Verification Continuum

HAPS® Prototyping User Guide

Version V-2025.06-3, April 2026

SYNOPSYS®

Copyright and Proprietary Information Notice

© 2026 Synopsys, Inc. This Synopsys software and all associated documentation are proprietary to Synopsys, Inc. and may only be used pursuant to the terms and conditions of a written license agreement with Synopsys, Inc. All other use, reproduction, modification, or distribution of the Synopsys software or the associated documentation is strictly prohibited.

Destination Control Statement

All technical data contained in this publication is subject to the export control laws of the United States of America. Disclosure to nationals of other countries contrary to United States law is prohibited. It is the reader's responsibility to determine the applicable regulations and to comply with them.

Disclaimer

SYNOPSYS, INC., AND ITS LICENSORS MAKE NO WARRANTY OF ANY KIND, EXPRESS OR IMPLIED, WITH REGARD TO THIS MATERIAL, INCLUDING, BUT NOT LIMITED TO, THE IMPLIED WARRANTIES OF MERCHANTABILITY AND FITNESS FOR A PARTICULAR PURPOSE.

Trademarks

Synopsys, Ansys, the Synopsys and Ansys logos, and other Synopsys trademarks are available at https://www.synopsys.com/company/legal/trademarks-brands.html.
Other company or product names may be trademarks of their respective owners.

Free and Open-Source Licensing Notices

If applicable, Free and Open-Source Software (FOSS) licensing notices are available in the product installation.

Third-Party Links

Any links to third-party websites included in this document are for your convenience only. Synopsys does not endorse and is not responsible for such websites and their practices, including privacy practices, availability, and content.

www.synopsys.com

Statement on Inclusivity and Diversity

Synopsys is committed to creating an inclusive environment where every employee, customer, and partner feels welcomed. We are reviewing and removing exclusionary language from our products and supporting customer-facing collateral. Our effort also includes internal initiatives to remove biased language from our engineering and working environment, including terms that are embedded in our software and IPs. At the same time, we are working to ensure that our web content and software applications are usable to people of varying abilities. You may still find examples of non-inclusive language in our software or documentation as our IPs implement industry-standard specifications that are currently under review to remove exclusionary language.

Feedback

Contents

1. **Preface** 16
   Related Documentation 17

2. **Getting Started With FPGA-Based Prototyping** 18
   About the HAPS® ProtoCompiler Solution 18
   HAPS® ProtoCompiler Design Flow 19
   HAPS Prototyping and Standard Prototyping Use Models 21
   HAPS ProtoCompiler Licensing Model 22
   Downloading and installing HAPS® ProtoCompiler 26
   Starting the HAPS® ProtoCompiler tool 27
      Setting up Additional Tools 28
      Configuring Environment Variables 28

3. **Converting ASIC Designs to FPGA** 30
   Getting ASIC Designs Ready for FPGA 31
   Handling I/O Pads and Replacing UDPs With RTL Function 32
   Converting ASIC Memories 33
   Making ASIC Netlists Suitable for FPGA Design 35
   Converting ASIC Clocks 36
      General Guidelines for ASIC Clocks 36
      Scaling External Clock Speed 38
         Analyzing the Clock Scaling Report 39
         Asynchronous Clocks 39
         Synchronous Clocks 40
   Intellectual Property 42
   Power Specifications 42
   Converting Constraints 43
   Checking Resources 44

4. **Creating and Compiling Databases** 45

Configuring a Design Database 46

Loading and Navigating Database States 49

Archiving a Database 51

Extracting Database Files from an Archive 54

Creating Lite Archives 56

Compiling the Design 58

Controlling the Compiler Run with Options and Constraints 59

Adding Design Files 62

Using UUM and Group Mapping 62

About Group Mapping and the lmf File 63

Creating an lmf File 63

Example: Group Mapping File 66

Example: Default Library Mapping 67

Configuring Library Options 67

Using Unified Compile 68

Preparing the Input for Unified Compile 69

Configuring RTL Design Files and Library Mapping 70

Configuring UTF File and the VCS Script 72

Adding IPs in a UC Design 74

Adding Complex SystemVerilog Port Modules 76

Running Unified Compile 77

Checking the UC Database 80

Validating VCS Design Files 81

Running Lint Checks 82

Checking for CDC Violations 84

Compiling the Design Incrementally 89

Incremental Unified Compile Flow: Design Database to SRS 90

Incremental Unified Compile Flow: Bypass Flow 91

Automated Bypass Flow (UC) 93

Manual Bypass Flow (UC) 97

SRS Bypass Flow with Incremental RTL Instrumentation 100

Incremental Unified Compile Flow: Structural VM 101

Incremental Unified Compile Flow: Bottom-Up Compilation 104

About Compiler Groups 106

Preparing the Design Using VCS Software 107

Running Compile on Compilation Groups 107

Debugging with the Verdi Software 108
Enhancing SRS Instrumentation for Partially Optimized Designs 108
Specifying Constraints 109
Adding Constraints to a Design Database 110
Configuring Directives in a CDC File 110
Configuring Options 112
Configuring a Tcl Options File 113
Configuring Options in Options Editor 115

5. Partitioning the Design 117
Design Methodology for Partitioning 119
Configuring Files for Partitioning 122
Defining the System in a TSS File 122
Exploring Partition Choices with a Basic TSS File 123
Defining the Target System in a Detailed TSS File 126
Configuring Clocks 130
Defining Hardware Interconnect 131
Configuring Connections with Auto-Cabling 131
Configuring Connections with board_system_create -interconnect 133
Defining Daughter Boards 137
Moving from Abstract PCF to TSS: Flow Example 148
Step 1: Define Initial Interconnect in PCF 148
Step 2: Check Results After Running with Initial Interconnect Definition . . 149
Step 3: Edit Interconnect Definition and Rerun Exploratory Partitioning . . 150
Step 4: Transfer Final Results to TSS and Run Partitioning 151
Validating TSS Cable Connections 152
Configuring Constraints in PCF Files 153
Using Abstract PCF Commands for Partition Exploration 154
Configuring PCF Constraints for Partitioning 156
TDM on Asynchronous Nets 157
Using Multiple PCF Files 157
Defining Global Information in the PCF 157
Assigning Ports in the PCF 158
Assigning Cells in the PCF 159
Assigning Clocks in the PCF 161
Assigning Nets to MGBLINKs 163
Assigning Inter-FPGA Nets to Traces 163
Working Iteratively with PCF Assignments 165

Working with HAPS Systems 166
Working with HAPS Global Clocks 166
Assigning PLL Clock Sources to the GCLK Network 167
Assigning Internally Generated Clocks to the GCLK Network 168
Assigning and Configuring Clocks 169
Minimizing Clock Skew 171
Using run pre_partition to Define Partitions 173
Partitioning the Logic 176
Creating Partitions 176
Partitioning Controls 179
Partitioning Restrictions 180
Partitioning Single-FPGA Designs 182
Working With Area Estimates for Partitioning 184
Reusing Area Estimation Data 184
Bypassing Area Estimation 185
Using Built-in Area Estimates 185
Analyzing Area Estimation Results 186
Using Estimates From FPGA Synthesis Tools 186
Using Your Own Custom Estimates 186
Using Timing-Aware Partitioning 186
Using Graph-Based Timing-Aware Partitioning 187
Partitioning Clocks 189
Replicating Clocks 189
Replicating Clock Control Managers 190
Rerunning Partition with Pre-Assigned Clocks 191
Locking Down Partitions 192
Locking Down Incremental Partitions 192
Locking Down Final Partitions 193
Using Interactive Partitioning 194
Interactive Partitioning File 196
Analyzing Partition Result Files 196
Checking the Target Specification (TSS) 197
Analyzing Partitioning at Different Stages 198
Analyzing Partition: Initial Partition Stage 199
Analyzing Partition: Second Partition Stage 200
Analyzing Partition: Abstract Partition Iteration Stage 202
Analyzing Partition: Final Partition Stage 203
Analyzing Partition: System Route Stage 204
Checking Partitioning Reports 205
Identifying Connector Availability 208
Parsing Partitioning Reports 208

Using Time Domain Multiplexing (TDM) 209
Setting General Controls for TDM 210
Using HSTDM and HSTDM ERD 212
Estimating System Frequency Based on HSTDM Delays 213
Constraining HSTDM Timing Paths 214
HSTDM Ratios 214
Recommended Bit Rate Settings 215
HSTDM Delay Tables 215
FPGA-to-FPGA Trace Delay Table 215
Analyzing HSTDM Results 216
Using HSTDM Top I/O Flow 217
PCF Commands for Setting Up Top I/O (Team Based Design) Flow 219
Exporting the PCF File from the Tool 219
Limitations 220
Using HSTDM Top I/O Flow with Debug Support 220
Using Multi-Gigabyte Transceivers for TDM 221
Enhancing Post-Partition Performance 224
Using Hierarchical Partitioning 225
Running Hierarchical Partitioning in Automatic Single-Pass Mode 226
Running Top-Down Hierarchical Partitioning in Manual Mode 227
Running System Route for the Top Level 228
Clock Scaling Support 232
Clock Scaling Flow 232
Reducing SLR Congestion 236
Generating FPGA Partitions 237
Analyzing *System Generate* Results 239
Interactive Partitioning Flow 240
Working with Inferred Clocks Driving Inter-FPGA Paths 244
Implementing Individual FPGAs 245
Running Synthesis for Individual FPGAs 246
Customizing Synthesis Scripts 249
Running Synthesis and Place and Route for a Multi-FPGA Design 251
Using Vivado Flags and Options in Tcl Script 252
Tcl and Environment Variables in Custom Scripts 252
Updating the Top Level with FPGA Implementation Results 254
Using Multi-Design Mode (MDM) 256
Instantiating CAPIM_UI 256
CAPIM_UI Example 256

Using UMRBus 3.0 with HAPS-100 Modules 257
Using UMRBus 3.0 MCAPIM for HAPS-100 Modules 257
Using UMRBus 3.0 Transactors 258
Using JTAG Communication 259
Modular Flow 259
Modular Flow Setup 261
First Pass - Partition 263
Subsystem definition 263
Cell assignment to subsystem 264
First pass system route 268
Pass 1: Output Files 269
Guided Partition Flow 271
Overview of Selective Rebuild 272
Bottom-up Compile 275
Debug Flow 282
Modular Flow Bottom up dumpvars support 285
Sync GSV in Modular Flow 287
Modular Flow Usage Recommendations 293

6. Running the Implementation Flow 294
The Basic Implementation Flow 294
Running Pre-Map 296
Mapping to Hardware 299
Defining HAPS Pins in Multi-FPGA Designs 299
Mapping the Design 299
Setting Auto-SPO 302
Synthesizing Based on Design Intent 305
Using Design Intent to Synthesize a Quick Prototype 305
Synthesizing for Optimal QoR Using Design Intent 307
Using Distributed Processing 308
Distributed Processing Schemes 308
Distributed Processing Operations 308
Setting Options to Run Distributed Processing 309
Using CDPL for Distributed Processing 310
Host File Examples for Distributed Processing 311
Running Distributed Compile 312

Running Distributed Synthesis . . . 315
Clock Conversion and Distributed Synthesis . . . 318
Distributed Synthesis Limitations . . . 319
Analyzing Results . . . 319
Checking Reports and Log Files . . . 319
Viewing Results in a Schematic . . . 323
Exploring a Schematic with find and expand . . . 326
Querying Incremental Results . . . 326
Querying Jobs . . . 328
Querying Metrics for a Design . . . 329
Dealing with Black Boxes in the Synthesized Netlist . . . 331
Checking Timing Results . . . 332
Generating Inter-FPGA Path Timing Report . . . 332
Generating Timing Report for Estimates of Design Performance . . . 332
Identifying Objects in Post-Mapped Timing Reports . . . 336
Viewing and Correlating Results with the Timing Report View . . . 339
Viewing Synthesis Results in the Timing Report View . . . 339
Correlating Synthesis Results With Place-and-Route Timing . . . 341
Generating Custom Timing Reports with the Timing Analyst . . . 343
Checking Clock Information . . . 347
Running SLTA . . . 349
Resolving Errors and Warnings . . . 350
Overriding Message Designations and Suppressing Output . . . 351
Setting Limits on the Number of Messages in a Log File . . . 354
Generating a Custom Error Message Report . . . 354
Downgradable Errors and Other Message Categories . . . 355
Running Simulation . . . 355
Setting up the Tools and Testbench . . . 356
Running Gate-Level Simulation for a Single-FPGA Design . . . 357
Generating a Testbench from FSDB . . . 359
Running Place and Route . . . 360
Running Place and Route . . . 360
Customizing Scripts to Place and Route a Single-FPGA Design . . . 362
Importing Place and Route Results for Back-annotation . . . 363
Resynthesizing the Design After Place and Route . . . 364
Analyzing Routing Congestion . . . 366
Minimizing Routing Congestion . . . 367
Troubleshooting Routing Congestion . . . 368

Minimizing Congestion During Place-and-Route Phase 368
Minimizing Congestion During the Map Phase 371
Minimizing SLL Congestion in Vivado 372
Running Synthesis Using Tcl Batch Script 374
Queuing Licenses 375
Releasing the Tool License 376
Working with Tcl Scripts and Commands 376
Generating a Tcl Script from the GUI 377
Creating a Tcl Script 378
Configuring Maximum Number of Parallel Jobs 379
Preparing to Run on the Hardware 380
Exporting Files for Runtime 381
Running HAPS Hardware Diagnostics 382
Running Data Expansion in a Multi-User Setup 382

7. Instrumenting and Debugging Designs 385
Debug Factors 386
HAPS ProtoCompiler Debug Functionality 386
The Instrumentor-Debugger Flow 389
Instrumenting the Design for Debug 391
Defining IICE Parameters 391
Instrumenting Multiple Clocks in a Single IICE 392
Instrumenting with Unified Compile 394
Verifying UC Designs 394
Debugging Designs with SVA 395
Running Incremental Unified Debug 396
Using Verdi to Debug a UC Design 398
Running the RTL Debugger 399
Debugging a Multi-Clock Design 399
Varying Buffer Depth Dynamically for DDR3 402
Verifying the Hardware Configuration for Deep Trace Debug 402
Configuring CEL-Based Triggers 403
Working with Deep Trace Debug 406
Comparison of Deep Trace Debug Schemes 406
About Single-FPGA Deep Trace Debug (DTD) 406
External DDR3 Memory 407

Running Deep Trace Debug (DTD) 407
Features for DTD with a DDR3 Daughter Board 408
RTL Data Expansion Using Verdi for Instrumented Signals 408
Using BRAM for Debugging 410
Running Incremental Debug for ECOs 411
Creating Incremental ECO IDC File Using Instrumentor User Interface 411
Creating Incremental ECO IDC File using the *report debug_visibility* command 413
Applying Debug ECO Instrumentation in the Design 414
Limitations to the Debug ECO Flow 415
Using Global State Visibility (GSV) 415
Asynchronous and Synchronous GSV 416
Using Asynchronous GSV for a Single-FPGA Design 419
Use Model Example: Synchronous GSV for a Single-FPGA Design 421
Use Model Example: Synchronous GSV with a Multi-FPGA Design 422
Configuring Clock Control Modules for GSV 424
Memory GSV 426
Running GSV with Clock Control 426
Using PLL Clocks for Global State Visibility 428
GSV or eGSV Flow 428
Configuring CCM V2 Clock Generator 429
Setting up to Run GSV or eGSV with HAPS-100 429
Configuring CCM V3 Clock Generator 432
Automatic Instrumentation of Driver Clock 436
IDC Syntax for Automatic Instrumentation 436
IDC File Example 437
CCM Debug Flow Enhancements 437
Limitations 437
Fast Track Debug 438
Fast Track Debug Flow 439
Always On Debug IICE 440
Automatic Cross-Triggering 442
Automatic Connection of Triggers to Clock Control Module (CCM) 444
Limitations 446
Enabling Clock Selection With DTD/Readback 447
Integrated Debugging with Verdi Software 448
Launch Verdi 450

Launching Verdi Through Commands 450
Launching Verdi from the HAPS-200 ProtoCompiler GUI 451
Using Verdi and Siloti for Debugging 453
Instrumenting Verdi Signals (ESDB) 455
Generating the Fast Signal Database (FSDB) 455
Exporting Files for Verdi 456
Debugging with the Waveform Viewer 456
Using the Verdi Flow with a UC Design 458
Running the Verdi Flow For Designs with GSV 459
Running the Verdi Flow For Designs with Instrumented/Probe Signals 461
Troubleshooting Verdi Flow Issues 461
Running Formal Verification with Checkpoints 462
Selecting a Verification Tool for Checkpoint Analysis 464
Analyzing Checkpoint Verification Results 465
Configuring Dynamic Force 468
Dynamic Force Backdoor Memory Initialization 469
Handling Memories with Gated Clocks 470
Design Clock Independent ROM Backdoor Access 471

8. Migrating Designs for Synthesis 473
Running Compilation or Mapping with Errors 473
Running Compilation with Errors 474
Analyzing Compilation Errors 476
Running Mapping with Errors 479
Including Unified Power Format (UPF) Specifications 479
HAPS Unified Compile UPF Flow 479
Power Aware Compilation 480
UPF Power Domain Corruption 482
Low Power (UPF) Reporting 485
Low Power (UPF) Debug 485
Debugging Low Power UPF During Runtime 486
Converting Gated Clocks 487
Gated Clock Conversion Flow 487
Guidelines for Generated Clocks 489
Guidelines for Gated Clocks 490
Guidelines for Integrated Clock Gating Cells (ICG) 491
Working with Gated and Generated Clocks 493
Defining Clocks for Gated Clock Conversion 495

Defining Generated Clocks with create_generated_clock 499
Combinational Logic and GCC Examples 501
Gated Clock Definitions for Black Boxes 502
GCC Examples for Generated Clocks 502
GCC Examples for Generated Clocks 505
Integrated Clock Gating (ICG) Stability Latch Removal Examples 508
Unsupported Loads for ICG Latches 511
Interpreting Gated Clock Error Messages 514
Clock from Clock Mux 517
Clock from Gated/Generated/Unconstrained Technology Primitive 517
Clock from Generated Clock Directive 518
Clock on Mux Select Line 518
Derived Clock on Input 518
Incompatible Clocks on Mux Inputs 519
Inferred Clock from Port 519
Instance Clocked by Multiple Variants of Clock 520
Multiple Clocks on Instance 521
Editing Netlists 521
Editing a Verilog Netlist 522
Using Gate-Level ASIC Netlists 522
Creating Verilog Code with *force* and *bind* Statements 522
Using Cross-Module Referencing (XMR) 523
Defining Cross-Module References Using a CDC Constraint 524
Defining Cross-Module References in the HDL 524
Using Hyper Source Modules for XMR 525
Hyper Source Example 527
Resolving XMR Multi-Driver Errors 529
Converting Designs for Unified Compile 530
Converting Older Designs for Unified Compiler 530
Known Limitations 532
Handling HDL Differences 532
Compile Units 532
Compilation into Different Libraries 533
Configurations in UC 533
Cross-Module Referencing 536
Designs with VHDL and Macros 537
Dynamic Force Statements 537
Force and Release Statements 539
Library Name Mapping 540
RTL Directory Paths 541
Virtual Classes 541

Including DesignWare Foundation Components 543
Custom DesignWare Compile Script Example 545
Integrating Memory Models 546

9. Verifying the Design 555
Guidelines for Successful Formal Verification 555
General Coding Guidelines for Formal Verification 556
Coding Guidelines for Verilog and SystemVerilog Designs 558
Limitations of Equivalence Checking 561
Prerequisites for Verifying FPGA designs 561
Enabling Formal Verification 562
Verifying Single-FPGA Designs 563
Synthesizing a Single-FPGA Design 564
Running Formal Verification 565
Analyzing Formal Verification Results 566
Re-synthesizing a Single-FPGA Design 569
Verifying Multi-FPGA Designs 569
Configuring Options for Formal Verification 574
Running VCS Simulation for Inferred Memories 575
Running Formal Verification with Checkpoints 577

10. Recommended Methodologies 580
Methodology for a Quick Prototype 580
Setting up the FPGA Design 581
Compiling a Quick Prototype 582
Synthesizing, Placing, and Routing a Quick Prototype 584
Tips to Reduce Turn Around Time (TAT) 585
Configuring a Best-Performance Prototype 586
Scripting Design Tasks 589

1

Preface

This section contains the following topics:

- Related Documentation

About this Guide

The *HAPS Prototyping User Guide* describes the functionality of the HAPS ProtoCompiler tool and how to use its features.

This document is intended for users who are familiar with the HAPS ProtoCompiler product range.

Contents of this Guide

The *HAPS Prototyping User Guide* have the following chapters:

- Getting Started With FPGA-Based Prototyping
- Converting ASIC Designs to FPGA
- Creating and Compiling Databases
- Partitioning the Design
- Running the Implementation Flow
- Instrumenting and Debugging Designs
- Migrating Designs for Synthesis
- Verifying the Design
- Recommended Methodologies

Related Documentation

The following product documents are included within the HAPS Prototyping tools. Unless otherwise specified, all documents can be accessed through the help system and as PDF documents:

- *HAPS® Prototyping User Guide*
- *HAPS® Prototyping Reference Manual*
- *HAPS® Prototyping Command Reference Manual*
- *HAPS® Prototyping Compiler and Mapper Guide*
- *HAPS® Prototyping Debugging Environment Reference Manual* (HAPS-100 only)
- *HAPS® Prototyping Debugger User Guide* (HAPS-100 only)
- *HAPS® Prototyping Instrumentor User Guide*
- *HAPS® Prototyping Message Reference Manual*
- *HAPS® Prototyping hRun User Guide* (HAPS-200 only)
- *HAPS® Prototyping Limited Customer Availability (LCA) Features Guide*
- *System Configuration Software Handbook* (HAPS-100 only)
- Free and Open-Source Software (FOSS) information is in PDF format and is located in the install_dir/doc directory.

Additional documentation is available outside the software:

- HAPS hardware documentation is available on https://spdocs.synopsys.com/dow_retrieve/latest/home_public/haps.html.

2

Getting Started With FPGA-Based Prototyping

FPGA-based prototyping is an effective method to validate systems. It offers a high-performance solution that supports software development before silicon becomes available.

FPGA-based prototypes are fully functional hardware representations of an IP block, subsystem or a complete SoC. FPGA-based prototyping is best used with relatively mature RTL, with much of it fixed or partly verified.

FPGA-based prototypes can run at high speed while keeping the RTL implementation as close to the ASIC original as possible.

The HAPS® ProtoCompiler tool can be used with the HAPS hardware systems for a prototyping solution.

See the following for more information:

- About the HAPS ProtoCompiler Solution
- HAPS ProtoCompiler Design Flow
- HAPS Prototyping and Standard Prototyping Use Models
- HAPS ProtoCompiler Licensing Model
- Downloading and installing HAPS ProtoCompiler
- Starting the HAPS ProtoCompiler tool

About the HAPS® ProtoCompiler Solution

HAPS® ProtoCompiler provides a development and debug environment for HAPS systems to prototype multi-FPGA designs.

Synopsys HAPS-200 is a scalable pre-silicon prototyping system. The HAPS-200 6 FPGA platform has high performance and capacity, fast compile times, and high debug capacity and bandwidth. HAPS-200 is compatible with the HAPS ecosystem, enabling mixed systems with HAPS-100. It uses the AMD Versal™ Premium VP1902 Adaptive SoC.

A separate executable, hRun, provides runtime functionality including debug.

The HAPS® ProtoCompiler solution provides the following functionality.

- Sophisticated FPGA logic synthesis engine.
- Built-in instrumentation and RTL debug engine.
- Integrated Xilinx place-and-route launch and run.
- Native Universal Multi-Resource Bus (UMRBus®) hardware and C, C++, and Tcl Application Programming Interfaces (APIs).

In addition, HAPS® ProtoCompiler works with other tools to further enhance functionality and provide a seamless prototyping solution:

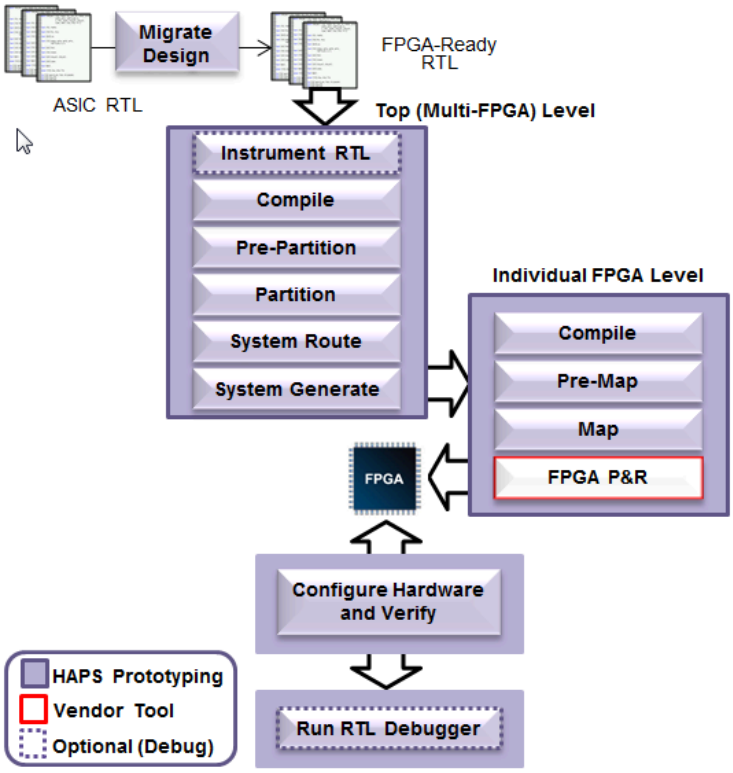
- Synopsys DesignWare® IP
- Synopsys VCS simulation
- Synopsys Verdi™ automated debug
- Synopsys HAV Utilities, comprising:
  - HAPS Resource Manager (HRM)
  - HAPS Hardware Utilities (hUtils)
  - Synopsys HAV Scheduler (SHS)

HAPS® ProtoCompiler Design Flow

Use the HAPS® ProtoCompiler tool to prototype a multi-FPGA design. Multi-FPGA design flow includes conversion, synthesis, place and route, and hardware configuration steps. You can run a basic implementation flow or include hardware-based debug.

Feedback

Figure 1 Multi-FPGA Design Flow



The implementation steps shown for the top multi-FPGA level and the individual FPGA level are part of the *protocompiler* executable. The hardware configuration uses hRun for configuration and debug. See HAPS Prototyping and Standard Prototyping Use Models for a diagram of the functionality

To get details about each of the stages, refer to this table:

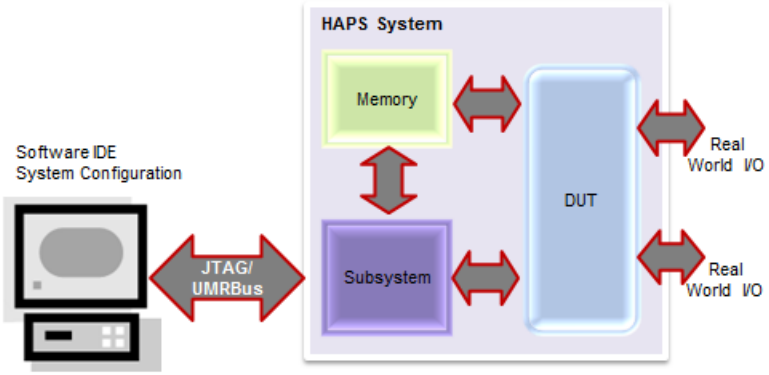
Migrating the design from ASIC	Converting ASIC Designs to FPGA
Starting the tool	Starting the HAPS ProtoCompiler tool
Setting up and compiling	Creating and Compiling Databases
Partitioning (includes pre-partitioning, system routing and system generation)	Partitioning the Design
Synthesizing (pre-map and map)	Running the Implementation Flow
Debug flow	Instrumenting and Debugging Designs

HAPS Prototyping and Standard Prototyping Use Models

You can plug the HAPS prototyping solution into three prototyping methodologies that are used today:

- Standalone Prototyping Use Model
- Prototyping Use Model With a PCI Express Connection
- Hybrid Prototyping Use Model

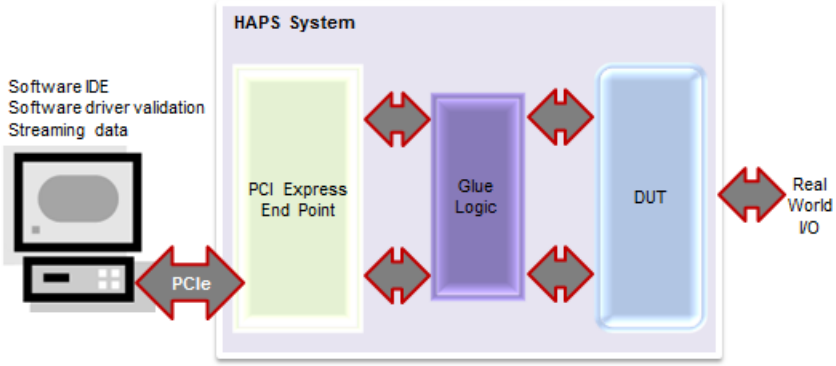
Standalone Prototyping Use Model



This use model is the most popular application for FPGA-based prototypes. It is a self-contained environment with real-time clock performance. It connects the design under test (DUT) to real world interfaces.

Use an embedded CPU subsystem of the prototype design to test and execute the tool stack. FMC or HapsTrak 3 daughter boards provide PHY interfaces for a wide variety of protocols and systems. The HAPS hardware provides a JTAG connection to external workstations running the tool Integrated Development Environment (IDE).

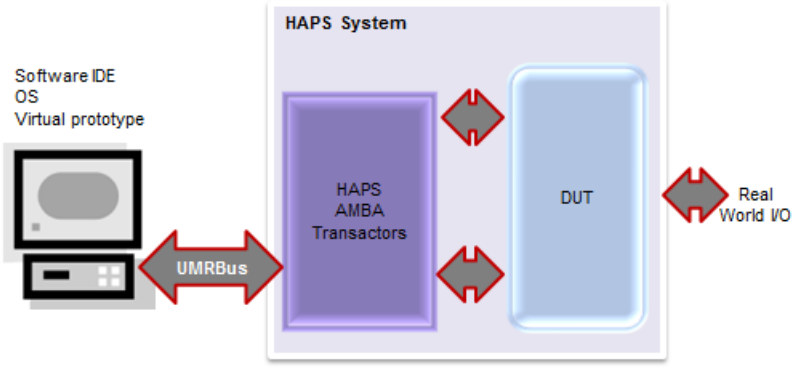
Prototyping Use Model With a PCI Express Connection



In this model, a prototype system is directly plugged into the PCIe slot of a host workstation. This enables high-volume data streaming to a DUT. This makes it a popular validation scenario for media controllers.

The HAPS Universal Multi-Resource Bus (UMRBus) provides the API to use custom applications to communicate and control the DUT logic. DUT connections are made within the RTL with a simple client interface module.

Hybrid Prototyping Use Model



In this model, you can mix SystemC/transaction-level models (TLMs) with FPGA-based prototype hardware. It eliminates RTL availability as a gating factor for system bring-up. It makes prototypes available sooner.

The DUT RTL is validated in a virtual processor subsystem running a software stack of OS and application software. A virtual prototype communicates with the DUT using bus protocol transactors to bridge loosely-timed models with cycle-accurate hardware.

HAPS ProtoCompiler Licensing Model

This section explains the details of license consumption for the ProtoCompiler software.

Terminology

ACP (Automated Compile Points)	Existing technology for parallel processing of compile points.
CDPL (Common Distributed Processing Library)	Parallel or distributed processing on multiple machines. Parallel processing does not necessarily mean using CDPL, because the processes can be run on a single local machine.
Core process	Mapper, compiler or other Synopsys processes launched by the tool.
Distributed compilation	Compile jobs use multiple parallel processes.

Distributed estimation	Estimation jobs use distributed synthesis technology and parallel estimation runs.
Distributed processing	Same as parallel processing, but does not necessarily mean the processes are distributed on remote machines.
Distributed synthesis	Mapper jobs use multiple parallel processes.
Main product license	The product license that is checked out when the tool starts up.
Parallel processing	Multi-processing; running multiple processes in parallel.
Product license	The license required by the product to run: *protocompiler*, *protocompiler100*
Token (mp) license	Second-level license used for multiprocess runs. For example *protocompilermp*

The main factors that determine the actual number of processes run in parallel are:

- The number of jobs to run in parallel (set by `max_parallel_jobs`).
- The number of licenses available.

One *protocompiler100* license can run an unlimited number of jobs in parallel.

One *protocompiler* license can run up to 16 jobs in parallel.

The product reduces the number of parallel processes according to the number of licenses available.

The Tcl window and the log file (protocompiler.log) record messages with license information such as:

- Number of licenses being checked in or checked out.
- The total count in use following the operation.

For example, this message is displayed when three implementations are run in parallel:

```
License checkout: 2 protocompiler - current checkout count 3
…
License checkin: 2 protocompiler - current checkout count 1
```

CDPL Processing and Distributed Processing

Distributed or parallel processing runs tasks using multiple processes. CDPL enables distributed processes to run on remote machines.

The `cdpl` Tcl option enables CDPL. When the option is disabled, parallel processing runs on the local machine only. All parallel processing and licensing behavior is independent of CDPL.

If CDPL is enabled and a core process consumes a license, the core process checkout occurs on the remote machine.

Licensing for the Mapper Flow

When running processes supporting multiprocessing the number of licenses required is:

- One protocompiler license for every 16 jobs.
- One protocompilermp license for every four jobs. 4 protocompilermp licenses for every 16 jobs.

The protocompiler or protocompilermp licenses are checked out at a 1:4 ratio. It is sufficient to cover the number of jobs running concurrently.

When started, the ProtoCompiler tool allows you to run up to 16 jobs. Extra licenses are required only if the *max_parallel_jobs* option is set to more that 16. For example:

max_parallel_jobs	**protocompiler**	**protocompilermp**
16	No extra licenses required	No extra licenses required
18	1 extra license required (total of 2)	1 license required (total of 1, allows 4 more jobs or 20 total)
24	1 extra license required (total of 2)	2 extra license required (total of 2, allows 8 more jobs or 24 total)
32	1 extra license required (total of 2)	4 extra license required (total of 1, allows 4 more jobs or 32 total)

If two processes are run in parallel, both the processes require separate licenses. The requirement is added together increasing the total number of licenses required.

Licensing When Using the Launch ProtoCompiler Command

You can run the synthesis databases, created by the *System Generate* data state, in two ways:

- Run the ProtoCompiler tool on each database at the system or shell level. You can use a shell script to run each of the ProtoCompiler instance.
- Use the `launch protocompiler` Tcl command to launch the ProtoCompiler tool for each database.

Feedback

The licensing rules are same irrespective of the way you run the tool at the system level or shell level. Each instance of the ProtoCompiler tool requires one license and allows up to 16 jobs. Extra licenses are required depending on the `max_parallel_jobs` option.

The top-level ProtoCompiler tool manages the license checkout to optimize the license usage for running synthesis.

The license pre-checkout at the top-level allows the license usage to be distributed among the synthesis runs. The launched ProtoCompiler instances checks out the *protocompilermp* licenses based on the *max_parallel_jobs* option.

Example 1: Four Synthesis Database and 4 Max Parallel Jobs

In the following example, there are four FPGA synthesis databases and four max_parallel_jobs.

```
launch protocompiler -max_parallel_jobs 4 -script f1.tcl -script f2.tcl
 -script f3.tcl -script f4.tcl  ...
```

This uses 4 jobs multiplied by 4 databases or 16 jobs and requires:

- One *protocompiler* license. Because 16 jobs are allowed by the top-level license.
- Four *protocompilermp* licenses. Each instance of the ProtoCompiler tool launched consumes one *protocompilermp* license at the startup.
- Five *protocompilermp* licenses if you are using the system or shell method.

Example 2: Sixteen Synthesis Database and 4 Max Parallel Jobs

In the following example, there are 16 FPGA synthesis databases and four max_parallel_jobs.

```
launch protocompiler -max_parallel_jobs 4 -script f1.tcl ...  -script
 f16.tcl  ...
```

This uses 4 jobs multiplied by 16 databases or 64 jobs and requires:

- Four *protocompiler* licenses. The main license and three additional licenses checked out by the top-level ProtoCompiler instance.
- 16 *protocompilermp* licenses. Each instance of the ProtoCompiler tool launched consumes one *protocompilermp* license at the startup and allows four jobs.
- 16 additional *protocompiler* licenses if you are using the system or shell method.

Example 3: Sixteen Synthesis Database and 8 Max Parallel Jobs

In the following example, there are 16 FPGA synthesis databases and eight max_parallel_jobs.

Feedback

```
launch protocompiler -max_parallel_jobs 8 -script f1.tcl ...  -script
 f16.tcl  ...
```

This uses 8x16 or 128 jobs and requires:

- Eight *protocompiler* licenses. The main license and seven additional licenses checked out by the top-level ProtoCompiler instance.
- 32 *protocompilermp* licenses. Two instances of the ProtoCompiler tool launched consumes two *protocompilermp* license at the startup and each instance allows four jobs.
- 16 additional *protocompiler* licenses if you are using the system or shell method.

Example 4

In this example, there are:

- 24 FPGA synthesis databases and 12 max_parallel_jobs for six databases.
- 4 max_parallel_jobs for the remaining 18 databases.

```
launch protocompiler -max_parallel_jobs 12 -script f1.tcl ...  -script
 f6.tcl   -max_parallel_jobs 4 -script f7.tcl ... -script f24.tcl
```

This uses (12 jobs x 6 scripts) + (4 jobs x 18 scripts) = 144 total jobs and requires:

- Nine protocompiler licenses: The main license and eight additional licenses checked out by the top-level ProtoCompiler instance. 9 x 16 jobs-per-license allows 144 jobs.
- 36 protocompilermp licenses: Six runs using 12 jobs requires three protocompilermp each. 18 runs using 4 jobs require 1 protocompilermp each.
- 24 additional protocompiler licenses (Total 25 licenses) if you are using the system or shell method.

Note:

Each ProtoCompiler instance requires at least one *protocompilermp* license and has at least 4 jobs available to run. So, using less than *-max_parallel_jobs* 4 does not reduce the number of *protocompilermp* licenses required.

Downloading and installing HAPS® ProtoCompiler

Use this procedure to download and install HAPS® ProtoCompiler.

1. Download and install the software according to the instructions.

   You also require the Xilinx Vivado software for place and route. You require the system configuration software to work with the board system. See Setting up Additional Tools for details.

Feedback

2. Set the *LM_LICENSE_FILE* environment variable to point to your license server. Set the *PATH* variable to include the path to the tool executables.

   For information about variables for other functionality used during the prototyping process, see Configuring Environment Variables.

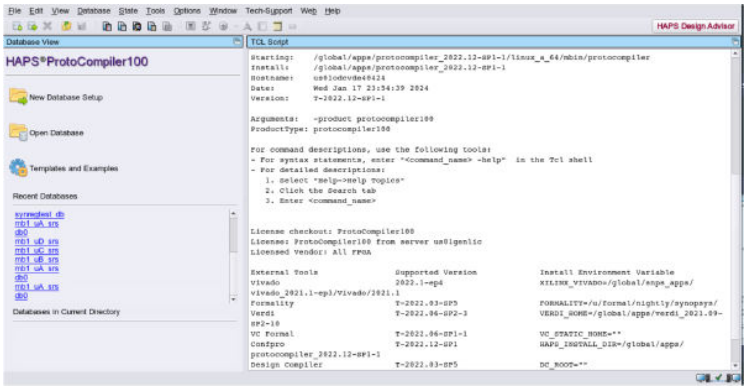
Starting the HAPS® ProtoCompiler tool

Use this procedure to start the HAPS® ProtoCompiler tool.

1. To start the tool, type the appropriate command at a shell prompt:

   - For HAPS ProtoCompiler: `protocompiler100 &`

     The tool console window opens.



2. Determine your working mode, according to what you intend to do:

   - Multi-FPGA partitioning

     Specify this mode after launching the tool, using the `option set design_flow partition` command. See Partitioning the Design for details. This mode is intended for partitioning your large design into multiple FPGAs. You can also instrument your design.

   - Single-FPGA implementation mode

     Specify this mode after launching the tool, using the `option set design_flow synthesis` command.

     This mode is intended for implementing the design at the individual FPGA level.

     At the single-FPGA level, you can run the basic implementation flow or the implementation flow with RTL debugging. See Converting ASIC Designs to FPGA,

Creating and Compiling Databases, or Instrumenting and Debugging Designs for the next steps.

See HAPS Prototyping and Standard Prototyping Use Models for an overview of the design flow.

See the following for details:

- Setting up Additional Tools
- Configuring Environment Variables

Setting up Additional Tools

HAPS® ProtoCompiler implements the prototyping design from RTL to synthesis. To place and route and verify the design with a board system, install additional tools listed in the following table. This is not an exhaustive list of tools. For example, you can use VCS for simulation.

Tool	Installation and Documentation Packages
Xilinx Vivado Design Suite	
Includes FPGA place-and-route and bitstream generation functionality required to program the FPGA device	*http://www.xilinx.com/products/design-tools/vivado*

Configuring Environment Variables

Configure environment variables with the *setenv* command, and provide the path to the functionality.

The following table lists the environment variables to set for functionality used during prototyping:

Variable	Description
LM_LICENSE_FILE	License server for the prototyping tool
PATH	Path to the prototyping software
CDPL_FPGA_HOST	Distributed processing with common distributed processing library (CDPL)
SYNOPSYS	Design Compiler® installation

Variable	Description
XILINX_VIVADO XILINX	Place and route with Vivado Place and route with ISE
VCS_HOME	VCS simulation
(ProtoCompiler Only)*FORMALITY*	Formality equivalence checking
HAPS_INSTALL_DIR	Internal system configuration libraries

3

Converting ASIC Designs to FPGA

ASIC or SoC designs are much larger and faster than FPGA designs.

The following guidelines describe best practices to follow with ASIC designs that are to be prototyped in FPGAs:

- Identify and deal with FPGA-hostile RTL. The following ASIC structures do not easily map to FPGA architectures. See the rest of the chapter for more information about handling these conversions.
  - Asynchronous delays in latches need to be removed, reworked or replaced with clocked processes.
  - ASIC memory is too big and must be split across multiple RAMs.
  - Clocks with gating. Gated clocks usually overflow the FPGA clock resources. See Converting ASIC Clocks for details.
  - Complex generated clocks need to be simplified to fit the FPGA resources. See Converting ASIC Clocks for details.
  - Clock mixing, as for embedded test logic.
  - Top-level pads. See Handling I/O Pads and Replacing UDPs With RTL Function for details.
  - Gate-level netlists.
  - Analog modules require wrappers to interface to external circuitry.
  - User-defined primitives (UDP) that specify the behavior of an element or module. If you include them in the design you get an error during logic synthesis. Replace them with equivalent RTL that describes the function of the UDP.
  - SoC leaf cells instantiations.
  - SoC memories.
  - SoC-specific IP without source RTL.
  - BIST and other test circuitry instantiated in the RTL. Handle them by leaving these signals dangling.

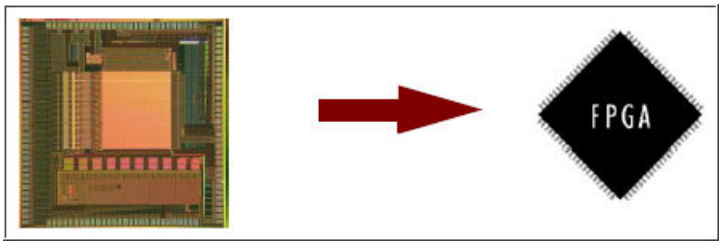
- Avoid latches because they are hard to time on an FPGA.
- Avoid combinatorial loops.
- Do not include optimizations such as clock gating, test insertion, and low-power in the original RTL. Instead, perform optimizations using the SoC tools to ensure that the ASIC RTL is portable to FPGA,.
- Use *‘define* and *‘ifdef* to include or remove code for prototyping. Use these constructs to isolate BIST (built-in self test), memory instantiations, and so on.
- Isolate RTL changes to within the library elements rather than outside them in the RTL structure. This improves portability and keeps the prototyping code as close to the original as possible.
- Share *make* files between SoCs and FPGAs by using macro-driven branching for different targets.
- Reduce the impact of changes to the ASIC source RTL, by using wrappers to make changes. Replace files instead of editing them, and back-annotate all changes.

This chapter describes the procedures to be done in HAPS® ProtoCompiler to automate or simplify ASIC-to-FPGA conversion for prototypes.

- Getting ASIC Designs Ready for FPGA
- Handling I/O Pads and Replacing UDPs With RTL Function
- Converting ASIC Memories
- Making ASIC Netlists Suitable for FPGA Design
- Converting ASIC Clocks
- Intellectual Property
- Power Specifications
- Converting Constraints
- Checking Resources

Getting ASIC Designs Ready for FPGA

Before an ASIC SoC design can be mapped to an FPGA, some parts of the design must be reworked. This is required because of architectural differences between FPGAs and ASICs and FPGA capacity limitations.



- ASIC gated clock and generated clock structures do not fit on the FPGA, and must be reworked. Internal fixed clock sources and PLLs must be replaced.
- ASIC memories cannot be used on the FPGA.
  - Replace simple memories with FPGA equivalent memories. These memories can be generated using:
    - FPGA vendor tools, or
    - Synopsys Synphony Model Compiler, or
    - SYNCore (included with the Synopsys FPGA synthesis tools).
  - For more complex memories, write replacement models for the FPGA.
- Latches and asynchronous delays must be reworked or replaced.
- ASIC analog modules require wrappers before they can be used on the FPGA.
- I/O control logic for dedicated board systems must be adjusted.

Dividing up the Design

ASIC or SoC designs are much larger than FPGA designs. ASIC designs are usually divided up into partitions across multiple FPGAs before prototyping at the individual FPGA level.

HAPS® ProtoCompiler provides various features to automate the partitioning process. See Partitioning the Design for details.

Handling I/O Pads and Replacing UDPs With RTL Function

Handling I/O Pads

The ways to handle the ASIC I/O pad ring are:

- Prototype the ASIC design core and ignore the I/O pad ring. The FPGA synthesis engine can infer and insert I/O buffers. So you can leave out the I/O pad ring. Make the dangling connections inactive or tie them to the top-level boundary.
- Keep the ASIC I/O pad ring in the FPGA prototype design.

- Create an FPGA-synthesizable equivalent for every ASIC I/O pad by writing the equivalent RTL or instantiating it. The FPGA equivalent only needs to model the logical connection from the design core to the outside world. You need to create one for each different kind of I/O pad.
- Replace each ASIC I/O pad with the corresponding synthesizable model you created.

Replacing UDPs With RTL Function

HAPS® ProtoCompiler does not synthesize user-defined primitives (UDPs). If you include UDPs for FPGA synthesis, the following error occurs:

```
@E:CG731 : test.v(3) | Can't synthesize UDP primitives
```

Replace any UDPs in your design with the equivalent RTL function.

Converting ASIC Memories

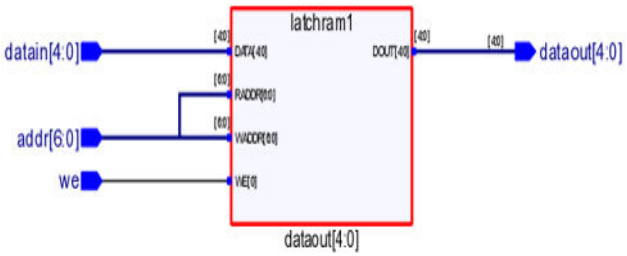
ASIC memory is typically larger and more complex than FPGA memory. FPGA memory is implemented within the FPGA, but ASIC memory can be off-chip.

Guidelines

- Remove ASIC memory that is not needed for the prototype, and implement the FPGA with external RAM.
- Replace ASIC memory with equivalent FPGA models whenever possible.
- Instantiate the memories or write them as HDL code from which the software can infer them. Then map them to the appropriate resources.
  - Create different wrappers for the memories in the HDL code, one for ASIC and one for FPGA. The FPGA wrapper either :
    - Describes the behavior so that the memory can be inferred and mapped correctly, or
    - Has a vendor-provided netlist describing the behavior.
  - Use the *syn_ramstyle* attribute to specify the kind of RAMs to infer. See Setting Attributes to Guide RAM Inference for details.

Feedback

- For latch-based RAMs, specify latch ram inference before compiling the design (`option set latchram 1` command). The compiler schematic shows the latch RAM inferred:



  Replace the inferred *latchram1* memories with FPGA equivalents. Make sure to do so before mapping; you get an error if there are *latchram1* instances when you map the design.

- Disable test and power pins on ASIC memory, by ignoring them or tying them to constants.
- Verify the functionality of FPGA-compatible memory models with testbenches before using them in your design. Some tools, such as SYNCore, provide testbenches.

For large ASIC memories, do the following:

- Map complex memories to FPGA models if possible. If the complex memory is too large to fit on the FPGA, then:
  - Implement complex memories using off-chip resources external to the FPGA, such as DDR3 or DDR4.

    Connect and synchronize the external memory to the FPGA. Black-box the memory on the FPGA.
- Create two RTL wrappers, one for the FPGA prototype and one for ASIC. The FPGA wrapper should have RTL that describes the behavior. Then the FPGA synthesis tool can infer the memory structure and map it to the appropriate vendor technology.

For small ASIC memories, do the following:

- Directly replace less complex memories with FPGA-specific memories generated by vendor tools.
- Use the netlists provided by FPGA vendors for certain memories.

Procedure

1. Convert ASIC memory as per your design needs.
2. Run logic synthesis.

   The tool automatically infers RAMs from the RTL code and reports them. The tool can infer and implement FPGA RAM from register banks in the RTL. You can also guide inference by specifying the *syn_ramstyle* attribute before synthesizing the design. For detailed information on inferring RAM, see Defining Objects for Inference.
3. Check the usage summary in the log file to see if the FPGA memory resources are overutilized:

```
RAM/ROM usage summary
Block Rams: 160 of 156 (102%)
Register bits not including I/Os: 2500 (50%)
```

4. Check the log file for a report of the inferred memories. For example:

```
#### START OF Block RAM DETAILED REPORT ####
Total Block RAMs:  1
mem_mem_0_0
----------------------------
READ_WIDTH_A          4
WRITE_WIDTH_A                 4
READ_WIDTH_B                  4
WRITE_WIDTH_B                 4
WRITE_MODE_A                  READ_FIRST
SRVAL_A                       0
----------------------------
#### END OF Block RAM DETAILED REPORT ####
```

Making ASIC Netlists Suitable for FPGA Design

Modify the original ASIC RTL netlist to avoid minor changes to the HDL source. Editing the netlist lets you keep the HDL code intact. Post-synthesis netlist editing is useful to:

- Implement engineering change orders (ECOs).
- Disable ASIC clocks.
- Reroute a fast clock from an external source to an internal (DCM) source.
- Insert IP blocks in an RTL netlist (`srs` file) at the required level of hierarchy.

For further information about netlist editing techniques, see Editing Netlists, which includes information about these topics:

- Editing a Verilog Netlist
- Using Cross-Module Referencing (XMR)
- Using Gate-Level ASIC Netlists
- Creating Verilog Code with force and bind Statements

Converting ASIC Clocks

To avoid clock skew, ASIC clocks consist of numerous clocks that are “balanced” using clock tree synthesis. Unlike ASICs, FPGAs have a finite number of low-skew global clocks that are built in as part of the architecture. In addition, ASIC clocks might be dynamically programmed by the tool. ASIC clocks have asynchronous clock domains or domains that are gated for power saving. If you have a gated clock in an FPGA implementation, it adds extra delay and upsets the balance. This introduces timing violations in the process.

ASIC clocks must be converted to fit the restricted resources of FPGA architecture. Gated clock conversion (GCC) automates the bulk clock conversion, but the clock information must be properly defined. You must also specify decisions about trade-offs to the tool.

For details, see the following:

- General Guidelines for ASIC Clocks
- Scaling External Clock Speed

Also, see Converting Gated Clocks.

General Guidelines for ASIC Clocks

Unlike ASICs, FPGAs have a finite number of low-skew global clocks that are built in as part of the architecture. In addition, ASIC clocks might be dynamically programmed by software, and have asynchronous clock domains, or domains that are gated for power saving. If you have a gated clock in an FPGA implementation, it adds extra delay and upsets the balance, introducing timing violations and other glitches in the process. The following guidelines describe some ways to handle the differences between ASIC and FPGA clocking schemes.

Simplifying the ASIC Clock Network

- Avoid scan or test logic on the clock path. For prototyping purposes, tie scan logic to 0, so that the tool automatically removes this logic when it runs. Test logic on the clock path prevents gated clock conversion, which results in high clock skew and hardware instability.
- If it is necessary that the Scan Clock and the associated mux logic on the clock path be retained, replace the logic mux with the FPGA BUFGMUX primitive. This ensures that the clock is routed on the dedicated clock network, this and reduces the clock skew caused by local routing.
- Swap the ASIC PLLs with functionally equivalent MMCM structures on the FPGA. Use the MMCMs to generate the subsystem, interface, IP, and peripheral clocks required for the design. The reference clocks for these MMCMs must come from the HAPS system PLLs whenever feasible. If the reference clock for the user FPGA PLL/MMCM has to be an external hardware clock source, you can further redistribute the clock back onto the HAPS global clock network.

Clock Mapping Guidelines

- Map design clocks to HAPS sources. Clocks that are not mapped can cross FPGAs, and can therefore cause skew.
- Map gated clocks to the HAPS sources or replicate the clocks as needed. Make sure to map system-level clocks generated within the FPGAs to the GCLK network.
- Avoid one-to-one mapping of cascaded dividers on clock sources when moving ASIC/SoC code to the FPGA. Instead, merge cascaded circuits and use the new divide value to program the MMCM output.
- Do not manually instantiate *BUFG* and *IBUFGDS* on clock nets and ports. The tool inserts them automatically, according to the loads on the clocks.

Synchronizing Design Resets and MMCM Lock Signals

- The lock times of different MMCMs are not guaranteed to occur at the same time. It is recommended that you poll the lock signals of all the MMCMs in the design as well as the locked signals, and then use this result to create a delayed power on master reset for the entire design.
- It is recommended that you use a separate reset to control the MMCMs. This reset must be different from the master reset, and is used to decouple the clock reset circuit from the design reset.

Replicating Clock Networks

- To avoid problems implementing the design in the hardware, replicate clock networks in these cases:
  - When you exhaust the HAPS clocks and need to use FPGA MMCMs to generate clocks, replicate clocks in each FPGA.
  - If you have source-synchronous clocks that are generated by the FPGA, replicate them in the FPGAs, because mapping source-synchronous clocks to HAPS clocks makes them asynchronous.
- Do not replicate MMCMs that generate odd multiples of the source clock across FPGAs. If such clocks are needed across systems, map them to the HAPS GCLKs, with the connecting FPGA as the source. MMCMs that create even multiples of the source clock across FPGAs (2^n) can be replicated.
- Do not replicate MMCMs that have divide-by clocks across FPGA boundaries.
- Monitor the status of the lock signals of replicated MMCMs across systems, and use the result to control the master reset for the design.

Scaling External Clock Speed

Clock scaling tunes the external clocks to the fastest clock speed. The design realizes the timing requirements. The idea is to find the fastest speeds of clocks in a design. This allows all delay paths to meet the timing requirements (all slacks are zero or positive).

The Tcl option to enable the clock scaling:

```
option set clock_scaling <multi_root>
```

Clock scaling is invoked at two points during the ProtoCompiler flow:

- Before the timing report is written out during system generate stage.
  - This mode uses estimated timing data.

  ```
  report timing -generate -out slta
  ```
- Before the timing report is written out during system-level timing analysis (SLTA).
  - This mode uses the back-annotated data from tool’s place-and-route run.

  ```
  report timing -generate -mode slta -out slta
  ```

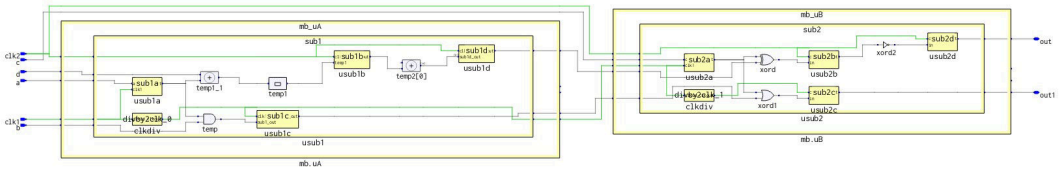
Feedback

Analyzing the Clock Scaling Report

The clock frequency scaling report is generated under the corresponding clock *Performance Summary* table, in the regular timing report. You need to observe the values in the *Estimated Frequency* or *Estimated Period* columns.

See the following example output in the timing report in system generate stage with estimated values and BA SLTA. Without clock scaling, all the clocks are scaled independently. When clock scaling is enabled, all external synchronous clocks are scaled using the same ratio. The asynchronous clocks are scaled independently.

Schematic



Asynchronous Clocks

FDC constraints

```
create_clock -name {clk1} [get_ports {p:clk1}] -period 40.0
create_clock -name {clk2} [get_ports {p:clk2}] -period 30.0
create_generated_clock -name {genclk1} -source {p:clk1} [get_pins
 {usub1.clkdiv.clkout}] -divide_by 2
create_generated_clock -name {genclk2} -source {p:clk1} [get_pins
 {usub2.clkdiv.clkout}] -divide_by 4
set_clock_groups -asynchronous -group {clk1 genclk1 genclk2}
set_clock_groups -asynchronous -group {clk2}
```

Performance Summary in the System Generate Stage

Without clock scaling:

Here, the root clocks *clk1* and *clk2*, and the generated clocks *genclk1* and *genclk2* are scaled independently.

```
@S0.0 |Performance Summary
*******************

Worst slack in design: 8.541

                  Requested     Estimated     Requested     Estimated               Clock                                                 Clock
Starting Clock    Frequency     Frequency     Period        Period        Slack     Type                                                  Group
-------------------------------------------------------------------------------------------------------------------------------------------------
HAPS_lock_clkin   1.0 MHz       NA            1000.000      NA            NA        declared {HAPS_lock_clkin}                            haps_lock_clkgroup
HAPS_umr_clk      100.0 MHz     1221.9 MHz    10.000        0.818         9.182     derived (from HAPS_umr_clkin) {HAPS_umr_clkin}        default_clkgroup
HAPS_umr_clkin    100.0 MHz     NA            10.000        NA            DCM/PLL   declared {HAPS_umr_clkin}                             default_clkgroup
clk1              25.0 MHz      1438.2 MHz    40.000        0.695         39.305    declared                                              group_1_107
clk2              33.3 MHz      186.9 MHz     30.000        5.351         24.649    declared                                              group_1_108
genclk1           12.5 MHz      719.1 MHz     80.000        1.391         79.413    generated (from clk1)                                 group_1_107
genclk2           6.2 MHz       107.0 MHz     160.000       9.346         150.654   generated (from clk1)                                 group_1_107
System            1.0 MHz       312.8 MHz     1000.000      3.196         996.803   system                                                system_clkgroup
=================================================================================================================================================
```

Feedback

With clock scaling:

The asynchronous clocks *clk1* and *clk2* are scaled independently, but the generated clocks *genclk1* and *genclk2* are scaled with respect to the root clock *clk1*. Scaling rate for *clk1* is 0.1 and for *clk2* is 0.18.

```
@S0.0 |Performance Summary
********************

Worst slack in design: 8.541
```

Starting Clock	Requested Frequency	Estimated Frequency	Requested Period	Estimated Period	Slack	Clock Type	Clock Group
HAPS_lock_clkin	1.0 MHz	NA	1000.000	NA	NA	declared {HAPS_lock_clkin}	haps_lock_clkgroup
HAPS_umr_clk	100.0 MHz	NA	10.000	NA	9.182	derived (from HAPS_umr_clkin) {HAPS_umr_clkin}	default_clkgroup
HAPS_umr_clkin	100.0 MHz	NA	10.000	NA	DCM/PLL	declared {HAPS_umr_clkin}	default_clkgroup
clk1	25.0 MHz	250.0 MHz	40.000	4.000	39.305	declared	group_1_107
clk2	33.3 MHz	185.2 MHz	30.000	5.400	24.649	declared	group_1_108
genclk1	12.5 MHz	125.0 MHz	80.000	8.000	79.413	generated (from clk1)	group_1_107
genclk2	6.2 MHz	62.5 MHz	160.000	16.000	150.654	generated (from clk1)	group_1_107
System	1.0 MHz	NA	1000.000	NA	996.803	system	system_clkgroup

Performance Summary in theSLTA

Without clock scaling:

Here, the root clocks *clk1* and *clk2,* and the generated clocks *genclk1* and *genclk2* are scaled independently.

```
@S0.0 |Performance Summary
********************

Worst slack in design: 8.282
```

Starting Clock	Requested Frequency	Estimated Frequency	Requested Period	Estimated Period	Slack	Clock Type
HAPS80_SYSTEMIP_ls_0_11s_440s_2016_06_1_1_mb_uA_slpgen|I_haps80_systemip_core_I_umr_clk_gen_mmcm_clkout2_derived_clock_CLKIN1	100.0 MHz	NA	10.000	NA	NA	derived (
HAPS80_SYSTEMIP_ls_0_11s_440s_2016_06_1_1_mb_uB_slpgen|I_haps80_systemip_core_I_umr_clk_gen_mmcm_clkout2_derived_clock_CLKIN1	100.0 MHz	NA	10.000	NA	NA	derived (
HAPS_clk_12_5	12.5 MHz	NA	80.000	NA	NA	derived (
HAPS_clk_200	200.0 MHz	NA	5.000	NA	NA	derived (
HAPS_lock_clkin	1.0 MHz	NA	1000.000	NA	NA	declared
HAPS_umr_clk	100.0 MHz	582.0 MHz	10.000	1.718	8.282	derived (
HAPS_umr_clkin	100.0 MHz	NA	10.000	NA	DCM/PLL	declared
clk1	25.0 MHz	198.4 MHz	40.000	5.253	34.747	declared
clk2	33.3 MHz	72.5 MHz	30.000	13.792	16.208	declared
genclk1	12.5 MHz	95.2 MHz	80.000	10.506	72.222	generated
genclk2	6.2 MHz	35.5 MHz	160.000	28.166	131.834	generated
System	1.0 MHz	121.2 MHz	1000.000	8.254	991.746	system

With clock scaling:

The asynchronous clocks *clk1* and *clk2* are scaled independently, but the generated clocks *genclk1* and *genclk2* are scaled with respect to the root clock *clk1*. Scaling rate for *clk1* is 0.18 and for *clk2* is 0.46.

```
@S0.0 |Performance Summary
********************

Worst slack in design: 8.282
```

Starting Clock	Requested Frequency	Estimated Frequency	Requested Period	Estimated Period	Slack	Clock Type
HAPS80_SYSTEMIP_ls_0_11s_440s_2016_06_1_1_mb_uA_slpgen|I_haps80_systemip_core_I_umr_clk_gen_mmcm_clkout2_derived_clock_CLKIN1	100.0 MHz	NA	10.000	NA	NA	derived (
HAPS80_SYSTEMIP_ls_0_11s_440s_2016_06_1_1_mb_uB_slpgen|I_haps80_systemip_core_I_umr_clk_gen_mmcm_clkout2_derived_clock_CLKIN1	100.0 MHz	NA	10.000	NA	NA	derived (
HAPS_clk_12_5	12.5 MHz	NA	80.000	NA	NA	derived (
HAPS_clk_200	200.0 MHz	NA	5.000	NA	NA	derived (
HAPS_lock_clkin	1.0 MHz	NA	1000.000	NA	NA	declared
HAPS_umr_clk	100.0 MHz	NA	10.000	NA	8.282	derived (
HAPS_umr_clkin	100.0 MHz	NA	10.000	NA	DCM/PLL	declared
clk1	25.0 MHz	138.9 MHz	40.000	7.200	34.747	declared
clk2	33.3 MHz	72.5 MHz	30.000	13.800	16.208	declared
genclk1	12.5 MHz	69.4 MHz	80.000	14.400	72.222	generated
genclk2	6.2 MHz	34.7 MHz	160.000	28.800	131.834	generated
System	1.0 MHz	NA	1000.000	NA	991.746	system

Synchronous Clocks

FDC Constraints

```
create_clock -name {clk1} [get_ports {p:clk1}] -period 40.0
create_clock -name {clk2} [get_ports {p:clk2}] -period 30.0
```

Feedback

```
create_generated_clock -name {genclk1} -source {p:clk1} [get_pins
 {usub1.clkdiv.clkout}] -divide_by 2
create_generated_clock -name {genclk2} -source {p:clk1} [get_pins
 {usub2.clkdiv.clkout}] -divide_by 4
```

Performance Summary in the System Generate Stage

Without clock scaling:

Here, the root clocks *clk1*and *clk2*, and the generated clocks *genclk1* and *genclk2* are scaled independently.

```
@S0.0 |Performance Summary
*******************

Worst slack in design: 3.851

                    Requested     Estimated     Requested     Estimated               Clock                                                                  Clock
Starting Clock      Frequency     Frequency     Period        Period        Slack     Type                                                                   Group
-------------------------------------------------------------------------------------------------------------------------------------------------------------------
HAPS_lock_clkin     1.0 MHz       NA            1000.000      NA            NA        declared {HAPS_lock_clkin}                                             haps_lock_clkgroup
HAPS_umr_clk        100.0 MHz     1221.9 MHz    10.000        0.818         9.182     derived (from HAPS_umr_clkin) {HAPS_umr_clkin}                         default_clkgroup
HAPS_umr_clkin      100.0 MHz     NA            10.000        NA            DCM/PLL   declared {HAPS_umr_clkin}                                              default_clkgroup
clk1                25.0 MHz      415.5 MHz     40.000        2.407         9.398     declared                                                               default_clkgroup
clk2                33.3 MHz      54.2 MHz      30.000        18.447        24.649    declared                                                               default_clkgroup
genclk1             12.5 MHz      20.3 MHz      80.000        49.192        3.851     generated (from clk1)                                                  default_clkgroup
genclk2             6.2 MHz       107.0 MHz     160.000       9.346         150.654   generated (from clk1)                                                  default_clkgroup
System              1.0 MHz       312.8 MHz     1000.000      3.196         996.803   system                                                                 system_clkgroup
===================================================================================================================================================================
```

With clock scaling:

Here, the synchronous clocks *clk1* and *clk2* are scaled to the same ratio. The generated clocks *genclk1* and *genclk2* are scaled with respect to the root clock *clk1*. Scaling rate for *clk1* and *clk2* is 0.62.

```
@S0.0 |Performance Summary
*******************

Worst slack in design: 3.851

                    Requested     Estimated     Requested     Estimated               Clock                                                                  Clock
Starting Clock      Frequency     Frequency     Period        Period        Slack     Type                                                                   Group
-------------------------------------------------------------------------------------------------------------------------------------------------------------------
HAPS_lock_clkin     1.0 MHz       NA            1000.000      NA            NA        declared {HAPS_lock_clkin}                                             haps_lock_clkgroup
HAPS_umr_clk        100.0 MHz     NA            10.000        NA            9.182     derived (from HAPS_umr_clkin) {HAPS_umr_clkin}                         default_clkgroup
HAPS_umr_clkin      100.0 MHz     NA            10.000        NA            DCM/PLL   declared {HAPS_umr_clkin}                                              default_clkgroup
clk1                25.0 MHz      40.3 MHz      40.000        24.800        9.398     declared                                                               default_clkgroup
clk2                33.3 MHz      53.8 MHz      30.000        18.600        24.649    declared                                                               default_clkgroup
genclk1             12.5 MHz      20.2 MHz      80.000        49.600        3.851     generated (from clk1)                                                  default_clkgroup
genclk2             6.2 MHz       10.1 MHz      160.000       99.200        150.654   generated (from clk1)                                                  default_clkgroup
System              1.0 MHz       NA            1000.000      NA            996.803   system                                                                 system_clkgroup
===================================================================================================================================================================
```

Performance Summary in the SLTA

Without clock scaling:

Here, the root clocks *clk1*and *clk2*, and the generated clocks *genclk1* and *genclk2* are scaled independently.

```
@S0.0 |Performance Summary
*******************

Worst slack in design: -7.807

                                                                
```

Feedback

With clock scaling:

Here, the synchronous clocks *clk1* and *clk2* are scaled to the same ratio. The generated clocks *genclk1* and *genclk2* are scaled with respect to the root clock *clk1*. Scaling rate for *clk1* and *clk2* is 1.79.

```
#S0.0 |Performance Summary
********************

Worst slack in design: -7.807

                                                
```

Feedback

ASIC and FPGA Power Domains

ASIC power domains are different from FPGA power domains:

FPGAs	ASICs
Volatile. Lose their configuration when they are powered down; configuration must be reloaded from onboard flash memory on power up.	Do not lose configuration when powered down.
The design cannot be powered down selectively for certain regions.	Can power down parts of the design.
No dedicated power switches.	Dedicated power switches.
No multi-voltage or user-configurable cells.	Power domains can operate at different voltages.
Single power and a single ground for mapping.	Multiple power and ground.

Because of these differences, power domains are implemented as virtual power domains on an FPGA. The device is not powered down, but the tool emulates the behavior of the powered-down state. For details on using UPF specifications, see Including Unified Power Format (UPF) Specifications.

Converting Constraints

The tool processes some standard formats, which reduces the time taken to translate constraints.

- SDC timing constraints

  Migrate to the FDC file for FPGA synthesis as is, because FDC uses the same syntax. If you have clocks that were not converted, use the following *find* commands to find unconverted clocks for analysis:

  ```
  find -seq -hier {*} -filter {@clock==*clockNamePattern*)
  select [find -seq -hier {*} -filter {@clock==*clockNamePattern*}]
  ```

  For information about including timing constraints in the design, see Running the Implementation Flow.

- UPF power constraints

  Put UPF constraints into a file and then read them into the prototype design. See Including Unified Power Format (UPF) Specifications for details.

Checking Resources

Check available resources when you map an ASIC to an FPGA.

1. Run synthesis.
   - For first-pass resource estimation, run fast synthesis strategy.
   - For optimized synthesis, synthesize as usual.
2. Check the Resource Usage report for I/Os, random logic, flip-flops, memory, and clocks. Aim for 50% utilization.

   ```
   RAM/ROM usage summary
   Block Rams: 160 of 156 (102%)
   Register bits not including I/Os: 2500 (50%)
   ```

3. If the design is overutilized, use register resources (distributed RAM) to implement the memories.

   Use the *syn_ramstyle* attribute to guide memory inference.

4

Creating and Compiling Databases

This section describes the basics of the prototyping database model and how to compile the database. The compile step is the initial step in an FPGA synthesis design flow.

After your design is FPGA-ready, set up your design database. A design database consists of all the data files required for the design. The database model consists of various database states. The database states represent the design at intermediate transformative stages.

1. Get your ASIC design ready for FPGA synthesis, placement, and routing.
   - See Converting ASIC Designs to FPGA for a discussion of SoC-to-FPGA conversion issues.
   - See Configuring Environment Variables for environment variable settings.
2. Open the prototyping tool and either create or load a database, with the `database create` and `database load` commands, respectively.

For details on creating and compiling databases, see the following topics:

- Configuring a Design Database
- Loading and Navigating Database States
- Archiving a Database
- Extracting Database Files from an Archive
- Creating Lite Archives
- Compiling the Design
- Adding Design Files
- Using UUM and Group Mapping
- Using Unified Compile
- Checking the UC Database
- Compiling the Design Incrementally

- Specifying Constraints
- Configuring Options

Configuring a Design Database

Every design requires a database. The first step in prototyping a design is to set up the design database.

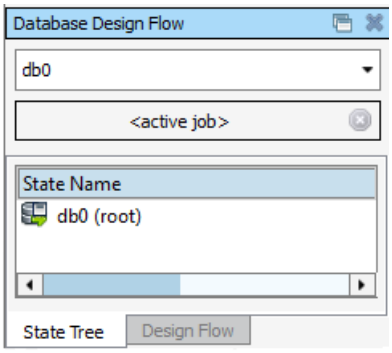
1. Type *database create* followed by the directory location at the tool prompt.

   ```
   database create databaseDirectory
   ```

   You can specify various options with the *database* command line; see database for the complete command syntax.

   To create a database, click **New Database Setup** in the left panel of the tool UI and specify the database directory.

   By default, the design database is created in the current directory. The default name of the database is *db0 (root)*. In the UI, the directory is displayed in the panel on the lower-left of the window. The green arrow next to the DB name indicates that it is in the active state.



   You can run operations on the root database using the **State** menu in the GUI to generate database states. A database state is an implementation of the root *db0* database at a particular design phase, based on the design options. The database states serve as points to check on performance and area requirements. You can tweak constraints and options to ensure that you achieve your design goals.

   The commands work on the active database state (*db0*), unless you load another database or switch to another state.

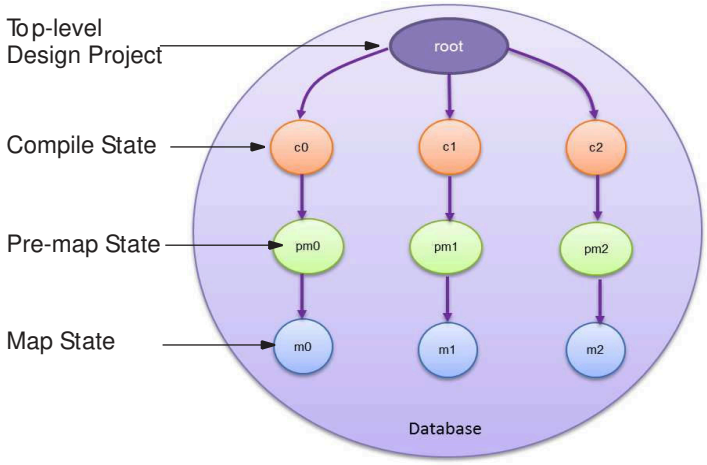
   The database is a closed database. Direct access and use of the files in the database directory is not recommended. Use the `export` commands to export the files you need. The structure of the database and file format can change without notice.

Feedback

The database hierarchy starts at *root* and branches through the states at different design stages. The source files apply to *root* and every state below it. But there might be additional files, such as constraint and option files, that you can specify for different database states.

The following figure is an illustration of synthesis database states with default names. You can return to any database state and rerun a command with a new set of options. This creates a new branch and another parallel database state. For example, you can return to *pm2* and rerun mapping to generate an *m3* state under *pm2*. See Design Database Structure for more information.

Figure 2 Database States



2. Specify the HAPS technology with the `database create -technology` command.

   The specified technology is the default technology associated with this database. It applies to subsequent database states, unless you reset it with the `database apply_state -technology` command at the compile stage. You can have different database states that use different target technologies.

3. If you do not want to use the default database names, you can specify a name:
   - For the *root* state, specify a name when you create the database, with `database create`. In the GUI, you can specify a different name in the dialog box that opens when you create the database.
   - For any other database state, specify the name with the *-out* option when you use the `run` command that generates that state. For example, specifying `run -compile trial1` generates a database state called *trial1* instead of the default, *c0*.
   - Rename an existing state with the `database rename_state` command. Alternatively right-click the state name in the GUI and select **Rename State**. For example, this command renames *sr10* to *sysroute_best_tdm8*:

     ```
     database rename_state {sysroute_best_tdm8} -state {sr10}
     ```

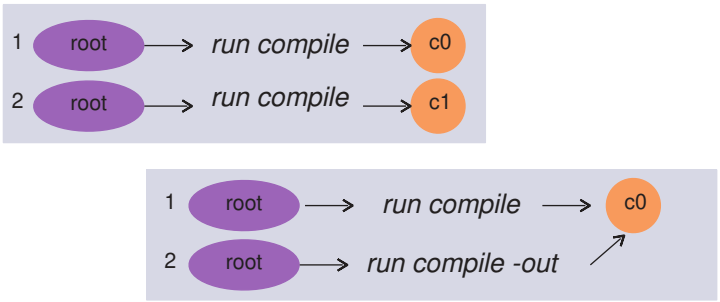
   Note:

   Renaming a state does not change the underlying directory. Both the old and new names are illegal names for new states until you remove the state.

4. To overwrite an existing database, specify the name of the database you want to overwrite using the *-out* option.

   The following figure illustrates how the `run compile -out` command overwrites a database state.

Figure 3 Overwriting a Database State



5. In the GUI, click the **Summary** tab at the bottom of the **Tcl shell** window.



   This tab shows details of all database states run for the current database tree.

6. To populate a database, you must specify the source files.

   For details about adding files, see Adding Design Files.

Loading and Navigating Database States

After you load a database state, you can execute commands on the database. The GUI panel on the lower-left indicates the active database.

Use the *report* commands to view the reports generated after the command is run for the database state. The structure of the database and file format can change without notice.

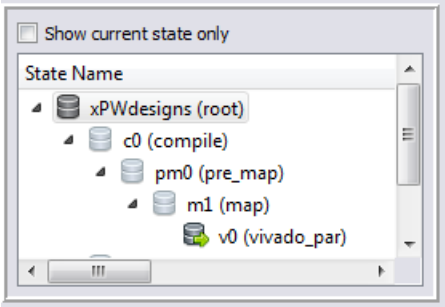
1. To load a database, do the following:
   - If you currently have another design database open, close the active database with the *database close* command. See database for the complete command syntax for the database commands.
   - Type this command at the console prompt:

     ```
     database load databaseName
     ```

     The GUI equivalents for this command are the **Database > Database Open** and **Database > Database Create** commands. If you are starting a design, you can now specify your source files and design options, as described in Adding Design Files.
2. Make sure you are at the right point in the database tree for the implementation commands you want to run.

   All states are preserved by default. The commands operate on data in the active database state. Any state can be accessed at any time. You must be in the appropriate state in the database hierarchy when you run the implementation commands.

   From the command line, use the *database get* command to identify the active database state. In the GUI, the State Name panel indicates the active database with a green arrow. A red database state indicates that the database state has errors.

3. Use the `database set {stateName}` command to switch to another database state.

   In the GUI, double-click on the database state you want in the **Database Evolution** panel to switch to that state.

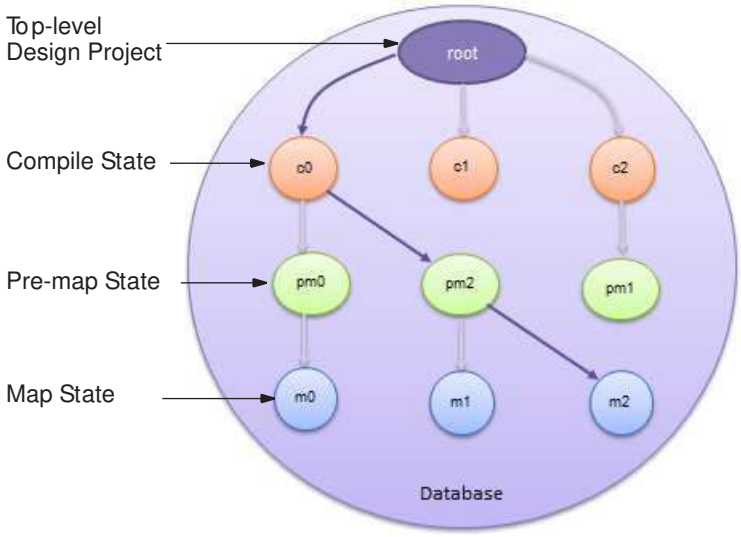
Feedback

You can return to any database state and rerun a command to branch and generate a parallel database state. This modular approach allows you to select the state with which you want to proceed.

The following example shows different branching database states created at the compile, pre-map and map phases.

The database states have default names. They are numbered consecutively as they are generated. The darker arrows indicate the chosen database states at each phase. Premap states are generated from the promising compile states, *c0* and *c2*. But the premap states generated from *c0* (*pm0* and *pm2*) are considered better to generate mapped database states.

Figure 4 Branching into Parallel Database States



The figure shows that you can return to a previous database state at any point. You can switch to the *m1* database state and run place and route to generate *v0*. Similarly, you can return to *c2* and run premap again to generate *pm3*.

4. To remove a database state that you do not want, use `database remove_state`.

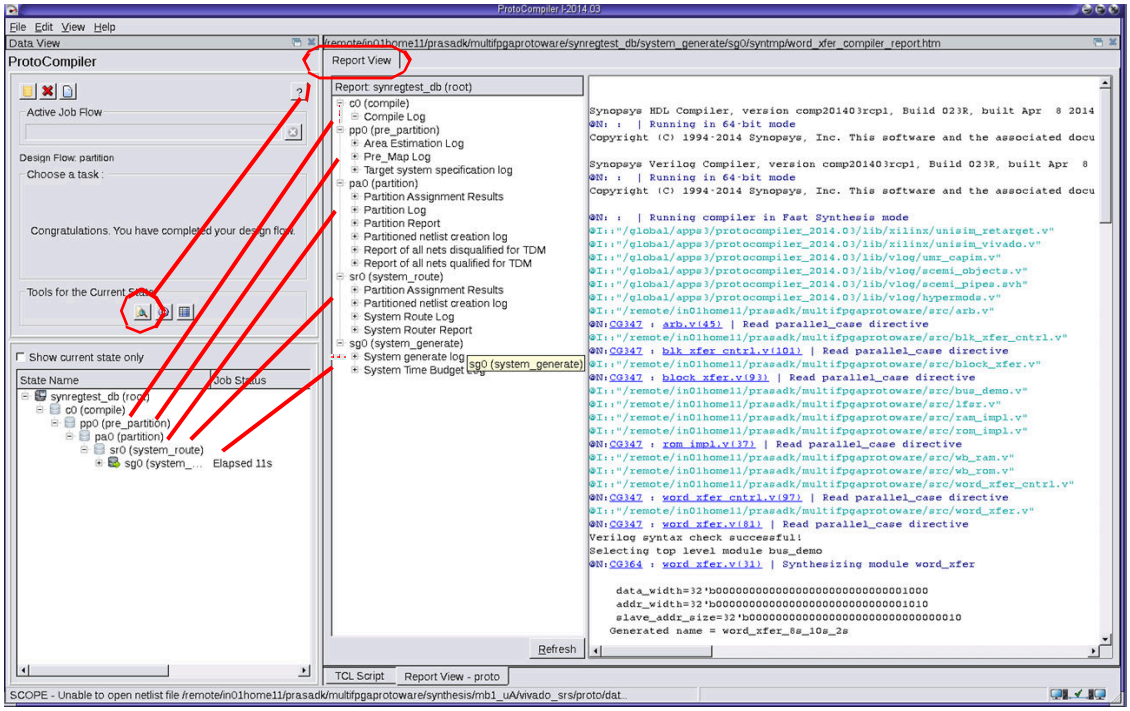
   Alternatively, you can overwrite an existing database that you do not want to keep. Specify the appropriate `run` command with the *-out* option, listing the name of the existing state you want to overwrite. For example, `run map -out m1` overwrites the existing *m1* state, instead of generating *m2*.

5. You can query a database state, or check the files generated for a database state.

Feedback

- For information about querying a database state use the `database query` and `database query_state` commands. For more information about querying, see Querying Incremental Results. For more information about the command syntax, use the `help` command. For example, type `help database query_state`.
- Each database state generates its own reports. Click the **Report View** icon in the GUI to display the reports for the active and previous database states. In the following figure, the active state is the System Generate state. The **Report View** displays the reports for the active and previous states.

Figure 5 Reports for Each Database State



Reports are included in the closed database. To open and save reports outside the database, export them (`export` command) or use the procedures described in Checking Reports and Log Files.

Archiving a Database

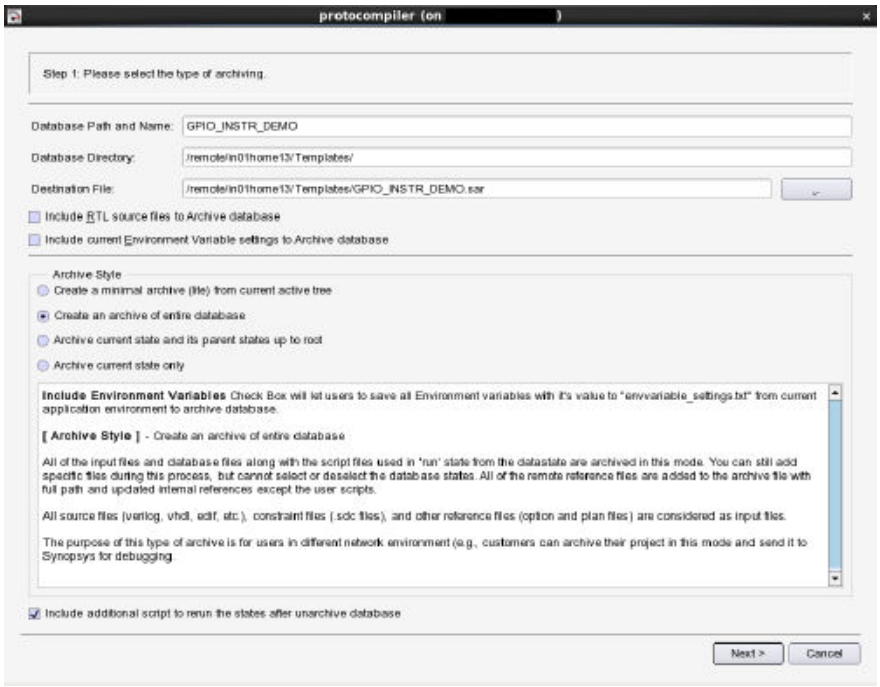
The archive utility stores the files for a database in a single archive file in the proprietary (*sar*) format. You can archive an entire database tree or selected files from the database.

Either use the `database archive` command to create an archive file or use the GUI. The latter provides an easy interface to add, delete, and view the files for archiving.

Feedback

1. In the **Data View**, select **Database > Archive Database** to open the utility.

   The Tcl equivalent to the GUI is the `database archive directoryPath` command. See database archive for the syntax.



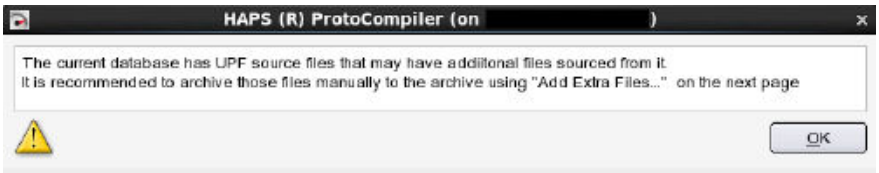
2. Do the following on the Step 1 screen:
   - Enter a location for the archive file in **Destination File**.
   - To include the source RTL files in the archive, select **Include RTL Source** (*database archive -include_source 1*). If the box is not selected the tool archives all user input files, except for RTL files.
   - To include all Environment variables with values in the archive database, select the **Include Current Environment Variable settings to Archive database**. All Environment variables and values are saved to the *envvariable_settings.txt* file in the archive database.
   - Select the **Archive Style**. The equivalent command line option is `-mode`. You can:
     - Archive the entire database (*database archive -mode full*).
     - Archive the current tree (*database archive -mode active_tree*)
     - Archive the current state (*database archive -mode current*)

The *active_tree* mode (default) archives the current database state and its parent states back to root. The *current* mode only archives the current database state. For information about creating a minimal archive, see Creating Lite Archives.

- If you only archive the current state, you cannot use any *run* commands on the archived state later. You can only generate reports and schematics. To use *run* commands to generate downstream database states, archive the active tree or the entire database.

  The archive utility archives just the database and state information files. It does not archive reports or other files. If the current tree contains a pre-map state, the tool archives all the input needed to re-run this state.

  If you have included UPF files to the list of files to be archived, the following message is displayed:



  For information about using run commands with a minimal archive (*-mode lite*), see Creating Lite Archives.

- Click **Next** to display the full uncompressed file size and summary of files to be included in the archive.

3. In the Step 2 screen, deselect the files you do not want to archive.
   - Double-click the directory categories and use the check boxes to add or delete files.
   - You can add additional script files to the archive database using the **Add Extra Folder** or **Add Extra Files** buttons. To add individual script files to the archive database, use the **Add Extra Files** button. To add a directory with multiple script files in it, use the **Add Extra Folder** button. These files are archived and unarchived at the same level as the database.
   - Click **Next** when you are done. This action advances you to the Step 3 confirmation screen.
4. On the next screen, verify the destination and other information.

- Verify that the archive contains the correct files and the archive file name is correct.
- If the list of files is not correct, click the **Back** button and include/exclude the files as needed.
- Click **Archive** to archive the entire database, including source files. If your design has Verilog *'include* files, the archive includes the Verilog files.

5. Check the archive utility report.

   The archive utility reports the file generation success and the path location of the archive file.

6. Click **Done** to close the dialog box.

Extracting Database Files from an Archive

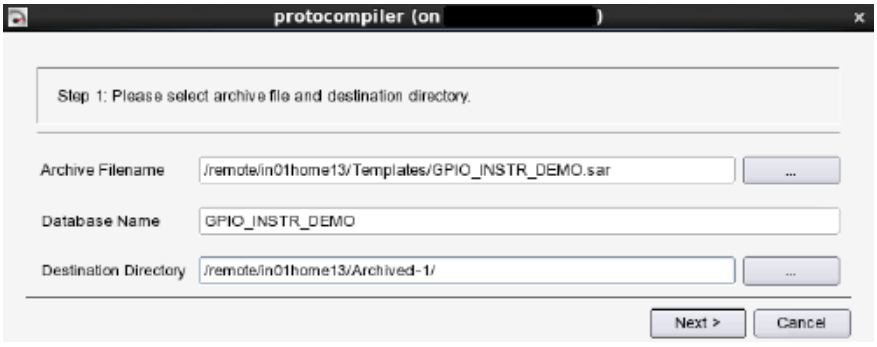
You can also restore the files from an archived database.

Use the following procedure to extract the design database files from a *sar* archive file, or *unarchive* the database. You can also use the `database unarchive` Tcl command to extract the files.

1. From the tool window, select **Database > Un-Archive Database** to display the utility.

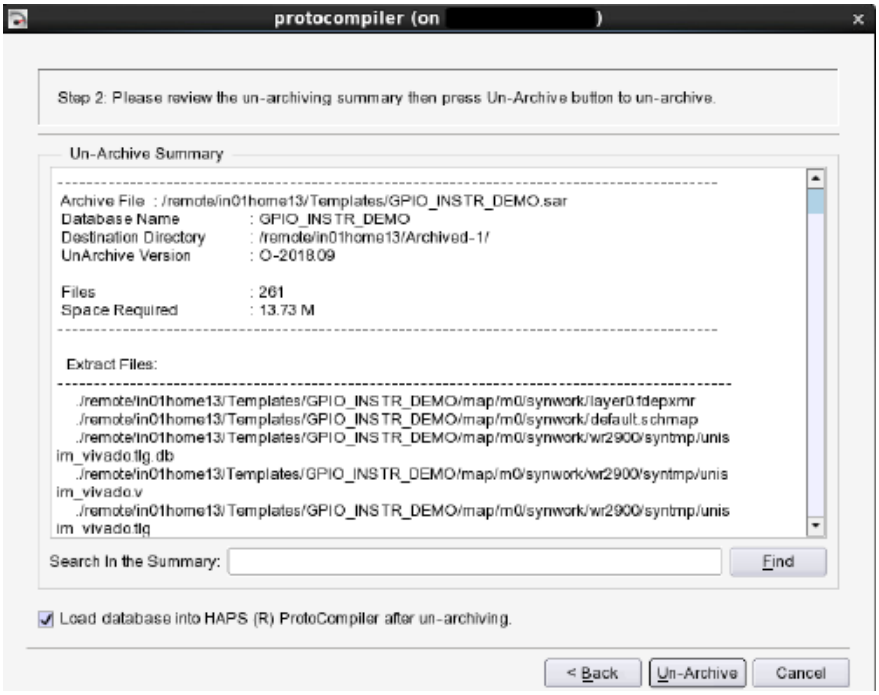
   The Tcl command equivalent is `database unarchive filename.sar`. See database unarchive for the syntax.

2. Specify the following information in the dialog box that opens:
   - Specify the name of the *sar* file containing the database files in **Archive Filename**.
   - Specify a location for the extracted files in **Destination Directory**. If you do not specify a directory, the archive file is written to the current directory.



   - Click **Next**.This displays Screen 2.

3. Do the following in Screen 2:
   - Review the summary list of files to be extracted that is displayed.
   - To load the archived database into the software tool after it has been extracted, check the **Load database into ProtoCompiler** check box.
   - Click **Un-Archive**.



   If the destination directory contains project files with the same name, a confirmation message displays. Click **Yes** or **Yes to All** to confirm that the existing files can be overwritten.

   After extracting the files, a message displays the status of the task and the location of the archive file.

   - Click **OK**.

     An archive summary log file —*archive_summary.log*— is created under the destination directory. Review this log file after you un-archive the database. Apart from the list of files, the log file provides the following details:

     - Version of the ProtoCompiler with which the archive was created
     - Database name
     - Root directory

- Date on which the archive was created
- Number of files
- Number of not readable files
- Total Size of the database excluding the summary log file

Creating Lite Archives

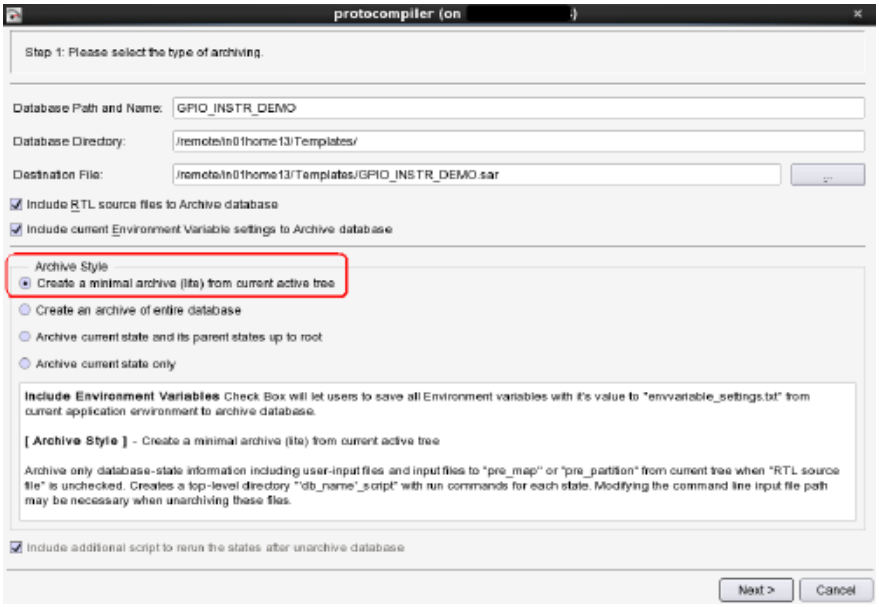
If some files cannot be archived for proprietary reasons, you can create a *lite* archive with the minimum required files.

For information about creating full archives, see Archiving a Database. The following procedure describes how to create a lite archive.

1. Start from a post-compile database state, such as pre-map, and make it the active state.

   The database state does not have to have passed successfully.

2. Create the minimal (lite) archive.
   - Specify the `database archive -mode lite` command from the command line or use the GUI. See database archive for the complete command syntax.
   - In the **Data View**, select **Database > Archive Database** to open the archive utility.
   - On the first page of the dialog box, set **Archive Style** to **Create a minimal archive (lite) from current active tree**.



- On the next dialog box, check the list of files and add or remove files as required.
- Fill in the other options to create the database (see Archiving a Database for details). To include the input RTL files, enable **Include RTL source files to Archive database**. Review the summary of files to be archived.

  The current and parent states up to root are archived. The input required to rerun this state, including the compiled netlists, are archived.
- Click **Finish**

You can now verify the archived files in the database, as described in the next steps.

3. Copy the *db.sar* archive file to a directory to extract and verify it.
4. Extract the archive files by running the `database unarchive` command in the Tcl window.
5. To use `run` commands to generate downstream database states from a lite archive, follow these steps:
   - Unarchive the lite database, which consists of the files for the active tree but which excludes proprietary information.
   - Check the script included in the archive for `run` commands. You might have to edit the paths to constraint files, depending on where you unarchived the lite

database. You can either use the script to execute the `run` commands or specify them manually, as described next.

- Start from the compiled database state and specify the `run` command. Even if you archived a terminal state (for example, map), you must regenerate the intervening states from the compiled state.

Compiling the Design

For the context of this tool, the *compile* operation is more granular than the definition in other Synopsys verification tools: *compile* here only refers to what other tools call “front-end compile.” Subsequent design stages that make up “back-end compile” in other tools (partitioning, mapping, and place-and-route) are separate design stages in the context of this tool.

The compilation stage is where HDL is simplified and mapped to logic gates. During this stage you also validate the input files. Compilation is an iterative process.

The tool uses, the unified compiler (UC), which is a shared VCS compiler front-end that is shared across many tools. With the UC flow, you compile the UCDB database (see Using Unified Compile). After compiling the design, the tool generates a compiled database that can be used for the next stage of the design. The unified compiler, reuses the RTL database generated in the VCS environment.

See these topics for information about compiling with UC and running incremental compile:

- Using Unified Compile
- Compiling the Design Incrementally

Compiler Comparison

The following table lists the features of Unified Compile.

Unified Compile
In other tools that use the same VCS compiler, *compileincludes “back-end” stages like mapping and partition. With this tool,compile* consistently refers to front-end compile only.
Uses the top-level module and Verilog standard defined in the Unified Compile database (*ucdb*).
Supports VHDL and mixed language
Reuses the existing VCS compilation scripts to target HAPS hardware.
The VCS tool analyzes and elaborates RTL files and generates a word-level netlist.

Feedback

Unified Compile
UPF is processed by VCS (unified UPF).
Supports both RTL and post-compile debug instrumentation.
Use the VCS command line options to define all synthesis macros and compiler directives.
Synthesis attributes can be specified as comments in the RTL as well as in a CDC file.
Supports formal verification with Formality.
UDPs are supported
EDIF/NGC input is supported as input to *run compilewith theucdb*option; not supported in the VCS script.
Supported if encryption is handled by the VCS tool: P1735 and *synenc* are supported with the VCS key). For more about encryption, refer to the chapter in the *VCS User Guide* on encryption.
Source-level partitioning not supported.
Only distributed compiler mode supported.

Controlling the Compiler Run with Options and Constraints

You can manipulate the results of the compiler through these controls:

- Specify arguments to the `run compile` command. For example, use the *hdl_define* argument to specify design parameters and compiler directives normally specified with *'ifdef* and *'define* statements in Verilog designs. See run compile for the complete syntax.
- Add compiler directives to a Tcl file with a *cdc* extension Specify them with the `define_directive` command, according to the language used for the module. See Configuring Directives in a CDC File for details.

Verilog	*define_directive {v:[libraryName.]entityName[(architectureName)]} {directive} {value}*
VHDL	*define_directive {v:[libraryName.]moduleName} {directive} {value}*

- Add *option set* command options in the Tcl window or in a Tcl file. See Configuring Options for details about working with options.

Examples of Compiler Options and Constraints

Option/Constraint	Description
Compiler Directives (*cdc* File)	
syn_black_box *name*	Defines a module as a black box for synthesis.
syn_no_prune 1|0	Determines whether unused output ports on instances and black-box modules, including technology-specific primitives, are optimized away. The default is 0, where the ports can be optimized.
syn_preserve 1|0	Prevents sequential optimizations like constant propagation, inverter push-through and FSM extraction for the specified module or entity. The default is 0.
syn_sharing off | on	Specifies whether resources are shared during the compile stage.
syn_rename_modulename	Renames a submodule to avoid naming conflicts between a generated name and the original name.
syn_unique_inst_modulename	Renames an instance of a module without affecting the other instances.
Option Settings (*option set*). Click the Options Editor icon for a comprehensive list.	
auto_infer_blackbox 0|1	Determines whether the tool errors out when it encounters an undefined Verilog module. The default (0) causes the compiler to error out, but when set to 1, the tool creates a black box for the undefined module, issues a warning, and continues with compilation.
automatic_compile_point 0|1	Determines whether the tool automatically divides up the design into compile points, without creating additional hierarchy. Compile points are RTL partitions of the design that are treated as independent blocks by the synthesis engine.
continue_on_error 1|0	When enabled (1), directs the compiler to continue after encountering a non-syntax error and to continue compiling the next design unit without stopping. The default is 1. See Running Compilation or Mapping with Errors for details.
dc_root	Specifies the path to the Design Compiler installation for access to Synopsys foundation and minPower DesignWare libraries. The default is *$SYNOPSYS*.
allow_duplicate_modules 1|0	Allows duplicate module names to be used when set to 1. The software uses the last definition of the module and ignores any previous definitions. The default is 0.

Feedback

Option/Constraint	Description
dw_foundation 1|0	Specifies the Synopsys DesignWare foundation library as the source of the DesignWare building blocks. The default is 1. If you also specify *dw_minpower*, the tool overlays the DesignWare minPower library over the foundation library.
dw_minpower 1 |0	When set to 1, uses the DesignWare minPower library as the source for the DesignWare building blocks. The tool replaces DesignWare foundation library building blocks with the corresponding blocks from the minimum power library. The default is 0.
dw_stop_on_nolic 1|0	Specifies whether compilation stops with an error message when the design contains a Synopsys DesignWare building block but there is no license for it. The default (1) is for the tool to stop with an error message. When set to 0, the tool black boxes the block and continues.
enable_prepacking 1|0	Prepares the netlist for LUT combining during place-and-route, so that the Xilinx tool can pack into the LUT6_2 depending on available resources
synthesis_strategy fast|routability	Runs the compiler in a fast mode that is optimized for prototyping. This mode does not run compiler optimizations. See the *Reference Manual* for a description.
hdl_param -setoption	Sets HDL parameter overrides for VHDL designs
include_pathpath	Defines the search path used by the*'include*commands in Verilog design files. *path* is a semicolon-delimited list of directories where the included design files can be found. The include paths are relative. The software searches for include files in the following order, and stops at the first occurrence of the included file:The source file directoryFiles in *path*, in the specified orderThe project directory
libext.libextName1 .libextName2 ..	Specifies additional library extensions for Verilog library files in addition to *.v*and*.sv*; for example: *.av, .bv, .cv, .xxx, .va, .vas*. The tool searches the directory paths you specified for Verilog library files with these extensions.
library_pathpath	Specifies the paths to the directories which contain Verilog library files. The tool uses *path*to find all included library files and determine the top-level module.
looplimitlimit	Lets you override the default compiler loop limit value of 2000.
max_parallel_jobsnum_of_jobs	Determines the maximum number of processing jobs to run in parallel. Use this in conjunction with *automatic_compile_point* to reduce runtime.

Option/Constraint	Description
top_modulename	Specifies the top-level entity, when the top-level entity does not use the default work library to compile the VHDL files. *name* is the VHDL library, followed by a dot (.) and the name of the top-level entity.
vhdl2008 1|0	Overrides the default VHDL standard (VHDL), and sets the standard to VHDL 2008.
vlog_std v95 | sysv	Sets the Verilog standard to Verilog 95 or SystemVerilog, instead of the default, *v2001*.

Adding Design Files

When you specify design files, they apply to the current database state and all underlying states. This table summarizes the different kinds of files:

Tcl Option Files	You can specify option files for any database state, instead of specifying individual options on the command line. The options specified apply to the current database state and the underlying states. Some options only apply to a specific database state, and can only be specified for that state. You must source the options file for the options to apply. See Configuring Options.
Constraint Files	There are different constraint files. Some, like the *cdc* file, can only be specified for specific database states to which they apply. Generally, you specify these files as an argument to the implementation command for that database state. See Specifying Constraints

For details, see these topics:

- Using UUM and Group Mapping
- Configuring Options
- Specifying Constraints

Using UUM and Group Mapping

Group mapping allows you to use the same design files across different tools through a library mapping mechanism called the Unified Use Model (UUM). Using UUM helps interoperability between tools and optimal design management. The goal is to use the same files for both VCS designs and prototyping. UUM also serves to integrate multiple

IPs from different vendors, where there might be overlapping IP module or *include* file names.

See the following for details:

- About Group Mapping and the lmf File
- Creating an lmf File
- Example: Group Mapping File
- Example: Default Library Mapping
- Configuring Library Options

About Group Mapping and the lmf File

Without UUM and an *lmf* group mapping file, the defines or macros specified from the command line are seen across the entire design regardless of the library where it was compiled. Similarly, defines or macros specified in an RTL file are accessible by all RTL files compiled after the RTL file with the defined macro, without regard to the library where it was compiled or the value of the *multi_file_compilation_unit* option.

The *lmf* group mapping file is a Tcl file that is used with Verilog designs to specify include paths, library directories, compiler directives and Verilog macros that are unique to a given library. The file provides guidance to resolve conflicts when there are overlapping include paths, overlapping modules in multiple libraries without a Verilog configuration file, or when there are macros in the global scope that need to be changed to the library scope. The library mapping file eliminates the need to pre-compile and merge files under a wrapper to specify unique settings, and also lets you restrict the scope of a macro to a library. It is similar to the VCS *synopsys_sim.setup* file, which is used to determine module binding when there are conflicts.

Creating an lmf File

The following steps guide you through the creation of group mapping files, which you then specify from the command line. You cannot specify group mapping files from the GUI.

1. Create one or more group mapping files (*lmf*).

   The following guidelines apply:

   - You can create multiple group mapping files for a design.
   - Each mapping file can provide mappings for any number of libraries.

- If a library has multiple mappings, only the last mapping is honored. The *lmf* mapping files are added in the order specified in the *-lmf* argument to the `run compile` or `run pre_instrument` commands.
- Ideally the *lmf* file should cover all the libraries in the design. If only a a small subset of the libraries have special settings, you can use *__DefaultLibraryMapping__* to set default settings for the libraries with common settings (see the next step for details). If there are missing libraries, the tool searches the other source files outside the mapping file using this order: *include path*, *define*, *library*.
- Settings defined in the library take precedence. For unspecified libraries, the tool uses the regular settings for the design. For example, if you do not explicitly specify the *work* library in the mapping file, the tool uses setting information from the project for files located in this directory.

2. In the *lmf* file, specify the setup to be used for each logical library.

   See Example: Group Mapping File for an example of this file.

   - Inside the *lmf* file, use *set_option* statements with the arguments shown in the following table to define the files and libraries:

-include_path	Specifies different *include_path* options for each library. Paths are separated by semicolons. `set_option -include_path {path1;path2;path3}` Relative paths are resolved relative to the *lmf* file.
-library_path	Specifies paths to Verilog library directories. The tool searches the directories for modules that are instantiated but not defined in the source code. Multiple paths are separated by semicolons. `set_option -library_path {libpath1;libpath2;libpath3}` Relative paths are resolved relative to the *lmf* file.
-hdl_define	Defines a text macro in your source code as a value or character string. You can test for this definition in your Verilog source code using the *'ifdef* compiler directive. `set_option -hdl_define -set {DEF1=32'hFF DEF2=1}` You can list multiple sets of *macro=value* pairs with the *-hdl_define* argument, separating them with spaces.
-lib_ext	Specifies a list of file extensions for the tool to search for in a library directory. The tool searches the directories specified with *library_path* and *include_path* for files with the specified extensions. `set_option -libext {*.v *.sv *.v1}` The example specifies that the tool only search for *.v or .sv* and *.v1* extensions in a library. The order of the file name extensions does not imply an order for the search.

- Use *__DefaultLibraryMapping__* to specify default settings for multiple libraries. Use this option when you have multiple libraries, but only a small proportion have individual sets of options. Specify specific settings for the individual libraries that have different settings, and use *__DefaultLibraryMapping__* to define the common settings for the rest. See Example: Default Library Mapping.

3. Specify library search order and binding options as *run compile -hdl_define* options, as needed to resolve ambiguities.

   For details, see Configuring Library Options.

4. Use the single-line comment style to include comments.

   ```
   //This is a Verilog single-line comment
   ```

5. Once you have created the file, add the *lmf* file with the *-lmf* argument to the `run compile` or `run pre_instrument` commands, so that it is compiled.

   For example: `run compile -lmf {libmap1; libmap2}`

   The tool adds files with the specified extensions from the directories you specified in the *lmf* file, in the order specified. If there are conflicts, the tool honors the last one specified.

   When the design is compiled, each IP is compiled in a separate directory, along with its library, include path and define information, so that different name spaces are maintained. The compiler reads in the *lmf* file, source files, and global options, and resolves them by first honoring the *lmf* options on a per-library basis, and then filling in any missing information in the *lmf* from the global options.

6. Check the results by specifying the `view report` command from a post-compile database state.

   - This command opens the log file, where you can check the library setup information that was added. Notes like the following list the library mapping file used; check that the correct *lmf* file was used.

     ```
     @N:: __SYN_GROUP_MAP__ set: reading /sample_vcs2pc/vloglib.map
     @N::Reading group mapping file: /sample_vcs2pc/vloglib.map
     ```

   - The *Before parsing* section of the log file contains details of the macros, include paths, library search paths, and library extensions for every library as specified in the *lmf* file. The *After parsing* section has the same information as well as information about any RTL defines found. Check that the details are as expected.

- Check for errors:

Error	Cause
Cannot open file	Check for missing include files.
CG389: Reference to undefined module	Check for missing library paths or libexts that result in undefined modules. Check for conflicting modules across libraries and resolve them using a config file or the libgrouporder macro
CG141: Creating black box for module	Check the same causes as for CG389.
Other errors	Check for missing or ignored defines or macros from the *lmf*. Check for syntax errors.

Example: Group Mapping File

Relative paths in include and library paths are resolved from the path to the *lmf* file. This means that in the following example, if the *lmf* file is in *dir1/dir2*, then the path to the include file defined on line 3 is *dir1/dir2/./ip1_includes*.

```
#Options for ip1_lib
 library ip1_lib;
 set_option -include_path {./ip1_includes}
 set_option -library_path {./ip1_libdir}
 set_option -libext .v1
 set_option -hdl_define -set {DEF1=32'd0 DEF2=32'd2}
 set_option -hdl_define -set {DEF11 DEF12}
 endlibrary

 #Options for ip2_lib
 library ip2_lib;
 set_option -include_path {./ip2_includes}
 set_option -library_path {./ip2_libdir}
 set_option -hdl_define -set {DEF1=32'hFFFFFFFF DEF2=32'hBAAD0000}
 set_option -hdl_define -set {DEF12}
 endlibrary

 #Options for ip3_lib
 library ip3_lib;
 set_option -include_path {./ip3_includes}
 set_option -library_path {./ip3_libdir}
 set_option -hdl_define -set {DEF1=32'd256 DEF2=32'd257}
 set_option -hdl_define -set {DEF11}
 set_option -libext .v3
 endlibrary
```

Feedback

Example: Default Library Mapping

If the design has multiple libraries but only a small set have specific settings, (library_path/include_path/*hdl_define*) define the common settings for the remaining libraries with *__DefaultLibraryMapping__*. In this example, *lib1* and *lib2* each have distinct sets of options, but the remaining 98 libraries share the same set of options.

This is an example of the library mapping file (*lmf*) for this design scenario:

```
library lib1;
set_option -library_path …
set_option -include_path …
…
Endlibrary

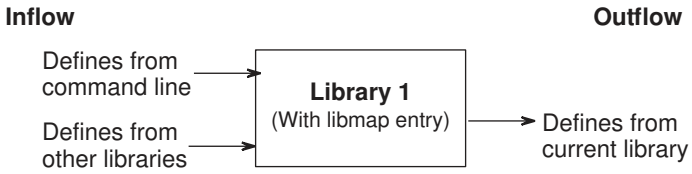
library lib2;
set_option -library_path …
set_option -include_path …
…
endlibrary

library __DefaultLibraryMapping__; #Applies from lib3 to lib100
set_option -library_path …
set_option -include_path …
…
endlibrary
```

Configuring Library Options

The scope of macros defined with *-hdl_define* is confined to the specified library only; it is not global. Say that *temp* is specified in both *lib1* and *lib2* but is defined with-*hdl_define* in *lib1* only, Then if the HDL contains *ifdef temp ... else*code, the *ifdef* code applies to *lib1* and the *else* code applies to *lib2*.

Use *-hdl_define* macros to affect the default inflow and outflow illustrated in the following figure. If an-*hdl_define* macro is defined for the library in the *lmf* file, it overrides the default settings.



Feedback

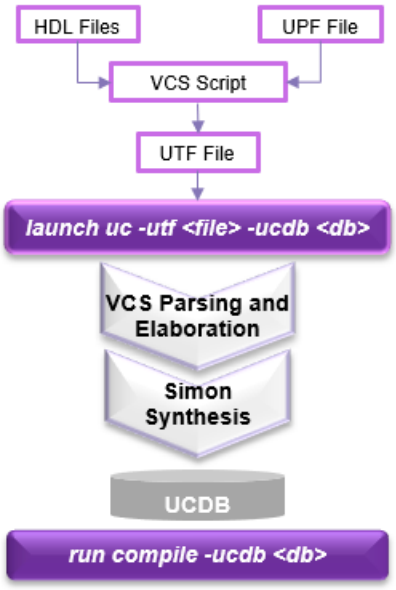
- Inflow is the ability of the macro that is defined in the RTL or the global scope to be available in a library scope, provided there is no *-hdl_define* specified for that library in the *lmf* mapping file.
- Outflow is the ability of the macro to be available in another library scope, as long there is no *-hdl_define* for that library in the *lmf* file.

Using Unified Compile

Use the Unified Compile flow to reuse VCS scripts and options, and to seamlessly move from the VCS environment to FPGA synthesis and prototyping. Semantics, language constructs, and library resolution are all consistent with the VCS software.

The following figure summarizes the compile stage of the UC design flow. Subsequent stages after this follow the regular design flow. The figure shows the two main compile steps, where word-level VCS synthesis run first to create a UC database (*ucdb*), followed by another compile command on the compiled UC database.

Figure 6 Unified Compile Design Flow



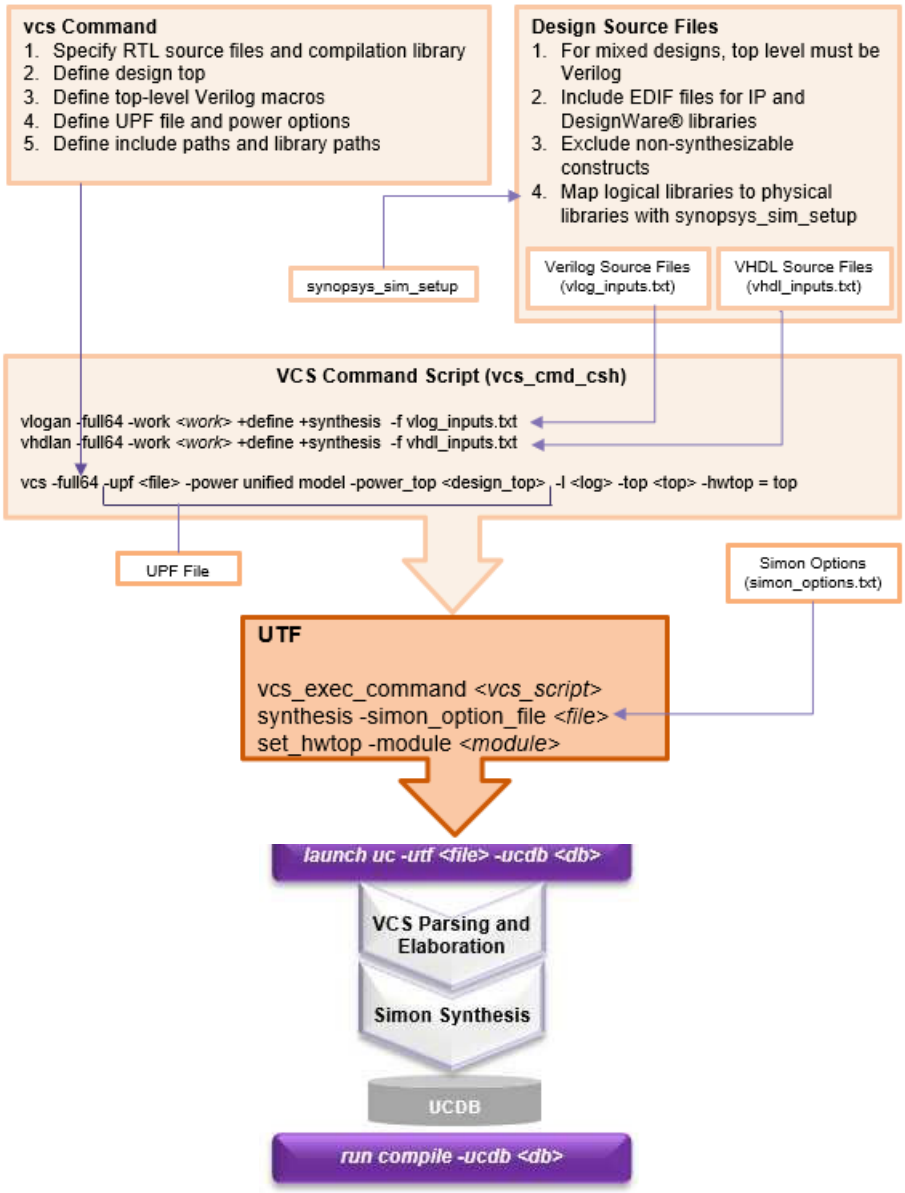
See the following for information on running the UC flow,

- Preparing the Input for Unified Compile
- Running Unified Compile

Feedback

Preparing the Input for Unified Compile

The following figure summarizes how the input for Unified Compile (UC) is structured.



UC Input Files

- HDL files: Verilog, SystemVerilog, or VHDL; or a mixture
- UTF file (points to the VCS script file)

- VCS script file (with or without pointer to UPF)
- UPF file (power information)
- Simon option file (Simon timing engine options)
- Synopsys_sim.setup file (library mapping)
- Tcl script (includes the `database load`, `launch uc`, and `run compile` commands to run UC in the prototyping tool)

For details on configuring these files, see Preparation for the UC Flow.

Environment Variables

Set up environment variables to point to the installations for these tools:

- VCS installation path (UC)
- ProtoCompiler path (prototyping and FPGA synthesis)
- Xilinx Vivado path (place and route)
- Verdi installation path (verification)
- Design Compiler installation path (IP)
- Formality installation path (*zFmCheck*)

Preparation for the UC Flow

The following topics provide tips to set up the files for the initial VCS front-end compile. Consult the *VCS User Guide* for further information.

- Configuring RTL Design Files and Library Mapping
- Configuring UTF File and the VCS Script
- Adding IPs in a UC Design
- Adding Complex SystemVerilog Port Modules

Also, see:

- Handling HDL Differences
- Converting Older Designs for Unified Compiler

Configuring RTL Design Files and Library Mapping

This procedure provides some guidelines on setting up the design source files and for mapping libraries; it is not exhaustive. The example snippets mainly show Verilog syntax;

Feedback

for the equivalent VHDL syntax and further details about the information here, see *VCS User Guide*.

1. Use design source files that are in Verilog or VHDL formats, or a mixture of these languages.
   - Make sure the top level of the design is the synthesizable DUT, not the testbench.
   - For mixed designs, the top-level module in the design must be Verilog. If you have a VHDL top-level entity, create a Verilog wrapper for it and then specify the top level as usual (see Step 4).
2. If you have modules that are included multiple times, use the recommended structure to avoid compile time errors.

   Define a macro which includes a declaration of the module:

   ```
   `ifndef sample_mac
   `define sample_mac
   module myModule;
   endmodule
    endif
   ```

   The tool uses the declaration of *myModule* for multiple instantiations.
3. Exclude non-synthesizable constructs using one of the following methods:
   - Enclose non-synthesizable constructs inside *translate_off/translate_on* pragmas, using either of the syntax styles:

     ```
     /* synthesis translate_off */
     …
     /* synthesis translate_on */
     ```

     or

     ```
     /* synopsys translate_off */
     …
     /* synopsys translate_on */
     ```
   - Add *-skip_translate_body* to *vlogan/vcs* to skip contents inside *translate_off/translate_on*. Use the same option with *vhdlan* to skip contents in VHDL designs.
4. Include IPs and other components by following the guidelines in Adding IPs in a UC Design.
5. To add instrumentation, use SystemVerilog assertions (SVA), and the *$dumpvars* (SystemVerilog (or $dumpports (VHDL) constructs.
6. Map logical libraries to physical libraries using the *synopsys_sim.setup* file.

Feedback

The VCS software allows multiple logical libraries, each of which can point to a different physical library. The file syntax to map a logical library to a physical library is shown. Logical library names are case-insensitive.

```
logical_name: physical_name
```

This is an example of the file. The first two lines specify that the default logical library, *WORK*, is mapped to the default physical library, work. The remaining lines specify the physical mapping for other logical directories and the DesignWare IP libraries to be added to the setup file:

```
WORK > DEFAULT
DEFAULT: ./work
L1 :  ./lib/lib1
L2 :  ./lib/lib2
L3 :  ./lib/lib3
Lw1 : ./lib/lw1
Lw2 : ./lib/lw2
dw01 : ./dw01
dw02 : ./dw02
dw03 : ./dw03
dw04 : ./dw04
dw05 : ./dw05
dw06 : ./dw06
dware : ./dware
gtech : ./gtech
```

Configuring UTF File and the VCS Script

The UTF (Unified Tcl Format) file is an input file that specifies the design information for unified compile. It must point to the VCS script. Create a VCS script or specify the individual commands. The essential command in the UTF file is `vcs_exec_command`.

For further details about the script and the UTF file, see *VCS documentation*.

1. Create a UTF file that points to the VCS script for compilation.

   The required command is `vcs_exec_command`.

2. Set up the VCS script.

   - Specify the required command, `vcs_exec_command`. For example:
     `vcs_exec_command {vcs_dut.csh}`
   - Include the mandatory `vlogan` and `vhdlananalysis` commands, according to the HDL used for the design. The `vlogan` and `vhdlan` commands must be separated in the file. For information about the VHDL commands and any details about the VCS flow, see *VCS User Guide*, which contains both Verilog and VHDL examples and detailed descriptions.

- Include the required *vcs* elaboration command.
- Include other VCS script commands that are optional. Some commonly used commands are described in the following steps.
- To get information about VCS script syntax, use `vcs -full64 -doc`.

3. Specify the top-level module in the VCS script.
   - Specify the synthesizable design under test (DUT) as the top-level for analysis and elaboration in the VCS script.
   - Define the top module with the *-hw_top* option.

     ```
     -hw_top= designUnderTest
     ```

     The *hw_top* model should be the top module of the synthesizable RTL. This option specifies that only the design below the specified module is to be elaborated.

     Examples:

     ```
     vlogan -full64 -sverilog sub_ip1.v
     vlogan -full64 -sverilog sub_ip2.v
     vlogan -full64 -sverilog dut.v
     vcs -full64 -sverilog dut -hw_top = dut
     ```

     or

     ```
     vcs -full64 -sverilog sub_ip1.v sub_ip1.v dut.v -top dut
     -hw_top=dut
     ```

   - For synthesis and prototyping, exclude the testbench and transactors from the design setup. If the DUT has cross-module references (XMRs) from the testbench or transactors to the DUT, exclude them as well.
4. Explicitly define macros in the VCS script.

   You cannot have implicit macros in the unified compile flow, so you must explicitly define any predefined macros. For example, if you have any *synthesis* or SYNTHESIS macros, they must be explicitly defined:

   ```
   vlogan -full64 +define+SYNTHESIS +define+synthesis -sverilog ip_1.v
   ```

5. Specify memory initialization files with *$readmemb/$readmemh*, and specify the associated include paths with the UTF `memory_preferences -scan_path` command.

Feedback

6. To include low power information, do the following:
   - Model low-power behavior in a UPF (Unified Power Format) file.
   - Add the *-upf* option to the VCS script to specify the UPF file. For example:

     ```
     vcs -full64 top -hw_top = top -upf designName.upf
     ```

     For the complete syntax, see *VCS documentation*.
   - If you have multiple UPF files, use one UPF file to source the others. VCS scripts only accept one UPF.

Adding IPs in a UC Design

A UC design can include IP and other components. Different components are described in the following sections:

- Adding Xilinx XPMs to the UC Design
- Adding EDIF or NGC Files in a UC Database
- Adding Encrypted IP to a Unified Compile Database

For further information, including encryption support, see *VCS User Guide*.

Adding Xilinx XPMs to the UC Design

Adding Xilinx Parameterized Macros (XPM) into a design enables the tool to read pre-defined RTL codes automatically and add the XPM definitions directly into the design during synthesis. It improves timing for the design and provides better synthesis results.

To use Xilinx XPMs in the UC flow, follow these steps:

1. Set the *XILINX_VIVADO* environment variable to a valid Vivado installation location.
2. Define the XPM library mapping in the *synopsys_sim.setup* file.

   ```
   xpm : ./xpm
   ```

3. Add the following code in the VCS script, to compile the SystemVerilog XPM files.

   ```
   ###   Create work library ###
   mkdir xpm


   ###  Compile XPM Files in VCS  ###
   vlogan -work xpm
    $XILINX_VIVADO/data/ip/xpm/xpm_memory/hdl/xpm_memory.sv
   vlogan -work xpm $XILINX_VIVADO/data/ip/xpm/xpm_fifo/hdl/xpm_fifo.sv
   vlogan -work xpm $XILINX_VIVADO/data/ip/xpm/xpm_cdc/hdl/xpm_cdc.sv
   ```

Feedback

Adding EDIF or NGC Files in a UC Database

Although the synthesis tool can directly read EDIF files, they must be handled differently when using unified compile because the VCS software cannot parse these files. You can either include the files with the *-srclist* argument, or use the bottom-up approach.

Including EDIF or NGC Using -srcList

1. Create black box modules for the IP.

   Specify the black box definitions when you compile the design. Link the modules into the compile state as described in the next two steps.

2. Create a text file that contains the files to be added.

   List the EDIF or NGC files in the text file, using this syntax:

   ```
   -type edif adder.edf
   -type ngc adder.ngc
   ```

3. When you compile, use the *-srclist* argument and specify the text file you created in the previous step:

   ```
   database create db0
   launch uc -utf run.utf -ucdb ucdb
   run compile -ucdb ucdb -srclist EDIFfiles.txt
   ```

Including EDIF or NGC Using the Bottom-Up Flow

1. Compile the EDIF and NGC into a separate database, using the `run compile` command.

   Use the *-srclist* argument with `run compile` to list the EDIF and NGC files. The EDF and NGC files can be listed in the same text file; use *-type* to distinguish the kind of files as described in Including EDIF or NGC Using -srcList.

2. For the rest of the design, launch UC and create the UC database as usual.

3. At the *run compile*stage, specify the *-src* option to “stitch” or incorporate the EDIF or NGC into the UC database.

   This is an example of a typical sequence of commands, where *db0* is the EDIF database that is compiled separately and then added to the UC database, *db1*. The *add_files.txt*file contains the *-type*list of EDF/NGC files to add.

   ```
   database load db0 -autocreate
   run compile -srclist add_files.txt -top_module adder -out add
   database load db1 -autocreate
   option set design_flow synthesis
   launch uc -utf run.utf -ucdb ucdb
   run compile -ucdb ucdb -type state -src db0|add
   ```

Feedback

Adding Encrypted IP to a Unified Compile Database

You can add encrypted IP to your design, if it is encrypted with the VCS key.

1. Encrypt the IP with the VCS key.

   For detailed information refer to the chapter on encryption in the *VCS User Guide*.

2. Add the RTL for the IP as with any other RTL file.
3. Run the UC flow, and the IP is treated as any other RTL.
4. Check the results.
   - Open the schematic viewer and check that the IP module is displayed. You should be able to view the details in the IP module.
   - If the IP module is not linked successfully and becomes a black box, review the *ucdb/src/wrapperFile* and the IP top-level file and check that the module interfaces are identical. Most linking errors are caused by a mismatch in the module interfaces.

Adding Complex SystemVerilog Port Modules

Use the method described here to create a wrapper for a complex port module that was synthesized with another Synopsys tools and instantiate it in the RTL design.

1. Compile the design using UC.
2. Use `zFMcheck` to generate a simple vector port wrapper:
   - Run the `zFMcheck` command from the *installDir/zebu/bin* directory.
   - To generate a wrapper based on the hierarchy instance name, use the following command, where *-d* specifies the UC database directory generated in step 1:

     ```
     zFmCheck -gen_rtl_wrapper_hier hier_instance_name -o zfm workdir -d
      ucdb/design/defaultGroup
     ```

   - To generate a wrapper based on the module name, use this command:

     ```
     zFmCheck -gen_rtl_wrapper_module module_name -o zfm workdir -d
      ucdb/design/defaultGroup
     ```

   Example

   ```
   $BUILD/zebu/bin/zFmCheck -d ucdb -gen_rtl_wrapper_module sub1 -o
    wrappermodule_sub1
   $BUILD/zebu/bin/zFmCheck -d ucdb -gen_rtl_wrapper_module sub2 -o
    wrappermodule_sub2
   ```

3. Prepare the files and compile the design.

Feedback

- In the RTL code, replace the original code for the instances (*sub1* and *sub2* in this example) with their top wrapper module names. The following examples show the wrapper file paths for complex port modules *sub1* and *sub2*:

```
wrappermodule_sub1/gen_rtl_wrapper/sub1_wrapper.sv
wrappermodule_sub2/gen_rtl_wrapper/sub2_wrapper.sv
```

- In the UTF file, declare the wrappers as black boxes:

```
vcs_exec_command {vcs_dut_wrap.csh}
synthesis -generate_db_for_fmcheck {true}
synthesis -blackbox sub1_sub_wrapper
synthesis -blackbox sub2_sub_wrapper
```

- Compile the design with the `launch uc` and `run compile` commands, as usual.

```
launch uc -utf runwrap.utf -ucdb ucdb_wrap
run compile -ucdb ucdb_wrap -out c0_wrap
```

4. Link in the port modules.
   - Compile the port modules using the native compiler:

```
run compile -srclist files_list.txt -top_module sub1_sub_wrapper
-out c_sub
run compile -srclist files_list.txt -top_module sub2_sub_wrapper
-out c_sub
```

   - Link these compiled databases for the port modules to the compiled database from the end of step 5.

```
run compile  -srcstate pcucdb_wrap|c0_wrap -srcstate
pcucdb_sub1|c_sub
-srcstate pcucdb_sub2|c_sub -top_module top -out c0_top
```

Running Unified Compile

The following procedure describes how to run Unified Compile from the FPGA prototyping tool.

1. Set up the design to run unified compile.
   - Follow the guidelines in Preparing the Input for Unified Compile.
   - Make sure to use compatible versions of the VCS and synthesis tools. Check the release notes for details.
   - Start with a VCS script.

- If you have IP, follow the recommendations in Adding IPs in a UC Design.
- For UPF information, create a UPF file and include it with the `vcs` command as described in Configuring UTF File and the VCS Script.

2. Create a UTF file that points to the VCS script for compilation.

   The UTF (Unified Tcl Format) file is a common file for unified compile, where you can specify the design information. Not all UTF commands are supported. Check that the main command (`vcs_exec_command`) covers both the analyze and elaboration steps.

3. Start the unified compile process.

   - Start the tool, and create a database. Make sure to specify the technology, which can vary. For example, use HAPS-1004F for HAPS-100 designs. For ProtoSynthesis designs, set the technology to *Virtex-UltraScalePlus-FPGAs*.

     ```
     database load uc_debug -autocreate -technology technology
     ```

     If you are working with an existing database, load it with the `database load` command.

   - To use formal verification, enable verification mode: *option set verification_mode 1*.
   - Launch the unified compiler from the Tcl window.

     ```
     launch uc -utf <path_to_utf_file> -ucdb <path_to_uc_database>
     ```

   For details about the command syntax, refer to the corresponding descriptions in the *Command Reference*. The `launch` command uses the VCS tool to generate a netlist in the UC flow. After the VCS software elaboration stage, the word-level synthesis tool generates an intermediate netlist, which is then processed by the synthesis tool.

   The default UC version is 2.0. You can specify version 1.0 with the *-v* option to the command, but it is not recommended.

4. Check the log file for errors and ensure that the results are clean.

   The *uc.log* file is generated by the VCS software and is located in the directory specified for-ucdb. For example: `ucdb/uc.log`. The log file contains the reports

Feedback

generated by the VCS tool. You can also check for UPF errors at this point. This is an excerpt:

```
=======================================================
VCS pre-synth CPU Time        : 0.28 sec
VCS pre-synth Wall-Clock Time : 1.00 sec
VCS pre-synth Current Memory  : 307 MB
SIMON Master CPU Time         : 0.02 sec
SIMON Total  CPU Time         : 0.02 sec
SIMON Master Wall-Clock Time  : 0.15 sec
SIMON Master Current Memory   : 309 MB
SIMON Master Peak Memory      : 309 MB
SIMON CGroup Memory           : 391 MB
=======================================================
Info: Simon VCS Finished
Info: Simon call complete
Info: Exiting after Simon Analysis
CPU time: .360 seconds to compile
```

If you get an error about previously declared modules (*Error-[MPD] Module previously declared*), you have two options. Either fix the duplicate modules as described in Configuring RTL Design Files and Library Mapping, or include the *-error=noMPD* option at the `vcs` command line to downgrade the error to a warning and allow duplicate modules.

5. Compile the database generated by the unified compiler.
   - Specify synthesis attributes as comments in the RTL or in a CDC file.
   - Set options in a Tcl options file. Most options, like resource sharing and retiming, are optional but the *technology* option is required. For encrypted designs, the `result_format` **vm** option is also required. If **vm** option is not specified, you might get the INBB-3 error at Vivado:

     ```
     ERROR: [DRC INBB-3] Black Box Instances: Cell
      'sysip_inst/haps_ufpga_pctd_ip_wrapper' of type
      'haps_ufpga_pctd_ip' has undefined contents and is considered a
     black box. The contents of this cell must be defined for opt_design
      to complete successfully.
     ```

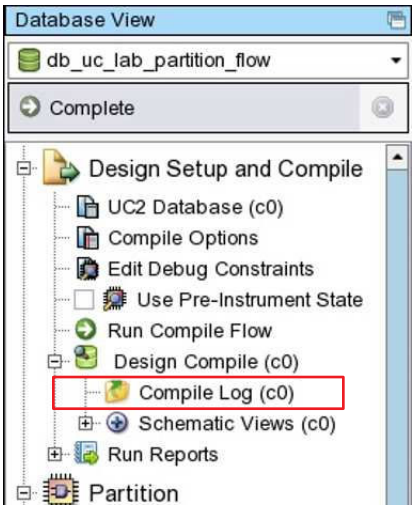
     Source the options file.
   - Compile the design:

     ```
     run compile -ucdb <pathUCdatabase> -cdc <cdcFilePath>
     ```

   The process runs in distributed compile mode, ignoring any specified setting for parallel processing. If the design cannot be divided into parallel processes for distributed compile, the compiler runs in monolithic mode and prints the following message:

   ```
   @N: This design does not have sufficient nodes/jobs for distribution,
   running in monolithic mode.
   ```

6. Check the results.
   - Check the *Unified Compiler Report* section of the log file generated after `run compile`. Use the `view report -state c0` command to open the report from the command line. From the GUI, go to the **Database View** panel and click **Compile Log**.



   - Run the `report ctc` command and check the log file it generates for clock information summaries.

7. Continue with the rest of the flow as usual.

   To run the partition flow, continue with the `run pre_partition` and `run partition` and other commands.

Checking the UC Database

Besides the UC log file (*ucDatabase/uc.log*), there are other tools and techniques available to validate the UC database.

See the following for details:

- Validating VCS Design Files
- Running Lint Checks
- Checking for CDC Violations

Validating VCS Design Files

You can check the elaborated RTL from VCS against the Simon output netlist using either the simulation (VCS) or formal verification (Formality) tools. The functionality used for validation is called *zFmCheck*.

1. Add this command to the UTF file:

   ```
   synthesis -generate_db_for_fmcheck true
   ```

   This creates readable RTL inside the *ucdb* folder: *ucdb/design/defaultGroup/src_for_fm/*

2. From a terminal, run the `zFmCheck` command on this database.
   - To run Formality equivalence checking, use the *-scm* argument with the command:

     ```
     zFmCheck -scm -d design_dir -o output_dir -l time_for_run -j parallel_jobs -t top_module
     ```

   - To run VCS simulation, use the *-scm_sim* argument with the command:

     ```
     zFmCheck -scm -scm_sim -d design_dir -o output_dir -l time_for_run -j parallel_jobs -t top_module
     ```

     The command runs VCS simulation when formal verification fails.

   - Use the `zFmCheck -help` command to get immediate information about the syntax.

   For an overview of the command and some examples, see zFmCheck Command.

3. Check the failures.
   - Check the log file generated after the run. See the example at zFmCheck Log File Example.
   - Check *ucdb/design/defaultGroup/failure_list/Readme.txt*.
   - Run simulation only on the failures.
   - Run the command with Verdi to debug issues: `zfmCheck -verdi_debug`

Feedback

zFmCheck Log File Example

```
#########################SCM Formality summary#####################################
  EQUIVALENT:   5899  (97%)
  DIFFERENT :     16  (0%)
  UNKNOWN   :    170  (3%)
      |--REF_SETUP_FAIL      :     22
      |--IMP_SETUP_FAIL      :      2
      |--UNMATCHED_POINT     :    127
      |--MULTIPLE_DRIVEN     :      4
      |--TIMEOUT             :     15
  BLACKBOX  :      0  (0%)
  UNVERIFIED:      0  (0%)
  SKIPPED   :      1  (0%)
      |--UNSUPPORTED_BY_FM   :      1
          |--CONDITION_MISMATCH_FOR_EVENT_CONTROL_LIST  :      1
  CLUSTER   :      0  (0%)
-------------------------------------------------------------------------------
               6086
###############################################################################
#####################Scm Simulation summary####################################
Modules       : 186
PASS          : 140
FAIL          : 7
ERROR/MISSING : 39
RUNTIME-ERROR : 0
```

Running Lint Checks

You can use VC Static functionality to run lint language and structural checks early in the design cycle.

1. Set up a VC Static configuration file (*.txt*), based on a sample file or a template.

   See the **VC Static Sample Configuration File** section in Checking for CDC Violations for more about the contents of the file .

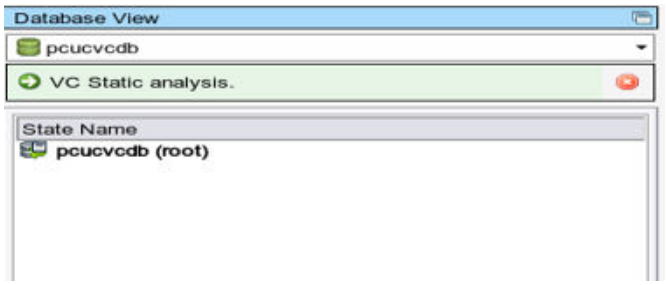
   Copy the sample file *(vc_static_config.txt)* from *install_dir/lib/spyglass* into the working directory.

   Set the mode to *PCLINT*.

   The sample file points to a default template for lint mode. To customize the template (*vcstatic_lint_template.tcl*), copy the template from *install_dir/lib/spyglass* to your working area, and customize it for your design. Do not modify the parts indicated in the template comments. Then point to the template you want to use from the sample file. For template format, see the **CDC Template File** section in Checking for CDC Violations.

2. Open the database and specify the configuration file, before running compile.

Feedback



- You can do this from the command line with the `report vc_static` command:

  ```
  report vc_static -utf myUTF.utf -path dir -config
  vstatic_config.txt
  ```

- Alternatively, select **Tools > Report vc_static** and specify the configuration file from the GUI.

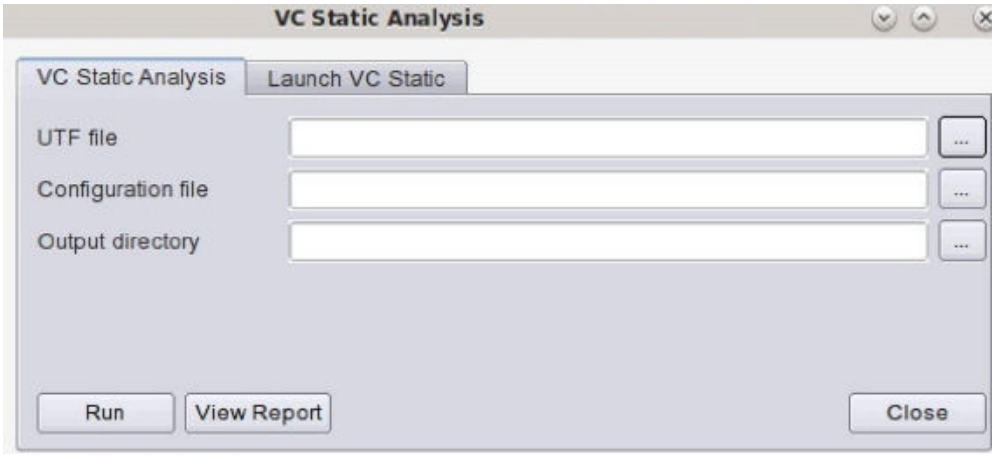
3. Run unified compile and compile, as usual.
4. Run pre-partition with VC static to generate a report.
   - From the command line:

     ```
     run pre_partition -vc_static_cdc_path outputDir -tss file -fdc file
     ```

   - From the GUI:

     Go to **Tools > Report vc_static**, specify the parameters, and click **View Report**. Note that the UTF file can be the same file as for unified compile, or can be a separate file.

     The path for the output directory must be the same relative or absolute path specified with the `report vc_static` command.



The tool runs static analysis and generates a generates a *cdc.pcf* file with constraints for partitioning. The tool also applies the *syn_preserve 1*, *syn_replicate 0*, and *syn_allow_retiming 0* attributes to all synchronization registers.

5. Export the results and analyze them:
   - Use this command to export results:

     ```
     export file -all -path outputDir
     ```

     Check the partitioning constraints in the exported *cdc.pcf* file.
   - Use the `view reports` command to display and check the log file, *report_lint.log*.

Checking for CDC Violations

In larger designs with asynchronous clock networks, clock domain crossing (CDC) violations can be a problem. Use VC Static to identify synchronization logic early in the cycle and set partitioning constraints that keep synchronization logic together.

1. Set up a VC Static configuration file (*.txt*), based on a sample file or a template.

   See VC Static Sample Configuration File for more about the contents of the file .

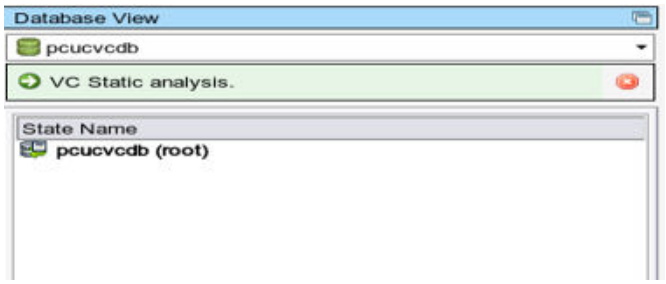
   Copy the sample file *(vc_static_config.txt)* from *install_dir/lib/spyglass* into the working directory.

   Set the mode to *PCCDC*.

   The sample file points to a default template for CDC mode. To customize the template (*vcstatic_cdc_template.tcl*), copy the template from *install_dir/lib/spyglass* to your working area, and customize it for your design. Do not modify the parts indicated in the template comments. Then point to the template you want to use from the sample file. For template format, refer to CDC Template File.

Feedback

2. Open the database and specify the configuration file, before running compile.



   - You can do this from the command line with the *report vc_static* command:

     ```
     report vc_static -utf myUTF.utf-path dir -config
      vstatic_config.txt -show_report 1
     ```

   - Alternatively, select **Tools > Report vc_static** and specify the configuration file from the GUI.

3. Run unified compile and compile, as usual.

4. Run pre-partition with VC static.

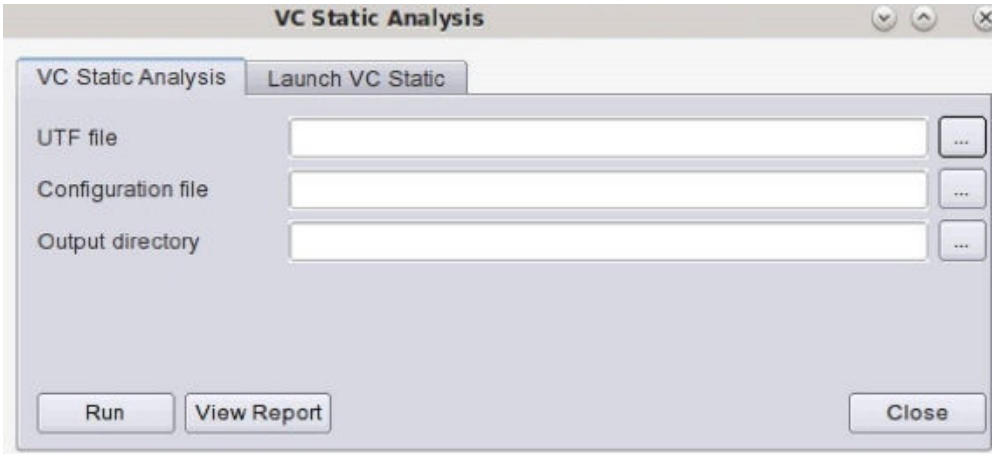
   - To run it from the command line:

     ```
     run pre_partition -vc_static_cdc_path outputDir -tss file
      -fdc file
     ```

     To run it from the GUI, go to **Tools > Report vc_static**, specify the parameters, and click **View Report**. Note that the UTF file can be the same file as for unified compile, or can be a separate file.

     Initially, ensure to set the environment variable *VC_STATIC_HOME* to the relevant branch before invoking the VC Static feature. You must unset it for the next VC Static analysis.

     The path for the output directory must be the same relative or absolute path specified with the `report vc_static` command.



The equivalent Tcl command to run VC Static and load the CPDB database is `launch vcstatic` (launch vcstatic in the *Command Reference*).

The tool runs static analysis and generates a generates a *cdc.pcf* file with constraints for partitioning. The tool also applies the *syn_preserve 1*, *syn_replicate 0*, and *syn_allow_retiming 0* attributes to all synchronization registers.

- Export the results and analyze them:

  ```
  export file -all -path outputDir
  ```

  Check the log file and the partitioning constraints in the exported *cdc.pcf* file.

5. Use the constraints to guide partition.

Example of Command Sequence for VC Static Integration

```
database load ucvcdb -autocreate -technology HAPS-200
report vc_static -utf uchaps.utf -path vcstatic -config
vcstatic_config.txt -show_report 1
launch uc -utf uchaps.utf -ucdb ucdb
run compile -ucdb ucdb -out c0
run pre_partition -vc_static_cdc_path vstatic -tss board.tcl -fdc
top.fdc -out pp0
export file -all -reports_pp0
run partition ...
run system_route ...
run system_generate ...
```

Feedback

VC Static Sample Configuration File

The sample file is located in the *install_dir/lib/spyglass* directory. The configuration file to run VC Static and identify clock domain crossing (CDC) violations must include these parameters:

Mode	PCCDC, PCLINT, or both
SDC(Synopsys Design Constraints)	CDC flow. Design constraints must match the FDC constraints. You can enter FDC constraints here, but do not use FDC prefix notation, like *n:*, which is not supported.
Templates: *vcstatic_cdc_template.tcl* *vcstatic_cdc_template.tcl*	(Customization optional). Default templates are located in *install_dir/lib/spyglass.*

This is the sample file, *vc_static_config.txt:*

```
## Choose PCCDC or PCLINT as the mode, or both separated by "|" (PCCDC|PCLINT).
MODE=PCCDC|PCLINT

## Provide a list of SDC files separated by "|" (file1.sdc|file2.sdc).
## SDC definitions should match exactly with the RTL definitions.
SDC=file1.sdc | /path1/file2.sdc

## Enter user-defined cdc_template.tcl file and lint_template.tcl files.
## The default templates are available in the <PC_BUILD>lib/spyglass/ directory.
## You may copy these templates and prepare customized templates to run the flow.
## If relative paths are used for the customized templates, make sure paths are relative to the current working di
.
## If the following lines are commented out, templates in the default directory <PC_BUILD>lib/spyglass/ are used.

cdc_user_template=<cdc_user_template.tcl>
lint_user_template=<lint_user_template.tcl>
~
```

CDC Template File

You can customize the template, but do not modify the parts indicated.

Feedback

```
################### DEFAULT CDC TEMPLATE FILE USED FOR VC_STATIC FLOW IN PROTOCOMPILER  ###################

set session_name vccdc.daidir/run_vcsgcdc

# Setting application variable to enable VCSGCDC-PC flow
set_app_var enable_cdc true
set_app_var read_synopsys_sim_setup true


##########################################################################
################## BELOW LINES SHOULD NOT BE MODIFIED ####################
##########################################################################
# Template file for "vcs -vccdc"
# Assume read_command variable is set

# RECORD:ON
eval $read_command
# RECORD:OFF

eval $sdc_command
configure_cdc_nff_sync -detect_full_chain true
check_cdc

# Report Generation
set log_path "."
if {[info exists ::env(SNPS_INTERNAL_PC_CDC_DIR)]} {
    set log_path $env(SNPS_INTERNAL_PC_CDC_DIR)
}

report_cdc -verbose -file $log_path/report_cdc.log
```

Lint Template File

You can customize the template, but do not modify the parts indicated.

Feedback

```
################### DEFAULT LINT TEMPLATE FILE USED FOR VC_STATIC FLOW IN PROTOCOMPILER FLOW ###################
set session_name vclint.daidir/run_vcsglint

# Setting app var to enable VCSGLINT-ZEBU flow
set_app_var enable_lint true
set_app_var enable_zlint true
set_app_var read_synopsys_sim_setup true

##########################################################################
################## BELOW LINES SHOULD NOT BE MODIFIED ####################
##########################################################################

# Template file for "vcs -vcstatic"
# Assume read_command, session_name variables are set

# RECORD:ON
eval $read_command
eval $sdc_command
# RECORD:OFF

check_lint

set log_path "."
if {[info exists ::env(SNPS_INTERNAL_PC_LINT_DIR)]} {
    set log_path $env(SNPS_INTERNAL_PC_LINT_DIR)
    report_lint -limit 0 -verbose -include_waived -gen_empty -file $log_path/report_lint.log
} else {

# Report Generation
report_lint -limit 0 -verbose -include_waived -gen_empty
}
checkpoint_session -session $session_name
# To view results, start vc_static_shell and execute:
# restart_session -session $session_name
# view_activity

quit
```

Current Limitations

The VC Static integration currently has some limitations:

- Data and control synchronizers are not supported.
- Specify FDC constraints through the SDC parameter in the configuration file.
- FDC prefix notation (*p:*, *c:*,*n:*) is not supported.

Compiling the Design Incrementally

In large and complex designs, minor design modifications can be expensive time-wise if the design cannot be recompiled incrementally.

With a UC incremental run, the tool skips partitioning and only reruns the changed FPGAs, thus saving turnaround time.

Note:

The UC flow does not have a system-level compiled database. The UC flow must work from the single-FPGA compiled databases.

See these topics for information about running incremental compile in the UC compiler flows:

- Incremental Unified Compile Flow: Design Database to SRS
- Incremental Unified Compile Flow: Bypass Flow
- Incremental Unified Compile Flow: Structural VM
- Incremental Unified Compile Flow: Bottom-Up Compilation
- Enhancing SRS Instrumentation for Partially Optimized Designs

Incremental Unified Compile Flow: Design Database to SRS

In this UC flow, only modules that are modified for the next run are compiled. Designs or modules that are not modified are reused.

Use the *-incremental* option with the `run compile` command to do incremental runs.

Use the same UCDB name for the first compile run and subsequent runs to ensure that only the changed modules are recompiled. Make a copy of the UCDB before rewriting it for debugging.

First Run

1. Run VCS.

   ```
   launch uc -utf <UTF_filepath> -ucdb <design_database_path>
   ```

2. Build the design

   ```
   run compile -ucdb <design_database_path> -out <compile_state_name>
   -incremental
   ```

   Example:

   ```
   launch uc -utf run.utf -ucdb ucdbrun compile -ucdb ucdb -out c0
   -incremental
   ```

Incremental Run

1. Build the design with changes in RTL.
2. Clone the compile state.

   ```
   database clone_state -state c0 -name cincr
   ```

3. Run the unified compiler with the same database name used during the first run.

   ```
   launch uc -utf run.utf -ucdb <database_name_in_first_run> -v 2.0
   ```

4. Compile only the modified modules using the *-incremental* option with `run compile` command.

   ```
   run compile -ucdb <design_database_path> -out <new_compile_state_name>
   -incremental
   ```

   Example:

   ```
   run compile -ucdb ucdb -out cincr -incremental
   ```

5. Check the compile_summary.rpt report to confirm the success of the incremental run. The report is available in the following path:

   <database name>_reports/ <new compile state name>/compile_summary.rpt

   Example:

   If the database name is *ucdb* and the new compile state name is *cincr*, then the report is ucdb_reports/cincr/compile_summary.rpt.

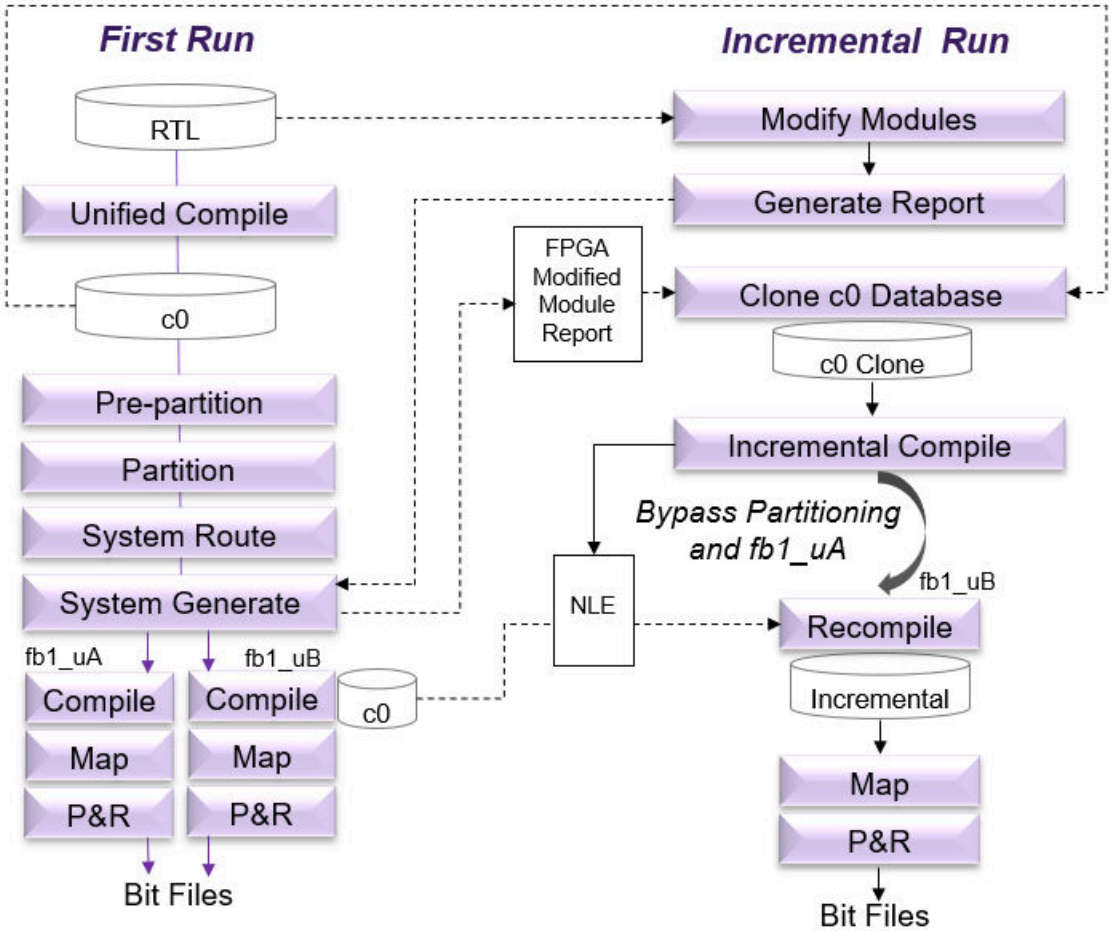
Incremental Unified Compile Flow: Bypass Flow

The bypass flow is used for minor RTL changes at the design level within an FPGA. It bypasses partitioning and routing, and only re-compiles the modified parts of the design in the affected FPGA. Updated views for the changed modules are linked to the rest of the design.

If changes are at the FPGA level, use the procedure: Incremental Unified Compile Flow: Structural VM.

The following figure summarizes the bypass flow, showing both the initial and incremental runs. The black arrows connect the steps in the incremental flow.

Feedback



You can use one of these flows:

- Automated Bypass Flow (UC)
- Manual Bypass Flow (UC)

Limitations

- Linking fails in the following scenarios:
  - RTL changes that modify ports of modules
  - XMR changes that impact port changes
  - Instrumentation changes that cause port changes
  - Modules partitioned across FPGAs
- Module names with special characters cannot be selected for the bypass flow. Use the parent module while linking.

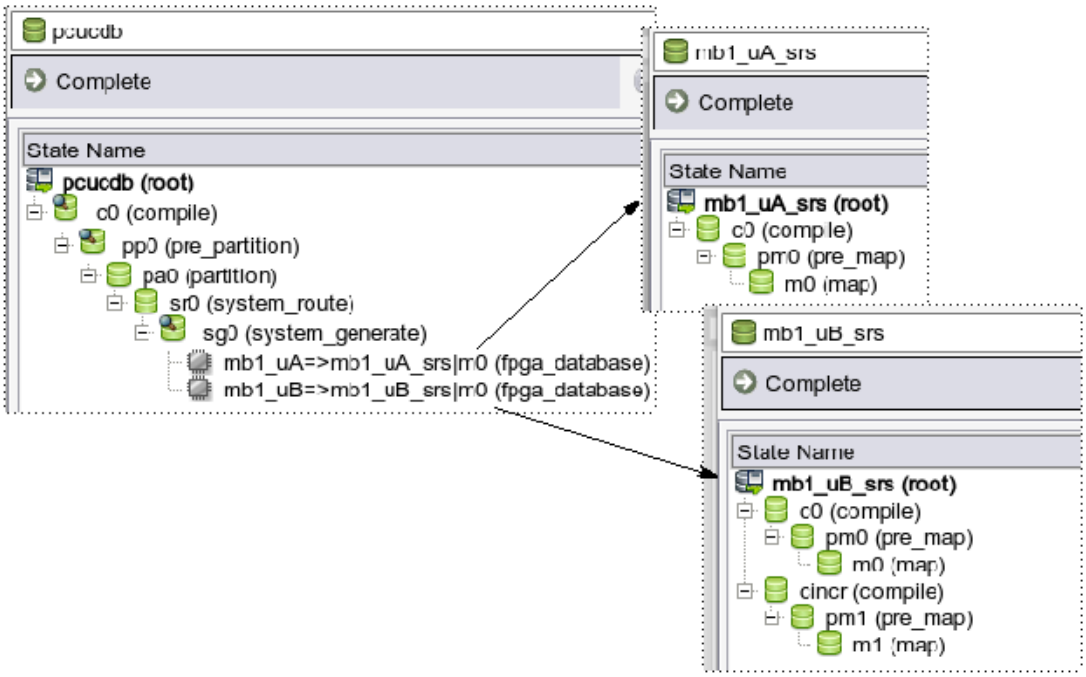
Feedback

- Modules impacted by partitioner are marked as *Modified*. These modules cannot be edited.
- If modifications affect critical paths, you must manually update the appropriate FDC constraints.

Automated Bypass Flow (UC)

In this UC flow, you need not find the module to which it is partitioned, or run individual FPGA scripts manually. The tool gets all necessary NLEs and constraints applied to the design.

1. Make incremental changes in a design, through the place-and-route stage.
   - The top-level design would have gone through all the design states through system generate, and the individual FPGAs would have database nodes for compile, pre-map, and map. The next figure shows parent-level and FPGA-level database states for a design with two FPGAs.



   - Make minor module-level design changes. The changes must be within a single FPGA.
2. Check the modules to be bypassed on the incremental run. Only modules not modified by the partitioner can be bypassed.

Note:

Modules impacted by global optimization operations such as constant propagation or sequential optimizations are listed in the report, but these modules cannot be edited with the bypass flow. For these cases, use the structural VM flow instead. You can generate a report that lists the modified modules with the `export file` command.

- To export a list of modules, run `export file` for each FPGA from the top-level *system_generate* state, and export a report that lists them:

```
export file Modified_Modules_mb1_uA.rpt
export file Modified_Modules_mb1_uB.rpt
```

- To see a list of all modified files available, run this command from the *system_generate* state:

```
export file -list -all
```

3. Clone the compile state to create a duplicate state for incremental changes.

   The main reason for cloning the state is control, so that you can run multiple modification iterations or revert to the pre-modified state if needed.

```
database load pcucdb
database set_state c0
database clone_state -name c0_clone
```

4. Launch incremental compilation and specify the original and clone states for *run compile*.

   - Start from the *root* database of the previous run, and launch UC.
   - Use the `run compile` command, as usual, but make sure to specify the clone compile state with *-out*. If there are multiple iterations with multiple clone states, make sure to point to the correct one.

```
database load pcucdb
database set_state root
launch uc -utf run.utf -ucdb ucdbincr
run compile -ucdb ucdb -gen_update_nle 1 -out c0_clone
```

   The tool runs incrementally and recompiles the modified modules. The `run compile` command creates *nle* files with replacement commands for the module views with changes.

5. Export the modules with changes.

- Use the following command to export the modules:

  ```
  export file update_modified_views.nle
  ```

  This is an example of the NLE file. The NLE `replace_view` command replaces the whole sub-hierarchy it points to. There is a checksum for modules in *vcs.xcui*.

  ```
  ## Generated by Synopsys Netlist Linker
  ## version comp202003pcp2, Build 015R on Jun  5 2020 09:17:51
  ## NLE Script to edit modified views for SRS Bypass Flow
  set srs_handle [load_design \
   "myDesign/pcucdb/compile/c0_clone/synwork/top_comp.srs"]
  replace_view  {top_clk_inst.U3} $srs_handle:seqlog3
  ```

- Manually edit the *nle* file if needed.
- In the NLE, comment out any replacement views that are not part of that FPGA; if not, linking (next step) fails.
- If you modify multiple modules that are targeted to different FPGAs, edit the NLE script manually to point the correct compiled (*srs*) handle for that FPGA.

6. Select system generate state

   ```
   database set_state {sg0}
   ```

7. Check which post-partition FPGA is affected and export the generated bypass log.

   ```
   report srs_bypass -compile_state c0_clone
   export srsbypass.log
   ```

   The report srs_bypass command generates a report with details of the modules changed, post-partition FPGAs affect, and whether the incremental SRS bypass flow can be used. The *srsbypass.log* contains details of each module that has been changed.

8. Apply the changes to a single FPGA node.

   ```
   export srs_bypass -compile_state c0_clone -path <path> -use_mapper
   ```

   The *-use_mapper* option finds the parent view if the child is in a non-incremental view. The tool traverses through the hierarchy till it finds a view which can be replaced.

   The `export srs_bypass` command exports the following files:

   - *sg0+cincr_setup.tcl*—Contains FPGA name that needs to be re-run after the update.
   - *sg0+cincr_top.tcl*—Calls individual FPGA scripts that are in the *pcucdb_synthesis_files/mb1_u** folder.

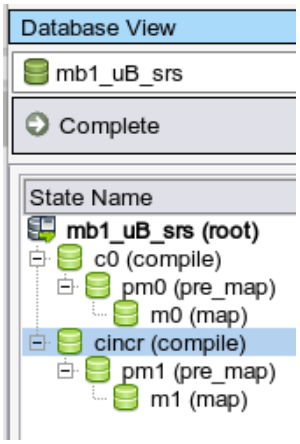
- *cincr_update_modified_views.nle*—Contains the replace view commands of updated module needed for stitching original SRS. This file saved in the *pcucdb_synthesis_files/mb1_u** folder. Note that if you modify across multiple FPGAs, multiple *cincr_update_modified_views.nle* files are generated. Each *cincr_update_modified_views.nle* file is saved in the respective modified FPGA directory.
- *cincr_mb1_uC_srs.tcl*—Calls FPGA script *mb1_uC_srs.tcl* to launch from beginning. Note that if you modify across multiple FPGAs, multiple *cincr<FPGA>_srs.tcl* files are generated. Each *cincr<FPGA>_srs.tcl* file is saved in the respective modified FPGA directory.
- *report_srs_bypass.log*—Log file stating whether the SRS bypass flow can be used. Lists the modules that are changed and provides details on whether a module can be linked or not. The parent view names are also listed in this log.

9. Link the updated compiled SRS to individual FPGAs.

```
cd <path>
source sg0+c0_clone_top.tcl
```

10. Runs the whole FPGA with all constraints applied on compile, pre_map, and map states. Any failure in linking can be debugged by checking the message in *netlist_optimizer.log*.

    The following figure shows the results of successful linking:

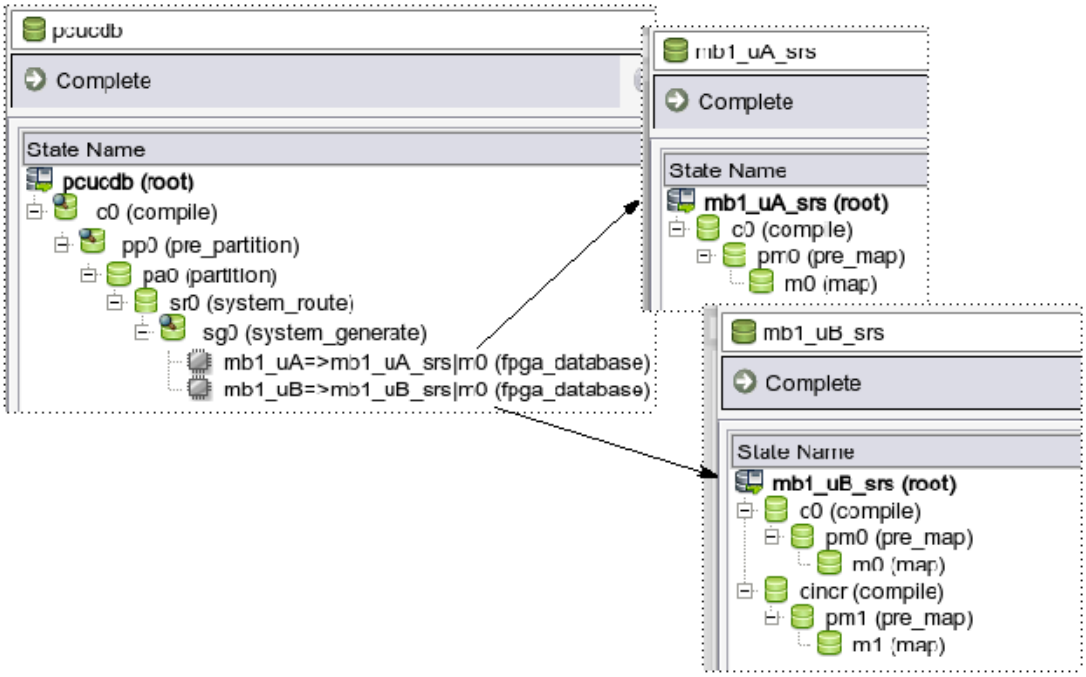


    Complete the incremental design by running the FPGA through the remaining design stages from map state to bit generation.

Feedback

Manual Bypass Flow (UC)

1. Make incremental changes in a design that has been run through the flow, through the place-and-route stage.
   - The top-level design would have gone through all the design states through system generate, and the individual FPGAs would have database nodes for compile, pre-map, and map. The next figure shows parent-level and FPGA-level database states for a design with two FPGAs.



   - Make minor module-level design changes. The changes must be within a single FPGA.

2. Check the modules to be bypassed on the incremental run.

   The bypass methodology can only be applied to modules that were not modified by the partitioner. Note that modules impacted by global optimization operations like constant propagation or sequential optimizations are listed in the report, but these modules cannot be edited with the bypass flow. For these cases, use the structural VM flow instead. You can generate a report that lists the modified modules with the `export file` command.

- To export a list of modules, run `export file` for each FPGA from the top-level *system_generate* state, and export a report that lists them:

```
export file Modified_Modules_mb1_uA.rpt
export file Modified_Modules_mb1_uB.rpt
```

- To see a list of all modified files available, run this command from the *system_generate* state:

```
export file -list -all
```

3. Clone the compile state to create a duplicate state for incremental changes.

   The main reason for cloning the state is control, so that you can run multiple modification iterations or revert to the pre-modified state if needed.

```
database load pcucdb
database set_state c0
database clone_state -name c0_clone
```

4. Launch incremental compilation and specify the original and clone states for `run compile`.

   - Start from the *root* database of the previous run, and launch UC.
   - Use the *run compile* command, as usual, but make sure to specify the clone compile state with *-out*. If there are multiple iterations with multiple clone states, make sure to point to the correct one.

```
database load pcucdb
database set_state root
launch uc -utf run.utf -ucdb ucdbincr
run compile -ucdb ucdb -gen_update_nle 1 -out c0_clone
```

     The tool runs incrementally and recompiles the modified modules. The `run compile` command creates *nle* files with replacement commands for the module views with changes.

5. Export the modules with changes.

   - Use the following command to export the modules:

```
export file update_modified_views.nle
```

     This is an example of the NLE file. The NLE `replace_view` command replaces the whole sub-hierarchy it points to. There is a checksum for modules in *vcs.xcui*.

```
## Generated by Synopsys Netlist Linker
## version comp202003pcp2, Build 015R on Jun  5 2020 09:17:51
## NLE Script to edit modified views for SRS Bypass Flow
set srs_handle [load_design \
```

Feedback

```
 "myDesign/pcucdb/compile/c0_clone/synwork/top_comp.srs"]
replace_view  {top_clk_inst.U3} $srs_handle:seqlog3
```

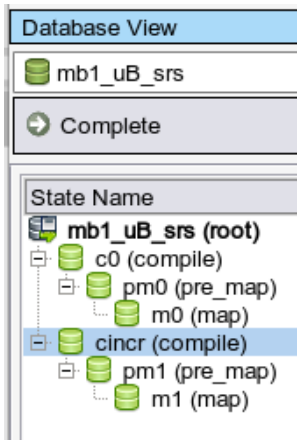
- Manually edit the *nle* file if needed.
- In the NLE, comment out any replacement views that are not part of that FPGA; if not, linking (next step) fails.
- If you modify multiple modules that are targeted to different FPGAs, edit the NLE script manually to point the correct compiled (*srs*) handle for that FPGA.

6. Link the modules with changes to the original compiled database by specifying the *nle* file:
   - For each modified FPGA, recompile with `run compile` and specify the *nle* file generated in step 5:

     ```
     database load pcucdb_synthesis_files/mb1_uB/mb1_uB_srs
     database set_state c0
     run compile -srcstate c0 -nle update_modified_views.nle -out cincr
     ```

   - You cannot recompile modules that have escaped identifiers as names . For these modules, use the module parent while linking.
   - If there are link failures, check messages in the *netlist_optimizer.log* file to debug it. Linking might fail because of these reasons: RTL changes that modify the ports of modules, XMR changes that cause port changes, changes to instrumentation that cause port changes, or modified modules partitioned across FPGAs.

   The following figure shows the results of successful linking:



7. Complete the incremental design by running the FPGA through the remaining design stages from pre-map to bit generation.

Feedback

SRS Bypass Flow with Incremental RTL Instrumentation

You can incrementally add and remove dumpvars signals for RTL instrumentation using SRS bypass flow. Perform the following steps:

- Make incremental dumpvars changes in the corresponding module.
- Follow the Manual Bypass Flow (UC) steps for incremental compilation until NLE generation and edit if needed.
- Link the modules with changes to the original compiled database by specifying the nle file. For each modified FPGA, recompile with run compile, specify the nle file generated, and specify the incr_srcstate which is incremental compilation state.

```
run compile -srcstate pcucdb_synthesis_files/mb1_uA/mb1_uA_srs|c0
 -nle update_modified_views.nle -top_module TraceBuildLib.mb1_uA
 -incr_srcstate db0|c0_incr -out cincr
```

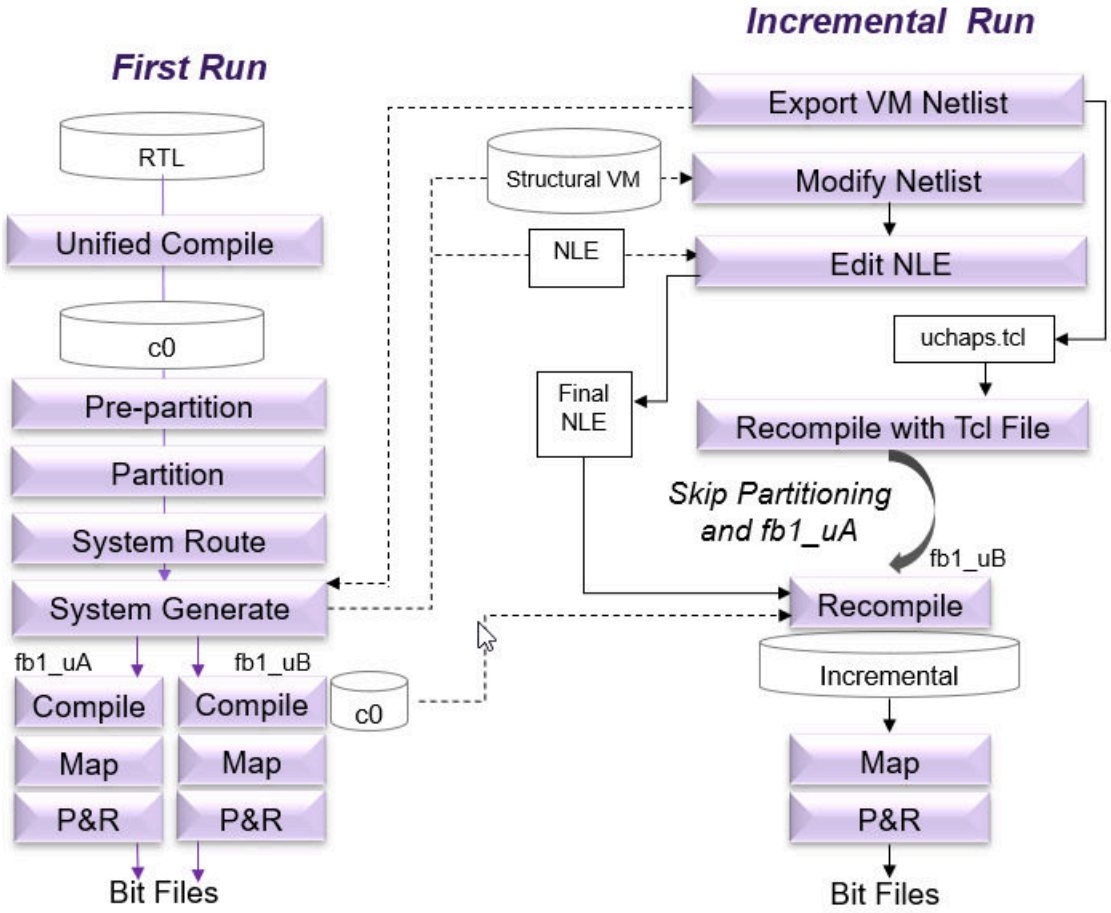
- Specify the idc file with mode dumpvars and define required IICE settings with sample clock during pre-map.

```
iice new {IICE_0} -type regular -mode {dumpvars}
```

- Complete the incremental design by running the FPGA through the remaining design stages from map to bit generation.

Feedback

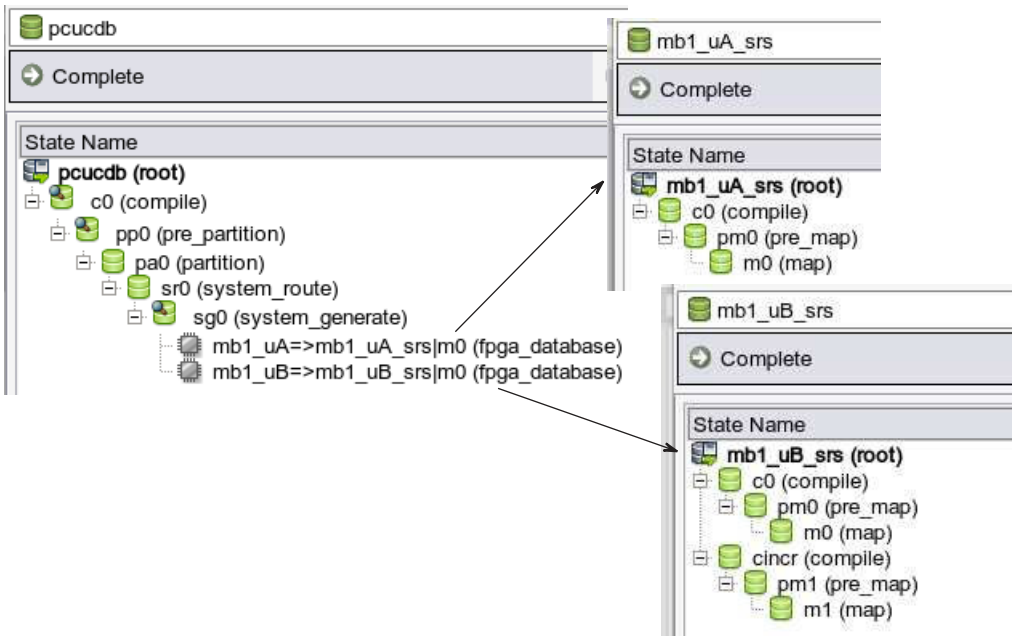
Incremental Unified Compile Flow: Structural VM



The structural VM flow is an incremental run methodology, for when changes are made at the FPGA level. A structural netlist is module-based instead of behavior-based. As the figure shows, the flow starts with the export of a structural netlist (*.vm*) from the System Generate stage. Changes are made in the structural netlist and linked back to the top-level design. This allows you to skip the partitioning stages at the top level and rerun only the FPGA with the changes. Use this method for changes within an FPGA, or when the bypass method (Incremental Unified Compile Flow: Bypass Flow) cannot be used.

Feedback

1. Make changes to a design that has run through the flow, through place and route.
   - In a partitioned design, the top-level design would have gone through all the design states through system generate, and the individual FPGAs would have database nodes for compile, pre-map, and map. The figure shows parent-level and FPGA-level database states for a design with two FPGAs.



2. Export the structural VM netlist from the System Generate state, using the `export netlist` command:
   - To export a complete FPGA, use this command:

     ```
     export netlist -path <OUT_DIR> -format structver
     ```

   - To export selected modules, use this syntax:

     ```
     export netlist -path <OUT_DIR> -format structver
     -modlist <FILE_WITH_LIST_OF MODULES>
     ```

     The modules can be specified in the file in these ways:

     ```
     i:<instanceName>
     v:<cellName>
     lib.cell.view.
     ```

     Note that if you specify multiple modules, the VCS script is only created for the first module in the list. You must modify the script to compile multiple modules.

The `export` command creates the UC scripts required to compile the structural VM netlist. It generates the following files:

nle_script	NLE file with *replace view* commands for exported module instances
struct_ver	Structural VM netlist
system_ip	System IP files
uchaps.utf	UTF commands
uchaps.tcl	UC compilation commands
update_modified_views.nle	NLE script for entire FPGA
vcs_cmd.csh	VCS commands for UC compilation

3. To make incremental changes to modules that are modified by the partitioner follow these steps:

   This flow does not support instrumentation that is added outside the DUT (design under test) instance in the structural netlist.

   - Make minor changes to the VM netlists for the modules.
   - Add the *-structver* option in the NLE file to force the linking of these modules. Use *-structver* the tool to accept netlist changes necessitated by modifications by the partitioner, like net name changes, logic optimizations, distribution across FPGAs etc.

     ```
     replace_view -structver {u1} $srs_handle:module_pplan
     ```

4. Recompile the structural netlist by sourcing the *uchaps.tcl* file created by the `export` command.
5. Go to the FPGA with the changes and do the following:
   - Recompile the FPGA and use the NLE file to link it to the old compiled database:

     ```
     run compile -srcstate c0 -nle<NLE_file> -out cincr
     ```

   - Continue with the rest of the flow for the FPGA as usual, through bit file generation.

Feedback

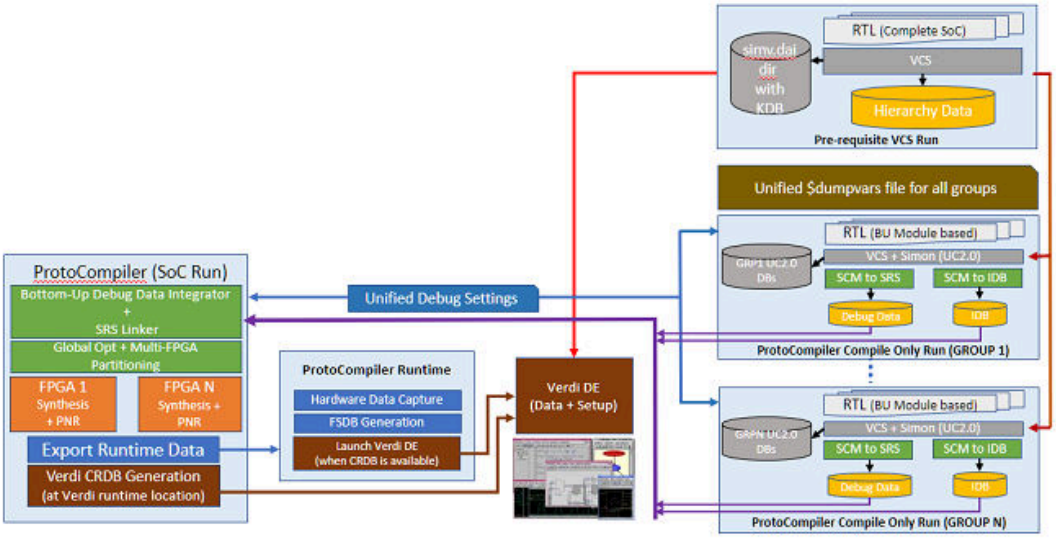
Limitations

- When exporting multiple modules, the VCS script is created for the first module as the top module. Modify this script if you have to compile different modules.
- Modules that have escaped identifiers as names cannot be selected by the compiler for the structural VM flow.
- Debug logic that is outside the DUT might not be instrumented.

Incremental Unified Compile Flow: Bottom-Up Compilation

Bottom-up compilation manages complex designs by distributing work between teams, to incorporate IP, or to isolate sections of designs.

Run bottom-up compile by first compiling individual blocks and then stitching them together and running compile at the top level. You can run bottom-up compilation on a single database where you stitch together multiple compile database states, or stitch together compiled database states from different databases.



The SRS bypass flow:

- Supports only the compilation groups run with Unified Compile. You cannot use the compilation groups run with standard compiler partition flow.
- Does not support for RTL instrumentation changes.

Prerequisites

- Preparing the design
  - The entire design is run using the VCS flow without any black boxes meant for compilation groups.
  - Generated Verdi KDB and hierarchy database remains available.
  - The database is regenerated for all RTL changes.
  - Design hierarchy is exactly the same as the one created after running and merging all compilation groups.
- Running compilation groups
  - RTL partitioning is done with appropriate black boxing.
  - Matching sub-tree hierarchies are available within each group with respect to the entire design.
  - Each compilation group is run at separate location with same debug options.
  - Hierarchy information is generated from VCS prerequisite run.
  - RTL instrumentation is applied and recommended only through $dumpvars, or SVA.
  - Top module compilation group requires IDC specifying all the debug IP configurations, clock specification. The IDC instrumentation and clock specification should only be within the top scope.
  - Debug visibility reports can be used within each compilation group.
- Runtime flow:
  - All compilation groups are provided for integration with appropriate top specification.
  - SRS-based IDC specification are applied at pre-partition after the compilation groups are integrated. The IDC specification can also be given during FPGA synthesis.
  - All the existing debug features are available after the compilation groups are integrated.
  - Verdi KDB generated from VCS prerequisite run is used during the runtime for debug session.

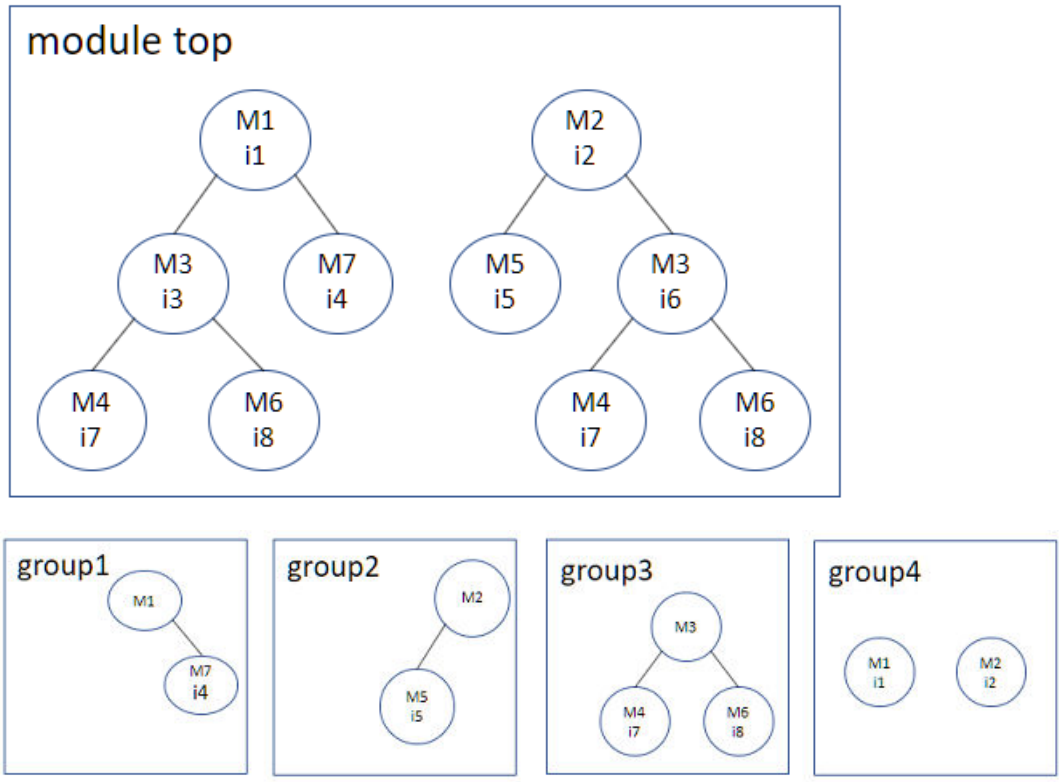
Feedback

To run the flow, see the following sections:

1. Preparing the Design Using VCS Software
2. Running Compile on Compilation Groups
3. Debugging with the Verdi Software

About Compiler Groups

Using compiler groups you can define modules or part of a design hierarchy that needs to be run through unified compile (*-compile_only*) flow independently.



Unified $Dumpvars Specification for the Flow

- VCS pre-requisite run for complete SoC
- Appropriate partitioning of the SoC
- Each Compilation group hierarchy is a sub-tree of SOC design hierarchy
- One specification is applicable across all groups
- Limit specification is not available

- Appropriate instrumentation is created based on path specifications
- Supports wild cards, dynamic force, and monitor specification

Example $Dumpvars specification for the compile group based on the previous diagram:

```
$dumpvars(1, top.i1.i4)
$dumpvars(1, top.i2.i5)
$dumpvars(1, top.i1.i3.i7)
$dumpvars(1, top.i1.i3.i8)
$dumpvars(1, top.sig*)
```

Unified SVA Specification for bottom-up flow

- SVAs can be written within any module
- Appropriate instrumentation is created for all module instantiations containing SVA

Preparing the Design Using VCS Software

1. Set the option *-hierarchy_info=<path>* to generate the hierarchy information for each module.

   ```
   export HIERARCHY_INFO=/bottom_up/pre_requisite_run
   ```

   or

   Provide the absolute path to the *hierarchy_info* argument directly.

   ```
   /bottom_up/pre_requisite_run
   ```

2. Set the option *-kdb* for each VCS command to generate RTL Knowledge Database (KDB) to be used for Verdi debug. Note that for each RTL change, for any module, you must do a rerun to regenerate the RTL KDB.

   ```
   /bottom_up/pre_requisite_run/run_vcs?
   vlogan -full64 -work -sverilog -f files.sfl -kdb
   vcs -full64 -top <name> -kdb
   -hierarchy_info=$HIERARCHY_INFO
   ```

Running Compile on Compilation Groups

Before your run the compile, make sure that same options are set on all compiler groups.

1. Configure *option set use_module_idb 1* on all compilation groups.
2. Make sure each group run maintains its own *simv.daidir*, and *ucdb* directories.
3. Use option *-compile_only 1* for each group run.
4. Specify the IDC file with all the required IICE settings and clock definitions. The IDC should only be given to the SoC group representing the top module.

Feedback

```
set dirLoc "/bottom_up/GRP_N"
database load db_grp_N -autocreate -technology HAPS-100
source /bottom_up/options.tcl
launch uc -utf ${dirLoc}/uc_data/run.utf -ucdb -v 2.0
# Specify IDC only for group representing "top" module
# Used for specifying debug IP settings
run compile -ucdb ${dirLoc}/ucdb_grp_N -out grp_N_c0 \
                      -top_module grp_N \
                      -compile_only 1
                      -idc <path>
```

5. Run the main link step using the `run compile` command.
6. Import all group runs using *-srcstate* option.
7. Specify the top module details using the `-top_module` command.

```
run compile -srcstate {/bottom_up/GRP_1/db_grp_1|grp_1_c0
 /bottom_up/GRP_N/db_grp_N|grp_n_c0} \
                      -out soc_top_c0 \
                      -top_module top
```

Debugging with the Verdi Software

Before you start, specify the RTL KDB directory pointing to the *simv.daidir* under the prerequisite run location. Make sure that RTL KDB is generated correctly for the entire design.

1. Run the prototyping and debug flow.
2. Run synthesis.
3. Export data for Runtime, hardware capture, FSDB generation and Verdi debug.

```
export runtime -path ${dirLoc}/runtime_soc_db
```

4. Generate Verdi correlation database (CRDB).

```
launch verdi -run_dir ${dirLoc}/runtime_soc_db/system/verdi -crdbgen
-fast_crdbgen -fsdbtype edif
```

Enhancing SRS Instrumentation for Partially Optimized Designs

HAPS-100

The SRS instrumentation flow enables users to monitor and debug specific signals within a design. During compilation, the synthesis tools optimize unused or redundant bits in bus signals, making some bits unavailable for instrumentation.

When the `report debug visibility` command is applied on system-level pre-map or post-SLP (system-level partition) pre-map netlists, it checks for the availability of

specific signals within the netlist. If any bits of the requested signal are not found or are unavailable, the entire signal is reported as *not found*. You can instrument the available bits or ranges within a partially optimized bus, thus improving the debug process.

In your design, identify the bus signals you want to instrument and request instrumentation. The tool detects the partially optimized buses.

For example, in a bus with [0:63] bits, the bits 3,17, and 35 are optimized. Then, you can de-instrument the existing bus and instrument the existing ranges. This design example consists of 4 signals with ranges [0:2], [4:16], [18:34], and [36:63].

If some bits in the requested bus are missing due to optimization, ignore the unavailable bits and instrument the available ranges as separate signals.

A message displays indicating which bits are not found and which ranges are instrumented.

Example message:

```
DI497 Cannot find net 'shift_reg_struct\.sh_r2[2:0]' for instrumented
 object 'SRS.shift_reg_struct\.sh_r2[2:0]' because some bits are missing
 in the provided bus signal. Instrumenting the following available
 signals instead:
SRS.shift_reg_struct\.sh_r2[0]
SRS.shift_reg_struct\.sh_r2[2]
```

Specifying Constraints

Constraints are optional. You can add constraints to refine your prototype.

Some constraints only work for particular database states. The following table lists the constraints and the design stages where they apply:

Constraint	Valid State	Description
CDC	Compile	Compiler constraints file
FDC	Pre-map, map	FPGA design constraints file
UPF	Pre-map, pre-partition	Power constraints file
IDC	Compile, pre-instrument, pre-map, pre-partition	Instrumentor constraints file

See:

- Adding Constraints to a Design Database
- Configuring Directives in a CDC File

Adding Constraints to a Design Database

Use this procedure to add constraints to a design database.

1. Define the constraints in a constraints file.

   The tool reads some file formats directly, such as *upf*. You can convert constraints which cannot be read by the tool. For example, you can save standard Synopsys SDC timing constraints defined for your ASIC design in an FDC file, which uses the same syntax. See Converting Constraints.

2. Add the constraints file to the design by using the appropriate argument to the `run` command at the appropriate database state.

   For example, `run map -fdc <file>` specifies design constraints to be used when mapping the design. Check the command syntax for speciifc `run` command for details.

   Some constraints can only be added at certain design states. The following table lists some constraints you can enter at different stages of the design:

Command	Constraints File
run compile	◦ cdc (Compiler directives) Configuring Directives in a CDC File ◦ idc (Instrumentation) Instrumenting the Design for Debug
run pre_map run map	◦ fdc (Synthesis design constraints) ◦ upf (Low power constraints) Including Unified Power Format (UPF) Specifications ◦ idc (Instrumentation)
launch vivado	par (Place-and-route constraints)

Configuring Directives in a CDC File

Use this procedure to create a CDC file and configure directives, without making changes to your HDL files.

Feedback

1. Create a CDC constraints file that contains the Tcl directives.

   You can specify the compiler directives on views, entities, architectures and modules. See Controlling the Compiler Run with Options and Constraints for a table that includes *cdc* directives. You can also refer to the description of a directive; if there is a *cdc* example, it is supported.

2. Use the following syntax for the directives you want. Use the syntax that matches the HDL source code.

VHDL	`define_directive {v:[libraryName.]entityName[(achitectureName)]} {directive} {valu`
Verilog	`define_directive {v:[libraryName.]moduleName} {directive} {value}`

   The following example sets the *syn_black_box* attribute on all architectures of the *sub* entity in the *MyLib* library:

```
define_directive {v:MyLib.sub} {syn_black_box} {1}
```

   Specify the attribute or directive on a view (`v:`). The *libraryName* and *architectureName* arguments are optional. If you do not specify a library, the tool defaults to all design libraries. If you include an architecture, ensure to enclose it in parentheses. Note that Verilog objects are case-sensitive, but VHDL objects are not.

   If you apply directives on a net, ensure to include the pipe (|) character, as shown in the example:

```
define_directive {v:work.sub} syn_rename_module {sub_new}
define_directive {n:work.sub_new|temp1} {syn_keep} {1}
define_directive {n:work.test|a} {syn_direct_reset} {1}
define_directive {n:work.ones_cnt|vout_reg} {syn_allow_retiming} {1}
define_directive {n:work.test|dout_t} {syn_dspstyle} {dsp}
define_directive {n:work.fsm|state} {syn_encoding} {one_hot}
define_directive {n:work.test|a} {syn_direct_set} {1}
define_directive {n:work.mult|mult_i} {syn_multstyle} {logic}
define_directive {n:work.pipeline|temp2} {syn_pipeline} {1}
define_directive {n:work.RAMB4_S4|mem} {syn_ramstyle} {select_ram}
define_directive {n:work.norep|DriveA} {syn_replicate} {true}
define_directive {n:work.d_p|h_data_pip_i} {syn_srlstyle}
 {extractff_srl}
```

3. Add the directives file when you compile the design, using the *-cdc* or *-cdclist* options to the command.

   When the design is compiled, the tool passes all active *cdc* files to the compiler. The compiler references the object names in these files with the original RTL objects and assigns the corresponding directives.

4. Check for constraint errors:
   - Execute the `report constrant_check` command to generate the following reports:
     - The *constraint_check.rpt* file lists the constraints that were applied, not applied or ignored.
     - The *syntax_constraint_check.rpt* file reports syntax errors with the constraints. To run the syntax check, execute `report constrant_check -syntax_only`.
   - Analyze the design and adjust constraints. It is recommended that you use simple constraints for initial runs.

Configuring Options

The following table provides different ways to set options for `run` commands at different points in the design cycle. Some options only affect a certain database state or `run` operation, while others affect multiple states.

Method	Description
option set commands in a Tcl file	A Tcl file containing multiple options, which can be reused in multiple sessions.
option set command typed in the tool console	Does not persist beyond the current session. Not ideal for multiple option settings.
Options Editor from the GUI ()	Easy way to create an options file, check current option values, edit them, and view differences from previous run.
Design Intent button in the GUI *design_intent* command	Preset options for either fast turnaround or performance. Preset options can also be customized. See Synthesizing Based on Design Intent for details.

For more information, see:

- Configuring a Tcl Options File
- Configuring Options in Options Editor

Feedback

Configuring a Tcl Options File

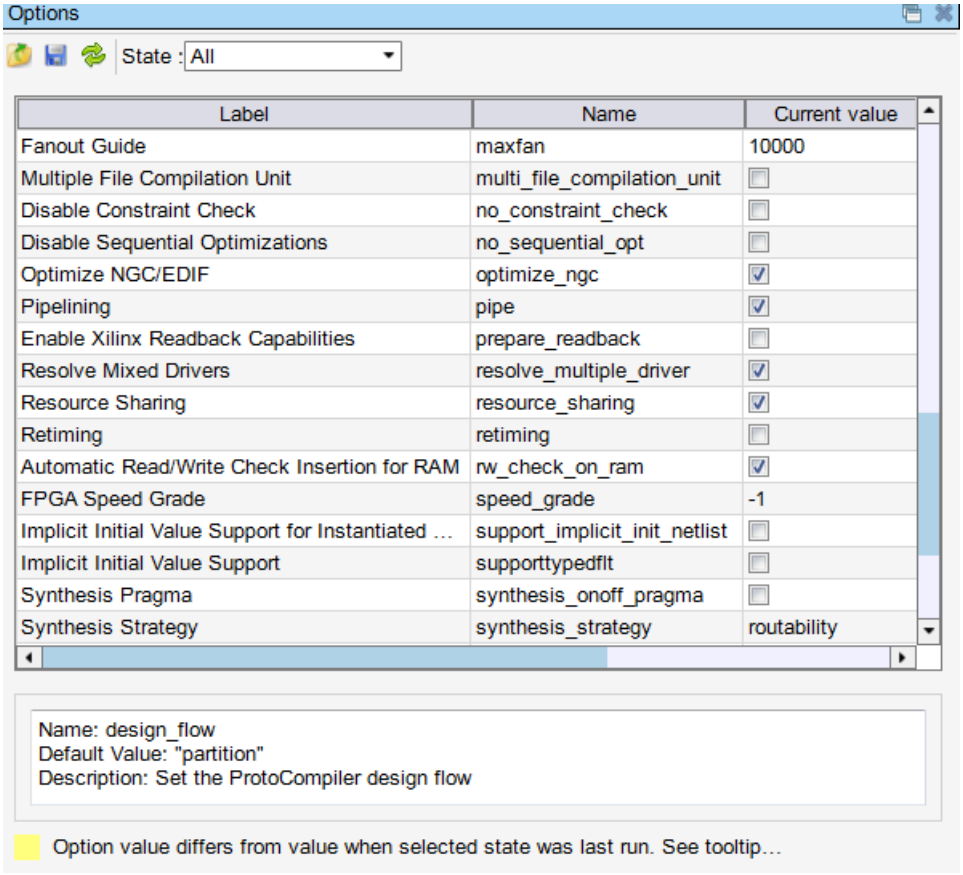
1. Create a Tcl options file, using one of these methods:
   - Start with a template file of options and edit it to suit your needs. The tool installation includes template files in the specified directory. To open one of these files and use it as a starting point, specify it with the `design_intent` command:

     ```
     design_intent -custom installDir/lib/design_intent/protocompiler/
     timing_qor.tcl
     ```

     If you do not specify a file with this command, the tool opens the *fast_turnaround.tcl* file by default. To see a list of available Tcl options files, use the `design_intent -list` command.
   - Type the options manually into a Tcl file, using this syntax for the `option set` command:

     ```
     option set optionName optionValue
     ```

     For example: `option set synthesis_strategy fast`. To see the available options, enter the `option list` command in the Tcl window, or see List of Options of the *Command Reference Manual*.
   - Click the **Options Editor** icon in the GUI (). The default view shows the options for the current database state. You can set option values and then click the **Save** icon at the top of the tab to save a file based on the current database options. Alternatively, you can first click **Show All Options** at the bottom right before setting the options and saving the file.



Create a single file for all the options, or create multiple option files to be used with different database states. For example, you might set these options specifically for the map stage:

```
option set enable_prepacking 1

option set synthesis_strategy routability

option set fanout 10000

option set retiming 1
```

2. Source the options file before issuing a `run` command:

   `run compile`, `run pre_partition`, `run partition`, `run system_route`, `run system_generate`, `run pre_map`, `run map`.

   ```
   source ./myCompileOptions.tcl
   ```

   Options are not written to the database and must be sourced to take effect. If you are using multiple options files, make sure to source the appropriate options file to set the

options. If you reopen the GUI, make sure to source the options again, as they are not stored.

To view option values, use the commands described in Configuring Options in Options Editor.

3. To set options automatically on startup, do the following:
   - Set the options you want in a Tcl file called .synopsys_pc.setup.
   - Save the file in one of the locations listed in the following table. The tool automatically reads options set in this file at startup. It reads the files in the order specified in the previous table. After reading the file, the tool uses the specified option setting as the default.

Setup File Location	Directory Description
installDir/.synopsys_pc.setup	Installation. Sets the option or default in the install.
~/.synopsys_pc.setup	Sets the option or default for your personal setup.
./.synopsys_pc.setup	Sets the option or default for the working directory, which is the directory from where you launched the tool.

4. To automatically use different options files for different experimental runs, follow these recommendations:
   - Create separate *.synopsys_pc.setup* files with the different option settings for each of the databases, and save them to the different database directories.
   - For the different runs, launch the tool from the locations of the *.synopsys_pc.setup* files. This ensures that the options are set automatically for that run, and no files need be sourced.

Configuring Options in Options Editor

The tool provides ways to view and edit options from the command line or from the GUI.

Feedback

1. To view the value of an individual option, use the **Options Editor** as described in the next step, or use these commands:

To View ...	Command
Current value of an option	◦ `option get <optionName>` ◦ `option list` ◦ Click the **Options Editor** icon in the GUI
Current default value for the option	`option get_default <optionName>` (No return value indicates the default)
Value used to run the database state	database `query_state -option <optionName>` either on its own, or a script. See Querying Incremental Results.

   Note that the option value for the loaded database might differ from the current value. The two commands display the current value to be used going forward for the next `run` command, and the value that was used to generate the current database. For syntax details, see the *Command Reference*.

2. To view all the option values for a particular database state, select the state and then open **Options Editor** by clicking the icon in the GUI ( ).

   The editor shows the relevant options for that database state and their current values. If the value of an option has changed since the previous run, it is highlighted.

3. To view all the current option values, open **Options Editor** and click **Show All Options** in the editor.

   The editor shows all the options and their current values.

4. To reset a current option value to the value it had when you last ran the database state, create a script like the following example, and run it.

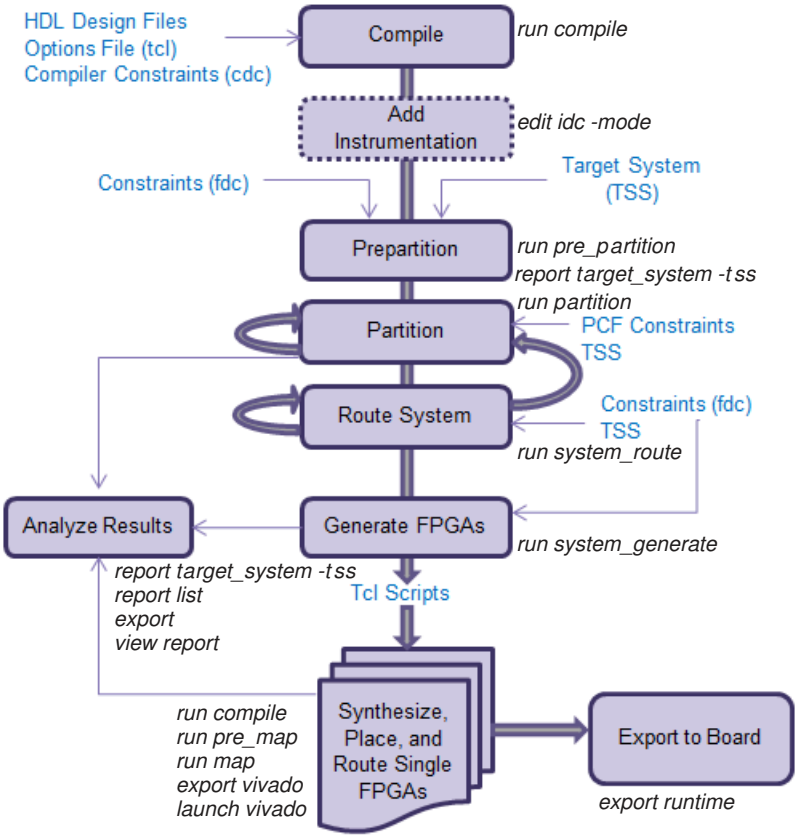
```
proc restore_state_options {} {
   foreach o [option list] {
       set cur [option get $o]
       if {[ catch {set prev [database query_state -option $o]} ]}
continue
 if { "$cur" == "$prev" } continue
       puts "Changing option $o to $prev"        option set $o $prev
   }
}
```

5

Partitioning the Design

Partitioning enables you to partition a large design into multiple FPGAs. This is the basis of multi-FPGA design.

Figure 7 Partitioning Flow



The figure shows the complete partitioning design flow and the main commands associated with each state. Partitioning itself consists of four states, each with an associated `run` command: pre-partition, partition, route system, and generate FPGAs. Although the diagram shows iterations at the partitioning stages only, you can re-iterate and rerun any stage in the design because of the flexible database model.

For more information about the steps shown in the preceding figure, see the following table:

Table 1 Partitioning Flow

Overview of Partitioning Flow	Design Methodology for Partitioning
Compile	Configuring a Design Database Adding Design Files Compiling the Design
Add Instrumentation	Instrumenting the Design for Debug
Pre-partition	Configuring Files for Partitioning Using run pre_partition to Define Partitions
Partition	Partitioning the Logic
Route System	Running System Route for the Top Level
Generate FPGAs	Generating FPGA Partitions
Synthesize, Place and Route Individual FPGAs	Implementing Individual FPGAs Running the Implementation Flow
Export to Hardware	Preparing to Run on the Hardware
Analyze Results	Analyzing Results

This chapter describes partitioning methodology and various commands to partition your design.

- Design Methodology for Partitioning
- Configuring Files for Partitioning
- Defining the System in a TSS File
- Configuring Constraints in PCF Files
- Working with HAPS Systems
- Using run pre_partition to Define Partitions
- Partitioning the Logic
- Analyzing Partition Result Files
- Using Time Domain Multiplexing (TDM)

- Using Hierarchical Partitioning
- Running System Route for the Top Level
- Clock Scaling Support
- Reducing SLR Congestion
- Generating FPGA Partitions
- Implementing Individual FPGAs
- Using Multi-Design Mode (MDM)
- Modular Flow
- Guided Partition Flow
- Overview of Selective Rebuild
- Debug Flow
- Sync GSV in Modular Flow
- Modular Flow Usage Recommendations

Design Methodology for Partitioning

Partitioning logic is an iterative process, where you refine results until you get what you want. Your goal is to find an optimal assignment of cells and ports, so that the connections effectively use the available routing resources.

Typically, this task is time-consuming and difficult, because you have to balance two inter-dependent considerations that influence and affect each other: the hardware topology and the design characteristics. The hardware configuration determines the functioning of the prototype, but design characteristics determine the best hardware configuration. The automatic partitioner (APTN) helps automate this task for faster iterations.

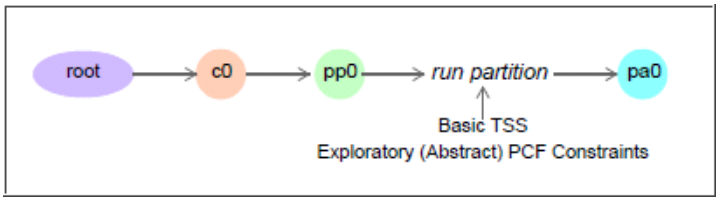
The partitioning approach described here provides a fast and flexible solution that allows for a high level of automation. The APTN flow consists of two phases: in the first, you quickly iterate through multiple experimental runs and refine results until you are satisfied; then, you implement the partitioning solution you decided was best.

1. Set up an initial target system specification (TSS) that models the available hardware resources.
   - At a minimum, this TSS file must define the FPGAs. Additionally, you can define the clocks; the port connections and daughter boards; and inter-FPGA cabling in this file, depending on your design restrictions and your knowledge of the architecture. See Exploring Partition Choices with a Basic TSS File.
   - For trace names and connection information, use the `report target_system -tss` command. See Checking the Target Specification (TSS) for details.
2. Refine the hardware setup, by defining exploratory constraints in *pcf* file.

   It is recommended that you separate different kinds of constraints into multiple *pcf* files, so that you can use them in a modular fashion. For example, create an *abstract.pcf* and a *setup.pcf*file. See Using Abstract PCF Commands for Partition Exploration for information about creating these files.
3. Start with a pre-partitioned database state, and partition it specifying the *TSS* file and the *pcf* setup constraints. For example:

   ```
   run partition -tss abstract.tss -pcf setup.pcf -pcf abstract.pcf
   ```

   See Partitioning the Logic for details about generating the partition database state.

   When you use a basic TSS, the partitioner generates a partitioned database state, called *pa0* by default. You cannot proceed to the next stage of design, system routing, from this database state because it was generated with a basic TSS file and exploratory constraints in the *PCF* file. To partition the design, you require a complete *TSS* file definition of the hardware, with no abstract constraints.

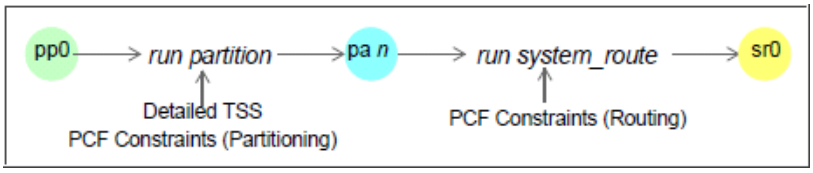
   The partitioner also generates a log file and a report. If you have conflicting constraints in multiple files, the partitioner implements what was specified in the last file.
4. Examine the log and report (*prf*) files using the `view report` command, to determine what constraints to add or adjust for the next run.

   See Analyzing Partitioning at Different Stages and Checking Partitioning Reports for details.
5. Rerun partitioning with updated TSS and *PCF* files.

- Rerun partitioning with these files and the TSS file.
- Analyze the log and report files.
- Repeat this set of steps until you are satisfied with the results. Partitioning with a basic TSS and *pcf* constraints is quick, so you can easily loop through several iterations.

6. When you are satisfied with the results, run partitioning with a complete TSS file, and run system routing.
   - Create a detailed TSS file for system routing. Convert the *pcf* constraints needed to implement the partitioning solution you want (step 5) into `board_system*` commands in a new, detailed TSS file, and remove the original abstract *pcf* constraints you used for earlier runs. The detailed TSS file is required to proceed beyond partitioning.

     See Defining the Target System in a Detailed TSS File for more information about creating this file, and Target System Specification Commands for command descriptions.
   - Run partitioning with the detailed TSS file. This generates a partitioned database state where you can run system routing.
   - Specify any constraints to guide system-level routing in a *PCF* file (*route.pcf*, for example) and specify it when you run system routing.
   - Run system routing as described in Running System Route for the Top Level.



7. To lock down a final partitioning solution, follow these steps:
   - When you have a solution you want to save, use the `export file` command to export the constraints file and TSS, if you have one.
   - Specify the exported constraints file and the detailed TSS when you rerun partitioning or system route.

   See Locking Down Partitions for information about using *pcf* files for incremental flows.

Configuring Files for Partitioning

To partition a design, configure the following files:

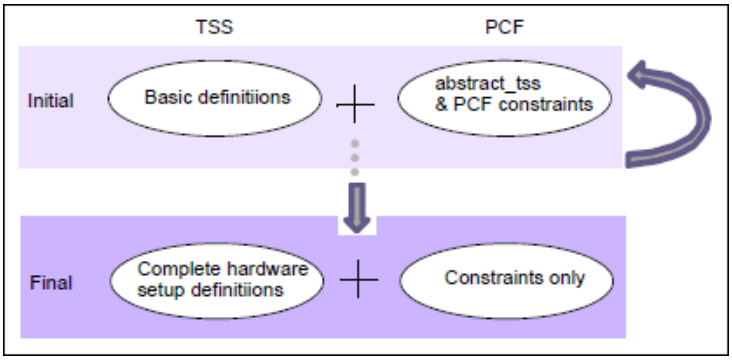
Files	Format/File
HDL logic design files. See Adding Design Files.	Verilog/VHDL or both
One or more partition constraint files to explore partitioning options. See Configuring Constraints in PCF Files.	*.pcf*
Board description or target system specification (TSS) file. This is the software model of the hardware configuration, and can be basic or detailed. See Defining the System in a TSS File.	*.tcl*

Defining the System in a TSS File

The Target System Specification (TSS) file is a Tcl file that defines the hardware configuration. The hardware configuration is described using `board_system` commands that define the hardware board, interconnect, clocks, resets, and voltage regions.

At the initial exploratory design phase, the TSS file contains basic partition information. It is used in conjunction with *abstract_tss* constraints entered in the *PCF* file. As you refine your partition choices, the TSS file specifies your final partition choice and the actual hardware. Partitions that can be routed are based on the specifications in this file.

Figure 8 TSS File



Use hUtils to generate the TSS file for the existing hardware. For more information, see *hUtils* documentation in SupportNet.

Feedback

For more information about using the TSS file, see these topics:

- Exploring Partition Choices with a Basic TSS File
- Defining the Target System in a Detailed TSS File
- Configuring Clocks
- Defining Hardware Interconnect
- Moving from Abstract PCF to TSS: Flow Example
- Validating TSS Cable Connections

For additional information, see these topics:

- Design Methodology for Partitioning
- Working with HAPS Systems

Exploring Partition Choices with a Basic TSS File

The basic TSS file contains basic system configuration commands, and is used in conjunction with abstract *pcf* constraints to explore partitioning choices. As you iterate through partitioning, update the TSS file to reflect your choices, until you run system routing with your final choices and a detailed TSS file. For example, see Moving from Abstract PCF to TSS: Flow Example.

The following procedure describes the basic TSS commands for system configuration. Detailed descriptions for these commands are in Target System Specification Commands.

1. Create a basic TSS file:
   - Use the `launch tss` command to open the TSS editor. You can use the built-in syntax help for guidance.
   - Enter `board_system_create` commands to define the systems. If you type `help` in the TSS editor, it displays help and syntax for the TSS commands. For an initial run, all that is required is a definition of the correct number of FPGAs. Here are some examples:

     ```
     board_system_create -haps -name haps-100sys
     board_system_create -add HAPS100_4F -name fb1
     board_system_create -haps -name haps80sys
     board_system_create -add HAPS80_S104 -name fb1
     ```

     To get the exact keywords, use the `board_system_list -objects` command.
   - Specify the speed grade with the `board_system_create -speed_grade` command in the TSS file. It is recommended that you do this for each system added.

Feedback

```
board_system_create -add HAPS80_S26 -name FB1 -speed_grade {-2-e}
```

To view the valid speed grades for the current system (which is set with *option set technology*) use `option get valid_speed_grades`. Specifying a speed grade sets it for all FPGAs on the HAPS system, but you can specify different speed grades for different systems. During the partitioning process, the speed grade and technology are written to the individual FPGA partition files for synthesis. Do not change these settings at the individual FPGA level.

For single FPGA designs, you can either define the speed grade in the TSS file or use the `option set speed_grade` command.

- Save the file.

As you proceed through more iterations, add details to the TSS, like the clock and reset plan. For more information, see Defining the Target System in a Detailed TSS File.

2. Set `abstract_tss` commands in the *pcf* file to define configuration schemes and I/Os that you want to experiment with. This example defines an interconnect scheme:

```
abstract_tss -add_port_bin -fpga FB1.uA -name \ ABS_PORTA -width 650
abstract_tss -add_port_bin -fpga FB1.uB -name \ ABS_PORTB -width 650
```

The *pcf* constraints do not all have to be defined at the same time or in the same file, but can be added successively to different files, as you develop the prototype. See Configuring Constraints in PCF Files.

3. Partition the design with `run partition`.

This run is very fast. The run does not generate a new database state, because it is run with abstract *pcf* commands and a preliminary *tss* file. The `run partition` command has options that offer additional controls for exploratory runs. See run partition for the complete syntax.

- Set *-effort* to *low*, *normal*, or *high*. The setting affects the default setting for other `run partition` command arguments.

Argument	Low	Normal	High
`-max_trials`	150	1000	1000
`-solutions`	1	3	6
`-hierarchy_dissolve_pass`	1	2	2
`-multi_hop_accuracy`	0 (estimate)	0 (estimate)	1 (full accuracy)

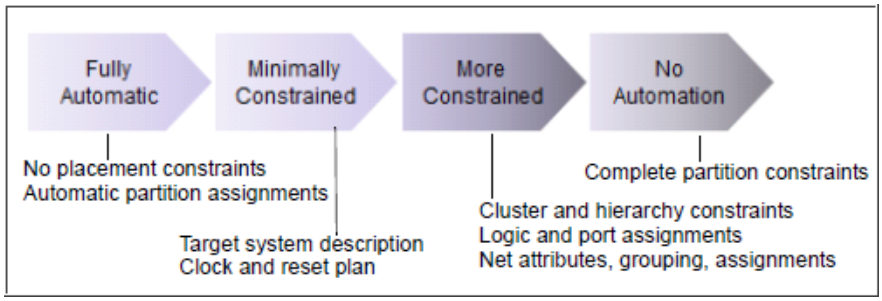
Feedback

- Specify other `run partition` arguments to override the defaults set by *-effort*:

Argument	Description
`-max trials`	Specify the number of exploratory trials you want the partitioner to run.
`-solutions`	Specify the number of solutions to evolve in parallel during the partitioning run.

- For initial partitioning runs, you can also set the advanced options `-clock_crossing_reduction` and `-llegal_feedthrough_reduction` to *none* or *low*.

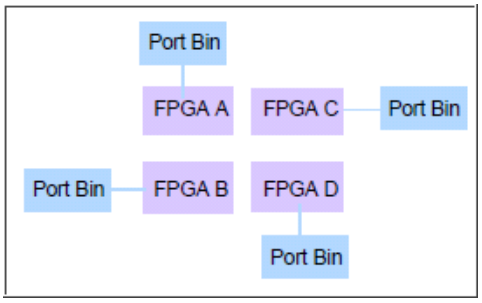
4. Adjust *pcf* constraints and *tss* definitions.
   - Change *pcf* constraints and rerun partitioning.
   - As you settle on design details, remove the `abstract_tss` constraints from the *pcf* file, and add the equivalent commands in the *tss* file. See for typical commands added to the *tss* file during exploration.
   - Iterate as needed, as described in Design Methodology for Partitioning. The following figure shows how the TSS constraint definitions evolve with the prototype, driven by the TSS rather than the *pcf*, until it is defined by the TSS file:



5. Define other configurations in the TSS file.
   - Create one bin for each FPGA. A bin is collection of target resources, a group of logical or physical elements of the HAPS system that is referenced in the TSS file. Basic TSS does not need to be set to physically realizable limits.
   - Create one port bin for each FPGA:

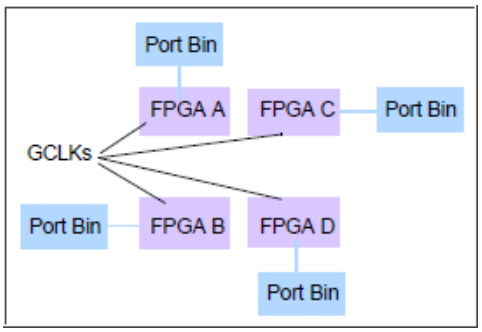
```
board_system_configure -top_io {FB1.A1} board_system_configure
-top_io {FB1.B1} board_system_configure -top_io {FB1.C1}
board_system_configure -top_io {FB1.D1}
```

Feedback



- Create one or more locked bins for clocks, resets, black boxes and internal memories. You must define all bins before defining traces. Map known design clocks to HAPS *pll* or *fpga* sources.

  ```
  board_system_configure -clock FB1.GCLK1 pll board_system_configure
  -clock FB1.GCLK5 pll board_system_configure -clock FB1.GCLK8 fpga
  ```



- Define traces that connect each port bin to its corresponding FPGA.
- Define traces that connect all bins in pairs.
- Define traces for clocks and resets.

6. Ensure that the TSS file reflects the *pcf* constraints you used to achieve the result, and use this detailed TSS file to partition the design and run system routing.

   See Defining the Target System in a Detailed TSS File for details. The *pcf* file must not contain any configuration constraints; it should only contain constraints for partitioning and system routing. The detailed *tss* file used for partitioning overrides previous *tss* files used for pre-partitioning.

Defining the Target System in a Detailed TSS File

To run system routing and create FPGA partitions, you need a detailed TSS file, which defines the actual hardware setup. If you used *abstract_tss*PCF commands to explore prototype configurations (Exploring Partition Choices with a Basic TSS File), you must make sure that the final configuration information from the *pcf* constraint file is reflected

Feedback

in the TSS file before you run system routing, and that the *pcf* file no longer contains the *abstract_tss* commands used for partition exploration.

The TSS is the only definition of the hardware that the prototyping software uses as a basis for partitioning, so it is important that it be modeled completely and accurately. The detailed *tss* file used for partitioning overrides previous *tss* files used for pre-partitioning, for example.

- For a partitioned individual FPGA, the final TSS file must define the HAPS system, the global clock, the global reset, and the UMRBus connections.
- For a multi-FPGA design, the TSS must additionally define CDE chaining, and clock-left and clock-right connections from board to board. CDE is the HAPS supervisor module.e Configuring Connections with board_system_create -interconnect for more information about UMRBus chaining.

The following procedure refers to various commands; the complete syntax for each is described in Target System Specification Commands. See Defining the Target System in a Detailed TSS File for an example of this file.

1. Prepare to enter the TSS commands.

   At the exploratory initial stage, a TSS file can be accompanied by a PCF file with `abstract_tss` commands that define board configuration information that is eventually defined in the TSS.

   - For a final TSS file, make sure that there are no `abstract_tss` commands defined in accompanying *pcf* files. If there are `abstract_tss` commands defined, they take priority and you cannot complete the later partitioning phases and implement the partitions.
   - Use the `launch tss` command to open the TSS editor. If you use the editor to enter the TSS commands, you can take advantage of the built-in syntax help. Type *help* to see syntax details.

2. Use the `board_system_create` command to specify the number of FPGA partitions:

   - Supply a name for the target system. For example, this command defines a target system called `six_chip_sys`:

     ```
     board_system_create -haps -name six_chip_sys
     ```

   - Add HAPS systems and daughter boards with the *-add* argument. Use a separate `board_system_create -add` command to add each board.

     ```
     board_system_create -add HAPS100_4F -name FB1
     ```

     For daughter boards, use `board_system_create` to define daughter board positions and connections. (See Configuring Connections with board_system_create -interconnect.)

Feedback

- Use the following command to check for available systems, daughter boards, and interconnect boards:

  ```
  board_system_list -objects
  ```

- To add HAPS-compliant custom daughter boards, use additional arguments to the `board_system_create` command as described in Defining Custom Daughter Board Connections.

-haps_custom_db	Specify the custom daughter board
-HT3 |*-HT2* | *-MGB* | *-HSIO*	Specify the connector for the custom board: HapsTrak 3, HapsTrak II, HapsTrak MGB, or HapsTrak High Speed I/O. You must specify the list of connectors and their types.
-guts	Create a Verilog *_guts.v* file that defines the connection model for the module. It must have the black box information and pin connections for the custom daughter board, and include a trace delay value (*.tdv*) file. See Example: _guts.v File for an example of the pin connections. Then specify the file with this argument so that traces are made available for assignment. If using a relative path, specify it relative to the pre-partition state. The contents of this file are included in the module definition for the custom daughter board.

  The following example specifies a custom adapter board, with pin connections defined in the *.v* file defined with the *-guts* option:

  ```
  board_system_create -haps_custom_db "HAPS_JUNO_ADAPTER" -HT3 {JX1
  JX2 JX3} -guts JUNO_ADAPTER.v
  ```

3. Define the board-to-board connections.

   You must model the top-level I/O ports, all the hardware connections between boards, and the cabling.

   - Specify board-to-board connections with `board_system_create -interconnect`, as described in Configuring Connections with board_system_create -interconnect. For examples of daughter board connections, see Examples: Daughter Board Connections.
   - If needed, use the *-disconnect* option to disconnect boards in an existing file:

     ```
     board_system_create -disconnect fb_3 board_system_create -disconnect
     connC
     ```

Feedback

4. Map known design clocks to HAPS sources with the `board_system_configure -clock` command.

   See Configuring Clocks and Working with HAPS Global Clocks for more information.

5. Define board system parameters.

   - To view available parameters for an instance, use this command:

     ```
     board_system_list -parameters -instance instanceName
     ```

   - To define voltage regions, use `board_system_configure -voltage`. The following example shows typical commands for a HAPS system:

     ```
     board_system_configure -voltage {fb1.uA_V1 fb1.uA_V2 fb1.uA_V3
     fb1.uA_V4 fb1.uA_V5 fb1.uA_V6 fb1.uA_V7 fb1.uA_V8 fb1.uA_V9
     fb1.uA_V10 fb1.uA_V11 fb1.uA_V12} 1.2 board_system_configure
     -voltage {fb1.uD_V8 fb1.uD_V9 fb1.uD_V10 fb1.uD_V11 fb1.uD_V12
     fb1.uD_V13 fb1.uD_V14} 0
     ```

   - Set the system (technology) and speed grade to match the target set for the compile stage. You get a warning message if you do not define these parameter. If you did not define the technology when you created the database with `database create`, define it as an option:

     ```
     option set technology haps-200
     ```

     Define the speed grade in the TSS file:

     ```
     board_system_create -add HAPS80_S26 -name FB1 -speed_grade {-2-e}
     ```

6. When editing an existing file, use the *-disconnect* option to disconnect extra traces or board systems:

   - In the following example 400 traces are created between FPGAs uA and uB because of the *-interconnect -auto* option:

     ```
     board_system_create -interconnect -auto -width 400 -devices {fb_1.uA
     fb_1.uB}
     ```

     If you only require 280 traces you can remove the 120 extra traces with the *-disconnect -width* option:

     ```
     board_system_create -disconnect -width 120 -devices {fb_1.uA
     fb_1.uB}
     ```

   - Use *-disconnect* option to remove systems or interconnect boards:

     ```
     board_system_create -disconnect fb_3 board_system_create -disconnect
     connC
     ```

Feedback

Configuring Clocks

Configure clocks in the final TSS file to set up global clocks (GCLKs) from the HAPS global clock network and define the sources for the global clocks. Clocks that are not mapped to HAPS clock sources can cross FPGAs and cause skew.

This is just one part of the clock definition. See Assigning Clocks in the PCF

1. Check the clocking scheme in the hardware manual for the board system. The TSS file supports direct and indirect clock synchronization.

   Consult the hardware Reference manual for details about the clock infrastructure for the system you are using.

2. Define port bins for the FPGAs with the `board_system_configure -top_io` command.

   You can use the `launch tss` command to open an interactive Tcl shell and enter the TSS commands there. If needed, use the `board_system_list` command to list available resources.

   ```
   board_system_configure -top_io{FB1.A4} board_system_configure
   -top_io{FB1.B1}
   ```

3. Specify a source (*fpga*, *pll*, or *external*) for GCLKs to be used for the design. For example:

   ```
   board_system_configure -clock FB1.GCLK8 fpga board_system_configure
   -clock FB1.GCLK1 pll
   ```

   The `board_system_configure -clock` commands tie the HAPS clock sources to specific clocks on the GCLK network for the HAPS system. For step-by-step details, see Working with HAPS Global Clocks.

   To complete the HAPS-aware clock definition, you must also connect the design clock nets to the GCLK network and the clock sources, using PCF constraints. For details, see Assigning Clocks in the PCF.

4. Define the clock frequency for the clock source with the *-frequency* option:

   ```
   board_system_configure -pll FB1.PLL1_1 -frequency 10000
   ```

5. Configure other connection parameters for the clocks, like feedback, the driving source for PLL, or indirect clock synchronization.

   This example instantiates a CDE_CABLE across the CLK connector pairs for a HAPS series system:

   ```
   board_system_configure -connect {fb1.CLK_RIGHT fb2.CLK_TOP}
   ```

Feedback

Defining Hardware Interconnect

Ensure that the TSS file models the hardware connections between FPGAs. This is required to correctly represent the physical setup to enable improved performance.

See the following:

- Configuring Connections with Auto-Cabling
- Configuring Connections with board_system_create -interconnect
- Defining Daughter Boards

Also, see Working with HAPS Systems.

Configuring Connections with Auto-Cabling

The choice of inter-FPGA cables has an immediate effect on partitioning, and ultimately affects routability and system performance. Automatic cabling or *auto-cabling* automates the normally iterative and time-consuming process of identifying the best configuration of HT3 cables.

The alternative is to manually define the connections. See Configuring Connections with board_system_create -interconnect for details about this method.

The following procedure describes how to take advantage of auto-cabling.

1. Create an initial TSS file that contains the following basic information:

Required Definitions	Optional
HAPS system/module definitions Daughter board connectivity Reserved top-level I/Os	Clock connectivity GCLK definitions

2. Add cable definitions based on your knowledge of the design, while keeping congestion and routability in mind.

   You can define the cables in one of two ways: set abstract definitions in the PCF or TSS files, or set specific TSS definitions. With abstract definitions, you cannot specify cable length, but with specific TSS definitions, you can define the HT3 cable lengths for the connections.

Feedback

- To define specific cable connections, include the `board_system_create -interconnect -manual` command in the TSS file. For example:

  ```
  board_system_create -interconnect -manual CON_CABLE_25_HT3 -name conn2 -connector {FB1.A23 FB2.B23}
  ```

  When auto-cabling is enabled, the tool can modify the connection for a better result and move the cable to a different pair of connectors.

- To define specific hard connections that auto-cabling honors, use the *-hard 1* option with the TSS `board_system_create -interconnect -manual` command. For example:

  ```
  board_system_create -interconnect -manual CON_CABLE_25_HT3 -name conn2 -connector {FB1.A23 FB1.B23} -hard 1
  ```

  Auto-cabling keeps this connection as-is and not modify it.

- For abstract cable definitions, use either of these methods:

PCF	*abstract_tss -add_trace_group {FB1.uA FB1.uB FB2.uA FB2.uB} -width 200 -connect_all*
TSS	*board_system_create -interconnect -auto -width 96 -devices {FB1.uA FB1.uB}*

  Do not specify a cable length with abstract definitions; auto-cabling uses 100 cm cables for all abstract connection definitions. The advantage to abstract connections is that they are easy to use.

- If there are multiple cable definitions, the order of precedence is abstract flow using PCF definitions, abstract definitions in the TSS, and lastly, defined HT3 cable connectivity.

3. Add definitions for certain cases, so auto-cabling handles them correctly.
   - For empty locked FPGA bins that are reserved for future use, define all these connectors as *-top_io*. If a locked FPGA bin contains logic, do not exclude the connections from consideration by automatic cabling.
4. Enable auto-cabling by adding the `auto_tss_control -enable 1` command to the PCF file.

   The default is to disable auto-cabling and specify all connections manually (`auto_tss_control -enable 0`), so you must specifically enable auto-cabling.
5. Partition the design.

- Partition the design using the TSS file and the PCF file.

  ```
  run partition -pcflist pcflist.txt -out a1 -tss basicTss.tss
  ```

  Auto-cabling runs at the partitioning stage. It keeps any cables defined with *-hard 1* as is, but changes other connections as needed to optimize the design. It considers daughter board connections and port bins to be hard, and does not change them.

  For single-FPGA designs and connections between FPGAs on the same system, auto-cabling does not alter cable lengths. For connections between systems in a multi-FPGA design, it automatically uses 100 cm cables for inter-system connections, keeping the same number of cables. Currently, you cannot specify available cable resources or the relative positions of connections.

  The tool creates a new TSS file called *auto_cable.tss.* which includes the automatic cabling definitions as well as the other TSS information previously defined. If you have manual hierarchical partitions, the new TSS is generated after the second pass, after partitioning.

- Export the generated TSS file:

  ```
  export file auto_cable.tss
  ```

6. Use the exported TSS file and PCF file as input to rerun partitioning.

   - Partition the design with `run partition`, using the exported TSS file and the PCF file. For example:

     ```
     run partition -pcflist pcflist.txt -out a1 -tss auto_cable.tss
     ```

     This example shows a typical sequence of commands:

     ```
     database set_state {pp0} # prepartition database
     run partition -pcflistpcflist.txt -out a0
     export file auto_cable.tss
     run partition -pcflistpcflist.txt -out a1 -tss auto_cable.tss
     ```

7. Check the results.

   For the results, check the Automatic Cabling section of the log file after partitioning. It contains details like the number of cables between FPGA pairs, and the number of occupied and usable connectors for each FPGA.

   When you are satisfied with the partitions and cable connections, continue with system route and system generate.

Configuring Connections with board_system_create -interconnect

After defining the component boards that make up the system, define the connections between them. It is very important that the TSS file model the hardware connections

Feedback

to correctly represent the physical setup. The way the interconnect is set up can greatly influence performance. There are manual and automatic methods to specify the interconnect between boards:

Manual	• Explicit interconnect specification: *board_system_create -interconnect -manual* • Implicit; defines the number of traces and leaves the connections to the tool:*board_system_create -interconnect -auto*
Auto cabling	Configuring Connections with Auto-Cabling

See the following for information about defining connections manually:

- Configuring Connections Manually
- Configuring Interconnections Explicitly or Implicitly
- Defining Daughter Boards

Configuring Connections Manually

The following procedure provides an overview of using `board_system_create -interconnect` commands to manually specify hardware board-level connections, top-level ports, and cabling. It covers both explicitly specified and implicitly specified connections.

See Target System Specification Commands for the syntax for the commands mentioned. For additional hardware specifics, refer to Working with HAPS Systems.

1. Create an initial TSS file that contains the following basic information:

Required Definitions	Optional
HAPS system definitions Daughter board connectivity Reserved top-level I/Os	Clock connectivity GCLK definitions

   See Defining the Target System in a Detailed TSS File for information about adding the boards to the TSS.

2. Define the interconnect between boards using explicit or implicit `board_system_create -interconnect` commands.

   A *board* in this context can be a multi-FPGA HAPS system with each FPGA having a different connector designation, or daughter boards with different connectors. For details, see Configuring Interconnections Explicitly or Implicitly.

3. Define the interconnect for the daughter boards with the `board_system_create -interconnect -manual` command.

   See Defining Daughter Boards for details about connecting different kinds of daughter boards and custom boards.

4. When editing an existing file, use the *-disconnect* option to disconnect existing interconnections before defining new connections.

5. Define port bins and top-level I/O bins with the `board_system_configure -top_io` command:

   ```
   board_system_configure -top_io {FB1.A1}
   ```

   To specify multi-FPGA I/Os, first create a breakout and then specify the `board_system_configure -top_io` command on one of the breakout connectors. This example creates a trace route from TOP_IO-A to B:

   ```
   board_system_create -interconnect -manual BREAKOUT_HT3 -name
     conn_topa1 -connector {fb1.A1 0 0}
   board_system_create -interconnect -manual CON_CABLE_100_HT3 -name
     connab -connector {conn_topa1.J1 fb1.B1}
   board_system_configure -top_io {conn_topa1.J2}
   ```

6. Define UMRBus chaining between boards with `board_system_configure` commands:

   ```
   board_system_create -add HAPS100_4F -name fb1
   board_system_create -add HAPS100_1F -name fb2
   board_system_configure -connect {umrbus_if.CDE_OUT fb1.CDE_IN}
   board_system_configure -connect {fb1.CDE_OUT fb2.CDE_IN}
   ```

   When you define UMRBus chaining, the tool automatically instantiates *CDE_CABLE* as the interconnect board across connectors. If you mix different board systems, the newer systems must be at the beginning of the chain.

Configuring Interconnections Explicitly or Implicitly

Use the `board_system_create -interconnect` command to define connections between boards. A *board* can be a multi-FPGA HAPS system. Each FPGA has a different connector designation or daughter boards with different connectors.

1. To directly specify the interconnect, use the `board_system_create -interconnect -manual` command.

   To specify a connector, use the name of the board instance, followed by the designation for the on-board connector. For example, *fb_1.C1* indicates the *C1* connector on the *fb_1* board.

   The following example defines the interconnect between *fb_1* and *fb_2*, using HAPS interconnect cables. The first command defines a *CON_CABLE* connection between

Feedback

the A2 and D1 connectors on *fb_1*. The second command defines a *CON_CABLE* connection between the A4 connector on *fb_1* and the D3 connector on *fb_2:*

```
board_system_create -interconnect -manual CON_CABLE -name connB
  -connector {fb_1.A2 fb_1.D1}
 board_system_create -interconnect -manual CON_CABLEX -name connC
  -connector {fb_1.A4 fb_2.D3}
```

If a warning displays, the specified target board does not support the connector and is incompatible with the interconnect board. Use the `board_system_list -type interconnectboard` command to see a list of available connections.

2. To use the implicit method, use the `board_system_create -interconnect -auto` command to define the number of traces required between FPGAs. Allow the tool to determine the interconnect.

   You can specify multiple trace requests between devices. The tool analyzes all the requirements and creates a TSS file after considering all the on-board direct traces. The following command defines 250 traces between FPGAs C and D on *fb_1*:

   ```
   board_system_create -interconnect -auto -width 250 -devices {fb_1.uC
   fb_1.uD}
   ```

   Do not use this TSS command to define more traces than are physically available at an initial stage. This causes an error. For early-stage exploration purposes, use abstract *pcf* commands instead.

   When you use *-auto*, do not define voltages (`board_system_configure -voltage`), or you could get an error.

3. To check the connectors and locations assigned to the board, add the `report_board_system fileName` command as the last line in the *tss* file.

   This generated file contains information about the connectors automatically selected by the tool to match your criteria. The following is a sample of the format used in this report:

   ```
   Interconnect Boards
   ********************
   Type                   Name            Connections
   ------------------------------------------------------
   CON_CABLE_100_HT3      conauto_0       fb1.A12  fb1.B12
   ```

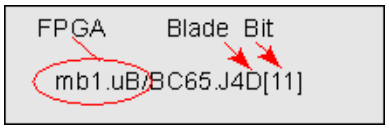
   In addition, the report file generated after *run system_route*, contains detailed interconnect information:

   ```
   @S2.2.13.1 AP414 |TraceGroup T_fb1.uB_fb1.uD_1
   trace fb1_B4_D[11] -group T_fb1.uB_fb1.uD_1 -pins {
     fb1.uB/BC65.J4D[11]
     fb1.uD/BC66.J7A[9]}
   ```

Feedback

The preceding snippet reports the following:

Pin 11 of the fourth blade/row (D) of Connector 4 (J4) is connected to Connector 7 on FPGA D. A blade is a row of pins. The HapsTrak 3 connector has four blades, A to D.



4. When editing an existing file, use the *-disconnect* option as needed to disconnect extra traces if needed.

   In this example, there are 400 traces created between FPGAs uA and uB because of the *-interconnect -auto* option:

   ```
   board_system_create -interconnect -auto -width 400 -devices {fb_1.uA fb_1.uB}
   ```

   If you only require 280 traces you can remove the 120 extra traces with the *-disconnect -width* option:

   ```
   board_system_create -disconnect -width 120 -devices {fb_1.uA fb_1.uB}
   ```

Defining Daughter Boards

This section describes how to add and connect Synopsys and custom daughter boards to define the hardware setup. It includes some examples, but is not exhaustive. For details, consult the individual daughter board manuals.

- Defining Daughter Board Connections
- Examples: Daughter Board Connections
- Defining Custom Daughter Board Connections

Defining Daughter Board Connections

This procedure describes how to define connections for Synopsys daughter boards and interface cards in the *tss* and *pcf* files. For custom daughter board connections, see Defining Custom Daughter Board Connections.

For details about the syntax of commands in the following procedure, refer to the *Command Reference Manual*.

1. Add daughter boards in the *tss* file.
   - Get a list of supported daughter boards for the HAPS system.

     ```
     launch tss -mode shell
     board_system_list -objects -type daughterboard
     ```

Feedback

◦ Add daughter boards in the *tss* file with the `board_system_create -interconnect -manual` command.

There are some differences in how you use the command for different kinds of daughter boards. The following table summarizes how to use `board_system_create -interconnect -manual` (abbreviated to *the command*, in the table) to define different daughter boards:

HAPS/HapsTrak daughter boards: SRAM, SDRAM, HTII_ADAPTER2_H3, etc.	▪ Instantiate with *board_system_create -interconnect -manual*. ▪ See Examples: Daughter Board Connections.
ECDB (External clock distribution board)	▪ Instantiate with the command. ▪ See also Example: DDR3 Daughter Boards.
DDR3 daughter boards	▪ Follow guidelines in step 3 below. ▪ Instantiate with the command. ▪ See also Example: DDR3 Daughter Boards.
Two-connection daughter boards, like LAB_HT3	▪ Make sure to specify both top and bottom connectors in the *tss* file. If you are not using one of the connections, set it to 0. ▪ See also Example: LAB_HT3 Daughter Boards.
HAPS-compliant custom daughter boards	▪ Use specific *board_system_create*command options, as described in Defining Custom Daughter Board Connections.
MGB interface cards	▪ Specify two *board_system_create -interconnect -manual* commands as described in step 4. ▪ See also Using MGB Interface Cards with HAPS-100 Modules.
QSFP (Quad Small-Form Pluggable) interface cards	▪ Specify two *board_system_create -interconnect -manual* commands as described in step 4. ▪ See also Example: QSFP+ MGB Interface Cards.
Two PCI Four-Lane Cards	▪ Use a separate riser card for each PCI interface card or share one riser card. ▪ See also Example: Two PCI Express Four-Lane MGB Interface Cards.

2. When editing an existing file, use the *-disconnect* option to disconnect existing interconnections.
3. To connect DDR daughter boards, use the `board_system_create -interconnect -manual` command, making sure to follow the additional guidelines below.

- Connect the daughter board to three HT3 connectors in the same SLR on the HAPS system. Refer to the HAPS system reference manuals for special or limited connectors.
- Configure the voltage regions by setting the VCCO of the selected HT3 connectors to 1.5 V.

  DDR3_SODIMM_HT3 and DDR3_SODIMM2R_HT3 daughter boards are powered by the HapsTrak 3 connector. Daughter boards contain an ID PROM on each HapsTrak 3 connector. The ID PROM is used for board identification and over-voltage protection. It ensures that the applied VCCO is compliant with the daughter board. Signals are impedance-controlled (50 Ohms).
- Add *pcf* constraints to assign DDR3 interface ports to the corresponding traces. Use this file as input when running partition and system route. Refer to the example in Example: DDR3 Daughter Boards for *pcf* constraints and *tss* specifications.

4. Instantiate MGB interface cards with two `board_system_create -interconnect -manual` commands.

   The TSS is interpreted sequentially. To add MGB cards, use a two-step process with two `board_system_create -interconnect -manual` commands. The first command adds a riser card to the interface card. The second command adds the interface card to the riser. For connector details, refer to the interface card documentation.

   - For QSFP interface cards, use these guidelines. The eight-lane RISER8_MGB riser card has two connectors, J2 and J1. Connect J1 to one of the MGB connectors available. The *QSFPPLUS_MGB* interface card plugs into the J2 connector on the riser card. See Example: QSFP+ MGB Interface Cards.
5. To connect boards with HapsTrak II connections, add an adapter board and follow this example: Example: HTII_ADAPTER2_HT3.
6. Assign daughter board interface ports to trace names.
   - View trace names for the daughter boards. Run pre-partition (or pre-partition and partition) with the *tss* file you created.

     Use the `report target_system -tss` and `view report` commands to generate *report pcf_tss_report.rpt* and view the trace names for the daughter boards, respectively.
   - Use `assign_port` command in the *pcf* file to assign interface ports from the daughter board to corresponding trace names.

Feedback

The tool converts these assignments into I/O standard and pin location constraints. The *pcf* constraints are read and used during partition and system route.

7. Use the *pcf* file with trace assignments and *tss* file with hardware setup description as input to run partition and system route.

Examples: Daughter Board Connections

The following examples illustrate how to use the *board_system_interconnect* command to connect daughter boards. Refer to the appropriate hardware manuals for complete details.

Example: HTII_ADAPTER2_HT3

To use HapsTrak II daughter boards with HapsTrak 3 systems an adapter board must be used. This example illustrates how to instantiate the HTII_ADAPTER2_H3 board.

1. In the *tss* file, add an adapter board and make the HapsTrak II side the top I/O, as shown here:

```
# Instantiate HTII_ADAPTER2_HT3 on HAPS system
board_system_create -interconnect -manual HTII_ADAPTER2_HT3 -name
ht2_adpt_ht3 -connector {0 FB1.A23 FB1.A22 FB1.A21}
# Make the HT2 side the top I/O
board_system_configure -top_io {ht2_adpt_ht3.J1 }
```

2. Consult the adapter board documentation to find out how to map the HapsTrak II pins to the HapsTrak 3 pins.

3. Use `report target_system` to find available HapsTrak 3 traces. The generated report shows information like this example:

```
#MY_FLASH connected to J1 of HTII_ADAPTER2_HT3
(which connects to HAPS on JX1, JX2, JX3)

    #A[26] #MY_FLASH JX1_A[27] HTII_ADAPTER2_HT3 JX1_B[9]
    #A[25] #MY_FLASH JX1_A[26] HTII_ADAPTER2_HT3 JX1_B[2]
    #A[24] #MY_FLASH JX1_A[25] HTII_ADAPTER2_HT3 JX1_B[3]
    #A[23] #MY_FLASH JX1_A[24] HTII_ADAPTER2_HT3 JX1_B[4]
    #A[22] #MY_FLASH JX1_A[23] HTII_ADAPTER2_HT3 JX1_B[5]
```

Top-level Pin — HapsTrak II Pin — HapsTrak 3 Pin

4. Assign the pins to the correct HapsTrak 3 traces in the *pcf* file with the `assign_port` command. For example:

```
assign_port {async_A[26]}}
-trace {FB1_A1_B[9]} assign_port {async_A[25]}}
-trace {FB1_A1_A[2]} assign_port {async_A[24]}}
-trace {FB1_A1_A[3]}} assign_port {async_A[23]}}
-trace {FB1_A1_A[4]}
```

Feedback

Example: HAPS SRAM Daughter Board Connections

To use a HapsTrak II daughter board on HapsTrak 3 connectors, instantiate the adapter board as shown in Example: HTII_ADAPTER2_HT3. Then instantiate the SRAM. The following command adds two *SRAM_1x1* HAPS SRAM daughter boards:

```
board_system_create -interconnect -manual SRAM_1x1 -name sram1 -connector
{fb_1.A1}
```

The next example stacks the SRAM boards by specifying the connector and including the *-connect_at top* option.

```
board_system_create -interconnect -manual SRAM_1x1 -name sram2 -connector
{sram1.J1} -connect_at top
```

Example: DDR3 Daughter Boards

The *DDR3_SODIMM_HT3* and *DDR3_SODIMM2R_HT3* daughter boards contain the DDR3 SODIMM memory module. If you are connecting these daughter boards, follow the connection guidelines described in Configuring Connections with board_system_create -interconnect, step 4.

This is an example of the *tss* commands to specify a DDR3 daughter board on a HAPS-100 system:

```
# Define board system
board_system_create -haps -name ddrcard_board
board_system_create -add HAPS100_1F -name fb1

# Define top-level I/Osboard_system_configure -top_io {fb1.A1 fb1.A7
fb1.A8 fb1.A9 fb1.A10}
board_system_configure -top_io {fb1.B1 fb1.B7 fb1.B8 fb1.B9 fb1.B10}
# Instantiate daughter board. Connect to A4, A5, A6 on the HAPS
#system, which represent HT3 connectors 3, 2, and 1 on FPGA A of# the
HAPS system. The I/O standard is automatically set to SSTL.
#
board_system_create -interconnect -manual CON_CABLE_100_HT3
 -name connab  -connector {fb1.A2 fb1.B2}
board_system_create -interconnect -manual  DDR3_SODIMM_HT3
 -name DDR3 -connector {fb1.A4 fb1.A5 fb1.A6}

# Configure voltage to 1.5V. The I/O standard is automatically set
# to SSTL.
board_system_configure -voltage fb1.uA_V4 1.5
board_system_configure -voltage fb1.uA_V5 1.5
board_system_configure -voltage fb1.uA_V6 1.5

# Configure clock; reset configured automatically
board_system_configure -clock fb1.GCLK0 100Mhz
board_system_configure -clock fb1.GCLK1 pll
board_system_configure -clock fb1.GCLK2 pll
board_system_configure -clock fb1.GCLK3 pll
```

Feedback

```
board_system_configure -clock fb1.GCLK4 pll
board_system_configure -reset {fb1.uA_reset fb1.uB_reset} 0
```

Use *assign_port* constraints in the *pcf* file to make corresponding trace assignments:

```
assign_port {ddr_odt[0]} -trace {DDR3.DDR3_ODT[0]}
assign_port {ddr_cs_n[0]} -trace {DDR3.DDR3_S_B[0]}
assign_port {ddr_we_n} -trace {DDR3.DDR3_WE_B}
assign_port {ddr_reset_n} -trace  {DDR3.DDR3_RESET_B}
assign_port {ddr_ras_n} -trace {DDR3.DDR3_RAS_B}
assign_port {ddr_dqs_p[7:0]} -trace {DDR3.DDR3_DQS_P[7:0]}
assign_port {ddr_dqs_n[7:0]} -trace  {DDR3.DDR3_DQS_N[7:0]}
assign_port {ddr_dq[63:0]} -trace {DDR3.DDR3_DQ[63:0]}
assign_port {ddr_dm[7:0]} -trace {DDR3.DDR3_DM[7:0]}
assign_port {ddr_ck_p[0]} -trace  {DDR3.DDR3_CK_P[0]}
assign_port {ddr_ck_n[0]} -trace {DDR3.DDR3_CK_N[0]}
assign_port {ddr_cke[0]} -trace {DDR3.DDR3_CKE[0]}
assign_port {ddr_cas_n} -trace {DDR3.DDR3_CAS_B}
assign_port {ddr_ba[2:0]} -trace {DDR3.DDR3_BA[2:0]}
assign_port {ddr_addr[15:0]} -trace  {DDR3.DDR3_A[15:0]}
assign_port {refclk_to_ddr3} -trace {DDR3.REF_CLK}
```

Example: GPIO_HT3

For this daughter board, you cannot specify discrete GPIO signals as top I/Os.

```
board_system_create -interconnect -manual GPIO_HT3 -name gpio_ht3
-connector {FB1.A23}
```

Example: HAPS-ECDB Clock Hub

Even if you do not use all available HAPS-ECDB connectors, you must initially identify each connector in the TSS file. Specify a value of 0 as a placeholder as shown in the example.

You cannot chain HAPS-ECDB units, but you can specify a single clock-distribution board. For a more complete example, refer to the *HAPS External Clock Distribution Board Reference Manual* on HAPS SupportNet.

```
board_system_create -interconnect -manual ECDB -name ecdb_mod
-connector {0 0 0 0 0 0 0 0 0 0 0 0 }

board_system_configure -connect
{ecdb_mod.<ecdb_source_connector>.<FPGA>.<clk_direction}
```

Example: LAB_HT3 Daughter Boards

Some daughter boards have top and bottom connectors. In this case, even if you do not use one of the connections, you must define them in the *tss* file. Set the unused connector to a value of 0. For example:

```
board_system_create -interconnect -manual LAB_HT3 -name lab_ht3-connector
{FB1.A3 0}
```

Feedback

Example: QSFP+ MGB Interface Cards

As with other MGB interface cards, you must add the riser card to the FPGA. Then add the MGB interface card to the riser. For QSFP interface cards, the right-angled eight-lane RISER8_MGB riser card has two connectors: J2 and J1. J1 is connected to an MGB connector on the FPGA as with other MGB interface cards. The QSFPPLUS_MGB interface card plugs into the J2 connector.

The first line of the example specifies the following:

The J1 connector of the RISER8_MGB riser card (*conn_MGB1_RISER8*) connects to FPGA A on board *fb1* through MGB slot AM1. J2 is not yet connected, so its value is zero.

The second line specifies the following:

The *QSFPPLUS_MGB* interface card (*conn_MGB1_QSFP*) is plugged into the J2 connector of the RISER8_MGB riser card (*conn_MGB1_RISER8*).

```
board_system_create -interconnect -manual RISER8_MGB
  -name conn_MGB1_RISER8 -connector {0 fb1.AM1}
board_system_create -interconnect -manual QSFPPLUS_MGB
  -name conn_MGB1_QSFP -connector {conn_MGB1_RISER8.J2}
```

The following example shows two QSFPPLUS_MGB interface card plugged into FPGA B MGB connectors BM1 and BM2:

```
# Connect MGB cards for QSFP_module0
board_system_create -interconnect -manual RISER8_MGB -name connBM1
-connector {0 fb1.BM1}
board_system_create -interconnect -manual QSFPPLUS_MGB
-name connBM1_MGB1_J2 -connector {connBM1.J2}

# Connect MGB cards for QSFP_module1
board_system_create -interconnect -manual RISER8_MGB
  -name connBM2 -connector {0 fb1.BM2}
board_system_create -interconnect -manual QSFPPLUS_MGB
  -name connBM2_MGB2_J2 -connector {connBM2.J2}
```

Example: Two PCI Express Four-Lane MGB Interface Cards

To add a PCI Express four-lane card, add a RISER1_MGB card. There are two ways to configure two PCIe and riser cards.

- Using two riser cards, one for each PCIe four-lane card.

```
# Add riser card 1 and connect J1 (edge connector in -connector
 position 3) to
 # an available connector on the FPGA: AM1 or AM2, BM1 or BM2 etc. The
 first
 # two -connector positions for J3 and J2 are zeros (unconnected)
 because
 # they are placeholders for daughter cards that have not yet been
 defined.
```

Feedback

```
board_system_create -interconnect -manual RISER1_MGB -name risercard1
  -connector {0 0 fb1.AM1}
# Connect the first PCI card through riser card 1
board_system_create -interconnect -manual PCIE4_MGB -name conn2
  -connector {risercard1.J2}
# Add riser card 2 and connect it as described for riser card 1
board_system_create -interconnect -manual RISER1_MGB -name risercard2
 -connector {0 0 fb1.AM2}
# Add PCI card 2 through riser card 2
board_system_create -interconnect -manual PCIE4_MGB -name conn3
 -connector {risercard2.J2}
```

- Using one shared riser card for both PCIe four-lane cards:

```
# Add riser card 1 and connect it to the FPGA, as described above
board_system_create -interconnect -manual RISER1_MGB -name risercard1
  -connector {0 0 fb1.AM1}
# Add PCI card 1 through riser card 1
board_system_create -interconnect -manual PCIE4_MGB -name conn2
 -connector {risercard1.J3}
# Add PCI card 2 through riser card 1
board_system_create -interconnect -manual PCIE4_MGB -name conn3
 -connector {risercard1.J2}
```

Use this method if the PCIe endpoint and controller in your design and the FPGA technology support this sharing.

Using MGB Interface Cards with HAPS-100 Modules

To use MGB interface cards with HAPS-100, a CON_CABLE_XX_MGB2 and an MGB_ADAPTER_MGB2 are required. This is an example connecting a PCIE4_MGB interface card to a HAPS-100 using an adapter and a 20 cm cable.

```
#MGB2 Sideband signals
board_system_configure -gpio {FB1.AM222}

#MGB2 Adapter with PCIE4_MGB
board_system_create -interconnect -manual CON_CABLE_20_MGB2 -name
mgb_cable1 -connector {FB1.AM222 NIL}

board_system_create -interconnect -manual MGB_ADAPTER_MGB2 -name
adapter -connector {NIL NIL mgb_cable1.J2}

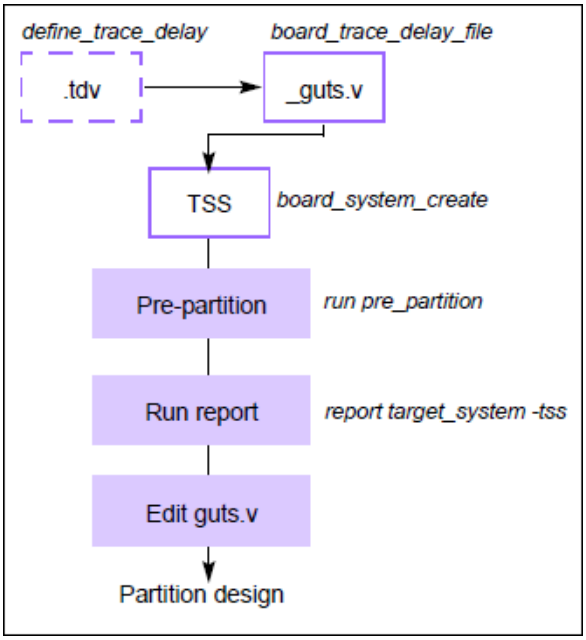
board_system_create -interconnect -manual PCIE4_MGB -name PCIE0
-connector {adapter.J3}
```

For information about the procedure to configure the MGB_ADAPTER_MGB2 with PCIe sideband signals see the *HAPS® MGB_ADAPTER_MGB2 Sideband I/O Signals Application Note*. See also *MGB_ADAPTER_MGB2 Reference Manual* and the*HAPS-100 4F Reference Manual* located on HAPS SupportNet.

Feedback

Defining Custom Daughter Board Connections

Use the following procedure to define connections for HAPS-compliant custom daughter boards. For Synopsys daughter boards, see Defining Daughter Board Connections.



1. Add custom daughter boards in the *tss* file using the `board_system_create` command.
   - Specify the custom daughter board with the `-haps_custom_db` command. This command adds the initial module, port and the trailing *endmodule* around the *_guts.v* file, making it a complete Verilog board description.

     ```
     board_system_create -haps_custom_db CUSTM_DB_HT3 -HT3 {JX1}
     -guts ../../../custom_db_ht3_guts.v
     ```

-haps_custom_db	Specify the name for the custom daughter board. Do not use connector names, like *FB1.AM1,* as the daughter board name. Avoid using a hierarchical names.
-HT3 |*-HT2* | *-MGB* | *-HSIO*	Specify the connector for the custom board: HapsTrak 3, HapsTrak II, HapsTrak MGB, or HapsTrak High Speed I/O. You must specify the list of connectors and their types.
-guts	Specify the file created in step 1 with this argument. If using a relative file path, specify it relative to the pre-partition state. The contents of this file are included in the module definition for the custom daughter board.

Feedback

You can specify multiple connectors. The following example defines two HT3 connectors (J1 JX1), one MGB connector (J2) and one HSIO connector (JH1):

```
board_system_create -haps_custom_db MY_DB -HT3 {J1 JX1} -MGB {J2}
-HSIO{JH1}  -guts ../../../my_guts.v
```

Define the connections for the daughter board with the*-interconnect* option. For example:

```
board_system_create -interconnect -manual CUSTOM_DB_HT3 -name CUST
-connector {FB1.A5}
```

2. Create a Verilog file (*_guts.v*) that defines the connections for the daughter board.

   The *guts.v file* contains the connection model for the daughter board and must define a black box and pin connections. See Example: _guts.v File for an example. This file is used together with the `board_system_create -haps_custom_db` command (described in the previous step) to provide a complete board definition.

3. Optionally, create a file that defines trace delays for the board module and specify it with the *board_trace_delay* file attribute:

   - Create the Tcl *.tdv* trace delay file when timing accuracy is important. This file defines trace delays for the custom daughter board, as shown in the following example:

     ```
     start_board_delay CUSTOM_DB_HT3
     define_trace_delay  -terminal  {t:COMP.TCK} -delay 5
     define_trace_delay  -terminal  {t:COMP.TDO} -delay 5
     define_trace_delay  -terminal  {t:COMP.DATABUS[0]} -delay 5
     define_trace_delay  -terminal  {t:COMP.DATABUS[1]} -delay 5
     define_trace_delay  -terminal  {t:COMP.DATABUS[2]} -delay 5
     end_board_delay
     ```

     Use `start_board_delay` and `end_board_delay` commands to enclose the trace delay information for a daughter board.

     Specify the delays for the custom daughter board traces using the `define_trace_delay` command. Use the *-terminal* argument and a prefix to identify one of the trace terminal pairs. Use the *b:* prefix for terminal ports (trace terminals inside the board definition). Use the *t:* prefix for terminal pins (instances connected to the outside world). Specify the delay (*-delay* in nanoseconds.

   - In the *guts.v*file, include a pointer to the *.tdv* file from the *CUSTOM_DB_HT3* module, using the *board_trace_delay* file attribute:

     ```
     /*synthesis board_trace_delay_file = "path" */
     ```

     Include the quotation marks and the = sign as shown, and use an absolute path.

Feedback

4. Rename board traces or alias them to define actual pin locations, using the *syn_trace_attr* attribute in the *guts.v* file:
   - Run pre-partition.
   - Use the *report target_system -tss* command to list the new daughter board bins and traces. This is needed to define the pin locations. See Checking the Target Specification (TSS) for more information about using this report.
   - Edit the *_guts.v* file and rename or alias trace names, using the *syn_trace_attr* attribute.
5. Continue with the partitioning process (*run partition* and so on) as usual.

Example: _guts.v File

This is an example of a Verilog file that defines the connections for a custom daughter board. The TSS *board_system_create -haps_custom_db* command combine the information in this file and creates a complete Verilog module definition. This information is shown as comments, and includes the initial module definition and the trailing *endmodule*.

```
////////////////////////////////////////////////////////////////
// Port declaration added by TSS board_system_create
// -haps_custom_db definition
//
// module CUSTOM_DB_HT3 (
//   inout [13:0] JX1_A,
//   inout [13:0] JX1_B,
//   inout [13:0] JX1_C,
//   inout [13:0] JX1_D,
//   inout        JX1_VRN,
//   inout        JX1_VRP
// );
////////////////////////////////////////////////////////////////
// Attributes for the main module, CUSTOM_DB_HT3
/* synthesis syn_partition = "board" */
/* synthesis board_trace_delay_file = "../CUSTOM_DB_HT3.tdv" */
/* synthesis syn_noprune = 1 */
// Black box signal wires and connections to HT3 ports, defined
// inside main module
wire      TCK_wire     = JX1_A[0];
wire      TDO_wire     = JX1_B[9];
wire[2:0] DATABUS_wire = {JX1_B[3],JX1_B[2],JX1_D[10]};
// Interface black box instance
CUSTOM_DB_HT3_BBOX COMP (
 .TCK     (TCK_wire),
 .TDO     (TDO_wire),
 .DATABUS (DATABUS_wire)
);
// Trace attributes
syn_trace_attr #(1,3,"syn_noprune=1 haps_allowed_iostandard=LVCMOS_18")
 TCK     (TCK_wire);
```

Feedback

```
syn_trace_attr #(1,3,"syn_noprune=1 haps_allowed_iostandard=LVCMOS_18")
 TDO     (TDO_wire);
syn_trace_attr #(3,3,"syn_noprune=1 haps_allowed_iostandard=LVCMOS_18")
 DATABUS (DATABUS_wire);
endmodule
// For CUSTOM_DB_HT3 custom daughter board
// Definition of black box module(s)
module CUSTOM_DB_HT3_BBOX (
 inout       TCK,
 inout       TDO,
 inout [2:0] DATABUS
);
// Attributes for black box module
/* synthesis syn_black_box */
/* synthesis syn_partition = "black_box=CUSTOM_DB_HT3_BBOX" */
/* synthesis syn_noprune = 1 */
////////////////////////////////////////////////////////////////
// Trailing endmodule added by TSS board_system_create definition
////////////////////////////////////////////////////////////////
```

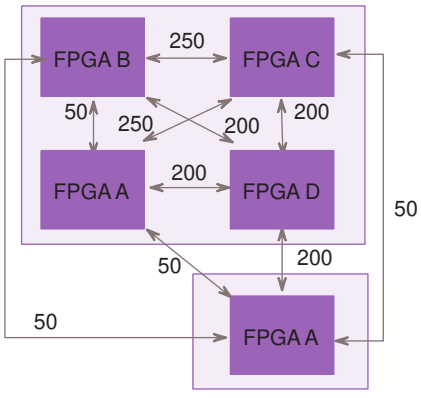
Moving from Abstract PCF to TSS: Flow Example

The example illustrates the abstract flow from initial trace group descriptions in the PCF file to the final hardware interconnect descriptions in the TSS file.

- Step 1: Define Initial Interconnect in PCF
- Step 2: Check Results After Running with Initial Interconnect Definition
- Step 3: Edit Interconnect Definition and Rerun Exploratory Partitioning
- Step 4: Transfer Final Results to TSS and Run Partitioning

Step 1: Define Initial Interconnect in PCF

This is the initial example design:



Feedback

- Define the corresponding FPGA-to-FPGA channels in the *pcf* file:

```
abstract_tss -add_trace_group {fb_1.uA fb_1.uB} -width 50
abstract_tss -add_trace_group {fb_1.uA fb_1.uC} -width 250
abstract_tss -add_trace_group {fb_1.uA fb_1.uD} -width 200
abstract_tss -add_trace_group {fb_1.uA fb_2.uA} -width 50
abstract_tss -add_trace_group {fb_1.uB fb_1.uC} -width 250
abstract_tss -add_trace_group {fb_1.uB fb_1.uD} -width 200
abstract_tss -add_trace_group {fb_1.uB fb_2.uA} -width 50
abstract_tss -add_trace_group {fb_1.uC fb_1.uD} -width 200
abstract_tss -add_trace_group {fb_1.uC fb_2.uA} -width 50
abstract_tss -add_trace_group {fb_1.uD fb_2.uA} -width 200
```

- Run partitioning.

Step 2: Check Results After Running with Initial Interconnect Definition

The Bin Usage Summary shows acceptable area utilization for individual FPGAs:

```
@S4.1 AP140 |Bin Usage Summary

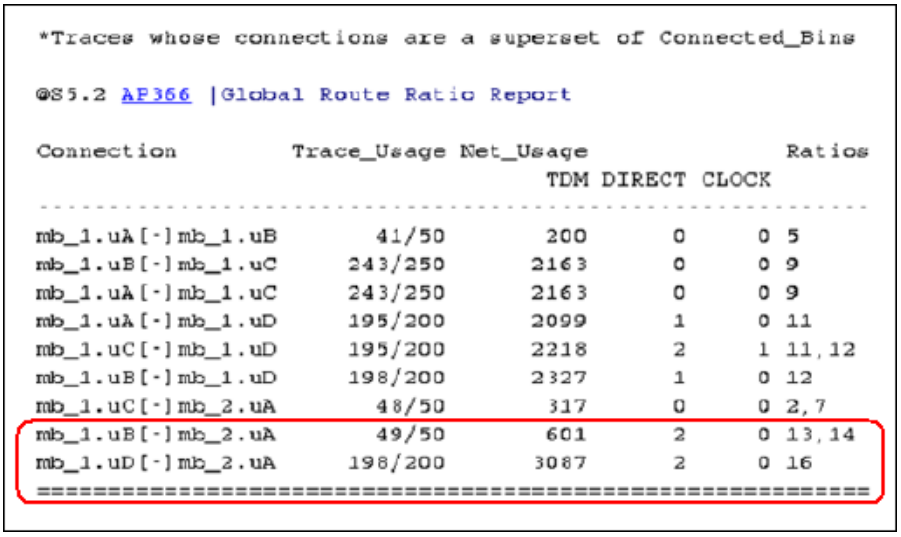
FPGA_Bin Cells Lock LUT       %  DFF       %  BRAM    %  DSP     %
-------------------------------------------------------------------------
mb_1.uD    1101       855770   69% 274421   11% 579    44% 75      3%
mb_1.uC       8       791127   64% 206801    8% 163    12% 1145   53%
mb_1.uB      29       710082   57% 173913    7% 274    21% 189     9%
mb_1.uA      16       970890   78% 300155   12% 466    35% 415    19%
mb_2.uA     214       587959   47% 318225   13% 628    48% 172     8%
=========================================================================
```

However, the Global Route Summary indicates that the maximum TDM ratio is 16:

```
@S5.4 AP267 |Global Route Summary
@N:AP367 :  | Routed 15049 Nets
@N:AP268 :  | Maximum TDM Ratio: 16
@N:AP368 :  | Feedthroughs: 0
@N:AP270 :  | Unrouted: 0
@N:AP371 :  | For global routing det
@N:AP152 :  | Time: 34
```

Feedback

Further investigation of the Global Route Ratio Report identifies the bottleneck between *fb2.uA* and *fb1.uD*:



```
*Traces whose connections are a superset of Connected_Bins

@S5.2 AP366 |Global Route Ratio Report

Connection          Trace_Usage Net_Usage              Ratios
                                      TDM DIRECT CLOCK
-------------------------------------------------------------
mb_1.uA[-]mb_1.uB        41/50       200      0     0 5
mb_1.uB[-]mb_1.uC       243/250     2163      0     0 9
mb_1.uA[-]mb_1.uC       243/250     2163      0     0 9
mb_1.uA[-]mb_1.uD       195/200     2099      1     0 11
mb_1.uC[-]mb_1.uD       195/200     2218      2     1 11,12
mb_1.uB[-]mb_1.uD       198/200     2327      1     0 12
mb_1.uC[-]mb_2.uA        48/50      317       0     0 2,7
mb_1.uB[-]mb_2.uA        49/50      601       2     0 13,14
mb_1.uD[-]mb_2.uA       198/200     3087      2     0 16
=============================================================
```

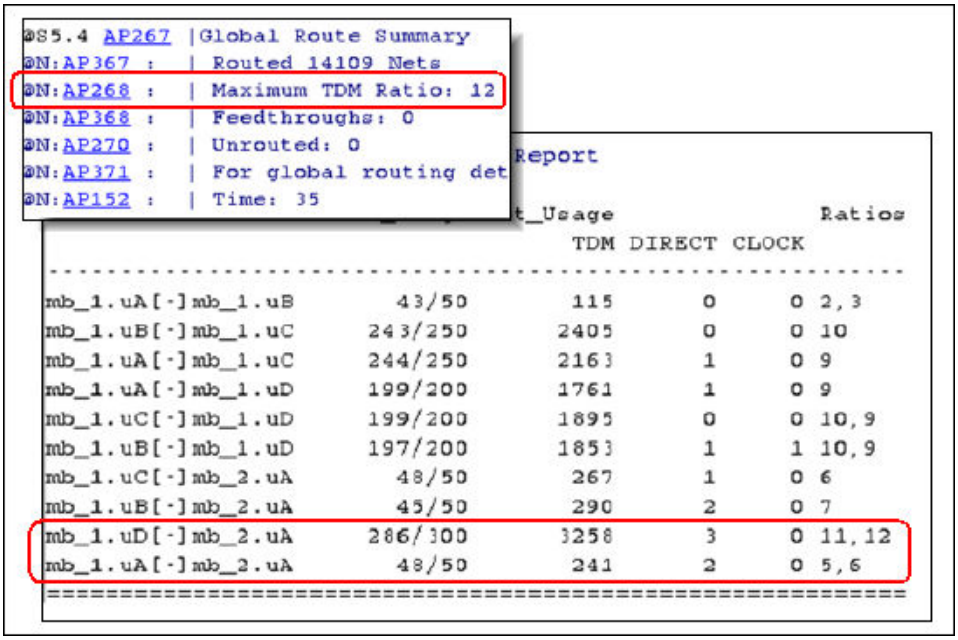
Step 3: Edit Interconnect Definition and Rerun Exploratory Partitioning

To ease the bottleneck, edit the *pcf* file to increase the interconnect capacity between *fb2.uA* and *fb1.uD* from 200 to 300:

```
abstract_tss -add_trace_group {fb_1.uD fb_2.uA} -width 300
```

Feedback

When run with the increased capacity, the maximum TDM ratio for the design goes down from 16 to 12:



Step 4: Transfer Final Results to TSS and Run Partitioning

Use the `board_system_create -interconnect` commands in the TSS file to define the hardware setup that produced the lower TDM ratio. Remove the original `abstract_tss` commands from the *pcf* file after you have a TSS definition with the hardware information. This is a sample of the TSS interconnect commands for *fb1A*:

```
board_system_create -interconnect -manual CON_CABLE_50_HT3
 -name fb1A9_fb1B1 -connector {fb_1.A9 fb_1.B1}
board_system_create -interconnect -manual CON_CABLE_50_HT3
 -name fb1A10_fb1C23 -connector {fb_1.A10 fb_1.C23}
board_system_create -interconnect -manual CON_CABLE_50_HT3
 -name fb1A11_fb1C22 -connector {fb_1.A11 fb_1.C22}
board_system_create -interconnect -manual CON_CABLE_50_HT3
 -name fb1A12_fb1C21 -connector {fb_1.A12 fb_1.C21}
board_system_create -interconnect -manual CON_CABLE_50_HT3
 -name fb1A22_fb1C12 -connector {fb_1.A22 fb_1.C12}
board_system_create -interconnect -manual CON_CABLE_50_HT3
 -name fb1A23_fb1C11 -connector {fb_1.A23 fb_1.C11}
board_system_create -interconnect -manual CON_CABLE_50_HT3
 -name fb1A13_fb1D23 -connector {fb_1.A13 fb_1.D23}
board_system_create -interconnect -manual CON_CABLE_50_HT3
 -name fb1A14_fb1D22 -connector {fb_1.A14 fb_1.D22}
board_system_create -interconnect -manual CON_CABLE_50_HT3
 -name fb1A15_fb1D21 -connector {fb_1.A15 fb_1.D21}
board_system_create -interconnect -manual CON_CABLE_50_HT3
 -name fb1A16_fb1D20 -connector {fb_1.A16 fb_1.D20}
```

Feedback

```
board_system_create -interconnect -manual CON_CABLE_50_HT3
 -name fb1A21_fb2A21 -connector {fb_1.A21 fb_2.A21}
```

Additionally, set the *tdm_control* constraint in the *pcf* file to fix the ratio and ensure that partitioning and system route are constrained. For example:

```
tdm_control -type HSTDM -max_ratio 16
```

Validating TSS Cable Connections

HSTDM refers to High Speed Time-Domain Multiplexing of signals.

In the following example, the hardware setup uses multiple hosts, where each host has superchain setups. Host 1 has two superchains, and Host 2 has three superchains. So you need to provide the host serial number CSV file to the `conspeed_hstdm` command. For this example, the CSV file is host_slno.txt.

CSV File Example

```
Host1, H000001 Host1, H000002 Host2, H000003 Host2, H000004 Host2,
H000005
```

The order of HAPS systems listed in the CSV file should match the order of systems in the TSS file. For example, FB1 should be H000001, FB2 should be H000002, and so on.

Use this procedure to validate cable connections and check channel connectivity defined in the TSS file.

1. Initialize the setup with *-init* and *-rundir* arguments to create a directory at the specified location.

   ```
   conspeed_hstdm -init -rundir <directory_path_name>
   ```

2. In the implementation phase. generate the bit files based on the defined hardware connections.

   Example:

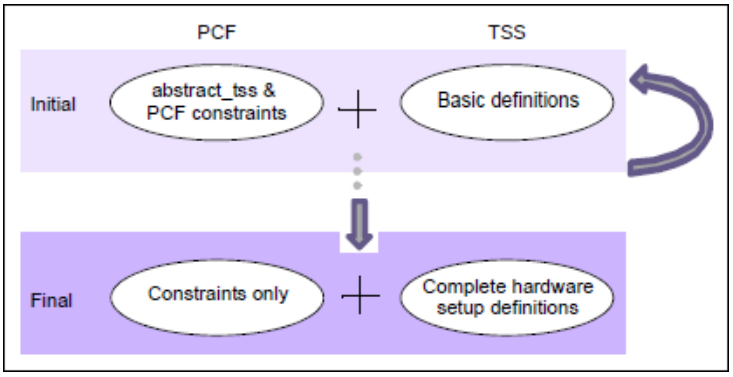
   ```
   conspeed_hstdm -technology HAPS80 -mode bidir -flow implementation
   -tss myTss.tcl -rundir <directory_path_name> -hstdm_ratio 15 -tdm_type
   HSTDM_ERD -bitrate 1400
   ```

3. In the runtime phase, run the bit files and validate the cable setup.
   - Make sure the hardware setup reflects the definitions in the TSS file.
   - Execute the `conspeed_hstdm` command from hUtils. For more information, see *hUtils* documentation in SupportNet.

Configuring Constraints in PCF Files

The Partitioning Constraints Format (PCF) file is used to configure constraints that guide the auto-partitioner during partitioning.

Partitioning is iterative, and you can use the PCF file in different ways, depending on the state of the design. It is recommended that you create multiple *pcf* files so that you can use them for different purposes in a modular fashion.



Note that the *pcf* file is used for two different purposes:

- To create abstract interfaces and interconnect schemes for initial partitioning and exploration.
- To define assignment constraints to guide actual partitioning and system routing, including constraints for bin utilization, floorplanning, hierarchy control, and assignments of cells and ports. Bins are groups of objects created by the partitioner.

This table summarizes where to find information about using the *pcf* file.

For Information About ...	See ...
• Partition exploration with the *pcf* file • Using the *pcf* file to develop specifications for the *tss* file • Using abstract *pcf* constraints	Using Abstract PCF Commands for Partition Exploration
• Getting *tss*information • Querying the design	Configuring Constraints in PCF Files
• Setting partition assignment constraints for clocks, ports, and cells • Setting assignment constraints to guide *run partition* • Manual trace assignment	Configuring PCF Constraints for Partitioning Assigning Inter-FPGA Nets to Traces

Feedback

For Information About ...	See ...
• Moving from abstract to detailed partitioning • Iterating between pre-partition and partitioning runs with the *pcf*	Moving from Abstract PCF to TSS: Flow Example Working Iteratively with PCF Assignments

Using Abstract PCF Commands for Partition Exploration

If you are in the early stages of prototyping, you experiment with various interconnect schemes before committing to one. For example, you might know you need four FPGAs and 500 ports, but not have any other design restrictions. Use *pcf* constraints to explore partition choices and determine the final target hardware configuration.

When you are exploring partitioning options, use the *pcf* file and abstract constraints to specify the interconnect for an abstract target system definition, as described here. For details about the *pcf* constraint syntax, see Partition Constraint File Tcl Commands.

1. Do a baseline partitioning run (*run partition*).
   - After the run, check the partitioning report and check the number of FPGAs and port bins.
   - Check the report (*report target_system*) for global clock names.
2. Create a *pcf* file with constraints that define the I/O interface for the FPGAs, based on the FPGA names from the initial partitioning run.
   - Get the names of the FPGA bins by checking the TSS report (Checking the Target Specification (TSS))
   - Use the `abstract_tss -add_port_bin` command to define the interface:

     ```
     abstract_tss -add_port_bin -fpga FB1.uA -name \ ABS_PORTA -width 650
      abstract_tss -add_port_bin -fpga FB1.uB -name \ ABS_PORTB -width
      650
      abstract_tss -add_port_bin -fpga FB1.uC -name \ ABS_PORTC -width
      650
      abstract_tss -add_port_bin -fpga FB1.uD -name \ ABS_PORTD -width
      650
     ```

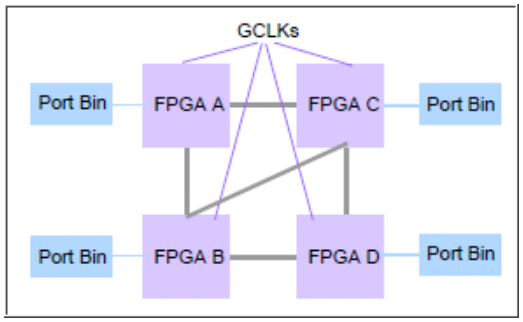
   - Set port bin sizes with the *-width* argument. You can expand the size of abstract ports beyond the connector size, to provide the tool with more freedom during this exploratory phase. The example increases the width to 650, even though one HT3 connector has just 50 pins.

     If you have existing *tra* and *prt* constraints that you want to convert to *pcf* constraints, contact Synopsys.

3. Define FPGA-to-FPGA link pairs to explore interconnect schemes, using the `abstract_tss -add_trace_group` command.

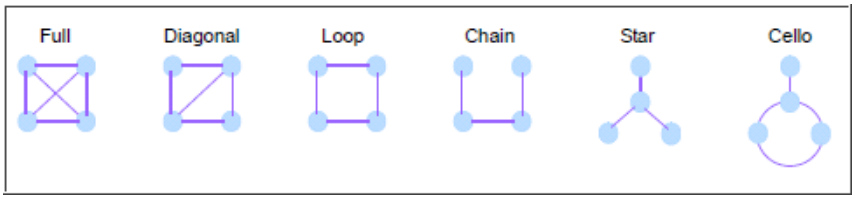
   This example shows a diagonal interconnect scheme for four FPGAs:

```
abstract_tss -add_trace_group {FB1.uA FB1.uB} -width 500
 abstract_tss -add_trace_group {FB1.uB FB1.uD} -width 500
 abstract_tss -add_trace_group {FB1.uD FB1.uC} -width 500
 abstract_tss -add_trace_group {FB1.uC FB1.uA} -width 500
 abstract_tss -add_trace_group {FB1.uC FB1.uB} -width 500
```



   Here are some guidelines for defining interconnect:

   - Target the smallest max ratio with the fewest cables
   - Explore other hardware topologies. The following figure shows the six hardware topologies possible for 4-FPGA systems. The Star and Chain configurations use the fewest cables.



   - Use inter-FPGA feedthroughs if necessary. Inter-FPGA feedthroughs are routed without pin multiplexing, and are usually faster than a pin-multiplexed route.
   - Avoid port feedthroughs.

   If your TSS file defines port bins, external bins, and interconnect like clocking architecture, use related *pcf* commands that modify the defined target system for exploration. For example, you can use *pcf* commands to assign clocks if your TSS file defines inter-FPGA clocks.

4. Iteratively run the partitioner and adjust the *pcf* constraints until you have a feasible interconnect scheme, and meet your minimum performance target.

5. You can now create a detailed *tss* that implements the abstract interconnect scheme, and use it for partitioning and system routing.

   When you switch to a detailed *tss*, the *pcf* file must not contain any abstract interconnect commands. All *pcf* constraints must be partitioning constraints related to clock assignments, cell assignments, port assignments, and so on. See the following for more information:

Setting final *pcf* constraints: clock, cell, and port assignments	Configuring PCF Constraints for Partitioning
Example illustrating the flow from *pcf* to *tss*	Moving from Abstract PCF to TSS: Flow Example
TSS file	Defining the Target System in a Detailed TSS File
Partitioning flow	Design Methodology for Partitioning

Configuring PCF Constraints for Partitioning

The PCF constraints described here are not abstract interconnect constraints used during the exploration phase (Using Abstract PCF Commands for Partition Exploration), but specific constraints for port, clock, and cell assignments. Not all the *pcf* constraints are described; for a complete lit of all the *pcf* constraints and their syntax, refer to Partition Constraint File Tcl Commands in the *Command Reference Manual*.

1. Create or update the *pcf* file from the pre-partition, partition, or system route states.

   Make sure to update the *pcf* file after partition, so that you can use the results of the partitioning run to define more accurate constraints.

   It is recommended that you use multiple PCF files for different constraints. See Using Multiple PCF Files.

2. Enter the constraints you need.

Global setup	Defining Global Information in the PCF
Port assignments	Assigning Ports in the PCF
Cell assignments	Assigning Cells in the PCF
Clock assignments	Assigning Clocks in the PCF

Feedback

Nets for MGBLINKs Assigning Nets to MGBLINKs

3. Check the partitioning report and partitioning log files for results. See Analyzing Partition Result Files for more information.

TDM on Asynchronous Nets

You can add TDM on an asynchronous net using the following PCF switches:

```
partition_optimization_control -relax_async_constraints 1
report_control -async_nets 1
```

The tool automatically detects the asynchronous nets in which the source and destination flip-flops are in different clock domains. To use these options set the license_feature, *FpgaInternalBeta*. See Target System Definition for more information.

Using Multiple PCF Files

It is recommended that you create multiple *pcf* files with different constraints so that you can use them in a modular fashion. Here are some suggested *pcf* files you might use at different points in the design cycle:

abstract.pcf	FPGA definitions and interconnect schemes defined with *abstract_tss* commands for design exploration. See Using Abstract PCF Commands for Partition Exploration.
setup.pcf	Global prototype information, like bin utilization, TDM controls, and cell dissolving. See Defining Global Information in the PCF.
ports.pcf	Port assignments. Initially, specify them as clusters or assign them to bins. Later, assign them to pins or traces. See Assigning Ports in the PCF.
clocks.pcf	Clock assignments. See Assigning Clocks in the PCF.
floorplan.pcf	Assignment of a few key cells, specific bin assignments and set assignments. See Assigning Cells in the PCF.
assignments.pcf	Assignment of all cells when you want to lock down a partition. These assignments can be exported out of the tool. See Assigning Cells in the PCF.

Defining Global Information in the PCF

This procedure shows you how to define global setup information using PCF constraints. Global information includes specifications for bin utilization, TDM controls, and cell dissolving, and it is recommended that you put this kind of information in *setup.pcf*. The

Feedback

following procedure describes some of the commands, but is not an exhaustive list of the options. Consult Partition Constraint File Tcl Commands for a complete list.

1. Define resource availability and set maximum utilization constraints with the `bin_utilization` command:
   - Check resources with the `design_list_cells -filter` command.
   - To define bin utilization as a ratio, use `bin_utilization -resource_ratio`:

     ```
     # Use 80% of the available resources for all bins bin_utilization
      -all_bins -resource_ratio {All 0.8 }

     # Use 90% of LUT resources for FPGA bin FB1.uA bin_utilization -bin
      FB1.uA -resource_ratio {LUT 0.9}
     ```

   - To define bin utilization as a number value, use `bin_utilization -resource`:

     ```
     # Use up to 100,000 LUT resources for FPGA bin FB1.uB
     bin_utilization -bin FB1.uB -resource {LUT 100000}
     ```

   For syntax details, see bin_utilization.
2. For controls on individual cells use `cell_attribute -dissolve`, as described in Assigning Cells in the PCF.
3. Set options for TDM with `tdm_control` commands.

Assigning Ports in the PCF

Initially, specify ports as clusters or assign them to bins. As the design progresses, assign them to pins or traces. You can put this information in a separate file, *ports.pcf*. The following procedure describes some of the commands, but is not an exhaustive list of the options. Consult Partition Constraint File Tcl Commands in the *Command Reference* for a complete list.

1. Assign top-level ports to legal port bins with the `assign_port` command.

   You can also use this command to specify a particular pin for a top-level port.
2. Assign clock and reset ports to special bins.
3. Group ports for logical interfaces together with the `cluster_port` command.

   The following example defines ports that must be kept together because they define an external interface:

   ```
   cluster port {addr[12:0] data[15:0] r_not_w strobe}
   ```
4. Optionally, assign nets with the `cluster_net` command.

Feedback

Use net assignments to group nets if you do not get the results you want with just cell and port assignments. With this command, the partitioner keeps the drivers of clustered nets together. You can assign a name to the cluster; the name can then be used in subsequent net commands to refer to the whole group.

5. Use `net_attribute` commands to assign nets to functional trace groups.

   This constraint specifies attributes on nets, which the router later routes automatically, using functional group traces that match the attributes. To manually route nets through specified bins, use the **`net_route`** command. The following of the *net_attribute* constraint:

   - Use the constraint to specify nets for later automatic assignment to MGBLINKs by the router.

     ```
     net_attribute {netNameList} -function MGT
     ```

     For details about manual assignment, see Assigning Nets to MGBLINKs.

6. Specify routing constraints for the partitioner and system router.

   - Use `trace_group_attribute` to define trace groups.
   - Remove traces from consideration by the partitioner or router with `reserve_trace`.

7. If you need to manually assign inter-FPGA nets to traces, use the `net_route` command for nets, and the `assign_port` command for daughter boards.

   It is recommended that you use the automatic partitioner for trace assignment. For details and examples of manual assignment, see Assigning Inter-FPGA Nets to Traces.

Assigning Cells in the PCF

This procedure shows you how to floorplan your design, using PCF constraints to guide logic assignments for ports and key cells. It is recommended that you put this kind of information in *floorplan.pcf* during the initial stages, and then *assignments.pcf* as the design is finalized and more detailed constraints added. You can export *assignments.pcf*.

The following procedure describes some of the commands, but is not an exhaustive list of the options. Consult Partition Constraint File Tcl Commands for a complete list.

1. Restrict logic assignment to specific FPGAs by locking them with the *bin_attribute -locked* constraint.

   No logic is assigned to locked FPGA bins. For example, the following command only assigns logic to FPGAs A and B, because C and D are locked:

   ```
   bin_attribute -locked fb0.uC
   bin_attribute -locked fb0.uD
   ```

Feedback

2. Assign legal bin locations for key cells with the `assign_cell` command.

   You can assign a cell to multiple bins. However, if a bin is locked, you can only specify a single bin. The tool can cluster assigned cells with other cells that have the same assignment. Some `assign_cell` commands create dissolve commands as a side effect.

   - For large functional blocks like CPUs that span multiple FPGAs, define a multi-FPGA region for the group of cells. The following examples confine the core cells to three FPGA bins each:

     ```
     # floorplan.pcf
     assign_cell {dut_mod_inst.sub0.core0} {FB1.uA FB1.uB FB1.uC}
     assign_cell {dut_mod_inst.sub1.core1} {FB2.uB FB2.uC FB2.uD}
     ```

   - Only assign black boxes to black box bins.

3. Define logic that must be kept together with the `cluster` command.

   Use these commands to group cells that should stay together for timing reasons, or to keep logical interfaces and associated logic together. The tool assigns clusters to the same FPGA or port bin.

   - Define hard cell clusters that must be assigned to the same bin:

     ```
     cluster {dut_mod_inst.sub0.blkA dut_mod_inst.sub0.blkB}
     ```

   - Use the *-soft* argument to indicate clusters that can be split if this leads to better results:

     ```
     cluster {dut_mod_inst.sub1.blkC dut_mod_inst.sub1.blkD} -soft
     ```

4. You can specify dissolve controls on individual cells. In general, leave the default automatic setting for cells (`cell_attribute -dissolve auto`), where the tool chooses which design hierarchies to dissolve when a block exceeds the FPGA bin resources. Use other arguments to override the default.

   Specify hierarchical logic that can be dissolved and freely assigned to different bins with `cell_attribute { list of cells} -dissolve 1` commands. Use `report schematic` to identify modules that contain mainly feedthroughs and add *dissolve* constraints for them. You can also use this command for blocks that are primarily made up of random logic.

   If the automatic partitioner takes many hours to run, this could be because too many cells are being dissolved. You can set `cell_attribute -dissolve` to *0* to mark self-contained interfaces that you want to keep intact.

   ```
   # Do not dissolve this cell cell_attribute {asdf.basdf.basdf}
   -dissolve 0
   ```

Feedback

- Check the partitioning report to see which cells were dissolved. For details of the dissolving process, see Auto Hierarchy Dissolve Controls. Check the *Bin Usage Summary* section of the partitioning report for the cell count, and the *Dissolving large cells and must-dissolve cells* section to identify which cells are being over-dissolved. Then apply `cell_attribute -dissolve 0` to these cells.

5. Replicate clock buffers in each FPGA with the `cell_attribute -auto_replicate 1` command.
   - To automatically replicate cells, use the `cell_attribute {list of cells} -auto_replicate 1` command.

     Although it is best to avoid replicating MMCMs because replication could result in cut clocks and cause hold violations and hardware failures, you might have to do this if MMCM clocks drive most of your partitioned logic. In this case you can manually replicate the MMCMs based on your knowledge of the design, but also make sure to connect MMCM-generated clocks to a GCLK that connects to all the FPGAs in the design. This helps avoid phase differences in clocks that connect to all the FPGAs in the design. For an example, refer to SolvNet article 2345509.
   - Use the *-force* option to override the default and ensure that the cell is replicated even if it does not have connections in the specified bin after partitioning.
6. Customize what is reported in the log file with the `report_control` command.

Assigning Clocks in the PCF

This is a generic procedure that shows you how to use PCF constraints to add clock and reset information to your design; specifics vary with the hardware used.

It is recommended that you use a separate file for clock constraints called *clocks.pcf*. The following procedure describes some of the commands, but is not an exhaustive list of the options. Consult Partition Constraint File Tcl Commands in the *Command Reference* for a complete list.

Be aware that there are some clock PCF command differences for HAPS-100, see Partition Constraint File Tcl Commands in the *Command Reference*

1. Get global clock and other names.
   - Do an initial partitioning run.
   - Use the `report target_system -tss` and `view report` commands to generate reports. Check that bins were created according to the commands you entered.
   - Get the names for global clock nets, resets, and clock bins from this report. Use these names when you declare the system clocks in the next step.
2. Assign clock and reset ports to special bins with the `assign_port` command.

Feedback

- Assign global clocks to the appropriate bins with the `assign_port` command. Global clocks can be driven by different sources: PLL, FPGA, or external. In the example, the `assign_port` commands assign the global clock to the PLL source driver bins, and the `assign global_net` commands assign the global clock traces.

```
# GCLK1
net_attribute gclk1_p -function GCLK -differential |
 gclk1_n -is_clock 1
# Assign global clock to the pll1 bin
assign_port gclk1_p FB1.clk.pll1
assign port gclk1_n FB1.clk.pll1
# Assign global clock trace
assign global_net gclk1_p FB1.GCLK1

# GCLK5
net_attribute clk25_p -function GCLK -differential \
 clk25_n -is_clock 1
# Assign global clock to the pll2 bin
assign_port clk25_p FB1.clk.pll2
assign port clk25_n FB1.clk.pll2
# Assign global trace
assign global_net clk25_p FB1.GCLK5

# GCLK8
assign_port gclk8 FB1.clk.bd_clk8
assign global_net gclk8 FB1.GCLK8
```

- Assign a port to the *GCLK0* bin. GCLK0 is the reserved system clock and is input to all FPGAs with a fixed frequency of 100 MHZ.
- See Assigning Internally Generated Clocks to the GCLK Network for details about assigning internally generated clocks.

3. Assign global clocks to global clock traces.
   - Find trace names for the global clocks from the reports described in the first step. This example shows that global clock bin *FB1.clk.pll1* drives global clock trace *FB1.GCLK1*:

     ```
     trace FB1.GCLK1 -function {GCLK hard} -from FB1.clk.pll1 -pins {
     ```

   - Assign traces for the known global clocks, using the identified trace names with the *assign_global_net* command. For example:

     ```
     assign_global_net xck_x1 FB1.GCLK1
     ```

   - Provide information about whether the clock has a single port or is differential, with the *net_attribute* command.

     ```
     net_attribute {xck_x1} -function GCLK -diffsingle -is_clock1
     ```

Feedback

- Assign resets to the global reset:

```
assign_port xxclr FB1.reset
 net_attribute xxclr -function RESET
```

Example 1 PCF File

```
Manual cell assignment
#####################################################################
Fuctional group and Trace assignment for external clock source
#####################################################################
#####################################################################
net_attribute {clk} -function {GCLK} -replicate 0
assign_global_net {clk} {mb1.GCLK3}
assign_port {clk} -trace {mb1.GCLK3}
assign_port {clk_out10} -trace {mb1.CLK_SRC[2]}
assign_port {clk_out5} -trace {mb1.CLK_SRC[1]}
assign_port {reset} -trace {mb1.A_USER_RESETN}
```

Assigning Nets to MGBLINKs

You can mark nets and let the tool automatically assign them to MGBLINKs, or you can assign them manually.

1. To mark nets for automatic assignment, specify the function of the nets you want to assign to MGBLINKs as *MGT*.

   ```
   net attribute netName -function MGT
   ```

   During system route, the tool automatically assigns nets marked with the *MGT* function to MGBLINKs.

2. To manually assign nets, do the following:

   - Use the `report target_system -tss` command to identify the usable MGBLINKs. Search for the traces that have the *MGT hard* function.
   - Define the net assignment with the `net_route` command, using the MGBLINK name you just identified in the *-using_trace* argument.

     ```
     net route {myNet} -from {FB1.uA} -to {FB1.uB} -using_trace
     {FB1.MGBLINK_BI2_TO_AI2_FIXED_P[7:0]}
     ```

Assigning Inter-FPGA Nets to Traces

It is recommended that you use the automatic partitioner for trace assignment. However, if the partitioning results are not ideal, you can manually assign nets to traces. For manual trace assignment, first identify the trace names and then assign the appropriate inter-FPGA nets to them, as described in the following procedure.

Feedback

1. Define your hardware setup in an initial TSS file.

   You must do this to determine the exact trace names, which could differ from the trace names defined in the hardware manuals, and can vary according to the connection setup. Once the setup is defined, the tool can report the correct names, based on the daughter boards defined and how they are connected in your setup.

   For example, the setup could have pass-through riser cards, MGBs with PCIe connections, or MGBs with SATA connections, and these differences affect the trace names. If you have more than one card and you give them differentiating names in the TSS, this can affect the trace names as well.

2. Find the trace names.

   Trace names are listed in the system route report, if you have run that phase. Check the *Trace Assignments* section of the system route report.

3. After finding the trace names, use them in net assignments, made with the `net_route` PCF command. Define them with the *-using_trace* argument. For example:

   ```
   net_route {a_r[4]} -from {fb1.uA} -to {fb1.uB} -using_trace
   {fb1_A3_B[4]}
   net_route {a_r[3]} -from {fb1.uA} -to {fb1.uB} -using_trace
   {fb1_A3_B[3]}
   net_route {a_r[2]} -from {fb1.uA} -to {fb1.uB} -using_trace
   {fb1_A3_B[2]}
   net_route {a_r[1]} -from {fb1.uA} -to {fb1.uB} -using_trace
   {fb1_A3_B[1]}
   net_route {a_r[0]} -from {fb1.uA} -to {fb1.uB} -using_trace
   {fb1_A3_B[0]}
   ```

   See net_route for more information about the command syntax.

4. For daughter board trace assignments, use `assign_port` commands in the *pcf* file instead of `net_route`.

5. To manually specify that TDM not be used between certain FPGAs, set the *trace_group_attribute* to *DIRECT*.

   Use this when you need more control over nets; for example when you do not want to use TDM for low slack nets. *DIRECT* changes the default assignment, and does not use TDM between the specified FPGAs.

6. Run system route (`run system_route`) after making the assignments.

   Check the actual assignments in the system route report. The following command indicates which traces were assigned.

   ```
   assign_trace a_r[4] fb1_A3_B[4]
   ```

Feedback

Working Iteratively with PCF Assignments

The assignment of partitioning constraints is iterative. It is recommended that you use multiple *pcf* constraint files for different phases and purposes. See Configuring PCF Constraints for Partitioning for a list of suggested *pcf* files.

Use the method to work on constraint assignments for partitioning:

1. Partition the design and analyze the reports generated.

   Move logic as permitted by the design, so as to optimize feedthroughs between partitions.

2. Once the feedthroughs have been fixed, assign traces.

3. Assign clocks last.

   - Check the target system report after partitioning to get the global clock nets, resets, and clock bin names.
   - Use these names when you declare the system clocks and assign them to special bins. For example:

```
# GCLK1
# Clocks driven from PLL1 PLL bin:
net_attribute gclk1_p -function GCLK -differential |
 gclk1_n -is_clock 1
assign_port gclk1_p FB1.clk.pll1
assign port gclk1_n FB1.clk.pll1
# Assign trace
assign global_net gclk1_p FB1.GCLK1
# GCLK5
# Clocks driven from PLL2 PLL bin:
net_attribute clk25_p -function GCLK -differential \
 clk25_n -is_clock 1
assign_port clk25_p FB1.clk.pll2
assign port clk25_n FB1.clk.pll2
# Assign trace
assign global_net clk25_p FB1.GCLK5
# GCLK8
# Clocks driven from FPGA:
assign_port gclk8 FB1.clk.bd_clk8
assign global_net gclk8 FB1.GCLK8
```

Working with HAPS Systems

This section contains some specifics about working with HAPS systems and specifying them in the TSS and PCF files. For comprehensive information refer to the appropriate hardware documentation.

- Working with HAPS Global Clocks

Use the following details in conjunction with the information on using the TSS and PCF files (Defining the System in a TSS File and Configuring Constraints in PCF Files).

Working with HAPS Global Clocks

The global clock (GCLK) network is distributed to all the FPGAs on a HAPS system. This low-skew, built-in distribution network is used to connect the clock sources with the ports or nets in the user design. HAPS systems have twelve usable global clocks and one fixed-frequency system clock, GCLK0.

GCLK0	GCLK0 is a fixed 100 MHhz clock and feeds all the FPGAs. Do not use it to drive the user design. It is reserved for HSTDM, UMRBus, CAPIMs, and system IP like GPIO. GCLK0 is also distributed on all HapsTrak® 3 (HT3) connectors. Define GCLK0 when modules interface with the UMRBus. This example defines GCLK0 as driving the GPIO IP. TSS: *board_system_configure -clock FB1.GCLK0 100Mhz* PCF: *net_attribute gpio_clk -function GCLK0_100MHz -is_clock 1 -diffsingle assign_global_net {gpio_clk} FB1.GCLK0*
GCLK1 - GCLK12	GCLK1 - GCLK12 also feed all the FPGAs. Use them to connect the clock source to the user design, through TSS and PCF definitions. The sources for these clocks can be PLL (up to 6 generated clocks), FPGA (up to 6 generated clocks) or external. GCLK1 - GCLK12 do not have a fixed frequency. For details, see xxx and Assigning Internally Generated Clocks to the GCLK Network. These clocks are also distributed on HT3 connectors 1-12, with one GCLK per connector.

See these topics for more information:

- Assigning PLL Clock Sources to the GCLK Network
- Assigning Internally Generated Clocks to the GCLK Network

- Assigning and Configuring Clocks
- Minimizing Clock Skew

Assigning PLL Clock Sources to the GCLK Network

A phase-locked loop (PLL) is a common clock source for design clocks with user-configurable frequencies. HAPS systems provide two PLLs with three programmable outputs each. You can use them to feed the design clocks through the HAPS global clock distribution network (GCLKs).

1. In the *tss* file, define the clock.
   - Set the selected GCLK to source the input from the PLL:

     ```
     board_system_configure -clock fb1.GCLK1 -pll
     ```

     This command identifies a clock source for GCLK1: *pll*. Do not use GCLK0 to drive a design clock. Consult the documentation for the HAPS system you are using for specifics of the clock distribution network and the exact PLL clock source for the GCLK.
   - Define the clock frequency for the clock source with the *-frequency* option:

     ```
     board_system_configure -pll FB1.PLL1_1 -frequency 10000
     ```

2. In the *pcf* file, connect the clock source to the design clock:

   ```
   net_attribute {designClk1} -function GCLK -diffsingle -is_clock 1
   assign_global_net {designClk1} fb1.GCLK1
   ```

   This command connects the global clock (GCLK1) to the design clock, thus connecting the clock source to the user design.
3. Before running the design, use FDC constraints to declare clocks and their frequencies for synthesis.
4. Check that the constraints and definitions have been properly applied.
   - After pre-partitioning, check the *Clock Settings* section to make sure that the clocks have been defined correctly.
   - Run `report constraint_check` to check that there are no errors with the FDC constraints.
   - Check the timing report after system generate and make sure that all the clocks are correctly declared.

Feedback

Assigning Internally Generated Clocks to the GCLK Network

FPGA or internal clocks are clocks that are generated by the user design. Using the global clock network to distribute them across FPGAs ensures that they are phase-aligned at the FPGA boundaries. Do the following to assign an user-generated clock (FPGA clock) to a GCLK.

1. In the *tss* file, use `board_system_configure` to configure your selected GCLK to source the input from the FPGA. For example:

   ```
   board_system_configure -clockfb.GCLK2 FPGA
   ```

   This command binds the clock source (FPGA) to GCLK2.

2. Define the functional group of internally generated clocks as GCLKs in the *pcf* file:

   ```
   net_attribute {clk_gen} -function GCLK -diffsingle -is_clock 1
   ```

3. Assign the internally generated clock to GCLK in the *pcf* file:

   ```
   assign_global_net {clk_gen} fb.GCLK2
   ```

   This command connects the global clock (GCLK2) to the internally generated clock nets, thus connecting the clock source to the user design.

4. Run checks after system generate.

   - In the Clock Relationships section of the system generate report, check that the user-generated clock or derived clock is reported.
   - Check the pin locations to make sure that GCLK is assigned to the internally generated clock.

     ```
     define_attribute {p:clk_gen_0} syn_loc {AB41}
      define_attribute {p:clk_gen} syn_loc {AC41}
      define_attribute {p:clk_gen} .certify_trace_name {fb.GCLK2}
     ```

5. Use FDC constraints to declare clocks and their frequencies for synthesis.

   This example specifies a clock divider that increases the source clock period by 16x.

   ```
   create_clock -name clk_pll [get_ports clk_pll] -period 10
   create_generated_clock -name clk_gen [get_nets {clk_gen}]
    -source [get_ports clk_pll]  -divide_by 16
   ```

   If you set this constraint before system generate, the report shows the clock as a generated clock (*clk_gen*) instead of a concatenated name.

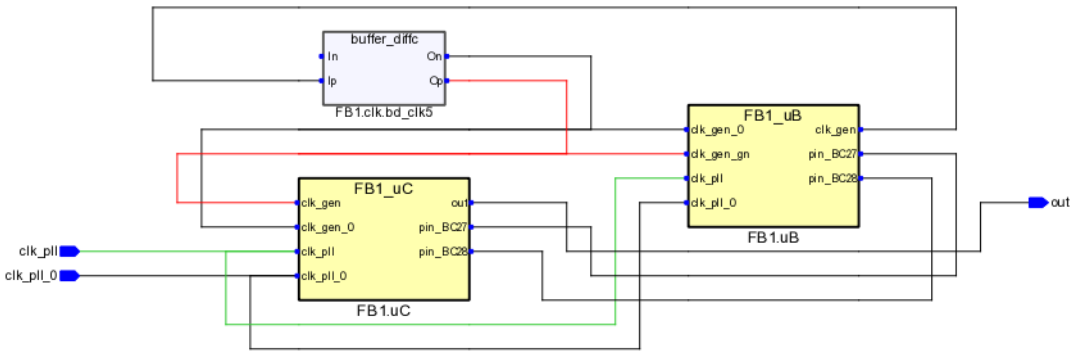
6. To distribute a generated clock to other FPGAs, use the GCLK network.

- Define the generated clock using FDC constraints.
- Check the hardware manuals to find out which GCLKs you can use to drive the clocks from a particular FPGA. For example, to drive a clock from FPGA B to FPGA C, you can use GCLK5, GCLK11, GCLK56, or GCLK12. Set up the TSS configuration accordingly. This statement declares GCLK5 on FPGA B as an FPGA-sourced clock:

  ```
  board_system_configure -clock  FB1.GCLK5 fpga
  ```

- Connect the generated clock to the GCLK you just defined, using PCF constraints. The generated clock now goes from FPGA B through GCLK5 and the global clock network to FPGA C.

  ```
  net_attribute clk_gen -function GCLK
  assign_global_net {clk_gen} {FB1.GCLK5}
  ```



Assigning and Configuring Clocks

1. Identify trace names for assignment, by following these steps:
   - After an initial partitioning run, specify the *report target_system -tss fileName* command to list board bins and traces.
   - Export the report.
   - Open the report and use these trace names for assignment. See Checking the Target Specification (TSS) for more information about using this report.

2. To assign specific clocks to a trace, use PCF *assign_global_net* constraints, as shown in the excerpt shown below:
   - The PCF `assign_global_net` command must be used directly with a trace name, which you find using the procedure described in step 1. For example:

     ```
     net_attribute {GCLK3_P} -function GCLK -differential
      GCLK3_N -is_clock 1
     assign_global_net {GCLK3_P} {fb1.GCLK3}
     ```

   - Configure the clock with a `board_system_configure -clock` command in the TSS file.
3. If you do not care about specific clock assignments, follow these steps to let the tool make the assignments:
   - Use `assign_port` syntax like this in the PCF file:

     ```
     net_attribute {GCLK3_P} -function GCLK -differential
      GCLK3_N -is_clock 1
     assign_port GCLK3_P fb1.GCLK3
     ```

   - Configure the clock with a `board_system_configure -clock` command in the TSS file.
4. To handle differential clocks, follow these instructions.
   - Use pad cells like IBUFDS, IBUFGDS, or OBUFDS for differential signals, because differential signals are on the pads and outside the FPGA, not inside the FPGA. Everything inside the FPGA is single-ended. Differential pad cells convert the differential signals to single-ended signals inside the FPGA.

     Although you can use MMCMs with differential or single-ended clocks outside the FPGA, remember that the MMCMs are internal to the FPGA and do not know about differential signals outside the FPGA.

     ```
     assign_global_net clkname_p globalClkTrace net_attribute -clkname
     -function {GCLK} -diffsingle clkname_n -is_clock 1
     ```

   - If you have one clock feeding a differential buffer, use `assign_global_net` syntax for the clock:
   - For two clocks feeding a differential buffer, use `assign_port` syntax:

     ```
     assign_port clkname_p clockbin assign_port clkname_n clockbin
     net_attribute -clkname -function {GCLK} -differential clkname_n  -
     is_clock 1
     ```

5. If the design uses GCLK0, make sure to follow the recommendations described in Working with GCLK0.

Feedback

Minimizing Clock Skew

Partitioning can cause clock nets to cross FPGAs; the crossing clocks are also called cut clocks. These clocks cause clock skew, which can cause the design to fail on the board.

Do the following to avoid cut clocks:

1. Connect the internal clock net to the HAPS global clock network using a *pcf* constraint.

   You do not need to edit the RTL. The following example assigns the clock net to GCLK7.

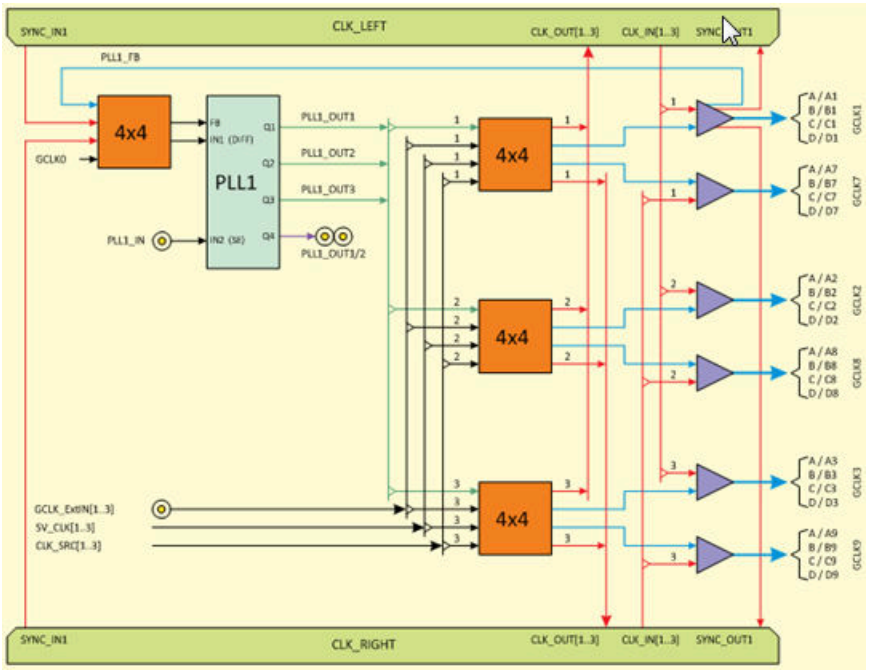
   ```
   net_attribute clk_out -function GCLK -diffsingle assign_global_net
   clk_out fb1.GCLK7
   ```

2. Connect the global clock back to all the FPGAs.

   Using the global clock network prevents the clock nets from crossing FPGA boundaries. The tool also replicates the clock-gating circuit on all the FPGAs.

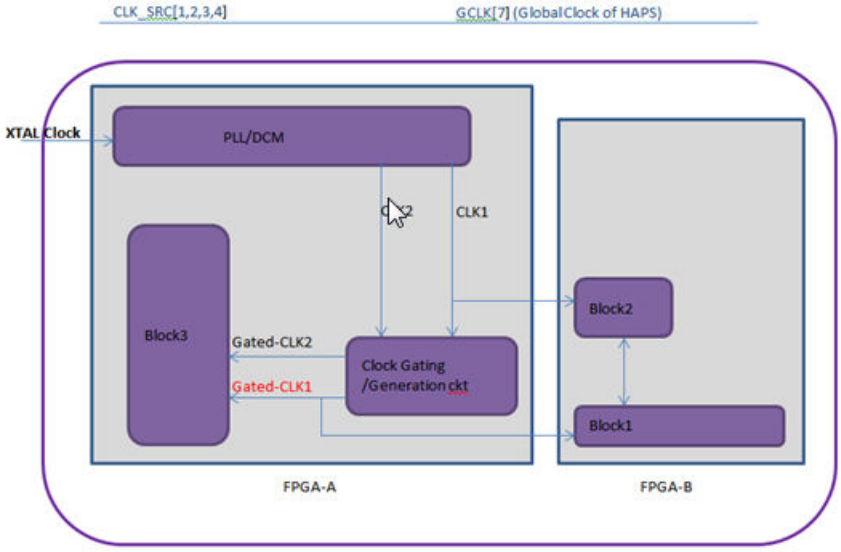
Examples

The following example shows a clocking scheme with a single PLL. Any of the GCLKs can be used in the FPGA design. GCLK0 is fixed at 100 MHz and the rest are PLL-generated clocks with configurable frequencies. The FPGA connected to the daughter board inputs the clock and redistribute it to the other FPGAs through a global clock net.

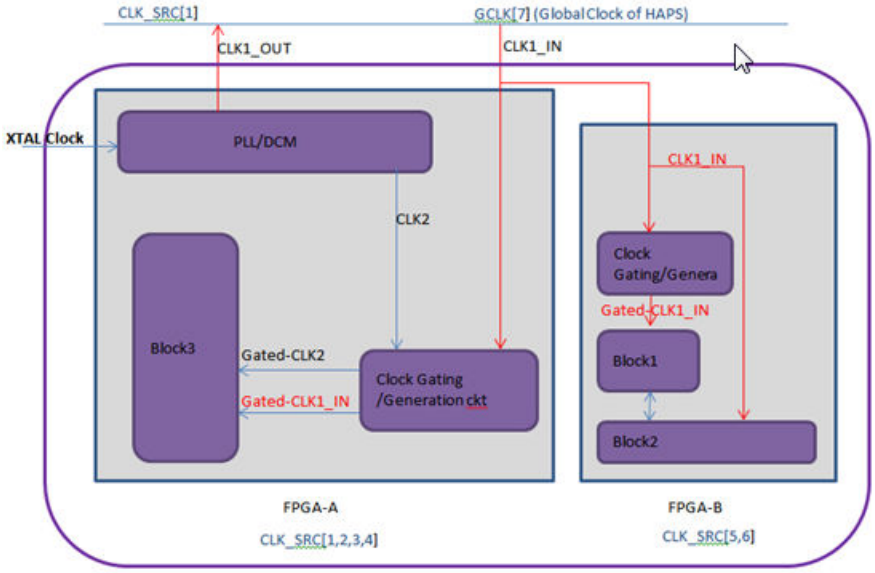


Feedback

After partitioning, the design has two cut clocks which cross the FPGA boundary: *CLK1* and *Gated-CLK1*.



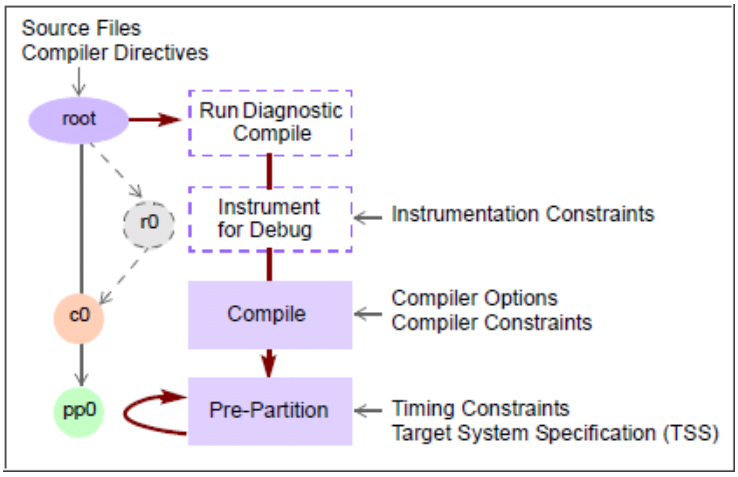
To avoid cut clocks in this case, connect CLK1 to *CLK_SRC[1]* of FPGA A, Then connect it back to all the FPGAs using a pcf constraint to assign it to the global clock GCLK[7]. The example shows this scenario, with the *clk_out* clock generated from the *CLOCK_GENERATION* block connected to the HAPS global clock network.



Feedback

Using run pre_partition to Define Partitions

After you have set up your design and the files are ready (Configuring Files for Partitioning), you can run the preparatory implementation steps before you partition the design. During pre-partitioning, the tool reads target system specifications (TSS), estimates area and applies constraints, including UPF constraints. If the design was instrumented, it also reads the *idc* file and inserts IICE™ logic. The following figure summarizes these steps and the corresponding database states generated. Dotted lines indicate optional steps.



Note:

Before running run `pre_partition`, ensure that the design has been compiled using the run compile command. The compile step generates the netlist and prepares the design database for partitioning. Without a compiled state, pre-partitioning and area estimation cannot proceed.

1. Compile the design.
   - Optionally, run the diagnostic compiler (**`report rtl_diagnostics`**) and make sure your RTL is clean.
   - If you want to instrument the design, use the **`run pre_instrument`** command to add RTL debug instrumentation.
   - Compile the design (**`run compile`**).
2. Create a board specification file, as described in Defining the Target System in a Detailed TSS File.

   If you are exploring partitioning options, use a basic TSS file for quicker turnaround times. See Exploring Partition Choices with a Basic TSS File for information about this file.

3. Set constraints and options as needed in *fdc* and Tcl options files respectively. Source the Tcl file.

   This is the first stage where FDC synthesis constraints and attributes are honored, so make sure to include this file. You can add additional FDC constraints at later stages.

4. Run prepartition:

   ```
   run pre_partition -tss boardFile.tcl
   ```

   - You must specify the TSS board file.
   - The tool runs pre-partition with built-in area estimation (fast area estimation) when set to *0*. To run regular area estimation run pre-partitioning with *-area_est 1*. Area estimation is critical to creating efficient partitions.

     To use previously generated area estimates for an incremental run, first run pre-partition with *-out* to generate the database. For the incremental run, set *-area_est 0*, and specify the previous database.

     See Working With Area Estimates for Partitioning for details about using area estimation.
   - For more efficient partitioning, set an effort level for area estimation with the **`option set area_effort`** command. The default setting is *low*, but you can set it to *medium* or *high*.
   - The default number of jobs you can run in parallel is eight, but you can increase that number with the **`option set max_parallel_jobs`** command.
   - Specify a *pcf* file that contains partitioning constraints. For example, set *cell_utilization* constraints in the *pcf* file to update area numbers for each instance.
   - If you have *fdc* constraint files, add them at the command line with the *-fdc* argument.

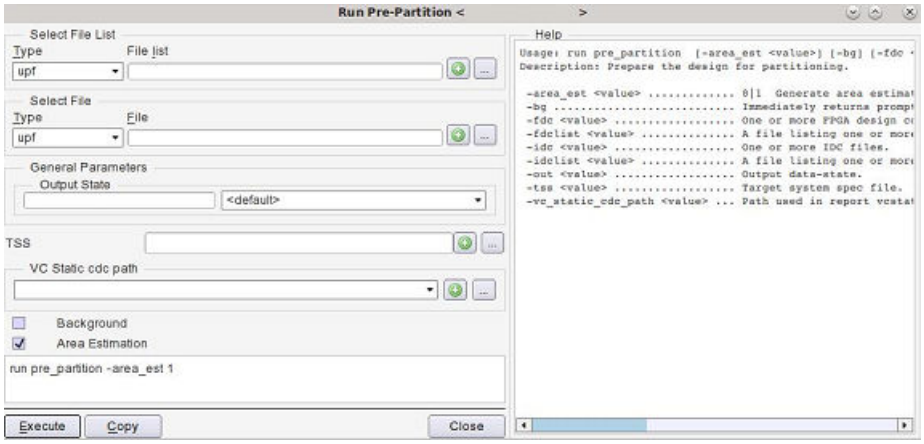
   For other options to the **`run pre_partition`** command, see *Command Reference Manual*.

   The first time through, the tool runs area estimation. It honors the constraints and options while performing technology-independent optimizations. If you instrumented the design, it inserts debug IICE logic. It creates a new database state (*pp)* that is ready for partitioning by eliminating feedthroughs, duplicate output, unconnected or no-load top-level input ports, and disabled tristates (connected to constants).

   The command generates an area estimate file called *design.est* and a compiled netlist (*srs*) that you can view with the **`view schematic`** command. To view the estimate file, use **`report list`** and select the area estimate file. You can also export this file for use during the partitioning phase. After running pre-partitioning, the tool generates a new, pre-partitioning database state.

Feedback

You can also pre-partition using the GUI instead of the command line. Start with a compiled database, select **State > Run Pre-Partition**, and specify the command parameters. For descriptions of the dialog box options, see Run Partition Dialog Box.



5. Analyze the results.

   - Check FDC constraints by running the `report constraint_check` command to generate the *constraint_check.rpt* file. Check this file for details about the constraints that were applied, not applied, or ignored. This command also generates a *syntax_constraint_check.rpt* file which reports any syntax errors with the constraints. To run just the constraint syntax check, use the `report constraint_check -syntax_only` command.
   - Use the `report list` and `export report` commands to view the reports generated after partitioning. These are the most important reports to check at this stage: Area Estimation Report, Clock Summary, Constraint Application Report, UPF Report, and Netlist Editing Report.

   For more information about analysis, see Analyzing Partition Result Files and Analyzing Results.

6. Make adjustments to options and constraints and rerun the command.

   Pre-partitioning is iterative and you can run it as many times as you need. After the initial run, the tool does not rerun area estimation, so subsequent runs take less time than the first run.

   - Check the channels and make feasible channels that yield a good TDM ratio, where the total number of pins per device does not exceed the maximum.
   - To lock results from a previous run, copy the appropriate assignments from the report into the *pcf* file for the next run.

Partitioning the Logic

Partitioning logic is an iterative process whereby you refine the design until you get the results. Your goal is to:

- Find a feasible partition that meets your minimum timing goals.
- Route the connections between bins using the trace groups specified in the TSS file.
- Find a suitable hardware topology and logic assignment for your design.

The design aspects determine the best hardware configuration. The partitioning solution provides a flexible way to iterate through high-capacity designs and partition them.

Partitions are implemented with the `run partition`, `run system_route`, and `run system_generate` commands.

The following topics focus on the `run partition` command:

- Creating Partitions
- Partitioning Single-FPGA Designs
- Working With Area Estimates for Partitioning
- Using Timing-Aware Partitioning
- Partitioning Clocks
- Locking Down Partitions
- Using Interactive Partitioning

For additional partitioning-related information, see these topics:

- Using run pre_partition to Define Partitions
- Analyzing Partition Result Files
- Running System Route for the Top Level
- Generating FPGA Partitions

Creating Partitions

After pre-partitioning, partitioning solutions are explored and implemented with the *run partition*, *run system_route*, and*run system_generate* commands. This procedure summarizes how to generate hardware-aware partitions for a multi-FPGA design with the *run partition* command. The partitions created can be exploratory or final, depending on the TSS hardware definitions and the assignments in the PCF file.

Feedback

To use hardware-based partitioning for a single-FPGA design, refer to the procedure in Partitioning Single-FPGA Designs.

1. Start from a pre-partitioned database state.

   See Using run pre_partition to Define Partitions for details about generating this state.

   If you are working iteratively and have already partitioned and implemented your FPGA partitions, you can refine top-level partitioning by using area estimates generated after implementing the FPGAs. See Updating the Top Level with FPGA Implementation Results for details.

2. Prepare the input files for partitioning.

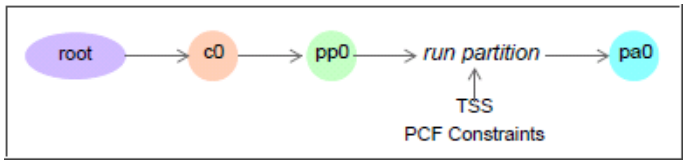
   The files are specified as part of the command line in the next step.

   - Describe the hardware setup in a TSS board specification file. (See Defining the Target System in a Detailed TSS File or Exploring Partition Choices with a Basic TSS File for more about creating this file.) You must have this file if you are doing a quick partitioning run and have skipped pre-partitioning. You do not have to specify the file again if you have run pre-partitioning and specified it at that time.
   - Specify partitioning constraints in a *pcf* file. See Configuring PCF Constraints for Partitioning.
   - Optionally, specify synthesis attributes and constraints in an *fdc* file. Check FDC constraint syntax, using the `report constraint_check -syntax_only` command.
   - Set other options as needed in *a* Tcl options file. Source the Tcl file.

3. Optionally, set up more accurate area estimates for use during partitioning.

   - Specify the `run pre-partition -area_est` command if you want to generate more accurate area estimates. Specifying this argument generates a *design.est* file which you can use when you run partitioning. See Working With Area Estimates for Partitioning for details.
   - Set *cell_utilization* constraints in the *pcf* file to update area numbers for each instance.

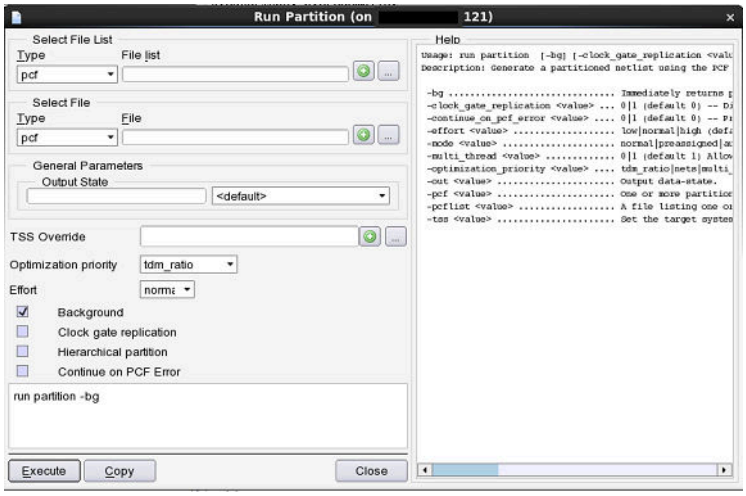
4. Use the `run partition` command or select **State > Run Partition** in the GUI to partition the design.



Feedback

- Set partitioning controls. See Partitioning Controls for some typical controls.
- For better area estimation, use the controls described inWorking With Area Estimates for Partitioning.
- For timing-aware optimizations, use the appropriate arguments to the `run partition` and `run system_route` commands. See Using Timing-Aware Partitioning and for details.
- Specify a simple TSS file with the *-tss* option and set *-effort* to *low*.
- Add constraints in a *pcf* file.
- Optionally, specify an *fdc* constraints file. Some constraints take effect at this stage.

To use the GUI to run the command, start with a pre-partitioned database, click **Partition**, and specify the command parameters. For descriptions of the dialog box options, see Run Partition Dialog Box.



The tool automatically partitions the design and does global routing. After partitioning, the tool generates a multi-FPGA netlist. It also generates a new, partitioned database state. It generates an assignment *pcf* file and reports about the global routing.

5. Analyze results.

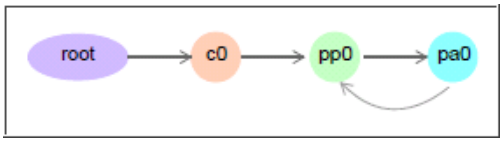
   See Analyzing Partition Result Files for information on interpreting results in

6. Make modifications to the TSS and *pcf* files as needed and rerun partitioning until you are satisfied with the results.

   Use the output *pcf* from the previous run as input for the next run. Analyze the reports from each run and make changes based on the results. You can update the top level with results from the partitioned FPGAs, as described in Updating the Top Level with

Feedback

FPGA Implementation Results. You can also loop back to pre-partitioning and make changes at that stage, before running partition again.



The next step is to route the system level, as described in Running System Route for the Top Level. You can iterate between partitioning and system routing, just as you do with multiple partitioning runs.

7. To freeze partition results, export the *assignments.pcf* file and use it on the next run, as described in Locking Down Partitions.

Partitioning Controls

The following table highlights some useful partitioning controls, but it is not an exhaustive list. See Partition Constraint File Tcl Commands and Target System Specification Commands for details about the available commands and constraints.

Description	**Command/Constraint**
run partition/*run system_route*Command Arguments	
Hardware specification	*-tss*
PCF constraints	*-pcf -pcf_continue_on_error*
Timing-aware partitioning	*-estimate_timing -optimization_priority -multi_hop_accuracy*
Effort level	*-effort*
Area estimation	*-builtin_est run pre-partition -area_est*
Clocks	*-clock_gate_replication -clock_crossing_reduction*
Feedthroughs	-feedthrough_reduction -illegal_feedthrough_reduction -allow_feedthrough_tdm
PMUX decomposition	-pmux_decomposition
PCF File Constraints	
Resource utilization	*bin_utilization*
Floorplanning	*assign_cell, cluster_cell, cluster_port*

Feedback

Description	Command/Constraint
Hierarchy control	*cell_attribute -dissolve 0|1|auto, dissolve_control*
Global routing controls	*net_attribute*
TSS File Commands	
System configuration	*board_system_create*
HAPS connections for top level	*board_system_configure -top_io*
Clock configuration	*board_system_configure -clock*
Inter-FPGA connections Daughter boards	*board_system_create -interconnect*

Partitioning Restrictions

The following restrictions apply when partitioning designs.

General Partitioning Restrictions

- Rotate right (*rotr*) and rotate left (*rotl*) primitives inferred by the compiler cause the partitioner to error out. Work around this by using the netlist-based (*srs*) single-FPGA scripts generated after system generate, and ignore the HDL-based scripts. To generate netlist-based scripts, run system generate with the *-database 1* argument.
- Enumerated data types are not supported.
- Support for member port types in composite data type records is limited. Only members with the following port types are supported for composite data type records: *std_logic*, *std_logic_vector*, *std_ulogic*, and *std_ulogic_vector*.
- Port types *bit*, *bit_vector*, *boolean*, *integer* (signed and unsigned), and single- and multi-dimensional arrays are not supported as members in records. However, the partitioner can handle these types if they are not part of a record.

Mixed-Language Restrictions

- You cannot use an entity with multiple architectures when Verilog is the language for the topmost level. This is because Verilog does not support multiple entity/architecture pairs. Here are some possible workarounds:
  - Run system generate with the *-database 1* argument to generate netlist-based (*srs*) single-FPGA scripts for the partitions, and use them to run synthesis. Ignore the HDL-based scripts.
  - Expose the mixed-project writer to only one entity/architecture pair. Do this by either not splitting views that contain entities with multiple views, or by generating wrappers for each entity/architecture pair with a unique entity name.
  - If only a single architecture is used for prototyping, insert *synthesis off* and *synthesis on* pragmas around the unused architectures such that only one architecture is compiled for each entity.
- When entities with ports that are elements of records are at the top of the partitioned FPGAs, VHDL and mixed-language generated RTL code is not properly constructed. This results in a failure to compile mixed-language projects. If the entire record appears as a port, code generation is correct. You can avoid this by either using the *srs* netlist partitions, or by not placing entities with record elements at the ports of an FPGA boundary.
- When entities with ports of type *character* or *string* are at the top of the partitioned FPGAs, VHDL and mixed-language generated RTL code is not properly constructed and this results in a failure to compile mixed-language designs. Avoid this by either using the `srs` netlist partitions or by not placing entities with these types of ports at an FPGA boundary.
- With mixed-language designs, do not use reserved words in the other language as top-level FPGA port names, because this can cause linking errors during compilation. Either avoid using reserved words in port names or avoid mixed-language designs when reserved words are used as top-level FPGA ports.
- With mixed-language designs, using an entity with the same name as a module can cause the partitioner to error out when SLP HDL project writing is enabled. As a workaround, avoid using entities and modules with the same name or disable HDL SLP project writing.

VHDL Restrictions

- Global VHDL signals (signal declarations in a package) are not supported in mixed-language partitions. Using these signal declarations in the RTL code restricts the tool to generating netlist-based partitions.
- With VHDL designs, entities that share the same name but which are located in different libraries and have logical differences can cause the partitioner to error out

when Verilog partition writing is enabled. Either avoid using entities with the same name and different logic in different libraries, or use *srs*-based single-FPGA scripts instead of HDL-based ones.

- When partitioning a VHDL design with signed division or remainder operations, the partitioner errors out when attempting to write out Verilog partitions. Because signed division or remainder operations in Verilog are not modeled in the same way as VHDL, you must disable the generation of Verilog partition output. Use mixed-language output when the source code must be used. For compiling and mapping the FPGAs, use the partition srs-based scripts instead of the HDL-based scripts.
- VHDL arrays are only supported with one-dimensional arrays of single-bit objects. Arrays of multi-bit objects (for example, *bit_vector*, *std_logic_vector*, and *integer*) are not supported.

Verilog/System Verilog Restrictions

- With System Verilog designs, using interfaces or complex data types (structures or unions, for example) as a port can cause the partitioner to error out if it does not match the original port declaration. One possible workaround is to only use the netlist-based scripts for partitioning, and ignore the HDL-based scripts.
- If the top-level project module in a Verilog design has the same name as a sub-module but with a different letter case, you get an error when you simulate the entire system in VHDL, because the top (`vhp`) and a sub-entity have the same name in the VHDL name space. To avoid this error, do not name a top module with the same name as a sub-module if you have to consider VHDL case insensitivity, or do not simulate with VHDL.
- When compiling a Verilog partition, you must use the Verilog 2001 standard because parameters are passed formally. If your original project uses a Verilog 2001 reserved word as a port, net, or instance name, you get an *"illegal use of reserved word"* error. To avoid encountering the error, rename the object in the original source code.

Partitioning Single-FPGA Designs

The typical prototyping flow with a single-FPGA target does not have a mechanism to specify hardware considerations, like daughter board interfaces or top-level port constraints. The hybrid flow described here shows you how to specify hardware considerations for a single-FPGA design where the logic is confined to one FPGA, and includes hardware configuration information (TSS and PCF).

This is particularly useful for IP designers who no longer have to blend single-FPGA and multi-FPGA flows to accomplish what they need. It can also be used to set up designs that share the hardware system in multi-design mode (MDM).

Feedback

1. Create a design database and compile it.

   ```
   database create name -tech run compile -srclist RTL_files
   ```

2. If you have set options, source the options file.

   The options file is not stored with the database.

3. Prepare the design for partition.

   - Create a TSS file with the system and daughter board definitions. For multi-user mode, make sure the TSS is compatible with other designs that share the hardware system.
   - Include the hardware configuration command that designates a single FPGA (*-single_fpga_target*) in the TSS file:

     ```
     board_system_configure -single_fpga_target <fpgaName>
     ```

     This command assigns all the logic to the specified FPGA and locks all the other FPGA bins so no logic is assigned to them.
   - Add timing constraints in an FDC file.
   - Pre-partition the design, using the TSS and FDC file as inputs.

     ```
     run pre_partition -tss <tssFile> -fdc <fdcFile>
     ```

4. Partition the design.

   - If you have mixed systems, you can block systems for partitioning.
   - Assign FPGA pins for the daughter boards in the PCF. Optionally, you can set cell utilization constraints.
   - Generate partitions based on the PCF assignments.

     ```
     run partition -pcf pcfFiles -area_est 0
     ```

   - Check the partition log file for errors. The tool runs a DRC check on the constraints and reports errors in the log file.

     ```
     @S2.3 |Single HAPS Target Flow DRC
     @W: : | Disregarding soft cell assignment of cell 'myCell' to bin(s)
      'fb1.uB' due to single HAPS FPGA 'fb1.uA' target flow.
     @E: : | Cell 'padder_' is replicated -hard to bin(s) 'fb1.uB fb1.uC'
      which is not supported for single HAPS FPGA 'fb1.uA' target flow.
     @E: : | Port 'out_304' is assigned -hard to bin(s)
      'TOP_IO_HT3_mb1.B1' which is not supported for single HAPS FPGA
      'fb1.uA' target flow.
     Error in single HAPS FPGA target constraints.
     ```

5. Run system route to assign the pins:

   ```
   run system_route -pcf pcfFiles
   ```

6. Create a single-FPGA database for synthesis:

   ```
   run system_generate
   ```

   The tool automatically turns off time budgeting when it generates the partitions.

Working With Area Estimates for Partitioning

Area estimation determines how much logic fits into each FPGA and directly affects partitioning quality. Accurate estimates are essential for:

- Determining the number of FPGAs required
- Balancing logic across devices
- Guiding partitioning decisions to optimize resource utilization

This section describes how to generate, import, reuse, and analyze area estimates to support the partitioning process.

By default, area estimation runs during pre-partitioning using the following commands:

```
run pre_partition
run partition
```

The default setting are:

- Pre-partitioning: `-area_est` is 1.
- Partition: `-builtin_est` is 0.
- Reusing Area Estimation Data
- Bypassing Area Estimation
- Using Built-in Area Estimates
- Analyzing Area Estimation Results
- Using Estimates From FPGA Synthesis Tools
- Using Your Own Custom Estimates

Reusing Area Estimation Data

If you already have an area estimate file (`.est`) from a previous run, you can reuse it to save runtime. This approach is suitable when design changes are minimal or when small differences in area can be tolerated.

Feedback

Use the following flow to export and reuse area estimates:

```
run pre_partition
export file top.est
database set_state c0
run pre_partition -area_est 0 -out noest
database apply_state -import_est top.est
run partition
```

The flow performs the following steps:

1. Runs pre-partitioning and exports the `.est` file.
2. Disables area estimation for the subsequent pre-partition run.
3. Imports the previously generated estimates into the current database.
4. Runs partitioning using the imported estimates.

Bypassing Area Estimation

Built-in area estimation is model-based and emphasizes runtime over accuracy.

Using Built-In Estimates for Fast Runtime

Built-in estimates provide the fastest runtime but are less accurate than standard area estimation. Use this method when:

- Regular area estimation is not available
- You are performing flow validation or fast iterations

Built-in estimates are not recommended for generating bit files or for on-board design verification.

```
run pre_partition -area_est 0
run partition -builtin_est 1
```

Overriding Standard Estimates with Built-In Estimates

If standard area estimation is available but you choose to override it, enable built-in estimation during partitioning:

```
run pre_partition
run partition -builtin_est 1
```

Using Built-in Area Estimates

For fastest runtime

Built-in area estimation is model-based, and is a very fast but less accurate method of area estimation. Use built-in area estimates when regular area estimation is not available.

Feedback

```
run pre_partition -area_est 0
run partition -builtin_est 1
```

Overriding with Built-in estimates

Use built-in are estimates when normal area estimation is available.

```
run pre_partition
run partition -builtin_est 1
```

Analyzing Area Estimation Results

After running pre-partitioning, the tool generates:

- An area estimate file (`.est`)
- An SRS netlist

To review the results:

1. Run pre-partitioning.
2. Use the `report list` command to locate the `.est` file.
3. View the estimate file using the report viewer.
4. Export the file, if needed, for reuse by downstream database states.
5. View the netlist using the `view schematic` command.

Using Estimates From FPGA Synthesis Tools

Import area data generated by Synopsys FPGA synthesis tools. Use more accurate estimates from synthesis instead of ProtoCompiler's estimate.

```
run pre_partition -area_est 0
database apply_state -import_est <file from synthesis>
run partition
```

Using Your Own Custom Estimates

Provide your own `.est` file and optionally set utilization constraints to update area numbers for each instance using `cell_utilization` PCF command.

```
run pre_partition
database apply_state -import_est <fileName>
run partition
```

Using Timing-Aware Partitioning

The tool provides different controls to drive timing-driven partitioning. The ones you choose to use depend on your design needs. Timing-aware partitioning is controlled by a

Feedback

combination of arguments to the `run partition` and `run system_route` commands and options.

The combination of the *-estimate_timing* to the `run system_route` command and *-optimization_priority* to the `run partition` command, determines the timing model used and let you tune your settings to suit your design needs. The following table summarizes how to use these options.

All setting combinations shown honor the TDM qualification mode you set (*tdm_control -qualification mode*), or default to *all*.

Goal	Settings	Result
Best run time	*optimization_priority tdm_ratio estimate_timing 0*	Optimizes the worst TDM ratio.
Get slack values on inter-FPGA paths	*optimization_priority tdm_ratio estimate_timing 1*	Optimizes the worst TDM ratio. Runs timing estimation and adds slack values on inter-FPGA paths to the TD reports.
Best performance	*optimization_priority multi_hop_path estimate_timing 1* With the run system_route command	Runs timing estimation and does slack-based TDM insertion and multi-hop optimization.
	optimization_priority slack -mapped_timing_models 1	Generates accurate timing models from mapped netlist to improve pre-tdm and post-tdm slack correlation between system route and system generate.*Improves correlation in multi-hop paths with clock groups, clock crossing paths and individual path timing exceptions.*
Best runtime and performance (without clock constraints)	*optimization priority multi_hop_path estimate_timing 0*	Does TDM insertion based on board delays, optimizes the worst TDM ratio including multi-hop paths, and uses actual cable delays and 0 slack value for inter-FPGA paths.

For faster partitioning runs or to take clock constraints into account, see Using Graph-Based Timing-Aware Partitioning.

Using Graph-Based Timing-Aware Partitioning

To achieve better partition results with a HAPS system, you can use top-down timing (TDT) which bases partitions on a timing graph model. Here, partitioning is based on

Feedback

timing graphs that the tool creates during pre-partitioning for each module, from the bottom up. During partition and system route, the tool composes a top-down timing graph based on the timing models, and uses it to drive partitioning. Graph-based partitioning (or timing-aware partitioning, or TDT) only considers routing delays and assumes that all logic delays, except for signal delays that cross FPGAs, are 0.

Unlike *estimate_timing* model, the TDT does not require calculation time, so it is much faster. It also saves an additional cycle because it lets you change clock constraints and check the results at the system route state, without having to rerun and re-calculate, as you would with *estimate_timing*.

1. Set timing constraints.
   - Specify clocks and set constraints for them.
   - Set input and output delay constraints.

   Timing-aware partitioning honors these constraints: *create_clock, set_clock_groups, set_false_path, set_multicycle_path, set_input_delay,* and *set_output_delay*. For information about these constraints, see Specifying FDC Constraints.

   There are some exceptions. Graph-based partitioning does not support the *set_max_delay* and *reset_path* constraints. It also does not support phase relationships defined with *create_clock -waveform*:

   ```
   create_clock -name {clk} -period 10 -waveform {5.0 9.5} [get_ports {clk_in}])
   ```

   Graph-based partitioning also does not support the propagation of two clocks when the constraint is applied on the same net, as shown here:

   ```
   create_generated_clock -master_clock [get_clocks <clk_alias>] -add
   ```

2. Run pre-partitioning (*run pre_partition*) and partition (*run partition*)

   The tool creates timing graphs for each module after pre-partitioning.

3. Specify that the timing graphs be used to drive routing with the `run system_route -optimization_priority slack` command.

   This command enables routing based on top-down timing graphs, and inserts slack-based TDM. The partitioner loads the timing models for the current state and composes a complete timing graph for the design from the modules. Based on the timing graph, inter-FPGA TDM delay, and the *fdc* constraints (including reset path, false path, and multicycle path timing exceptions), it tries to minimize the worst slack of the design when it creates the partitions.

   The router optimizes the design to minimize the worst case slack of the design using an estimate of the inter-FPGA TDM delay, top-down timing graph, and *fdc* constraints.

Feedback

Use option *-mapped_timing_models* with *-optimization_priority slack* to generate accurate timing models from mapped netlist to improve pre-tdm and post-tdm slack correlation between system route and system generate.

4. Check the paths and timing reported in the critical path summaries in the log file.

Partitioning Clocks

The following procedures provide more information about handling clocks across partitions:

- Replicating Clocks
- Replicating Clock Control Managers
- Rerunning Partition with Pre-Assigned Clocks

Replicating Clocks

If your design has clocks that cross FPGA boundaries, clock skew can degrade performance and introduce functional problems. One solution to this problem is to replicate the clock tree and insert clock buffers in the different FPGAs.

For the syntax of the commands mentioned in the following procedure, see Partition Constraint File Tcl Commands.

1. Check the partitioner log for clocks and asynchronous resets that cross FPGA boundaries.

   The report includes messages like the following:

   @S4.5 AP406 | Partitioner Estimate of Clock & Asynchronous Reset Crossings

   Cut clock: fb1.uD->{fb1.uc:} icore_sys.iarchipelago.iCCalb_cpu _top_u_alb_cpu_clk_gated

2. Specify clock tree replication when you rerun partition:

   ```
   run partition -clock_gate_replication 1
   ```

   The software automatically replicates the clock tree in the FPGAs, and reports the results in the *Automatic Clock Replication Summary* section of the partition log, as shown in this example:

```
885 AP425 |Automatic Clock Replication Summary
Clock_Net                                                    #Replicants Target_Bins          #Inputs Prim
-------------------------------------------------------------------------------------------------------------
*icore_sys.iarchipelago.iCCalb_cpu_top.u_alb_cpu.clk_gated   4           mb1.uC,mb1.uB,mb1.uA 1       xck_
^N_2                                                         1           mb1.uC,mb1.uB,mb1.uA 0       xck_
TOTAL                                                        4
=============================================================================================================
*some replicants are counted multiple times
```

It also generate a *pcf* file with the appropriate replication constraints, called *auto_replication.pcf.*

3. To manually replicate a cell to all bins, set this command in the *pcf* file:

   ```
   replicate_cell {cellName} -all_bins 1
   ```

   Use `clock_gate_replication_control` command to unlock bins that are otherwise locked, to allow clocks to be replicated. The bins are locked for all other operations

4. Use these *pcf* commands to disable replication:
   - To disable manual clock replication, use the `replicate_cell` command:

     ```
     replicate_cell {cellName} -all_bins 0
     ```

   - To disable automatic replication and manual clock replication, use the `cell_attribute` command:

     ```
     cell_attribute -auto_replicate 0
     ```

Replicating Clock Control Managers

In general, it is recommended that you do not replicate clock managers like MMCMs (mixed-mode clock managers) on multiple FPGAs. The reason is that this could introduce phase differences between clocks, and can cause incorrect system-level timing. Replication might also result in hold violations and hardware failures. If design needs dictate the replication of MMCM logic on multiple FPGAs, you must do so manually, using PCF constraints.

Proceed with care when replicating CCMs (clock control managers).

1. Replicate the clock divider modules from the CCM.
   - Instantiate the CCM in the top-level design and specify parameter values.
   - Assign the clock module to an FPGA with a PCF constraint. For example:

     ```
     assign_cell clock_control FB1.uA
     ```

   - Replicate the clock module on other FPGAs, including the one to which it was assigned. This ensures that signals are synchronized. Use a PCF constraint, as in this example:

     ```
     replicate_cell clock_control.cclk_gen_replica.cclkgen0 {FB1.uA
     FB1.uB}
     ```

   For details about the PCF commands, see Partition Constraint File Tcl Commands.

2. Assign the clock enable to the GCLK network with PCF constraints.

Feedback

It is recommended that you connect CCM-generated clocks to the HAPS GCLK network and use the GCLK in all the FPGAs in your design.

```
assign_cell {clock_control.clk_cntr_en_keeper} {FB1.clk.bd_clk4}
net_attribute {clock_control.clk_cntr_en} -function GCLK -diffsingle
-is_clock 1 net_attribute {clock_control.clk_cntr_en_int} -function
GCLK
```

3. In the PCF, assign the trigger input port to gate the CCM, as in this example:

   ```
   assign_port {trigger_in} -trace {FB1_A5_A[7]}
   ```

4. In the TSS file, define the GCLK to be used as the FPGA source. For example:

   ```
   board_system_configure -clock {FB1.GCLK4} FPGA
   ```

   For details about this command, see board_system_configure in the *Command Reference Manual*.

   For an example of MMCM replication, see SolvNet article 2345509. For specifics about replicating clock managers for debug with GSV, see Instantiating and Configuring Clock Control Modules for GSV.

Rerunning Partition with Pre-Assigned Clocks

Use the following procedure when you are iterating, to remove cut clocks that were not resolved during initial partitioning. The technique reuses previous clock assignments, so you can build on existing results. It also saves runtime on subsequent partitioning runs.

1. Do an initial partitioning run with run partition, where clock replication assignments are made, as described in Replicating Clocks.
2. Export the constraints from this run with this command:

   ```
   export file assignments.pcf
   ```

   The *assignments.pcf* file is generated by the tool, and contains the clock constraints from the preceding run.

3. Reuse current clock assignments for the next run, with hard or soft constraints.
   - Do not edit the file if your intent is to resolve cut clocks. Leaving the constraints in the file as soft constraints (default) signals that the partitioner can optimize the clock assignments. Once the cut clocks have been resolved, you can make the constraints hard instead of soft.

- To change the current constraints to hard constraints for subsequent runs, edit *assignments.pcf*. The file includes Tcl variables which are set to *soft* by default. Change the value to *hard*. These are the Tcl variables:

  ```
  set port_soft {-soft |-hard} set cell_soft {-soft | -hard} set
  repl_soft {-soft |-hard}
  ```

  You can set each of these variables independently of the others.

- Use the variables to control assignments and replications, adding them to the commands in the *assignments.pcf* file. For example:

  ```
  assign_port $port_soft port1 {portbin} assign_cell $cell_soft cell1
  {fb1.uA} replicate_cell $repl_soft cell2 {fb1.uA fb1.uB}
  ```

  The partitioner does not replicate clocks for cells that have hard constraints.

4. Rerun partition using *assignments.pcf* and the *-optimization_priority preassigned* argument.

   ```
   run partition -optimization_priority preassigned -pcf assignments.pcf
   -clock_gate_replication 1 otherArgs
   ```

   With *preassigned* set as an optimization priority, the partitioner uses the constraints set in *assignments.pcf* as a starting point. If you have unassigned cells, you get an error. The partitioner applies TDM qualification. This partitioner run is faster because it is incremental, and does not rerun certain partitioner operations.

Locking Down Partitions

You can lock down partitions for incremental use or lock down the final solution.

Locking Down Incremental Partitions

You can incrementally lock down partitions and save or freeze them so that you can make minor changes to them and reuse them on subsequent runs. You export a *pcf* file with final constraints to lock down partitions. The following steps demonstrate how to stabilize results for an incremental partitioning run.

1. Start with a database (partition or system route) where you are satisfied with the results.
2. Export the *pcf* file with the results.

   The *pcf* file you export is different, depending on the state of your design.

- After partitioning, export *assignments.pcf*. For example:

```
database load db0
database set pa0
export file -list
export file assignments.pcf
```

  Edit the *assignments.pcf* file to reflect minor incremental changes, without changing the RTL. The *pcf* file contains assignments to all the cells in the design, and can be used for predictable results. For example, if you want to add instrumentation to one FPGA but keep the others as is, delete the references to the FPGA you want to change, and use this updated file as input when you rerun partitioning.

- After system route, export *router_constraints.pcf*. For example:

```
database load db0
database set sr0
export file -list
export file router_constraints.pcf
```

  The *router_constraints.pcf* file contains assignments to all the cells in the design, and can be used for predictable results. Make minor changes to this file if the partition changes, or if you need to move a net or cable to meet timing.

3. Specify the exported *pcf* file during the next run, using the *run* command appropriate to the design stage:

Partition	*run partition -pcf assignments.pcf*
System Route	*run system_route -pcf router_constraints.pcf*

4. To fix incremental trace assignments, use the *net_route pcf* constraint as shown below:

   - To control trace assignments to nets, use the *-using_trace* option:

```
net_route {net1t}  -from {FB1.uB} -to {FB1.uC}
 -using_trace {FB1_A12_A1[4]}
```

   - To control how nets are grouped for TDM, use *-using_trace* with *-tdm_module*:

```
net_route {net1 net2 net3 net5} -from {FB1.uA} -to {FB1.uC}
 -using trace {FB1_C12_C[6]} -tdm_module {HSTDM_4by2}
 -tdm_net_name {tdm_net_1}
```

Locking Down Final Partitions

When you are satisfied with the partition solution, lock down the solution by translating all abstract PCF constraints into TSS constraints.

Feedback

1. Make sure the PCF does not contain any abstract definitions.
   - Make sure that all abstract definitions are translated into TSS constraints and that there are no abstract definitions in the PCF.
   - The PCF file should no longer contain any abstract constraints; it should only have assignment constraints.
2. Make sure the TSS fully defines the setup.

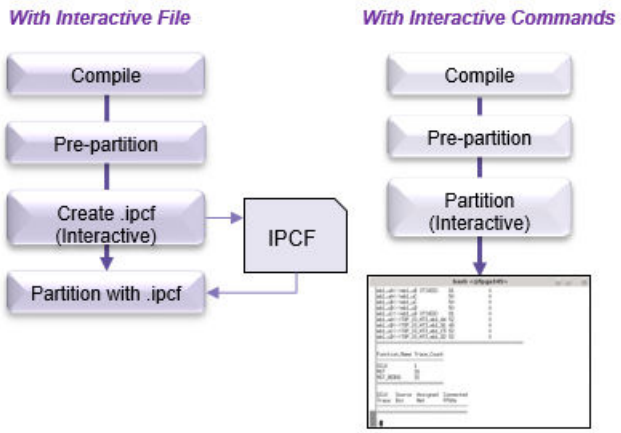
   Translate all abstract PCF definitions into corresponding TSS definitions.
3. Run partition with these files.

   The tool generates a partitioned database for system routing.

Using Interactive Partitioning

Interactive partitioning is a way to access the partitioner and fine-tune partitioning by moving cells to different FPGAs. You can also get feedback on the impact of changes and reiterate.

You can run interactive partitioning through a file or run the commands individually in the shell window.



Feedback

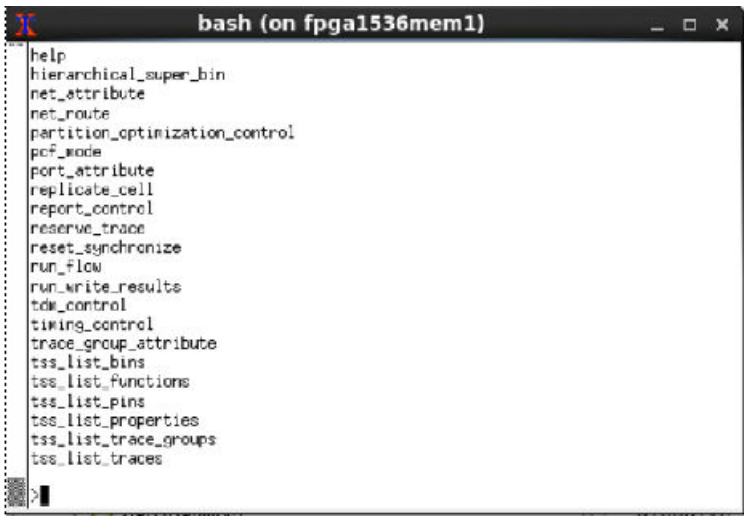
The procedure describes the steps to follow:

1. Set the license to enable the interactive partitioner: *-license_feature FpgaInternalBeta*
2. To run interactive partitioning through a file, do the following:
   - Create an interactive PCF file with the commands. See Interactive Partitioning File for an example.
   - Specify the command with the *-ipcf* argument and the interactive PCF file:

     ```
     run partition -interactive 1 [-ipcf filename]
     ```
3. To run an interactive partitioning session from the console, use this command:

   ```
   run partition -interactive 1
   ```

   When you specify the command without the file, an interactive shell window becomes available where you manually enter individual partitioning commands.
   - Type *help* in the window to see a list of available commands at that stage. The list of available commands changes, according to the stage of interactive partitioning you are in.

   - Start an interactive partitioning session with *run_flow*. This command signals the switch to manual partitioning. If you type *help* again, you see a different set of available commands. Note that these interactive partitioning commands are slightly different from the regular PCF constraints, and that not all PCF choices are available for interactive partitioning. From this point up to the end of the session, you can only specify the interactive partitioning commands.

     See Interactive Partitioning Commands for command descriptions. For descriptions of individual commands, you can also type *commandName-help* in the window.

- Enter manual commands to make changes to partitions or get reports. For example, use these commands to get the worst paths and a report summary:

```
report_timing -type hops -worst_paths 10
report_timing -summary
```

- Run multiple iterations until you get the results you want. You can save and write out the results with the `solution_commit` and `run_write_results` commands.
- End an interactive session with this command: `exit`.

Interactive Partitioning File

The commands in the file typically cover these phases:

1. Start manual partitioning with the `run_flow` command.
2. Analyze the design.
3. Assign cells and check results. Iterate through the analysis and assignment steps.
4. Save the solution and write out results (`solution_commit` and `run_write_results`).
5. End interactive partitioning with the `exit` command.

```
run_flow
report_timing -summary
report_timing -type haps -worst_paths 10
assign_cell F2.fb1.uA
assign_cell F3.fb1.uA
report_timing -summary
report_timing -type haps -worst_paths 10
solution_commit
run_write_results
exit
```

Analyzing Partition Result Files

The following section provides general guidelines on what to check in the results file after one of the partitioning commands.

- Checking the Target Specification (TSS)
- Analyzing Partitioning at Different Stages
- Checking Partitioning Reports
- Parsing Partitioning Reports

Feedback

Checking the Target Specification (TSS)

Check the target specification after a basic TSS file has been generated. You can do this after pre-partitioning or partitioning.

1. Specify the `report target_system -tss tssFile.tcl` command to generate a report.

   The report file (*report pcf_tss_report.rpt*) contains detailed information about the run. It provides information on design considerations, like logic assignments, grouped objects, and dissolved cells. It also lists implied constraints, which are created as a side effect of other constraints. The report also shows the results of global routing for each external net, indicating nets with no legal routing with the *-unrouted* flag.

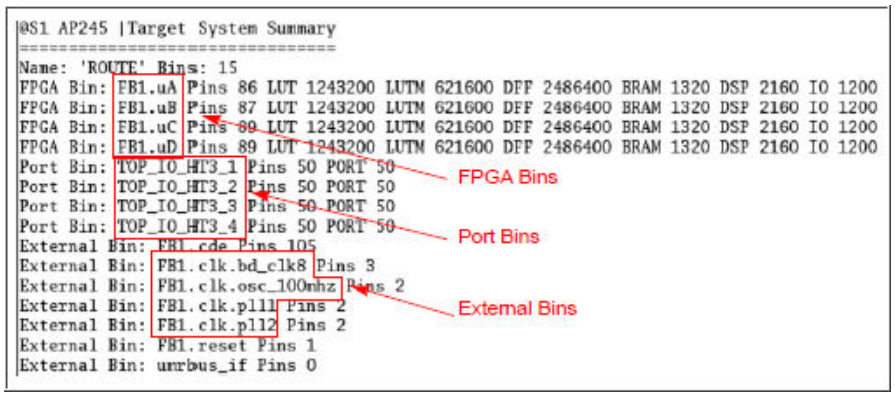
   The file also provides information on hardware requirements, like the connection model, target system, the number of FPGAs required, and the requisite cabling. It lists the board bins and traces.

2. Run `view report target_system.rpt` to view the report.

   Alternatively, list the generated reports with *report list* and export the file.

3. Check the report for the definitions of various bins:

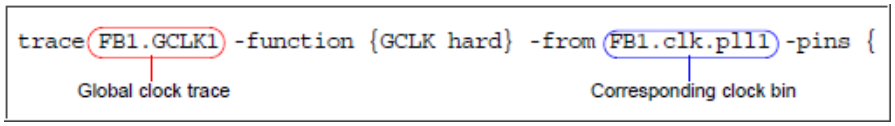
FPGA bin	Logic elements of the target FPGA
Port bin	Top-level port declarations
External bin	External objects
Clock bin	HAPS clock sources



4. You can also check the traces and identify clock drivers.

Feedback

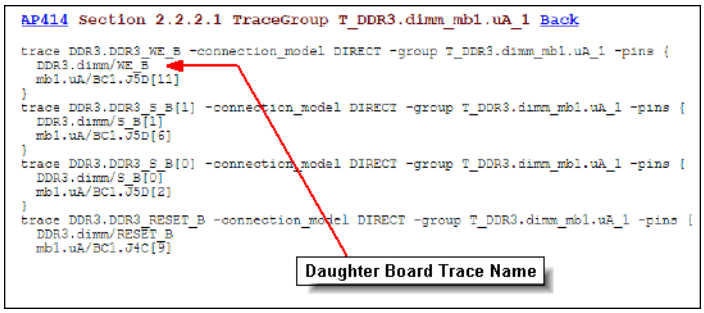
The `report target_system -tss` command lists point-to-point traces and special functions like clock and reset networks. In the following example, the global clock bin *FPB.clk.pll1* drives the *FB1.GCLK1* clock trace:



It also indicates net splitting for multi-terminal trace assignments with the *-split* flag, and feedthroughs with the *-feedthrough* flag. It also shows the detailed trace assignments and clock routing.

5. Check trace information.

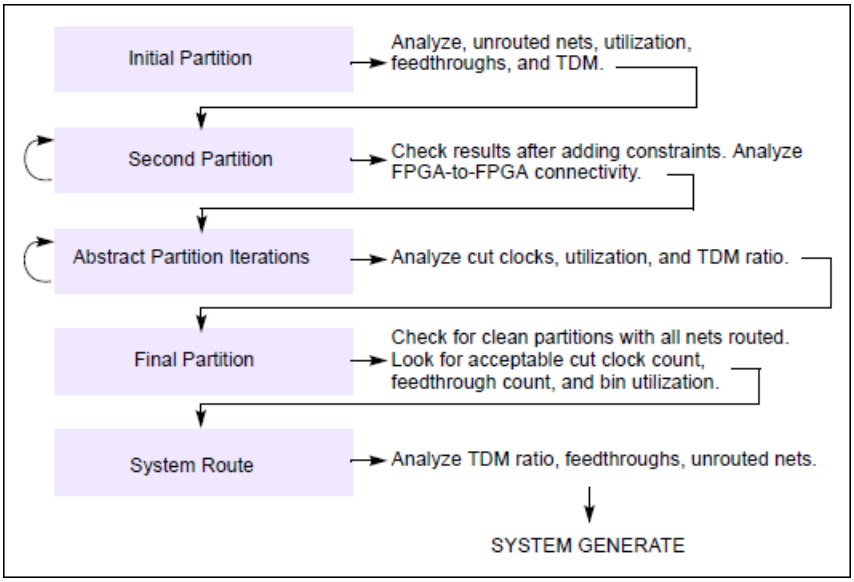
   Go to the **Traces > Traces Connecting Bins** section, which shows connection information for daughter boards like this:



Analyzing Partitioning at Different Stages

The following figure shows five stages of partitioning, excluding pre-partitioning. The first four use the `run partition` command to generate partitioning solutions and fine-tune them, and the last uses the `run system_route` command to implement the final solution.

Feedback



This section describes how to leverage the partitioning reports in the context of the various phases; for report-centered usage information, see Checking Partitioning Reports.

- Analyzing Partition: Initial Partition Stage
- Analyzing Partition: Second Partition Stage
- Analyzing Partition: Abstract Partition Iteration Stage
- Analyzing Partition: Final Partition Stage
- Analyzing Partition: System Route Stage

Analyzing Partition: Initial Partition Stage

1. Set up for the initial partition.
   - Pre-partition the design with `run pre_partition`.
   - Define connectivity using `abstract_tss -add_port_bin` commands to define cable connectivity.
   - To use port constraints to guide the solution, set *-constrain_connections* when you make port assignments in the PCF file.
2. It is recommended that you run partition with these arguments at the initial stage:

   ```
   run partition -optimization_priority nets -hierarchy_dissolve_pass 1
   ```

   These arguments reduce runtime.

For some designs, the *-effort high* argument can be very effective when used with *-optimization_priority nets*. However, the best options vary widely with the design, so experiment with the *-optimization priority* options for the best results.

3. Analyze nets in the partition log file.
   - Check whether partition goals can be achieved with the current parameters.
   - Check that constraints are applied correctly.
   - Check the Global Route Summary section for the number of unrouted nets. If unrouted nets are reported, check top-level port assignments, clock constraints, and cell assignment constraints.
   - If the total number of nets is high, it indicates possible partitioning problems because of the high trace utilization.
4. Check utilization values reported in the Bin Usage Summary section of the log file.
   - Check that the resource utilization value reported for FPGA bins meets your goal values.
   - If you see higher utilization values at this stage, determine if your utilization goal is achievable. You can manually add up the utilization for all the bins and divide by the number of FPGAs to determine if setting additional constraints can improve the values or whether you have to adjust your utilization goals.
5. Check the Global Route Summary for feedthroughs and TDM.

   At this point, just check and note if the value is feedthrough value is higher than the number of traces in an abstract channel at the maximum possible TDM ratio. If it is higher, this needs to be adjusted.

Analyzing Partition: Second Partition Stage

The emphasis at this stage is to adjust physical constraints to get more realistic partition results.

1. Confirm that there are no unrouted nets.
2. Create a separate *pcf* file for each type of constraint, or put them all in one *pcf* file:
   - Bin utilization constraints for bins, and a minimum LUT utilization constraint to balance utilization across bins. Example:

     ```
     bin_utilization-all_bins-resource_ratio{0.65}
     bin_utilization-min -all_bins-resource_ratio{LUT 0.30}
     ```

- Make high-level cell assignments to specific bins. For example:

```
assign_cell{sys} {fb_1 fb_2}
assign_cell{gpu} {fb_3 fb_4 fb_5}
```

- Add dissolve constraints for specific cells. In generall, do not dissolve cells that can fit into a bin at 65% utilization.
- Add cluster constraints for specific cells, according to the design and cell connectivity.

3. Run partition with the new *pcf* constraints.
4. Analyze results.
   - Check that the utilization constraints were honored.
   - Check that there are no unrouted nets.
5. Analyze and adjust connectivity.

   The goal is to create an abstract *pcf* file.

   For HAPS-100, you can use 34HT3 connectors out of 36 available, for HSTDM cabling and J1 & J30 are not HSTDM compatible. These are just the guidelines for the abstract *pcf* file to arrive at a better TSS for a good partition.

   - Check the Partitioner Ratio Estimate Report section of the partition log file for TDM ratio estimates. These estimates are based on partition engine estimates. Do not use the Global Route Ratio Report section, where the estimates are based on post-global route estimates.

     Remember that the TDM ratio reported is single-ended. Double the value if you are using differential HSTDM.
   - Look for the next biggest TDM ratio available to physically account for the estimate. For example:

Table 2 $S4.2.1 AP407 |Partitioner Ratio Estimate Report*

Connectivity	Available TDM Traces	Direct Usage	Feedthrough Usage*	TDM Usage	TDM** Ratio
fb_1.uA<->fb_.uD	80	5	0	4507	60.09

For differential HSTDM, you double the estimated TDM ratio of 60.09 to get 120.18. The next larger TDM ratio is 128:2.

- Use this data to modify the abstract *pcf*. Set constraints to lower the maximum ratio while making sure that the specified number of physical traces is available in the current system.

6. Create a new abstract *pcf* file based on the estimated TDM ratio results, and use it going forward.
   - Change all cell assignments within this file from hard to soft by adding *-soft*. This change ensures that they are treated as guides instead of hard constraints.

Hard Assignment	Soft Assignment
assign_cell {ufs_rx} {fb_1.uB}	*assign_cell -soft {ufs_rx} {fb_1.uB}*

Analyzing Partition: Abstract Partition Iteration Stage

The goal of this stage is to achieve a partition that satisfies initial results and a board file with some leeway for HT3 utilization. You might have to iterate through several times, checking reports and adjusting constraints. You might even have to return to the previous phase and start over with a full abstract connectivity matrix but different cell constraints for placements, dissolving, and clustering.

1. Check for unrouted nets.

   This might be caused by under-utilized channels between bins being removed. Fixes are design-specific. Some techniques are adding a new channel between bins, specifying cell placement, or replicating cells.

   Check that there are no unrouted nets before proceeding.

2. Analyze and fix cut clocks in the *Partitioner Estimate of Clock & Asynchronous Reset Crossings* section of the log file.
3. Analyze utilization.
4. Analyze the TDM ratio results, as described in Analyzing Partition: Second Partition Stage.
5. Run and rerun partitioning until you have a satisfactory result.

These are the characteristics of a partition that is ready for final partitioning:

- It is fully routed
- It satisfactorily minimizes number of nets between FPGAs
- It has minimal cut clocks
- It has a low number of feedthrough nets
- It has a low number of multi hop nets
- It has reasonable FPGA LUT and BRAM utilization, to make place and route faster and easier
- It uses an appropriate number of HT3 connectors for each FPGA

Analyzing Partition: Final Partition Stage

The goal is to transfer the abstract connectivity to real cable assignments and create a database that is ready for the next phase, which is system routing.

1. Make sure the design is ready for the final partition, as described in Analyzing Partition: Abstract Partition Iteration Stage.
2. In the TSS file, use the auto connect feature (details) to specify only the widths of the FPGA-to-FPGA channels.

   Note that when you use auto connect, the SLRs are not aligned. You can fix this later.
3. Run partition.
   - Partition with each FPGA utilizing HT3 connectors effectively Partition results are estimates only. Leave room for cable adjustments during the next stage (System Route) if needed.
   - Partition with desired maximum TDM ratio Partition results for TDM ratio are derived without considering timing. During system route, the tool assigns TDM ratio based on timing requirements. Allow a margin, for later adjustments during system route.
   - Use the *-optimization_priority multi_hop_path* argument to minimize multi-hop paths and improve timing.
4. Check the results.
   - There should be no unrouted nets.
   - There should be minimal nets between FPGAs
   - Cut clocks should be minimal.

- There should be a minimal number of feedthroughs.
- Bin utilization values should be satisfactory.

Analyzing Partition: System Route Stage

1. Transfer abstract connectivity to a TSS file.

   Check that there is some leeway for HT3 connector usage to modify cable connections if needed.

2. Add timing constraints in an *fdc* file.

   The system route algorithm uses these constraints to optimize TDM ratios. It can apply lower TDM ratios on critical paths and relax the ratios on non-critical paths. However, this is still an estimate; the most accurate system-level timing results are reported after the system generate stage.

3. Run system route, specifying the files from the preceding two steps.

   There are other useful options you can specify with the command:

   - To use estimated timing numbers instead of the timing constraints, use the *-estimate_timing 1* option.
   - Specify *-optimization_priority slack* to run timing estimation on paths that cross FPGA boundaries. Use this value as guidance. Currently, slack optimization does not take long net delays due to SLR crossings into account.

4. Analyze the results.

   - Check the Global Route Summary of the system route log file for information about the TDM ratio, feedthroughs, and unrouted nets.
   - Check the Clock Summary section for information about the clock pairs (to/from clocks) that clock data on the FPGA interconnect and worst path data information before and after slack optimization.

5. Analyze the slack to determine if the estimated numbers are enough to move forward.

   - Check that the slack estimates are positive, regardless of the TDM ratio. Use *Ctrl-f* to search for "*slack*" in the log file. You can find slack information in the Clock Summary which reports the estimated slack on paths that cross FPGA boundaries.

     If you have positive slack, you can move to the System Generate phase and confirm the estimates with system-level timing analysis.

   - Analyze the slack estimation on critical paths. The High-Level System Performance Path Summary section provides details about the critical paths in the FPGA interconnect before and after slack optimization. It there is a large negative slack,

analyze the design and adjust constraints to improve timing. In some cases, you might have to go back to partitioning and redo the design. In many cases, adding an HT3 cable might be sufficient to lower the TDM ratio and fix the negative slack. If the TSS file allowed for extra cabling, you might not have to re-partition.

- Check the *Multi Hops* section of the log file for a summary of multi-hop paths, which can affect system clock frequency.

Checking Partitioning Reports

The partitioner generates an *srr* log file and a *prf* report file after it finishes running. The log file contains high-level information about the run in chronological order, and includes error and warning messages. The *prf* report file contains detailed information about the assignments and is organized by function.

The files use a standardized format and syntax, which you can leverage to build your own parser and generate custom reports. See Parsing Partitioning Reports for more information.

The following procedure describes what to check in a partitioning report; for information about leveraging information from partition reports generated at different stages, see Analyzing Partitioning at Different Stages.

1. Check the log file for errors and warnings.

   When constraints are applied to a collection of instances which are partitioned into separate FPGAs, you get a warning message. This is because constraints applied to a collection are assumed to apply only to the FPGA that contains the relevant logic and related path. As long as the logic related to a constraint is in a single partitioned FPGA, the constraints work without modification. When instances are in separate FPGAs, the constraints are only applied to the portion of the logic that is partitioned into that particular FPGA.

2. Check the status of TDM on asynchronous nets (*AP255*). The report displays whether an asynchronous net is qualified for TDM or not. For example, for the net, *R*, the values under TDM are:

   - DT (Disabled Timing) - The nets are under different clock groups. This implies that the net is not qualified for TDM.
   - ET (Enabled Timing) - The nets are under same clock groups.



3. For timing, check the following files:
   - Check the *partition.log* file for information about multi-hop paths. Check the hop table and the hop path table. The tables list the connections between FPGAs. Check the *Multi-hop usage* column and multi-hop ratio to see if multi-hop paths require TDM, based on the number of traces available between FPGAs.
   - Check *timing.rpt* for timing values.
   - Check the multi-hop reports in the partition log for information about inter-FPGA timing. See for information about reducing the number of inter-FPGA hops.
   - Check the partition log for information about clock skew and determine if clocks need to be replicated. See Partitioning Clocks for details.
4. In the *prf* report file, check the *Bin Usage* section for bin utilization and the distribution of cells and ports in bins.

   The following excerpt shows low utilization of *FB1.uD*, which can then be explored further.

```
fb.uA(0) Cells: 316 LUT 487466 DFF 334089 BRAM 1041 DSP 24 IO 4
fb.uB(1) Cells: 2562 LUT 715535 DFF 301760 BRAM 579 DSP 6
fb.uC(2) Cells: 3472 LUT 1154379 DFF 549655 BRAM 796
fb.uD(3) Cells: 6 LUT 402741 DFF 319405 BRAM 377
TOP_IO_HT3_1(4) Cells: 20 PORT 20
TOP_IO_HT3_2(5) Cells: 20 PORT 20
```

5. Check the *Global Route Summary* in the report file.
   - Check *TDM Ratio*, and see if it meets your performance needs. TDM is time-domain multiplexing. See Using HSTDM and HSTDM ERD for details about this technique.

     ```
     @N: AP268 |Maximum TDM Ratio: 9
     ```

   - Check for unrouted nets, and decide how to eliminate them.

     ```
     @N: AP270 |Unrouted: 1
      FB1.clk.osc_100mhz -> FB1.uA 1
     Details:
     Unrouted net 'gclk8' from FB1.clk.bd_clk8 to FB1.uA - channel
      function mismatch
     ```

   - Check for feedthrough routes and judge their effect on timing.

     ```
     @N: AP268 |Feedthroughs: 0
     ```

   - In the system route report, look for feedthrough errors reported on asynchronous set and reset nets:

     ```
     @E: Feedthrough not allowed on async net
     ```

Feedback

By default, the tool does not allow asynchronous set/reset nets to be used as feedthroughs to route signals through an intermediate FPGA when there are no available direct traces between the source and destination FPGAs. To override the default and use asynchronous sets/resets as feedthroughs, set a PCF feedthrough constraint:

```
net_attribute -feedthrough_allowed 1 netName
```

6. Check the *Net Path Summary* in the *prf* file for inter-FPGA trace requirements.

```
@S4.1 AP271 |Net Path Summary
@N: AP272 |4101 nets connected to bins FB1.uD FB1.uC  with 500
 traces available
@N: AP272 |3223 nets connected to bins FB1.uC FB1.uA  with 500
 traces available
@N: AP272 |3120 nets connected to bins FB1.uC FB1.uB  with 500
 traces available
@N: AP272 |1411 nets connected to bins FB1.uD FB1.uB  with 500
 traces available
@N: AP272 |710 nets connected to bins FB1.uB FB1.uA  with 500
 traces available
```

Convert these requirements into firm trace constraints for the trace assignment stage, after taking available connectors and cables and the effect of TDM into consideration. To get a list of available connectors, see Identifying Connector Availability, next.

7. Check the *Trace Usage* section in the *prf* file to derive cabling requirements.

   This example shows 10 connectors and cables required between FB1.uD and FB1.uB.

```
@S4.2 AP265 |Trace Usage
@N: AP266 |Trace _T_1419 (ix=25) Usage: 480/500 F: DEFAULT
@N: AP267 |   FB1.uD->{FB1.uB} Net Usage 1370 (TDM 1370 DIRECT 0
 CLOCK 0) Async Ratio 3 Trace Usage 458
@N: AP267 |   FB1.uB->{FB1.uD} Net Usage 42 (TDM 42 DIRECT 0
 CLOCK 0) Async Ratio 2 Trace Usage 22
```

   If the total cabling and connector count exceed the resources, use one of these approaches to close the gap:

   - Increase the TDM ratio while keeping the current connection scheme
   - Use constraints to reduce interconnect
   - Explore other connection schemes

Feedback

Identifying Connector Availability

To identify which connectors are free, do the following:

1. From the TCL shell, launch the TSS editor: `% launch tss`
2. From the TSS editor interface, run these commands to get the connector status: `% source <your TSS file> % board_system_list -connectors -available`
3. The software lists the connectors that are available for you to use. To find the connectors that are unavailable because they are being used by the system, use this command: `% board_system_list -connectors -occupied`

Parsing Partitioning Reports

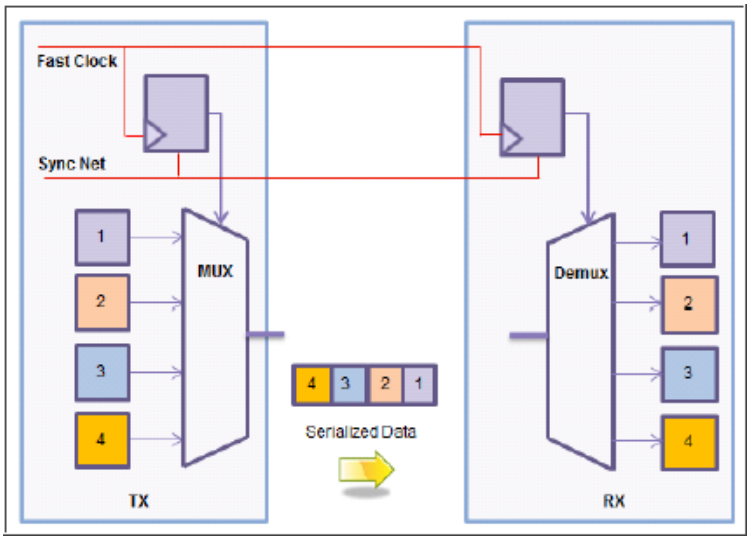
The partitioning log file and *prf* report files contain useful information about the partitioning run. You can build scripts that leverage the standard syntax in these reports and use them as pseudo commands to generate custom reports. Some of the commands are the same as those in the *pcf* constraints file.

The following table highlights some of the report syntax you can leverage. For details about the report syntax, refer to Configuration Commands for Partitioning Reports.

To check for ...	**Report Syntax**
Pin names, trace names	*target_system*
Dissolved modules	*dissolve*
Logic assignments: implied assignments, cell assignments, random logic assignments	*assign_cell*
Port assignments	*assign_port*
Unrouted nets (*-unrouted*), feedthroughs (*-feedthrough*), pin multiplexing ratios (*-ratio*)	*global_route*
Net splitting (*-split*), feedthrough insertion (*-feedthru*)	*modify_net*(After system route)
Clock routing assigned to global clock buffer port bins, connections to system PLLs (*-pll*)	*assign_clock*(After system route)
Trace assignments	*assign_trace*(After system route)

Using Time Domain Multiplexing (TDM)

Partitioning an SoC design requires the interconnect signals of the SoC to be split across multiple FPGAs. You must take into consideration how to reconnect signals across FPGA boundaries and synchronize the FPGAs, so that the FPGAs function in the same way as they would in a single SoC. Typically the number of traces and pins required for the interconnect exceed the available I/Os on the FPGA. This results in interconnect congestion and bottlenecks. The solution to reducing congestion and the number of inter-FPGA I/O pins required, is to multiplex the pins.



The tool lets you specify time-domain multiplexing or TDM. This section describes these multiplexing models: HSTDM (high-speed TDM), HSTDM ERD (HSTDM with error detection). It covers the general TDM controls to set for all three models, as well as specific controls needed for each one.

See the following sections for details:

- Setting General Controls for TDM
- Using HSTDM and HSTDM ERD
- Estimating System Frequency Based on HSTDM Delays
- Analyzing HSTDM Results
- Using HSTDM Top I/O Flow
- Using Multi-Gigabyte Transceivers for TDM
- Enhancing Post-Partition Performance

Feedback

Setting General Controls for TDM

You can specify various time-domain multiplexing controls in the *pcf* file. See Partition Constraint File Tcl Commands for detailed descriptions of the commands mentioned here.

1. Set attributes for traces with the `trace_group_attribute` command in the *pcf* file.
2. In the *pcf* file, specify the TDM model with the `tdm_control -type` command.

   You can set it to one of the following, with HSTDM being the default:

   - HSTDM (High-speed time-domain multiplexing). This is the default mode. It uses source-synchronous differential signaling to eliminate the effects of skew and provide faster multiplexing speeds and increased pin capacity. It requires a HAPS board to be specified.
   - HSTDM_ERD (High speed time domain multiplexing with error detection). This mode is similar to HSTDM, but the transmitter uses some of the bandwidth for sending checksum information with the user data. The receiver then separates and verifies the checksum information and, sets flag for checksum errors. The errors are subsequently processed by the HSTDM monitoring feature in the debugger through the UMRBus. The HSTDM *xx_ERD* module type reflects the number of available data signals.
   - DIRECT No multiplexing
3. Optionally, set TDM ratios with *tdm_control -min_ratio* and *tdm_control -max_ratio* in the *.pcf* file.
   - Specify the `run system_generate` command, and check the slack in the *design_top_time_est.srr* file generated after the command has run.
   - Select a TDM ratio using the options described next. The tool sets a default ratio based on the model you selected, but you can use these options to set specific limits. For more about TDM ratios, see HSTDM Ratios.

     For HSTDM designs, you can estimate the system clock frequency by starting with the TDM ratio and adding the trace delay and user logic delay. These delays can vary greatly from design to design, depending on the number of partitions, amount of logic between flip-flops, and so on. See Estimating System Frequency Based on HSTDM Delays for details about TDM.
   - Set maximum and minimum limits with these options:

     ```
     tdm_control -min_ratio 8 -max_ratio 16
     ```

- If you want a specific ratio instead of a range, set the maximum and minimum to the same value.
- Set a higher TDM ratio for paths with higher positive slack and a lower ratio for paths with lower positive slack.

4. Set the scope for TDM.
   - If you did not specify a TDM ratio as described in the previous step, you can use the `tdm_control -allowed_modules` command to specify all the module types that are to be considered for TDM.
   - Qualify nets for TDM with the `tdm_control -qualification_mode` command.

     Analyze the slack based on the TDM ratio selected. The slack must be positive, both at the start and end points of the path. The best paths for qualification start and end at flip-flops. The next best choices either start or end at a flip-flop. Avoid feedthroughs. Typically you also exclude reset paths, clocks, critical paths with very high frequencies, and paths with negative slacks after TDM insertion.
   - Override the general qualification by specifying the `net_attribute -tdm_qualified 0|1` command for individual nets or groups of nets. These settings on individual nets are always honored during the system route phase. I
5. Set other model-specific controls.

   See Using HSTDM and HSTDM ERD for details.
6. Calculate cabling requirements:
   - Check the partitioning report for the number of traces required between FPGAs.
   - Calculate one trace for each non-qualified net.
   - For qualified nets, the number of traces depends on the TDM ratio and signal direction. Remember to count two traces per channel for differential (diff) and one trace for single ended (SE).
7. To manually specify that TDM not be used between certain FPGAs, set the PCF *trace_group_attribute* to *DIRECT*.

   Use this when you need more control over net routing; for example when you do not want to use TDM for low slack nets. *DIRECT* changes the default assignment, and uses direct connections instead of TDM between the specified FPGAs.
8. Run partitioning and system route.

- See Partitioning the Logic for details about the partition run.
- To influence the TDM results, specify the *-optimization_priority* and *-estimate_timing* options with the `run system_route` command. For other information about running system route, see Running System Route for the Top Level. The following table summarizes system route options that affect TDM; the following option settings do not override any *net_attribute -tdm_qualified* settings on individual nets.

Goal	Option Settings	Effect on TDM
Fast run time	*optimization_priority tdm_ratio estimate_timing 0*	Optimizes the worst TDM ratio. Honors the TDM qualification mode set with *tdm_control -qualification_mode*. The default is *all*.
Get slack values on inter-FPGA paths	*optimization_priority tdm_ratio estimate_timing 1*	Optimizes the worst TDM ratio. Estimates timing to generate the slack values on inter-FPGA paths and adds the information to the TDM qualification report. Honors the TDM qualification mode set with *tdm_control -qualification_mode*. The default is *all*.
Best performance	*optimization_priority multi_hop_path estimate_timing 0*	Performs slack-based automatic TDM insertion and multi-hop optimization. Ignores the user-set TDM qualification mode.
Fast runtime and performance estimate without clock constraints	*optimization_priority multi_hop_path estimate_timing 1*	Performs TDM insertion based on board delays, and optimizes the worst TDM ratio including multi-hop paths. Uses actual cable delays and a slack value of 0 for inter-FPGA paths. Ignores the user-set TDM qualification mode.

Using HSTDM and HSTDM ERD

HSTDM is the default model for pin multiplexing. HSTDM uses source-synchronous clocking for data, and synchronizes the data using circuit training. HSTDM ERD is similar to HSTDM, but includes error detection.

To use HSTDM or HSTDM ERD, do the following:

1. Set TDM controls like TDM ratios.

   See Setting General Controls for TDM.

2. Set HSTDM-specific controls, including HSTDM training.

   See .

3. Partition the design as usual.

4. Analyze results.

   See Analyzing HSTDM Results.

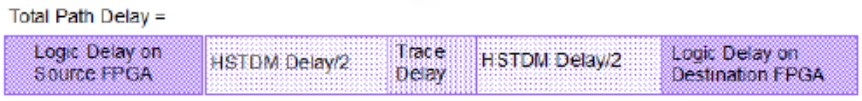
Estimating System Frequency Based on HSTDM Delays

Estimating system frequency requires that you estimate total path delays. Delay calculations for sequential-to-sequential paths with TDM must take trace delay into account, in addition to the user logic delay and HSTDM delay.

The following procedure and subsequent tables illustrate how to compute the total delay of paths with HSTDM to estimate system frequency.

1. When estimating system frequency, make sure that the system period is greater than the total delay of the path with HSTDM.

   The following figure shows the components of delay on a path with HSTDM (not to scale):



2. Use realistic estimates for the user logic delay.

   The data path delay on user logic is heavily dependent on the eventual placement and routing solution. When calculating the clock period, it is recommended that you leave enough margin to compensate for the potential extra delay introduced by place-and-route variances.

3. Estimate the HSTDM ratio and bit rate.

   See the following for more information:

HSTDM ratio	HSTDM Ratios
Bit rate	Recommended Bit Rate Settings

For mixed system configurations, use the recommended bit rate that it is supported by both systems.

4. Find the sum of the HSTDM delay and trace delay.
   - HSTDM delay is based on the ratio and bit rate.
   - Add trace delay based on cable length provided in FPGA-to-FPGA Trace Delay Table. See FPGA-to-FPGA Trace Delay Table for more information.

     Trace delay is the inter-FPGA pin-to-pin delay and does not include the input and output buffer delays. The buffer delays are included in the HSTDM delay.

Constraining HSTDM Timing Paths

There are three kinds of HSTDM paths: user logic to user logic; intra-HSTDM paths, and paths between HSTDM and user logic (to/from). The most critical HSTDM timing paths are usually the paths between the HSTDM and the user logic.

The following procedure outlines the steps needed to set up HSTDM timing paths.

1. Runtime budgeting to create constraints between the user clock and the HSTDM clock.

   The user clock and the HSTDM clock are asynchronous, so define timing exception constraints for them before the design goes through synthesis, place, and route. The tool creates DPO (data path only) constraints, which are like maximum delay constraints but are independent of clock delay.

2. Check these *set_datapath_only* constraints for negative values.

   The paths have negative values when the period is less than the TDM delay. Negative values affect place and route quality and runtime. Try changing the clock period to a more reasonable value, and rerun the design stage.

3. Check timing closure on intra-HSTDM timing paths.

   There are two primary clocks in the HSTDM IP: a 550 MHz bit clock used by IOSERDES and a 137.5 MHz word clock used by the HSTDM logic to clock the user interface. The bit clock is sent to the receiver logic on the FPGA as a source-synchronous (SS) clock. The tool automatically ensures word clock timing.

HSTDM Ratios

These are some guidelines to follow when setting HSTDM ratios:

- All paths are not critical, so they do not all need to be fast; for example, false paths, asynchronous clock crossings, slow clocks, and debug paths.
- You can mix HSTDM ratios. The source sync clock is shared across ratios.

- Match TDM ratios to the slack on the path. Use lower ratios on critical paths and higher ratios on non-critical paths.
- Free up traces by using higher ratios on non-critical paths.
- An HSTDM ratio multiplexes the number of nets specified in the ratio, and each original signal needs a constraint. Each net or bit requires its own DPO constraint, and each clock edge needs a different DPO constraint.
- The default bit rate of 1400 Mbps does not support a ratio of 4; this bit rate only supports ratios of 8 and above. To use a ratio of 4, change the bit rate to 1200.

Summary:

- HSTDM ratio 4 is supported only at speeds of 1200 Mbps or lower.
- For 1400 Mbps and above, only ratios of 8 or higher are supported.
- This limitation is due to tool and hardware constraints to ensure reliable operation.

Recommended Bit Rate Settings

Recommended bit rate settings for different configurations vary, depending on the HAPS system configuration and cable type. When using a global bit rate for chained HAPS systems that are part of the same design, the recommended setting is the lowest of the recommended bit rates, even if that cable is not used for HSTDM.

To enable bit rates based on cable type set the PCF control *tdm_control -hstdm_auto_bitrate_selection* to *1*. The *system_route.log* shows that HSTDM Bitrate is set to auto and HSTDM Bitrate per Cable as on.

HSTDM Delay Tables

HSTDM delay is primarily a function of ratio and bit rate.

The values are subject to change with the release, depending on ongoing enhancements. HSTDM support ratios up to 256, while HSTDM ERD supports up to 240.

FPGA-to-FPGA Trace Delay Table

Trace delay is predictable, because it is a function of HT3 cable length. Trace delay here refers to more than just cable delay; it includes the FPGA pin-to-connector, cable delay, and connector-to-FPGA-pin delay.

The following table shows approximate delay values for different *CON_CABLE* lengths. For the exact values for your hardware setup, consult the appropriate hardware documentation, which lists cable delays.

CON_CABLE Length (cm)	25	50	100	150	200	300	400

Total Trace Delay (ns)	4	5	7	10	12	16	21

Add the total trace delay to the HSTDM delay in *HSTDM Delay Tables*.

Analyzing HSTDM Results

Use this procedure to check the results after HSTDM.

1. After routing, go to the *vivadoImplementation/HSTDM_Qor_rpt* directory.

   The tool generates reports from the post-routing DCP database, and puts them in the *HSTDM_Qor_rpt* directory. If the tool cannot open the DCP files, it does not generate the reports.

   The directory contains two kinds of reports:

   - Clock-oriented reports, which report problems between two clocks
   - Channel-oriented reports, which indicate HSTDM channels with problems.

2. Check the following clock-oriented reports:

clock_summary.rpt	Generated if the design contains HSTDM logic.
clock_summary_from_to_HSTDM_path_issues.rpt	Generated if there are timing violations between user and HSTDM logic. Paths between UMR clock and HSTDM clock are considered as from/to HSTDM paths.
clock_summary_intra_HSTDM_ path_issues.rpt	Generated if there are timing violations within HSTDM logic. It also includes multi-hop paths (from HSTDM receiver to sender).
*timing_from_to_hstdm*and *timing_intra_hstdm*directories	Detailed clock-to-clock reports.

   Report names in the *timing_from_to_hstdm* and *timing_intra_hstdm* directories follow this scheme: *timing_fromClk_to_toClk<Suffix>.rpt*. The report does not have a *suffix* if there are no problems. The table lists the suffixes:

Suffix	File Description
_VIOLATED_max	Hold timing violations between clocks
_VIOLATED_min	Setup timing violations between clocks
_ASYNC	Untimed paths and unapplied constraints because the paths are between clocks in asynchronous groups.

Check the following reports in the *timing_intra_hstdm* directory.

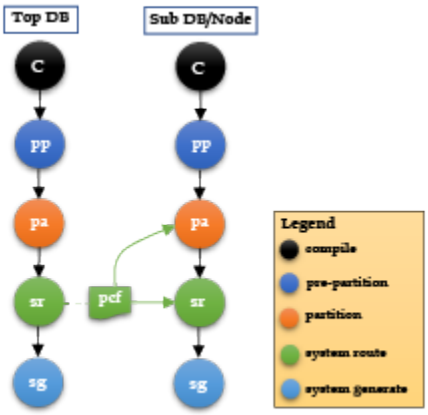
File	Description
timing_cpm_rcv_to_cpm_snd<suffix>.rpt	Multi-hop path report
timing_hstdm_mux_from_rcv<suffix>.rpt	Timing report for HSTDM receiver MUX
timing_hstdm_mux_to_snd<<suffix>.rpt	Timing report for HSTDM transmitter MUX

3. Check the following reports in the *HSTDM_constraints* directory for problems with the HSTDM channels:

File	Description
HSTDM_constraints_async_clk_grp.xdc	Reports asynchronous clock group problems.
HSTDM_constraints_missing.xdc	Reports missing constraints if the Report Missing Constraints option was enabled. problem.This file reports the name of the HSTDM receiver/sender module and the pin index of the problem paths.
HSTDM_constraints_too_tight.xdc	Report missing constraints that cause timing violations. This file reports the name of the HSTDM receiver/sender module and the pin index of the problem paths.

Using HSTDM Top I/O Flow

The HSTDM top I/O flow allows you to work on different subdesigns in parallel with the top-level design and then place them together. To set up a top I/O flow, identify the modules (subsystems) and mark them as top I/O, then run the regular flow on the two databases/nodes.



Follow these steps to run the top I/O flow on a design.

1. Identify the modules to be selected as candidates for a top I/O flow from the top-level design.
2. Mark the modules as black boxes in the compile stage using this compiler directive

   ```
   define_directive {v:subsystemName} {syn_subsystem} {1}
   ```

3. Run compile on the top-level design.
4. Run pre-partition on the top-level design. Use the same TSS file for implementing both the main and the subsystem databases.
   - In the partition stage include a PCF file defining the module identified as a top I/O.

     ```
     subsystem_top_io -cell subsystemName -bin fpgaBins
     assign_cell mainInstance fpgaBins

     #set -ignore to 1 to ignore global clocks
     subsystem_top_io -pinsubsystemClk -ignore 1
     ```

   - The FPGA bins selected for implementing the sub-system are dedicated to the sub-system. Do not assign any other logic to those FPGAs.
5. In the system route step use the same PCF file as for partitioning.
   - Always use the slack mode optimization priority so that the tool exports timing information.
   - Export two PCF files for the *system_route node* which are used while implementing the subsystem database/node.

     ```
     #contains clock and timing information
     subsystem_top_io_ subsystemViewName_clk.pcf
     ```

Feedback

```
#containing HSTDM trace/ratio information
subsystem_top_io_subsystemViewName_loc.pcf
```

6. Run system generate.

   Bit files are generated for the main design FPGAs.

Subsystem Database/Node

1. Set the subsystem as top-level and run compile.
2. Run pre-partition using the same TSS file used for implementing the main design.
3. In the partition stage include the PCF file exported from the system route stage of the main design.

   This file defines the HSTDM trace, ratio, and clock information of the main design.

   Lock all other FPGA bins except the FPGAs used while running the subsystem's partition flow. Use the following pcf command to lock all other FPGAs automatically except those specified in the command.

   ```
   subsystem_top_io -use_bins {subsystemFpgas}
   ```

4. In the system route step use the same PCF file exported from the system-route stage of the main database.
5. Run system generate.

   Bit files are generated for the subsystem FPGAs

PCF Commands for Setting Up Top I/O (Team Based Design) Flow

To define subsystems:

```
subsystem_top_io -cell {subsystemInstanceName} -bin {fpgaBins}
```

To ignore global clock pins:

```
subsystem_top_io -pin {subsystemClk} -ignore 1
```

To lock all unused FPGAs automatically:

```
subsystem_top_io -use_bins {subsystemFpgas}
```

Exporting the PCF File from the Tool

The PCF file exported from the tool contains routing and timing information.

For *subsystem_top_io_subsystemViewName_clk.pcf*:

```
subsystem_top_io $types(instanceName) [objName {instanceName} {netName]
-bin {fpgaBin}
-qualified <tdm_qual 1/0> -clock_event {clkName:edge:delay:hop}
```

For *subsystem_top_io_subsystemViewName_loc.pcf*:

```
subsystem_top_io $types(instanceName) [objName {instanceName} {netName]
-bin {fpgaBin}
-qualified <tdm_qual 1/0> -trace {traceName} -tdm_slot {slotName}
-tdm_module {tdmRatio}
```

Limitations

Currently, there are some limitations to the HSTDM Top I/O flow.

- Gated clock and reset passing from main design to subsystem.
- Identification of cut clocks.
- Reset synchronization across subsystem FPGAs (both from top-level port and reset-generator net).
- Bidirectional bus passing across subsystem and the main design.
- A complete timing picture is not available since the system is not presently timing aware.
- Clock gate replication in the subsystem FPGA.
- SLR partitioning.
- Subsystem FPGA feedthroughs.
- Reports requiring information from the subsystem are not correct.
- Unified Compile flow with Verilog wrapper over VHDL RTL.

Using HSTDM Top I/O Flow with Debug Support

Debug insertion in the HSTDM top I/O flow is supported using $dumpvars at the design subsystem level. The following are the guidelines to run the flow:

- If you want to cross-trigger between subsystems, add the RTL cross-triggering in the design.
- Edit the TSS file to configure the complete system.
- In synchronous GSV, each clock control module (CCM) can be instantiated into individual subsystems. During Runtime, each subsystem should work independent as a separate design.

Debug Support Limitations

Currently, there are some limitations:

- Fast Track debugging of the design is not supported.
- Debugging the design using the Verdi tool is not supported.
- Dynamic force is not supported.
- See existing limitations of the HSTDM Top I/O flow.

Using Multi-Gigabyte Transceivers for TDM

To significantly improve TDM speeds on HAPS-100 modules, use the multi-gigabyte transceivers (MGBs). You can run MGTDM (multi-gigabyte TDM) automatically or manually. Additionally, there are two modes to choose from: default mode at 12.5 Gbps, or performance mode at 20 Gbps.

1. Define the MGB2 connections for MGB2 quads in the TSS file.

   A channel is 1 GTY/GTX transceiver (1 Tx and 1 Rx pair). A quad is a group of two GTY/GTX transceivers.

   - To automatically use fixed traces, do not specify anything. There are 5 fixed quads per HAPS-100 module for fixed transceiver interconnect, but only quad 226 is used for MGTDM. Quad 226 is a full quad with four channels; quads 220, 225, 230 and 235 are half quads.
   - In addition to the fixed traces, you can also specify connectors from the MGB2 quad connectors. HAPS-100_4F modules have 6 quads connected to MGB2, so there are 24 MGB2 connectors available for MGTDM in addition to the fixed interconnect. All MGB2 quads are full quads, with 4 channels per MGB2. The quads that are connected to MGB2 are shown below.

FPGA	MGB Quad Connectors
A	*AM221, AM222, AM231, AM232, AM236, AM237*
B	*BM221, BM222, BM231, BM232, BM236, BM237*
C	*CM221, CM222, CM231, CM232, CM236, CM237*
D	*DM221, DM222, DM231, DM232, DM236, DM237*

Use the connector names from the previous table to manually specify MGTDM cables in the TSS file. Do not specify fixed quad 226, which the tool uses automatically. For example:

```
board_system_create -interconnect -manual CON_CABLE_100_MGB2 -name
mgb1 -connector {FB1.AM221 FB1.BM221}
```

- Make sure to use the correct cable for the chosen speed grade and planned MGTDM mode. See this table:

Speed Grade				
FPGA 1	**FPGA2**	**Cable Length**	**Performance Mode**	**Default Mode**
-1	*-2*	*100 cm*	Not Supported	Supported
-1	*-2*	*200 cm*	Not Supported	Supported
-1	*-1*	*100 cm*	Not Supported	Supported
-1	*-1*	*200 cm*	Not Supported	Supported
-2	*-2*	*100 cm*	Supported	Supported
-2	*-2*	*200 cm*	Not Supported	Supported

2. For MGTDM default mode (12.5 Gbps), specify these constraints in the PCF file:
   - Enable MGTDM with this command:

     ```
     tdm_control -enable_mgtdm 1
     ```

     To disable MGTDM insertion, set the constraint to 0:

     ```
     tdm_control -enable_mgtdm 0
     ```

   - Also specify the inter-FPGA nets for MGTDM with PCF constraints:

     ```
     net_attribute {list_of_nets} -function MGT -tdm_group
     {MGTDM_module_with_ratio}
     ```

   The MGTDM module is called *HSTDM_MGT*. These are examples of the PCF constraint:

   ```
   net_attribute {u4.dat1[39:0] u4.dat2[39:0] u4.dat3[39:0]}
   -function MGT -tdm_group HSTDM_MGT_256 net_attribute {u4.dat1[39:0]
   u4.dat2[39:0] u4.dat3[39:0]} -function MGT -tdm_group HSTDM_MGT_128
   ```

The MGTDM delays for various TDM ratios in default and performance modes are shown below:

Ratio (Fixed Traces)	MGTDM Delay: Default	MGTDM Delay: Performance
64	*95*	*63*
128	*103*	*69*
256	*113*	*75*
512	*133*	*87*
1024	*173*	*113*

3. For MGTDM performance mode (20 Gbps), set this PCF constraint instead of the ones described in the previous steps:

   ```
   tdm_control -low_latency_mgtdm 1
   ```

4. To run MGTDM in automatic mode, do the following:
   - Specify this constraint in the PCF file:

     ```
     tdm_control -enable_mgtdm auto
     ```

     The tool runs MGTDM automatically in default mode, using the fixed trace (quad 226) for MGTDM.
   - To run the tool automatically in performance mode, specify these PCF commands:

     ```
     tdm_control -enable_mgtdm auto tdm_control -low_latency_mgtdm 1
     ```

5. Run system route with the following optimization options:

   ```
   run system_route -optimization_priority multi_hop_path
   -estimate_timing 1
   ```

6. Check results.
   - If the design was run with distributed synthesis, you can check the *constraints_application.srr* log file to see which quads are used. If distributed synthesis was not used, this information is available in the pre-map log file, *pre_map.srr*.

Feedback

```
Multi-gigabit transceiver TDM report for FB1_uA
================================================================
MGTDM Info:
Quad_id Direction       Channel_Id      Bit rate(Mbps)  Ratio   Trace Type
----------------------------------------------------------------
221     Receive         3               12500           256     MGB2
221     Transmit        0               12500           256     MGB2
221     Transmit        2               12500           256     MGB2
221     Receive         1               12500           256     MGB2
221     Receive         2               12500           256     MGB2
221     Transmit        1               12500           256     MGB2
221     Transmit        3               12500           256     MGB2
221     Receive         0               12500           256     MGB2
222     Transmit        3               12500           256     MGB2
222     Receive         1               12500           256     MGB2
222     Transmit        1               12500           256     MGB2
222     Receive         3               12500           256     MGB2
222     Transmit        2               12500           256     MGB2
222     Transmit        0               12500           256     MGB2
222     Receive         0               12500           256     MGB2
222     Receive         2               12500           256     MGB2
226     Transmit        0               12500           256     FIXED
226     Receive         3               12500           256     FIXED
226     Receive         2               12500           256     FIXED
226     Transmit        2               12500           256     FIXED
```

- Check the Global Route Ratio Report (AP366) for the MGTDM ratio, trace usage, and bit rate.

```
@S3.1 AP267 |Post-Optimization and Cable Assignment Global Route Report

@S3.1.1 AP366 |Global Route Ratio Report

Connection            Trace_Usage Net_Usage              Ratios   Bitrate Cablelength
                                        TDM DIRECT CLOCK          in Mbps in cm
-------------------------------------------------------------------------------------------
FB1.uA<->FB1.uB            25/26         0     25     0
FB1.uA<->FB1.uB (MGT)        4/8      1920      0     0 512      12500
FB1.uC<->FB1.uD             0/81         0      0     0 Not used
===========================================================================================
```

- Check the post-optimization routing summary (AP635) for details about the TDM module and post-TDM clock range.

```
@S3.1.4 AP655 |Post-Optimization Routing Summary
Connection       Trace Usage  No. of Net     TDM Module              Available Slack Range  Mean Slack
                              failed timing                          (Post TDM)
-------------------------------------------------------------------------------------------------------
FB1.uA<->FB1.uB  25/26
                              0/25           NONE (4 ns)             1190 to 1195 ns        1190 ns
FB1.uA<->FB1.uB  4/8
                              0/1920         HSTDM_MGT_512 (132 ns)  1066 to 1066 ns        1066 ns
=======================================================================================================
```

Enhancing Post-Partition Performance

Partitioning a design across multiple FPGAs makes the inter-FPGA path as a critical path. This results in interconnect bottlenecks and reduces partition performance. To reduce the inter-FPGA path delay, perform retiming of registers in the inter-FPGA path.

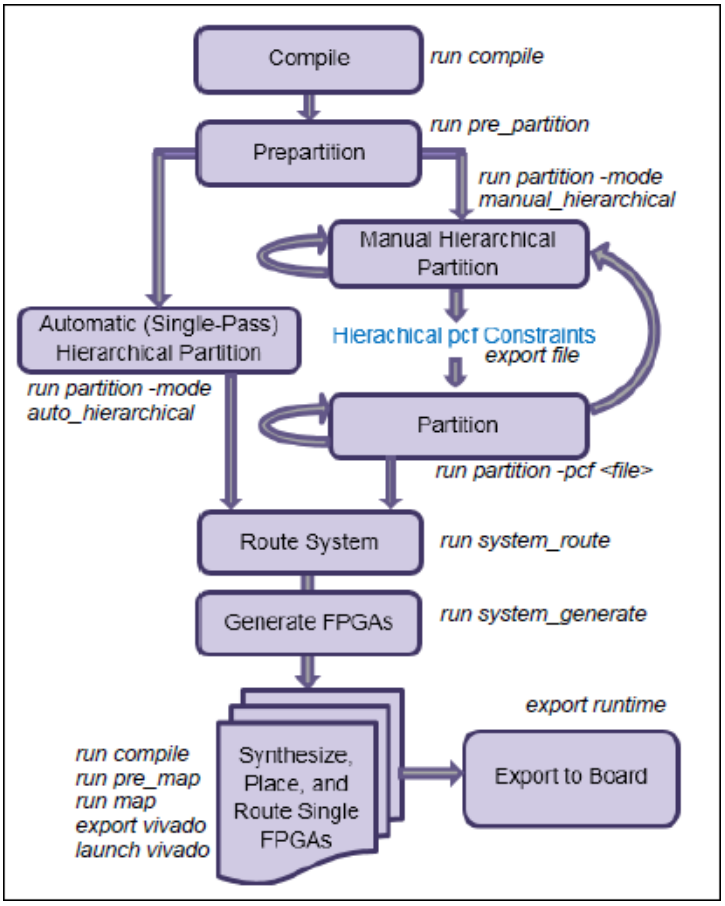
Feedback

To perform retiming of registers and improve the post partition performance, use the *-inter_fpga_path_retiming* option in the `run system route` command. See run system_route.

Using Hierarchical Partitioning

For complex prototypes with eight FPGAs or more, use hierarchical partitioning to break down the problem into smaller pieces. Hierarchical partitioning is based on the concept of super-bins. A super-bin consists of the FPGAs on a particular *board* on the prototype. The idea is to first partition the multi-FPGA prototype into board-level super-bins, and then use the constraints generated by this first step to complete partitioning down to the FPGA level.

You can run hierarchical partitioning automatically or manually. In automatic mode, the tool runs single-pass recursive partitioning instead of the two-pass methodology required for manual mode.



Feedback

See the following topics for details:

- Running Hierarchical Partitioning in Automatic Single-Pass Mode
- Running Top-Down Hierarchical Partitioning in Manual Mode

Running Hierarchical Partitioning in Automatic Single-Pass Mode

Use this procedure to run top-down hierarchical partitioning automatically, in a single-pass recursive mode.

1. Define the configuration in the TSS file and complete the compile and pre-partition stages.
2. Define FPGA superbins.

   By default the partitioner creates *superbins* from each “board” when it runs in hierarchical mode. You can use either a basic or detailed TSS file, depending on where you are in the design cycle. The following TSS definition automatically creates superbins for FB1 and FB2:

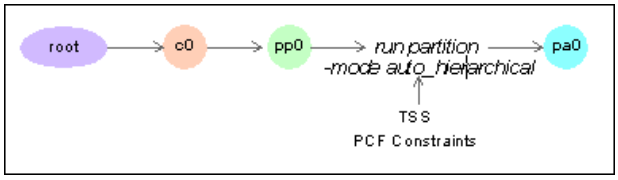
   ```
   board_system_create -add HAPS-100_4F -name FB1 # Creates FPGA bins
   {FB1.u[A-D]} board_system_create -add HAPS-100_4F -name FB2 # Creates
   FPGA bins {FB2.u[A-D]}
   ```

   You can change the defaults and define your own superbins using *pcf* constraints:

   ```
   hierarchical_super_bin -name g1 {fb1.uA fb1.uB fb1.uC}
   hierarchical_super_bin -name g2 {fb1.uD fb2.uA fb2.uB}
   ```

3. Run partition, specifying automatic hierarchical partitioning with the *-mode* argument.

   ```
   run partition -mode auto_hierarchical -tss tss1.tcl -pcf clocks.pcf
   -pcf ports.pcf
   ```

   The tool automatically runs recursive hierarchical partitioning and generates a constraint file with port and cell assignments.

4. To reuse these constraints, export the file:

   ```
   export file assignments.pcf
   ```

5. Continue with the rest of the partition flow, by routing system traces and generating FPGA partitions.

Feedback

For additional information about the commands and tasks mentioned in the preceding procedure, refer to the following topics:

- Partitioning the Logic
- Target System Specification Commands
- run partition

Running Top-Down Hierarchical Partitioning in Manual Mode

Use this procedure to run top-down hierarchical partitioning in a two-pass manual mode.

1. Define the configuration in the TSS file and complete the compile and pre-partition stages.
2. Define FPGA superbins.

   By default the partitioner creates *superbins* from each “board” when it runs in hierarchical mode. You can use either a basic or detailed TSS file, depending on where you are in the design cycle. The following TSS definition automatically creates superbins for FB1 and FB2:

   ```
   board_system_create -add HAPS-100_4F -name FB1 # Creates FPGA bins
   {FB1.u[A-D]} board_system_create -add HAPS-100_4F -name FB2 # Creates
   FPGA bins {FB2.u[A-D]}
   ```

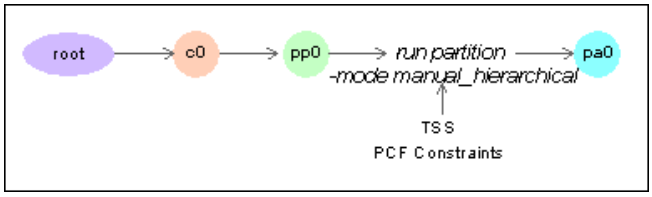
   You can change the defaults and define your own superbins using *pcf* constraints:

   ```
   hierarchical_super_bin -name g1 {fb1.uA fb1.uB fb1.uC}
   hierarchical_super_bin -name g2 {fb1.uD fb2.uA fb2.uB}
   ```

3. Run a first pass of hierarchical partitioning.

   Start with a pre-partitioned database state, and run the partitioner in manual hierarchical mode (*-mode*) and the appropriate TSS and *pcf* files. For example:

   ```
   run partition -mode manual_hierarchical -tss my_tss.tcl -pcf GCLKS.pcf
   -pcf abstract.pcf
   ```

   After this pass of hierarchical partitioning, the tool generates a constraint file that constrains all the cells in the design to the bins that make up the superbins. The output

for a cell is constrained by its assignment to a superbin and any user constraints you might have set. Replication constraints are directly copied to the output constraints. The superbin assignments are in the *cell_constraints.pcf* file.

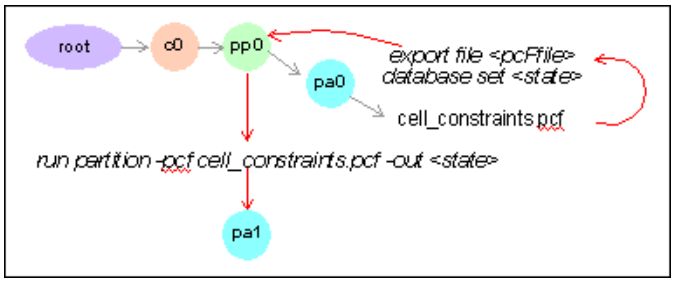
4. To reuse hierarchical files generated from this database state, first export the constraints:

   ```
   export file cell_constraints.pcf
   ```

5. Rerun partitioning with the hierarchical constraints.

   The following example shows the commands used to rerun partitioning, with the red arrows indicating the second run with reused constraints:

   ```
   database set pp0 run partition -pcf cell_constraints.pcf -out pa1
   ```



6. Continue with the rest of the partition flow by routing system traces and generating FPGA partitions.

   For additional information about the commands and tasks mentioned in the preceding procedure, refer to the following topics:

   - Partitioning the Logic
   - Target System Specification Commands
   - run partition

Running System Route for the Top Level

After you have partitioned the design, the next step is system routing at the inter-FPGA level. During this phase, the tool assigns inter-FPGA nets to HAPS board traces and performs TDM as needed. This is the stage where the partitions are actually created. After you route the system level for the first time, the tool generates a database state called *sr0*.



You can skip this step and go directly from partitioning to generating the individual FPGA partitions, but you might have more pins than are allowed. You can skip this step if you are exploring different solutions.

1. Start with a partitioned database state that meets your goals.

   See Partitioning the Logic for details.

2. Gather other input for system routing.

   The files mentioned below are specified from the command line in the next step.

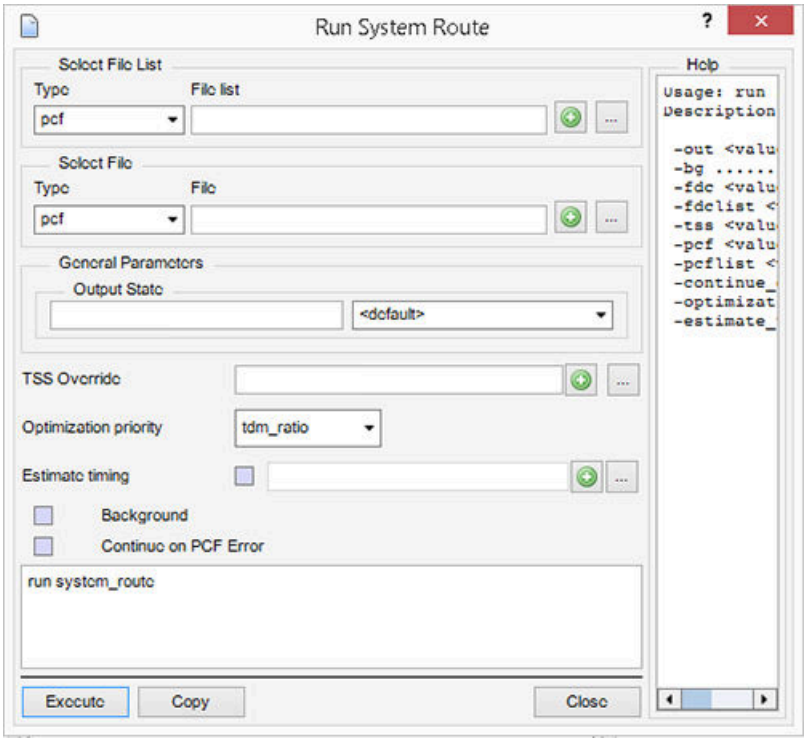
   - Use the target system information (*tss* file) from the report generated after partitioning to see the pin and trace name assignments.
   - Specify routing constraints in a *pcf* file. See Configuring PCF Constraints for Partitioning. Add constraints as needed, but do not make drastic changes or the results might not be routable. For example, add keepbufs for feedthroughs, add net attributes to control post-partition routing, separate ports and nets for nets that are routed on two different channels, and define end pins for differential clocks.

     System route requires that a *pcf* file with clock and reset information be specified.
   - Optionally, specify an FDC file with constraints for compiling and mapping the partitions. Generates a *syntax_constraint_check.rpt* file with the *report constraint_check-syntax_only* command, and make sure that all constraint syntax has been specified correctly.
   - Set other options as needed in a Tcl options file, and source the Tcl file.

3. Use the *run system_route* command to create system-level routing.

Feedback

- To use the GUI to run the command, start with a partitioned database, click **System Route**, and specify the command parameters. For descriptions of the dialog box options, see Run System Route Dialog Box.



- If you have *pcf* constraints, specify them with the *-pcf* argument or in the appropriate field in the GUI.
- Optionally, specify an *fdc* constraints file. Some constraints take effect at this stage.
- To specify timing-aware optimizations for TDM and multi-hop paths, use the *-estimate_timing* and *-optimization_priority multi_hop_path* arguments with the *run system_route* command. See Setting General Controls for TDM for a tabular description of the effect of these options. Use the *net_attribute -tdm_qualified* command in the *pcf*file to override these general option settings and qualify nets for TDM on a per-net basis.

See run system_route for details about the command and its arguments.

The tool routes the inter-FPGA traces for the whole system. This includes TDM with ratio assignment and grouping, and detailed routing assignments. If you specified multi-hop optimizations, it runs timing-aware optimizations to reduce the number of hops between FPGAs.

4. Analyze results.

- Specify the *report target_system -tss tssFile.tcl* command to generate the report file. Run *view target_system.rpt* to view the report. For more information, see Checking the Target Specification (TSS).
- Check the slack value in *system_route.rpt* for each net.
- Check *timing.rpt* for updated values. The report contains FPGA names and timing.
- Check the updated TDM report: *tdm_qualified_timing.rpt*.

For more information about partitioning reports, see Analyzing Partition Result Files and .

5. To freeze partition results, export the *router_constraints.pcf* file and use it on the next run, as described in Locking Down Partitions.

SLR Congestion Reduction

Beta

After partitioning, use *option set enable_conn_table 1* to reduce congestion. This feature requires beta license. Use the following `run system_route` commands to optimize the SLR crossing:

```
run system_route -mapped_timing_models 1 -optimization_priority slack
```

Or

```
run system_route -estimate_timing 1 -optimization_priority multi_hop_path
```

Pin Table Optimization

In this mode, the ProtoCompiler tool creates a FPGA pin connectivity table based on the design. This includes connectivity crossing flip-flops and DUT pins. The tool calculates an estimate of the SLR crossings induced by the router trace assignment. The tool uses the pin table optimization after routing, to reduce the estimated SLR crossings.

Cable Optimization

In this mode, the ProtoCompiler tool conducts SLR auto-cabling and the DUT pins connected to each SLR are redistributed across the SLR in the *System Route* state. The tool uses cable optimization to realign the cable connector pairs within the FPGA to reduce the I/O induced estimated SLR crossings and DUT pins per SLR and reduces congestion.

In the first *System Route* state, use the PCF command `auto_tss_control -enable 1` to generate the *auto_cable.tss* file. This file contains the modified cable connections.

In the second *System Route* state, run the `run system_route` command again with the *auto_cable.tss* file. If the number of DUT pins in the single SLR is high, they are

Feedback

automatically realigned along the modified cables to other SLRs within the FPGA. This reduces the estimated SLR crossings and improve congestion in the FPGAs.

Check the *system_route.log* for the SLR crossing enhanced report:

```
"@S4.7 AP612 |Cable Assignment Optimization " section is system_route.log
 file is enhanced SLR crossing related info.
```

Clock Scaling Support

Beta

Managing high performance during compilation using asynchronous clock domains increases the tool runtime. Therefore, if the clock performance is lowered by selectively scaling the clock period might help to bring down the overall runtime of the tool.

Clock scaling enables clock period adjustments by specifying the clock period scaling factor, using a PCF or a FDC scaling command.

A scaled clock is generated by updating the clock periods and forward-annotate them to the later stages in the partitioning flow (system_route, system_generate, and synthesis stages).

Clock scaling is supported only for:

- Guided partition mode and the optimization priority slack with mapped timing model enabled.
- The Create Clock constraints that are specified in the FDC and not in the related user-specified exception (false path, multicycle path, and so on).

Enable the following commands to ensure consistent timing behavior during clock scaling:

- `-run system_route -estimate_timing 1`
- `-run system_route -mapped_timing_models 1`
- `-report timing -generate`
- `-run system_generate`

Clock Scaling Flow

Clock scaling is enabled in different stages of the partitioning flow. Depending on which stage the scaling has been performed, the tool would implicitly forward-annotate the scaling constraints to the rest of the flow.

Feedback

The following are the recommended use modes:

Note:
In all these use modes, `create_clock` command constraints provided originally in the user-provided FDC should not be modified. Continue to be use the `create_clock` command constraints, provided in the pre-partition, for the rest of the stages in the flow.

Scaling at Partitioning Stage

To scale a clock during the partition stage, use the PCF clock scaling command, timing_control.

```
timing_control -scale_clocks {clock_name1 clock_name2 ...}
```

The *clock_name* is the root clock name as specified in the `create_clock` command constraint, in the user-provided FDC file. The clock names specified in the Create Generated Clock constraints cannot be used for scaling, as these generated clocks are automatically accounted if defined in the FDC constraints.

If the generated clock constraints are not defined in the FDC, then such generated clock constraints would get the same scaled clock period value as that of its root clock period.

One can reiterate the partition node with different scaling factors without having to re-compile the pre-partition node, provided that the original user FDC has not been modified.

For example

```
timing_control -scale_clocks {clk1 4.0 clk2 4.0 i2c_clk 0.3 clkin_by2_neg
1.7 }
```

Scaling at System Route Versus Partitioning Stage

If clocks were scaled during the partition state you can re-scale the same clocks during the system route stage. Scaling performed during system route scales the original user specified fdc create clocks constraints. If a scaling factor is specified in the partition state, it is overridden. Clocks scaled in partition, but not rescaled in system route are honored for the rest of the flow.

The following table illustrates the influence of the scaling factor when scaling is performed at both partition and/or system route stages.

Scaling in Partition state (clock period* scaling factor)	Scaling in system route (clock period* scaling factor)	Expected final clock period after scaling
clk1: 20ns * 0.5 clk2: 100ns * 2	clk1 20ns * 1.5 clk2 100ns * 1	clk1: 20ns*1.5 clk2: 100ns*1

Feedback

Scaling in Partition state (clock period* scaling factor)	Scaling in system route (clock period* scaling factor)	Expected final clock period after scaling
clk1: 20ns * 0.5 clk2: 100ns * 2	clk1 20ns * 1.5 No scaling of clk2	clk1: 20ns*1.5 clk2: 100ns*2

To scale a clock during the system route stage, use the FDC clock scaling command, set_clock_scaling.

```
set_clock_scaling -factor {scaling factor1} {clock_name1 clock_name2 ...}
set_clock_scaling -factor {scaling factor2} {clock_name4 clock_name3 ...}
```

The *clock_name* is the root clock name as specified in the `create_clock` command constraint in the user-provided FDC file. The clock names specified in the Create Generated Clock constraints cannot be used for scaling, as these generated clocks are automatically accounted if defined in the FDC constraints.

If the generated clock constraints are not defined in the FDC, then such generated clock constraints would get the same scaled clock period value as that of its root clock period.

For example:

```
set_clock_scaling -factor {0.5} {clk1 lsio_fpga_i2c_top_chiplet_clk}
set_clock_scaling -factor {1.5} {clk2 clk3}
```

Scaling at System Route Stage

To scale a clock directly during the system_route stage, use the FDC clock scaling command:

```
set_clock_scaling -factor {scaling factor1} {clock_name1 clock_name2 ...}
set_clock_scaling -factor {scaling factor2} {clock_name4 clock_name3 ...}
```

For example:

```
set_clock_scaling -factor {0.5} {clk1 lsio_fpga_i2c_top_chiplet_clk}
set_clock_scaling -factor {1.5} {clk2 clk3}
```

Note:

You need to provide the same FDC file with the scaling command in both the system route and system generate stages.

Clock Scaling Log Messages

After scaling of the clocks, the clock scaling information can be viewed in the partitioning, system route, system generate, and pre-map/map log files.

Feedback

- Clock scaling information can be viewed in the `partition.log` file in `AP1084` messages. The following are the message examples:

```
@N:AP1084 :  | Scaling clock 'clk1' from 125.00 ns to 500.00 ns
@N:AP1084 :  | Scaling clock 'clk_pos' from 250.00 ns to 1000.00 ns
```

- Clock scaling information can be viewed in the system_route stage in the `system.log` and `estimate timing.log` file. The following are the message examples:

```
Clock [clk1] is scaled to period [37.500] (scaling factor 0.300)
Clock [clk2] is scaled to period [66.000] (scaling factor 3.300)
Clock [lsio_fpga_i2c_top_chiplet_clk] is scaled to period [100.000]
 (scaling factor 2.000)
```

- Clock scaling information can be viewed in the system_generate stage in the *Performance Summary* report. The following image shows the message example:

```
@S0.0 |Performance Summary
********************


Worst slack in design: -20.120

                                Scaling     Requested     Estimated     Requested     Estimated                 Clock
Starting Clock                  Factor      Frequency     Frequency     Period        Period        Slack       Type
-------------------------------------------------------------------------------------------------------------------
clk1                            0.300       26.7 MHz      NA            37.500        NA            NA          decla
clk2                            3.300       15.2 MHz      NA            66.000        NA            NA          decla
lsio_fpga_i2c_top_chiplet_clk   2.000       10.0 MHz      4.3 MHz       100.000       234.507       47.648      decla
mb_uB_clk_neg                   0.000       7.6 MHz       3.2 MHz       132.000       309.550       -5.380      gener
mb_uB_clk_pos                   0.000       13.3 MHz      35.6 MHz      75.000        28.104        15.632      gener
System                          0.000       1.0 MHz       204.5 MHz     1000.000      4.891         995.110     syste
===================================================================================================================
Estimated period and frequency reported as NA means no slack depends directly on the clock waveform
```

- Clock scaling information can be viewed in pre-map/map stages in the *Performance Summary* report. The following image shows the message example:

```
@S0.0 |Performance Summary
********************


Worst slack in design: -15.801

                                Scaling     Requested     Estimated     Requested     Estimated
Starting Clock                  Factor      Frequency     Frequency     Period        Period        Slack
-----------------------------------------------------------------------------------------------------------
System_mb_uA                    0.000       1.0 MHz       NA            1000.000      NA            NA
clk1                            0.300       26.7 MHz      3995.2 MHz    37.500        0.250         37.25
clk2                            3.300       15.2 MHz      3995.2 MHz    66.000        0.250         65.75
```

Limitations

The following are the scenarios where the scaling is not supported:

- Guided Partition and mapped timing model with Slack Optimization mode both in partition and system route stages are the only supported modes. Other modes are not supported
- MMCM output clocks with CGC constraints
- Scaling of CCM clocks
- System Level Timing analysis
- Scaling at pre-partition, system_generate, and pre-map stages
- Automated scaling of user DPO and exception. You have to manually scale these values based on the scaled clocks
- Cut clocks and certain `create_clock` command options (`-waveform, -add, -invert`)

Reducing SLR Congestion

Beta

After partitioning, use `option set enable_conn_table 1` to reduce congestion. This feature requires beta license. Use the following run system_route commands to optimize the SLR crossing:

```
run system_route -mapped_timing_models 1 -optimization_priority slack
```

Or

```
run system_route -estimate_timing 1 -optimization_priority multi_hop_path
```

Pin Table Optimization

In this mode, the ProtoCompiler tool creates a FPGA pin connectivity table based on the design. This includes connectivity crossing flip-flops and DUT pins. The tool calculates an estimate of the SLR crossings induced by the router trace assignment. The tool uses the pin table optimization after routing, to reduce the estimated SLR crossings.

Cable Optimization

In this mode, the ProtoCompiler tool conducts SLR auto-cabling and the DUT pins connected to each SLR are redistributed across the SLR in the System Route state. The tool uses cable optimization to realign the cable connector pairs within the FPGA to reduce the I/O induced estimated SLR crossings and DUT pins per SLR and reduces congestion.

Feedback

In the first System Route state, use the PCF command `auto_tss_control -enable 1` to generate the `auto_cable.tss` file. This file contains the modified cable connections.

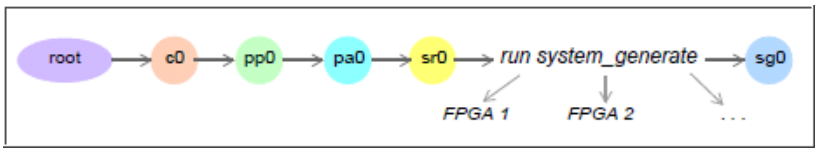
In the second System Route state, run the `run system_route` command again with the `auto_cable.tss` file. If the number of DUT pins in the single SLR is high, they are automatically realigned along the modified cables to other SLRs within the FPGA. This reduces the estimated SLR crossings and improve congestion in the FPGAs.

Check the `system_route.log` for the SLR crossing enhanced report:

```
"@S4.7 AP612 |Cable Assignment Optimization " section is system_route.log
 file is enhanced SLR crossing related info.
```

Generating FPGA Partitions

The *system_generate* stage is the terminal step in the basic partitioning flow. At this stage, the tool automatically creates a timing budget and generates partitions based on the hardware specification in the *tss* file and the *pcf* and *fdc* constraints.



You can also return to the system generate stage after synthesizing individual FPGAs or to back-annotate place and route results for system-level timing analysis.

Use this procedure to generate individual FPGAs.

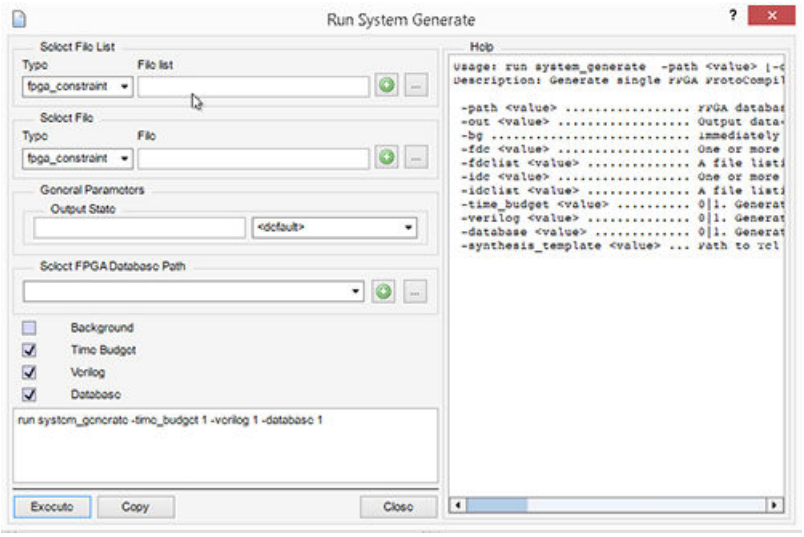
1. Start with a database state that has been partitioned (`run partition`) or where you have run system routing (`run system_route`).
2. Specify other input.
   - Specify additional partitioning constraints as needed in a *pcf* file. See Configuring PCF Constraints for Partitioning. Specify the *pcf* file on the command line in the next step.
   - Set other options as needed in a Tcl options file. Source the Tcl file.
3. Use the `run system_generate` command to partition the design between multiple FPGAs.
   - Specify *fdc* constraints and debug point constraints (*idc*). You must specify the FDC file so that the generated partitions are constrained properly. Generate a *syntax_constraint_check.rpt* file with the `report constraint_check-syntax_only` command, and check that all FDC constraint syntax is correct.
   - Set the *-time_budget* argument to 0 if you do not want timing budgets to be used.

Feedback

See run system_generate for details about the command and its arguments.

The tool creates a directory for each FPGA. If you are using Multi-FPGA DTD, the tool generates an additional device called *dtd2*, which you must treat like any other FPGA. Each FPGA has its own single-chip database, HDL, and constraints. The directory also contains a file with paths to all generated and original HDL files, and a script to run the partitioned single-FPGA design through place-and route.

To use the GUI to run the command, start with a database state where system level routing is complete, click **System Generate**, and specify the command parameters. For descriptions of the dialog box options, see Run System Generate Dialog Box.



Once you have completed the first round of system routing, you can implement individual FPGAs, as described in Implementing Individual FPGAs. You can also simulate the post-partitioned design using simulation-ready RTL logic models. See Running Simulation for details.

You can also return to the system route state after synthesizing individual FPGAs for analysis and backannotation.

See the following for details:

- Analyzing System Generate Results
- Interactive Partitioning Flow
- Working with Inferred Clocks Driving Inter-FPGA Paths

Feedback

Analyzing *System Generate* Results

1. Use the `report syntax_constraint_check` command to check the FDC for the generated partitions.

   View the *syntax_constraint_check.rpt* file for details.

2. Use the `report timing` command to generate timing results.

   - Use *report timing -generate* to generate the report. Timing analysis at this stage always reflects system-level timing analysis (SLTA), and only reports inter-FPGA paths. HSTDM IPs are modeled as through points. The contents of the report vary, depending on the status of the system generate state:

System Generate State	Timing Report Contents
With time budgets	Contains time budget estimates
After mapping FPGAs	Post-synthesis (mapped FPGAs) timing
After backannotating FPGA P&R	Post-P&R timing

   - Specify the kind of SLTA with the *-mode* argument. You can check what modes are available with `report timing -list`. See Running SLTAfor a detailed procedure on running system-level timing analysis.

   - Customize the timing report by using specific arguments with the `report timing -generate` command, like *-from, -rise_to, -slack_margin*, and so on. See report timing for the complete list of arguments.

     You can also use *get_** or *find* query commands to narrow down the report further. For example:

     ```
     report timing -generate -from [get_cells {fb1.uB.wb_add[*]}
     ```

     When you use *-from, -to*, or *-through* filters, remember to use mapped names, not RTL names. RTL instance names can change during mapping; for example, an RTL RAM could have changed from *{i:dest_u4.ram1[7:0]}* to *{i:dest_u4.ram1_ram1_0_0}* after mapping.

3. Check timing budget results.

   The *set_datapathonly_delay* constraint is used to model direct paths between partitions, instead of input and output delays. These constraints are set on I/O ports, for every clock event going through that port instead of just the most critical event. See Time Budgeting and set_datapathonly_delay for details.

- Look for warnings about negative slack and negative timing constraints in the log and budget *fdc* constraint files for the FPGAs. A time budget of 0 indicates that the design is over-constrained and does not meet timing after the partitions are synthesized. This could show up as a requested period of 0 or a DPO of 0 down the line, after mapping.

  Prevent this by checking the time budget log for slack details and for warnings. Some typical causes include clocks with misaligned edges between start and endpoints, missing timing exceptions between user clocks, large arrival times at the startpoint, or large output delays on the receive side.

  There are some techniques to address time budget issues found at this stage. One method is to reduce the requested clock frequency. You can also work on the critical paths. Check the nets considered for TDM, and the TDM ratio used. If possible, remove the nets from TDM, and route them directly. If you need the performance, try adding more cables, or get the target frequency in the ballpark, and then explore advanced partitioning options.

- Check the UMR logic and set exception constraints (false path, multicycle path or max delay) as needed on UMR paths. You cannot change the UMR clocks, which the tool automatically inserts. The UMR clocks are listed in the *Clock Summary* section of the log file.

4. After compile and map, use the *check_hstdm_timing* utility to check for missing time budget constraints.

   For information about using this utility, see Checking Timing Results.

5. Analyze results.

   - Check for warning and error messages.
   - Use the `view schematic time_budget` command to view the results in a schematic. Use `view schematic -list` to see a list of views available.

     See Analyzing Results for information about using analysis tools.

Interactive Partitioning Flow

The interactive partitioner is a Tcl shell in the HAPS automatic partitioner (APTN). You can refine the partitioning process by moving cells into multiple FPGAs and get reduce multihop paths.

Phases in Interactive Partitioning

There are phases in interactive partitioning. Interactive partitioner can be used to fine-tune partition and get impact of specific changes rather than running the entire flow which needs more runtime. Obtain the *FpgaInternalBeta* license to enable this feature.

Feedback

To invoke the interactive partitioner enable the option *-interactive 1,* and provide constraints into pcf files using the `run partition` command.

- The `run partition` command with the *-interactive* option activates the partitioner in interactive mode.
- The ipcf file contains information to run the APTN in interactive mode.

Example syntax:

```
run partition -tss abstract.tss -pcf setup.pcf -pcf abstract.pcf
 -interactive 1 -ipcf ipcfFileName
```

Constraint phase

In this phase, launch the interactive partitioner from the GUI or in batch mode, and provide the setup constraints using *pcf* files. The tool reads the inputs such as the netlist and the *.pcf* files, and the *.ipcf* file. The ipcf file contains information to run the APTN in interactive mode.

Use the `run_flow` command to run the partitioner in interactive partitioning shell. It executes the partitioning algorithm and switches to the next phase, manual partitioning. The ipcf file starts with the `run_flow` command and end with the exit command.

Manual partitioning phase

In this phase, make changes to the partition and analyze the results and the QoR impact. You can go through multiple iterations and refine the results until you are satisfied. The *-ipcf* option is used with the `run partition` command and it contains commands that are needed in the manual partitioning phase. The *-ipcf* option starts with `run_flow` command, which switches the partitioner into manual partitioning phase.

The `exit` command quits the interactive session.

Commands in this phase are different from commands in constraint phase and some of the commands that were available in constraint phase are not available in manual partition phase.

`help` is used to access the entire list of commands.

Feedback

```
>help
assign_cell
assign_port
design_list_cells
design_list_nets
design_list_ports
design_list_properties
dissolve_cell
exit
fix_multihop_path
help
replicate_cell
report_cell_connections
report_cost
report_interconnection_table
report_solution
report_timing
run_clock_replication
run_flow
run_route
run_write_results
solution_commit
solution_discard
solution_restore
solution_save
tss_list_bins
tss_list_functions
tss_list_pins
tss_list_properties
tss_list_trace_groups
tss_list_traces
```

`<command name> -help` gives details of each command. For example:

```
run_flow -help
```

Flow commands	
Command	**Description**
run_flow	Runs the partition algorithm and switches to manual partition phase.
run_route	Runs the global router and prints summary (AP267)
run_write_results	Runs global route and srp build. Required to complete the flow.

exit	Quits interactive session. Returns success if `run_write_results` was called successfully.
Solution commands	
Command	**Description**
assign_cell {cell(s)} {bin}	Assigns one or more cells to a FPGA bin in the temporary solution.
assign_port {ports(s)} {bin}	Assigns one or more ports to a Port bin in the temporary solution.
replicate_cell {cell(s)} {bins}	Replicates one or more cells to multiple FPGA bins in the temporary solution.
solution_commit	Accepts all pending moves and applies them to the final solution.
solution_discard	Discards all the pending moves.
solution_save {file}	Saves the *temp.solution* into the file.
solution_restore {file}	Restores *temp.solution* from the file.
Report commands	
Command	**Description**
report_cell_connections	Reports cell connections similar to *partition.rpt*.
report_solution	Reports bin usage summary (AP140).
report_cost	Reports solution cost (depending on the optimization priority).
report_interconnection_table	Prints information about interconnections and their *tdm* qualification, also saved to *.csv* file.
report timing	Reports timing. *-type <hops> reports hops.* *-type<value> ..... hops|slack (default hops).* *-worst_paths <value> ... report worst paths.* *-level<value> ..... report multihop paths of length n.* *-net<value> ..... report worst paths for specified net.* *-summary<value> ..... report summary.* *-file<value> ..... write report to file.*
List commands	

Feedback

*design_list_** and *tss_list_** commands lists various properties as mentioned in the command. For details, check the command using *-help*. For example: `design_list_cells -help`	
Fix multihop command	
Command	**Description**
fix_multihop_path -net name	The tool automatically attempts to fix worst multihop for the given net.

Working with Inferred Clocks Driving Inter-FPGA Paths

During time budgeting, there can be many inferred clocks for inter-FPGA paths. These clocks and DPOs associated with them can cause long runtime issues during place and route.

There are two flows when working with inferred clocks for inter-FPGA paths. The default flow infers a clock for every unconstrained clock object in the design. The alternative flow merges clocks inferred for every clock object into one inferred clock. The following procedure shows how to use both flows.

Default flow

1. After running pre-partition, check the clock information.
   - Specify clock definitions with `create_clock` for any inferred clocks.
2. Do the following for the default flow:
   - -option set merge_inferred_clocks 0 (setting is default)
   - Partition the design as usual.
   - Generate timing budgets at the System Generate stage (`run system_generate -time_budget 1`). With the default settings (*option set merge_inferred_clocks 0*), the tool infers multiple inferred clocks (as necessary) for unconstrained clocks clocking inter-FPGA paths. It creates constraints (*create_clock*) for each of them that are passed to the individual FPGAs.

     ```
     create_clock -name {test|c1} -period 1000.00 -add [get_ports {c1}]
     create_clock -name {test|c2} -period 1000.00 -add [get_ports {c2}]
     create_clock -name {test|c3} -period 1000.00 -add [get_ports {c3}]
     ```
3. Check for DE107 errors. These errors indicate inter-FPGA paths that are clocked by inferred clocks. You can downgrade this error to a warning, so that you can continue. To downgrade the error, use the *TCL* command `message_override -warning DE107`

and rerun System Generate. Or right-click on the error message and choose the option **Downgrade to Warning**. If you downgrade the error, the tool generates DPOs for the paths clocked by inferred clocks, in addition to the create_clock constraints.

Merged Flow

With this option, the tool merges clock nets, ports, and pins into only one inferred clock, and creates a clock constraint. This constraint is passed to the individual FPGAs for use during synthesis and place and route.

1. Execute the following command to merge inferred clocks and generate only one inferred clock for the whole design.

   ```
   option set merge_inferred_clocks 1
   ```

2. Inferred clocks are forward-annotated to synthesis with the argument to its constraints.

   ```
   create_clock -name {test|c1} -period 1000.00 -add [get_ports {c1} {c2} {c3} {c4} ...]
   ```

3. Check for DE107 errors, which can be downgraded as described in the Default flow. If you downgrade the error, the tool generates DPOs for the paths clocked by inferred clocks, in addition to the *create_clock* constraints.

Implementing Individual FPGAs

After partitioning the top-level prototype, you can synthesize the individual FPGAs.

See the following for details:

- Running Synthesis for Individual FPGAs
- Customizing Synthesis Scripts
- Running Synthesis and Place and Route for a Multi-FPGA Design
- Using Vivado Flags and Options in Tcl Script
- Tcl and Environment Variables in Custom Scripts
- Updating the Top Level with FPGA Implementation Results

To reduce runtime, you can use distributed processing to work on the FPGAs in parallel. See Running Distributed Synthesis.

Feedback

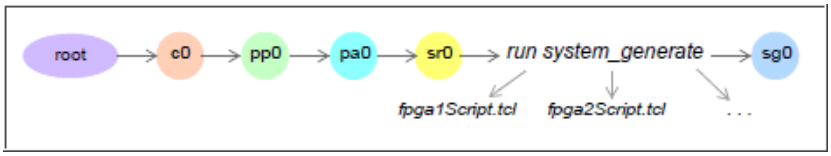
Running Synthesis for Individual FPGAs

There are two ways to implement the individual FPGA partitions, an automatic, script-based method, and a manual one. Both methods are described below, but it is recommended that you use the first procedure.

Synthesizing Individual FPGAs Using a Script

To use the recommended, script-based method to synthesize FPGAs, follow these steps:

1. Generate individual FPGA partitions with `run system_generate -path`(Generating FPGA Partitions).



   This command generates Tcl scripts for implementing the individual FPGAs. The *-path* option specifies a location for the Tcl scripts. For example, `run system_generate -out g0 -path synthesis_files` generates the following Tcl files for a design with two partitions, *uA* and *uB*:

```
synthesis_files/fb1_uA/fb1_uA_srs.tcl
synthesis_files/fb1_uB/fb1_uB_srs.tcl
```

   The scripts include commands to compile, pre-map, and map the FPGAs. Each FPGA script also includes the `export vivado` command to generate the files needed for place and route.

   You can customize the scripts as described in Customizing Synthesis Scripts. However, do not change the speed grade or technology that are written to these Tcl files, because the partitions use this information. The speed grade for individual FPGAs is set with the `board_system_create` command in the TSS file, and the technology is determined by the `database create -technology` command.

2. To run the individual databases in parallel, specify the `launch protocompiler -script` command:

   - Use this basic syntax to process each script and implement the FPGAs in parallel. See launch protocompiler for the complete syntax description.

```
launch protocompiler -script fpga1.tcl -script fpga2.tcl ...
```

Feedback

- You can also set options for individual FPGA runs from the command line by preceding the *-script* argument with a *-tclcmd* argument. See Customizing Synthesis Scripts for details.
- If you are using deep trace debug, make sure to implement the DTD device generated after the system generate step, just as you do with all the other FPGAs.

3. To run FPGA implementation scripts in parallel and minimize the overall runtime for a multi-FPGA design, do the following:
   - Specify the `launch protocompiler` command with the *-max_parallel_jobs* option to run individual FPGA implementations in parallel. You can run four jobs in parallel with each license. The actual number of jobs run depends on the number of available licenses and the value specified.

     ```
     launch protocompiler -max_parallel_jobs 8 -script fpga1.tcl -script
     fpga2.tcl ...
     ```

     If you do not specify this argument, the command uses the default *max_parallel_jobs* value, or a value that was previously set. For more about specifying parallel jobs, see Configuring Maximum Number of Parallel Jobs.

     You can override the *max_parallel_jobs* setting used with the `launch` command by setting an option in an individual FPGA during synthesis, after the `launch` command has been issued.
   - To allocate different values for parallel jobs for different FPGAs, specify *-max_parallel_jobs* before that FPGA script.

     ```
     launch protocompiler -max_parallel_jobs 4 -script fpga1.tcl
     -max_parallel_jobs 8 -script fpga2.tcl ...
     ```
   - To use CDPL as the basis for distributed processing, set it up as described in Using CDPL for Distributed Processing. If you do not use CDPL, the tool uses the available processors on the current machine to run the parallel processes.
   - Specify the scripts to run in parallel when you specify the `launch` command. The tool divides the jobs and uses distributed processing.
   - You can also set options for the run from the command line. See Customizing Synthesis Scripts for details.
   - If you are using deep trace debug, make sure to implement the DTD device generated after the system generate step, just as you do with all the other FPGAs.
4. Run place and route for individual FPGAs with the `launch vivado` command.

   The script generates files for the Vivado run with the `export vivado` command.

Feedback

- To use the default Vivado Tcl script for place and route, run Vivado with commands like the following example, where *myVivadoDir* is the directory generated by the script after running `export vivado`:

  ```
  set vivado_tcl_script [export vivado -path myVivadoDir] launch
  vivado $vivado_tcl_script -run_dir $fb_uA1_dir
  ```

- To use a custom place-and-route script, specify it instead of using the Tcl script generated by `export vivado`, as shown here:

  ```
  launch vivado /home/my_custom_vivado_script.tcl -run_dir $fb_uA1_dir
  ```

This is an example of the command sequence:

```
launch protocompiler -max_parallel_jobs -script path/slp1.tcl -script
path/slp2.tcl -script path/slp3.tcl
set vivado_tcl_script [ export vivado -path myvivadodir]
launch vivado $vivado_tcl_script -run_dir $fb_uA1_dir
```

Synthesizing Individual FPGAs Manually

To work on each FPGA manually, follow these steps:

1. Generate individual FPGA partitions with *run system_generate*. See Generating FPGA Partitions.

   This command generates Tcl scripts for implementing the individual FPGAs. The scripts include commands to compile, pre-map, and map the FPGAs. Each FPGA script also includes the `export vivado` command to generates the files needed for place and route.

2. Set the tool to synthesis mode with the `option set design_flow synthesis` command.

3. Open the database for an individual FPGA and use the generated design files to compile and map the design.

   See The Basic Implementation Flow, for details.

4. Run place and route for individual FPGAs.

   - Export the files for place and route with the `export vivado` command. You can now run place and route with the `launch vivado` command.

   - To use the default Vivado Tcl script for place and route, run Vivado with commands like the following example, where *myVivadoDir* is the directory generated after running `export vivado`. This directory includes the default script.

     ```
     set vivado_tcl_script [export vivado -path myVivadoDir]
     launch vivado $vivado_tcl_script -run_dir $fb_uA1_dir
     ```

Feedback

- To use a custom place-and-route script, specify it instead of using the Tcl script generated by `export vivado`, as shown here:

  ```
  launch vivado /home/my_custom_vivado_script.tcl -run_dir $fb_uA1_dir
  ```

  See Customizing Synthesis Scripts for more information about using customized scripts for individual FPGAs.

Customizing Synthesis Scripts

The `run system_generate` command creates individual synthesis scripts for each FPGA based on the template in `installDir/lib/protocompiler/sysgen_template.tcl`. You can customize these instructions by either editing the FPGA scripts directly, or setting options from the command line when you launch the ProtoCompiler tool to run synthesis.

The customized scripts to synthesize individual FPGAs can include Tcl variables as well as environment variables.

Note:

The `-tclcmd` argument is positional. It applies only to the scrip that follow it. Each new `-tclcmd` block overrides the previous one for subsequent scripts.

Variables Used in Modular Flow (Exception Handling)

- `ENABLE PAR`: Enables place-and-route.
- `ENABLE BACKANNOTATION`: Enables the back annotation flow.
- `CUSTOM PAR`: Specifies a custom Vivado place-and-route script.

1. Create the synthesis Tcl scripts by generating partitions with `run system_generate`.

   See Running Synthesis for Individual FPGAs for details. Separate scripts are generated for each FPGA, and are customizable.

2. To customize an individual script directly, open the Tcl script file you need, and edit the appropriate line in the Tcl file.

   For example, to enable place and route for *fb1_uA*, open the Tcl file generated for that FPGA and edit this line:

   ```
   set_if_unset ENABLE_PAR 1
   ```

   The customized scripts can include Tcl variables and environment variables. See Tcl and Environment Variables in Custom Scripts for a list of variables that can be set.

3. To customize synthesis scripts from the command line, specify options to the `launch protocompiler` command when you synthesize an individual FPGA, using the *-tclcmd* argument.

Feedback

- Use the `launch protocompiler -tclcmd` command to specify Tcl code to execute before launching the subsequent script. This can include options you want to set for that FPGA. For example, you can specify sub-hierarchical EDIF files for Vivado. Then define the FPGA Tcl script to which it should apply with a subsequent *-script* argument. The *-tclcmd* argument applies to all subsequent scripts, until another *-tclcmd* argument is specified.

  ```
  launch protocompiler -tclcmd "set ENABLE_PAR1"
  -script synthesies_files/fb1_uA/fb1_uA_srs.tcl
  ```

  The options (*-tclcmd*) must precede the scripts to which they apply (*-script*).

- Specify multiple options by separating them with semi-colons and surrounding them with curly braces:

  ```
  launch protocompiler -tclcmd {set ENABLE_BACKANNOTATION 1; set
  ENABLE_PAR 1} -script $scriptA
  ```

- Specify multiple scripts with separate *-script* arguments. The *-tclcmd* options apply to all subsequent scripts. For example, this command runs place and route in both *$scriptA* and *$scriptB*:

  ```
  launch protocompiler -tclcmd {set ENABLE_PAR 1} -script $scriptA
  -script $scriptB
  ```

- To specify different settings for different scripts, use multiple sets of *-tclcmd* and *-script* arguments. This example runs place and route in fb1_uA (*$scriptA*) and runs both place and route and backannotation in fb1_uB (*$scriptB):*

  ```
  launch protocompiler -tclcmd {set ENABLE_PAR 1} -script $scriptA
  -tclcmd {set ENABLE_PAR 1; set ENABLE_BACKANNOTATION 1} -script
  $scriptB
  ```

- For modular flow:

  Launch ProtoCompiler Arguments:

  - `COMP_ARG`: Passes arguments to compile scripts (for example, synthesis options).
  - `SR_ARG`: Passes `pcf` and `tss` arguments in the launch protocompiler command.
  - `PART_ARG`: Passes `pcf` arguments in the `launch protocompiler` command.

  ```
  launch protocompiler -parallel 4 -tclcmd {set PART_ARG
   "-optimization_priority slack -clock_gate_replication 1
   -pcf ./constraints/fpga_bin_utilization.pcf"; set SR_ARG
   "-continue_on_pcf_error 1 -pcf ./constraints/feedthrough.pcf
  ```

Feedback

```
-tss ./constraints/board.tss" ; set ENABLE_PAR 1 ; set
CUSTOM_PAR ../../custom_export_vivado.tcl) } -script [database
query_state -script]
```

Running Synthesis and Place and Route for a Multi-FPGA Design

The *run system_generate* command creates individual synthesis scripts for each FPGA based on the template in *installDir/lib/protocompiler/sysgen_template.tcl*. You can customize these instructions by either editing the FPGA scripts directly, or setting options from the command line when you launch the ProtoCompiler tool to run synthesis.

You can customize the place-and-route options by creating a custom file to override the default settings.

This procedure describes how to use customized scripts to direct synthesis or place and route for individual FPGAs in a multi-FPGA design.

1. Partition the design and generate the individual FPGAs with `run system_generate`. For example:

   ```
   run system_generate -out g0 -path synthesis_files
   ```

   This command creates the partitions at the specified location, with a directory for each FPGA partition:

   ```
   synthesis_files/fb1_uA
   synthesis_files/fb1_uB ...
   ```

   Each FPGA directory contains a Tcl script to automate the synthesis of that FPGA:

   ```
   synthesis_files/fb1_uA/fb1_uA_srs.tcl synthesis_files/fb1_uB/
   fb1_uV_srs.tcl ...
   ```

2. To set custom synthesis options, customize the individual Tcl scripts as needed, either directly in the script or from the command line when you start synthesis.

   - Start with the template in *$BUILD/lib/protocompiler/sysgen_template.tcl*.
   - You can further customize the behavior with command line arguments to the `launch protocompiler` command.

     See Customizing Synthesis Scripts for details.

     After the compile and map stages finish, the script generates a *run_vivado_haps.tcl* file on a per-FPGA basis, which is used to run place and route with Vivado. The next steps describe how to override the Vivado settings with custom settings.

3. Create a Tcl file with the place-and-route settings you want to run Vivado.

Feedback

The settings in this custom options file override the settings in the *run_vivado_haps.tcl* file. For example, you can generate a Verilog netlist after place and route or decide that you do not want to generate a bit file:

```
set write_post_par_verilog "1"
set write_bitstream_enable "0"
```

4. Run the scripts for the FPGAs with the `launch protocompiler` command and the customized Vivado script.
   - For custom PAR explorer scripts, make sure to specify *set CUSTOM_PAR*:

     ```
     launch protocompiler -tclcmd {set ENABLE_PAR 1; set CUSTOM_PAR ../
     ../custom_export_vivado.tcl) -script ./synthesis_files/fb1_uA/
     fb1_uA_srs.tcl
     ```

   - For other custom place-and-route scripts, use *setVIVADO_OPTION_FILE*:

     ```
     launch protocompiler -tclcmd {set ENABLE_PAR 1; set
     VIVADO_OPTION_FILE ../../custom_export_vivado.tcl) -script ./
     synthesis_files/fb1_uA/fb1_uA_srs.tcl
     ```

     Note that the custom script is located two levels up relative to the Tcl files. For information about using the *-tclcmd* option, see Customizing Synthesis Scripts.

Using Vivado Flags and Options in Tcl Script

You can use Vivado flags and options in custom *run_vivado_haps.tcl* scripts during the initial exploration of a design to reduce the runtime. Most of these switches are reported by the Vivado in the log. These switches help in implementing a script-based flow without adding the switches to individual Vivado runs.

For example, you can add the Vivado switch *route.enableHoldExpnBailout* in a custom script *run_vivado_haps.tcl*:

```
catch {set_param route.enableHoldExpnBailout 1}
```

For details on the Vivado flags and options, see the Vivado documentation.

Tcl and Environment Variables in Custom Scripts

You can add the following Tcl and environment variables in customizable scripts:

TCL Variable	Defaults	To Enable Variable...
ENABLE_PRE_INSTRUMENT	Value: 0 State: *p0*	*{set ENABLE_PRE_INSTRUMENT 1}*Runs pre_instrument

Feedback

TCL Variable	Defaults	To Enable Variable...
ENABLE_COMPILE	Value: 1 State: *c0*	*{set ENABLE_COMPILE 1}*Runs compilation
ENABLE_PRE_MAP	Value: 1 State: *pm0*	*{set ENABLE_PRE_MAP 1}*Runs pre-map.
ENABLE_MAP	Value: 1 State: *m0*	*{set ENABLE_MAP 1}*Runs map.
ENABLE_EXPORT_VIVADO	Value: 1	*{set ENABLE_EXPORT_VIVADO 1}*Allows additional TCL script to be sourced with *run_vivado_haps.tcl script.*
RUN_VIVADO_SCRIPT	-	*{set RUN_VIVADO_SCRIPTrun_script.tcl}*Overrides the default *run_vivado_haps.tcl* script for P&R.
ENABLE_AUTO_DEBUG	Value: 0	*{set ENABLE_AUTO_DEBUG 1}*: Uses FTD IICE and enables automatic instrumentation of visible FPGA probles.
ENABLE_PAR	Value: 0	*{set ENABLE_PAR 1}*: Enables Vivado place and route.
ENABLE_BACKANNOTATE	Value: 0	*{set ENABLE_BACKANNOTATE 1}*Enables backnnotation.
VIVADO_OPTION_FILE	-	*{set VIVADO_OPTION_FILE ../../additional_vivado_option.tcl}*Optional file to source in *run_vivado_haps.tcl*. Used with *ENABLE_EXPORT_VIVADO.*
USER_DEBUG_SIGLIST	-	*{set USER_DEBUG_SIGLIST ../../post_slp.islf}*
USER_RTL_IDC	-	*{set USER_RTL_IDC project_user.idc}*For single-FPGA debug.
USER_NETLIST_IDC	-	*{set USER_NETLIST_IDC project_netlist.idc}*For single FPGA debug.
Read-only Tcl Variable		Setting

Feedback

TCL Variable	Defaults	To Enable Variable...
VIVADO_DIR	Value: *./vivado_$RTL_INPUT_FORMAT*	Defaults to *RTL_INPUT_FORMAT*, either HDL or SRS.(Cannot be set from *launch protocompiler*)
Environment Variables	Default	Setting
VM_NETLIST_DIR	-	*setenv VM_NETLIST_DIR ./Netlists*Use with *export netlist -path*.
VIVADO_ENABLE_MULTI_MACHINE_MPF	Value: 0	*setenv (VIVADO_ENABLE_MULTI_MACHINE_MPF)*Enables multi processing flow (MPF) in a multi-machine scenario.
VIVADO_REMOTE_MPF_SUBMIT	-	*setenv*(VIVADO_REMOTE_MPF_SUBMIT)Configures the end-user environment for multi-machine MPF.Example: `setenv(VIVADO_REMOTE_MPF_SUBMIT) "qsub -P bnormal -l mem_free=64G -pe mt 4 -cwd"`
VIVADO_REMOTE_MPF_KILL	-	*setenv*(VIVADO_REMOTE_MPF_KILL) Stops the specified job in a multi-machine MPF.Example:`setenv(VIVADO_REMOTE_MPF_KILL) "qdel"`
VIVADO_ENABLE_SINGLE_MACHINE_MPF	Value: 0	*setenv (VIVADO_ENABLE_SINGLE_MACHINE_MPF)*Enables multi processing flow in a single machine scenario.

Updating the Top Level with FPGA Implementation Results

You can update the top level with results from various database states at the FPGA level, either because the FPGAs have more accurate values or because they are more current.

You use the *database apply_state* command, as described in the following procedure, to update the top level with area estimates or a mapped FPGA. See database for command syntax details.

1. Partition your design, generate FPGA partitions and implement them as usual.

   For individual FPGAs, run the commands you need to generate the results you want. You can then use the `database apply_state` command to update existing database states with the new results. See the following steps for details.

2. Do the following to backannotate area estimates to the top-level design:
   - Pick the critical module or modules for which you want to estimate the area.
   - To estimate the area of a sub-module, use the *run compile -top_module* command on the submodule. Then execute the `run pre-map` and `run map` commands to generate an area estimate file (*.est*).
   - Export the area estimate file for the FPGA with the `export file estimationFile.est` command.
   - Open a pre-partitioned database state for the top level.
   - Specify the `database apply_state -update_est` command with the *.est* file from submodule, to update the top level with the more accurate area results from the submodule. The following example updates the top level with the results from *mod1, mod2,* and *mod3*:

     ```
     database apply_state -update_est {mod1.est mod2.est mod3.est}
     ```
   - Partition the top-level design with the `run partition` command.
3. To update the top level with another version of a mapped FPGA, follow the steps below.

   From the top level, the tool automatically links to the default database state. Use this procedure if you have customized the default name of the FPGA mapped database state, or if you want to change the link to an alternative mapped state.
   - Generate a mapped database state by running the `run compile`, `run pre-map`, and `run map` commands on the individual FPGA.
   - Open a system generate database state for the top level.
   - Specify the `database apply_state -link_module` command with the name of the mapped module and the name of the state. Note the bar between the path and the name of the mapped state:

     ```
     database apply_state -link_module moduleName pathtoModule|stateName
     ```

     This command links the specified mapped state to the top-level design.
   - You can now rerun `run system_generate` at the top level.
4. At the individual FPGA level, use the `database apply_state -import_vivado` command to import place-and-route results back into the FPGA. See Importing Place and Route Results for Back-annotation for details.

Using Multi-Design Mode (MDM)

Multi-design mode (MDM) lets you pool your hardware resources in HAPS farms and share the same HAPS systems between multiple users or designs. You can run multiple designs concurrently, either locally or remotely. This mode is also called multi-use or multi-user mode. Use multi-design mode to prototype designs or to validate them.

Multi-design mode is not limited to single FPGAs, but can be spread across multiple systems. For example, a 5-FPGA design can be implemented on FPGAs A, B, C, and D of system 1 and FPGA A of system 2, while another 3-FPGA design can be implemented on the remaining FPGAs (B, C, and D) of the system 2.

You can use mixed system designs for multi-design mode.

For details, see the following:

- Instantiating CAPIM_UI

Instantiating CAPIM_UI

For HAPS-100 modules, refer to Using UMRBus 3.0 with HAPS-100 Modules.

The CAPIM_UI eliminates the need for multiple compile runs to address conflicts caused by multiple copies of the same bit file. CAPIM_UI uses dynamic addressing instead of static addressing. The dynamic addresses are assigned by the supervisor at runtime.

Instantiate CAPIM_UI in your design.

Get the module definition from the */umr_capim.v* file, located in *installDir/lib/vlog*. The CAPIM address must include the FPGA location ID and non-zero MSB bit address. See CAPIM_UI Example.

CAPIM_UI Example

```
capim_ui  #(
 .UMR_CAPIM_ADDRESS(1),
 .UMR_CAPIM_TYPE(16'hA001),
 .UMR_DATA_BITWIDTH(UMR_DATA_BITWIDTH),
 .UMR_CAPIM_COMMENT_STRING("I_CAPIM_1"),
 .UMR_USE_LOCATION_ID(1)   )
capim_led_control (
 .umr_clk  (umr_clkout),
 .umr_reset  (),
 .wr (capim_data_wr),
 .dout (capim_data_do),
 .rd  (capim_data_rd),
 .din (capim_data_di),
 .intr (1'b0),
 .inta  (),
```

```
 .inttype (16'h0000)
);
 .din (capim_data_di),
 .intr (1'b0),
 .inta  (),
 .inttype (16'h0000)
);
```

Using UMRBus 3.0 with HAPS-100 Modules

See these topics for information about using UMRBus 3.0 with HAPS-100:

- Using UMRBus 3.0 MCAPIM for HAPS-100 Modules
- Using UMRBus 3.0 Transactors
- Using JTAG Communication

Using UMRBus 3.0 MCAPIM for HAPS-100 Modules

Only UMRBus 3.0 MCAPIM (*umr3_capim_ui*) is supported for HAPS-100 modules. UMRBus 3.0 allows bi-directional communication with a higher bit width.

These are the guidelines:

- UMRBus 3.0 is backward-compatible; replacing prior CAPIMs with the new ones. However new features, such as DMA, are not backward compatible.
- Do the following to convert designs that use UMRBus 2.0 CAPIMs (*capim_ui* or *capim_wp*) to UMRBus 3.0 MCAPIM (*umr3_capim_ui*).
  - For a simple *capim_ui*, replace it with *umr3_capim_ui* and connect the UMRBus 3.0 ports. See the example that follows.
  - Remove or comment the parameter *//.UMR_USE_LOCATION_ID(1)*.
- You can downgrade the UMRBus 2.0 error to a warning by setting the environment variable *disable_umr2_capim_error* to *TRUE*.

Example: UMRBus 2.0 to UMRBus 3.0

The following is an example of converting UMRBus 2.0 to UMRBus 3.0.

UMRBus 2.0

```
capim_ui #(.UMR_CAPIM_ADDRESS(UMR_CAPIM_BASE_ADDRESS),
 .UMR_CAPIM_TYPE(16'hD000),
 .UMR_DATA_BITWIDTH(UMR_DATA_BITWIDTH),
 .UMR_USE_LOCATION_ID(1),
 .UMR_CAPIM_COMMENT_STRING("I_CAPIM_1")
 )
capim_ua (
```

Feedback

```
 .umr_clk (umr_clkout),
 .umr_reset (),
 .wr (capim_data_wr),
 .dout (capim_data_do),
 .rd  (capim_data_rd),
 .din (capim_data_di),
 .intr (1'b0),
 .inta  (),
 .inttype (16'h0000)
);
```

UMRBus 3.0

```
umr3_capim_ui #(.UMR_CAPIM_ADDRESS(UMR_CAPIM_BASE_ADDRESS),
 .UMR_CAPIM_TYPE(16'hD000),
 .UMR_DATA_BITWIDTH(UMR_DATA_BITWIDTH),
 //.UMR_USE_LOCATION_ID(1),
 .UMR_CAPIM_COMMENT_STRING("I_CAPIM_1")
 )
capim_ua (
 .umr_clk (umr_clkout),
 .umr_reset (),
 .wr (capim_data_wr),
 .dout (capim_data_do),
 .rd  (capim_data_rd),
 .din (capim_data_di),
 .intr (1'b0),
 .inta  (),
 .inttype (16'h0000)
);
```

Using UMRBus 3.0 Transactors

Follow these steps to convert existing transactor designs to HAPS-100. If you use a new transactor design that is implemented for HAPS-100, *XACTORS_GEN* automatically sets the parameters and these steps are not required.

1. Add the *define XACTORS_UMR3* Verilog define as compile option.
2. Add following file to file list

   ```
   $env(BUILD)/lib/synip/uc2/umrbus3/umrbus3.v
   ```

3. Comment out the *umr_clk_reset* instance in the top-level file.

   ```
   // umr_clk_reset
    // I_umr_clk_reset
    //(
     // .umr_clk_out(CCLOCK),
     // .umr_reset_out(CRESET)
    // );
   ```

Using JTAG Communication

There is no dedicated JTAG connector on the HAPS module. Instead the built-in UMRBus offers JTAG communication for your application. The following two step process uses the virtual cable (XVC) from Vivado Hardware Manager, which is a TCP/IP-based protocol that acts like a JTAG cable.

1. Start the JTAG server using hRun commands on the host computer.
2. Connect to JTAG using the Vivado Hardware Manager.

```
>connect_hw_server
>open_hw_target -xvc_url localhost:65137
```

Modular Flow

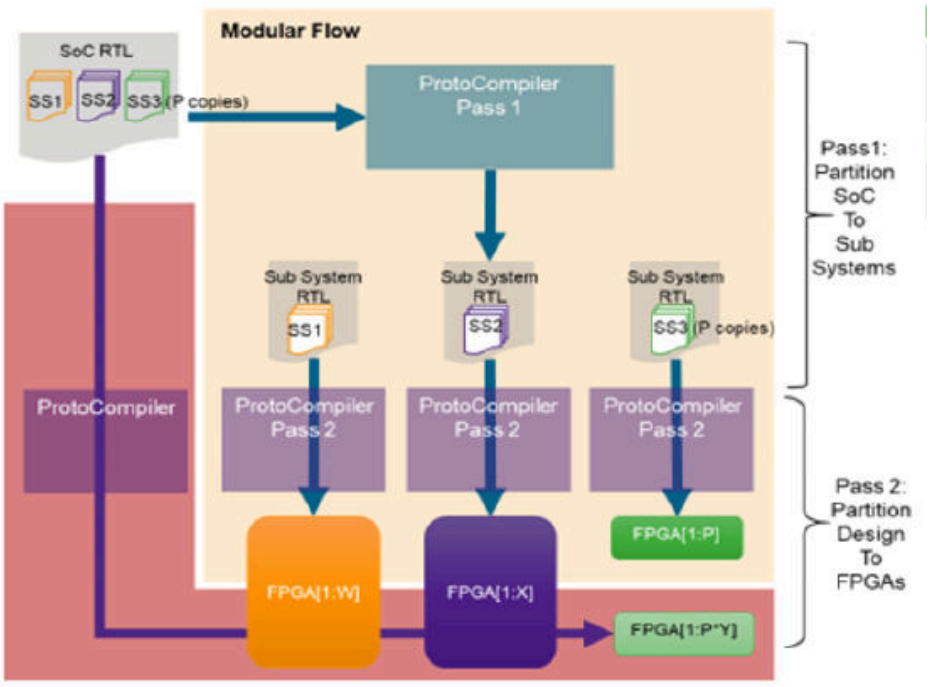
Large SOC and GPU designs if done top down can have huge impact on runtime. Replicated modules must be compiled separately adding up to the total runtime.

You can use enhanced modular flow if the design has large number of replicated modules or if the design has incremental changes. To reduce compile time, you can re-use bit files of replicated modules. Incremental compile of subsystems allows compile of only modified modules to preserve performance or reduce Turn Around Time (TAT).

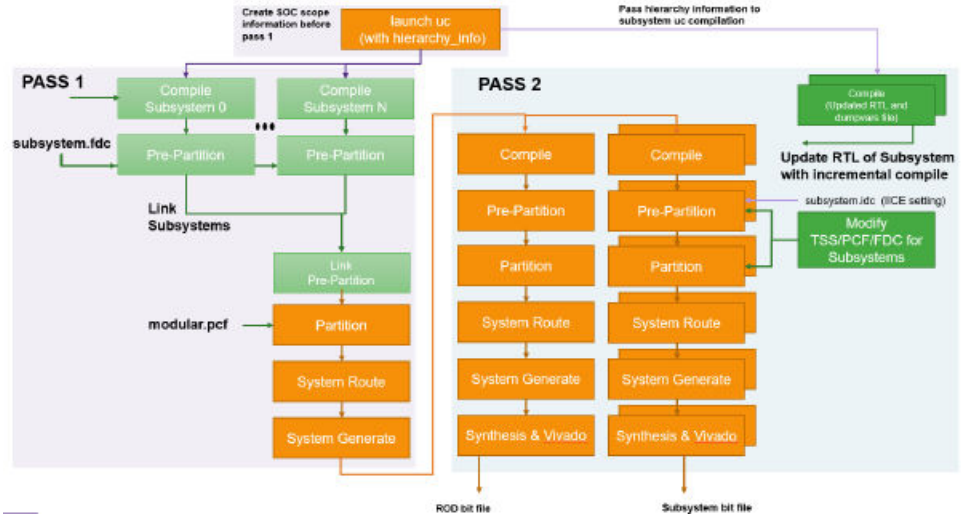
Modular flow is a two-pass partition flow. In the first pass, the SOC is partitioned into subsystems. In the second pass, guided or automated subsystems are partitioned to HAPS FPGAs. Each subsystem follows partition flow to generate bit stream for each of the FPGAs that form the subsystem.

The following figure is a representation of modular flow.

Feedback



The following figure is a representation of various steps of modular flow in first and second pass. First pass includes partition flow in which the design is divided into subsystems. Second pass flow partitions the design into FPGAs using partition flow and generates bit stream for each of the FPGAs.



Feedback

Modular Flow Setup

The flow supports cell assignments to specific subsystems and FPGAs in the first pass. Replicated modules are assigned to replicated subsystems. Replicated subsystem consists of multiple cells from replicated tile modules. These modules are assigned to replicated subsystems.

The following image details the subsystem definition and assignment to subsystems and FPGAs.

Name of subsystem	Instance assigned to sub-system	Type of sub-system
SS_MCell_DHier_Rep	{i:iSoC.RocketSystem.tile.rocket} **i_SS_MCell_DHier_Rep_1** {i:iSoC.RocketSystem.tile.xing} **i_SS_MCell_DHier_Rep_1** {i:iSoC.RocketSystem.tile_1.rocket} **i_SS_MCell_DHier_Rep_2** {i:iSoC.RocketSystem.tile_1.xing} **i_SS_MCell_DHier_Rep_2** {i:iSoC.RocketSystem.tile_2.rocket} **i_SS_MCell_DHier_Rep_3** {i:iSoC.RocketSystem.tile_2.xing} **i_SS_MCell_DHier_Rep_3** {i:iSoC.RocketSystem.tile_3.rocket} **i_SS_MCell_DHier_Rep_4** {i:iSoC.RocketSystem.tile_3.xing} **i_SS_MCell_DHier_Rep_4**	Multi cell & replicated
SS_Mcell_Non_Rep	{{i:iSoC.i_axi_mmio.axi_uartlite_0} {i:iSoC.i_axi_mmio.axi_interconnect_mmio} {i:iSoC.i_axi_mmio.blk_mem_gen_0} {i:iSoC.i_axi_mmio.axi_bram_ctrl_0}} **{i_SS_Mcell_Non_Rep_1}** {{i:iSoC.i_mig} {i:iSoC.i_axi_mmio_master_xactor} {i:iSoC.i_axi_master_xactor} {i:iSoC.i_axi_extmem}} **{i_SS_Mcell_Non_Rep_1}**	Multi cell from different hierarchy and Non replicated subsystem
ROD	Cells not explicitly assigned to a sub-system are auto assigned by the tool to this default sub-system	Rest of Design

Non-replicated subsystem SS_Mcell_Non_Rep consists of multiple cells from different hierarchies.

ROD subsystem is added by the tool and this includes all FPGAs that are not locked by the user and that are not used in user defined subsystems. Instances that are not assigned by the user are assigned by the tool to ROD and to non-replicated subsystem.

In the following figure, highlighted modules that are identical are assigned to replicated subsystem and multiple cells from SoC are assigned to non-replicated subsystem.

Feedback


i:tile (SyncRocketTile_tile)
i:rocket (RocketTile_rocket)
i:xing (IntXing_xing)
Nets (410)
Ports (47)
Primitives (0)
Black Boxes (0)
i:TLBroadcast (TLBroadcast)
i:TLToAXI4 (TLToAXI4)
i:yank (AXI4UserYanker_yank)
G:SystemBus_tile_TLBuffer_[3:1] (TLBu
G:tile_[3:1] (SyncRocketTile_tile)
i:tile_1 (SyncRocketTile_tile)
i:rocket (RocketTile_rocket)
i:xing (IntXing_xing)
Nets (410)
Ports (47)
Primitives (0)
Black Boxes (0)
i:tile_2 (SyncRocketTile_tile)
i:rocket (RocketTile_rocket)
i:xing (IntXing_xing)
Nets (410)
Ports (47)
Primitives (0)
Black Boxes (0)
i:tile_3 (SyncRocketTile_tile)
i:rocket (RocketTile_rocket)
i:xing (IntXing_xing)
SS_MCell_DHier_Rep
SS_MCell_DHier_Rep
SS_MCell_DHier_Rep
SS_MCell_DHier_Rep

i:SoC (SoC_0000)
i:i_axi_extmem (axi_extmem)
i:i_axi_master_xactor (axi_master_xactor)
i:i_axi_mmio (axi_mmio)
i:axi_bram_ctrl_0 (axi_mmio_axi_mmio_axi_bram_ctrl_0_0)
i:axi_interconnect_mmio (axi_mmio_axi_mmio_axi_interconnect_0_0)
i:axi_uartlite_0 (axi_mmio_axi_mmio_axi_uartlite_0_1)
i:blk_mem_gen_0 (axi_mmio_axi_mmio_blk_mem_gen_0_0)
Nets (1075)
Ports (79)
Primitives (0)
Black Boxes (0)
i:i_axi_mmio_master_xactor (axi_mmio_master_xactor)
i:i_gpio (gpio)
i:l_xactor_connect (xactor_connect)
i:resetSyncInvGPIOReset (reset_sync_inv_1)
i:RocketSystem (ExampleRocketSystem)
Nets (2106)
Ports (28)
Primitives (4)
Black Boxes (1)
i:i_mig (mig)

Feedback

First Pass - Partition

Subsystem definition

You can use the subsystem create command to define the subsystem. It consists of subsystem name, instance name, and FPGAs that form the subsystem. Replicated subsystems have the same subsystem name and different instance names. ROD subsystem is added by the tool and includes all FPGAs that are not locked by the user and that are not used in user defined subsystems. Assignment of replicated modules have to be done to replicated subsystems. Replicated subsystems are locked after user assignment to preserve the identical nature of the replicated subsystems.

Tcl command: – option set subsystem_partition 1

PCF command: – subsystem_create -bins

For example:

```
subsystem_create SS_MCell_DHier_Rep i_SS_MCell_DHier_Rep_1 -bins
{mb_c0.uA mb_c0.uB}

subsystem_create SS_MCell_DHier_Rep i_SS_MCell_DHier_Rep_2 -bins
{mb_c1.uA mb_c1.uB}

subsystem_create SS_MCell_DHier_Rep i_SS_MCell_DHier_Rep_3 -bins
{mb_c2.uA mb_c2.uB}

subsystem_create SS_MCell_DHier_Rep i_SS_MCell_DHier_Rep_4 -bins
{mb_c3.uA mb_c3.uB}

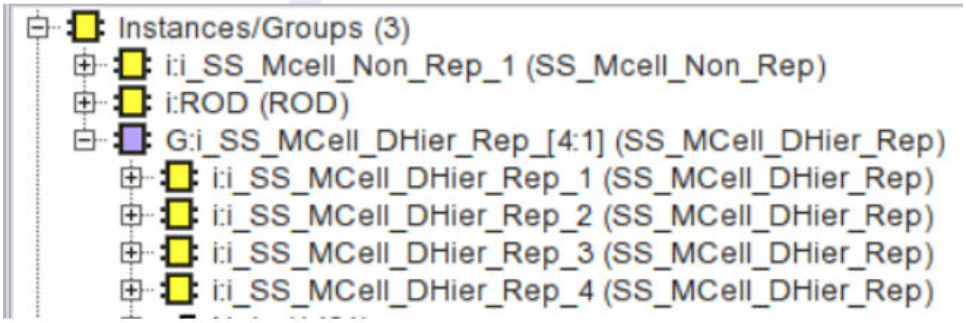
subsystem_create SS_Mcell_Non_Rep i_SS_Mcell_Non_Rep_1 -bins {mb_c0.uC
mb_c0.uD}
```

In the example:

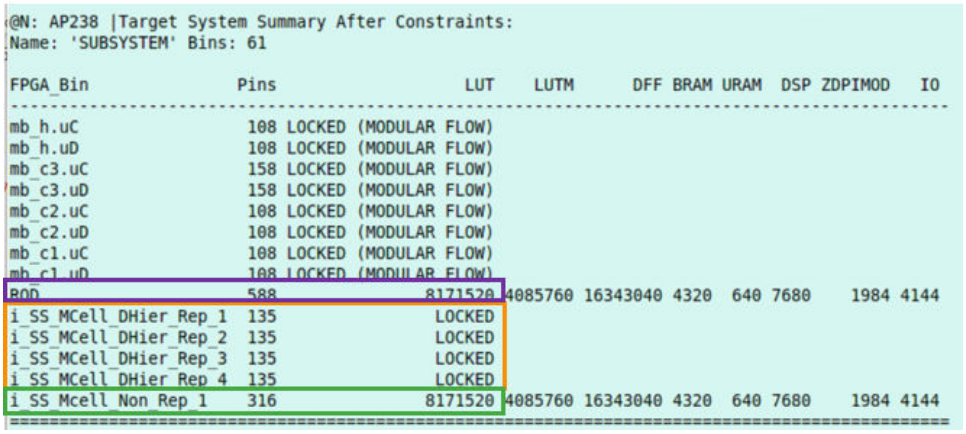
SS_MCell_DHier_Rep is replicated subsystem. The i_SS_MCell_DHier_Rep_1, i_SS_MCell_DHier_Rep_2, i_SS_MCell_DHier_Rep_3, and i_SS_MCell_DHier_Rep_4 are 4 copies of the replicated subsystem. SS_Mcell_Non_Rep is non-replicated subsystem.

The following partition schematic image displays replicated subsystem, non replicated subsystem, and ROD. FPGAs are not displayed in the first pass partition schematic.

Feedback



The following Target System Summary After Constraints image displays pins and resources of ROD and non replicated subsystem. Replicated subsystem highlighted with orange box is displayed as LOCKED as the replicated subsystem is locked after user assignment of replicated modules to the subsystem.



```
@N: AP238 |Target System Summary After Constraints:
Name: 'SUBSYSTEM' Bins: 61

FPGA_Bin                Pins                  LUT    LUTM      DFF BRAM URAM  DSP ZDPIMOD   IO
-------------------------------------------------------------------------------------------------
mb_h.uC                  108 LOCKED (MODULAR FLOW)
mb_h.uD                  108 LOCKED (MODULAR FLOW)
mb_c3.uC                 158 LOCKED (MODULAR FLOW)
mb_c3.uD                 158 LOCKED (MODULAR FLOW)
mb_c2.uC                 108 LOCKED (MODULAR FLOW)
mb_c2.uD                 108 LOCKED (MODULAR FLOW)
mb_c1.uC                 108 LOCKED (MODULAR FLOW)
mb_c1.uD                 108 LOCKED (MODULAR FLOW)
ROD                      588              8171520 4085760 16343040 4320  640 7680    1984 4144
i_SS_MCell_DHier_Rep_1   135                LOCKED
i_SS_MCell_DHier_Rep_2   135                LOCKED
i_SS_MCell_DHier_Rep_3   135                LOCKED
i_SS_MCell_DHier_Rep_4   135                LOCKED
i_SS_Mcell_Non_Rep_1     316              8171520 4085760 16343040 4320  640 7680    1984 4144
=================================================================================================
```

Cell assignment to subsystem

Instances are assigned using assign_cell command.

Syntax:

assign_cell <instance name> <subsystem instance name>

PCF assignment of cells to subsystems:

```
assign_cell {i:iSoC.RocketSystem.tile.rocket} i_SS_MCell_DHier_Rep_1

assign_cell {i:iSoC.RocketSystem.tile.xing} i_SS_MCell_DHier_Rep_1

assign_cell {i:iSoC.RocketSystem.tile_1.rocket} i_SS_MCell_DHier_Rep_2

assign_cell {i:iSoC.RocketSystem.tile_1.xing} i_SS_MCell_DHier_Rep_2

assign_cell {i:iSoC.RocketSystem.tile_2.rocket} i_SS_MCell_DHier_Rep_3
```

Feedback

```
assign_cell {i:iSoC.RocketSystem.tile_2.xing} i_SS_MCell_DHier_Rep_3

assign_cell {i:iSoC.RocketSystem.tile_3.rocket} i_SS_MCell_DHier_Rep_4

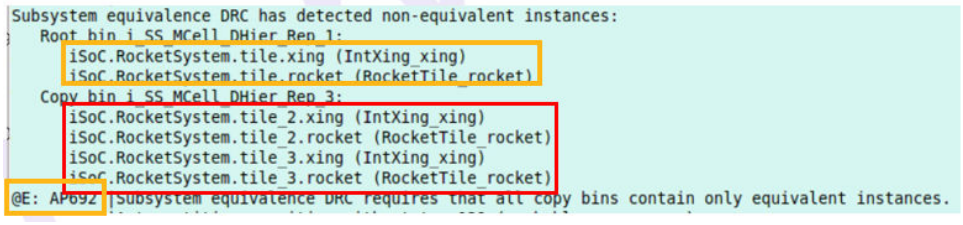
assign_cell {i:iSoC.RocketSystem.tile_3.xing} i_SS_MCell_DHier_Rep_4

assign_cell {{i:iSoC.i_axi_mmio.axi_uartlite_0}
{i:iSoC.i_axi_mmio.axi_interconnect_mmio}
{i:iSoC.i_axi_mmio.blk_mem_gen_0} {i:iSoC.i_axi_mmio.axi_bram_ctrl_0}}
{i_SS_Mcell_Non_Rep_1}
```

PCF assignment of cells to FPGA:

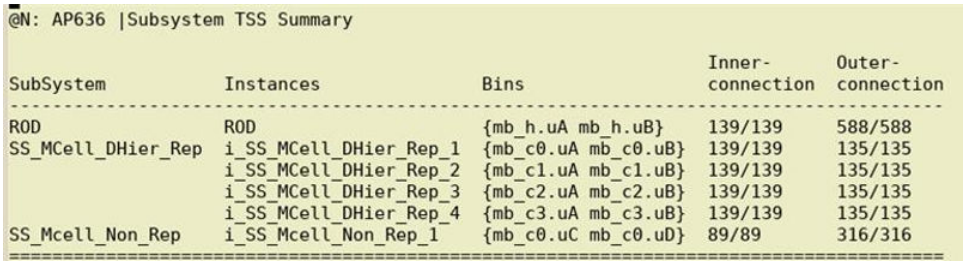
```
assign_cell {{iSoC.i_mig} mb_c0.uA
```

DRCs are added for replicated subsystems. The following image is an example of DRC when non equivalent instances are assigned to replicated subsystems. Partition errors out as two instances are assigned to one copy of the replicated subsystem and four instances are assigned to another copy of the replicated subsystem which makes the assignment to subsystems non equivalent.



Subsystem TSS summary, displayed in the following figure, includes subsystem name, instance names, and FPGAs that form the subsystem. Inner connections indicate connections within the subsystem. Outer connections indicate connections across subsystems.

Replicated subsystems have to be pin compatible. The summary displays any differences in pin compatibility.



The following log file displays information with respect to subsystems instead of FPGAs as in the case of non-modular flow. A snippet of connectivity across subsystems and bin usage summary of subsystems is displayed.

Feedback

```
Connectivity                                    Point-to-Point Mult-Terminal
-------------------------------------------------------------------------------
mb_h.uC<->mb_h.uD (FIXED)                       81             0
mb_c3.uC<->mb_c3.uD (FIXED)                     81             0
mb_c3.uC<->mb_c3.uD                             50             0
mb_c2.uC<->mb_c2.uD (FIXED)                     81             0
mb_c1.uC<->mb_c1.uD (FIXED)                     81             0
ROD<->i_SS_MCell_DHier_Rep_1                    100            0
ROD<->i_SS_MCell_DHier_Rep_2                    100            0
ROD<->i_SS_MCell_DHier_Rep_3                    100            0
ROD<->i_SS_MCell_DHier_Rep_4                    100            0
ROD<->i_SS_Mcell_Non_Rep_1                      100            0
ROD<->TOP_IO_HT3_mb_h_A24                       52             0
i_SS_Mcell_Non_Rep_1<->TOP_IO_HT3_mb_c0_D24     52             0
i_SS_Mcell_Non_Rep_1<->ddr3_sodimm2r_ht3.dimm   124            0
i_SS_Mcell_Non_Rep_1<->ddr3_sodimm2r_ht3.osc    2              0
i_SS_Mcell_Non_Rep_1<->ddr3_sodimm2r_ht3.led    3              0
===============================================================================
```

In the following figure, Bin usage summary is displayed for each subsystem in the modular flow.

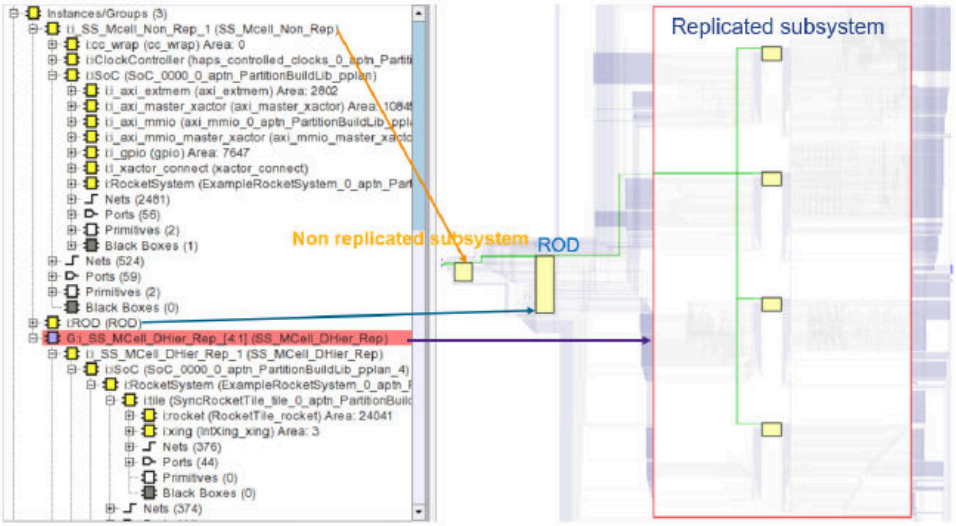
```
@S6.1 AP140 |Bin Usage Summary

FPGA_Bin               Cells Lock   LUT     %   DFF     % BRAM     % DSP     %
-------------------------------------------------------------------------------
ROD                      460      17271    0% 13689    0%    0    0%   1    0%
i_SS_MCell_DHier_Rep_1     4 LOCK 24051    0% 13339    0%    8    0%  13    0%
i_SS_MCell_DHier_Rep_2     4 LOCK 24051    0% 13339    0%    8    0%  13    0%
i_SS_MCell_DHier_Rep_3     4 LOCK 24051    0% 13339    0%    8    0%  13    0%
i_SS_MCell_DHier_Rep_4     4 LOCK 24051    0% 13339    0%    8    0%  13    0%
i_SS_Mcell_Non_Rep_1     269      38158    0% 38318    0%  521   12%   4    0%
===============================================================================
```

The following partition schematic displays replicated subsystem, non replicated subsystem, and ROD.

There are four identical copies of the replicated subsystem. Non-replicated subsystem hierarchy displayed in the image also includes cells that were auto assigned by the tool in addition to user assignments.

Feedback



Manual replication of cut clock

Manual replication of cut clock can be performed using the following command:

Syntax:

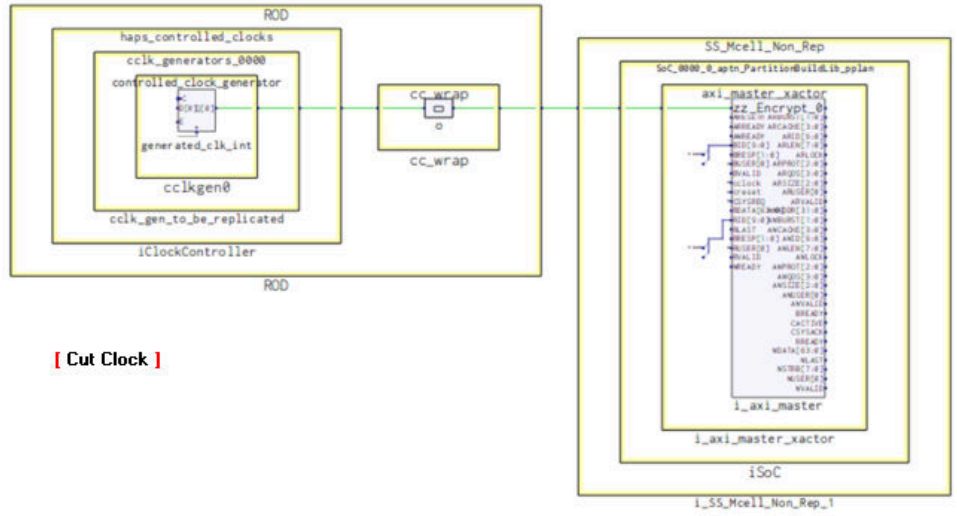
```
replicate_cell
```

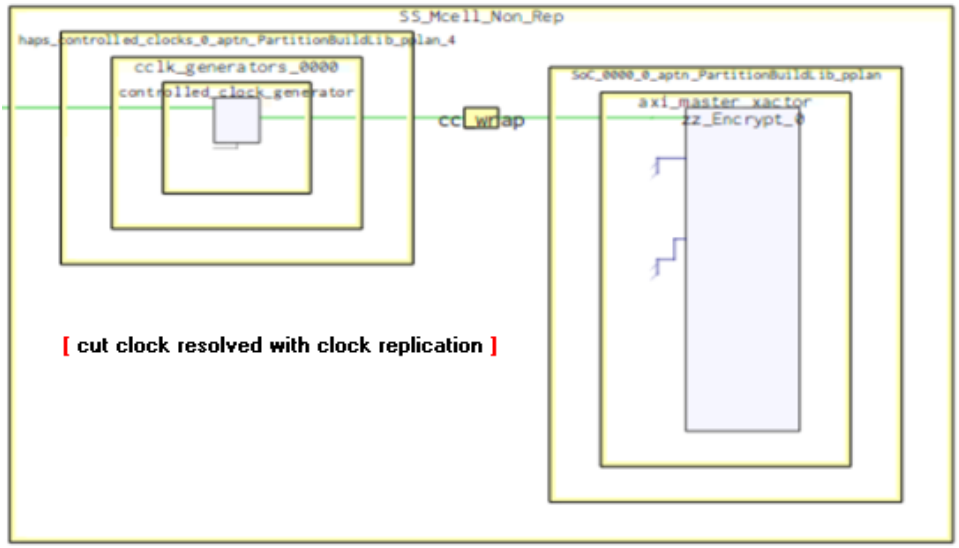
For example:

```
replicate_cell {i:iClockController.cclk_gen_to_be_replicated}
{i_SS_MCell_DHier_Rep_1 i_SS_MCell_DHier_Rep_2 i_SS_MCell_DHier_Rep_3
i_SS_MCell_DHier_Rep_4 i_SS_Mcell_Non_Rep_1 ROD}
```

The following image displays cut clock schematic that is resolved using manual replication of clock generator.

Feedback





If cut clock is observed, partition log file reports cut clock and source and destination bins which are subsystems in the first pass.

First pass system route

Interconnections are represented with respect to subsystems in case of modular flow in the log file of first pass system route. HSTDM insertion is timing aware.

For example, the following image displays slack information post TDM insertion. Since no nets failed timing, the summary indicates that none of the nets failed timing post TDM insertion.

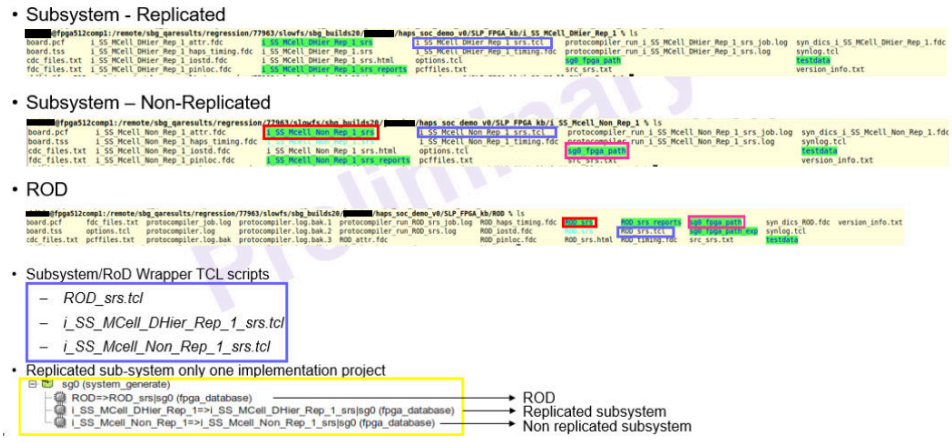
Feedback

```
@S5.1.4 AP655 |Post-Optimization Routing Summary
Connection                    Trace Usage  No. of Net      TDM Module          Available Slack Range  Mean Slack  Untimed
                                           failed timing                       (Post TDM)                         nets
-------------------------------------------------------------------------------------------------------------------------
ROD<->i_SS_MCell_Non_Rep_1    100/100
                                           0/46            NONE (11 ns)        84 to 989 ns           206 ns      2
                                           0/349           HSTDM_8 (40 ns)     152 to 159 ns          155 ns      0
                                           0/2             HSTDM_16 (50 ns)    147 to 147 ns          147 ns      0
ROD<->i_SS_MCell_DHier_Rep_1  100/100
                                           0/50            NONE (11 ns)        87 to 989 ns           207 ns      0
                                           0/311           HSTDM_8 (40 ns)     152 to 159 ns          155 ns      0
                                           0/15            HSTDM_256 (226 ns)                                     15
ROD<->i_SS_MCell_DHier_Rep_2  100/100
                                           0/50            NONE (11 ns)        87 to 989 ns           207 ns      0
                                           0/311           HSTDM_8 (40 ns)     152 to 159 ns          155 ns      0
                                           0/15            HSTDM_256 (226 ns)                                     15
ROD<->i_SS_MCell_DHier_Rep_3  100/100
                                           0/50            NONE (11 ns)        87 to 989 ns           207 ns      0
                                           0/311           HSTDM_8 (40 ns)     152 to 159 ns          155 ns      0
                                           0/15            HSTDM_256 (226 ns)                                     15
ROD<->i_SS_MCell_DHier_Rep_4  100/100
                                           0/50            NONE (11 ns)        87 to 989 ns           207 ns      0
                                           0/311           HSTDM_8 (40 ns)     152 to 159 ns          155 ns      0
                                           0/15            HSTDM_256 (226 ns)                                     15
=========================================================================================================================
```

Pass 1: Output Files

This section describes the output of replicated, non-replicated, and ROD subsystems. The files are similar to that of non-modular flow.

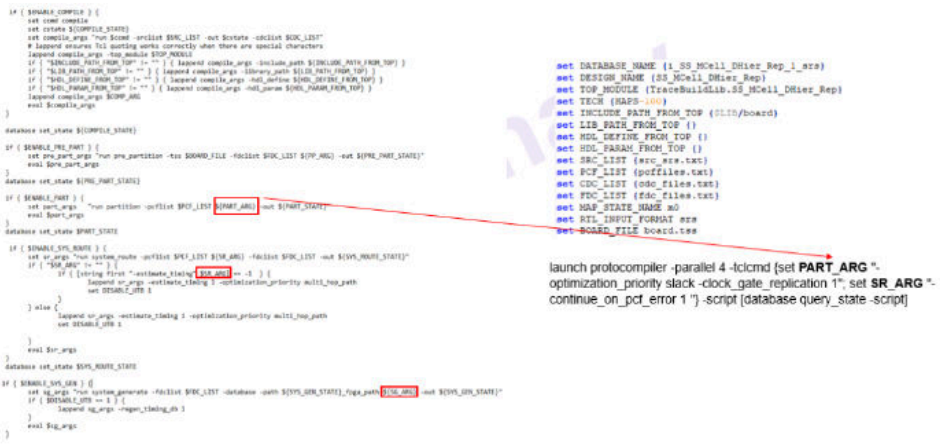
For example, in the following figure, FPGA name is replaced by subsystem instance name. The script to launch subsystem database is <subsystem_instance_name>_srs.tcl. Subsystem and ROD wrapper tcl scripts are highlighted in blue. File name is _srs.tcl. Highlighted section in yellow indicates SLP databases generated for ROD, replicated subsystem, and non replicated subsystem. There is only one database generated for replicated subsystem.



The following figure is an example of tcl file that is auto generated by the tool which is used to launch subsystem database. Modules assigned to various subsystems are partitioned to FPGAs using partition flow. It compiles subsystem database, partitions instances to multiple FPGAs in the subsystems, and generates bit steams for individual FPGAs.

Feedback

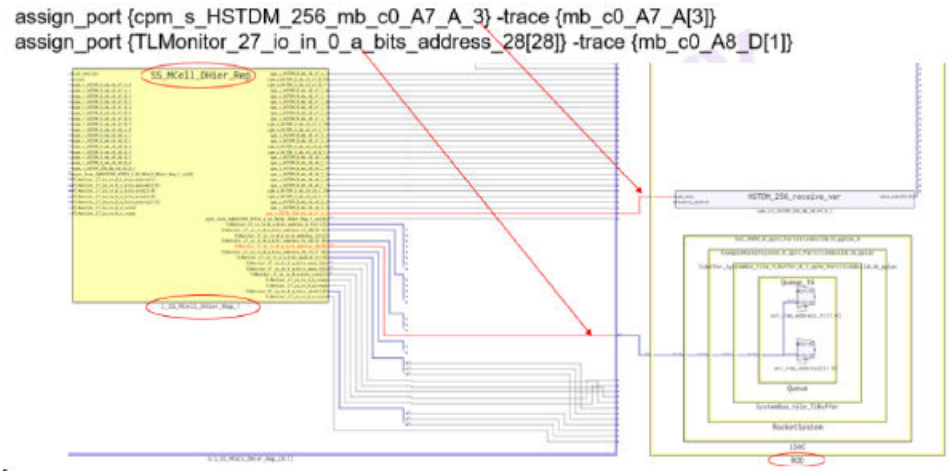
Use the PART_ARG, SR_ARG, and SG_ARG arguments to pass additional parameters to partition, system route, and system generate flows, respectively, in a modular flow. The arguments can be passed using –tclcmd option in the launch protocompiler command.



System generate in first pass generates inter subsystem trace assignments in *board.pcf* file for the individual subsystem. The following figure displays assignment of design ports and HSTDM ports to traces.

```
assign_port {cpm_s_HSTDM_256_mb_c0_A7_A_3} -trace {mb_c0_A7_A[3]}

assign_port {TLMonitor_27_io_in_0_a_bits_address_28[28]} -trace
{mb_c0_A8_D[1]}
```



Feedback

Guided Partition Flow

Guided partitioning refers to a methodology and set of tool features designed to help you optimally partition large designs across multiple FPGAs. This approach emphasizes:

- Maximizing performance
- Minimizing constraints
- Reducing compile time

The guided partition flow integrates automation, iterative refinement, and user-defined constraints to achieve high-quality partitioning. Correlation here refers to Timing QoR Metrics between Partition - System Route and between System Route - System Generate. It includes specific optimizations related to:

- Partition convergence
- Partition – System Route – System Generate correlation

These optimizations are bundled under a single switch - `-setup ps1`.

Key Features of `-setup ps1`:

- Enables the auto-partitioner to explore multiple options for optimal performance.
- Iteratively improves partition quality based on the initial iteration of a given partition run.
- Requires minimal user input, such as port assignments and IP assignments.

-setup ps1 Option

The `-setup ps1` option must be included in both `run partition` and `run system_route` commands, along with all other required parameters. See run partition for more information on this option, in the *Command Reference Manual*.

```
run partition -pcf pcf_guided.pcf -setup ps1 -out ps_ps1_0
 run system_route -pcf pcf_guided.pcf -fdc top_fpga_clock.fdc -setup ps1
 -out sr_ps1_0
```

The `-setup ps1` switch includes several arguments, including `optimization_priority` and `clock_gate_replication`. If these arguments are also specified in the `run partition` command, the partition process overrides these options and continues the flow. The following warning is generated and saved to `protocompiler.log`:

```
Warning: -setup and -optimization_priority options are mutually
exclusive. Tool is automatically using -setup option.
```

Only input files - PCF and TSS -- should be used with the `-setup ps1` switch.

Result Analysis

Review the following parameters from the `partition.log` to determine if the partition has successfully converged:

- `TargetSlack` — Desired slack in nanoseconds (default is 0ns).
- `Cuts` — Total number of cuts in the partition.
- `HopCost` — Multi-hop reduction cost, calculated as the sum of squares of hop values across all nets.
- `Unroute` — Count of unrouted net segments. This should be zero.
- `NoRepClk` — Number of cut clocks that cannot be resolved by using clock replication. These require manual resolution.
- `CutClk` — Total number of cut clocks. After clock replication, this should be zero in the final partition cost.
- `ChMax` — Ratio of *trace usage* to *available traces*. A value greater than 1 indicates the router may fail to meet timing.
- `ChOvf` — Number of channels where the *trace usage* to *available traces* ratio is ≥ 1.

```
@S5.2 AP751 |Final Partition Cost

Total:Slack Cost = 1.94278e+08;
  Slack Cost: Sol 791 Cuts 186152 FT 637 Timing 7 ChMax 1.10 ChOvf 1 MGT 0.00
```

Overview of Selective Rebuild

Selective rebuild is used to incrementally compile a module and generate bit stream for the affected subsystems. This helps to avoid recompilation of the entire design and bit stream generation of all FPGAs when the effect of RTL modification is localized to a subsystem.

Selective rebuild can be achieved using the following methodologies:

- Selective rebuild using nle file
  - Used for incremental changes.
  - Used when view can be replaced.
- Bottom-up compile

Feedback

- Used when there are lot of changes except for top level ports.
- Used for changes in primitives in top module.

Example scenario

This is an example scenario in which a module is compiled in *core_c* node. This module is defined as subsystem and rest of the design is compiled in *LC1 compile* node. Prepartition is run on the individual compile nodes and these are linked using *link_pre_partition* step.

RTL modification is made in ALU submodule within the module compiled in *core_c* node. The change in this example is to replace adder with subtractor. Hence, only *core_c* node is affected and the change is localized to a single subsystem. Clone of *core_c (core_c_incr)* is generated, and the module is incrementally compiled.

Export the *nle file* from the cloned module. This includes *replace_view* command for the module that has been modified. This *nle file* is passed in the run compile command of the subsystem in which the RTL modification has been affected. Run the subsystem compile with *nle file* and proceed with the remaining steps as in regular non modular flow to generate bit stream for FPGAs in the subsystem.

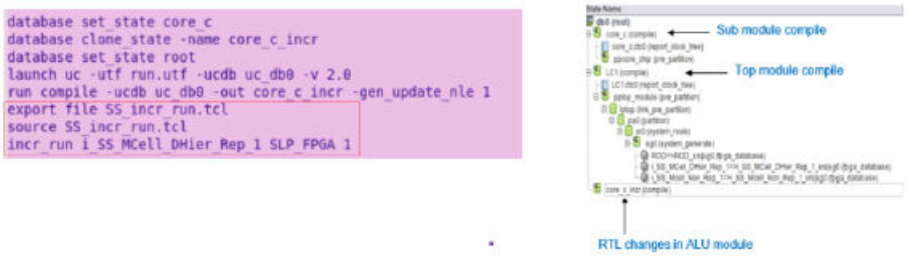
Perform the following steps to run selective rebuild:

1. Create a clone of the compile node which includes rtl files that have been modified.
2. Run uc2 compile using updated rtl files. Compile has to be run with gen_update_nle 1 to generate nle file.
   - Note: The ucdb database and filelist must be same as the compile node in which rtl file has not been modified. Create a backup of rtl files and ucdb database so that the rtl files and ucdb names remain the same.
3. Export SS_incr_run.tcl. Call the procedure with required arguments to rerun SS with the generated nle file.

```
set srs_handle [load_design "/remote/sbg_qaresults/regression/10044/incoming_tests/        /feature_f
e_c_incr_4/synwork/RocketTile_rocket_comp.srs"]
#In the line below, replace RocketTile_rocket by the name of the view of dut_inst e.g. FB1_uA_dut
#define_current_view RocketTile_rocket
replace_view  {core.alu} $srs_handle:ALU
replace_view  {core.div} $srs_handle:MulDiv
```

Feedback

- The following figure displays changes in the user script to source *SS_incr_run.tcl*.



- The following figure displays updated SS_incr_run.tcl.

```
## Generated by Synopsys Netlist Linker
## version comp202212pcp1, Build 240R on Feb 15 2023 09:42:02

## Side TCL file for Modular Flow

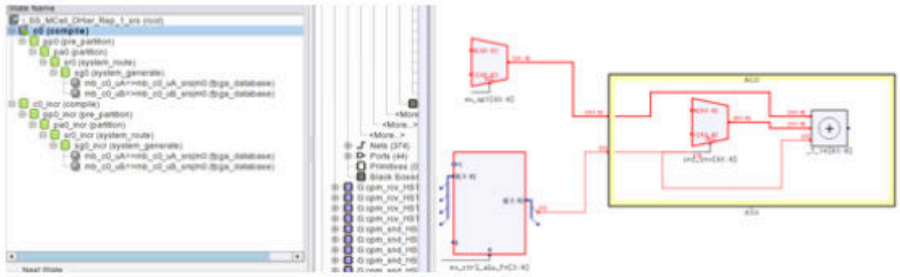
set STATE_SUFFIX {cincr}
set COMPILE_NLE {update_modified_views.nle}

proc set_if_unset {var val} {
        upvar $var v
        if {! [ info exists v]} { set v $val } elseif { "$v" != "$val" } { puts "Note: '$var' has been overridden to '$v'" }
}
set_if_unset LAUNCH_RUN 0
proc incr_run {SS SLP LAUNCH_RUN} {
        export file update_modified_views.nle -path $SLP/$SS
        exec sed -i "s/#define_current_view/define_current_view/g" $SLP/$SS/update_modified_views.nle
        export file modified_modules.txt -path $SLP/$SS
        export file SS_incr_run.tcl -path $SLP/$SS
        exec echo "source $SS\_srs.tcl" >> $SLP/$SS/SS_incr_run.tcl
        if {$LAUNCH_RUN} {
                launch protocompiler -script $SLP/$SS/SS_incr_run.tcl
        }
}
source i_SS_MCell_DHier_Rep_1_srs.tcl
```

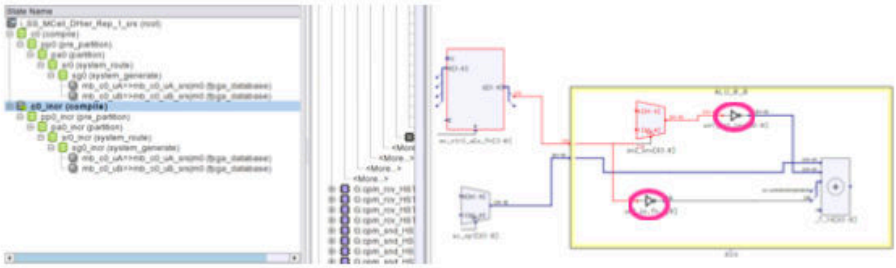
- The following figure displays SS Compile before and after RTL modification.

Feedback

SS compile before RTL modification



SS compile after RTL modification



- The following figure displays RTL changes in ALU mode.

RTL changes in ALU module

assign _T_11 = io_in1 + in2_inv; → assign _T_11 = io_in1 - in2_inv;
assign _T_14 = _T_12 + _GEN_1 → assign _T_14 = _T_12 - _GEN_1;

Bottom-up Compile

Bottom up compile is used to update entire module without change in interface. Bit stream can be generated for the subsystems that are affected by RTL modification.

Example scenario

Perform the following steps for bottom-up compile:

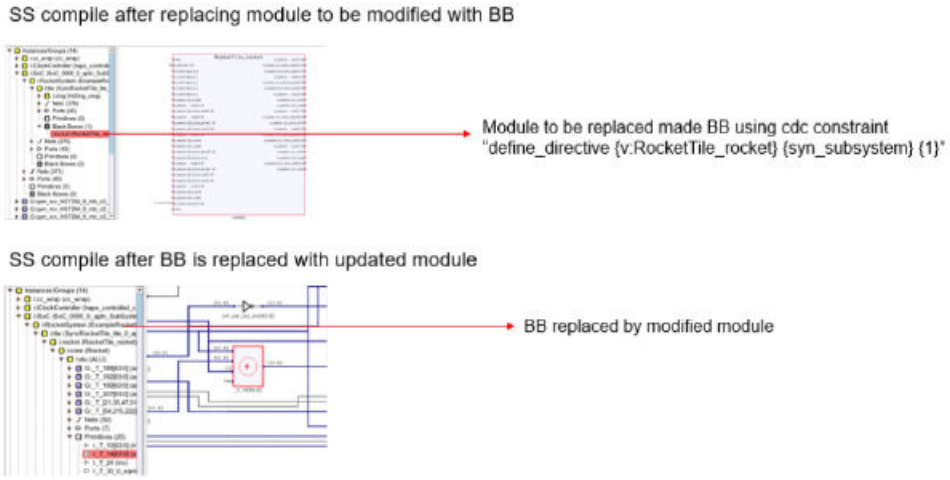
1. Modify the module compiled in *core_c* compile node.
2. Run compile with modified *rtl*.
   - Run compile `-ucdb uc_db0 -out core_c_incr`
3. Rerun subsystem compile by replacing modified module with blackbox. Use *syn_subsystem* file in *cdc file* of subsystem compile on the modified module.

Feedback

- Run compile `-cdcfile cdcfiles.txt top_module TraceBuildLib.SS_MCell_DHier_Rep -out c0_BB`.

4. Link the modified module with updated compile node.
   - Run compile `-srcstate c0_BB -srcstate db0|core_c_incr -out compile_incr`.

The following figure displays the scenario of SS compile after replacing the module to be modified with BB and SS compile after BB is replaced with the updated module.



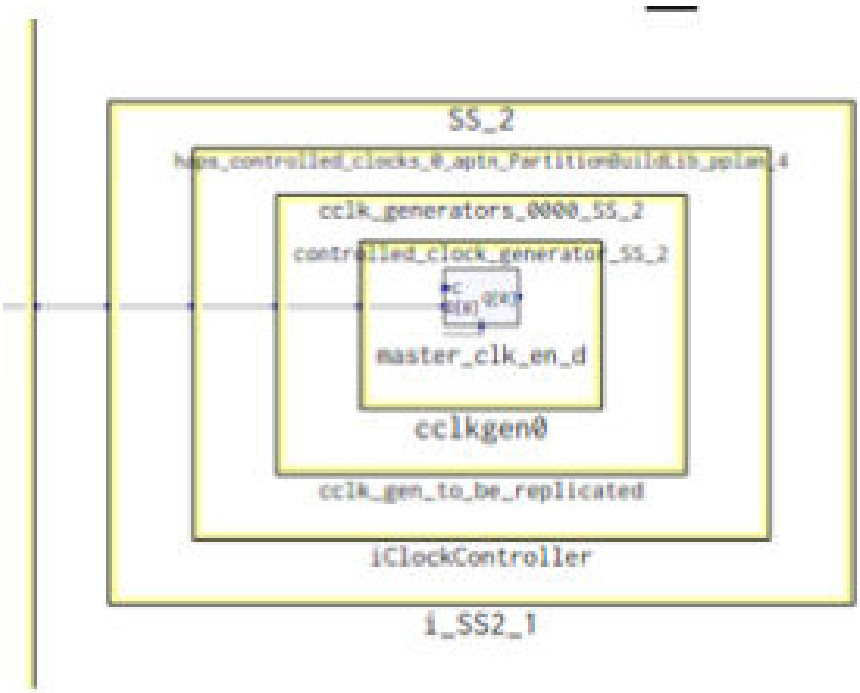
Modular Time Budget

DPO is written from input ports of SS (which are connected to ROD) to clock in SS.

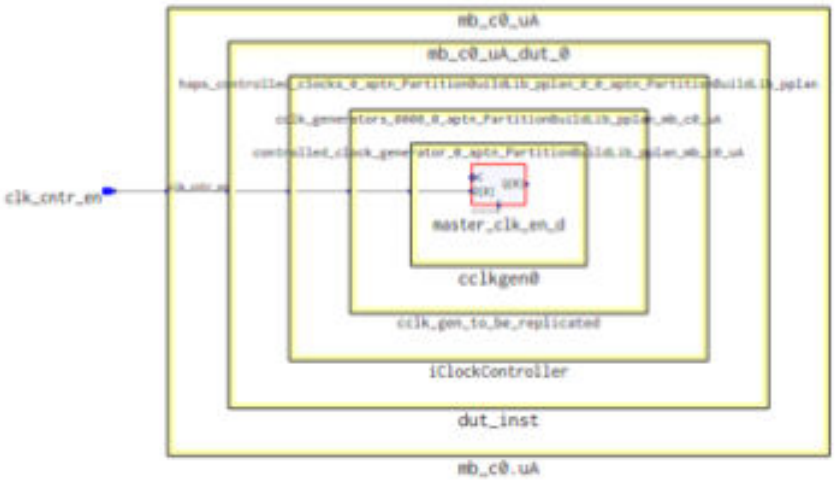
Scenario 1: First pass - Time budget

Feedback

The following figure displays Pass-1: RoD ->SS_2 connection:



The following figure displays path inside the subsystem:



For replicated sub-systems, the worst path among all sub-systems are considered for DPO.

Feedback

The following figure displays the path constrain written by Pass 1 for the RoD-SS connection.

31.3 is the DPO –from {p:clk_cntr_en} -to MASTER_CLOCK

```
# BudgetableDelay (1,1) + SlackDistribution (30,2) = 31.3  (after mapping)
set_datapathonly_delay -high_priority -rise_to [get_clocks -include_generated_clocks {MASTER_CLOCK}] -from {p:clk_cntr_en} 31.3
```

Scenario 1: Second pass - Time budget

In this scenario, both first pass (I_SS1_1_timing.fdc) and second pass FDC files are used in SLP run.

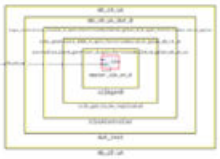
DPO allocated in First pass (31.3) is used in second pass as a requested period.

Timing report path consists of the entire path that is defined in first pass (clk_cntr_en to MASTER clock).

FDC file generated in Pass1 is read in Pass2 run

```
Reading constraint file: /remote/sbg_qaresults/regression/44107/slowfs/sbg_builds20/      /haps_soc_demo_v0/SLP_FPGA_ib/i_SS1_1/i_SS1_1_timing.fdc
Reading constraint file: /remote/sbg_qaresults/regression/44107/slowfs/sbg_builds20/      /haps_soc_demo_v0/SLP_FPGA_ib/common_files/clockgen_cons.fdc
Reading constraint file: /remote/sbg_qaresults/regression/44107/slowfs/sbg_builds20/      /haps_soc_demo_v0/SLP_FPGA_ib/i_SS1_1/i_SS1_1_haps_timing.fdc
```

Path report in Pass2 with the constraint generated in Pass1



```
Path information for path number 1:
    Requested Period:                       31.300
  - Setup time:                             -0.018
  = Required time:                          31.318

  - Propagation time:                       8.311
  = Slack (critical) :                      23.007

  Number of logic level(s):                 1
  Starting point:                           clk_cntr_en / clk_cntr_en
  Ending point:                             wb_c0.uA.dut_inst.iClockController.cclk_gen_to_be_replicated.cclkgen0.master_clk_en_d / D
  The start point is clocked by             System [rising]
  The end   point is clocked by             MASTER_CLOCK [rising] {rise=0.000 fall=50.000 period=100.000} on pin C
--Timing constraint applied as path with max delay (datapathonly) 31.299999 (from p:clk_cntr_en rise_to [get_clocks -include_generated_clocks MASTER_CLOCK])

Instance / Net                                                                  Pin      Pin                 Arrival   No. of
Name                                                                Type        Name     Dir     Delay     Time      Fan Out(s)
-------------------------------------------------------------------------------------------------------------------------------
clk_cntr_en                                                         Port        clk_cntr_en  In      0.000     0.000 r   -
clk_cntr_en                                                         Net         -        -       7.248     -         1
wb_c0.uA.clk_cntr_en_ibuf                                           IBUF        I        In      -         7.248 r   -
wb_c0.uA.clk_cntr_en_ibuf                                           IBUF        O        Out     0.790     8.038 r   -
clk_cntr_en_c                                                       Net         -        -       0.273     -         1
wb_c0.uA.dut_inst.iClockController.cclk_gen_to_be_replicated.cclkgen0.master_clk_en_d  FDR         D        In      -         8.311 r   -
-------------------------------------------------------------------------------------------------------------------------------
Total path delay (propagation time + setup) of 8.293 is 0.772(9.3%) logic and 7.521(90.7%) route.
```

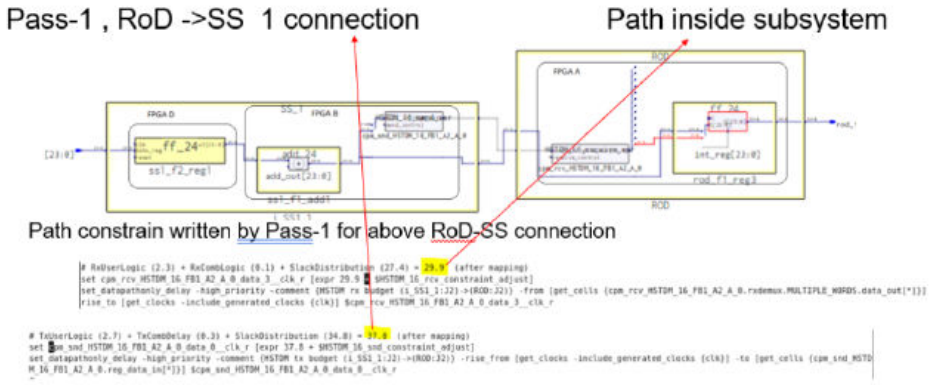
Scenario 2: Three FPGA path - First Pass – Timebudget

In this scenario, HSTDM is inserted across ROD-SS.

DPO of first pass of SS1 is from clock to HSTDM data in port.

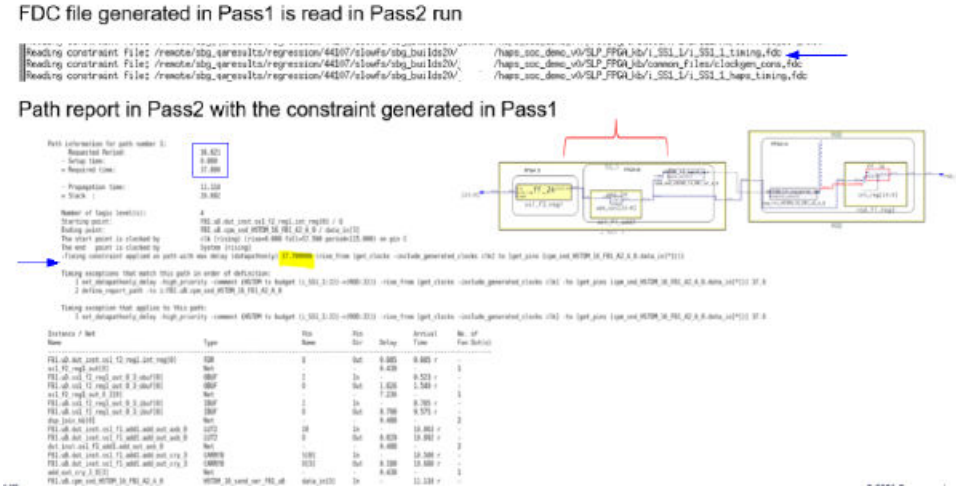
For SS1, DPO is from ss1_f2_reg1/clk to HSTDM data_in --- 37.8.

For ROD, DPO is from HSTDM data_out to rod_f1_reg3/clk --- 29.9



Scenario 2: Three FPGA path Second Pass – Time budget

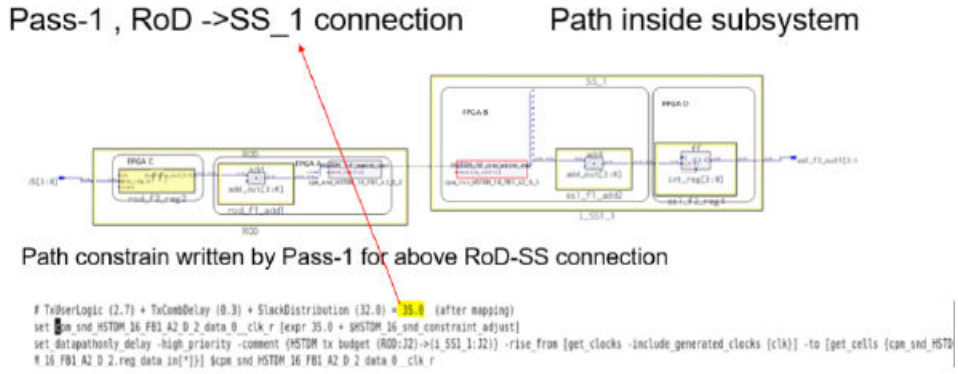
In this scenario, both First pass (I_SS1_1_timing.fdc) and Second pass FDC file are used in SLP run. DPO allocated in first pass(37.8) is passed to second pass as Requested Period. Timing Report path consists of entire path which is defined in First pass (ss1_f2_reg1/clk to HSTDM data_in) as highlighted in red.



Scenario 3: Four FPGA - First Pass – Time budget

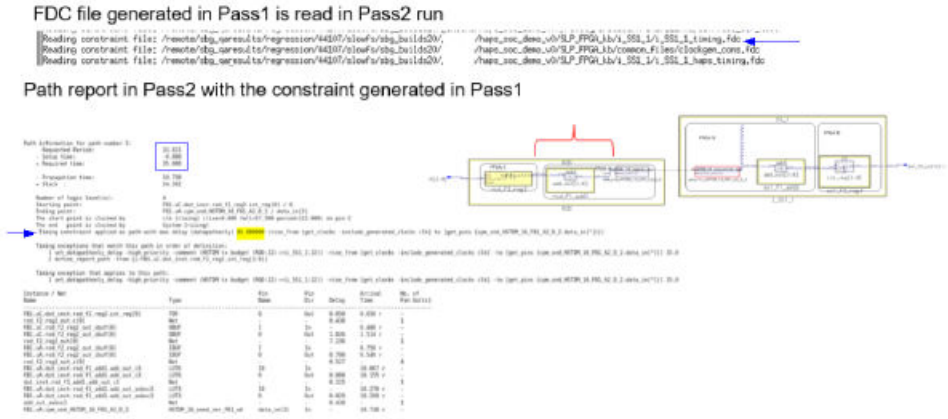
In this scenario replicated subsystem is based on worst path. The scenario is same as Scenario 2: Three FPGA path Second Pass – Time budget, except for the number of FPGAs used for path analysis.

Feedback



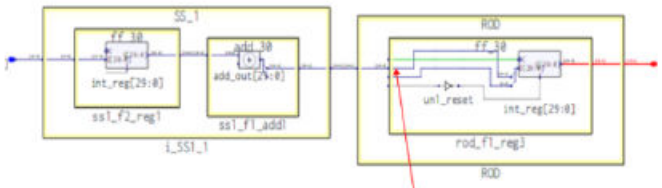
Scenario 3: Three FPGA path Second Pass – Time budget

This scenario is same as Scenario 2: Three FPGA path Second Pass – Time budget, except for the number of FPGAs.



Scenario 4: Feedthrough – Pass 1 Time budget

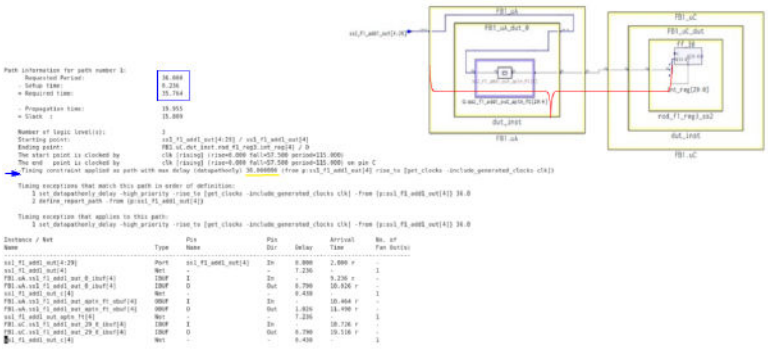
In this scenario, DPO is written from input ports of ROD (which are connected to SS) to clock in ROD.



Path constrain written by Pass-1 for RoD

```
# BudgetableDelay (1.1) + SlackDistribution (34.9) = 36.0  (after mapping)
set_datapathonly_delay -high_priority -rise_to [get_clocks -include_generated_clocks {clk}] -from {p:ss1_f1_add1_out[4]} 36.0
```

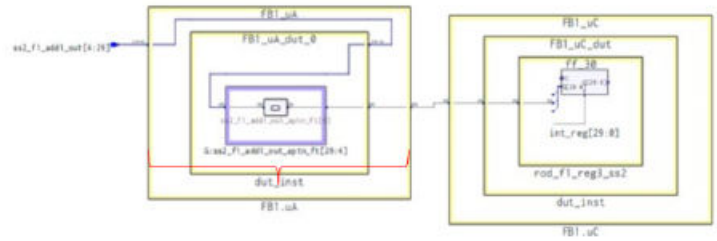
In this scenario, timing report path consists of entire path defined in First pass (ss1_f1_add1_out[4] to clk). DPO allocated in first pass is passed to second pass as a requested period --> 36. Feedthrough path is also taken care in timing path report.



Scenario 4: Feedthrough – Pass 2 Time budget

In this scenario, FB1.uA has feedthrough as highlighted in red in the following figure.

DPO written for Feedthrough path in Pass 2.



Path constrain written by Pass-2 for above feedthrough connection

```
# BudgetableDelay (2.1) + SlackDistribution (10.7) = 12.8  (after mapping)
set_datapathonly_delay -high_priority -from {p:ss1_f1_add1_out_0[4]} -to {p:ss1_f1_add1_out_aptn_ft[4]} 12.8
```

Debug Flow

This section describes the RTL instrumentation in modular flow.

Perform the following to enable RTL debug in modular flow.

First Pass

1. Set the following options to enable RTL instrumentation.
   - Option `set debug_dumpvars 1`
   - Option `set subsystem_partition 1`
2. Provide RTL signal to be instrumented in $dumpvars
   - Make sure $dumpvars have a sampling clock.
3. Run launch UC.
4. Run compile to SG without inserting the IDC file.

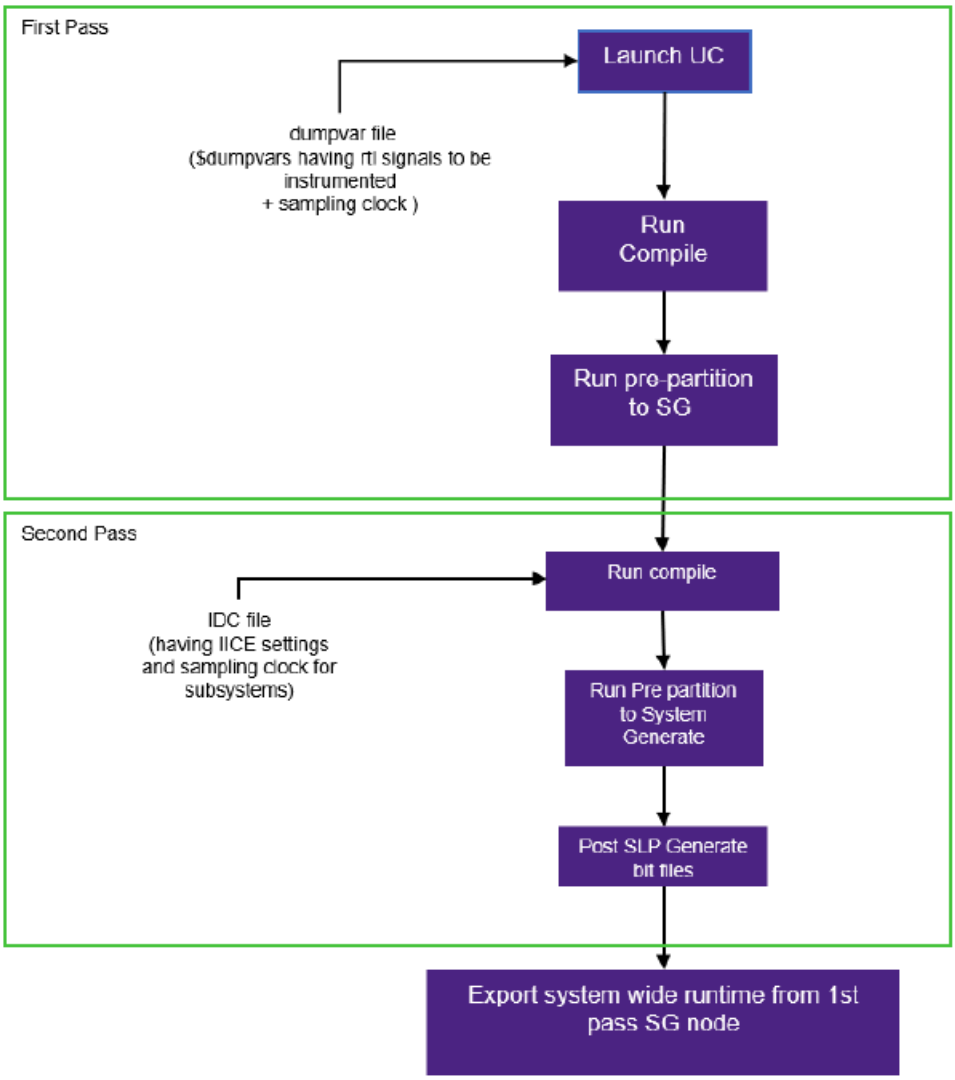
   Second Pass

5. For subsystem, provide IDC at compile.
6. Make sure IDC has IICE settings and sampling clock (RTL hierarchy) defined.

- Tool instruments signals are present in subsystem automatically based on signals in $dumpvars.
- After successful generation of all bit files, you can perform export runtime for the whole system from first pass SG node.

Feedback

The following is a flow chart describing the modular flow:



First Pass: Initial Debug Support

Debug insertion is supported for second pass and SLP FPGA stage. You cannot add debug in first pass. RTL instrumentation is possible using $dumpvars file. Signals in $dumpvars provided in the first pass is instrumented automatically in the second pass.

Cross trigger for inter-subsystem

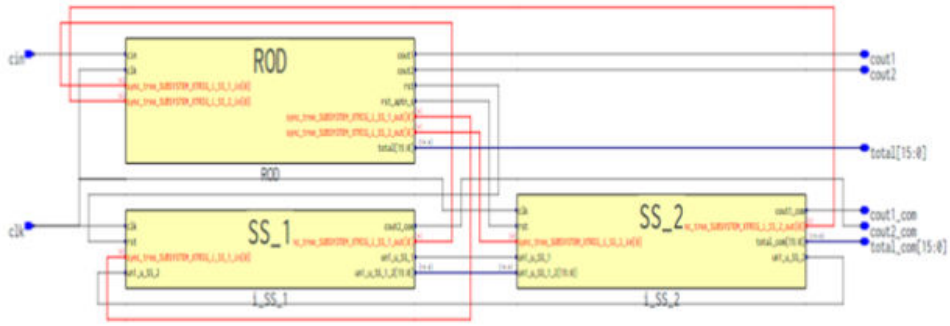
Cross triggering across subsystems is done using a simple trigger-in / trigger-out tree. Each subsystem, even if not instrumented, adds a cross trigger IP (like a bypass subsystem). During runtime, a signal is available to trigger based on other subsystem's instrumentation.

Feedback

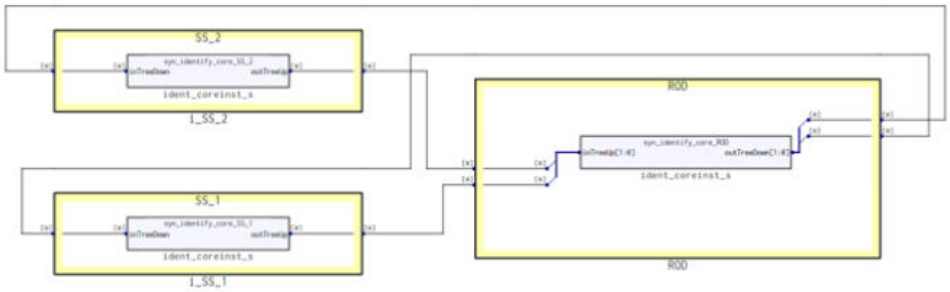
First Pass: Debug Xtrigger connections

The xtrigger network is established at system generate state in first pass.

The following figure represents first pass system generate schematic view of inter-subsystem cross trigger tree where Xtrigger ports are added in first pass of system route and a connection between subsystem is established.



The following figure represents first pass system generate schematic view of inter-subsystem cross trigger tree. In system generate, the ports are connected to ident_coreinst_s within each subsystem.

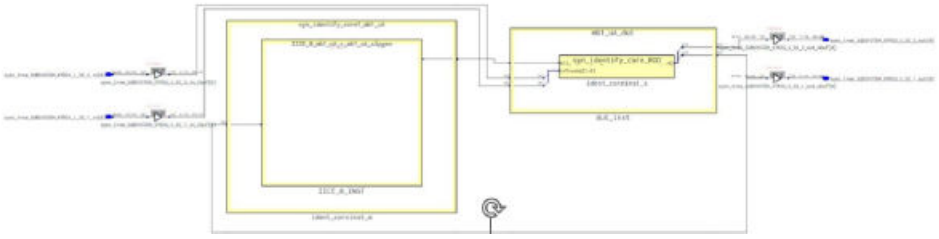


Second Pass – Debug IICE connection

In this example scenario, hyper source and hyper connect to debug IP is connected at post SLP map stage with xtrigger network. Subsystem will have SRS.ident_coreinst_s.SS_xtrigger automatically instrumented and displayed in the system generate *instrumentation.log* file. The IICE is connected to cross trigger combiner IP.

The following figure displays ROD 's IICE connection with cross trigger ports.

Feedback

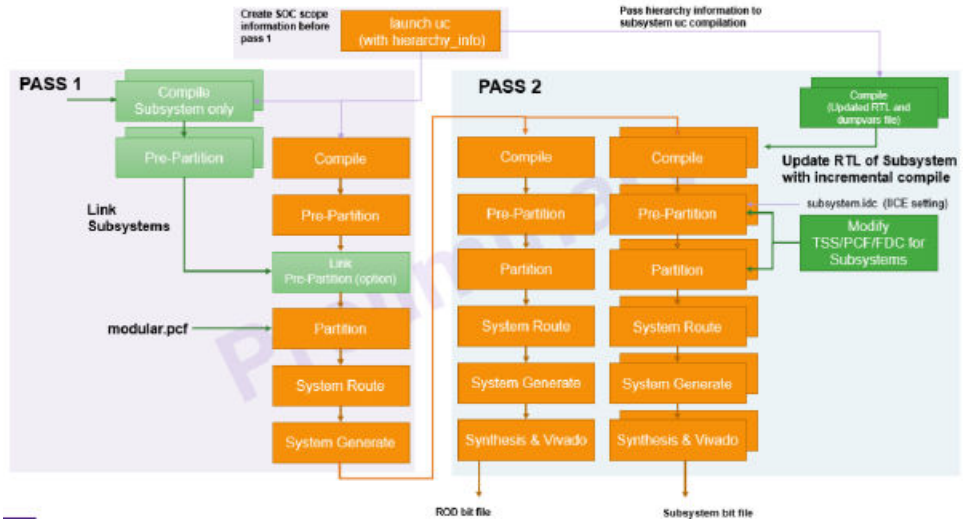


Modular Flow Bottom up dumpvars support

The modular flow bottom up dumpvars support helps with:

- TaT Reduction with Incremental RTL instrumentation in pass 2
- Generation of scope information through VCS before pass1
- Usage for RTL instrumentation during any independent UC compilation with pass1 or pass 2.

The following is a flow diagram for modular bottom up debug flow:



Example: Create SOC scope information

- Generate the SOC scope information before pass 1.

  Direct usage during launch uc with a separate run before pass1.

  Requires FpgaInternalBeta license, does not require -ucdb argument

  ```
  database load pre_db -autocreate -technology HAPS-100

  launch uc -utf run.utf -v 2.0 -hierarchy_info <path_to_hier_info>
  ```

Feedback

- Pass hierarchy information to subsystem compilation during pass 1.

  Direct usage during *launch uc* for pass1, requires *FpgaInternalBeta* license.

```
c
database load dbTop -autocreate -technology HAPS-100
option set subsystem_partition 1
launch uc -utf run_ss1.utf -v 2.0 -ucdb ucdb_ss1 -hierarchy_info
<path_to_hier_info>
run compile -ucdb ucdb_ss1 -out c_ss1
report ctc -fdc ss1.fdc
run pre_partition -fdc ss1.fdc -out pp_ss1
launch uc -utf run_top.utf -v 2.0 -ucdb ucdb_top -hierarchy_info
<path_to_hier_info>
run compile -ucdb ucdb_top
run pre_partition …
run link_pre_partition -srcstate pp_ss1 …
```

Incremental RTL instrumentation at subsystem

Example usage during incremental re-compilation of subsystem during pass2:

```
database load dbSS1 -autocreate -technology HAPS-100
launch uc -utf run_ss1.utf -v 2.0 -ucdb ucdb_ss1_incr -hierarchy_info
<path_to_hier_info>
run compile -ucdb ucdb_ss1_incr -out c_ss1_incr -compile_only 1
database set root
run compile -srclist $SS1_SRS_LIST -cdclist $CDC_LIST -out c0_ss1
-top_module TraceBuildLib.SS1
database set root
run compile -srcstate c0_ss1 -srcstate dbSS1|c_ss1_incr -top_module
TraceBuildLib.SS1
run pre_partition -tss board.tss -fdclist fdc_files.txt -idc ss1.idc
```

Export runtime structure in modular flow

Feedback

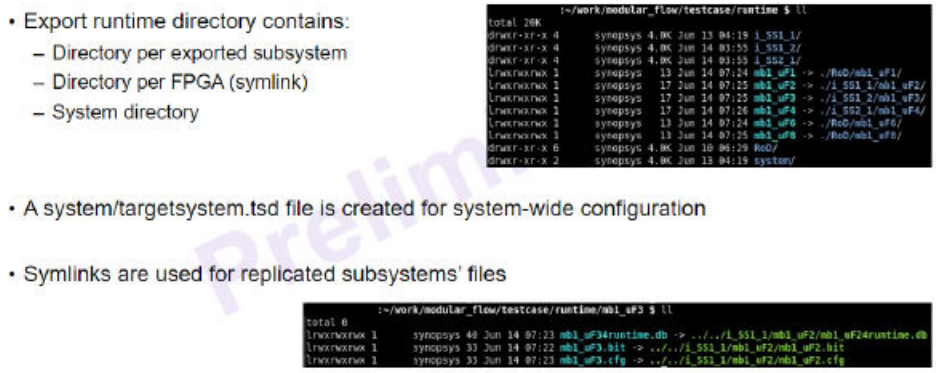
Export runtime directory contains the following:

- Directory per exported subsystem
- Directory per FPGA (symlink)
- System directory

A system/targetsystem.tsd file is created for system-wide configuration.

Symlinks are used for replicated subsystem files.

The following figure displays the structure of export runtime:

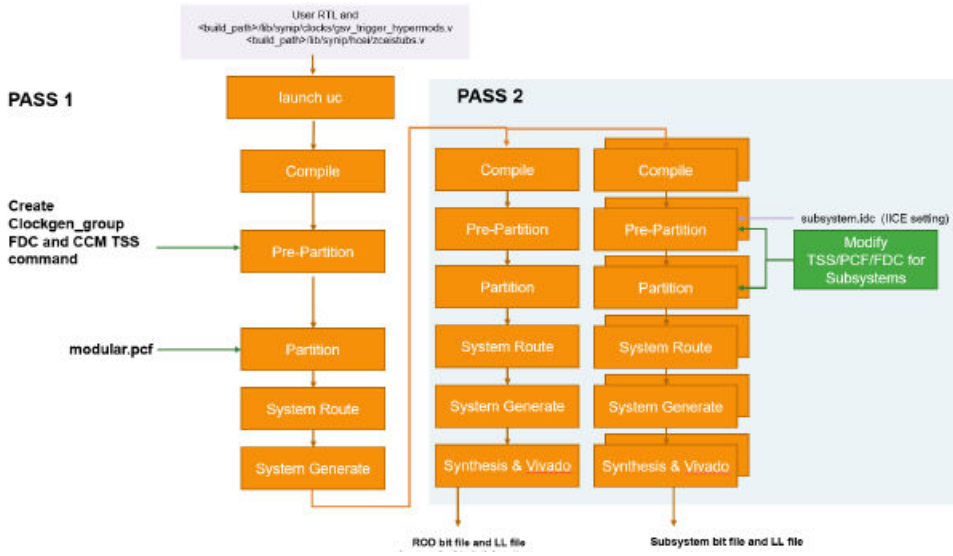


Sync GSV in Modular Flow

Sync GSV Flow in modular is supported only with new clock generator.

Feedback

The following figure displays the software flow use model:



To enable CCM insertion in modular, perform the following steps:

1. In Pass1 stage of the flow, to enable GSV/eGSV flow, HAPS clock generator, and other required options:
   - Enable readback by setting the prepare_readback option:
     - For GSV: *set option prepare_readback 1* (default is 1)
     - For eGSV: *set option prepare_readback 2* (needs to be turned on manually)
   - Enable RTL instrumentation through $dumpvars and connect debug IP trigger to stop clocks.

     ```
     option set debug_dumpvars 1 (default ON)
     ```
   - Enable the HAPS clock generator mode.

     ```
     option set haps_clockgen_mode 1
     ```
   - Enable Verdi correlation. (mandatory for UC2 flow).

     ```
     option set verdi_mode 1
     ```
2. For Unified Compile flow, explicitly include the following .v files for the clock port along with other source files:

   ```
   <build_path>/lib/synip/clocks/gsv_trigger_hypermods.v
   ```

   ```
   <build_path>/lib/synip/hcei/zceistubs.v
   ```

3. Declare the CCM source using the TSS command. For example:

   ```
   board_system_configure -create_ccm_clocks -source mb1.uA -load {mb1.uA mb1.uB}
   ```

   This ensures that the CCM source resides in FPGA A and the CCM load is replicated on both FPGA A and FPGA B.

4. Create CCM Clocks. To create CCM clocks, define the clock constraint in the FDC file with the required frequency and add it to the design as input by declaring the clock port input. For example:

   ```
   create_clock -name Clock1_CCM [get_ports {p:clk0}] -period 40 -waveform {0 20}

   create_clockgen_group {CLOCK1_CCM} -group {grp1}
   ```

   In this example, a frequency of 25 MHz is generated and is fed as input to the design through top level clock port clk0.

5. To stop the CCM clocks for readback, you can connect the debug IICE triggers in the RTL to gsv clock stop module as shown below:

   Example for IICE Triggers:

   ```
   iice_trigger #("IICE_0") trigger_inst1(trigger_iice_0);

   iice_trigger #("IICE_1") trigger_inst2(trigger_iice_1);

   assign trigger_in[0] = trigger_iice_0;

   assign trigger_in[1] = trigger_iice_1
   ```

6. Connect the triggers to a user GSV clock stop trigger module as shown below:

   ```
   ccm_gsv_clk_stop_user #(2) clk_stop_inst (trigger_in);
   ```

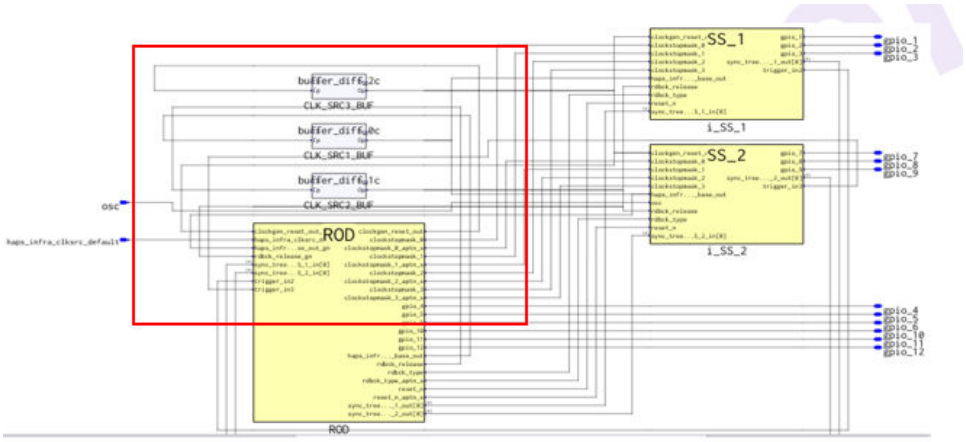
7. Make sure that the trigger wires from each IICE is manually partitioned to the correct FPGA. If the trigger is auto partitioned to other FPGA where IICE is not located, the mapper fails with hypersource-hyperconnect error.

8. Run through the rest of the flow to generate bit files and LL files. You can check the CCM reports at the following stages:

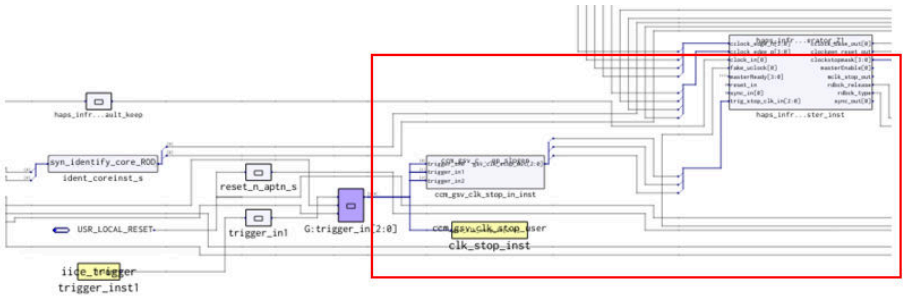
   - prepartition.srr (clock summary)
   - system generate time_budget.log (performance summary)
   - Vivado timing reports

Feedback

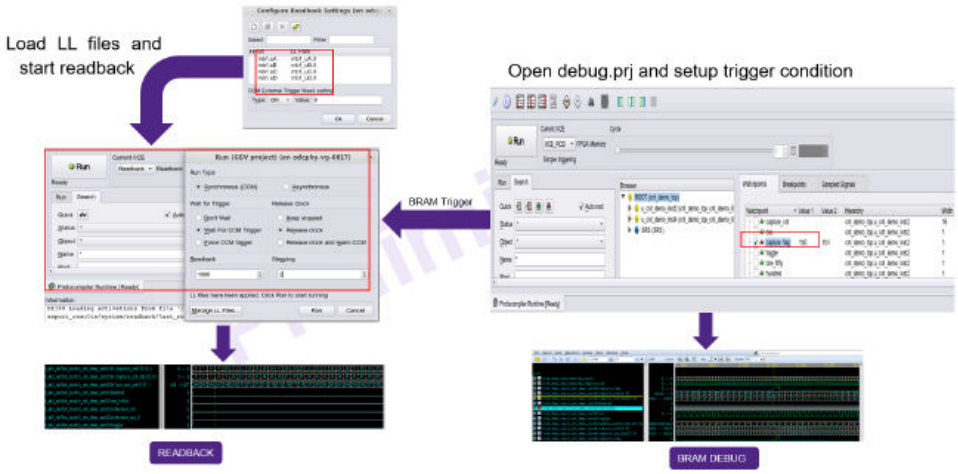
The following figure displays the CCM clock connection between ROD and SS:



The following figure displays the GSV clock stop connection to CCM source trig stop in port.



The following figure displays the hardware flow use model:



Multi-die Design

Multi-die design is a scenario where you can connect multiple replica of the same die together. These dies are interconnected only through a set of mirrored Tx-Rx connections. RTL design in the software contains the implementation of one die only. You must manually add a loopback connection within the same die by editing the RTL. You can specify which manually-created loopback nets inside the design hierarchy are to be brought out to the FPGA level and connect to the Tx/Rx of the same die for the software flow.

For example, you can specify design loopback net

{ u_die_1.data1_ss1_lpbk[1599:0]} to be connected externally using the cable specified in connectors for *conn_ss1_lpbk_1*. In the software flow, they are connectors of same Die/ system. But during real hardware, they perform as equivalent connector to another die/ system.

PCF

```
loopback_nets

<list of nets>

-trace_group < list of trace groups >

-function < function group name of the loopback connections >
```

TSS connectivity within the same FPGA in software flow:

```
board_system_create -interconnect -manual HT3C_CABLE_200 -name
conn_ss1_lpbk_1 -connector {FB1.F6J2 FB1.F6J3}
```

TSS connectivity on real hardware:

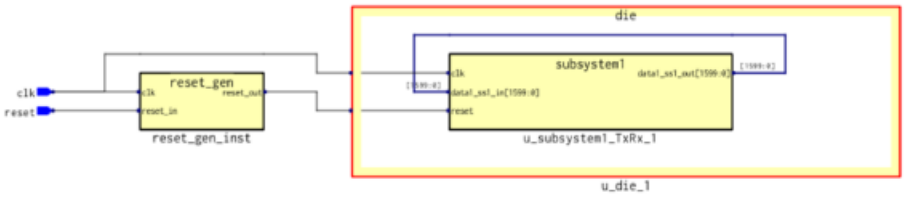
```
board_system_create -interconnect -manual HT3C_CABLE_200 -name
conn_ss1_lpbk_1 -connector {FB1.F6J2 FB2.F6J3}

board_system_create -interconnect -manual HT3C_CABLE_200 -name
conn_ss1_lpbk_1 -connector {FB1.F6J3 FB2.F6J2}
```

Compile Schematic

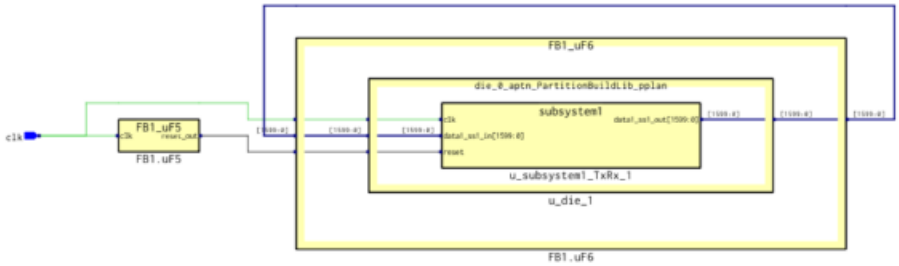
The following figure is an example for loopback inside hierarchy.

Feedback



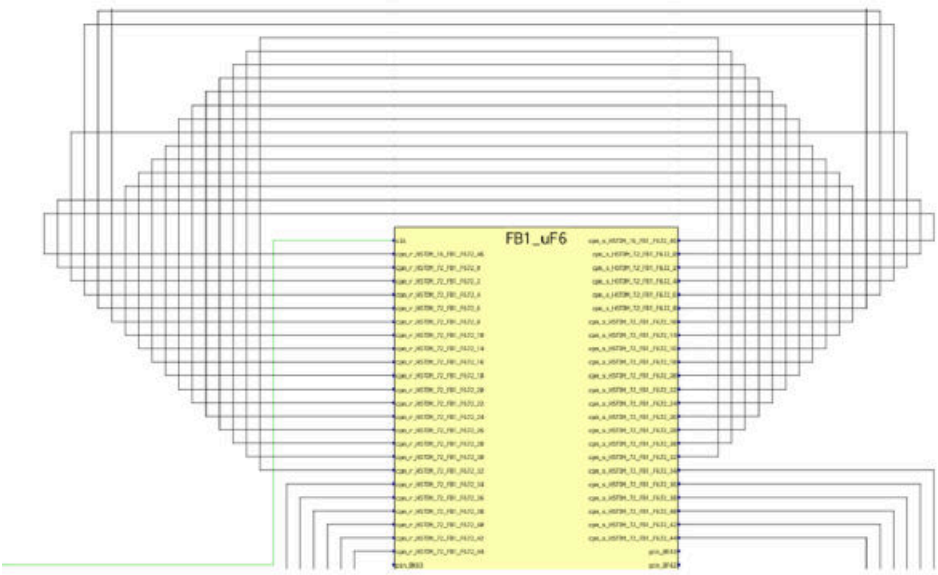
Partition Schematic

The following figure is an example of loopback pushed out to FPGA level.



System Route Compile Schematic

The following figure is an example of TDM added in loopback path.



See loopback_nets.

Modular Flow Usage Recommendations

Best practices and limitations are described in this section:

Best Practices

- You must rerun the first pass if time budgets are significantly impacted by an incremental change.
- Second pass partition should be run in slack mode.
- Debug-$dumpvars: provide as a separate, standalone RTL file.
- Debug-GSV: use new HAPS clockgen/Clock Control Module (CCM).

Limitations

- Memory models in replicated subsystem (SS) are not supported.
- Dynamic force in replicated subsystem is not supported.
- Dynamic force is not supported for incremental modular bottom up flow.
- The `database apply_state -idc` command is not supported in the modular flow.
- Selective rebuild capability is limited by gated clock conversion across subsystems, specifically when there are replicated clocks from subsystem to Rest Of Design (ROD).
- Debug: Cross triggering between subsystems is asynchronous.
- Debug-GSV: Verdi correlation and data expansion is currently not supported.
- Debug-GSV: Use mode enabling dynamic switching of driver and controlled clock not supported.
- Debug-GSV: For replicated subsystems, the replicant subsystem names are the same as the source subsystem hierarchy names in readback.
- Debug-SVA: Do not group SVA signals using the `-uc_groups` option.

6

Running the Implementation Flow

This chapter describes the basic flow through synthesis, placement, and routing, without debugging.

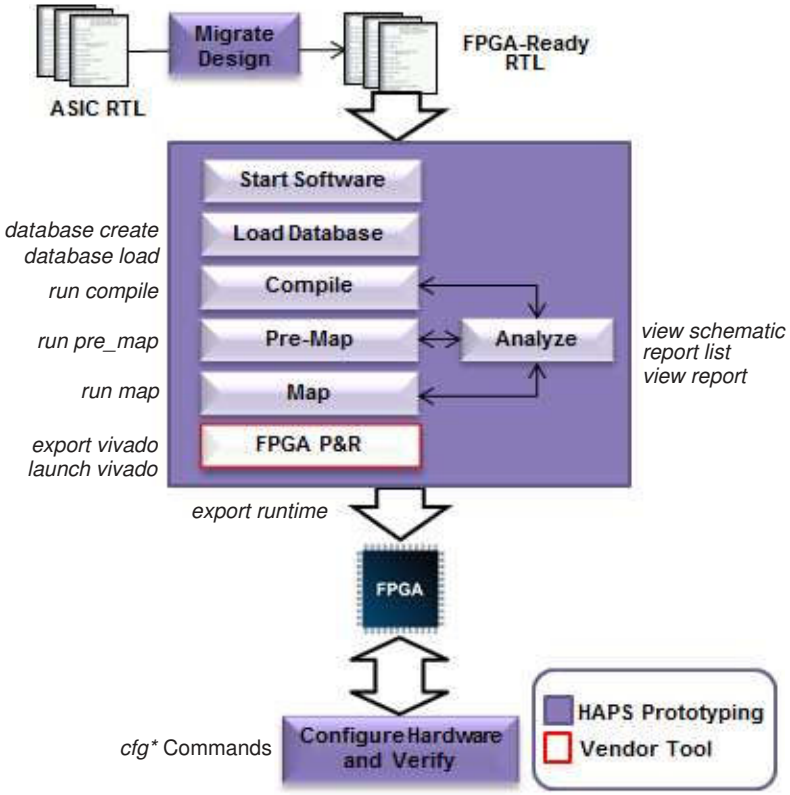
- The Basic Implementation Flow
- Running Pre-Map
- Mapping to Hardware
- Mapping the Design
- Setting Auto-SPO
- Synthesizing Based on Design Intent
- Using Distributed Processing
- Analyzing Results
- Checking Timing Results
- Resolving Errors and Warnings
- Running Simulation
- Running Place and Route
- Analyzing Routing Congestion
- Running Synthesis Using Tcl Batch Script
- Working with Tcl Scripts and Commands
- Preparing to Run on the Hardware

For information about a debug flow, see Instrumenting and Debugging Designs.

The Basic Implementation Flow

The following figure shows the steps in the basic design flow to implement an FPGA, along with the main commands used at each step. For details about the commands, see the *Command Reference Manual*.

Feedback



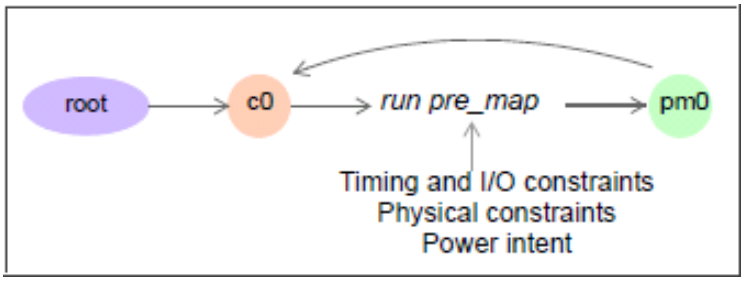
Some steps have been described earlier and are not repeated in this chapter. The following table points to the information for each step in the flow.

Migrate Design	Migrating Designs for Synthesis
Load Database	Creating and Compiling Databases
Compile	Adding Design Files Compiling the Design
Premap	Running Pre-Map
Map	Mapping the Design
FPGA Place and Route	Running Place and Route
Analyze Results	Analyzing Results
Export to Bit File	Exporting Files for Runtime
Configure Hardware and Verify	

Feedback

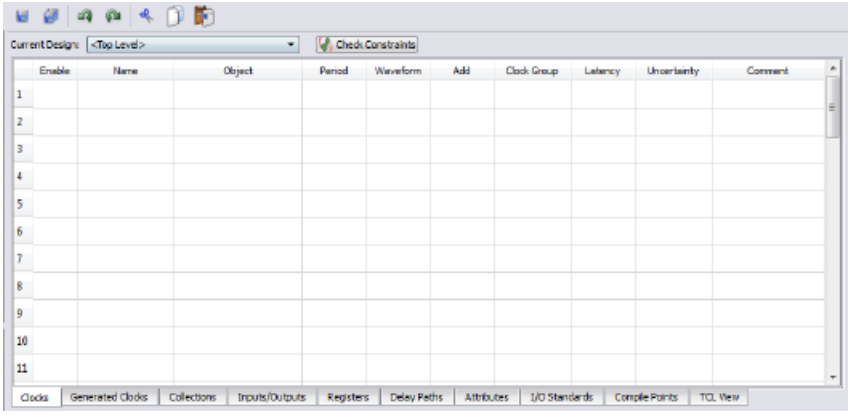
Running Pre-Map

After compiling the design, you can run the pre-map phase of synthesis. At this stage, specify constraints that optimize timing and set signal-to-pin assignments, power intent, and post-compile substitutions in the gate-level netlist.



The following procedure describes how to implement this phase as a standalone step, but you can also run it as part of a complete synthesis process based on design goals, by following the steps described in Synthesizing Based on Design Intent.

1. Start with a compiled design.
2. Set timing constraints in an existing *fdc* file or create a new one.
   - Use *edit fdc -mode gui* to open the GUI where you can specify the constraints. This opens the SCOPE® spreadsheet, where you can edit constraints and add attributes. Alternatively, use a text editor to create or edit an *fdc* file.



   - You must define the global clock with the *create_clock* constraint. For example: `create_clock {gclk0} -period {0}`.
   - Include the *fdc* file as input to the pre-mapper with the *-fdc* argument to the *run pre_map* command (step 6).
3. Set synthesis attributes for I/O planning in the *fdc* file.

Feedback

- If you are using the GUI to define the attributes, go to the *Attributes* tab in the SCOPE spreadsheet.
- Include the *fdc* file as input to the pre-mapper with the *-fdc* argument.

4. Optionally, specify power requirements using the UPF format.
   - Define power domains and specify isolation and retention requirements in a UPF file. See Including UPF Specifications for detailed information.
   - Add the UPF file to the design with the `load_upf` command, or specify the file with the *-upf* argument when you run the pre-mapper.
5. Specify other synthesis options.
   - Specify options, such as automatic conversion of gated clocks, using the `option set` command in a Tcl file. See the following table for the options you can set at this design stage.

Table 3 Pre-Map Options

Option	Description
Command Arguments (*run pre-map* Command)	
-fdcfileName	Loads the specified FDC file, and uses the defined constraints in the pre-map phase.
-upffileName	Loads the specified UPF file and uses the power constraints specified in the pre-map phase. See Including Unified Power Format (UPF) Specifications for information about using this file.
Option Settings (*option set*). Click the **Options Editor** icon for a comprehensive list.	
automatic_compile_point 0|1	Determines whether the tool automatically divides up the design into compile points, without creating additional hierarchy. Compile points are RTL partitions of the design that are treated as independent blocks by the synthesis engine.
fix gated_and_generated_clocks1|0	Determines whether the gating is separated from the clock inputs in gated clocks, and whether generated clock logic is replaced with logic that uses an initial clock with an enable. The default, 0, does not specify these conversions. For details about gated clock conversion, see Converting Gated Clocks.
force_async_gencl k_conv 1|0	Determines whether generated clocks are converted, and whether datapath latches that are driven by generated clocks are converted to a flip-flop and multiplexer when the latch is set or reset by asynchronous signals. By default, the latches with asynchronous control signals are not converted. See Defining Clocks for Gated Clock Conversion for information about its use and an example.

Feedback

Option	Description
max_parallel_jobs number	Determines the maximum number of processing jobs to run in parallel. Use this in conjunction with *automatic_compile_point* to reduce runtime.

- For partitioned FPGAs, you must define these options before partitioning. Define the technology when you create the database (`database create`) and the speed grade in the TSS file (`board_system_create`). Specify one speed grade per HAPS system; you can use different speed grades in the same setup. Do not change these inherited settings at the individual FPGA level.
- Save the Tcl options file and then source it. When you source the file, the options specified in it apply to this database state and the ones below it in the database tree.

  The first command launches a Tcl editor to create an options file called *MyPrepOptions.tcl*, and the second sources the file:

  ```
  edit file MyPrepOptions.tcl
  source file MyPrepOptions.tcl
  ```

6. Start with a compiled design and run pre-mapping with the `run pre_map` command, or click *Pre-Map* in the left panel to run it from the GUI.
   - If you have FDC constraints, specify them with the *-fdc* argument to the command. For example: `run pre_map -fdc myConstraints.fdc`.
   - If you have UPF constraints, put them in a *upf* file and specify it with the *-upf* argument to the command.

     When you run this command, the tool generates a new database node, called *pm0* by default. The tool generates a log file and a schematic, which you can view in a schematic window to analyze the results of the run.
7. Analyze the results of the reports and view the schematic.
   - To view the I/O report with the HAPS pin mapping assignments, select it from the Report View in the GUI from the list of reports generated after the pre-mapping stage. The **RTL View** section of the report lists the RTL ports and their corresponding HAPS connectors. The HAPS View section lists all legal HAPS I/Os, with the RTL bit ports, connectors, direction, and type.

   To export the file, first use the `report list` command to view the list of reports and get the name of the file. Then go to the premapped database state and use `export report` to export the HAPS I/O report.

- See Analyzing Results for more information about checking the log file and other results of the run.

8. Rerun premapping as needed until the design is properly constrained and intermediate logic substitutions and optimizations are properly applied.

   You can then proceed to mapping your design (Mapping the Design).

Mapping to Hardware

This section describes some hardware I/O planning considerations to take into account when describing the hardware for pre-map and map operations.

See the following for more information:

- Defining HAPS Pins in Multi-FPGA Designs
- Working with HAPS Systems

Defining HAPS Pins in Multi-FPGA Designs

For multi-FPGA designs, defining HAPS pins is a two-part process:

1. Define the hardware connections in the *tss* file. For example:

```
board_system_create -interconnect -manual DDR3_SODIMM2R_HT3 -name
DDR3_MEM -connector {fb1.A9 fb1.A10 fb1.A11}
```

2. Assign the pins with constraints in the *pcf* file. For example:

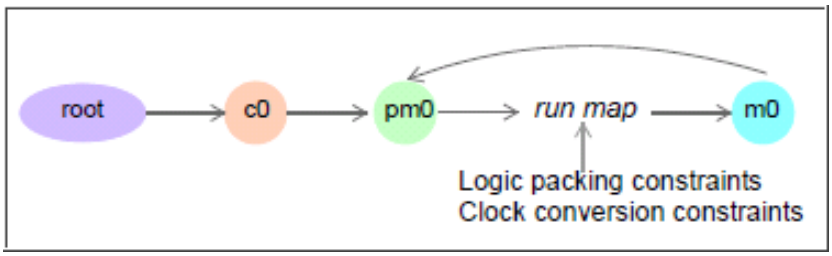
```
assign_port {ddr3_addr[15:0]} -trace {DDR3_MEM.DDR3_A[15:0]}
assign_port {ddr3_ba[2:0]} -trace {DDR3_MEM.DDR3_BA[2:0]}
assign_port {ddr3_dq[63:0]} -trace {DDR3_MEM.DDR3_DQ[63:0]}

port_attribute -ignore [portBusNameList]
```

Mapping the Design

Mapping is the last synthesis phase of FPGA synthesis, where the logic is mapped to technology-specific resources. At this stage, you can set options to convert and optimizes gated and generated clocks, and for packing.

Feedback



The following procedure shows you how to run this phase individually, but you can run the entire synthesis process based on design goals, by following the steps described in Synthesizing Based on Design Intent.

1. Start with a design that has gone through the pre-map stage (Running Pre-Map).

   If needed, use the appropriate *database* commands to close the current database and load an appropriate database node. See Loading and Navigating Database States for more information.

2. Specify synthesis mapping options in a Tcl file. Save the Tcl options file and then source it.

   The first command launches a Tcl editor to create an options file called *MyMapOptions.tcl*, where you can enter *option set* commands. The second command sources the file:

   ```
   edit file MyMapOptions.tcl
   source file MyMapOptions.tcl
   ```

   See the following table for the options you can set at this design stage.

Table 4 Mapping Options

Option	Description
Option Settings (*option set*). Click the Options Editor icon for a comprehensive list.	
automatic_compile_point 1|0	Determines whether the tool automatically creates compile points in the design.
retiming 1|0	Determines whether registers can be moved into combinational logic to improve performance. The default is 1, which allows this optimization.
continue_on_error	Specifies whether the mapping operation continues instead of stopping when it encounters errors in a compile point. When enabled (1, the default), the tool continues mapping the rest of the design. See Running Compilation or Mapping with Errors for details.

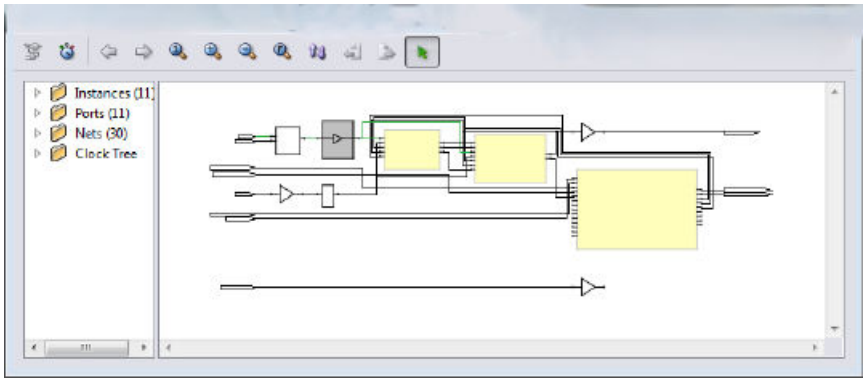
Option	Description
synthesis_strategy advanced|fast|routability	*advanced*– Performs additional optimizations to provide an output netlist with better QoR, but requires longer runtime compared to other modes. *fast* – Runs the mapper in a fast mode that is optimized for prototyping. *routability* – Performs placement-aware congestion reduction techniques to produce a routable netlist. There is a trade-off in timing QoR to improve synthesis runtime. This is the default.
no_sequential_opt 1|0	Determines whether sequential optimizations are performed. The default is 0, where optimizations are performed.
enable_prepacking 1|0	Determines whether the synthesis netlist uses advanced LUT combining, so that they can be packed more efficiently during place-and-route. This option is enabled (1) by default.
verification_mode 1|0	Enables or disables formal verification with the Formality tool. It is disabled by default. When enabled, the tool generates an SVF file for equivalence checking.
fix_gated_and_generated_clocks 1|0	Enables or disables gated and generated clock optimizations. This option is enabled by default.
force_async_genclk_conv1|0	Determines whether generated clocks are converted, and whether datapath latches that are driven by generated clocks are converted to a flip-flop and multiplexer when the latch is set or reset by asynchronous signals. By default, the latches with asynchronous control signals are not converted. See Defining Clocks for Gated Clock Conversion for information about its use and an example.

3. Enter the `run map` command: `run map`

   When you run this command, the tool generates a new database node, called *m0* by default. The tool synthesizes the design and maps the logic to technology-specific resources. In addition to the log file, the tool also generates an *srm* file, which you can view in a schematic window to analyze the results of the run. The schematic shows design logic as well as support functions like UMRBus and initialization blocks for the hardware system.

4. Analyze the results.

   See Analyzing Results for more information about checking the results of the run.



In particular, check these results: clock conversion, timing performance, cell and I/O usage, and hierarchical area usage.

5. Rerun mapping as needed to refine gated clock conversion and optimize the packing of logic into resources.

   When you are satisfied with system performance and resource utilization, you can perform one of these operations:

Place and route exploration	Running Place and Route
Place and route	Running Place and Route
Simulation	Running Simulation
Equivalence checking	Verifying the Design

Setting Auto-SPO

The Auto-SPO (Smart Parameter Optimization) feature automatically launches Parallel Automatic Recompile for Failing FPGA (PARFF) jobs with synthesis and place-and-route options for a successful compilation. The Vivado compilation in the original run is continuously monitored for certain design metrics and the PARFF job is launched based on heuristics or thresholds using the tool recipe file.

Note:

The Auto-SPO feature might not run with the fpga tcl scripts generated with the Protocompiler versions before V-2024.09.

Feedback

You can set the Auto-SPO feature by enabling the environment variables, as described in the following procedure:

1. Set the environment variable `PC_ENABLE_AUTO_SPO` to 1 in the Protocompiler compile script.

   ```
   PC_ENABLE_AUTO_SPO = 1
   launch protocompiler <script>
   ```

   The strategies mentioned in the recipe are run by default, for both the synthesis and place-and-route PARFF jobs. Synthesis results from the original run are reused in the PARFF run for the same synthesis strategy run.

2. Set the environment variable `PC_ENABLE_AUTO_SPO` to 2 for launching the Vivado only PARFF runs with the original run.

   ```
   PC_ENABLE_AUTO_SPO = 2
   ```

   The default recipe file is used for the PARFF jobs. You can see the default recipe file for the syntax in the build at the following location: $LIB/protocompiler/parff.recipe

   ```
   L0_FastSynth_EmulationPAR:
   {
   SynthesisOpts:parff_synoptn.tcl
   SynthesisStrategy:fast
   PARMode:Emulation
   }
   L1_RoutabilitySynth_EmulationPAR:
   {
   SynthesisStrategy:routability
   PARMode:Emulation
   PARenv:L1_vivado_env.tcl
   }
   L2_RoutabilitySynth_DefaultPAR:
   {
   SynthesisStrategy:routability
   PARMode:default
   sll_optimize:0
   }
   L3_RoutabilitySynth_MPFPAR:
   {
   SynthesisStrategy:routability
   MPFMode:Single
   }
   L4_AdvancedSynth_EmulationPAR:
   {
   SynthesisStrategy:advanced
   PARMode:Emulation
   }
   L5_AdvancedSynth_DefaultPAR:
   {
   SynthesisStrategy:advanced
   ```

Feedback

```
PARMode:default
}
L6_AdvancedSynth_TimingQoRPAR:
{
SynthesisStrategy:advanced
PARMode:timing_qor
PARenv:L6_vivado_env.tcl
opt_design_flags:
place_design_flags: -directive Explore
route_design_flags: -directive Explore
}
L7_FastSynth_Reentrantflow:
{
SynthesisStrategy:fast
PARenv:L7_vivado_env.tcl
};
```

3. Optionally, set a custom recipe file with customized options and environment. Make sure that you provide the absolute file path for the custom recipe file.

   ```
   env(PC_SPO_RECIPE_FILE) < absolute path to recipe file >
   ```

   After the PARFF jobs are launched using the tool recipe file and the following error message appears in the Protocompiler FPGA srs log file, protocompiler_run__srs.log:

   Detected COMPLEX FPGA, Launching PARFF for <FPGA>

   The compile monitoring and PARFF trigger information status is available in the */ vivado_srs/fpga_info.log* file:

   ```
   <timestamp> |- Compiling Original
   <timestamp> |- <reason for trigger> TRIGGER PARFF
   <timestamp> |- PARFF TRIGGERD
   ```

   Note:

   You need not monitor the recipe runs; the successful recipe result is copied to the FPGA directory. The first successful recipe file is saved in the FPGA directory as “*success.recipe*”.

4. Set the environment variable `PC_SPO_CONTINUE_RUNS` to 1 to maintain the original or SPO jobs running, even though the bit file is generated for an original or a SPO recipe.

   ```
   PC_SPO_CONTINUE_RUNS = 1
   ```

Custom Modified script

The Auto-SPO flow runs with only the Protocompiler tool generated fpga tcl script (fpga_srs.tcl). To update this script and to add sourcing of custom scripts, make sure that you:

- Prepend the `${appnd_path}` variable before the file path.

```
if { [file exists ${appnd_path}custom_script.tcl] == 1 }
   {source ${appnd_path}custom_script.tcl
```

- Provide absolute file paths for any script which is sourced from within such custom scripts.

Synthesizing Based on Design Intent

The prototyping tool includes an intuitive GUI where you can execute a complete synthesis run (compile, pre-map, map) with options that are optimized for the design goal you specify.

See the following procedures for details:

- Using Design Intent to Synthesize a Quick Prototype
- Synthesizing for Optimal QoR Using Design Intent

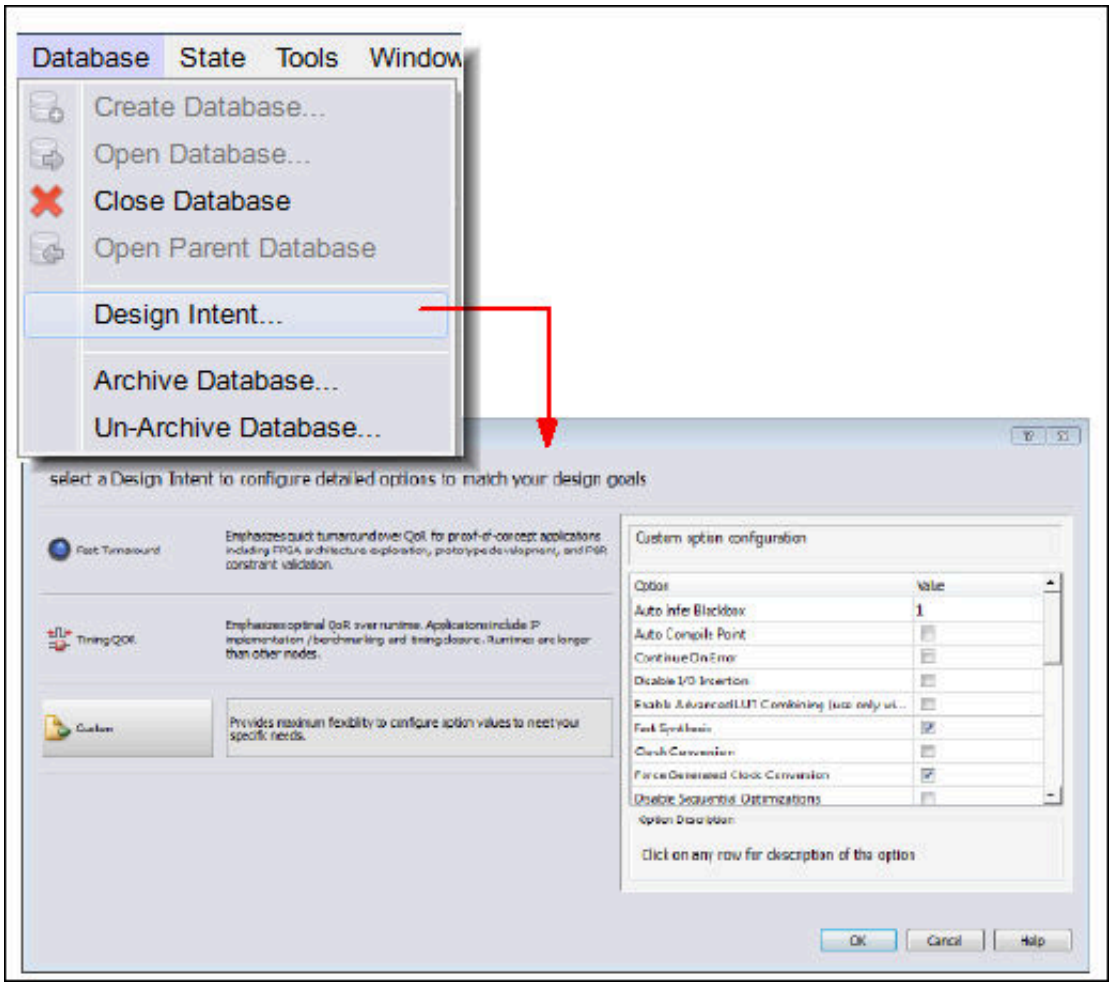
Using Design Intent to Synthesize a Quick Prototype

The tool offers a shortcut for a quick, first-pass synthesis run. You can do this to generate an initial prototype, or to estimate how to allocate resources and divide up the design. The following procedure shows you how to signify design intent from either the command line and the GUI.

1. To specify design intent without using the GUI, load a database and then do the following:
   - Specify the **`design_intent fast_turnaround.tcl`** command. This command opens the named file located in */lib/design_intent/protocompiler*; the file contains the default set of options for quick synthesis. You can synthesize using these default options.
   - To use a custom set of options, specify the **`design_intent -custom`** command. This command opens the *fast_turnaround.tcl* file, and you can use this as a template and customize the options. Save the options file, and then synthesize using the customized set of options.
2. Load a database and select **Design Intent** from the **Database** popup menu.

Feedback

The **Design Intent** window opens.



3. To use the default settings for a quick prototype, click **Fast Turnaround** and then click **OK**.

   The tool automatically sets the best options to minimize runtime and ensure a quick turnaround and synthesizes the design with these options. The table on the right of the dialog box shows the settings used.

   This mode uses the TTFP (Time To First Prototype) compiler, which is a thin compiler that is faster than the default compiler. The compiler generates a schematic where you can check the results.

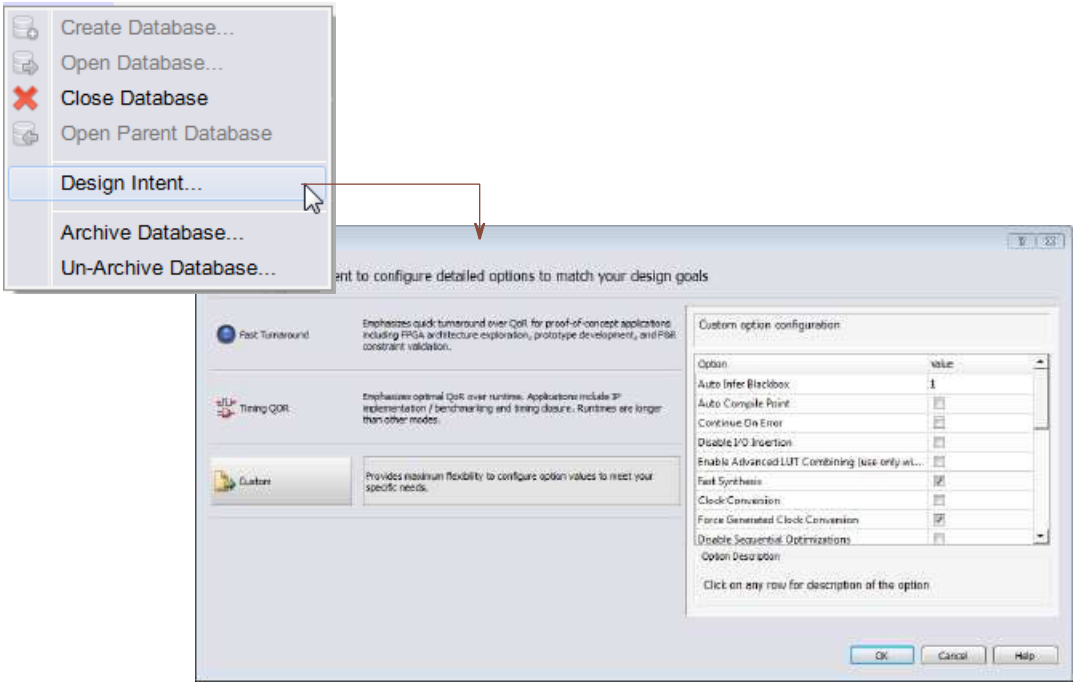
4. To customize the options, click **Custom**, select the option settings you want, and then click **OK**.

Feedback

Synthesizing for Optimal QoR Using Design Intent

The synthesis tool offers a shortcut to setting options to achieve the best QoR for the prototype. You can set design intent from the GUI or from the command line, as described in the following procedure:

1. To specify design intent without using the GUI, load a database and do the following:
   - Specify the `design_intent timing_qor.tcl` command. This command opens the named file located in */lib/design_intent/protocompiler*; the file contains the default set of options for best results. You can synthesize using these default options.
   - To use a custom set of options, specify the `design_intent -custom timing_qor.tcl` command. This command opens the default *timing_qor.tcl* file, which you can use as a template to customize the options. Save the options file, and then synthesize using the customized set of options.
2. To use the GUI, load a database and select **Design Intent** from the **Database** popup menu.



3. To use the default settings, click **Timing QoR** and click **OK**.

When you synthesize, and place and route the design, the tool automatically sets the best options to for an at-speed prototype and synthesizes the design with these options. The table on the right of the dialog box shows the settings used.

4. To customize the options, click **Custom**, select the option settings you want, and then click **OK**.

Using Distributed Processing

Distributed or parallel processing is a technique used to reduce runtime. With this technique you divide and conquer, splitting up processes so that they can run in parallel instead of serially, thus saving time.

The number of processes run in parallel is determined by the number of available licenses and available jobs, as well as the-*max_parallel_jobs* option.

See the following topics for more information:

- Distributed Processing Schemes
- Distributed Processing Operations
- Setting Options to Run Distributed Processing
- Using CDPL for Distributed Processing
- Running Distributed Compile
- Running Distributed Synthesis
-

Distributed Processing Schemes

There are two schemes for multiprocessing:

With the CDPL (Common Distributed Processing Library) scheme: CDPL distributes processes across multiple machines
Without CDPL: Processes are distributed among the processors on a single machine

Distributed Processing Operations

The following table lists processes that can be processed in parallel in distributed mode.

Feedback

Process	Usage
Distributed compile	Limitations: multiple configurations, hierarchical defparam, signed parameters
Distributed synthesis	Limitations: constraint checker, continue on error, and compile points are ignored.
Compile point processing	Compile points are ignored in distributed synthesis mode, but can otherwise be processed in parallel. SeeWorking with Compile Points.
Quick area estimation	Enable with the `run pre_partition -area_est 1` command.
Multiple tool invocations to process partitioned FPGAs	Use the `launch protocompiler -max_parallel_jobs` command.

Setting Options to Run Distributed Processing

There are various factors that affect a distributed processing run, like the number of tool licenses, the maximum number of jobs to multiprocess (four jobs in parallel per license, the distribution scheme, and which processes that can be run in parallel.

You can specify distributed processing when you start a run with the launch command, or set it as an option. The following procedure describes how to do this.

1. Set the maximum number of jobs to run.

   There are two ways to set the number of jobs, depending on the operation you want to multiprocess:

   - (*HAPS-100*) Use the `option set` command to set another value for distributed processing in the compile and map phases a single FPGA flow. This value is used instead of the default to process compile points, compiler runs, or synthesis operations in a single FPGA flow:

     ```
     option set max_parallel_jobs num_of_jobs
     ```

     See Running Distributed Synthesis for details. For details on the number of parallel jobs you can run with each license, see the HAPS ProtoCompiler Licensing Model section.

   - Specify the *-max_parallel_jobs* argument to the `launch protocompiler` command to run multiple invocations of the tool and process the partitioned FPGAs in parallel.

2. If you want to use CDPL to distribute processing jobs over multiple Linux machines, set up the CDPL machines and enable the CDPL option (*option set cdp1 1*).

Feedback

See Using CDPL for Distributed Processing for details. If you do not use CDPL, the tool distributes processing among multiple processors on the same machine.

3. To run distributed synthesis or distributed compile, set additional options as described in Running Distributed Compile and Running Distributed Synthesis.
4. To distribute area estimation runs, specify the *-area_est 1* argument with the `run pre_partition` command.
5. To launch multiple tool invocations to process partitioned FPGAs in parallel, use the `launch protocompiler -max_parallel_jobs` command.

   See Synthesizing Individual FPGAs Using a Script for details.
6. To multiprocess compile points, enable the compile point option and disable distributed synthesis.

   If distributed synthesis is enabled, compile points are not run in parallel during the map phase.

Using CDPL for Distributed Processing

You can specify that the tool use the Synopsys CDPL (Common Distributed Processing Library) scheme for distributed processing or parallel processing. CDPL is a mechanism for deploying multiple processors to run processes like compiling and mapping in parallel on different machines. It also allows you to launch third-party applications, like place-and-route for example, if you have the requisite license set up.

CDPL consists of a leader-worker distributed processing framework that distributes jobs across multiple machines. It only runs on Linux platforms. For Windows, or Linux machines without CDPL, the tool supports parallel processing on a single machine using multiprocessing.

For comprehensive information about CDPL, see the *Common DP Library User Guide*, a PDF version of which is included in the *doc* directory of the tool installation. The following procedure summarizes what you need to do to use CDPL in the prototyping tool. If you do not use CDPL, by default the tool distributes the processing jobs among the available processors in the current machine.

1. Specify the Linux machines to use for CDPL.

   CDPL only runs on Linux platforms.

   Create a host file that lists the machines for distributed processing, using the following syntax to specify them:

   ```
   flag| hostname | slots | tmpDir | protocol | command
   ```

Feedback

Four parallel processing jobs are allowed per license. If you have multiple licenses, you can specify more distributed processing jobs. See Host File Examples for Distributed Processing for an explanation of the syntax and examples.

2. Set the *CDPL_FPGA_HOST* environment variable to point to the CDPL executable and libraries:

csh	*setenv CDPL_FPGA_HOSThostFilePath*
bash, ksh	*export CDPL_FPGA_HOST=hostFilePath*

Make sure to specify an absolute path. Using a relative path causes a failure.

3. In the prototyping tool, do the following:
   - Set the number of jobs to run in parallel with the *-max_parallel_jobs* option. You can set this as an option with the **`option set`** command to distribute compile and map processes, or specify it as an argument to a the **`launch protocompiler`** command to process FPGA partitions in parallel.

     The actual number of jobs that run in parallel depends on the number of available licenses and available jobs as well as the *-max_parallel_jobs* value.
   - Enable CDPL with the *cdpl* option:

     ```
     option set cdpl 1
     ```

You can now leverage CDPL for the parallel processing of various operations. See Running Distributed Compile and Running Distributed Synthesis for information about running various distributed synthesis processes.

When CDPL is specified, the following command distributes the running of multiple scripts across the specified CDPL machines to implement the partitioned FPGAs:

```
launch protocompiler -max_parallel_jobs 2 -script path/slp1.tcl
-script path/slp2.tcl -max_parallel_jobs 10-script path/slp3.tcl
```

Host File Examples for Distributed Processing

Host files specify machines for distributed processing using this syntax:

```
flag| hostname | slots | tmpDir | protocol | command
```

flag	Specifies whether a machine can be used. Valid values: • 0 signfiies the host cannot be used • 1 signifies a usable worker machine

Feedback

hostname	Name of a valid worker machine. This field is empty for the *SGE* and *LSF* protocols.
slots	The maximum number of worker slots available on the host for *RSH* and *SSH* protocols, and in the farm for *LSF* and *SGE*. -1 indicates unlimited worker slots for LSF and SGE protocols.
tmpDir	Any directory, to be used for local data storage as required.
protocol	Protocol used to connect to the worker machine. Valid values are *RSH, SSH, SGE*, and *LSF*, for Remote Shell, Secure Shell, Sun Grid Engine and Load Sharing Facility, respectively.
command	Corresponding connection command for the protocol, which is used to connect to the worker machine. If *protocol* is defined as *RSH*, *rsh* is the corresponding connection command. For *LSF* and *SGE*, you can also specify command parameters.

Examples

RSH Host File

The following host file specifies a five-worker, two-node configuration for a distributed processing run. The tool launches up to two workers on the *sbgemtin01-25* machine, and up to three workers on the *sbgemtin01-22* machine.

```
1|sbgemtin01-25|2|/myDir/cdpl/tmp|RSH|rsh

1|sbgemtin01-22|3|/myDir/cdpl/tmp|RSH|rsh
```

SGE Host File

You must properly configure the end-user environment, and it must be capable of starting SGE jobs. This is an example of an SGE host file entry:

```
1| |32|/tmp|SGE|qsub -P bnormal -l os_bit=64,mem_free=64G,h_vmem=64G
```

LSF Host File

You must properly configure the end-user environment, and it must be capable of starting LSF jobs. This is an example of an LSF host file entry:

```
1| |32|/tmp|LSF|bsub -q bnormal -R os_bit==64 -R "rusage[mem=32000]"
```

Running Distributed Compile

You can run distributed compile on single-FPGA or multi-FPGA designs. In multi-FPGA designs, use distributed compile for both the top-level RTL compile process as well as the FPGA-level compile.

Feedback

Distributed compilation can be run as an initial run or incrementally. It is recommended that you run distributed compile on all designs to improve runtime.

1. If you are using CDPL for distributed processing, make sure the setup is in place and you have enabled the CDPL option.

   See Using CDPL for Distributed Processing for details. If you do not specify CDPL, the tool uses available processors on the current machine to distribute the jobs. You can try running distributed synthesis without CDPL first and then try it using CDPL.

2. Specify the number of compiler jobs to distribute and run in parallel.

   ```
   option set max_parallel_jobs num_of_jobs
   ```

3. Set the distributed compile option:

   - Use this command to specify distributed compilation:

     ```
     option set distributed_compile 1
     ```

     This option only runs distributed compile; for information on distributed mapping, see Running Distributed Synthesis.

   - Specify the distributed compile option last, after the options described in the previous steps.

4. Compile the design as usual with the **`run compile`** command.

   The tool splits the design into smaller parts (distribution points or nodes) and compiles the nodes in parallel. It creates separate post-compile database files for each group. The log file reports the separate processes that are being distributed with messages like this:

   ```
   Compiling work_cm3_dts_verilog as a separate process
   Compiling work_cm3_sched_verilog as a separate process
   Compiling work_cm3_srb_verilog as a separate process
   ...
   ```

   The top-level log file contains a *Distributed Compiler Report*, which shows the status of each group. This is an example of an entry in that report:

   ```
   ************************************************************************
   **********
   DP Name  Status Start time End Time Total Real Time Log File
   ------------------------------------------------------------------------
   ---------
   work_cm3_dts.verilog Success 0h:00m:01s 0h:00m:03s 0h:00m:02s
    synwork//
        distcomp
        distcomp1.log
   ```

5. Check for errors and black boxes.

Feedback

- If you get an error for a node during distributed compilation, you can bypass the node by setting the CDC *syn_distcomp_node* directive on the failing module:

  ```
  define_directive {v:libName.moduleName} {syn_distcomp_node} {0}
  ```

  The directive forces the compiler to create new groups for the next run, and to avoid using the failing node as a distribution node.

  Re-compile the design after setting the directive.

- If black boxes are created after distributed compile, it means that the tool encountered linking issues. Typically, this happens with certain constructs. You can either run in non-distributed mode or bypass the failing node.

6. Check the names after running distributed compile.

   Distributed compilation might create different names than non-distributed compile, so if you need to set constraints, make sure that the names match the names generated after distributed compile.

7. To run distributed compile incrementally, do the following:

   - Make any changes required.
   - Rerun distributed compile. The tool only recompiles and generates new post-compile databases for groups where changes were made and their parents. This results in runtime savings on iterative passes, depending on the number of nodes affected. If the change is to a top-level parameter or a common file, like package, the recompile affects many more nodes, and there might not be much of a runtime advantage.

Limitations

There are some limitations to keep in mind when running distributing compilation.

- UC and Distributed Compile:

  The UC flow only supports distributed compiler mode and runs in this mode regardless of the distributed compile setting. If the design cannot be divided into parallel processes for the distributed compiler, it runs in monolithic mode and reports the following message:

  ```
  @N: This design does not have sufficient nodes/jobs for distribution,
  running in monolithic mode.
  ```

- XMRs:

  Do not define XMRs in a *cdc* file; you must specify them in the RTL. If you do not specify them in the RTL, the tool issues a warning, and runs in non-distributed mode.

Feedback

- Constructs:

  Some constructs cause linking issues:

  - Some complex parameters, such as unions
  - Sign extensions with signed parameter override

The tool creates black boxes when it encounters linking issues. In such cases, either run compilation in non-distributed mode, or bypass failing nodes by setting *syn_distcomp_node* to 0.

Running Distributed Synthesis

To speed up runtime, run synthesis operations in a distributed fashion. When you follow the procedure given below, the tool runs mapping processes in parallel. You can run distributed synthesis on single FPGAs, so that *run pre_map* and *run map* operations run in parallel.

1. If you are using CDPL for distributed processing, make sure the setup is in place and you have enabled the CDPL option.

   See Using CDPL for Distributed Processing for details. If you do not specify CDPL, the tool uses available processors on the current machine to distribute the jobs. You can try running distributed synthesis without CDPL first and then try it using CDPL.

2. Check that the following option values have been set:

   - Use `option set max_parallel_jobs num_of_jobs` to specify the number of jobs to process in parallel. Do this for both distributed compile and distributed synthesis.
   - For distributed synthesis, make sure the following options are also set:

Option Setting	Description
option set max_parallel_jobsnumOfJobs	Number of jobs to process in parallel. Set the number higher than the default value.For details on the number of parallel jobs you can run with each license, see the *HAPS ProtoCompiler Licensing Model* section.
option set automatic_compile_point 0	Controls the automatic creation of RTL partitions (compile points).

   Make sure to disable the automatic compile point options; otherwise distributed synthesis does not run.

3. Set the distributed synthesis options and synthesize.

- Set this option in a single-FPGA design to specify distributed synthesis:

  ```
  option set distributed_synthesis 1
  ```

  This option runs the mapping operations from the pre-map and map stages in parallel.

- Make sure that the distributed processing option is specified last, after all other option settings described in the previous steps.

- Run the mapping phases (*run pre_map, run map*) as usual on the single FPGA. The tool runs synthesis processes like constraint application, clock conversion, and mapping in distributed mode. It reports the separate processes being run in the log file.

  See Clock Conversion and Distributed Synthesis for information about how clock conversion is handled.

  Distributed processing affects some synthesis features. See Distributed Synthesis Limitations for details.

- To distribute the synthesis of partitioned FPGAs in a multi-FPGA design, launch and run scripts in parallel, as described in Running Synthesis for Individual FPGAs.

4. To run distributed synthesis incrementally, do the following:

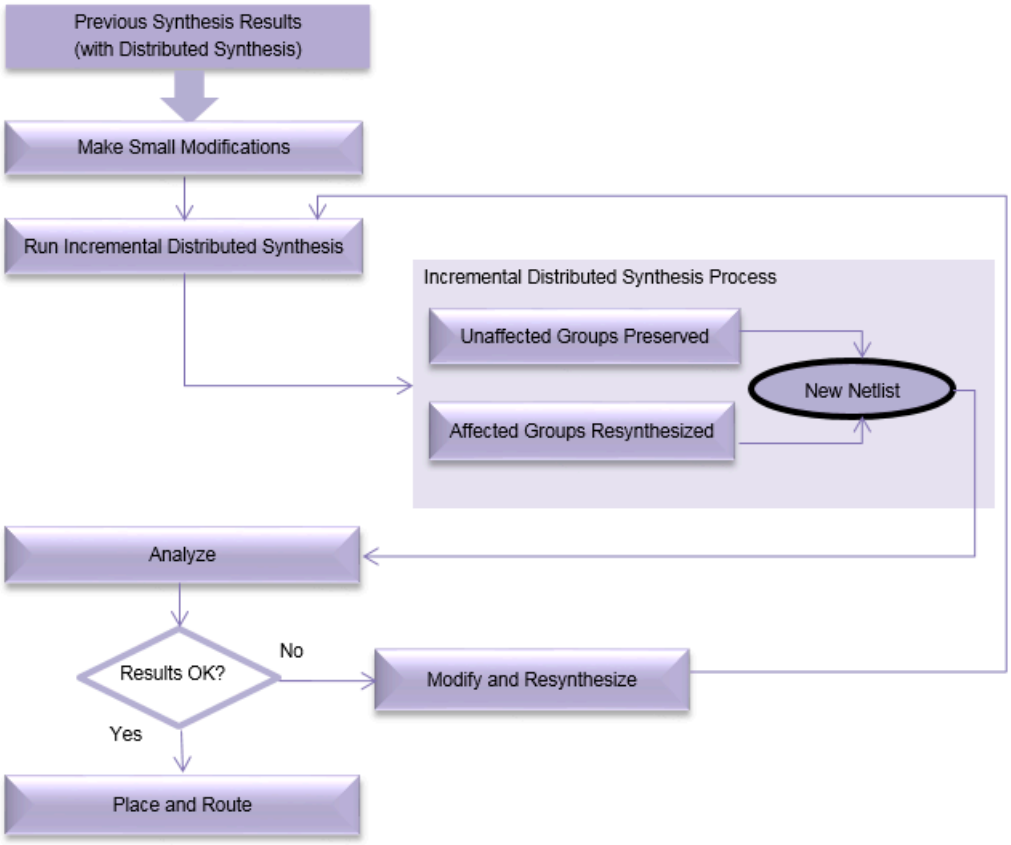
   - After the initial distributed synthesis run, make the changes you need to fix errors or improve your design.

   - Set the incremental distributed synthesis option.

     ```
     option set -incremental_synthesis 1
     ```

   - Rerun the map stages to resynthesize the design using distributed processing (*run pre_map, run map*).

     When the design is resynthesized, only those synthesis groups whose source code logic has been modified are resynthesized, using distributed processing. The mapped files generated from the previous run are used for the remaining, unchanged groups. If you modify existing constraints or options, the entire design is resynthesized.



- Check the log file for changes.

  The following figure shows the results of an incremental distributed synthesis run. After the first run, logic changes were made to the design. This figure shows that

Feedback

incremental synthesis only resynthesized the top-level *eight_bit_uc* and *prgmcntr* modules.

```
@N:MP797 :  | Total number of groups created: 9
@N:MP796 :  | Maximum number of parallel jobs set to 2

=================================================Summary for Mapping=======================
Hierarchical       Mapping Status                          Total Time  Total      Peak
Instance Name                                                          CPU        Memory
                                                                       Time       used
-----------------------------------------------------------------------------------------
uc_alu             Unchanged|Reused previously mapped design  0h:00m:03s  0h:00m:00s  165 MB
dmux               Unchanged|Reused previously mapped design  0h:00m:02s  0h:00m:00s  165 MB
decode             Unchanged|Reused previously mapped design  0h:00m:02s  0h:00m:00s  164 MB
io_buff            Unchanged|Reused previously mapped design  0h:00m:02s  0h:00m:00s  164 MB
regs               Unchanged|Reused previously mapped design  0h:00m:02s  0h:00m:00s  164 MB
work.eight.verilog Remapped|Design changed                    0h:00m:02s  0h:00m:00s  165 MB
prgmcntr           Remapped|Design changed                    0h:00m:02s  0h:00m:01s  165 MB
rom                Unchanged|Reused previously mapped design  0h:00m:02s  0h:00m:00s  164 MB
special_regs       Unchanged|Reused previously mapped design  0h:00m:02s  0h:00m:00s  165 MB
=========================================================================================
```

Clock Conversion and Distributed Synthesis

Clock conversions are handled the same during distributed synthesis and in a traditional (non-distributed) synthesis flow as follows:

- Clock conversion runs at the pre_map stage, on compiler primitives.
- The names of referenced objects and the numeric values in the Clock Optimization report are defined with the conditions below:
  - *Driving Element/Drive Element Type* references compiler objects (e.g. MUX).
  - Clock trees with *n* number of instances and with the same inputs are reported *n* times.
- ICG latch removal is performed in conjunction with clock conversion.
  - The ICG Latch Removal Summary is listed just before the Clock Optimization Report at the pre-map stage.
- ICGs that drive latches are supported and are converted to registers.
- When the *syn_hier=hard* attribute is applied to hierarchies that “cut” a gating cone of logic, the tool does not prevent clock conversion.

Distributed Synthesis Limitations

Currently, distributed synthesis has some limitations:

- Does not support continue on error
- Does not support FSM Explorer
- The stability latch removal summary does not contain debug information for clock conversions.

Analyzing Results

The following sections describe procedures to analyze the results of the `run` command:

- Checking Reports and Log Files
- Viewing Results in a Schematic
- Exploring a Schematic with find and expand
- Querying Incremental Results
- Querying Jobs
- Querying Metrics for a Design
- Dealing with Black Boxes in the Synthesized Netlist

Checking Reports and Log Files

Each database generates its own reports, appropriate to the command used to generate that database state. The database is closed, so there are different ways to access the reports and log files. They are summarized in the table below; details are in the procedure that follows.

Reports from the GUI (Within the database)
This is the recommended way to access the reports, as it is tightly integrated and includes features for navigation, locating errors, finding information about errors, and cross-probing. There are two ways to display the reports: Report View (step 1)`view report` and `report list` commands in the Tcl shell (step 2)
Exported Reports Outside the Database
The `export report` command exports reports outside the database. This method is best suited for scripts (step 3).
Reports Outside the Database (HTML)

Feedback

It is recommended that you use the HTML version of the files if you need to view reports offline. It includes some of the support features provided by the GUI report (step 4).
Reports Outside the Database (Text)
This version contains the same content, but does not have the navigation and other features available in the HTML or GUI versions of the report (step 5).
Obfuscated Files that Cannot be Exported
Use the `edit` command from the GUI to view internal files that are referenced in messages but which cannot be exported (step 6).

1. Follow the steps below to display the reports in the GUI Report View.
   - Go to the appropriate state in the database hierarchy.
   - To view all the reports up until that stage, click the **Report View** icon in the left panel in the tool GUI () or type *view report*. This opens the Report view and displays the reports available for the active stage, as well as the reports for all its ancestors in the database tree. It does not display reports for any parallel branches.
   - To display an individual report, use the list on the left of the **Report view** to navigate to the report you want.
2. To display reports in the GUI using the command line, do the following:
   - To see a list of all available reports for a database state, use `report list`. This command lists the names of the reports in the Tcl console.
   - To open all the reports available from the current database back to the root, specify the `view report` command with no arguments.
   - To open a specific report, specify the `view report` command and the report name. The tool opens the specified report in the Report View.

   The database is closed, so you must use these commands to access the reports and log files. See Shell Command Reference for details about the commands.
3. To export a log file or report outside the database, use the `export report` command.
   - Use `export report -list` to view the list of files available for export.
   - Use `export report -path` with the name of the file to export it to the specified location outside the database. Use the *-all* option to export all the available reports and log files.

   This command is particularly useful in scripts.
4. To view reports offline, use the HTML version of the report.

It is recommended that you use this method of offline log viewing if you want to view results without opening the database. The GUI version of the reports described in the previous steps is part of the closed database.

- From outside the tool, open the HTML report file, which is a single-page summary of all the log files and reports with links to each one. It is located outside the database hierarchy, at the same level as the database, in the same directory. You can access all reports from the current database state back to the root. The reports displayed are the same reports as are available from the GUI, but they can be accessed from outside the database.
- The HTML report is not saved, but is overwritten by results from a subsequent run. To keep a report, save it to another location or another name.

5. To view a text version of a report offline, follow these steps:
   - By default, the tool automatically generates text versions of the report files and puts them in a parallel directory to the database directory. If you do not see this directory, check that the *auto_export_reports* option is set to 1, using the **`option set`** command to reset it if necessary.
   - Go to the directory and open the reports you want.
   - These reports are overwritten by subsequent runs, so save any reports you want to keep to another location.
6. To display obfuscated files from the closed database that are referenced in messages but cannot be exported, follow the procedure below:

   Although the *edit* command is also available from the command line, you can only open and edit obfuscated files from the GUI using the method described here.

   - Locate the plain text file name referenced in the log file message. For example:

     ```
     Error in subprocess: See log file /myDesign/results/EMAC/db0/
     pre_partition/pp2/synlog/addrcheck_netlist_optimizer.srr
     ```

   - Open the database in the tool GUI. The database is required to be open for the **`edit`** command to open the file.
   - Use the *edit* command in the Tcl window to open the file. You can then save the file to another location if you want.

     ```
     edit /myDesign/results/EMAC/db0/pre_partition/pp2/synlog/
     addrcheck_netlist_optimizer.srr
     ```

   The `runtime_capture.tcl` script helps you collect compile time data for your design. It generates a CSV file with the compile time data of each and every stage in the HAPS Protocompiler flow. This script is useful because it allows you to track and analyze how

Feedback

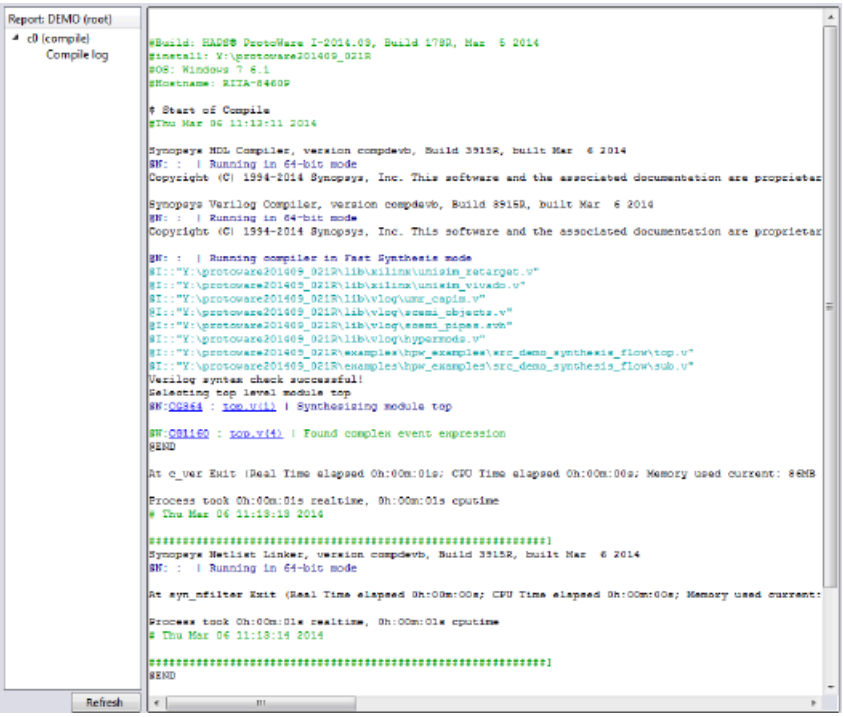
long different stages of your design compilation take. By understanding compile times, you can identify bottlenecks and improve your design flow.

The `runtime_capture.tcl` script is included with your installation. You can find it at:

`<protocompiler_installation_path>/examples/haps_utilities/runtime_capture.tcl`

7. After you have opened the report you want, check the file.

   The report typically lists the details and results of the current operation. The GUI version of the report includes color-coded errors, warnings and notes. See Resolving Errors and Warnings for information about working with these messages.



8. Navigate to view specific pieces of information.

   - In the GUI version, use the panel on the left to navigate to the section you want. In the HTML version, use the links.
   - To search the body of the report, use *Control-f* to open a dialog box for search criteria.

   The areas of the log file that are most important are the warning messages and the timing report, which lists the most critical paths. The following table lists common starting points for searches in the log file.

To find ...	Search for ...
Notes	@N or look for blue text
Warnings and errors	@W and @E or look for colored text
Performance information	Performance Summary
The beginning of the timing report	START TIMING REPORT. Also see Checking Timing Results.
Detailed information about slack times, constraints, arrival times, and so on.	Interface Information
Resource usage	Resource Usage Report
Gated clock conversions	Gated clock report

Viewing Results in a Schematic

Schematics present graphic views of your design. The tool includes the HDL Analyst tool, where you can graphically view the design at different stages, as it progresses through the sequence of database states.

See the following table for details about each one in the *HAPS® Prototyping Compiler and Mapper Guide*:

HDL Analyst (New)
Analyzing With the HDL Analyst Tool
Working in the Schematic
Exploring Design Hierarchy
Finding Objects
Crossprobing

Working in the Schematic

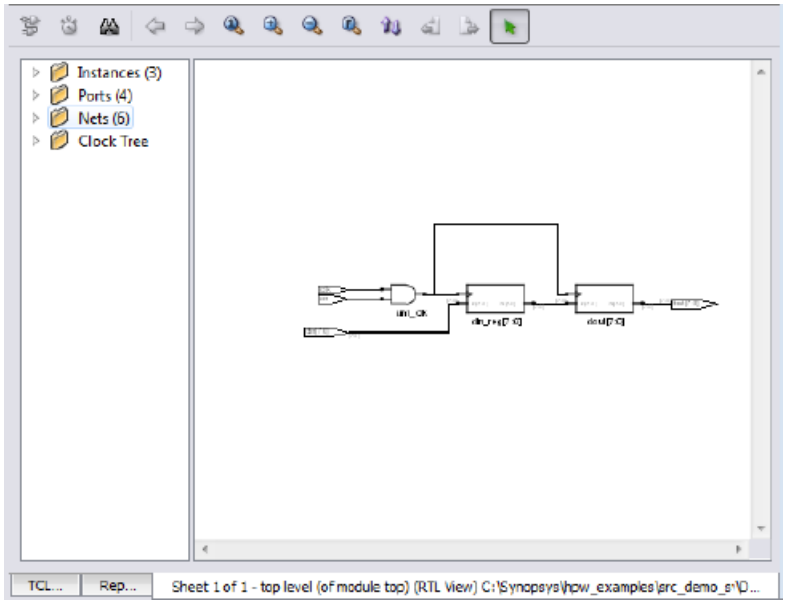
The following steps summarize how to analyze results in a schematic. For details about operations mentioned, refer to the appropriate section from the table.

Feedback

1. Type *view schematic* in the tool Tcl console, or click the schematic icon in the left panel of the tool GUI ( ) for GUI mode.

   You can use the **`view schematic`** command in either GUI or shell mode, but GUI mode offers more functionality. In shell mode, the command does not open a schematic of the netlist, but enables the use of query and analysis commands like **`find`** and **`expand`**.

   If you run the tool in GUI mode, the tool opens an HDL Analyst window with a graphic view of your design that reflects the state of the design at the given database state: for example, compiled, partitioned, mapped, routed, post-place-and-route, and so on. Use the **`database set_state`** command to change to the database state you want. You can use the object query commands and other HDL Analyst commands in this window to visually analyze the design.



   There is a difference in memory usage, depending on the mode you use. In GUI mode, the memory associated with the design is released back to the system when the HDL Analyst window is closed; in shell mode, the memory is released when the database is closed.

2. In GUI mode, use these techniques to navigate design hierarchy in the HDL Analyst view:
   - Use the browser on the left side, which lists objects by type.
   - Push down into a hierarchical object by placing the pointer over it and clicking. To go up a level, click in a blank area of the design.
   - If it is a multi-sheet schematic, move from one sheet to the next by clicking the appropriate icons.
3. In either GUI mode or shell mode, analyze the data in the schematic, using the techniques as described below:

Operation	Action
Find objects	Use the Tcl *find* command to locate objects. See the next step for details.
View connected objects	Select an object as a starting point and then filter the design.
Expand logic	Select a pin on the object or a net; then right-click and select the *Expand* command that is appropriate. In shell mode, use the *expand* command.
Filter the objects displayed	Use any of these methods: Select the objects you want to isolate and click F12 or the *Filter Schematic* icon. Click the *Back* icon to return to the previous view. Identify and select objects by type, using the browser on the left side of the view. Use the Tcl *find* command to locate objects.
Flatten the hierarchy	Right-click and select *Flatten Schematic*.
Display object properties in the schematic	To temporarily display properties, hover over an object. A tooltip shows the information. Select an object, right-click, and select *Properties*. The properties and their values are displayed in a table.

For more information about the commands mentioned, see Tcl Find and Expand. For usage information, see Finding Objects.

The commands apply to the current schematic, and remain relevant for that database state until you either close that database or issue another `view schematic` command for a different database state. If you are using GUI mode, the commands additionally require the HDL Analyst window to be open to take effect.

Exploring a Schematic with find and expand

You can use object query commands like *find* and *expand*, and collection commands like *c_diff* to explore and analyze a design even if you are not using GUI mode.

1. This command enables you to use the query and analysis commands in the next step. For more information, see Viewing Results in a Schematic.

   From the database state you want to analyze, type *view schematic* in the shell window.

2. Use the *find, expand*, or other commands to explore the database.

   For details about the command, its syntax and examples, see Tcl Find and Expand. Depending on the HDL Analyst version you use, you can find additional usage information in Finding Objects.

Querying Incremental Results

Prototyping is iterative, and involves ongoing changes, because of design evolution and different versions, tweaks to optimize performance or address RTL changes, or debug instrumentation changes. Use the **`database query_state`** commands to get useful information about incremental results for a particular database state. For the complete syntax for this command, refer to database. The following steps highlight some ways to use this command to extract useful information after a run:

1. Go to the database state that you want to query.

   The command works on the current database state.

2. To use this command to get information about source files, do the following:

   Specify the **`database query_state`** command with the appropriate argument:

   `database query_state argument`

To view ...	database query_state Argument
A list of HDL *'defines*	*-hdl_define*
A list of HDL parameters	*-hdl_param*
The top module name	*-top_module*

3. To query options for a state, use the **`database query_state -option`** command.

- To find the value set for a particular option, use this command:

  ```
  database query_state -option optionName
  ```

- To print the settings for multiple options for the database state in readable text, include the **database query_state** command in a Tcl command like this:

  ```
  foreach o [option list] {puts "The value of $o when this state was
     generated: '[database query_state -option $o' "] }
  ```

- Alternatively, use the **database query_state -option** command in a simple script, as shown in this example:

  ```
  proc get_present_option_settings {} {
  set state_type [database  query_state -run_type]
  set state_name [database  get_state ]
  puts ""
  puts "Option settings for state \"$state_name\" which is a state
   of type \"$state_type\""
  puts ""
  puts "Option                                Current Value"
  puts "------                                -------------"
  foreach o [option list] {puts "[format "%-33s" $o]  '[database
   query_state -option $o]'" }
  }
  ```

  One good practice is to generate the list of option settings when a database state is first generated, before you begin experimenting with different values.

- To restore option settings to the values they had when you generated the database state, with a command like this one:

  ```
  foreach o [option list] { option set $o [database query_state
  -option $o ] }
  ```

4. To check whether a database state needs to be rerun, use the **database query_state -out_of_date** command.

   Use this command to check if the current state or states in the currently active tree are out of date and need to be rerun. This could be because input files or global options have changed, because an earlier state was rerun more recently than the current state, or because an earlier state was rerun with a different version of the software. For example:

   ```
   % database query -out_of_date
   pm0 is out of date:
    master.fdc has changed
   m0 is out of date:
    Its parent pm0 is out of date
   ```

Use the *-verbose* option to see more details. You can also generate a report with this information by using the **report out_of_date -mode** command.

5. To check database state inputs, use the following commands:

To...	Commands
Compare inputs for current state to another state	*database query_state -state_diff report state_diff*
List all inputs for current state	*database query_state -input_files*

With the *-state_diff* option, the command returns a 1 if the inputs are the same, and 0 if they are different. The equivalent **report** command generates a report with the comparison information for the specified states or trees.

The following example shows the results of comparing two mapper runs where changes were made for the second run (*m1*):

```
m0 differs with respect to m1:
 arguments have changed
 my.fdc has changed
0
```

6. To generate reports about incremental changes use the following commands:

```
report out_of_date
report state_diff
```

These commands are similar to the corresponding **database query_state** commands, but generate report files. For the complete syntax for the **report** commands, refer to report out_of_date and report state_diff.

Querying Jobs

1. Use the *job* command to query pending, running, or completed jobs.
   - Use the **job list** command to list the jobs and their IDs.
   - Use the **job query** command to query a particular job, using its ID. For example:

     ```
     job query job232 -run_time 11
     ```

   See job for details about the syntax of this command.
2. You can also create a simple Tcl script.

   This example lists all the job IDs and job names, and reports their status.

Feedback

```
set aa [job list -all];
puts "Job ID\t Job Name\t\tJob Status";
foreach ele $aa {
puts [format "%-12s%-22s%5d" $ele [job query $ele -job_name]
 [job query $ele -return_code]]
}
```

Querying Metrics for a Design

Keeping track of metrics is important for measuring and tuning the QoR of a design. Metrics include data from various steps in the design flow; the data is saved and can be retrieved anytime. The design metrics you can query include the number of LUTs, the runtime for each process step, the worst slack, the slack for specific clocks, and the number of unconverted gated clocks.

1. Start from the tool Tcl window.
2. To find the names of the metrics available for the design, use one of the following arguments with the **`database query_state`** command:

database query_state	Use it to...
-dump_metrics	Shows the metrics that can be queried for the design.
-available_metrics	Show metrics that can be queried for the design as a Tcl list that can be used for scripting.

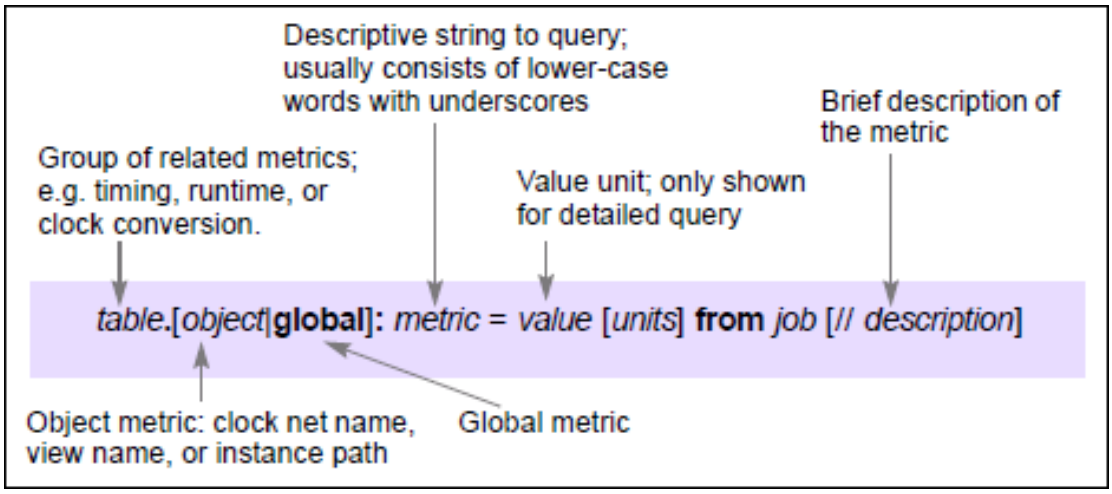
   For details about the command syntax, see database query_state.

3. Use the metric names with one of the following *database query_state* commands, according to the level of detail you want to see.

database query_state	Use it to ...
-metric	Query information about a QoR metric. For example:`% database query_state -metric runtime.realtime -jobname compiler 3.154000`
-metric_details	Show metrics and values available for the current database.`% database query_state -metric_details clock_conversion.clean_clock_pins -jobname fpga_mapper 271 {} {Number of clock pins driven by non-gated/non-generated clock trees}`

Feedback

Metrics can be global for the entire design, or specific to an object, such as a clock, module, or net. The command returns values with the default output format shown below:



This is an example of the main fields for clock conversion metrics:

***Table* Value**	***Metric* Name**	**Description**
clock_conversion	*clean_clock_trees*	Number of non-gated/non-generated clock trees
clock_conversion	*clean_clock_pins*	Number of clock pins driven by non-gated/non-generated clock trees
clock_conversion	*gated_clock_trees*	Number of gated/generated clock trees
clock_conversion	*instances_converted*	Number of sequential instances converted
clock_conversion	*instances_notconverted*	Number of sequential instances left unconverted

This is an example of reported clock conversion metrics:

```
clock_conversion.global: instances_converted = 0 from fpga_mapper
 //Number of sequential instances converted
runtime.global: realtime = 4.160000 seconds from compiler
runtime.global: cputime = 3.073220 seconds from compiler
```

Dealing with Black Boxes in the Synthesized Netlist

Black boxes in the synthesized netlist can cause problems during place and route. If you have black boxes, first identify why the information was missing for the black boxes, and then handle the situation as needed.

1. Identify the reason why the black boxes were created. There are two main reasons for this:
   - In designs that include DesignWare® IP, black boxes can be created for the IP if the appropriate tools cannot be accessed. See step 2 for information about dealing with these black boxes.
   - In mixed HDL designs, the tool creates black boxes when it cannot identify entities because of mismatches. See step 3 for information about dealing with these black boxes.

   These are some examples of possible messages:

   ```
   CG389 @E: Reference to undefined module.
   CD280 @W: Unbound component <name> mapped to black box.
   ```

2. To eliminate DesignWare IP black boxes, check that you have followed the guidelines below.

   Not following these guidelines could cause the tool to generate black boxes for the IP.
   - Run synthesis on a Linux machine.
   - Ensure that you have licenses for both DesignWare and Design Compiler®.
   - Make sure to use a compatible version of DesignWare; check the release notes for compatible versions.
   - Make sure you have correctly specified the path to the Design Compiler installation.
   - Make sure you have specified the DesignWare libraries appropriate to the IP you are using.
   - Provide the black box definitions before running place and route.
3. For black boxes in mixed-language designs, check the following:
   - Check for warnings related to port mismatches or missing module definitions.
   - Make sure VHDL entities have been correctly identified.
   - Add the definition for the module or entity to the source file list before compiling. The definition can be in Verilog or VHDL format, or an EDIF or NGC netlist.

Checking Timing Results

You can generate timing reports at different design stages to check the performance. The timing reports are based on estimates. The more mature the design, the more accurate the timing estimates reported.

See the following for more information:

- Generating Inter-FPGA Path Timing Report
- Generating Timing Report for Estimates of Design Performance
- Identifying Objects in Post-Mapped Timing Reports
- Viewing and Correlating Results with the Timing Report View
- Generating Custom Timing Reports with the Timing Analyst
- Checking Clock Information
- Running SLTA

Generating Inter-FPGA Path Timing Report

Generate an inter-FPGA path timing report after System Route for timing analysis of the most critical nets crossing FPGA boundaries including multi-hop paths per unique start and end clock event pair. The report can aid in identifying what is causing major slacks and you can make changes accordingly to partition-routing and clock constraints. By fixing the timing on required paths, you can avoid multiple runs of runtime-intensive System Generate, Synthesis and Place & Route. After making appropriate changes, re-run from Pre-Partition, Partition, or System Route states until a satisfactory inter-FPGA path timing report is achieved.

To generate the report, set the state to System Generate and run the following command.

```
report timing -generate
```

The *ta.rpt* file is generated in the directory, *databasename_reports/systemgenerate*.

The complete syntax for the `report timing -generate` command is described in report timing.

Generating Timing Report for Estimates of Design Performance

The following procedure describes how to generate a timing report with estimates of design performance. You can run intra-FPGA timing analysis on a single FPGA or inter-FPGA timing analysis in a multi-FPGA design.

Feedback

1. To run timing analysis and generate a timing report, use this basic command:

   ```
   report timing -generate
   ```

   The tool runs timing analysis on the current database (mapped or partitioned netlist) and generates a report file, which is displayed automatically. The *ta.rpt* file is generated. The timing report varies, based on the netlist analyzed.

   The complete syntax for the report timing -generate command is described in report timing.

2. Use the *-mode* argument to specify the netlist you want to analyze and correspondingly, the type of timing analysis you want to run:

Timing Report Type	Netlist Type	-mode Option
Single-FPGA. Command run from mapped database		
Post-synthesis	Mapped	None, or *-mode default*
With backannotated place-and-route data	Mapped, with backannotated data	*-mode ba*
Multi-FPGA. Command run from system generate database. See Running SLTA.		
Post-partitioning	Partitioned	None, or *-mode default*
With backannotated place-and-route data for FPGAs	Partitioned, with backannotated data	*-mode slta*

   To use backannotated data, you must run place and route and import the backannotated data before generating the report. You can then run a command like this one that is issued from the mapped state:

   ```
   report timing -generate -mode ba
   ```

   If you do not specify *-mode*, timing analysis uses the netlist for the current map or system generate database state as the default.

3. To customize the timing report, do the following:

   - Open a schematic from a particular database state with the `view schematic` (map database state) or `view schematic time_budget` (system generate state) command. You must explicitly do this; if not, the tool opens the compiled schematic, which does not use post-mapping names for the objects. See Identifying Objects in Post-Mapped Timing Reports for more information about naming issues.
   - Narrow down the results by specifying the arguments you want when you run the command.

To Report Paths ...	Command Arguments
To/from/through specified ports, registers, pins, or clocks	*-from -to -through*
Based on slack times	*-slack margin*
Based on rise/fall clock edges	*-fall_from -fall_to* *-rise_from -rise_to*

- To make the results even more specific, use object query commands in combination with the from/to/through options. For example:

```
report timing  -generate -through [get_pins {ram_to_bus_Q_2_1.O}]
  report timing  -generate -through [get_pins {ram_to_bus_Q_2_1.O}]
 -mode ba
```

See Object Query Commands for the syntax of these commands. If you use the object query syntax and run the `report timing -generate` command from within a script, make sure to open the schematic first, so that the specified objects can be located.

4. Use the *-out* option with `report timing -generate` to create multiple reports for a particular database state.

   You can run multiple timing analysis runs on the same netlist and from the same database state. For each run, a new timing report is generated with the following file-naming convention: *<SG_Node_Name>.tim<n>*, where *n* starts from 0 and increments with each run. When you specify a name with the *-out* option, the tool creates a substate under the current database state with the name you specified. It then generates a timing report for the substate, called *stateName.substateName.ta.rpt*. For example, if you specify *-out trial1* from the *m0* database state, the timing report is called *m0.trial1.ta.rpt.*

5. Check the timing results by viewing the report.

   See Checking Reports and Log Files for tips on working with reports.

6. After mapping, run the *check_hstdm_timing* utility using the options you need to identify missing constraints. Check that all HSTDM paths are covered by DPO timing exceptions. To save runtime, specify the options you need; otherwise the utility runs all the reporting options.

   Common reasons for missing budget constraints are GCC issues, inferred clocks, differences in sequential optimization between time budgeting and synthesis, and IP synthesis issues.

- Specify the *-rx2user* and *-user2tx* options with `check_hstdm_timing`. The paths from receiver to user logic and from user to transmit logic are the ones that most commonly have missing constraints, so use these corresponding options to identify them.

  The utility generates a timing report: *dir/timing_correlation_dbStateName/fpgaHstdmTiming.rpt*.The report is a table that shows start and end point details.

- If the Constraints column in the report shows *None*, it indicates an error.

  Paths that go through HSTDM points in SLTA actually start or end in HSTDM logic for FPGA synthesis. The tool handles this by generating *fdc* time budget files with data path only (DPO) constraints like the ones shown in the following example.

```
# Time budget constraints from user logic to HSTDM logic (transmit)
# Constraints for Ratio 4
set cpm_snd_HSTDM_4by2_fb1_B23_A_4_data_0__c_pciclk_r [expr 46.5
 + $HSTDM_4by2_snd_constraint_adjust]
set_datapathonly_delay -comment {HSTDM tx budget HSTDM_4by2
 (fb1.uB:J23)->(fb1.uC:J13)} -rise_from [get_clocks
 -include_generated_clocks {c:pciclk}] -to
 {t:cpm_snd_HSTDM_4by2_fb1_B23_A_4.bit_tx.mserdes.D1}
 $cpm_snd_HSTDM_4by2_fb1_B23_A_4_data_0__c_pciclk_r
# Time budget constraints from HSTDM logic to user logic (receive)|
# Constraints for Ratio 4
set cpm_rcv_HSTDM_4by2_fb1_B23_A_4_data_0__c_pciclk_r [expr 26.1
 + $HSTDM_4by2_rcv_constraint_adjust]
set_datapathonly_delay -comment {HSTDM rx budget HSTDM_4by2
 (fb1.uB:J23)->(fb1.uC:J13)} -from
 {i:cpm_rcv_HSTDM_4by2_fb1_B23_A_4.rxbit.ISERDES_RX_DATA_00}
 -rise_to [get_clocks -include_generated_clocks {c:pciclk}]
 $cpm_rcv_HSTDM_4by2_fb1_B23_A_4_data_0__c_pciclk_r
```

- Look for inferred clocks and declare them.

7. Write out the missing DPO constraints in the FDC and XDC files by running `check_hstdm_timing` with the appropriate options.
   - Use the *-default_delay* option to specify a DPO delay value to use when writing out the missing constraints; otherwise the default value is 8. Base the DPO value on the TDM ratio, using a higher value for a higher ratio. Check the relative values from DPO categories that are not missing to get an idea.
   - Run the utility with the-*write_fdc* and *-write_xdc* file options, to write out the missing constraints in the FDC and XDC files, respectively.
   - Check the constraints in the *xdc* file and add the file to the Vivado run when you place and route.

Syntax

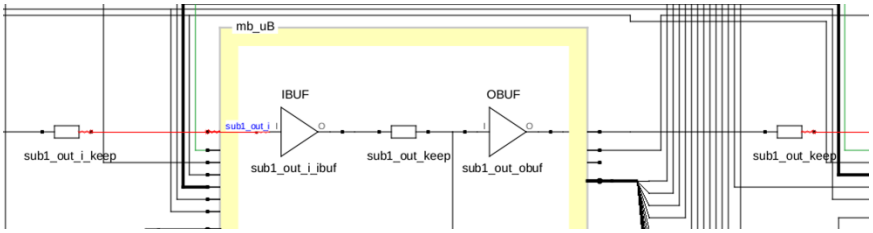
The *check_hstdm_timing* utility is a Tcl utility that identifies missing budget constraints and also writes out the missing constraints to the *xdc* and *fdc* files. Run it from the tool Tcl window after mapping. The utility has optional arguments; if it is run without any options, the utility takes longer because it runs all the reporting options.

Argument	Description
-default_delayvalue	Specifies the DPO delay value to use when writing missing constraints. The default is 8.
-port2tx	Reports paths from input ports to the transmit logic.
-rx2port	Reports paths from the receive logic to output ports.
-rx2tx	Reports paths from the receive logic to the transmit logic (multi-hop).
-rx2umr	Reports paths from the receive logic to the UMR logic.
-rx2user	Reports paths from the receive logic to user logic.
-slackvalue	Reports only those paths with a slack less than the given value. Default is 1500 ns.
-umr2rx	Reports paths from the UMR logic to the receive logic.
-user2tx	Reports paths from user logic to the transmit logic.
-write_fdc	Writes out an FDC file with DPO constraints for paths that were missing timing exception constraints.
-write_xdcfileName	Adds missing DPO constraints to the specified XDC file. You must specify the file name.

Identifying Objects in Post-Mapped Timing Reports

When timing reports are based on mapped databases, object names might no longer match the original RTL names. For example a RAM that was *{i.dest_u4.ram1[7:0]}* in the RTL might be called *{i:dest_u4_ram1_ram1_0_0}* in the mapped database. When you generate customized timing reports, you cannot use the RTL name of the object; the name must match the mapped name.

Nets with *-through* timing constraints can get segmented, and parts of the original constraint might no longer be applicable for SLTA. In the following figure, because of segmentation, the original false path constraint no longer applies to the left part of the net shown. However it still applies to the segment of the net on the right.



To make sure you identify objects correctly, do the following:

1. Open the schematic of the design with the `view schematic` command (map database state) or `view schematic time_budget` command (system generate state).

   This command opens a post-map view that has the correct names.

2. Use *fdc* query commands to identify the object correctly so that you can use it in your timing report generation command. For example:

```
% find -hier -inst {i:dest_u4.ram1*}
i:fb.uB.dest_u4.ram1_ram1_0_0
% report timing -generate -netlist slta -to [get_cells -hier
 {i:dest_u4.ram1*}] -out SLTA
```

Feedback

Figure 9 Example of Timing Report with BA Data

This timing report is generated from the map stage with back-annotated data (*-mode ba*).

```
Top view:               diff_clk_domains2
Requested Frequency:    100.0 MHz
Wire load mode:         top
Paths requested:        1
through:                ram_to_busQ_2_1.0
The system contains the following FPGAs:
Constraint File(s):
@N:MT320 :  | Timing report estimates place and route data. Please look at the place and route timing report for final timing.


Worst From-To Path Information
*******************************


Path information for path number 1:
      Requested Period:                      10.000
    - Setup time:                            0.000
    + Clock delay at ending point:           0.577
    = Required time:                         10.577

    - Propagation time:                      2.002
    - Clock delay at starting point:         2.307
    = Slack   :                              6.268

    Number of logic level(s):                2
    Starting point:                          RAM1.par[13] / Q
    Ending point:                            ram_to_busQ / D
    The start point is clocked by            dcm_test1|CLKO_BUF_derived_clock [rising] on pin C
    The end   point is clocked by            dcm_test1|CLKO_BUF_derived_clock [rising] on pin C

Instance / Net                Pin      Pin               Arrival     No. of         Location
Name                  Type    Name     Dir     Delay     Time        Fan Out(s)
-----------------------------------------------------------------------------------------------
RAM1.par[13]          FD      Q        Out     0.269     2.576       -              (TILE_x227y258)
ram_par[13]           Net     -        -       0.894     -           2              -
ram_to_busQ_2_1       LUT4    I1       In      -         3.470       -              (TILE_x227y258)
ram_to_busQ_2_1       LUT4    O        Out     0.053     3.523       -              (TILE_x227y258)
ram_to_busQ_2_1_i     Net     -        -       0.344     -           1              -
ram_to_busQ_2         LUT6    I4       In      -         3.867       -              (TILE_x227y258)
ram_to_busQ_2         LUT6    O        Out     0.106     3.973       -              (TILE_x227y258)
ram_to_busQ_2         Net     -        -       0.336     -           1              -
ram_to_busQ           FD      D        In      -         4.309       -              (TILE_x225y258)
===============================================================================================
Total path delay (propagation time + setup) of 2.002 is 0.428(21.4%) logic and 1.574(78.6%) route.
Path delay compensated for clock skew. Clock skew is added to clock-to-out value, and is subtracted from setup time value
```

Feedback

Viewing and Correlating Results with the Timing Report View

The Timing Report View provides a convenient way to view timing reports at certain stages of the design. It lets you view and query critical timing paths, and correlate certain timing results. You can view system-level timing, post-synthesis timing results alone, or in a composite view where they are correlated with the place-and-route timing results.

See these topics for details:

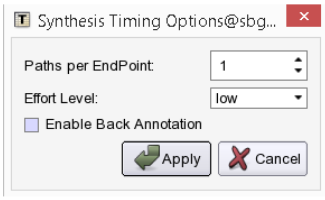
- Viewing Synthesis Results in the Timing Report View
- Correlating Synthesis Results With Place-and-Route Timing

Viewing Synthesis Results in the Timing Report View

The following procedure describes how to use the Timing Report view to analyze synthesis timing information from the *Map* state (Synthesis flow).

1. Display the **Timing Report** View.
   - From a mapped view, select **Tools > Timing Report Viewor**. Click the **Timing Report** view icon ( ).

     The Timing Report view opens (see Timing Report View). By default, the view shows the synthesis timing summary in the **Synthesis Timing and Correlation** tab. The summary matches the critical paths shown in the **Clock Relationships** section of the log file after synthesis.
   - Go to the **Synthesis Timing and Correlation** tab (default), if it is not open at this tab. The default timing view shows logic synthesis timing.
2. Set reporting and display options.
   - To expand all paths for the clocks and instances, click **Expand All**.
   - To collapse all paths for the clocks and instances, click **Collapse All**.
   - To change the number of paths displayed per endpoint, select the path and click **Options**. Set the number of paths to display in the **Synthesis Timing Options** dialog box.

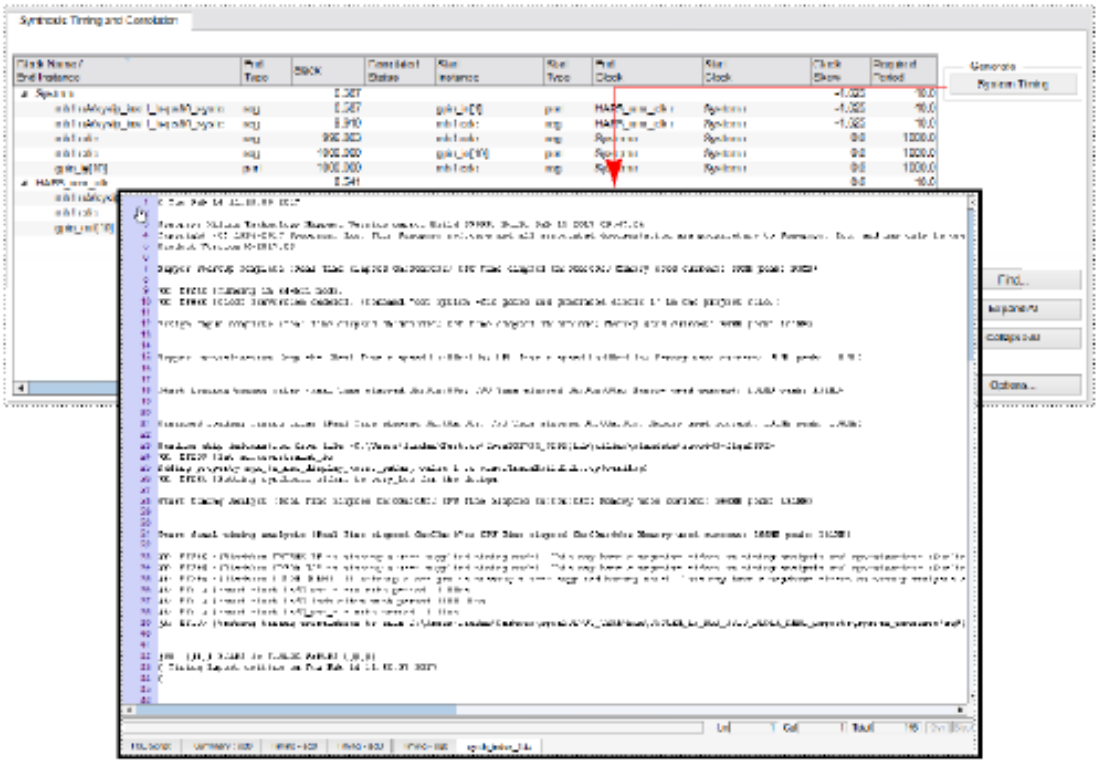


3. For **Synthesis Timing and Correlation** that correlate to the place and route results, you can specify the following effort levels:
   - *low* - Runs a default timing report for Vivado (no *-from*/*-to* filtering). From this report a first-pass correlation is run. For paths that did not correlate, switch to effort *Medium* and rerun for *All Paths*, or select individual paths and run *Selected Path*.
   - *medium* - Performs a low effort run, but continues to run the second-pass correlation until there is no further convergence of results. This occurs when the number of correlation failures does not change from the previous run.
   - *high* - Performs a medium effort level run. However, whenever convergence stops, it then continues running correlation on the remaining individual paths until all have been attempted. Be aware that effort level high can incur large runtimes, because Vivado is called repeatedly.
4. To locate objects like clocks or instances, click **Find** and type in the search term:



Feedback

5. To generate a timing report summary for a path, select the path and click **Synthesis Report**.



Correlating Synthesis Results With Place-and-Route Timing

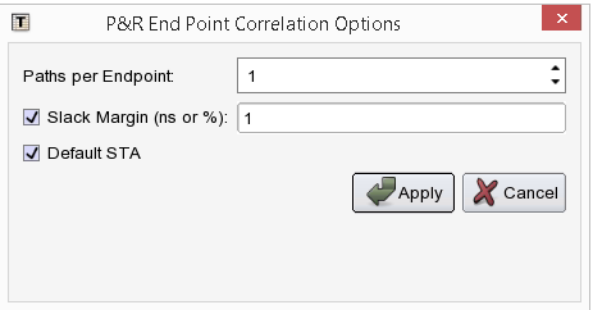
The following procedure describes how to use the Timing Report view to generate information from place-and-route timing, and compare it to the results from synthesis. Synthesis timing is considered the golden timing when correlating to P&R when in the **Synthesis Timing and Correlation** tab and P&R Timing results are considered golden when correlating to Synthesis when in the **P&R Timing and Correlation** tab. If you want to view the synthesis timing results only, use the procedure described in Viewing Synthesis Results in the Timing Report View.

1. To generate place-and-route timing information, follow these steps:
   - Synthesize, place, and route the design. See Running Place and Route for the basic procedure.
   - Display the Timing Report View by selecting **Analysis > Timing Report View** or by clicking the **Timing Report** view icon ( ). By default the view displays the timing results after mapping.
2. Set up to correlate results.

- Select the **P&R Timing and Correlation** tab.
- In the **P&R Directory** field specify where the Vivado P&R results are located. Typically this is the directory specified with the `export vivado -path` command for the Vivado run.

3. Run correlation and analyze results.
   - To correlate all paths, click the **All Paths** button on the right.
   - To only correlate selected paths, click **Selected Path** and select the paths you want.

     The window displays the place-and-route timing information and correlates it to the synthesis information. For example, you can compare synthesis timing results with P&R Static Timing Analysis (STA) results. The view reports the status of end points, start points, and required periods. Paths are reported against the end clock. The P&R path is shown first, with the synthesis path to be correlated below.
   - Look for red X marks () in the *Correlated Status* column, which indicate a mismatch. A green check mark () means that the synthesis and P&R timing results for end points, start points, and requested periods on that path match.
   - Float over the mismatched cell for a tool tip that describes the error. For a complete description of the Timing Report View options, see Timing Report View .
4. To view and analyze individual paths, use the following features in the view:
   - To generate a detailed timing report for a particular path, select the path and click **Synthesis Report**.
   - To generate a detailed P&R timing report for a path, select the path and click **P&R Report**.
5. Set other reporting and display options as needed.
   - Order the paths in ascending or descending order of slack values by sorting on the *Slack* column.
   - Change P&R timing correlation options by clicking the **Options** button and specifying the settings you want. For example, you can specify the number of paths to report per end point. If you do not enable any of the limiting options, the report includes paths for all the constraints. Rerun correlation by clicking **All Paths** or **Selected Paths** to view the updated results.



- To expand all paths for the clocks and instances, click **Expand All**.
- To collapse all paths for the clocks and instances, click **Collapse All**.
- To change the number of paths displayed per end point, select the path and click **Options**. Set the number of paths to display in the **Synthesis Timing Options** dialog box that opens.
- To locate objects like clocks or instances, click **Find** and type in the string to locate.

Generating Custom Timing Reports with the Timing Analyst

You can generate a customized timing report (`.ta`) when you need more details about a specific path, or for a path that is not one of the top five timing paths (covered in the log file by default). To generate the custom timing report, the tool uses the built-in timing analyst engine.

You can generate a custom timing report from a system generate or mapped database state. From the mapped state, you can alternatively launch Vivado and use the place-and-route engine to generate the timing report.

The following procedure shows you how to generate a custom report:

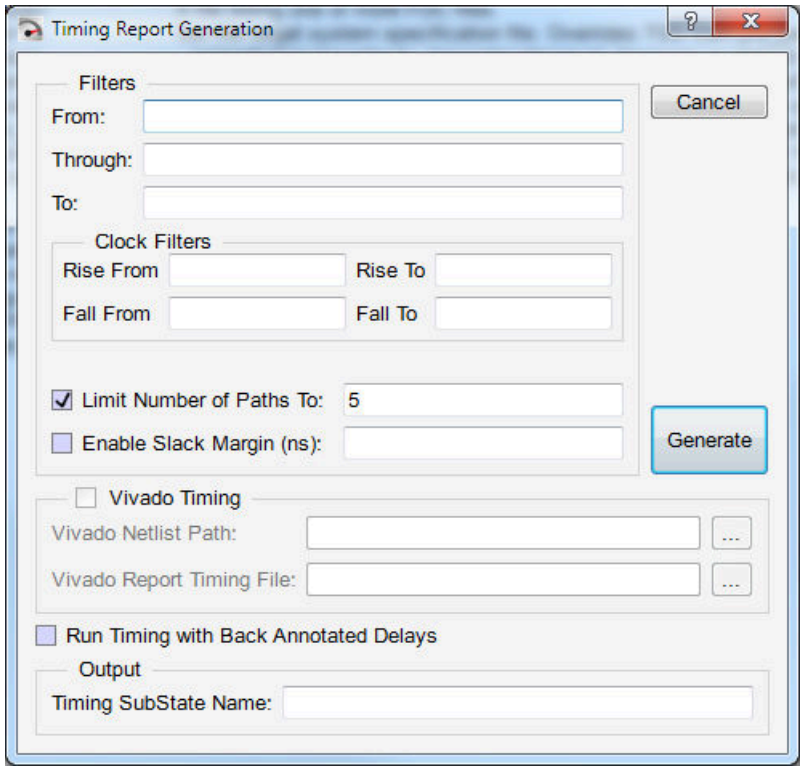
1. From a system generate or mapped database state, click the **Timing Analyst** icon ( ) or select **Tool > Timing Analyst**.

   From a mapped state, you can run detailed timing analysis using either the Timing Analyzer or the Vivado timing engine. From a system generate state, the tool runs system-level timing analysis (SLTA).

2. In the dialog box that opens, use the *From, Through*, and *To* fields to filter and specify the paths you need.
   - For timing analysis with the Timing Analyst, start from the map or timing budget schematic. Either cut and paste or drag and drop valid objects from there into the dialog box. Include the appropriate prefix if you type in the names: *{i:u.1.q}*. You can also use the `get` command to enter names.

Feedback

- For Vivado timing analysis, enter names in the Vivado format with / as the hierarchy separator. If they are entered in the Synopsys format (*{i:u.1.q}*, the tool attempts to convert them automatically to the Vivado format: *{u1/q})*. You can also use the *get** query commands in these fields.
- Specify the objects using the guidelines described in Table 5.



3. Specify other options for the custom timing report.
   - For the rise and fall options in the Clock Filters section, make sure to enter clock aliases with the *c:* prefix.

     For Vivado timing analysis, enter Vivado clock names. Get the names either from the Vivado place and route timing summary generated by the prototyping tool (*vivadoDir/*_timing_summary.txt*) or from Vivado (*vivadoDir/vivado.log*).

     You can also use the *get** query command to filter the clocks. See Table 5 for details.
   - If needed, change the number of paths generated by setting the **Limit Number of Paths To:** and **Enable Slack Margin** options. If both are set, the report is based on the specification that is the most constricting.
   - Specify a name for the output timing report (ta).

4. To generate a custom timing report from the map state, click **Generate**.

   The tool generates a timing substate under the current database state, and a custom report file (*.ta*). You can also view the corresponding schematic for the timing paths.

5. To generate a custom timing report from the map state using the Vivado engine instead of the Timing Analyst, do the following:

   - Start from a mapped state and run `export netlist` to create and set up the directory structure for place and route. Then use `launch vivado` to run place and route.
   - From the mapped state, start the Timing Analyzer and enter the filter criteria as described in steps 1 to 3. Use the Vivado format for names, with the / hierarchy separator, if possible. You can also use the `get` command to enter names: *[get_cells {u1/q}]*.
   - Enable **Vivado Timing**.
   - In **Vivado Netlist Path**, point to the Vivado directory created by `export netlist`.
   - Specify a name for the timing report file in **Vivado Report Timing File**. If no path is specified, it defaults to the current working directory.
   - Click **Generate**.

   The tool launches and runs Vivado timing analysis for the selected paths instead of the Timing Analyst.

6. To run system-level timing analysis, do the following:

   - Start from a system generate state.
   - Start the Timing Analyzer and enter the filter criteria as described in steps 1 to 3.
   - Click **Generate**.

   The tool runs system-level timing analysis (SLTA). The timing analysis reports run from this state report inter-FPGA paths; to see detailed intra-FPGA timing, run timing analysis from a mapped state. For additional information about the SLTA process, see Running SLTA.

7. To run system-level timing analysis with backannotation, do the following:

   - Partition the design, and synthesize and place and route the FPGAs.
   - Backannotate the place and route results into each FPGA as described in Importing Place and Route Results for Back-annotation.
   - Start the Timing Analyzer from a system generate state and enter the filter criteria as described in steps 1 to 3.

Feedback

- Enable **Run Timing with Back Annotated Delays**.
- Click **Generate**.

The tool runs SLTA using the back-annotated data.

8. Analyze results using the timing file and schematic.

Table 5 Specifying From, To, and Through Points for Custom Timing Reports

Specifying Through Points
Specify a through point by itself or in combination with *From* and *To*. Specify a net, hierarchical port, or instantiated cell pin. You can specify multiple objects.You can also enter a *get** query command. Typed-in names must include the appropriate *n:*or *t:*object prefix. Drag and drop automatically enters the syntax correctly. Synopsys-style names are automatically converted for place and route. To type in Vivado-style names for Vivado timing analysis, use the *get* command. For example: *[get_cells {u1/q}].*You can use wildcards, as described in Timing Analyst Examples with Wildcards. To generate a report for paths that go through ANY of the points specified (OR list), use a space-separated list that is optionally enclosed in curly braces. This example reports every path that passes through either port a12 or port c3: *{t:a12 t:c3}*See Path Filter Combinations for the Timing Analyst for examples.
Specifying From/To Points
Specify a port, register, register pin, or clock alias. You can specify multiple objects. You can also enter a *get** query command; for example: *[get_cells {u1/q}].* Typed-in names must include the appropriate *p:* (top-level port), *t:*, (hiearchical port or instance pin)*i:*(instance), or*c:*(clock alias) object prefix. Drag and drop automatically enters the syntax correctly. Synopsys-style names are automatically converted for place and route. To type in Vivado-style names for Vivado timing analysis, use the *get* command. For example: *[get_cells {u1/q}]..* See Path Filter Combinations for the Timing Analyst for examples. You can use wildcards in the names, as described in Timing Analyst Examples with Wildcards. Narrow down results by specifying both *From* and *To*, or specify just one of them.
Specifying Clock Rise/Fall From/To Points
Specify a clock alias as listed in the Performance Summary. Make sure to include the *c:*(clock alias) object prefix. You can also use the *get* command to specify names.To specify clocks for Vivado timing, use the clock names form the Vivado Place and Toute Timing summary in the tool, or from the Vivado log. You can use the *get* command to specify names. You can use wildcards in the names, as described in Timing Analyst Examples with Wildcards.

Feedback

Checking Clock Information

You can check clock information at different points in the prototyping process.

1. Check the clock information in the log file (`view report` command or **Report View** icon) at any stage of the design.

   This lists the reports available for the current database state.

2. After compile, check for constraint errors with run `report constraint_check`.

   The command generates a report that lists errors in constraints. Fix the errors before proceeding.

3. Check the gated clock report for information about gated clock conversion.

   See Interpreting Gated Clock Error Messages for information about specific messages.

4. Check clock information after pre-partition.

   - Run `report target_system` to generate the reports. Use the `view report pcf_tss_report.rpt` command to view the TSS report, which includes details about the target system.
   - Check that all clocks are listed in the **Clock Summary** section.
   - Check that all clocks have been declared. If there are any inferred clocks, specify clock definitions for them with FDC *create_clock* constraints.
   - Check the values in the **Clock Settings** section of the TSS report.
   - Check for inapplicable constraints and fix them before proceeding.

5. Check clock skew.

   - After partitioning, check the log file for clocks and asynchronous resets that cross FPGA boundaries.
   - Specify clock tree replication and rerun partition:

     ```
     run partition -clock_gate_replicaton 1
     ```

   For details, see Partitioning Clocks for details.

6. After system generate, check the following clock information:

   - View the log file, and check that there are no inferred clocks.
   - Check for derived clocks in the **Clock Relationships** section. Set *create_generated_clock* constraints for them in the FDC.
   - Run SLTA and check results. See Running SLTA for details.

7. Run the following checks at the map stage:
    - Before mapping, view the log file, and check that there all clocks have been declared. If there are any inferred clocks, specify clock definitions for them with FDC *create_clock* constraints.
    - Check for inapplicable constraints and fix them.
    - After mapping, check the **Clock Relationships** section to make sure that asynchronous domains have been properly defined for asynchronous clocks. If needed, adjust constraints and rerun mapping.

Understanding the Clock Usage Report

The Clock Usage report is printed in `partition.log` and it gives a summary of how much of your design is controlled by each clock. Sometimes, the total clock usage shown in the report is more than 100%. This can be confusing.

The following explanation clarifies how to interpret this behavior.

Consider a design with two clocks: CLK_A and CLK_B.

- CLK_A controls 100% of the design.
- CLK_B controls 10% of the design.

This does not mean the total design size is 110%. Instead, it reflects overlapping clock domains. For example:

- A sub-block (e.g., Block A) might be controlled by both CLK_A and CLK_B.
- If Block A represents 10% of the design, then both clocks are counted as driving that 10%.

Thus:

- CLK_A drives logic in Block A and other blocks, so it is 100% usage.
- CLK_B drives only Block A, so it is 10% usage.

Note:

- The report performs a top-level estimation of clock influence.
- If the report states that CLK_A controls 100% of the logic, it means that, at the top level, CLK_A reaches all relevant instances.
- It does not imply that CLK_A controls every net inside each hierarchy.
- The deeper the hierarchy is analyzed, the more accurate the estimate becomes.

Running SLTA

System level timing analysis (SLTA) gives you timing estimates of the inter-FPGA paths and this can help identify bottlenecks. It includes timing models for TDM, cable, and I/O delays that local static timing analysis does not account for. You can run SLTA to analyze all the data paths between FPGAs after partitioning, instead of analyzing individual path segments. You can also run SLTA with backannotated results from place and route.

1. Partition the design as usual (*run pre_partition, run partition, run system_generate*).
2. To generate a report using back-annotated place-and-route data, do this:
   - Synthesize, place, and route the FPGAs. You can run the jobs in parallel using the `launch protocompiler -script` command.
   - Import the place-and-route data to back-annotate the design, using the `database apply_state -import_vivado` and `database apply_state -backannotate` commands. See Importing Place and Route Results for Back-annotation for details.
3. From the system generate database state, run SLTA with the `report timing -generate -mode slta` command.

   The SLTA timing report covers inter-FPGA paths. HSTDM is treated as through points.

   There are some limitations when you use backannotated results to run SLTA:
   - Only net delays are back-annotated, not cell delays.
   - SLICE packing might prevent some nets from being updated.
   - TDM is an approximation, and not the actual physical result.
   - Do not modify your top-level timing constraints, as this can lead to unpredictable results.
4. Check the results and make adjustments to the design.
   - Check if the selected HSTDM ratios meet performance targets.
   - Check for clock skews between FPGAs, especially if clocks cross on HT3 connectors.
   - Check for other performance bottlenecks like multi-hop paths.
   - For SLTA with back-annotation, check inter-FPGA data path delays and clock path delays. Verify back-annotated data by checking the location information in the report.

   If both the Vivado timing report for the intra-FPGA paths and the back-annotated SLTA report are clean, it is a good indicator that the prototype runs on the hardware at the specified frequency.

Resolving Errors and Warnings

Open the log file or report as described in Checking Reports and Log Files and analyze the messages. Errors are prefixed with @*E* and warnings are prefixed with @*W*. You must resolve all errors and check all warnings.

1. Resolve all errors.
   - Search for messages with an @*E* (error) prefix, which indicate error messages.
   - If a message references an error in a file that cannot be exported, use the `edit` command to open the file, as described in Checking Reports and Log Files.
   - Fix all errors, because you cannot continue with the implementation of a design with errors. If the message is not self-explanatory or if you are unsure about how to handle it, click the message ID to go to the documentation for the message.
   - If the software crashes, check for error messages that help narrow down the search and pinpoint what caused the crash. For example:

     ```
     @E BN591 More debug information: The last object processed
     before the error was <objectName> in module <moduleName>.
     ```

     You can then check the specified object, module, or view to identify errors and fix them. Depending on the design, you could black-box the identified module and continue working on the rest of the design.
2. If you get warnings (@*W* prefix) after a run, do the following:
   - Search for and review messages with a @*W* (warning) prefix.
   - Check the warnings and make sure you understand them. Decide if the situation is something you need to act on, or whether you can ignore it.
   - In the GUI and HTML reports, if the message is not self-explanatory or if you are unsure about how to handle the error, click the message ID to go to online information about the condition that generated the warning.
3. Check other messages.
   - Skim through the notes (@*N)*. You can usually ignore notes, because they are typically informational in nature.
   - If you see *Automatic dissolve at startup* messages, you can usually ignore them. They indicate that the tool has optimized away hierarchy.
4. Press *Ctrl* + *F* or type the prefix you want as the criteria on the **Find** window(for example, @*W)*.

Feedback

5. Click *Mark All*. The software inserts bookmarks at every line that has the prefix you specified. You can now page through the file from bookmark to bookmark using the commands in the *Edit* menu.
6. You can display only the messages you want to see, using the techniques described in Overriding Message Designations and Suppressing Output.
7. To save a report, use the `export report` command and specify a directory location.

   Do this if you want to save or modify results from a file, because the database files cannot be accessed directly.

You can display only the messages you want to see by suppressing them. For a certain class of messages, you can also change their designation and downgrade them. You can also upgrade other messages to errors. Overriding a message with another message designation lets you elevate or downplay a particular message. You can also set limits and specify the messages to be reported.

See the following topics for more information:

- Overriding Message Designations and Suppressing Output
- Setting Limits on the Number of Messages in a Log File
- Generating a Custom Error Message Report
- Downgradable Errors and Other Message Categories

Overriding Message Designations and Suppressing Output

You can change certain message designations or suppress messages, using either the GUI or Tcl.

1. Create a Tcl file that contains `message_override` commands that customize the display of messages.
   - Start the file with `message_override -clear`, so that previous override settings are negated.
   - Specify `message_override` commands to customize the display of a message. For example, you can use this command to designate downgradable errors non fatal errors (*DE* prefix) as warnings, or upgrade warnings to errors. See Downgradable Errors and Other Message Categories for a descripton of specific changes you can make to each category of messages.

To ...	Use ..
Treat a *DE*message as a warning	*message_override -warningmessageIDlist*

To ...	Use ..
Treat a warning or critical warning (*CW)*as an error	*message_override -errormessageIDlist*
Treat a warning as a note	*message_override -notemessageIDlist*

Do not include the @ symbol when you specify the message number. For example:

```
message_override -warning DE130
```

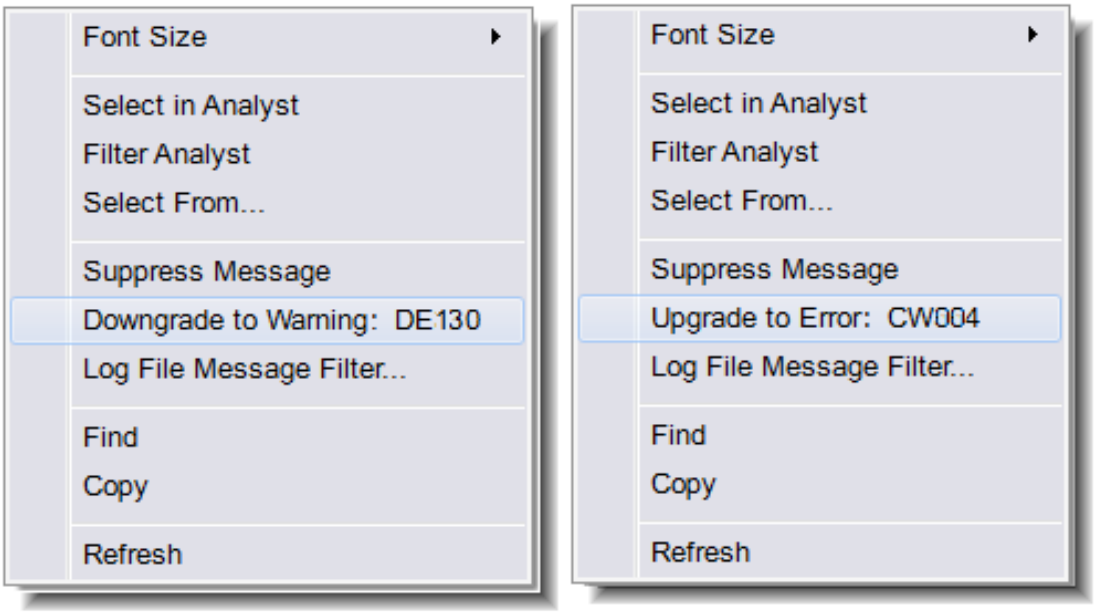
- Specify limits on the number of messages displayed with the *-suppress*, *-limit* and *-count* arguments to *message_override*. See Setting Limits on the Number of Messages in a Log File for details.
- Source the Tcl file from the tool window when you want to change a setting, or when you first start the tool.

  To automatically invoke the message settings every time you start the tool, add the `source` command to any of the setup files listed below. These are the setup files for the installation, personal setup, and the current working directory, respectively, and the tool reads them in that order.

  ```
  installDir/.synopsys_pc_setup
    ~/.synopsys_pc_setup
    ./.synopsys_pc_setup
  ```

Instead of creating a Tcl file, you can alternatively type individual `message_override` commands in the tool window.

2. Do the following to manipulate message output from the GUI:
   - Go the message ID in the log file.
   - Right-click the five-character message code, and select the appropriate command to upgrade or downgrade the message from the popup menu. You can only downgrade errors with a *DE* message prefix, and only upgrade warnings with a *CW* prefix.



The selected action moves the message from the upper to the lower section of the message filter. The *Override* column reflects the disposition of the message. The TCL window prints a message to confirm the change. For example:

```
message_override -warning DE130
```

3. To see the changes reflected, rerun the state in database state where you changed the message designation.

   For example if you suppressed a message in the *c0* compile database state, recompile the *c0* state to see the changes.

   The tool handles the changes as shown in the following table:

ID Status Change	Tool Action
Changed to an error	Stops executing at the first occurrence of the message and prints the message in the @E:*msgID* :*messageText* format
Changed to a warning	Prints the message in the @W:*msgID* :*messageText* format.
Demoted to a note	Prints the message in the @N:*msgID* :*messageText* format.
ID suppressed	Excludes the message from the `srr` file.

Modifying the classification does not affect the message string for reporting. A message originally categorized as an error continues to be reported as an error regardless of its user-assigned status.

4. Change the message classification back to its original designation.
   - Fix the cause of the upgraded warning (critical warning) or complete the rest of the flow (downgradable error).
   - To use Tcl to change a message back to its original designation, use the `message_override` command with the original designation argument. For example:

     ```
     message_override -error DE130
     ```
   - To use the GUI to change back to the original classification, right-click one of the modified messages in a report and select the appropriate **Revert** option from the pop-up menu.
   - Rerun the state again to confirm that the message status has reverted to the original.

Setting Limits on the Number of Messages in a Log File

The number of messages reported in the log file can become overwhelming. Use the *-suppress*, *-count* and *-limit* arguments to the `message_override` command to control the number of messages reported.

1. Use *-suppress* to prevent specified notes and warnings from being reported. You cannot suppress errors.

   ```
   message_override -suppress BN10
   ```
2. Use *-limit* to set the number of messages reported in the log file.

   This example limits MF580 and MF581 to 1000 each in each log file:

   ```
   message_override-limit {MF580 MFS581} 1000
   ```
3. To report all occurrences of a particular message, use *-limit* and *-count unlimited* together:

   ```
   message_override -limit {CL118} -count unlimited
   ```

Generating a Custom Error Message Report

Use this procedure to generate custom error message reports. For example, you can generate reports that contain only @*E* messages.

1. To generate a list of messages from a specific report file, execute the following command.

   ```
   report message <reportName>
   ```

2. To generate a list of all messages, execute the following command without arguments.

   ```
   report message
   ```

Downgradable Errors and Other Message Categories

Downgradable errors are specially designated messages whose classification can be temporarily changed to warnings. They have a special *DE* prefix that identifies them.

You can downgrade or upgrade messages from the GUI or through Tcl, as described in Overriding Message Designations and Suppressing Output.

The following table describes the message designations you can change:

Downgradable errors (@*DE*)	Can be downgraded to warnings. This is a small set of *non-fatal* errors where you can temporarily postpone addressing the error and continue with the design flow and verification of other aspects of the design. For example if you are pipe cleaning a design, you can downgrade errors about tight user clock constraints or incomplete trace assignments, even though this negatively affects the results, because the quality of the results is not your focus at that point. You must deal with the over constraining or assignment errors on later runs. See Overriding Message Designations and Suppressing Output for comprehensive details.
Errors (@*E*)	Cannot be downgraded or suppressed.
Critical warnings (@*CW*)	Can be upgraded to errors or suppressed. This small set of warnings represent critical problems. Elevating them to errors ensures that they are recognized and dealt with, because the error status forces the tool to stop.
Warnings (@*W*)	Can be upgraded to errors or suppressed. Elevating them to error status forces the tool to stop when it encounters them.
Notes (@*N*)	Can be upgraded to warnings or suppressed.

Running Simulation

You can check generated RTL by running simulation with the Synopsys VCS® product, a high-capacity simulator for Verilog code. The first step is to verify the testbench. After that,

you follow the normal steps in the design flow and simulate at different points to check your design against the RTL.

You can run VCS simulation on native RTL or after mapping. VCS is the recommended tool for simulation because it includes functionality to handle HAPS-specific encrypted RTL, and because the PLI server for UMRBus is precompiled in the VCS tool.

Simulation Flows

You can run simulation on a single-FPGA or multi-FPGA design.

See the following for details about the steps in the flow:

- Setting up the Tools and Testbench
- Generating a Testbench from FSDB

Setting up the Tools and Testbench

The following steps describe how to set up the tools and verify the testbench so that you can run simulation with VCS.

1. Set up the tools.
   - Make sure the VCS and Vivado tools are installed. Vivado is required to run gate-level simulation, because it includes libraries for Xilinx primitives.
   - Set up the environment variables for the tools needed to run simulation:

     ```
     export VCS_HOME=path
     export XILINX_VIVADO=path
     export VM_NETLIST_DIR =path
     ```

VCS_HOME	Path to the VCS installation; required.
XILINX_VIVADO	Path to the Vivado installation; required for Xilinx primitive libraries.
VM_NETLIST_DIR	Path to the directory where synthesis files are copied for simulation; required for top-level simulation in multi-FPGA designs. (Multi-FPGA designs only.)

When you set *VM_NETLIST_DIR, t*he tool replaces placeholders in the top-level wrapper file (*top_hdl.vp*) with the directory you specified in *VM_NETLIST_DIR* when you run System Generate. When you export netlists, the synthesis files in this directory include a partitioned netlist for each FPGA (*fb1_uA.vm*, *fb1_uB.vm...)* as well as a *design.hapsinit.vm* file for each FPGA *(fb1_uA_hapsinit.vm,*

*fb1_uB_hapsinit.vm...)*Specifying the directory with the files ensures that VCS finds the *.vm* netlists and *design.hapsinit* files for the individual partitions when simulation is run.

2. Make sure you have the following additional files to run simulation:
   - Original RTL source files for comparison
   - VCS testbench
   - For gate-level simulation, add the Xilinx primitives library: *$XILINX/verilog/src/unisims/**
   - To simulate the GPIO interface, make sure to include the appropriate GPIO *.v* file in the simulation: For example:

     ```
     installDir/lib/synip/gpio/verilog/HAPS70_2000T_GPIO.v
     installDir/lib/synip/gpio/verilog/HAPS70_GPIO.v
     ```

3. Verify the testbench by simulating it.

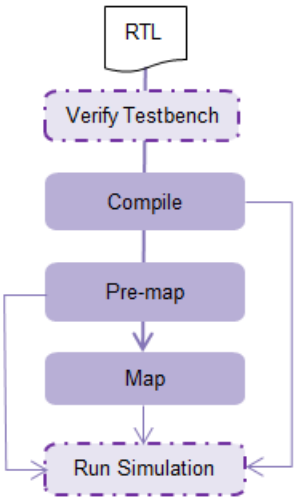
   It is recommended that you use VCS to validate the testbench with the original RTL, before running the prototyping tool. Once it is verified, you can use the testbench to run simulation.

   You can now run simulation. See Running Gate-Level Simulation for a Single-FPGA Design for more information. You can run gate-level or RTL simulation on your design, depending on the design stage.

   See the following for more information:

   - Running Gate-Level Simulation for a Single-FPGA Design

Running Gate-Level Simulation for a Single-FPGA Design



Feedback

The following steps show you how to run gate-level simulation for a single FPGA and functionally verify the RTL. You can run VCS simulation after the compile, pre-map, or map stages.

1. Complete the setup and validate the testbench.

   See Setting up the Tools and Testbench.

2. Load the database you want to simulate.

   You can run simulation on compiled, pre-map, or mapped databases. For example:

   ```
   database set_state {m0}
   ```

   For the testbench, it is recommended that you initialize all inputs to the design to a known logic value; avoid *z* or *x* initial values. For example:

   ```
   reg in1;
   reg in2;
   ...
   initial
   begin
    #0;
    in1 <= 1'b0;
    in2 <= 1'b0;
   ...
   ```

   When you synthesize the design, the tool generates the compiled or synthesized netlist (*.vm*). Different stages generate different files for simulation:

Compiled database Pre-map database	*design_compile.vm* (Netlist)
Mapped database	*design_compile.vm* (Netlist) *design_hapsinit.vm* (HAPS Initialization)

3. Export the files needed for simulation using the `export netlist` command.

   ```
   export netlist -path directoryPath
   ```

   This command writes out the *vm* netlist file to the specified location. If run on a mapped design, it also writes out the *design_hapsinit.vm* file, which is only generated after mapping. This file is required to correctly handle the mapped design in simulation.

4. If running simulation on a mapped design, instantiate the *hapsinit.sim* module in the testbench.

   Make sure to connect clk_0 to the inverted clock net. For example:

   ```
   // Required testbench changes:
   dut dut_inst (
   ```

Feedback

```
.clk (clk_net),
.reset (reset),
.clk_0 (~clk_net), ........);
hapsinit_sim hapsinit_sim_uut ();
```

This module is only generated after mapping. It is not required for simulation at earlier stages of the design.

5. Launch the VCS software and run simulation.
   - This first step is optional. If you have a mixed language design or need to analyze libraries, use the *vhdlan* or *vlogan* commands to run analysis.
   - If you are simulating a mapped design, make sure to specify the *design_hapsinit.vm* file as input to the `vcs` command. Specify it after the model file. For example:

     ```
     vcs design.vm hapsinit.vm testbench libraries
     ```

     You do not need this file to run simulation after compile.
   - If you have Xilinx primitives, specify the Xilinx simulation libraries from the Vivado installation (*$XILINX_VIVADO/data/verilog/src/unisims*).
   - Run simulation with the VCS `simv` command.

Generating a Testbench from FSDB

If you have a fast signal database (FSDB), you can generate a testbench from it instead of writing one. Use the Verdi and VCS tools to generate a testbench from an FSDB. For details of the syntax and commands, refer to the corresponding tool documentation.

1. From the Verdi tool, extract the testbench using the *fsdbextract* command.
   - Extract the FSDB for the scope that you want, which is typically the module you want to simulate. This is an example of the syntax:

     ```
     fsdbextractinputFsdb -s scope -o outputFsdb
     ```
   - Convert the FSDB to VCD with the *fsdb2vcd* command:

     ```
     fsdb2vcd inputFsdb -o outputVCD
     ```
2. Create a *cfg* file for the VCS *vcat* functionality to generate the testbench.

   The *cfg* file has three things defined inside it: <*scope*>, testbench, <*top part of module definition*>.This is used as input to *vcat*. The scope here must be the same as the scope defined in the previous step.
3. In VCS, create the testbench by running *vcat* with this command:

   ```
   vcat -vgen <cfg file created in step 2> <input VCD file>
   ```

Use the testbench created in the previous step to run simulation in the prototyping tool.

Running Place and Route

When you are ready to place and route the design, the only launch method is to run Vivado in normal mode.

Note:
The Exploratory Place-and-Route feature or the `run par_explorer`command is no longer used or recommended for placement and routing of designs. Use the Auto-SPO (Smart Parameter Optimization) (Setting Auto-SPO) feature for design exploration and optimization.

For more information about the place and route techniques and the backannotation of results after place and route, see these topics:

- Running Place and Route
- Customizing Scripts to Place and Route a Single-FPGA Design
- Importing Place and Route Results for Back-annotation
- Resynthesizing the Design After Place and Route

For information about exporting the design to run on the hardware after place and route is complete, see Preparing to Run on the Hardware.

Running Place and Route

After you have finished mapping, you can launch Vivado placement and routing as described here.

See Shell Command Reference for details about the commands mentioned in the following procedure.

Before you begin

Ensure that you have a mapped design. See Mapping the Design to map the design.

If you have not already done so, ensure you have Xilinx Vivado properly installed and licensed, and that you have set the appropriate environment variables to run the place-and-route software. See Setting Environment Variables.

1. Export the database with the *export vivado* command.
    - You must specify the path to the target directory with *-path*. This command creates a directory with all the input files needed to run place and route. It includes the default EDIF netlist generated for Vivado, corresponding constraints in an *xdc* file,

Feedback

and a *run_vivado_haps.tcl* automation script, which contains design variables and commands for setup, running place and route, and reporting.

For further information about the programming interface, refer to the Vivado Design Suite documentation on Tcl commands and scripting.

- To write out the netlist for Vivado in Verilog instead of the default EDIF, include the *-format verilog* option when you specify the *export vivado*command. With this option, the command writes out a *designName.vm* file instead of EDIF. It also writes out a corresponding *xdc* file, and the *run_vivado_haps.tcl* script to run Vivado uses *vm* as the input format instead of EDIF.

  If you transfer exported files to another location, you must copy all exported files. This includes encrypted files like the *.cfg* file, which contains LUT configuration information that is required for HAPS system bring-up and configuration. This file is generated for each run, and is a required file for successful bring-up.

2. Run place and route by specifying the *launch vivado* command with the name of the place-and-route Tcl script to run.

   The tool launches the Xilinx place-and-route software and runs place and route using the options you set in the Tcl script. It generates the following files in the specified place-and-route directory:

area.txt	FPGA resource utilization
clock_utilization.txt	Clock utilization
pinloc.txt	I/O assignments
design_post_par.xdc	I/O and logic placement constraints
post_route_drc.txt	DRC report

3. To view the place-and-route results, follow these steps:
   - Ensure the place-and-route state is active and type *view report* in the Tcl window.
   - In the Xilinx P&R Report, check the *Timing Summary Report* section for an overview of the timing. Judge whether the results meet your prototype requirements. There are two Vivado timing summaries, one with worst case conditions and the other with optimal conditions for process variation, voltage and temperature.
   - Check the *area.txt* output file for area utilization summaries by slice, memory, DSP, and I/O resources of the FPGA.

As with other states in the process, you can return to a previous database state and rerun it. If the results meet your requirements, you can generate a bit file (Exporting Files for Runtime).

4. To run place and route incrementally, use the design checkpoint (*dcp*) file as the starting point for the next run.

   This is a typical sequence of Xilinx commands:

   - Load the current design (*link_design*).
   - Run *opt_design*.
   - Read the incremental design checkpoint file (*read_checkpoint -incrementaldcpFile*).
   - Run *place_design*.
   - Run *route_design*.

   The tool uses an incremental flow with the *dcp* file used as a reference point. The place or route operation is run incrementally from the reference point on. Refer to the Xilinx documentation for the commands.

   For ECO updates, look for commands like *disconnect_net* and *connect_net,* which are commonly used for ECO changes. Also use the Vivado schematic and search tools.

5. To use the Vivado register readback feature, see the procedure described in Using Global State Visibility (GSV).

Customizing Scripts to Place and Route a Single-FPGA Design

This procedure describes how to use customized scripts to directly place and route a single-FPGA design. For information about using customized scripts in a multi-FPGA design, see Customizing Synthesis Scripts.

1. Start with a mapped design.
2. Create a Tcl file with the custom options you want to run Vivado.

   The options in this custom options file override the settings in the *run_vivado_haps.tcl* file. For example, you can change the default settings to generate a Verilog netlist after place and route and not write out a bit file:

   ```
   set write_post_par_verilog "1"
    set write_bitstream_enable "0"
   ```

3. Specify the options file when you export the database to Vivado.

   ```
   export vivado -vivado_option_file custom_export_Vivado.tcl
   ```

Feedback

This example exports the options file called *custom_export_Vivado.tcl*to the top-level working directory. It also sets a variable in the *run_vivado_haps.tcl* file, so that the custom options file is sourced and the custom settings in this file are used to run Vivado. You can now run place and route as usual.

4. For customized par exploration scripts, use *export vivado* with the *-script*argument instead of *-vivado_option_file*. For example:

   ```
   export vivado -script par_explore1.tcl
   ```

Using Vivado Flags and Options in Run Vivado HAPS Tcl Script

You can use Vivado flags and options in custom *run_vivado_haps.tcl* scripts during the initial exploration of a design to reduce the runtime. Most of these switches are reported by the Vivado in the log. These switches help in implementing a script-based flow without adding the switches to individual Vivado runs.

For example, you can add the Vivado switch *route.enableHoldExpnBailout* in a custom script *run_vivado_haps.tcl*:

```
catch {set_param route.enableHoldExpnBailout 1}
```

For details on the Vivado flags and options, see the Vivado documentation.

Importing Place and Route Results for Back-annotation

After running place and route, you can back-annotate the results from the Vivado database into a mapped state in the prototyping database.

1. Place and route the design. See or Running Place and Route.
2. Open the mapped database state from where you started place and route.
3. Execute the following command to set up the location of the place-and-route results for the mapped design state.

   ```
   database apply_state -import_vivado
   ```

4. To back-annotate place and route information for resynthesis or analysis in the prototyping tool, use the *database apply_state -backannotate* command.

   This command imports the Vivado net delay values so that you can run more accurate timing analysis from the mapped state. It creates a new database with back-annotated place-and-route information.

   For information about resynthesis, see Resynthesizing the Design After Place and Route.

5. To use back-annotated information as the basis for system-level timing analysis (SLTA), do the following:
   - For each FPGA, back-annotate the place-and-route results by running the*database apply_state -import_vivado* and *-backannotate* commands as described in the preceding steps.
   - From the system generate stage at the top level, run *report timing -generate -mode slta* command to generate a system-level timing report that is based on the back-annotated information for the FPGAs. See Running SLTA for details.

Note:

You can check *ba.log* for additional information.

Resynthesizing the Design After Place and Route

If your design continues to have poor slack numbers or still fails to route after you have tried various alleviation techniques, you can run post-place-and-route resynthesis to generate a new netlist. This resynthesis run is based on information from place and route, and can improve routability and help meet timing. The run preserves most of the design and only makes changes to congested hotspot regions and timing paths with violations, so it takes only a fraction of the normal synthesis runtime.

1. Run place and route with the *export vivado* and *launch vivado* commands.

   For more about placing and routing the design, see or Running Place and Route.

   For post-place and route resynthesis, you can either import the results back and then resynthesize (step 2) or resynthesize without importing, based on a specified place-and-route *dcp* file (step 3).

2. To resynthesize based on imported information, do the following:
   - Start with the mapped database state from where you exported the files for Vivado in step 1, and import the place-and-route database back to the same state, using the *import vivado* command.

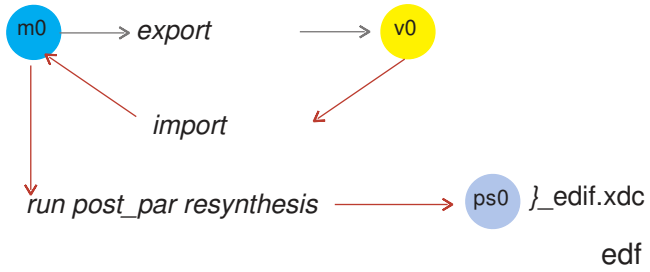
     The import operation imports the following netlist, constraint, and script files from Vivado:

design`_edif.xdc`	*design*`.edf`	`run_vivado.tcl`
design`.dcp` or post_place.dcp	*design*`.dly`	

   - Resynthesize the design with the *run post_par resynthesis* command.

     ```
     run postpar_resynthesis
     ```

Feedback



You do not need to specify any arguments to the *run post_par resynthesis* command; the tool uses the imported netlist and *dcp* file to generate a re-mapped netlist. The command writes out new EDIF and constraint files, and a new Vivado Tcl run script. It also preserves the original SRD, EDF, and XDC files.

The command also generates a new post-place-and route state with a new netlist and constraints. The default name for the state is *PS0*. The *PS0* state is like a mapped state, and you can look at reports, export files, or run PAR Explorer as usual. The only exception is that you cannot run Formality from this state.

3. Resynthesize with the *run post_par resynthesis* command and the appropriate *dcp* file. You do not have to import the Vivado files back.
   - Start from the mapped state from which you exported files for place and route.
   - If the problem is that the design does not meet timing, specify the post-route *dcp* file to resynthesize the design:

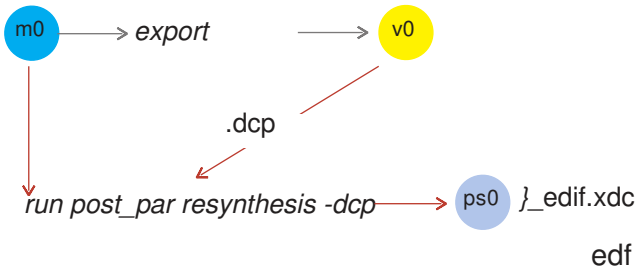
     ```
     run postpar_resynthesis -dcp <path>/design.dcp
     ```

   - If the design failed to route, use the post-place *dcp* file for resynthesis:

     ```
     run postpar_resynthesis -dcp <path>/post_place.dcp
     ```

Feedback

The command writes out new EDIF and constraint files, and a new Vivado Tcl run script. It also preserves the original SRD, EDF, and XDC files. It also generates a new post-place and route mapped state, with the default name *PS0*.



4. Check that resynthesis was successful in the *pr_resyn.log* file.
5. To place and route again after resynthesizing, export the files needed to an output directory with the *export vivado -pathpath* command.

   This command exports the *edif* and *xdc* files needed for the new place and route run.
6. To run the Vivado incremental flow after resynthesis, do the following:
   - Edit the *run_vivado.tcl* script and change the flow to *Incremental* (*set Flow "Incremental";)*.
   - Rename *design.dcp* to *design_pr_resyn.dcp* to ensure that Vivado reads the *dcp* file.
   - Run the Vivado incremental flow.

Analyzing Routing Congestion

Congestion analysis is an iterative process. Synthesize the design, place and route the design, then analyze the impact of congestion. Then, make changes and iterate through the process again, including re-partitioning, if needed.

For more information, see:

- Minimizing Routing Congestion
- Troubleshooting Routing Congestion
- Minimizing SLL Congestion in Vivado

Feedback

Minimizing Routing Congestion

It is recommended to minimize routing congestion in your prototype starting from the most recent design phases and working backwards, to avoid redesign.

Minimizing routing congestion during place-and-route phase

- Spread the design logic across the FPGAs and set a 60% utilization rate for the first pass.
- Check that the timing constraints are not too tight, as this can lead to sub-optimal placement.
- Check the place-and-route strategy. You can try different strategies in parallel.
- Use the congestion-specific techniques listed in the following table:

General Techniques	Design-Specific Techniques
Congestion_BalanceSLLs place_design -directive SSI_BalanceSLLs phys_opt_design route_design -directive NoTimingRelaxation	*Congestion_SpreadLogic_high place_design -directive SpreadLogic_high phys_opt_design -directive AggressiveFanoutOpt route_design -directive MoreGlobaliterations*
Congestion_BalanceSLRs place_design -directive SSI_BalanceSLRs phys_opt_design route_design	*Congestion_SpreadLogic_medium place_design -directive SpreadLogic_medium phys_opt_design -directive AggressiveFanoutOpt route_design -directive HigherDelayCost*
Congestion_CompressSLRs place_design -directive SSI_HighUtilSLRs phys_opt_design route_design	*Congestion_SpreadLogicSLLs place_design -directive SSI_SpreadSLLs phys_opt_design route_design -directive NoTimingRelaxation*

For additional information about reducing congestion at this stage, see Minimizing Congestion During Place-and-Route Phase.

For information about running place and route, see Running Place and Route.

Minimizing routing congestion during compile, pre-map, and map phases

1. Run with the *synthesis_strategy routability* option. This option reduces the competition for routing resources by mapping LUT6 to LUT4 in high-congestion areas. This single technique might be sufficient to fix congestion issues in designs with a relatively low number of node overlaps.
2. Check for high resource utilization after mapping. Aim to use less than 70% of LUT resources and less than 60% fo BRAMs and DSPs.

3. Check that logic packing is not using a high number of LUT6 primitives as compared to LUT5s, LUT4s, LUT3s or LUT2s. Make sure that the number of control sets is below 5000.
4. Avoid gated clocks on high-fanout clocks.
5. Use the *syn_insert_buffer* attribute to insert buffers on high-fanout nets.
6. Do not over-constrain the design. Aggressive timing constraints with negative slacks that are 10% over the period can cause congestion.

   For additional information about minimizing congestion at this stage, see Minimizing Congestion During the Map Phase.

Troubleshooting Routing Congestion

Routing congestion factors are design-dependent and you might have to use a combination of solutions to address the issues you find. Watch for typical indications throughout the design cycle, such as timing violations, high utilization numbers for LUTs, BRAMs, muxes and DSP, wide buses, a large number of interconnections between modules, many high-fanout nets, overutilization of SLRs and and long place-and-route runtimes.

Identify congestion early in the design cycle and fix the underlying issues. However, if congestion is only identified after place and route, work backwards through the flow and check whether you can minimize congestion at that design stage, to minimize re-design.

To minimize routing congestion during different design stages, see:

- Minimizing Congestion During Place-and-Route Phase
- Minimizing Congestion During the Map Phase

Minimizing Congestion During Place-and-Route Phase

1. Generate the congestion map and visually check it.
   - Select *File->Open Check Point* command to open the *dcp* file.
   - Check both horizontal and vertical congestion by right-clicking and selecting *Metric->Vertical/Horizontal routing congestion per CLB*.
2. Check for congestion errors.
   - Run *run_route_status* to get a full status of the routing, and then check the log file.
   - Vivado error: ERROR: [Route 35-162] 1868 signals failed to route due to routing congestion.

- Router Utilization Summary in log file Check the extent of the congestion (*Number of Unrouted Nets* and *Number of Node Overlaps*). Ideally, there should not be any node overlaps. The more node overlaps there are, the more congested the design.

  If overall routing utilization values are not high, check for local congestion. You can also check tile areas to identify local congestion.

- *vivado.log* file: Look for *Verification failed* messages, and *CRITICAL_WARNING*s in the *route_design* section.

3. Check timing.
   - Make sure that the paths being optimized are truly critical paths.
   - Check that all timing exceptions and I/O delays are properly constrained with constraints like *set_multicycle_path*, *set_false_path*, and *set_datapath_delay*.
   - Check routability by running place and route with relaxed constraints. Either relax the constraints in the *_edf.xdc* file and run place and route, or run place and route without this file. If the design then routes with this best-case scenario, re-introduce the constraints gradually and rerun. If it does not work, relax the timing constraints. You can also try alternate placements to improve routability.
4. Check the complexity report.
   - Generate an on-demand complexity report with the *report_design_analysis -complexity*. This report reports interconnect density per hierarchy as an exponent value.

     ```
     report_design_analysis -complexity -hierarchical_depth numLevels
     -name complexity -file name.rpt
     ```

   - Check that the interconnect density exponent value is less than 0.65.
   - Check number of ports and number of cells per hierarchy for density.
   - Check LUT, BRAM, DSP, and MUXF7 and MUXF8 usage.
5. Check SLR and SLL usage.

Feedback

- Check that SLR (Super Logic Region) usage is balanced. To fix congestion caused by unbalanced SLR usage, you might have to modify I/O placement or block constraints.

  If you specify advanced mode for the synthesis strategy in the prototyping tool, you can get estimates of the usage, as shown in the report below:

```
Estimated SLR Usage Report:
===========================
SLR   LUT       %  DFF      %  BRAM    %  DSP    %  IO     %
-------------------------------------------------------------------
SLR-0 0          0% 1        0% 0       0% 0      0% 146    9%
SLR-1 287831    94% 69030   11% 5       1% 2      0% 290   17%
SLR-2 233489    76% 17975    3% 0       0% 1      0% 326   19%
SLR-3 0          0% 0        0% 0       0% 0      0% 232   14%
===================================================================
```

- SLL (Super Long Lines) are connections between SLR regions. Cut constraint warnings about SLLs indicate that reruns might cause routing issues. High interconnectivity and the overuse of SLLs can cause congestion. You can check for SLL usage estimates in the post-map file for the single-FPGA level. Correct over-utilized SLLs by using a new I/O placement .

  If you enable the advanced synthesis option in the prototyping tool, you can get early estimates of possible problems, reported as follows:

```
Estimated SLL Crossing Report:
==============================
              Available  SLLs
Crossing           SLLs  Used  Usage
----------------------------------
SLR-0<->SLR-1      13720   135  0.01
SLR-1<->SLR-2      13720  8492  0.62
SLR-2<->SLR-3      13720    25  0.00
==================================
```

6. Redistribute connectors to balance congestion.
   - If you check the congestion map and find some SLRs are under-utilized, balance connectors across SLRs. I/O placement affects connectivity because connected logic moves with the I/O.
   - Fill the connectors across a single SLR, rather than vertically using multiple SLRs on just one side.

- Localize the connectors instead of spreading them out, as long as this does not interfere with device utilization.
-

7. Try an alternative placement solution.

   Use *Auto SPO* to run different combinations of strategies automatically. Use distributed synthesis to run them in parallel.

   If the design remains congested, resynthesize the design to get a more routable netlist.

Minimizing Congestion During the Map Phase

1. Run post-P&R resynthesis.

   This run uses a Xilinx *dcp* checkpoint file, and does not rerun all the synthesis steps. See Resynthesizing the Design After Place and Route for details. Use this fast iteration for routable but congested designs, or placed designs that do not route.

2. Check timing.
   - Make sure that the paths being optimized are truly critical paths.
   - Check that all timing exceptions and I/O delays are properly constrained.
   - Check the log file stages for constraint checker messages, high negative slack, or other timing-related warnings.
3. Check the design setup for areas that can be improved:
   - Manually check that all clocks are being routed on global resources, not local ones. Use HDL Analyst to look for buffers in the mapped view. Also check that the number of BUFGs and clocks are as expected, with more BUFGs than clocks.
   - Check the gated clock conversion report in the post-map log file to make sure that there are no gated clocks left unconverted.
   - Check for excessive replication. If necessary, increase the global fanout threshold or use the default (10000).
4. Check for high macro usage, and balance resource utilization.

   Check these attribute and directive settings and make sure that they are being used effectively and that there are no unnecessary restrictions on optimizations.

Attribute	How to Alleviate Congestion
syn_ramstyle	Map RAMs to BRAMs Remove glue logic by specifying *no_rw_check*

Feedback

Attribute	How to Alleviate Congestion
syn_dspstyle	Map arithmetic operators to DSPs
syn_srlstyle	Map SRLs to registers
syn_hier *syn_noclockbuf* *syn_preserve* *syn_keep*	Allow free optimizations by removing these attributes when they are not needed. Use the *Find in files*option to search the RTL, *cdc*, and *fdc* files and remove the ones that are not needed.

5. Turn off LUT6 packing to increase routing density.

   ```
   option set enable_prepacking 0
   ```

6. For local congestion, try a different synthesis flow:
   - If LUT utilization is over 60% and high performance is required, use the synthesis strategy advanced mode, which takes congestion into account when optimizing:

     ```
     option set synthesis_strategy advanced
     ```

   - If LUT utilization is over 60% and some timing QoR can be sacrificed for improved runtime, use the synthesis strategy routability mode, which takes congestion into account when optimizing:

     ```
     option set synthesis_strategy routability
     ```

   - If LUT utilization is less than 50% and high performance is not required, use the synthesis strategy fast mode, which reduces the number of optimizations:

     ```
     option set synthesis_strategy fast
     ```

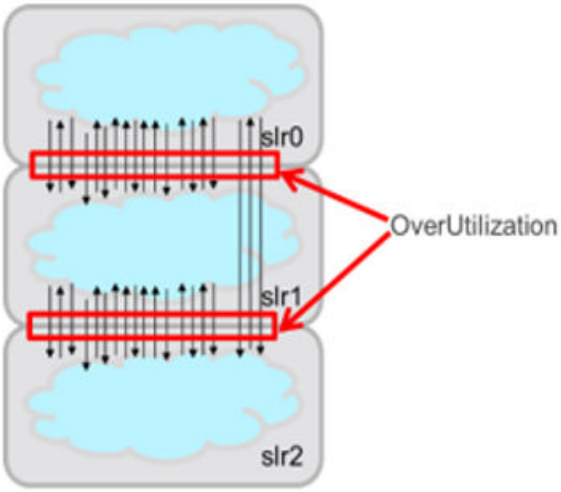
7. Resynthesize the design and rerun place and route.

Minimizing SLL Congestion in Vivado

With HAPS-100 designs, you can use TDM in Vivado to minimize congestion caused by high SLL demand. This lets you place and route designs that would otherwise fail with the following Vivado error message:

```
ERROR: [Place 30-99] Placer failed with error: 'SSI partitioning failed!'
```

Feedback



1. If you encounter the Vivado error message about SSI partitioning failing, set the environment variable *VIVADO_SLL_TDM_ENABLE* in the ProtoCompiler tool. To enable the variable use *1* and to disable the variable use *0*
2. To selectively allow or prohibit clock domain nets from being considered for TDM, set the appropriate attributes in the FDC file:

   ```
   define_attribute {c:clk} {.sll_tdm_allow} 1
   define_attribute {c:clkin2} {.sll_tdm_prohibit} 1
   ```

   Note that by default all DUT clocks are considered for SLL TDM optimization in Vivado. so use the attributes if you want to only run TDM on specific nets.
3. Run Vivado using the ProtoCompiler-generated Vivado script.

   This procedure might not work with custom scripts or scripts generated with versions before R-2020.12.
4. Check for messages after the script runs.

   The scripts provide timing signoff for TDM paths.

   - If you see the bit file is not written out, check that the TDM fast clock is not failing.
   - If you see the following critical warning, check that there are no missing constraints between the DUT and TDM.

     SLL Opt might cause timing violation in the DUT clocks. Bit file is generated. However, the design might only be functional at runtime when clock frequencies are adjusted.

Running Synthesis Using Tcl Batch Script

You can create a Tcl batch script to run compile, pre-map and map operations from the command line.

1. Create a Tcl batch script. See Creating a Tcl Script for details.
2. Save the file with a *tcl* extension to the directory that contains your source files.
3. From a command prompt, go to the directory with the files and type one of the following as appropriate:

   ```
   protocompiler -batch <Tcl_script>.tcl
   ```

   The tool executes the script, running in batch mode. The results and errors are written to the log file. The tool also reports success and failure return codes.
4. Check for errors.
   - For source file or Tcl script errors, check the standard output for messages. On Linux systems, this is generally the monitor in addition to the *stdout.log* file; on Windows systems, it is the *stdout.log/protocompiler.log* file in the directory where it was run.
   - For run errors, check the log file in the working directory.

Table 6 Possible Error Codes

Error Code and Description
0 - OK
2 - Logical error
3 - Startup failure
4 - Licensing failure.
5 - Batch mode not available (Not applicable)
6 - Duplicate-user error (server setups that do not permit multiple runs)
7 - Project-load error (Not applicable)
8 - Command-line error
9 - Tcl script error
20 - Graphic resource error
21 - Tcl initialization error

Error Code and Description
22 - Job configuration error
23 - Parts error (Not applicable)
24 - Product configuration error
25 - Multiple top levels

For related information, see:

- Queuing Licenses
- Releasing the Tool License

Queuing Licenses

A common problem when running in batch mode is that the run fails because all of the available licenses are in use. License queuing allows a batch run to wait for the next available license when a license is on the server but not immediately available. You can specify either blocking or non-blocking queuing.

Queuing Style	Tool Behavior
Blocking-style queuing	Waits until a license becomes available
Non-blocking-style queuing	Waits the specified length of time for a license to become available

For more information, see:

Queuing Synopsys DesignWare IP Licenses

In batch mode, the synthesis tool waits for an available IP license. When no license is available, the synthesis tool waits for either a specified time period or indefinitely. You can queue DesignWare IP licenses by adding a command line parameter to the tool invocation command.

The procedure to queue DesignWare licenses is given below; refer to ProtoCompiler Startup Commands for details about the syntax.

1. Take the following points into consideration:
   - The queuing mechanism does not take precedence over other Synopsys products such as the Design Compiler tool, and operates independently from their license queues.
   - Requested Synopsys DesignWare IP licenses are queued when the license is not immediately available.
2. To enable DesignWare IP license queuing in batch mode, use *-ip_license_wait* along with *-batch*. For example:

   ```
   protocompiler -batch -ip_license_wait waitTime
   ```

   The following examples illustrate command usage:

   waitTime is the number of seconds to wait for a license. If you specify 1, the synthesis tool waits indefinitely for the requested IP licenses.

   ```
   protocompiler -batch -ip_license_wait 60 myScript.tcl
   ```

   If an IP license is still not available when the wait period ends, the tool continues processing using any available license. An IP block without a requested license is processed as an error or a black box, if the*option set -dw_stop_on_nolic* command has a value of 0.

Releasing the Tool License

When invoking a third-party place-and-route tool from the Synopsys tool, the tool lets you release the license and run place and route in batch mode.

1. To release the FPGA license, execute the following command:

   ```
   toolName -batch -license_release
   ```

2. The tool checks all prototyping tool licenses immediately after the place-and-route job is launched. When licenses are released, the following message displays:

   ```
   Exiting session due to -license_release option
   ```

Working with Tcl Scripts and Commands

The tool uses extensions to the popular Tcl (Tool Command Language) scripting language to control synthesis and for constraint files.

Feedback

1. To get help on Tcl syntax, do any of the following:
   - Refer to the online help (*Help->Tcl Help*) for general information about Tcl syntax.
   - Refer to the *Command Reference Manual* for information about the run commands.
   - Enter *help* * in the tool window for a list of all the tool Tcl commands.
   - Enter *help commandName* in the Tcl window to see the syntax for an individual command.
2. To run a Tcl script, do the following:
   - Create a Tcl script. Refer to Creating a Tcl Script.
   - Run the Tcl script by entering *source Tcl_scriptfile* in the tool window.

   The tool runs the selected script by executing each command in sequence. For more information about Tcl scripts, refer to the following sections.

See the following for more information:

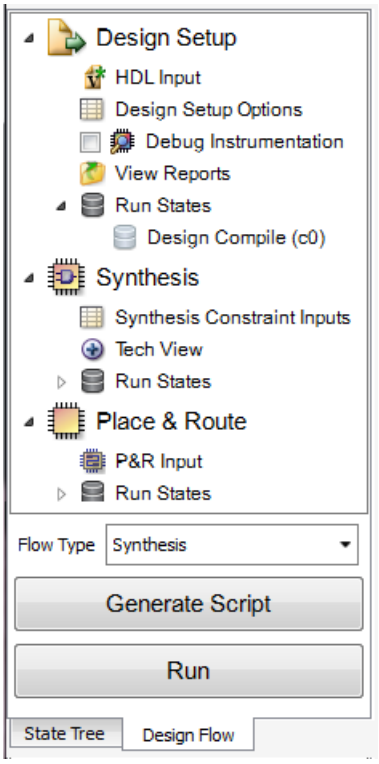
- Generating a Tcl Script from the GUI
- Creating a Tcl Script
- Configuring Maximum Number of Parallel Jobs

Generating a Tcl Script from the GUI

The Design Flow tab in the GUI provides a convenient interface where you can generate a Tcl script based on the parameters you provide, instead of manually creating a script.

Feedback

1. Create a root database.
2. Select the *Design Flow* tab (lower-right of the tool window) and do the following:
   - Click the first task (*HDL Input)*, and add the source files in the tab that opens on the right side of the window. It is the standard GUI interface for entering source files.



   - Similarly, walk through each of the steps in the order shown in the panel, adding options and constraints, and specifying *run* commands (Run States) as needed.
3. To generate a script, click *Generate Script* at the bottom of the panel.

   The tool generates a Tcl script in the working directory, based on the options you set. It also displays the script in a tab on the right side of the tool window. Review the script.
4. Click *Run* to run the script.

Creating a Tcl Script

Tcl scripts are text files with a `tcl` extension. You can use the graphic user interface to help you create a Tcl script. Interactive commands that you use actually execute Tcl commands, which are displayed in the Tcl window as they are run. You can copy the command text and paste it into a text file that you build to run as a Tcl script.

The following procedure covers general guidelines for creating a synthesis script:

1. Use a text file editor or select *File->New*, click the *Tcl Script* option, and type a name for your Tcl script.
2. Start the script by specifying the *database* command.
3. Add files using the *-src or -srcfilelist* argument to the*run compile* or *run pre_instrument* commands.

   The files are added to their appropriate directories based on their file name extensions. Make sure the top-level file is last in the file list.
4. Add attributes and constraints as needed in *cdc* and *fdc* files.
5. Set other design synthesis controls with the *option set* command.
6. Include *run* commands to run through the various design phases.

   For example: *run compile, run pre_map, run map*.
7. Check the script syntax.
   - Check case (Tcl commands are case-sensitive).
   - Start all comments with a hash mark (#).
   - Always use a forward slash (/) in directory and path names, even on the Windows platform.

Configuring Maximum Number of Parallel Jobs

Some operations, like distributed processing, run jobs in parallel if there are licenses available. You can set the maximum number of parallel jobs by specifying the *max_parallel_jobs* variable as a command argument or by setting an option either directly from the Tcl window or in a Tcl file.

With multiple definitions, the option setting takes precedence over the command argument. A setting defined later in the flow takes precedence over one that was defined

earlier. Details of the different methods to define the number of parallel jobs are described below:

1. To specify the maximum number of parallel jobs using the *option* command, do the following:
   - Use this syntax to specify *max_parallel_jobs*with the *option*Tcl command:

     ```
     option set -max_parallel_jobsvalue
     ```

     Set the value based on the number of licenses you have available. With each license, you can run four jobs in parallel.
   - Determine how you are going to issue the command. You can type it directly in the Tcl window or include it in a Tcl script or a Tcl file, like an options or setup file. If you specify it in a Tcl file, you must source the file. If you specify it in the Tcl window, the tool does not save the value, and it is lost when you end the current session.

     The tool applies the value globally to the design at the system level and does not apply to the individual FPGA runs. The maximum number of parallel jobs remains in effect until you specify a new value. When you set a new value, it takes effect immediately, going forward. For example, if you defined a value in a Tcl options file when you first compiled, but then typed in a value in the Tcl window at the map stage, the tool uses the value typed in the Tcl window going forward.
2. To set the number of parallel jobs as a command argument when you launch the tool, do the following:
   - Specify it as an argument when you first start the tool from the command line:

     ```
     protocompiler -max_parallel_jobs 8
     ```

     For details on the number of parallel jobs you can run with each license, see the HAPS ProtoCompiler Licensing Model section.
   - ```
     launch protocompiler -max_parallel_jobs 8 -script script1.tcl
      -max_parallel_jobs 10 -script script2.tcl
     ```

     This command is used to launch a synthesis script with 8 jobs and a seperate synthesis script with 10 jobs.

   The number of jobs specified in a command argument can be overridden at any time by using the *option set* command to specify a new value.

Preparing to Run on the Hardware

Bit files are generated for the FPGAs after place and route completes.

Take the full output of export_runtime command to configure hRun.
     ```

Feedback

HSTDM training is automated in the `setup configure` command in hRun.

- Exporting Files for Runtime
- Running HAPS Hardware Diagnostics
- Running Data Expansion in a Multi-User Setup

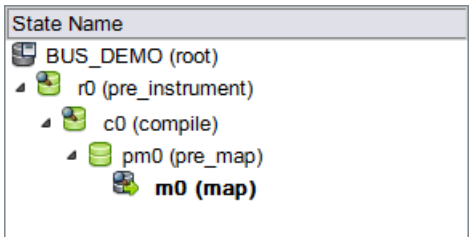
Exporting Files for Runtime

Once place and route is complete, export the files so that they can be used by the hRun. The following procedure describes how to set up the design for use with the hardware:

1. Start from the mapped database state of an FPGA after it has been placed and routed.

   The design must be properly mapped to the HAPS hardware. See Mapping to Hardware for more information, and consult the relevant HAPS system documentation.

   If needed, use the appropriate *database* commands to go to the mapped database state. See Loading and Navigating Database States for more information.

2. Ensure that all associated files, like IP files, are part of the design to be exported.
   - Run the *import vivado vivadoDir* command and specify the name of the Vivado directory where place and route was run.
   - If you skip this step, you can get an error message about missing files, if all the associated files are not imported back into the mapped state.
3. Export the files.
   - Specify this command to export the files:

     ```
     export runtime -path outputDirectory
     ```

     This command creates a directory with the files needed, including the bit file generated after the place and route. It prints messages about the files that are exported in the console.

- If you manually copy the exported database to another location, you must copy all exported files. This includes the database file (e.g., *.rtdb), encrypted files, which might be required for successful bring-up.
- You can now use the exported files to debug the product.

Running HAPS Hardware Diagnostics

You can run diagnostics on the HAPS hardware from the runtime environment using hUtils. For more information, see the *hUtils* documentation in *SupportNet*.

Running Data Expansion in a Multi-User Setup

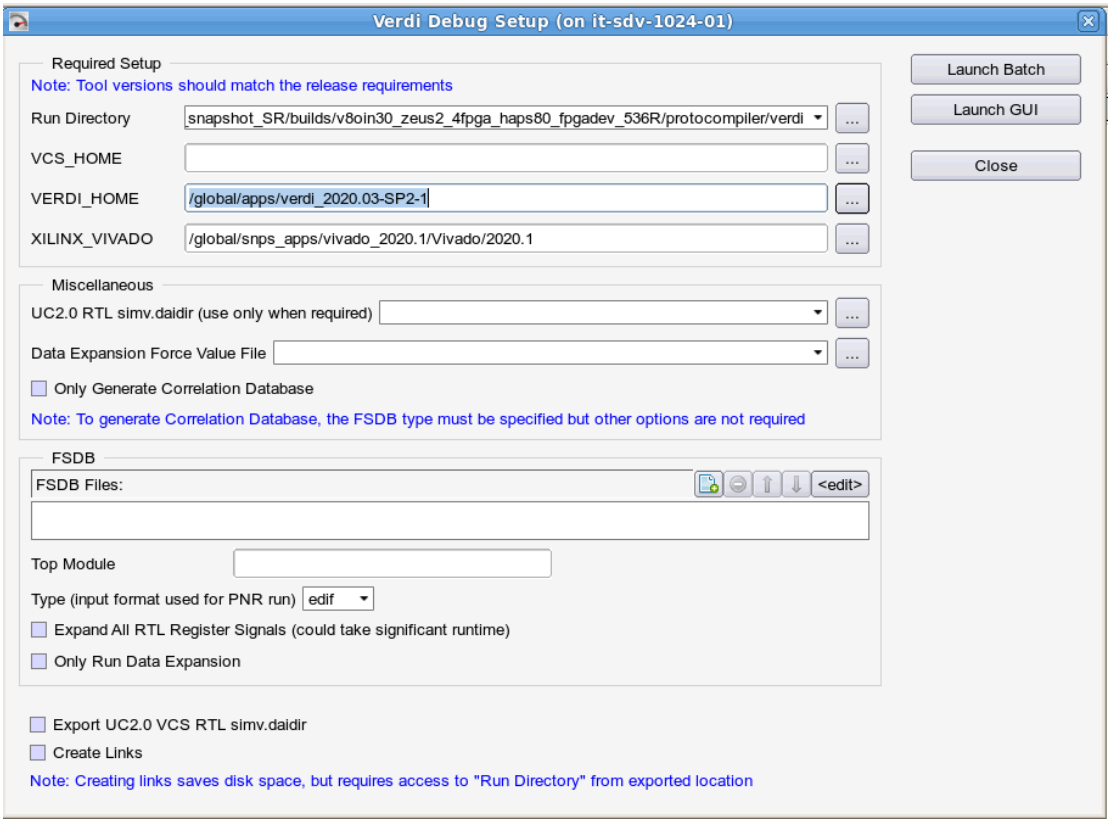
You can run data expansion (DE) in large-scale multi-user setup, where one host machine is connected to multiple HAPS systems and multiple users access these systems from various locations. The correlation database (CRDB) and the data expansion (DE) setup are exported to remote machines.

Feedback

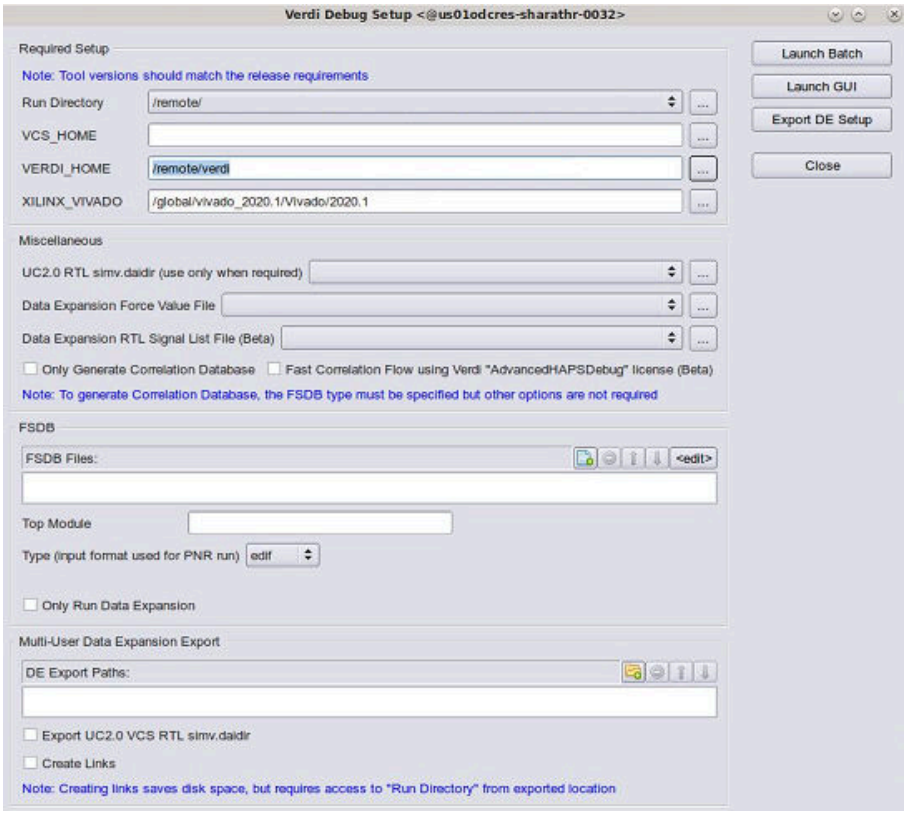
► ProtoCompiler Primary user:

◦ Generates CRDB using the ProtoCompiler tool:

Run *Launch Batch* with *Only Generate Correlation Database* selected.



◦ Adds the directory paths to the *Multi-User Data Expansion Export* using hRun. The setup and data for multiple users are exported to this path.



- Exports the DE setup to the given paths using the *Export DE Setup* button.

7

Instrumenting and Debugging Designs

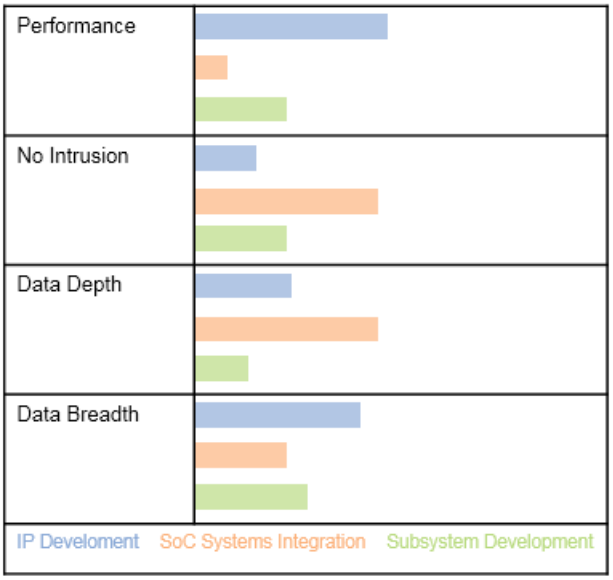
HAPS® ProtoCompiler includes functionality for instrumenting the design and then debugging the RTL. RTL debugging ensures functional correctness that a post-synthesis debugger cannot guarantee. This is because synthesis optimizations cause gate-level netlists to significantly differ from the functional RTL description, making it hard to trace back bugs to the RTL.

The following sections provide details about the instrumentation and debugging flow.

- Debug Factors
- HAPS ProtoCompiler Debug Functionality
- The Instrumentor-Debugger Flow
- Instrumenting the Design for Debug
- Verifying UC Designs
- Running the RTL Debugger
- Working with Deep Trace Debug
- RTL Data Expansion Using Verdi for Instrumented Signals
- Using BRAM for Debugging
- Running Incremental Debug for ECOs
- Using Global State Visibility (GSV)
- GSV or eGSV Flow
- Fast Track Debug
- Enabling Clock Selection With DTD/Readback
- Integrated Debugging with Verdi Software
- Using the Verdi Flow with a UC Design
- Configuring Dynamic Force

Debug Factors

There is a wide variance in the relative importance of debug factors, according to the type of prototype development. For example, IP subsystem development prioritizes data breadth (coverage) and performance over non-intrusiveness, whereas SoC system integration puts a premium on non-intrusiveness and depth of data coverage.



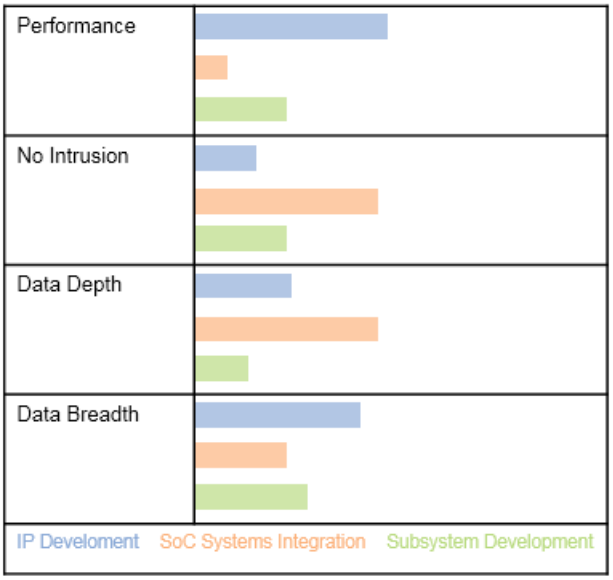
HAPS ProtoCompiler Debug Functionality

Debug factors

The relative importance of debug factors varies based on the type of prototype development. For example, IP subsystem development prioritizes data breadth (coverage) and performance over non-intrusiveness. SoC system integration puts a premium on non-intrusiveness and depth of data coverage.

Feedback

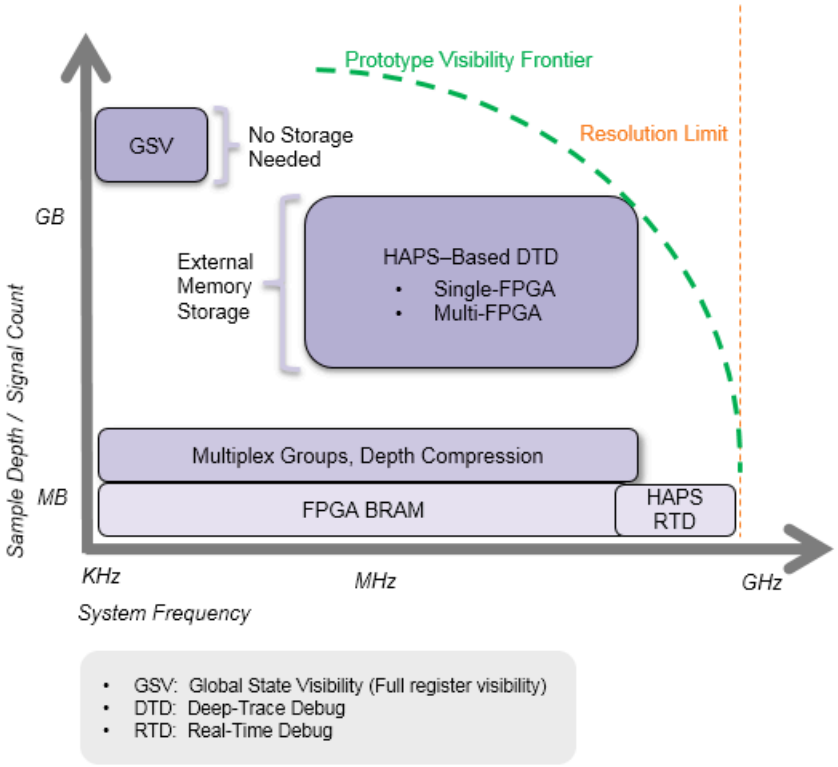
Figure 10 Debug Factors



The debugging approach is determined by performance requirements, debug visibility needs, and available hardware resources for the design.

The following figure is a summary of the debug functionality, based on two of the criteria: performance and sample depth.

Feedback



The following table lists the points of comparison for various debug methodologies and contains links to the information:

Interactive	Not Interactive
GSV: • Single/Multi-FPGA • Wide, emulator-like visibility • No memory storage Using Global State Visibility (GSV)	BRAM: • Single/Multi-FPGA • Broad coverage • Uses internal memory: BRAM resources on the FPGA • Fast Using BRAM for Debugging
	Built-in DTD: • Minimally intrusive • Large visibility window

Memory requirements affect the choice of debug methodology. Global state visibility (GSV) requires no memory storage. Deep-trace debug (DTD) schemes are memory-intensive, requiring either built-in system hardware resources or additional external memory.

Debug memory can be categorized as:

- Internal memory: Refers to BRAM memory that is part of the resources available in the FPGA modules.
- External memory: Refers to an additional memory module that is added to an FPGA for memory expansion. For example, daughter cards or a debug hub.

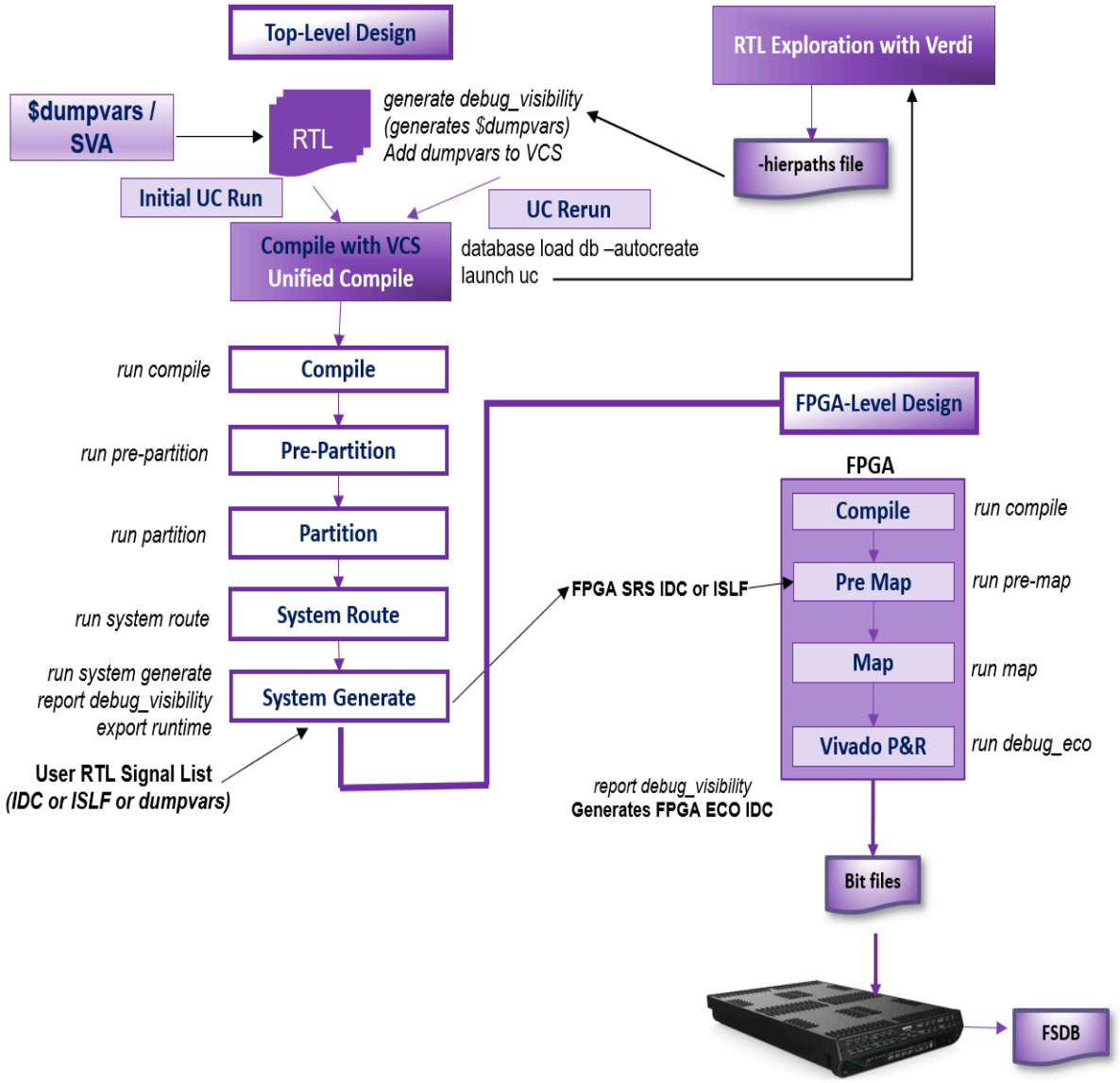
The Instrumentor-Debugger Flow

The Instrumentor-Debugger flow is implemented in two stages:

- Select the signals you want to observe at the HDL level. Add signal probes and triggers directly into the HDL source files.
- When you debug the design, check the instrumentation and functionally verify the RTL code.

The control of hardware triggers and annotation of captured data offers a comprehensive design view and easy probing. The built-in functionality adds speed and accuracy of results.

The following figure shows the various phases of this debug flow and the main commands associated with each stage.



The following steps provide an overview of the flow shown in the previous diagram. For syntax details for the commands mentioned, see Shell Command Reference.

1. Start the tool using the *protocompiler* command, and load the database.
2. Launch the instrumentor and instrument the design, as described in Instrumenting the Design for Debug.

Optionally, you can instrument the post-compile netlist.

1. Export the bit file with the *export runtime* command.
2. Debug the design as described in Running the RTL Debugger.
   - Launch the runtime executable.
   - Start debugging with hRun.
   - Debug the design.

Instrumenting the Design for Debug

The first step to prototyping with debug is to set up the design and mark signals for debugging. Do this by adding debug point logic, which is used to run on-chip debugging later, after programming the FPGA. It is a good practice to add debug point logic early in the cycle and reserve the resources needed, instead of doing it after the design is in place. Incorporating debug points as part of the initial phase reduces turnaround time. This step allows you to pipe clean the flow and identify logic, memory, timing and other limitations early in the cycle.

You can add instrumentation to a pre-compiled netlist or to one that has already been compiled. For more information, see the *HAPS Prototyping Debug Guide*.

For step-by-step procedures, see the following topics:

- Defining IICE Parameters
- Instrumenting Multiple Clocks in a Single IICE
- Instrumenting with Unified Compile

Defining IICE Parameters

IICE™ (Intelligent In-Circuit Emulator) units are blocks for probe and communication logic to trigger and sample the design. You can add multiple IICEs to handle multiple clock domains.

To configure IICE, see the following topics in the *HAPS Prototyping Debug Guide*:

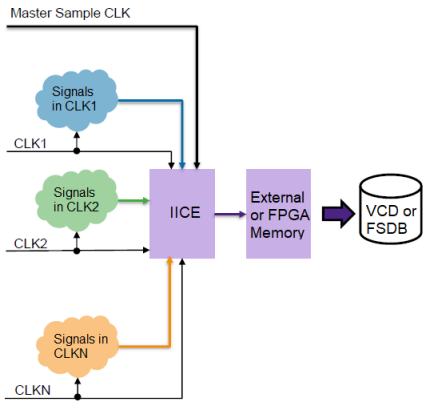
- Launching the Instrumentor
- Configuring Instrumentor Preferences
- Configuring IICE Units
- Configuring Individual IICE Parameters

- Configuring IICE Sampler Parameters for an IICE
- Configuring IICE Sample Clock
- Configuring IICE Controller and Other Options

Instrumenting Multiple Clocks in a Single IICE

Traditionally, separate Intelligent In-Circuit Emulators (IICE™) were defined for each clock domain, and the associated were instrumented and debugged in their respective IICEs. With multi-clock instrumentation, you can instrument multiple clock domain signals in a single IICE. To do this, you group signals in different clock groups.

The following block diagram provides a top-level overview of the multi-clock feature. The single IICE contains multiple clocks. Signals associated with different clock domains within the IICE can be triggered. This eliminates the need to use different IICEs for asynchronous clocks. This means that cross-triggering is not required and the instrumented signals from different clock groups can be viewed in a single waveform viewer session.



The following steps describe how to instrument multiple clocks in one IICE:

1. Create the database and pre-instrument the design.
2. Use FDC constraints to define all the clocks in the design.

- Define clocks with the *create_clock* command.If there are clocks derived from the design, define them in FDC using *create_generated_clock* command.
- Group the clocks as needed using the *set_clock_groups* command. The FDC file for the example shows the top-level asynchronous clocks grouped in the FDC file:

```
create_clock  -name {clk_40}  {p:clk_40} -period {25}
create_clock  -name {clk_30}  {p:clk_30} -period {33.33}
create_clock  -name {clk_20}  {p:clk_20} -period {50}
create_clock  -name {clk_10}  {p:clk_10} -period {100}

set_clock_groups -asynchronous -name clk40_grp1 -group {clk_40}
set_clock_groups -asynchronous -name clk30_grp2 -group {clk_30}
set_clock_groups -asynchronous -name clk20_grp3 -group {clk_20}
set_clock_groups -asynchronous -name clk10_grp4 -group {clk_10}
```

3. Open the instrumentor, and create a new IICE.
4. Specify the fastest clock in the design as the sample clock for that IICE.

   The sample clock (or fast clock) must always be active. Sample clocking must not be stopped once it is started. The sample clock must not be a gated clock. The sample clock must not be defined in clock group, as this is the fastest clock in the design.
5. Group multiple clocks using the *clock_group* command.

   Clocks in the same group are synchronous, so use different clock groups for asynchronous clocks. Once clock groups are defined for each clock domain, the signals can be instrumented in these clock groups. For example, if *sig_a* and *sig_b* signals are from two different clock domains, *clk_a* and *clk_b*, then the signals are instrumented as follows:

   ```
   clock_group add -name group1 -freq {30} -edge {pos} {/clk_a} -iice
   {IICE_0} clock_group add -name group2 -freq {50} -edge {pos} {/
   clk_b} -iice {IICE_0} signals add -iice {IICE_0} -clkgroup {group1}
   -silent -sample -trigger {/sig_a} signals add -iice {IICE_0} -clkgroup
   {group1} [-silent] -sample [-trigger] {/sig_a}
   ```
6. In the IDC file, instrument the signals in different clock domains by grouping them together in different clock groups.

Feedback

A sample of the IDC file is shown:

```
device jtagport umrbus
device umr_pipe 1
device prepare_incremental 1
iice new {IICE_0} -type regular
iice controller -iice {IICE_0} none
iice sampler -iice {IICE_0} -depth 2048
iice sampler -iice {IICE_0} -pipe 3

iice clock -iice {IICE_0} -edge positive {/clk_40}
signals add -iice {IICE_0} -silent -trigger -sample  {/cnt_64}
signals add -iice {IICE_0} -silent -trigger -sample  {/valid_40}

clock_group add  -iice {IICE_0} -name grp2_30  -freq {30}  -edge {POS} {/clk_30}
clock_group add  -iice {IICE_0} -name grp3_20  -freq {20}  -edge {POS} {/clk_20}
clock_group add  -iice {IICE_0} -name grp4_10  -freq {10}  -edge {POS} {/clk_10}

signals add  -clkgroup grp2_30 -iice {IICE_0} -silent -trigger -sample  {/cnt_128}
signals add  -clkgroup grp3_20 -iice {IICE_0} -silent -trigger -sample  {/cnt_192}
signals add  -clkgroup grp4_10 -iice {IICE_0} -silent -trigger -sample  {/cnt_256}

signals add -clkgroup grp2_30 -iice {IICE_0} -silent -trigger -sample  {/valid_30}
signals add -clkgroup grp3_20 -iice {IICE_0} -silent -trigger -sample  {/valid_20}
signals add -clkgroup grp4_10 -iice {IICE_0} -silent -trigger -sample  {/valid_10}
```

7. Continue with the normal design flow from compile to bit-file generation of individual FPGAs, and export the design for runtime debugging.

   You are now ready for the debug stage of the design, described in Debugging a Multi-Clock Design.

Instrumenting with Unified Compile

You can designate probe signals and instrument the RTL design with the UC flow:

- Using SystemVerilog assertions, or
- Using *$dumpvars*.

In addition to these RTL instrumentation techniques, you can add post-compile instrumentation.

The instrumented signals are managed through the Instrumentation-Debug Constraints file (IDC). The IDC file contains constraints for the instrumented signals and the Intelligent In-Circuit Emulator (IICE) which is the debug IP.

See the *HAPS Prototyping Debug Guide* for more information.

During the runtime phase, you can run the design on the HAPS hardware and debug the design. You can then use the Synopsys Verdi software to check the waveforms from the instrumented signals against a golden version.

Verifying UC Designs

UC designs are verified by using the Verdi waveform viewer to check results against a golden design. The Verdi software is used for designs with RTL instrumentation as well as non-instrumented designs using global state visibility (GSV).

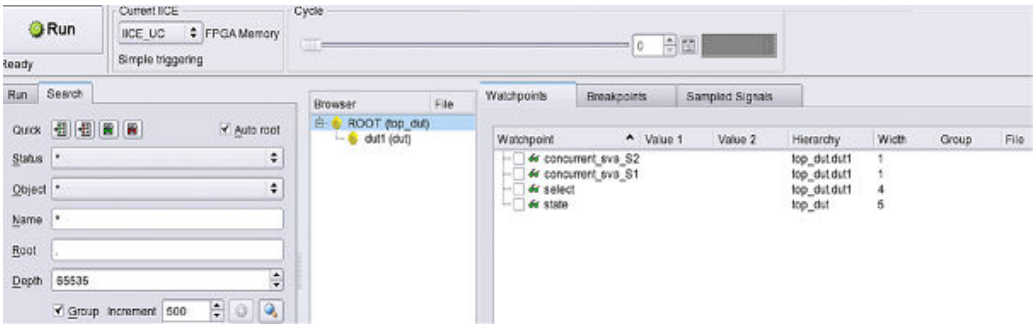
Feedback

See:

- Debugging Designs with SVA
- Running Incremental Unified Debug
- Using Verdi to Debug a UC Design

Debugging Designs with SVA

The tool lists all the synthesized assertions with SVA instance names that were passed through the IDC file, along with other debug signals.



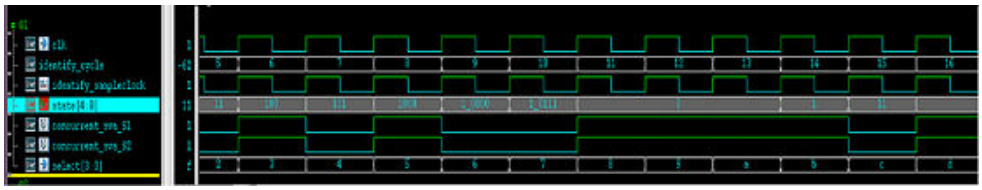
1. Enable the debug signal of the SVA instance using regular runtime commands to monitor assertion failures. For example:

   ```
   watch enable -language verilog {top_dut.dut1.concurrent_sva_S2} {1'b0} {1'b1}
   ```

2. Check for signal value changes in the waveform.

   When the assertion fails, the corresponding debug signal value changes from 0 to 1 in the waveform.

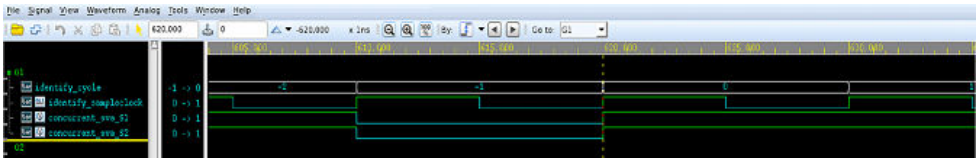
   If you evaluate the SVA property with respect to the negedge clock when the IICE sampling is at the posedge clock, or vice versa, the SVA samples appear one cycle earlier in the waveform. For example: `assert property (@(negedge clk) req##1 grant)`.



3. To filter SVAs in the waveform, use these commands:

   - `signals set_property -name show_in_waveform -value 0 [hierarchy find -all -maxdepth 100 -name "*" -stat "*" -type "signal" .]`
   - `signals set_property -name show_in_waveform -value 1 [hierarchy find -all -maxdepth 100 -name "*" -stat "*" -type "assert" .]`
   - `write fsdb`

     The following figure illustrates the filtered SVA signals.

     For information on using the Verdi integration flow, see Using Verdi to Debug a UC Design.

Running Incremental Unified Debug

Incremental debug lets you modify the instrumentation signals and rerun, without having to recompile the entire design. Incremental debug is a useful way to address these common scenarios:

- To remove instrumented signals in over-instrumentation scenarios.
- To modify the IICE settings and add new instrumentation to an IICE block.
- To change the instrumentation type. By default, all signals specified using *$dumpvars* are instrumented as *sample and trigger*. You can change the instrumentation to *sample only* and/or *trigger only*
- To add clock groups and mux groups for multi clock and mux set features.

The following procedure provides details on running incremental debug on RTL instrumentation:

1. Set this option to enable incremental debug: *option set incremental_debug 1.*
2. Compile the design, and then run the *export idc -pathvalue* command to export the IDC.

   You can only make instrumental modifications at the compile stage for incremental debug.
3. Modify the IICE settings of the exported IDC as needed.

You can adjust the sample depth, trigger type, etc. but you cannot change the IICE name. Note the following commands and syntax details:

- Delete instrumented signals with the *signals delete* command. For example:

```
signals delete -iice {IICE_UC} {top.inst1.sig1}
```

- To instrument new signals, use *signals add*: For example:

```
signals add -iice {IICE_UC} {top.inst1.temp}
```

- Hierarchical signal paths must start with the top-level module name.
- A dot (.) is used as the signal path hierarchy separator, not the slash (/).
- Signals in the generate scope must be specified with square brackets [] instead of parentheses (), because there is no need to specify the range for a signal in IDC.
- Signal path for VHDL does not need an architecture name.

  For example:

```
iice clock -iice {IICE_UC} -edge positive {top.clock_c}
signals add -iice {IICE_UC} -silent -trigger -sample {top.i1.i2.s1}
```

4. To incrementally update the design return to the compile stage and use the modified IDC to recompile the design:

```
database apply_state -idc <modified_idc.idc>
```

   This is an example of the sequence of commands:

```
option set incremental_debug 1
launch uc -utf run.utf -ucdb ucdb -v 2.0
run compile -ucdb ucdb -idc my_idc.idc
export idc -path unified_debug_idcs
database apply_state [-idc <modified_idc>] [-idclist
 modified_idc_list_file_path>]
```

5. To run incremental debug with post-compile instrumentation, do the following:
   - Run compile.
   - Edit the instrumentation in the generated IDC file.
   - Specify the updated IDC file when you run pre-partition or pre-map stages of the flow. Make sure to specify the correct IDC file. If you use a previous file with RTL instrumentation, you get an error.

   For information on using verdi integration flow, see Using Verdi to Debug a UC Design.

Feedback

Using Verdi to Debug a UC Design

The Verdi software is integrated with HAPS® ProtoCompiler to provide a seamless design debug environment to validate UC designs.

Use the Verdi tool in conjunction with the Global State Visibility (GSV) functionality. GSV leverages the Xilinx readback feature and provides a snapshot of all registers in a configured design at a particular clock cycle, thus offering full visibility. It does not require instrumentation and is instantly available. There are two GSV modes: legacy GSV and enhanced GSV (eGSV). The eGSV mode is implemented only when the design is instrumented but there are no other changes in the use model.

With this approach, you can perform Verdi data expansion using the FSDB that is generated from the GSV flow using the Vivado *.ll* files.

The application also checks the tool version compatibility and prints the message in the *novas.log* file. The tools prints an error if the versions are not matching.

```
######################
Current running Verdi version: U-2023.12
Verdi version used to generate CRDB: T-2022.06-SP1
HAPS VCS Version Data: VCS S-2021.09-SP2-3_Full64
Error: Current running Verdi version: (U-2023.12) does not match Verdi
 version used to generate CRDB: (T-2022.06-SP1)
*****************************************************************
        End Tool Version Compatibility Report
*****************************************************************
Error: Version compatibility check failed, DE Setup cannot be completed.
Info: DE setup fails.
######################
*****************************************************************
        Begin Tool Version Compatibility Report
*****************************************************************
Current running Verdi version: U-2023.12
Verdi version used to generate CRDB: T-2022.06-SP1
HAPS VCS Version Data: VCS S-2021.09-SP2-3_Full64
Warning: Current running Verdi version (U-2023.12) does not match Verdi
 version used to generate CRDB (T-2022.06-SP1)
*****************************************************************
        End Tool Version Compatibility Report
*****************************************************************
```

See these topics:

- Running the Verdi Flow For Designs with GSV
- Running the Verdi Flow For Designs with Instrumented/Probe Signals

Running the RTL Debugger

This section provides an overview on how to use the debugger. For more comprehensive information, refer to the help built into the debugger.

1. Set up the design for the debugger.
   - Verify that the hardware is set up correctly for the debug scheme you are using. See the following for details:
     - Verifying the Hardware Configuration for Deep Trace Debug
     - Working with Deep Trace Debug
   - Check that the runtime package is installed and that the runtime executable is available. This installation is required to run the debugger.
   - Instrument a design, and complete synthesis and place-and-route, as described in Instrumenting the Design for Debug.
2. Synthesize, place, and route the design, and then export the required design files by typing the following command at the system prompt in the ProtoCompiler tool:

   ```
   export runtime -path <path> -debug_export_sources
   ```

   The command creates a directory with all the files you need for debugging your design, including the bit file and a project file.

See:

- Debugging a Multi-Clock Design
- Varying Buffer Depth Dynamically for DDR3
- Verifying the Hardware Configuration for Deep Trace Debug
- Configuring CEL-Based Triggers

Debugging a Multi-Clock Design

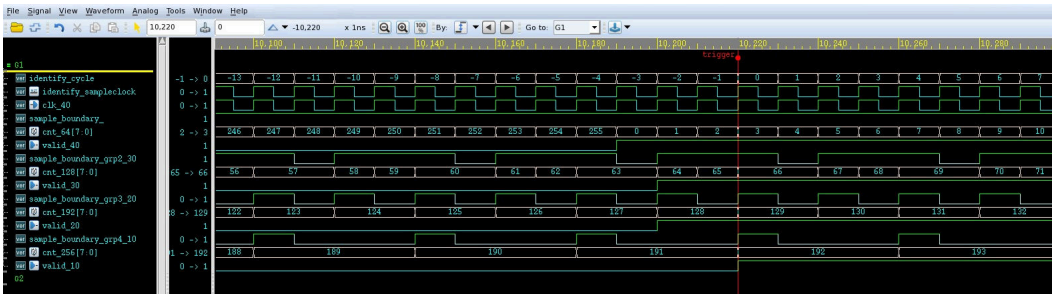
Once you have properly set up the clocks and synthesized the design (see Instrumenting Multiple Clocks in a Single IICE), you can debug the design.

1. Program the hardware using hRun, defining the frequencies and configuring bit files for each FPGA.
2. Debug the design using hRun and observe the captured waveforms.

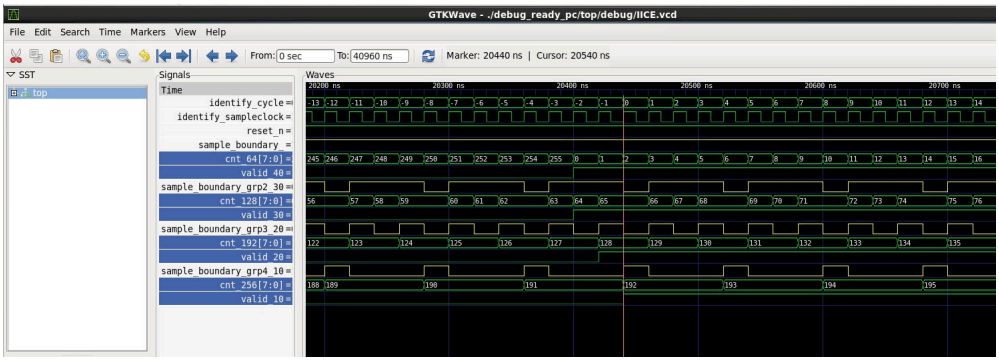
Feedback

- You can view multi-clock domain signal waveforms synchronously.
- View the sampled signals with reference to their *sample_boundaries*. See Sample Boundaries.

  The following figure shows the waveforms for *sample_boundaries* and their associated signals, captured in *FSDB* format. See FSDB Waveform Calculations for Clock Groups.



The next figure shows the waveforms for *sample_boundaries* and their associated signals, captured in *VCD* format:

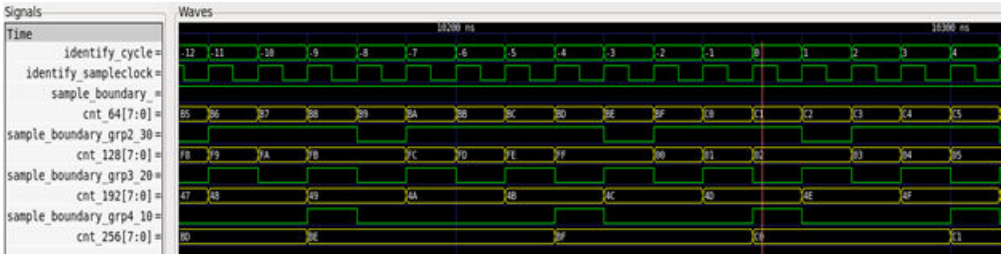


Sample Boundaries

Each clock domain has its own sample boundary. The sample data for signals in different clock groups is captured with reference to the *sample_boundary* for that clock domain.

The minimum resolution for the displayed waveform is the fast sample clock period, but the sub-sample clocks can be asynchronous to the fast sample clock. The following waveform example shows four sample boundary signals (in green) for four different clock domains. Data signals associated with the clock domain (shown in yellow) can be viewed with reference to their sample boundaries.

Feedback



The *sample_boundary** signals specify when the sampled data is valid. When the sample boundary is*1*or *high*, its associated data signal values are valid. And when the sample boundary is *0* or *low*, the signal values are not valid as the previous data values remain in place.

The software automatically creates FIFOs to buffer data that is captured from asynchronous clock domains, so that they can be displayed by the fast sample clock. The sample clock or fast clock cannot be a gated clock. Note that the sample clock must always be active. After it is started, the sample clock must not be stopped.

FSDB Waveform Calculations for Clock Groups

The FSDB waveform depends on the fastest clock frequency, which is the sampling clock frequency for the design. The remaining clock signals in the waveform are shown as de-skewed by n-clock cycles of their period.

The number of clock cycles by which the slower clock signals (*n*) are de-skewed is calculated using this formula:

```
Number of clock cycles to de-skew(n) = (Sampling Frequency/Slow clock
frequency) - 1
```

The figure shows waveforms for an example design where the sampling clock frequency is 40 MHz, and three additional clocks in the design run at 30 MHz, 20 MHz, and 10 MHz. The signals associated with the 30 MHz clock group (G1) are de-skewed according to the formula. The 0 indicates that de-skewing is not required for this group.

```
Number of clock cycles to de-skew(n) for 30 MHz clock group = (40/30) - 1
= 1/3 = 0
```

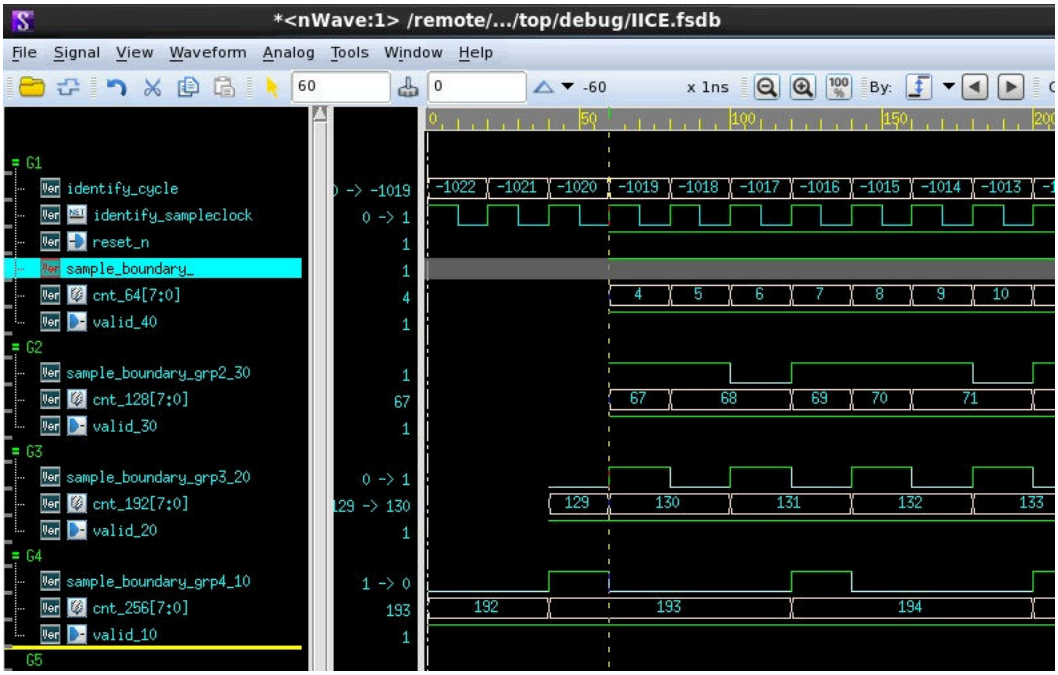
The waveform shows that the signals in the G2 group are de-skewed by 1 cycle, according to this formula:

```
Number of clock cycles to de-skew(n) for 20 MHz clock group = (40/20) - 1
= 1
```

Similarly, the signals in the G3 group are de-skewed by 3 clock cycles in the waveform.

```
Number of clock cycles to de-skew(n) for 10 MHz clock group = (40/10) - 1
= 3
```

Feedback



Varying Buffer Depth Dynamically for DDR3

The tool automatically calculates the maximum buffer depth for DTD, based on the number of instrumented bits. However, you can dynamically vary the buffer depth for DDR3 buffers or DDR3 memory modules. Setting a limit on the buffer depth can help with data offload times.

See *Configuring the IICE Sampler* in the *HAPS hRun User Guide*.

Verifying the Hardware Configuration for Deep Trace Debug

You can verify the deep trace debug memory configuration for a HAPS DDR3 daughter board or the on-board DDR3 module by running a self-test.

1. To run the self-test from the debugger console window, use the following command:

   ```
   iice sampler -runselftest 1|0
   ```

   There are two patterns (0 and 1) that you can run. The self-test writes data patterns to the external memory and reads back the data pattern to detect configuration errors and connectivity problems.

2. To run the self-test from the debugger GUI, follow these steps:
   - Open the project view and click the **IICE** icon.
   - Select *pattern 0* or *pattern 1* from the **Self-test** drop-down menu.
   - Click **Run external sample memory test**.
3. Repeat the command using the second pattern to ensure adequate testing.
4. You can run the self-test multiple times, at different points in the design cycle:
   - After the initial setup, to verify the hardware configuration against the instrumentation
   - During routine operations, when a hardware problem is suspected
   - When the physical configuration is modified by changing the IICE configuration settings.

Configuring CEL-Based Triggers

Complex Event Language (CEL) is a Verilog-like language to describe state machines. You can use CEL to configure triggers, by using it to describe DUT status monitoring and notification conditions.

Only state machine triggers are supported in CEL-based configuration for HAPS-100 systems.

1. Create a CEL file.

   For information about the file, see CEL Syntax in the *Debugger Reference*.

   - For instrumented SRS signals, use the following syntax to add a new hierarchy SRS name before the original hierarchy name. Use the complete SRS signal name defined in instrumentation).

     For the standard compile flow:

     ```
     var var_1 top_module.SRS.srs_hierarchy_name;
     var var [msb : lsb] top_module.SRS.\hdl_hierarchy [msb : lsb] ;
     ```

     For the unified compile flow:

     ```
     var var_1 SRS.srs_hierarchy_name;
     var var [msb : lsb] SRS.\hdl_hierarchy [msb : lsb] ;
     ```

2. From the debugger GUI, select **Debugger > Load CEL**.
3. Select the IICE, specify the CEL file, and click **Compile**.

The equivalent command is *cel compile*. Other commands are *cel set* and *cel get* (see cel in the *Debugger Reference*).

4. Check for errors.

   See CEL Errors for descriptions of the errors.

CEL Errors

There are various kinds of errors that are reported:

- CEL file syntax errors

  If there is a syntax error in the CEL file, you get this message: *Error: Failed to parse*. The error also lists the line where the error occurred and the last token that caused the error. For example:

  ```
  Error: Failed to parse {syntax error on line 8: last token is event}
  ```

- Undefined reference errors
  - Variable not found.

    These messages begin with *VAR_NAME at line NUMBER:*. If the variable is missing, use the *hierarchy found*Tcl command to get a list of available variables.

  - Undefined reference in event declarations

    These messages begin with *Event EVENT_NAME at line NUMBER:*. If an undefined reference is found, declare the variable first, followed by a watchpoint activation as described in CEL Syntax. It is also essential for an event to be declared first if it is going to be reused.

  - State machines with empty operations

    These messages begin with *State STATE_MACHINE at line NUMBER:*. Delete states with empty operations.

- Unsupported operation errors

  These error messages indicate the operation is not currently supported. These messages use the following prefixes:

  ```
  Unsupported operation at line NUMBER: Unsupported operator at line
  NUMBER:
  ```

- Invalid operation errors

These error messages indicate that the specified usage is not correct. For example, you cannot assign true or false to a multi-bit variable. Most of these messages use the following prefix:

```
Invalid operation at line NUMBER:
```

However some invalid operations in variable declarations use a general variable prefix, and the invalid operation error is described in the part after the colon. For example:

```
VAR_NAME at line NUMBER: Bit-width (X-bit) is not equal to Y-bit in
hierarchy name.
VAR_NAME at line NUMBER: Bit-width (X-bit) is not equal to Y-bit in
IICE.
VAR_NAME at line NUMBER: Variable range [msbX:lsbX] is not in the
range [msbY:lsbY] in IICE.
VAR_NAME at line NUMBER: Cannot do cross triggering with itself.
VAR_NAME at line NUMBER: Cannot assign variable range to
identifier/IICE.
VAR_NAME at line NUMBER: Cannot assign variable range to breakpoint.
VAR_NAME at line NUMBER: Signal ESCAPED_NAME is an escaped identifier
without backslash.
VAR_NAME at line NUMBER: Signal ESCAPED_NAME is an escaped identifier
without space suffix.
```

- Resource limitation errors

  These messages are displayed when the number of conditions or states or other resources are not sufficient. These messages often use this prefix:

  ```
  Invalid Usage:
  ```

  However, other resource errors are often only distinguished by the description after the colon. These are some examples of messages with assorted prefixes:

  - Counter declaration

    ```
    Event EVENT_NAME at line NUMBER: Excess maximum counter value
    MAX_COUNTER_VALUE for occurs, sustains. At line NUMBER: Excess
    maximum counter value MAX_COUNTER_VALUE.
    ```

  - Event declaration

    ```
    Event EVENT_NAME at line NUMBER: Excess maximum number of
    MAX_CONDITION_NUMBER conditions. At line NUMBER: Excess limited
    operator/operand number MAX_OPERAND_NUMBER.
    ```

  - State machine declaration

```
FSM FSM_NAME at line NUMBER: Excess maximum number of states
MAX_STATE_NUMBER. FSM at line NUMBER: Excess maximum number of
states MAX_STATE_NUMBER. Introduce new state when using occurs,
sustains.
```

Working with Deep Trace Debug

Deep Trace Debug (DTD) is a post-processing debug methodology that uses dedicated memory to store sample data for debug. It expands the storage capacity required for larger visibility windows by using external memory instead of local FPGA memory, like BRAM resources.

There are different DTD schemes that offer different speeds and visibility, according to the hardware resources they use.

See the following for more information:

- Comparison of Deep Trace Debug Schemes
- About Single-FPGA Deep Trace Debug (DTD)
- External DDR3 Memory
- Running Deep Trace Debug (DTD)

Comparison of Deep Trace Debug Schemes

There are different DTD schemes, depending on the kind of storage memory used. Storage memory could be external or internal or built-in.

About Single-FPGA Deep Trace Debug (DTD)

Single-FPGA HAPS Deep Trace Debug (DTD) lets you run RTL debug or post-compile debug on a single FPGA. In the multi-FPGA context, single-FPGA debug refers to debug of a partitioned individual FPGA. You might want to debug a single FPGA for to save runtime. There are some differences in the memory storage used with different HAPS systems.

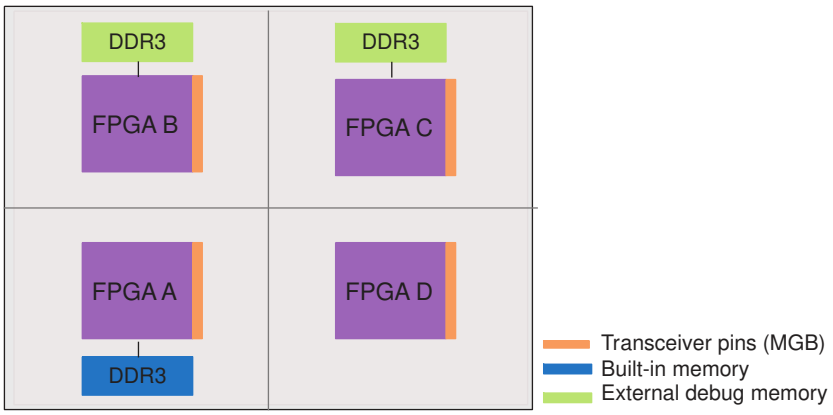
The following table summarizes the key features:

Debug Interface	UMRBus
Sample Buffer Memory	Dual Rank HT3 DDR3 daughter card Single Rank HT3 DDR3 daughter card

Number of Instrumented Signals	Up to total 2042 signals instrumented
Maximum Sample Frequency	Up to 250 signals: 140 MHz Up to 506 signals: 70 MHz Up to 1018 signals: 35 MHz Up to 2042 signals: 17.5 MHz
Over-Instrumentation	MUX set of 8 signal groups for up to 16K signals. Each signal group can have up to 2042 signals.

External DDR3 Memory

You can add DDR3 memory for debug storage as needed. The following figure shows additional DDR3 memory added to debug FPGAs B and C:

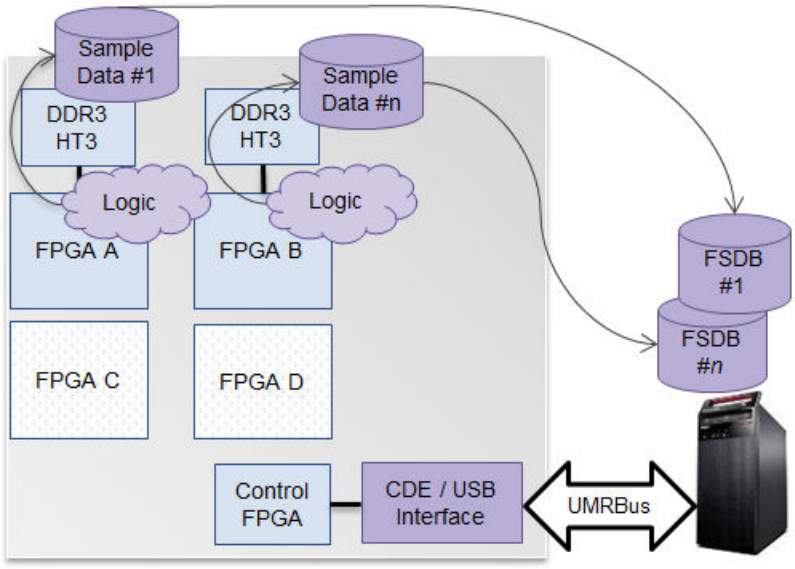


Running Deep Trace Debug (DTD)

Single-FPGA HAPS Deep Trace Debug (DTD) lets you run RTL or post-compile debug on a single FPGA. You can also use it to debug a single partitioned FPGA in a multi-FPGA design. This DTD scheme uses external DDR3 or DDR4 memory for debug storage.

The following figure shows the tool process to run debug on single FPGAs using a DDR3 memory. The HT3 DDR3 provides sample storage, up to 8 GB for each FPGA. Note that each FPGA is a separate debug process and that separate FSDB databases are generated for individual FPGA debug runs.

Feedback



See the following topics for details:

- Running the RTL Debugger

Features for DTD with a DDR3 Daughter Board

The following table summarizes the key features. The numbers apply to DTD with a DDR3 daughter board.

Debug Interface	UMRBus
Number of Instrumented Signals	Up to total 2042 signals instrumented
Maximum Sample Frequency	Up to 250 signals: 140 MHz Up to 506 signals: 70 MHz Up to 1018 signals: 35 MHz Up to 2042 signals: 17.5 MHz
Over-Instrumentation	MUX set of 8 signal groups for up to 16K signals. Each signal group can have up to 2042 signals.

RTL Data Expansion Using Verdi for Instrumented Signals

Beta

HAPS Deep Trace Debug (DTD) provides debug infrastructure which has fast hardware capture and higher capacity for instrumenting large set of signals. Although, the number

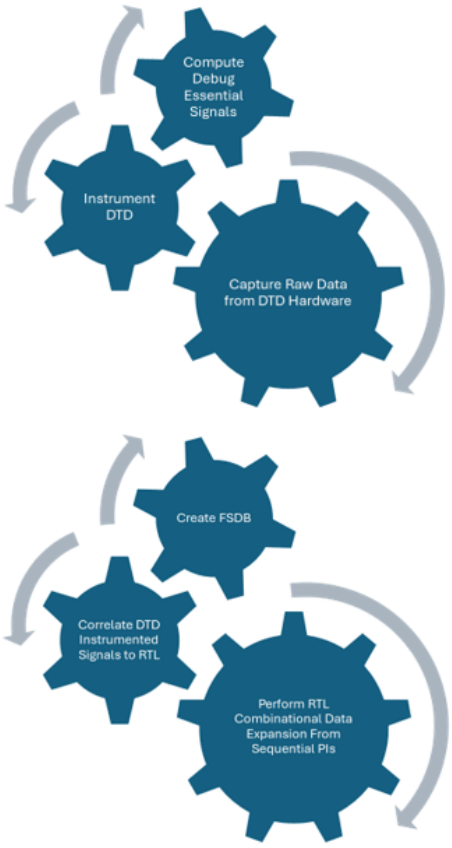
Feedback

of signals that can be instrumented for debugging are limited by the available hardware resources.

Instrumenting only the debug essential signals through DTD which are treated as the primary inputs (PI) for the combinational cone of debug interest and then performing RTL data expansion maximizes the instrumentation capacity without requiring any additional hardware resources and maintaining the performance.

The RTL data expansion uses the HAPS-Verdi flow. In this methodology, you are first guided into instrumenting the essential sequential PIs for a given set of combinational signals and then add the DTD instrumentation. Then all these signals are captured from the hardware and a complete SOC FSDB is created. This FSDB is then used in the existing HAPS-Verdi flow to perform the RTL data expansion on the corresponding driven combinational cone using the sequential PIs.

The following image shows the RTL data expansion methodology:



To enable RTL data expansion of DTD instrumentation using the HAPS-Verdi flow, use the following Tcl commands:

Note:
Beta features require a special license. Contact Synopsys Customer Support.

- `report debug_visibility` command for better visibility by incorporating essential registers. See report debug_visibility.
- `report debug_essential_signals` command for essential register reporting for combinational signals.
- `write_fsdb -DE` command is required for FSDB generation for RTL data expansion, see write fsdb. FSDB is required to be generated in the HAPS-Verdi flow compatible format for DTD instrumented signals.

Limitations

Currently, RTL Data Expansion supports sequential (register) signals DTD instrumentation.

Future releases support visibility with module input ports or RTL combinational signals but has a Verdi dependency.

Using BRAM for Debugging

You can use the on-board BRAM blocks to store sample data for debugging operations. BRAM-based debug offers many advantages. It is a platform-independent, fast, method that does not consume I/O resources. It leverages existing system resources, so it does not require additional hardware resources. It uses existing TSS connectivity so no major TSS changes are required. The performance limits and width are determined by the FPGA resources.

You can use it for multi-FPGA debug. You can use it for any sampling frequency, but it is best suited to high-frequency debugging.

1. Instrument the design.
   - For single-FPGA designs, instrument Instrument the design at the pre-instrument state (RTL-based instrumentation) or after compile (compiled database instrumentation).
   - For multi-FPGA designs, instrument the design at the top level at the pre-instrument state (RTL-based instrumentation) or the pre-partition state (compiled database instrumentation).

- Set the buffer type to internal memory:

  ```
  iice sampler iice internal_memory
  ```

- Set simple triggering only.

2. Run through the partitioning or synthesis implementation flow as usual.

   The tool uses four pins per slave IICE. It automatically inserts a cross-trigger network

3. Start hRun and run debug.

   The tool uses distributed RAM blocks to store sample data. You can view the results in a single waveform view (FSDB, VCD). It generates a balanced network where waveforms do not need to be post-processed.

Running Incremental Debug for ECOs

This instrumentation flow allows Engineering Change Orders (ECOs) to modify existing instrumentation for an IICE. It leverages the Xilinx incremental re-route feature. When ECO-based incremental re-route is used for a new debug implementation, turnaround time is much faster than running a full compile for a few instrumentation changes to the IICE logic.

Incremental debugging for ECOs is currently supported on single FPGA synthesis flow for BRAM and DTD buffer types.

You can use either of the following methods to run debug ECO post SLP:

- Creating Incremental ECO IDC File Using Instrumentor User Interface
- Creating Incremental ECO IDC File using the report debug_visibility command

Creating Incremental ECO IDC File Using Instrumentor User Interface

Perform the following steps to create the ECO IDC using the Instrumentor User Interface:

1. Complete the initial instrumentation of a design, run regular synthesis flow, followed by Vivado place-and-route and bit file generation, and import the Vivado results to map state using *import vivado* command.
2. Create an incremental *idc* file.
   - Run `edit idc`*<idc_filename.idc>*`-mode gui`from the mapped state and provide a new name for the incremental IDC file. See the *Command Reference Manual* for syntax details.

Feedback

3. Edit the instrumentation for ECO modifications.

   You can edit the existing instrumented signals and also instrument new signals. Use the *INCR_FILE* hierarchy in the Instrumentor. This is applicable only for the SRS database of a single FPGA. The *INCR_FILE*has a list of the Vivado EDIF mapped signals of your design, which is available for incremental instrumentation.

   Use the following Tcl command in *Instrumentor* shell window to generate a list of instrumented signals:

   ```
   % hierarchy find -stat instrumented -type signal
   ```

   To quickly filter the signals that you can edit, click the *Instrumentor Search*icon () in the upper left of the Instrumentor window to open the *Instrumentor Search* dialog box. Set the *Instrumentation* option to *not-instrumented*, and click *Search*. The tool displays a filtered list that only contains the un-instrumented signals you can modify.

   Use the following Tcl command in *Instrumentor* shell window to generate a list of un-instrumented signals:

   ```
   % hierarchy find -stat not-instrumented -type signal
   ```

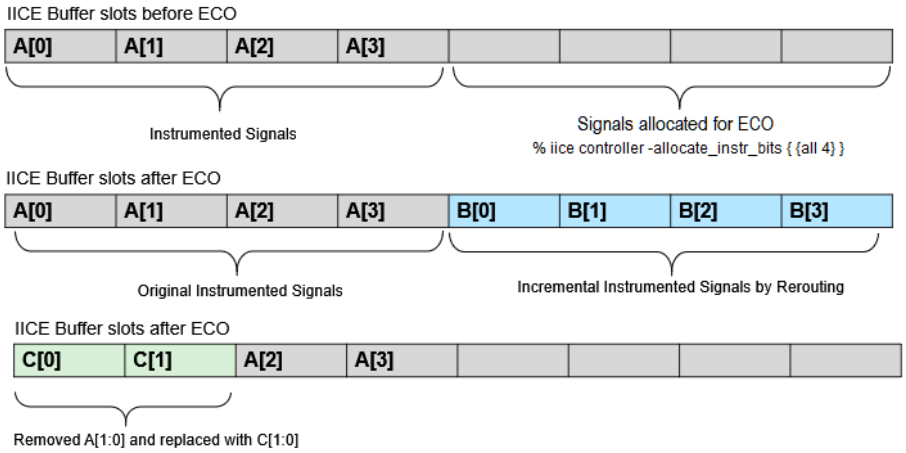
   - Un-instrument the signals, which are expected to be replaced from the initial instrumentation.
   - Instrument the new signals.

   The total number of signals or bits cannot exceed the original number, so for every new signal or bit added, you must remove an existing signal or bit. To allocate more signal bits than the original instrumentation bits, you can use the following Tcl command during the initial instrumentation step:

   ```
   % iice controller -iice <IICE_name> -allocate_instr_bits {{all <n>}}
   ```

Feedback

where, *n* represents the number of bits to be allocated for the future instrumentation during ECO. The following image provides an overview of allocate bits scenario:



- Save the instrumentation. The tool creates an *idc* file that only contains the edits, which it uses in conjunction with the original *idc* file from the mapped state.

4. Incorporate the debug ECO instrumentation into the design. See Applying Debug ECO Instrumentation in the Design.

Creating Incremental ECO IDC File using the *report debug_visibility* command

You can perform ECO using the *report debug_visibility* command after successful *map state*. This method is called as *RTL-to-EDIF correlation* for ECO. It is recommended to run the *report debug_visibility* command to check if a signal is optimized or renamed. For more details, see the *Command Reference Manual* for syntax details.

Perform the following steps to create ECO using the *report debug_visibility* command (RTL-to-EDIF correlation):

1. Enable *incremental_debug* option using the command *option set incremental_debug 1* in the database before running the design.
2. Complete the initial instrumentation of a design, run regular synthesis flow, followed by Vivado place-and-route and bit file generation, and import the Vivado results to map state using import vivado command.
3. Run the *report debug_visibility* command at the map state. The signal names contained in the input IDC (RTL or SRS) are converted into register names that are required for the Debug ECO and the availability of the signal names are checked and reported. You can instrument new signals based on RTL or SRS signal names instead of Xilinx register names. For example: `% report debug_visibility -export_path`

Feedback

```
<new_directory_name> -idc <IDC_file_mentioning_new_signals_for
ECO>.idc
```

- Based on the availability of signals queried, an ECO-based IDC file is generated that can be used to *run debug_eco* command.

  Note:

  The signal name provided in the command is correlated with all the possible matching signals available at the required stage, and the found names are provided in the generated IDC file. The IDC file is a template; review and make the appropriate changes in the generated IDC file before you use it.

4. Incorporate the debug ECO instrumentation into the design. See Applying Debug ECO Instrumentation in the Design.

Applying Debug ECO Instrumentation in the Design

Perform the following steps to create the ECO IDC:

1. Incorporate the ECO instrumentation into the design using the following command:

   ```
   run debug_eco -idc <filename>.idc -writeincrreports
   ```

   The command uses the specified *idc* file with the ECO changes, and generates an incremental Vivado script for place and route. The *run debug_eco* command creates a new state (*de0)* below the parent map state. This is a terminal state that does not generate any other sub-states. You cannot use *run debug_eco* command again from the existing *debug_eco* state. At the *de0*state, check the *<top_module_name>_identify_script.log* file generated for the incremental instrumentation information. See run debug_eco in the *Command Reference Manual* for syntax details.

2. Re-route the design and generate the bit file.

   - From the *debug_eco* state (*de0*), run the *export vivado* command. This command writes out the newly generated Vivado script (*debug_eco_vivado.tcl*) to the specified location. If you have multiple debug ECO runs and corresponding sub-states, make sure to run the *export vivado* command from the correct sub-state (*de1*,*de2*,and so on).
   - Run *launch vivado* to run the incremental re-route and generate new bit files. The script runs the Vivado router incrementally. The incrementally re-routed signals information can be found in the *vivado.log*.
   - From the *debug_eco* substate, run *import vivado* to import the place-and-route results into the database.

3. Link the debug ECO states to the system generate node using the *database apply_state -link_module* command after System Generate state. The following example shows how to link the debug ECO states of FPGA A and B to the system generate state:

```
database apply_state -link_module
 FB1_uA ./synthesis_files/FB1_uA/FB1_uA_srs|de0
database apply_state -link_module
 FB1_uB ./synthesis_files/FB1_uB/FB1_uB_srs|de0
```

4. Run *export runtime* to export the incremental ECO instrumentation for debug.

   Export runtime files for the linked modules at the System Generate state, if all the FPGAs are modified, or at FPGA-level debug ECO state to export for individual FPGA.

```
export runtime -path incr_export_results_link_module
```

5. Program the ECO generated bit files on the hardware, and open the debug projects exported after incremental ECO instrumentation in hRun. The incrementally instrumented signals are now available for debug.

Limitations to the Debug ECO Flow

Currently, these are some limitations with the debug ECO flow:

- Supports only single-FPGA design.
- Does not support the following features:
  - Clock groups for multi-clock instrumentation
  - MUX groups with preserve bits
  - Breakpoint instrumentation
  - Debug ECO for partition flow
- IICE buffers must use BRAM or *haps100_DTD_builtin* buffers.

Using Global State Visibility (GSV)

Global State Visibility (GSV) provides a snapshot of all registers in a configured design at a particular clock cycle. It offers the advantage of full visibility. It does not require instrumentation and is instantly available. GSV correlates the low-level signal names back to the original RTL names. It makes the entire register chain visible for debug, by leveraging the Xilinx register readback capability. It reads the current states of all internal registers within a Xilinx FPGA at a particular clock cycle.

You can use the information for hardware debugging and functional verification. You can run GSV separately or in combination with other debug functionality for single-FPGA designs, single-FPGA partitions, or multi-FPGA designs. You can run GSV in:

- Asynchronous mode for semi-static analysis, or
- Synchronous mode for system-wide state correlation and clock control.

There are two GSV modes: legacy GSV and enhanced GSV (eGSV).

In eGSV mode, the download speed of the GSV data from the target device is improved. To enable eGSV mode, set the *prepare_readback* option to 2. This mode is implemented only when the design is instrumented. But there is no change in the GSV use model described in the following subsections.

Note that the term GSV is used broadly to describe both GSV and eGSV, as the use models are similar. However, the term eGSV is used when something only applies to eGSV.

See these topics for details:

- Asynchronous and Synchronous GSV
- Using Asynchronous GSV for a Single-FPGA Design
- Use Model Example: Synchronous GSV for a Single-FPGA Design
- Use Model Example: Synchronous GSV with a Multi-FPGA Design
- Configuring Clock Control Modules for GSV
- Memory GSV
- Running GSV with Clock Control
- Using PLL Clocks for Global State Visibility

Asynchronous and Synchronous GSV

GSV can be asynchronous or synchronous. Use asynchronous mode for analysis of static or semi-static signals, or synchronous mode for system-wide state correlation and clock control.

The following table compares the features available in asynchronous and synchronous GSV:

Feature	Synchronous GSV	Asynchronous GSV
Time-synchronized register values within and across FPGAs	CCM (clock control module) guarantees synchronous readback of all registers in and across FPGA	Register values might not be synchronized
Clock stepping	CCM provides stepping	Stepping is NOT possible
Trigger support	Yes	No
External hardware	Hardware required for fine grain control mode, for arming, releasing, stepping. Trigger out with distributed IICE. None required for asynchronous CAPIM control which is triggered from the host system command line.	Not required
Usage scenarios	System freeze diagnostics Dynamic system activity (limited window)	System freeze diagnostics Semi-static signals/ discrete signals

The following provide more details about the differences between the two modes:

- Asynchronous GSV

Asynchronous GSV represents a snapshot of the register states in a running design, with the captured asynchronously to the design clock. All register data within the design are loaded into a parallel rank register and shifted out. However, as the design clock is still running, there is no guarantee that the captured data is aligned to the same clock cycle. In asynchronous mode, the state of the system is unknown. Asynchronous GSV is best suited for hang conditions, where you are looking at static or semi-static signals, like control signals, to get an idea of where to look for your next instrumentation cycle. For example, you can use this mode when programming an IP, to confirm the state of the IP.

See Use Model Example: Synchronous GSV for a Single-FPGA Design for information about using this mode.

- Synchronous GSV

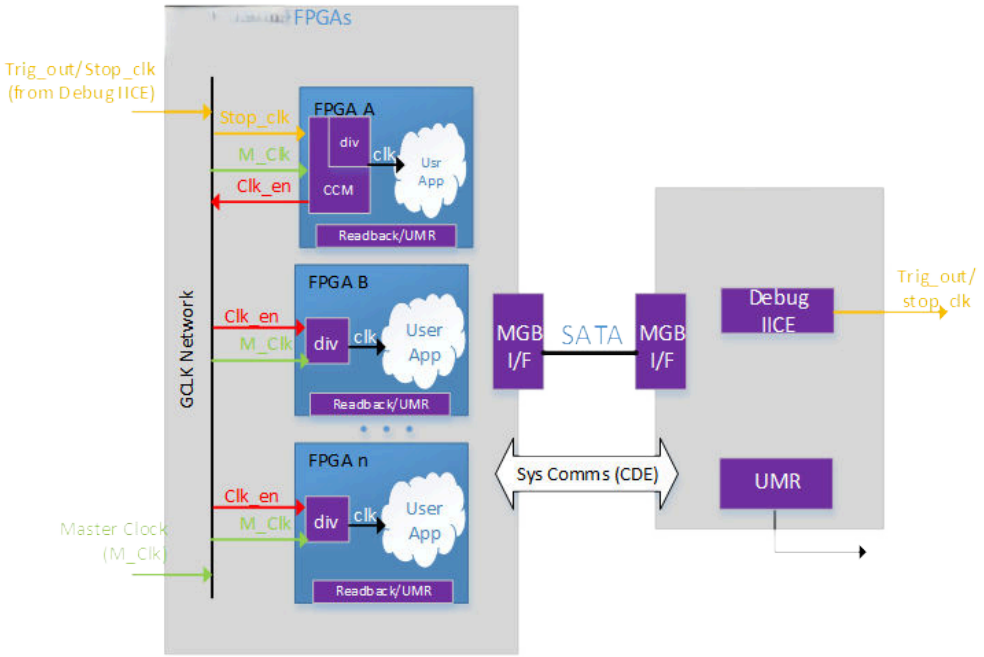
  Synchronous GSV correlates the entire system and aligns all the registers. It is intended for use in a hung processor scenario, where there is no DTD visibility. It provides a snapshot of register values while the design clock domain is stopped. The drawbacks to synchronous GSV are the need to stop clocks and zero depth.

Feedback

Synchronous GSV requires a clock control module (CCM) to be instantiated. The CCM is inserted at the base of the clock tree for the design. It takes the master clock source as input and forwards the clock to the rest of the design. When an enable signal is asserted, the CCM gates the clock and facilitates the downloading of captured registers off the device.

You can program the CCM with a command sequence, with options to step the clock for a specified number of cycles after the CCM module has been triggered.

There are two ways to control CCM assertion: direct assertion using an input port to the CCM, or through a command from the debugger shell. The following diagram shows a HAPS system used as a deep trace debug (DTD) buffer, and also as the mechanism to assert the control line to the clock enable module.



In the synchronous use model, the DTD debug IICE monitors signals under test at system speed. You set a trigger event, and when this occurs, the debug IICE is also configured to assert a trigger out signal. The trigger out signal is routed back to the user FPGAs to assert the CCM in order to start the GSV capture sequence.

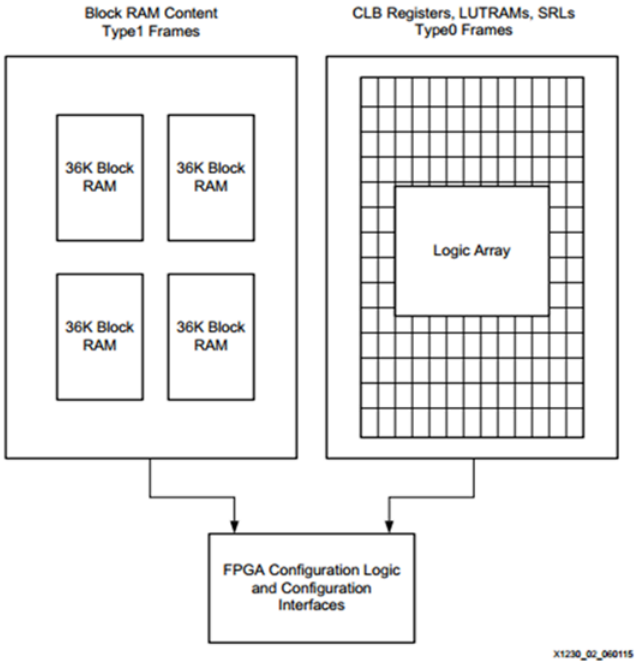
GSV Limitations

Currently, there are some limitations:

- For mixed HAPS system, legacy GSV must be adapted.
- The GSV capability uses GPIO and HT3 pins that are usually configured as user design pins, so there could be place-and-route conflicts if you do not set the *readback* option.
- Synchronous GSV limitations:
  - The phase and frequency from the clock control module can be controlled independently for each clock. Controlled clocks have a duty cycle of 50%.
  - You cannot independently stop an individual clock from the clock control module.
- Use the latest version of *haps_controlled_clocks*.v; do not use older versions saved in local areas.

Using Asynchronous GSV for a Single-FPGA Design

Asynchronous GSV represents a snapshot of the register states in a running design. It is a simple snapshot with no data alignment between signals. It is intended for use with static or semi-static signals. It does not support VM flow for back-annotation.



The following procedure describes how to use asynchronous GSV for a single, unpartitioned FPGA.

► Set the compile option for GSV or eGSV:

GSV: `option set prepare_readback 1`

eGSV:`option set prepare_readback 2`

You must set this option to generate the logic location (*.ll*) file later, during bit file creation.

► Run through the other stages in the flow as usual: compile, map, placement and routing.

When the bit file is created, Vivado generates a *.ll* file, which maps the register information from the FPGA to design signal locations. GSV uses the signals in this file.

► Import the results from Vivado and backannotate results from place and route:

- Return to the map database state in the prototyping tool and run the *database apply_state -import_vivado* command. This command imports the Vivado *.ll* file and place-and-route results into the prototyping database.
- Run the *database apply_state -backannotate* command with the *-gsv*option to map the objects in the Vivado.*ll* file to the prototyping mapped database. GSV uses the Vivado *.ll*file to map values back to the RTL.

```
database apply_state -backannotate gsv
```

1. Set up files for debug, including information from the *.ll* file:
   - Run the *export runtime* command to generate a GSV database that includes the Vivado and back-annotated *.ll* files under *exportResultsDir/system/readback*. The command also creates a parallel *debug* directory with the debug database in the same *system* export directory.
2. Run debug.
   - Arm the readback IICE to perform a single readback operation. Signal values are overlaid in the RTL view per signal line.
   - Run debug. If you instrumented the design, the command runs debug and GSV together.
   - To append samples to the waveform for asynchronous readback, use the*readback sample_buffer -append* command.

   For information about the syntax for the *readback* and *run* commands, see Instrumentor and Debugger Commands in the *HAPS ProtoCompiler Debugging Environment Reference*.

Use Model Example: Synchronous GSV for a Single-FPGA Design

This following procedure describes a use model which uses synchronous GSV for a single, unpartitioned FPGA, with a DTD IICE for triggering. For multi-FPGA partitions, follow the steps described in Use Model Example: Synchronous GSV with a Multi-FPGA Design.

1. Set up the design for register GSV.
   - Generate a root database for the design.
   - Specify *option set prepare_readback 1* to enable the GSV flow. To enable the eGSV flow, the *prepare_readback* option must be set to 2.
2. Continue with the normal design flow through the pre-instrument, compile and map stages.

   To run debug with GSV, you must run the pre-instrument phase, even if you do not actually instrument any signals. The debug directory, which is required for debug, is generated at this phase.
3. Place and route as usual, using the *launch vivado* and *export vivado* commands.

   The place-and-route tool generates a logic location (*.ll*) file, based on the GSV information. This file maps the register information from the FPGA to design signal locations. It contains all matched signals, and all signals in this file are automatically used for GSV.
4. Set up files for debug, including information from the *.ll* file:
   - Return to the map database state in the prototyping tool and run the *database apply_state -import_vivado* command. This command imports the Vivado *.ll* file and place-and-route results into the prototyping database.
   - Run the *database apply_state -backannotate* command to map the objects in the Vivado*.ll* file to the prototyping mapped database. This back-annotated file is called **_ba.ll*.
5. Run debug.
   - Run the *export runtime* command to generate a GSV database that includes the Vivado and back-annotated *.ll* files under *exportResultsDir/system/readback*. It also creates a parallel *debug* directory with the debug database in the same *system* export directory.
   - Start hRun.

     hRun reads back the register values of the matched signals and applies them in the source code viewer in the debugger. It also lists unmatched *ll* signals under LL tree in same view. After GSV is complete, the values are appended to the signal names.

Use Model Example: Synchronous GSV with a Multi-FPGA Design

This following procedure describes a use model which uses synchronous GSV for a multi-FPGA design, with a DTD IICE for triggering. For single-FPGA partitions, follow the steps described in Use Model Example: Synchronous GSV for a Single-FPGA Design.

1. Prepare the design for GSV.
   - To use the clock control capability, instantiate a CCM, and use the generated clocks from the CCM. (See Configuring Clock Control Modules for GSV for details). If you do not instantiate the CCM, there is no guarantee that the state information is aligned with the same system clock cycle.
   - Generate a root database.
   - Specify the*option set prepare_readback 1* command to enable GSV.
2. Continue with the normal design flow through the pre-instrument and compile stages.

   You must run pre-instrument. To run GSV with debug and clock control, you must instrument the signals to enable DTD2, and configure the DTD IICE with the export trigger option enabled. The export trigger signal serves as an assertion to the CCM to signal the start of a readback operation.
   - Configure the instrumentor for the DTD2 flow.
   - Set the export trigger option:

     ```
     iice controller -iice {IICE_0} -exporttrigger 1
     ```
   - Compile the design.
3. Follow these steps to set up the hardware and partition the design.
   - If you are using a CCM, do the following additional steps at the pre-partition and partition stages:

Pre-partition	▪ In the *tss* file, assign the trigger input port to the trace used to gate the CCM.
Partition	▪ Replicate the clock divider modules from the CCM to ensure that signals are synchronized by adding assignment and replication constraints to the *pcf* file. ▪ Assign the trigger input port, to gate the CCM. ▪ Assign the clock enable to GCLK network. See Configuring Clock Control Modules for GSV and Running GSV with Clock Control for details.

   - Run system route as usual.

4. Run system generate and do the following for each FPGA generated:
   - Launch the ProtoCompiler tool for the individual FPGA.
   - Synthesize, place, and route the individual FPGA as usual. The GSV capability uses the same pins (GPIO and HT3), that are usually configured as user design pins. When GSV is enabled, the tool ensures that there is no configuration conflict.

     When it generates the bit file, the place-and-route tool generates the logic location (*.ll*) file, which maps the register information from the FPGA to design signal locations. The *.ll* file contains all matched signals, and all signals in this file are automatically used for GSV.
5. Use the *export runtime* command to export the bit files of the individual FPGA, the two debug and GSV projects, and backannotated *ll* files back to the *export_results*directory.

   The tool creates two directories, one for debug and one for GSV, with the appropriate files.
6. Return to the System Generate state and link the partitioned FPGA to a corresponding mapped state of a single FPGA, using the *database apply_state -link_module* command.
7. Map Vivado *.ll* files to RTL-compatible *.ll* files (*_ba*)

   ```
   database apply_state -backannotate
   ```

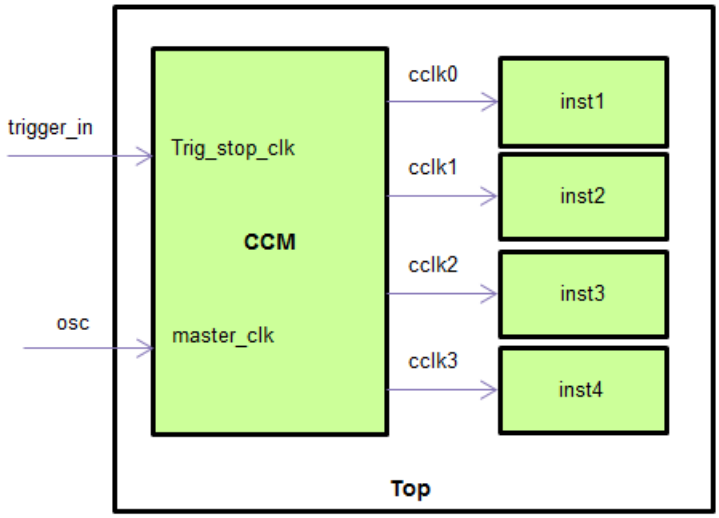
   This command backannotates information from the *.ll* files generated after place and route back to the map stage. Use these backannotated **_ba.ll* files for GSV.
8. Run debug.
   - From the System Generate state, run *export runtime* to export all debug files, including the *.ll* files for the FPGAs. The tool exports a single SoC project instead of multiple files for the FPGAs.

     There are two databases created, with the files located in the *debug* and *readback* directories under *export_results/directory*. The *readback* directory is generated if *prepare_readback* was set, and contains the files for GSV debug, along with all *.ll* files. The *debug* directory contains the instrumented database files needed to run deep trace debug.
   - Start debug using hRun.

     hRun reads back the register values of the matched signals and applies them in the source code viewer. After GSV is complete, the tool appends the values to the signal names.

Configuring Clock Control Modules for GSV

To ensure that data can be read synchronously across the FPGA and that register GSV is reliable, you must stop the design clock. The clock control module (CCM) that governs this process uses an uncontrolled input clock as the master clock and derives generated clocks from it to drive the instances in a synchronized fashion. The inputs to the CCM are a master clock and a clock stop, and the outputs are four clocks generated from a single master source, as shown in the figure. One example of a CCM is MMCM (mixed-mode clock manager).



Do the following to use the CCM:

1. Instantiate the CCM in your top-level design.
   - Get the CCM source file: *protocompiler_installDir/lib/di/haps_controlled_clocks.v*
   - Edit the CCM parameters that control the phase and frequency of uncontrolled clocks to values for your design. Do not edit any other CCM parameters. The following table illustrates the parameters to be edited.

Parameter	Description
CCLK{0 to 3}_COUNTER_MAX	Specifies an integer value to control the period of the generated clocks. The following equation where M is the parameter value, specifies its relation to the master clock: *T_CCLK(n) = 2*(M +1)*T_master_clk* If the master clock period is 10 ns and the parameter value M is 3, then the generated clock period is 8*10 ns. Default = 2

Parameter	Description
CCLK{0 to 3} _COUNTER_PHASE	Specifies a value to control the phase of the generated clock. Define the phase of each generated clock with respect to the smallest phase value among the four clocks. Default = 0
UMR_CLK_CTRL_CAPIM_ADDRESS	Sets the CAPIM address for the CAPIM that is used within CCM. Default =30

- Instantiate the CCM in the top-level design.

2. For multi-FPGA designs, add the following constraints to the *pcf* file to replicate the CCM across all FPGAs driven by clkout [3:0]:
   - Assign the clock module to FPGA A with *assign_cell* PCF commands:

```
assign_cell clock_control systemName.uA
assign_cell clock_control.te systemName.uA
assign_cell clock_control.haps_clk_ctrl inst systemName.uA
```

   - Use *replicate_cell* commands to replicate just the clock generate modules in all the FPGAs, including FPGA A. The following commands replicate clock generator modules *clkgen0, clkgen1, clkgen2,*and*clkgen3* in FPGAs A and B.

```
replicate_cell clock_control.cclk_gen_to_be_replicated.cclkgen0
 {FB1.uA FB1.uB}
replicate_cell clock_control.cclk_gen_to_be_replicated.cclkgen1
 {FB1.uA FB1.uB}
replicate_cell clock_control.cclk_gen_to_be_replicated.cclkgen2
 {FB1.uA FB1.uB}
replicate_cell clock_control.cclk_gen_to_be_replicated.cclkgen3
 {FB1.uA FB1.uB}
```

   - Assign the clock control enable signal to the GCLK network to ensure that the signal is uniformly distributed to all the replicated clock generator modules across the FPGAs. The following example assigns the clock enable to GCLK*4*, whose source is in FPGA A:

```
assign_cell {clock_control.clk_cntr_en_keeper}
 {FB1.clk.bd_clk4}
net_attribute {clock_control.clk_cntr_en} -function GCLK
 -diffsingle -is_clock 1
net_attribute {clock_control.clk_cntr_en_int} -function GCLK
```

   - Use the *assign_port* command to specify the hardware connection to export the external trigger signal which stops the clock. Make sure it feeds the FPGA that contains the CCM.

3. For multi-FPGA designs, define the GCLK to be used as FPGA source in the *tss* file:

   ```
   board_system_configure -clock {FB1.GCLK4} FPGA
   ```

   This command distributes the clock enable signals over the GCLK network.

Memory GSV

This feature supports readback of an RTL memory array using the Verilog or VHDL hierarchy path in hRun. To perform readback, the user-defined memory must be mapped to the Xilinx BRAM.

Running GSV with Clock Control

Follow this sequence of steps to run GSV with clock control:

1. Launch a debugger session and load the debug project generated by *export runtime.*
2. Train the GTX links for deep trace debug setup, either from the GUI or the Tcl prompt.
   - From the GUI, open the project for the debug IICE, and click the IICE icon. In the *Enhanced settings for IICE unit* dialog box, click *Check and setup Multi-FPGA HDTD Links*.
   - Alternatively, type this command at the Tcl prompt:

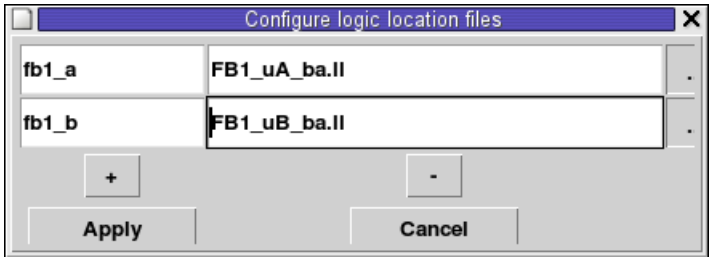
     ```
     haps setup_mfhdtd_links
     ```

   If the GTX links are correctly trained, the following message is reported. If you do not see this message, there could be problem with the setup.

   ```
   Wait for link training ...
   Link Status: (1 1)
   Align data word ...
   Reset system ...
   Check debug link alignment ...
   INFO: RX STATUS - 11110100000000000000000000000000
   Setup ready for debug. (1 1)
   ```

3. Launch the second debugger session and load the readback project in this exported directory: *export_results/system/readback*
4. In the GSV IICE, click the Readback icon ()to load the backannotated logic location files for the individual FPGAs.

   Use this FPGA naming convention: *boardName_fpgaName*. The *boardName* must be *fb1, fb2, fb3 ...*, and the *fpgaName* must be *A/a, B/b, C/c and D/d*.



5. In the GSV IICE, type the following *run* command with the *-readback* argument to start the operation.

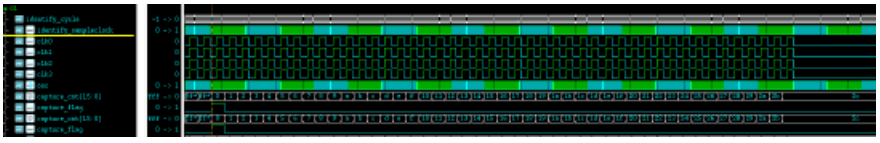
   For example:

   ```
   run -iice readback -readback 2 -stepping 2 -wait_for_trigger 1
   -release_clock 1
   ```

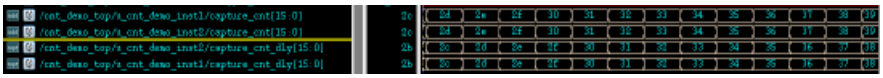
   This command performs three readbacks, and lets the controlled clocks tick two times between successive readbacks. The readbacks wait for the trigger and all the controlled clocks are released after GSV is completed. Alternatively you can use the *readback* command instead of *run*. For the complete command syntax, see Instrumentor and Debugger Commands in the *Debugger Environment Reference*.

6. Switch to debugger session where the debug project is loaded and do the following to arm the trigger.

   - Arm the trigger condition.
   - Click *Run*. When the debug IICE is triggered, the GSV IICE gets triggered and the GSV sample is downloaded in the GSV IICE. The GSV IICE receives the trigger signal some clock cycles of the master clock after the point where the trigger has occurred. The debug waveform shows the point where the export trigger is received and the controlled clocks have stopped to perform GSV, using clock stepping mode.

     The next GSV waveform shows the point where the GSV is performed by stepping the clock

7. Write out the GSV waveform by running this command in the GSV IICE:

   ```
   write fsdbfile_name.fsdb
   ```

   You must use this command to dump out the GSV waveform for viewing GSV data because the waveform viewer icon is not available at this point.

Using PLL Clocks for Global State Visibility

This section explains usage of PLL clocks as source clocks for the Clock Control Module (CCM), which generates controlled clocks for synchronized readback in GSV flows.

Term	Definition
PLL Clock	A programmable clock generated by the Phase-Locked Loop on the HAPS board.
CCM Clock	Controlled clocks generated by the Clock Control Module using PLL input.
GCLK	Global clock net used to distribute clocks across FPGAs.

Note:

In GSV flows, PLL clocks are used as input to the CCM. The CCM then generates controlled clocks for readback. GCLKs are global routing nets and should not be confused with CCM-generated clocks.

1. Instantiate CCM in your top-level design.
2. Feed PLL clock as the master input to CCM.
3. CCM generates controlled clocks for readback.
4. Use GSV instrumentation to capture internal states.
5. Ensure clock frequency is ≤16 MHz for reliable readback.
6. Use OR combiner for multiple trigger inputs.
7. Run readback dump using ProtoCompiler or runtime tools.

GSV or eGSV Flow

GSV or eGSV flow is supported with two different types of clock generators.

- Configuring CCM V2 Clock Generator
- Configuring CCM V3 Clock Generator

Feedback

- Automatic Instrumentation of Driver Clock
- CCM Debug Flow Enhancements
- Limitations

Configuring CCM V2 Clock Generator

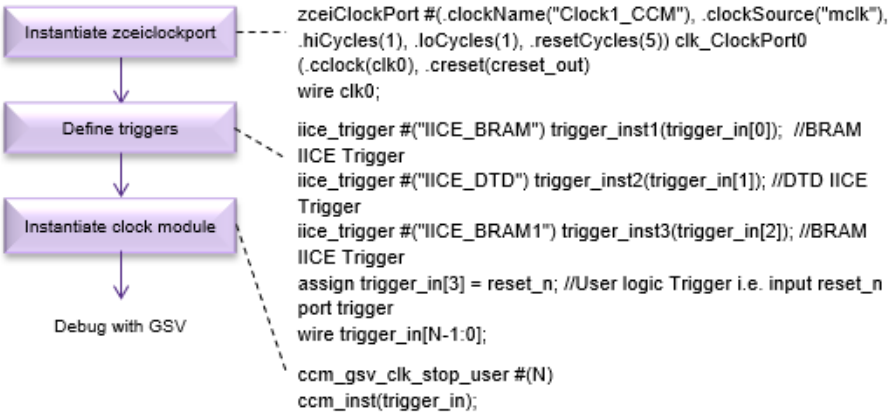
The following flow for HAPS-100 designs describes how to use *zcei* clock ports to stop the clocks and enable readback capture for GSV or eGSV. The following procedure describes an example that uses *zcei* clock ports to run synchronized GSV or eGSV debug on a two-FPGA design.

See these sections for details about the flow:

- Setting up to Run GSV or eGSV with HAPS-100
-
-
-

Setting up to Run GSV or eGSV with HAPS-100

This figure summarizes the main setup steps. Refer to the subsequent procedure for detailed steps.



► Enable readback by setting the *prepare_readback* option:

- For GSV, set this value: `set option prepare_readback 1`
- For eGSV, set this value: `set option prepare_readback 2`

eGSV achieves faster readback download speeds than GSV.

Feedback

► Set up the RTL files:

◦ Explicitly include these *.v* files for the clock port along with the other source files.

```
<build_path>/lib/synip/clocks/gsv_trigger_hypermods.v
<build_path>/lib/synip/hcei/zceistubs.v
```

◦ Instantiate *zcei* clock ports in the RTL. The number of clock ports depends on how many clocks need to be generated for the design. For example, if you want to generate four clocks (i.e. divide by 2, 4, 8 and 16), add four *zceiClockPort* definitions, as shown in the following example.

```
wire clk1;
wire clk2;
wire clk3;
wire clk4;
zceiClockPort
 #(.clockName("Clock1_CCM"), .clockSource("mclk"), .hiCycles(1), .lo
Cycles(1), .resetCycles(5)) clk_ClockPort0
 (.cclock(clk1), .creset(creset_out));
zceiClockPort
 #(.clockName("Clock2_CCM"), .clockSource("mclk"), .hiCycles(2), .lo
Cycles(2), .resetCycles(5)) clk_ClockPort1 (.cclock(clk2));
zceiClockPort
 #(.clockName("Clock3_CCM"), .clockSource("mclk"), .hiCycles(4), .lo
Cycles(4), .resetCycles(5)) clk_ClockPort2 (.cclock(clk3));
zceiClockPort
 #(.clockName("Clock4_CCM"), .clockSource("mclk"), .hiCycles(8), .lo
Cycles(8), .resetCycles(5)) clk_ClockPort3 (.cclock(clk4));
```

Here, the base clock, i.e. the clock input source to the *zcei* clock ports, is defined as *mclk*. Use the clock name from the constraint instead of the RTL clock port name when defining clocks that must be controlled or stopped for GSV capture. For example, if the top-level RTL clock port input is *osc*, and the FDC file defines the clock constraint as *mclk*, use the corresponding clock constraint name from the FDC file (*mclk*) to define the clock source to the *zceiclockport*, not *osc*.

```
create_clock -name {mclk} {p:osc} -period {50}
```

► Define the triggers.

◦ Define up to a maximum of 10 user triggers in the *idc* file. You can specify a combination of triggers from different IICE blocks, and user logic triggers.

IICE Triggers:

```
iice_trigger #("IICE_0") trigger_inst1(trigger_iice_0);
iice_trigger #("IICE_1") trigger_inst2(trigger_iice_1);
iice_trigger #("IICE_2") trigger_inst3(trigger_iice_2);
iice_trigger #("IICE_3") trigger_inst4(trigger_iice_3);
iice_trigger #("IICE_4") trigger_inst5(trigger_iice_4);
```

Feedback

```
iice_trigger #("IICE_5") trigger_inst6(trigger_iice_5);
iice_trigger #("IICE_6") trigger_inst7(trigger_iice_6)
```

User logic triggers:

```
assign trigger_in[0] = trigger_iice_0;
assign trigger_in[1] = trigger_iice_1;
assign trigger_in[2] = trigger_iice_2;
assign trigger_in[3] = trigger_iice_3;
assign trigger_in[4] = trigger_iice_4;
assign trigger_in[5] = trigger_iice_5;
assign trigger_in[6] = trigger_iice_6;
assign trigger_in[7] = gpio_1;    // User logic trigger
assign trigger_in[8] = gpio_4;   // user logic trigger
assign trigger_in[9] = gpio_7   // user logic trigger
```

- Connect the triggers to a wire called *trigger_in[9:0],* as shown in the example.
- Next, connect the triggers to a user GSV clock stop trigger module.

```
ccm_gsv_clk_stop_user #(10) clk_stop_inst (trigger_in);
```

The number 10 defines the trigger input parameters, and the current maximum is 10 triggers.

The control clock module (CCM) is responsible for stopping all the CCM clocks to enable GSV capture. At runtime, use the clock module to select which of the 10 triggers to use to stop the clocks. You can also use it to combine different triggers by masking them with a MCAPIM register. By default, all triggers are ORed.

► Define the CCM base clock out in the TSS file.

The CCM base clock out is routed to other FPGAs as the FPGA clock source.

- Define the clock routing to other FPGAs. This example shows the TSS definition for the master clock source, *osc*. The master clock source uses GCLK1 in the PLL1 as the clock source, and is routed to both FPGAs A and B in this 2-FPGA design:

```
board_system_configure -pll mb1.PLL1.CLK1 -frequency 20000 -name
d_clk
board_system_configure -clock {mb1.uA.CLK1} d_clk
board_system_configure -clock {mb1.uB.CLK1} d_clk
```

- Define the FPGA clock source for the CCM clock base out. The following example defines CLK3 as the FPGA clock source.

```
board_system_configure -clk_src mb1.uA.CLK_SRC3 -name ccm_clk1
-frequency 10000
board_system_configure -clock {mb1.uA.CLK3} ccm_clk1
board_system_configure -clock {mb1.uB.CLK3} ccm_clk1
```

► Add constraints in the PCF file for the clock module and clocks:

◦ Define the FPGA where the clock module is instantiated. This PCF constraint specifies that it be instantiated in FPGA A:

```
zcei_assign_route -master_bin mb1.uA
```

◦ Define the PCF constraints for the master clock and the FPGA clock source as shown in the PCF commands.

Definition for CCM clock input:

```
net_attribute osc -function GCLK
assign_global_net osc d_clk
```

Definition of CCM clock base out as FPGA clock source:

```
net_attribute {haps_infra_clksrc_mclk_cclock_base_out}
 -function GCLK
assign_global_net {haps_infra_clksrc_mclk_cclock_base_out}
 ccm_clk1
```

► Run through the regular GSV flow through place and route, to generate the *ll* file and bit files.

Configuring CCM V3 Clock Generator

Use this procedure to configure a two-FPGA synchronous GSV or eGSV flow to stop the clocks and enable the readback capture using the CCM V3 clock generator.

1. Execute the following command to enable readback:

   ```
   option set prepare_readback [1|2]
   ```

   Note:

   1 is for GSV and *2* is for eGSV. eGSV achieves faster readback download speeds than GSV.

2. Execute the following command to enable RTL instrumentation:

   ```
   option set debug_dumpvars 1
   ```

3. Execute the following command to enable the clock generator mode:

   ```
   option set haps_clockgen_mode 1
   ```

4. Enable Verdi correlation. (Mandatory for UC flow).

Feedback

5. Set up the RTL files to insert the CCM V3 clock generator in the design and set up the CCM clock:
   - For Unified Compile flow, include the following *.v* files for the clock port along with other source files:

     ```
     option set verdi_mode 1
     ```

     <build_path>/lib/synip/clocks/gsv_trigger_hypermods.v <build_path>/lib/synip/hcei/zceistubs.v
   - Declare the CCM source using the TSS command. For example:

     ```
     board_system_configure -create_ccm_clocks -source <source_fpga>
     -load <load_FPGAs> -source_frequency <source clock freq.>
     ```

     This ensures that the CCM source resides in *FPGA A* and the CCM load is replicated on both *FPGA A* and *FPGA B*.
   - Create CCM clocks. Define the clock constraint in the FDC file with the required frequency and add it to the design as input by declaring the clock port input. For example:

     ```
     create_clock -name Clock1_CCM [get_ports {p:clk0}] -period 40
     -waveform {0 20} create_clockgen_group {CLOCK1_CCM} -group {grp1}
     ```

     In this example, a frequency of 25 MHz is generated and is fed as input to the design through top-level clock port *clk0*.
   - To connect the triggers to the clock stop input of the CCM, manually instantiate the triggers and CCM clock stop modules in the RTL as shown in the example. You can define up to a maximum of 10 user triggers in the *idc* file. These triggers can be a combination of triggers from different IICE blocks, or user logic triggers.

Example 2 IICE Triggers

```
iice_trigger #("IICE_0") trigger_inst1(trigger_iice_0);
iice_trigger #("IICE_1") trigger_inst2(trigger_iice_1);
iice_trigger #("IICE_2") trigger_inst3(trigger_iice_2);
iice_trigger #("IICE_3") trigger_inst4(trigger_iice_3);
iice_trigger #("IICE_4") trigger_inst5(trigger_iice_4);
iice_trigger #("IICE_5") trigger_inst6(trigger_iice_5);
iice_trigger #("IICE_6") trigger_inst7(trigger_iice_6)
```

Example 3 User Logic Triggers

```
assign trigger_in[0] = trigger_iice_0;
assign trigger_in[1] = trigger_iice_1;
assign trigger_in[2] = trigger_iice_2;
assign trigger_in[3] = trigger_iice_3;
assign trigger_in[4] = trigger_iice_4;
assign trigger_in[5] = trigger_iice_5;
```

Feedback

```
assign trigger_in[6] = trigger_iice_6;
assign trigger_in[7] = gpio_1;
// User logic trigger assign trigger_in[8] = gpio_4;
// user logic trigger assign trigger_in[9] = gpio_7
// user logic trigger
```

- Connect the triggers to a wire called *trigger_in[9:0],* as shown in the example.
- Connect the triggers to a user GSV clock stop trigger module:

  ```
  ccm_gsv_clk_stop_user #(10) clk_stop_inst (trigger_in);
  ```

  The number 10 defines the trigger input parameters, and the current maximum is 10 triggers.

  The GSV clock stop trigger module is responsible for stopping all the CCM clocks to enable GSV capture. At runtime, use the clock module to select which of the 10 triggers to use to stop the clocks. You can also use it to combine different triggers by masking them with a MCAPIM register. By default, all triggers are ORed.

6. Run the GSV flow to generate the *.ll* file and bit files.
7. Enable the required stepping mode for readback.
   - Driver clock readback (default)—Readback happens by stopping or stepping the driver clock. Execute the following command to use this option:

     ```
     package require CCM CCM::open ../../ ccm stepping_mode -driver
     ```

   - CCM clock readback—Readback happens on the CCM generated clocks. Execute the following command to use this option:

     ```
     package require CCM CCM::open ../../ ccm stepping_mode
     {-pos_cclock_id | -neg_cclock_id}[CCM::cclock_id <clockID>]
     ```

     Details on CCM clocks in the design and its relative path is available in the *zebu_config.tcl* file under the *<exported_runtime_path>/system* directory.

     You can control whether the readback happens on the positive edge (*-pos_cclock_id*) or the negative edge (*-neg_cclock_id*) of the CCM clock. In the following example, the readback happens on the CCM generated clock (*clk0*) on positive edge:

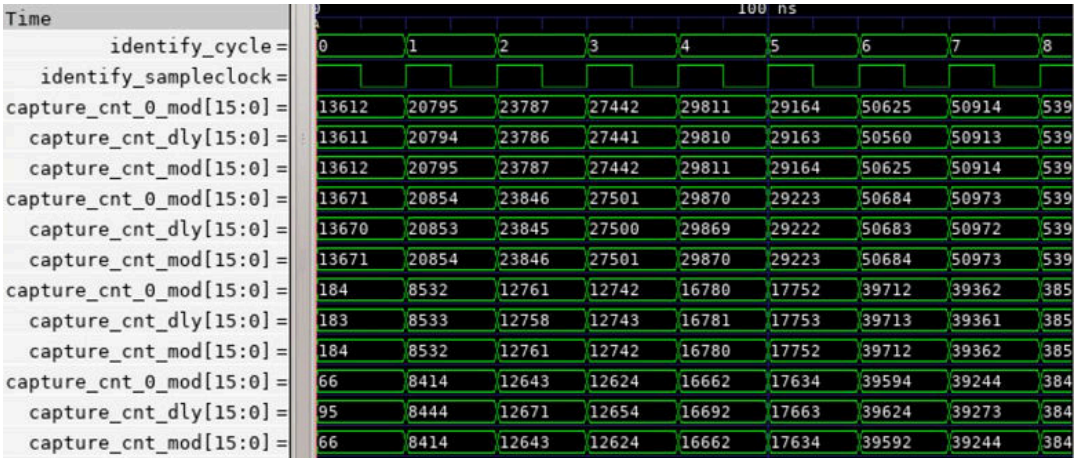
     ```
     ccm stepping_mode -pos_cclock_id [CCM::cclock_id cnt_demo_top.clk0]
     ```

     You can also enable or disable CCM clocks while performing the readback using the *-uncontrolled* option with the *ccm clock_mask* command. By masking the clocks, you can make sure the readback happens only on CCM clocks that are

Feedback

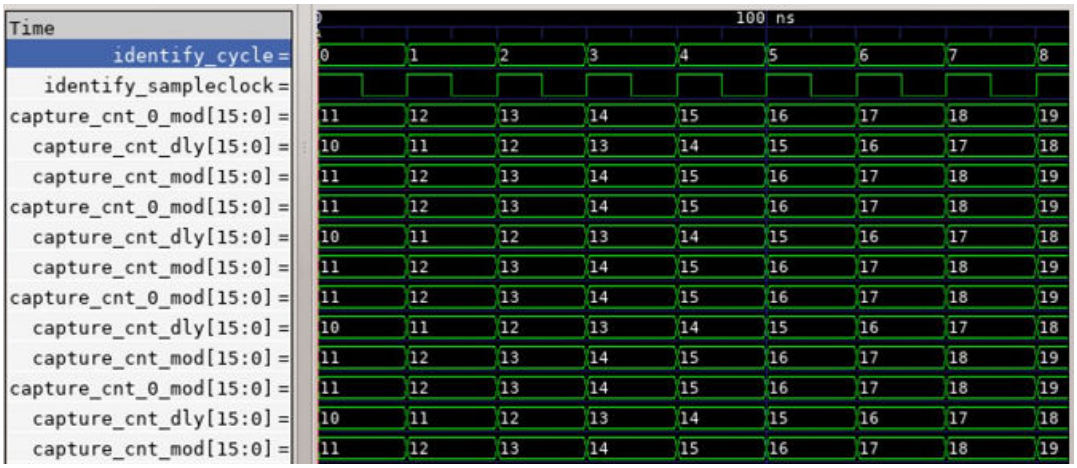
not masked. In the following example, the clock *cnt_demo_top.clk0* is disabled or masked:

```
ccm clock_mask -cclock_id [CCM::cclock_id cnt_demo_top.clk0]
-uncontrolled 1
```



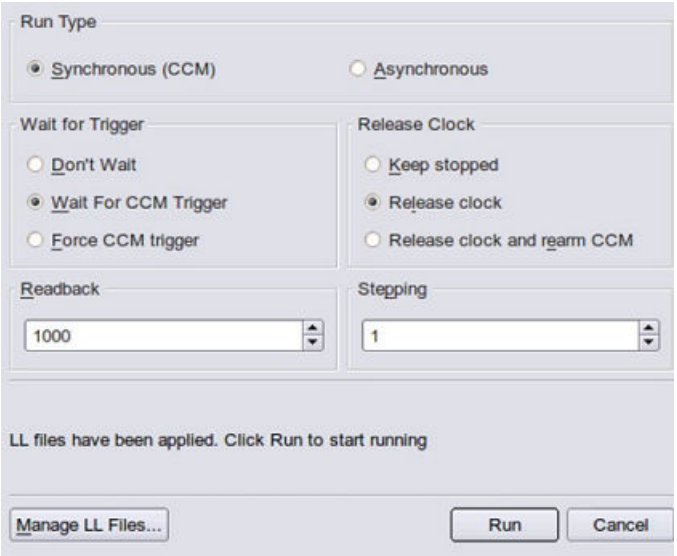
To unmask or enable the CCM clocks again, use the following command:

```
ccm clock_mask -cclock_id [CCM::cclock_id cnt_demo_top.clk0]
-uncontrolled 0
```



8. Use the regular runtime options in the GUI to stop clocks and perform readback by stepping the clocks.

Feedback



9. Use the waveform streaming to save time on writing the FSDB. For details, see the *readback stream_waveform* command in the *HAPS Prototyping Debugging Environment Reference Manual*.

Automatic Instrumentation of Driver Clock

The driver clock of the new clock generator is unavailable for use till the pre-partition stage of the design flow. You also need to manually instantiate the driver clock using hyperconnect in the RTL and use it as sampling clock in the IDC file.

As an enhancement, you can now automatically instrument the driver clock in the IDC file, without any manual instantiation of driver clock hyperconnect in the RTL.

IDC Syntax for Automatic Instrumentation

```
iice clock -iice {IICE_0} -edge positive -driver_clk
```

Consider the following:

- This feature requires the new clock generator flow to be inserted in the design as a prerequisite to use the driver clock instrumentation.
- The IDC command is supported for both system and post SLP level instrumentation (RTL or SRS) using the same syntax.
- This feature is not supported in any other clock generator or CCM flow.

IDC File Example

```
device jtagport umrbus3 device umr_pipe 1 iice new {IICE_0} -type regular
iice controller -iice {IICE_0} none iice sampler -iice {IICE_0} -depth
1000 iice sampler -iice {IICE_0} -pipe 3 iice clock -iice {IICE_0} -edge
positive -driver_clk
```

CCM Debug Flow Enhancements

The following are the debug enhancements with the clock generator flow:

- Supports automatic DTD link training during DTD and eGSV flow, when the sampling clock is not active.

Previously, a free running sampling clock was required for DTD link training to complete successfully. Now this has been enhanced to check if the CCM clock is running or stopped and complete the link training even if the clocks are not running.

To check if the clocks are running or stopped, see the link training log for these messages:

Controlled clocks are stopped

Controlled clocks are running

- Provides DTD trigger-detected information early using readback.

Previously, both the *Trigger detected* and *downloading samples*messages were shown after completing the download of samples from the DTD.

Now, in the log, the DTD Trigger detected message is displayed in the following sequence:

Reports below message after the trigger is detected:

DI242 IICE 'IICE_0' Trigger detected

Reports below message after samples are downloaded:

IICE 'IICE_0' downloading samples...

- Supports readback for very low frequencies in the range of 25 Khz to 50 Khz, with PCIe drivers.

Limitations

- CCM clock readback above 25 MHz is not recommended.
- GSV plus DPI flow does not support CCM clock readback.
- Clock generator for mixed system is supported only on HAPS-100 systems.

- Only OR combiner is supported for multiple trigger inputs to CCM.
- Phase shifting of clocks is not supported.

Fast Track Debug

If you plan to debug your design later in the HAPS ProtoCompiler debug flow, adding debug resources at that point of testing poses certain challenges:

- Since debug resources were not available in the original implementation, it requires changes leading to congestion, routing issues and timing violations and so on.
- To use the debug solutions, rerun the flow from post SLP (System Level Partitioning) pre_map for a single FPGA debug or rerun the whole flow for system debug. This means another N hours to wait for debugging the design.

To prevent such issues, the ProtoCompiler tool inserts the debug IP into design without the idc applied to the flow. The inserted debug IP must be visible in the ECO (Engineering Change Orders) flow and be able to connect the design probes with the regular IICE. Hence you can debug the design faster.

The features of fast track debug flow are:

- Auto Insertion of DTD IICE, if DTD IICE is not provided at FPGA or at system level.
- ECO of up to 8K instrumentation bits.
- Automatic cross-triggering between IICEs.
- Automatic sample clock selection.
- Automatic connection of fast track debug IICE to the clock control module (CCM), in CCM designs, so that the IICE can be used to stop controlled clocks for *readback*.

For more detail, see these links:

- Fast Track Debug Flow
- Always On Debug IICE
- Automatic Cross-Triggering
- Automatic Connection of Triggers to Clock Control Module (CCM)
- Limitations

Feedback

Fast Track Debug Flow

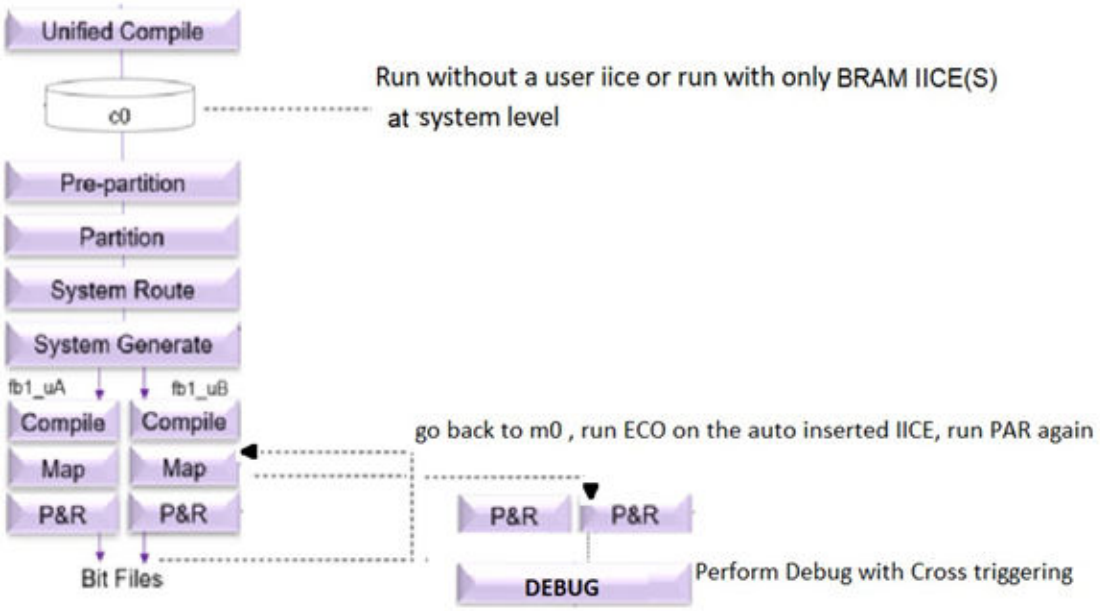
The fast track debug feature is enabled by default. To disable the feature use:

```
option set fast_track_debug 0
```

See the `fast_track_debug` option in option in *Command Reference*.

This option enables the following debug options:

- `option set debug_dumpvars 1`
- `option set debug_sva 1`
- `option set debug_utfprobe 1`
- `option set incremental_debug 1`
- `option set prepare_readback 1`
- `option set use_module_idb 1`



There are two scenarios to be considered in this flow:

Scenario - No IDC at System or Post SLP Level

In this scenario, the flow goes through from compile stage to system generate and post slp level compile without any insertion of IICE, since there is no idc provided at the system level or post SLP level.

1. At the pre-map stage, the auto insertion of the fast track debug DTD IICE takes place. The flow continues through map and place-and-route stages to generate the bit files.
2. Once the bit files are generated, revert to the post slp map state, do instrumentation of the incremental signals available.
3. Run debug ECO flow with the incremental idc and continue with place-and-route to generate the incremental bit files. See Applying Debug ECO Instrumentation in the Design.
4. Proceed to debug the newly generated bit file with hRun.

Scenario - IDC is Provided at System level or Post SLP level

If the idc is provided, then it has precedence over fast track debug. For example, if you insert the DTD IICE at system level or post SLP, the tool detects this and prevent insertion of fast track debug DTD IICE. If you insert the BRAM IICE, the tool continues with the fast track debug DTD IICE insertion.

Always On Debug IICE

ProtoCompiler automatically inserts a static debug IP logic to prepare the design for instrumentation. You can use Always On Debug (AOD) IICE to instrument the design either during the initial prototyping run or as part of incremental instrumentation.

This Fast Track Debug (FTD) methodology allows the user to do incremental instrumentation with the ECO using the AOD IICE.

HAPS Target	AOD IICE Instrumentation bits capacity	
	Max Sampling Frequency	Max Number of Instrumented bits
HAPS-100	40 MHz	2K
HAPS-200		8K

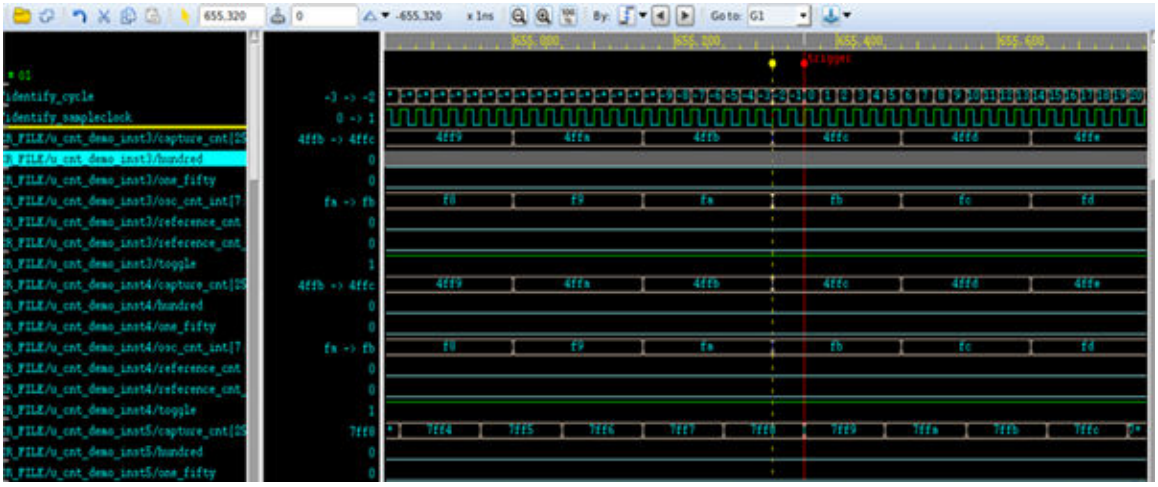
- Type of IICE inserted - DTD comes as best solution as most of the IP is in assistant FPGA and you can instrument thousands of signals and have exceptionally large depth as well.

  You can instrument up to 8K signals per FPGA at 20 MHz max sample clock. More width is more intrusive on user design.
- IICE configuration - More conditions require more HT3 traces for the sync tree, because of that state machine with 4 conditions as default value is chosen. This provides enough flexibility for initial debug.

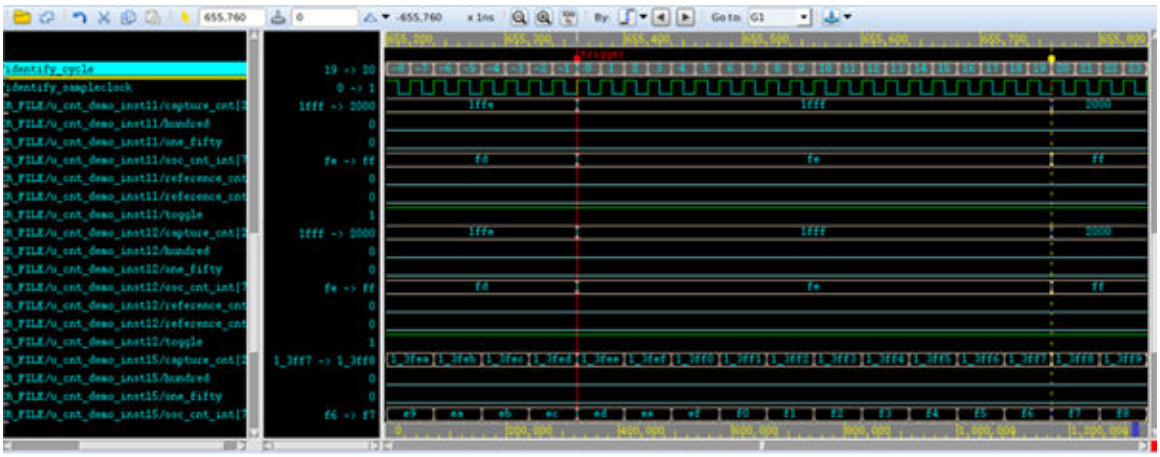
Feedback

- Sample Clock Selection:
  - Designs with CCM – Sample clock is a driver clock and the controlled clock selection feature is enabled by default. You can use any controlled clock as a sampling clock.
  - Designs without CCM – If the design does not have new CCM along with the DTD IP sampling is done using a high frequency oversampling clock. The signals are sampled and aligned with respect to the oversampling clock and then downsampled to 20 MHz to match the DTD at 8K bit instrumentation frequencies.

  The following waveforms can be used to understand oversampling. This example is a multi-clock single FPGA design with 16 instantiations of counter in which,
  - instances 1 to 4 are signals with userclock 5MHz,
  - instances 5 to 8 are signals with userclock 8MHz,
  - instances 9 to 12 are signals with userclock 2MHz,
  - instances 13 to 16 are signals with userclock 20MHz



Feedback



Automatic Cross-Triggering

If fast track debug is not enabled, the RTL cross-triggering is enabled by inserting the IICE blocks in RTL and then instrumenting the cross-trigger signal `trigger_out` of an IICE into other IICEs.

If fast track debug is enabled, automatic cross-triggering is enabled for each IICE. Hence the cross trigger signal is instrumented automatically and any IICE can trigger another IICE. You can also cross-trigger between DTD or BRAM IICEs within the same FPGA. Here all the IICEs are automatically connected to the cross-triggering network, if you introduce any system or post SLP IICEs using its own idc. Max number of cross trigger connections is 16 per FPGA.

The `ALWAYS_ON_DBG_IICE` is automatically inserted with the state-machine triggering mode which also has a cross trigger signal instrumented automatically.

Cross trigger signal — `modflow_ctrigger_out_MTRIG`

Cross-Triggering Procedure

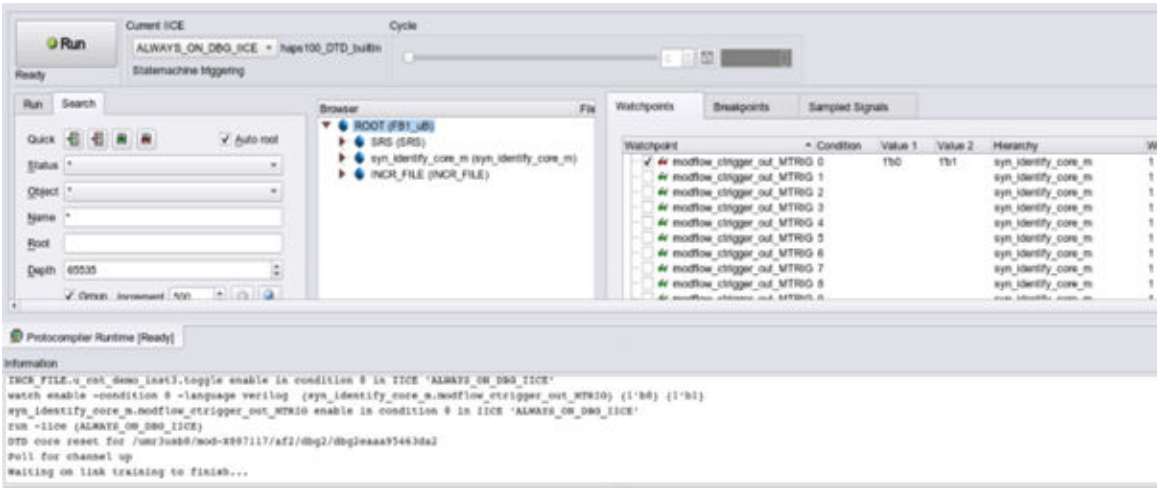
The following procedure describes how to do cross-trigger between the system level IICE and fast track debug DTD IICE, during runtime:

1. After successfully programming the hardware, launch the `protocompiler100_runtime` separately for both the IICEs.
2. Open the respective debug projects for both the IICEs.

   Since the `modflow_ctrigger_out_MTRIG` signal is automatically instrumented, it is seen in `Watchpoint` window.

Feedback

3. In Runtime window of fast track debug DTD IICE, configure the state machine trigger and set the cross-trigger signal `modflow_ctrigger_out_MTRIG`:



4. Run the link training for fast track debug DTD IICE. Once the link training is complete, run the IICE.

   It waits for the trigger.

5. In the other Runtime window of the system-level IICE, set the signal to be triggered and run the IICE.

   The trigger gets detected in both IICEs and samples captured. Now the waveform can be viewed with respect to cross trigger signal nd proceed with debug.



Note the following:

- There is a delay during cross- triggering; the `modflow_ctrigger_out_MTRIG` signal triggers after a few clock cycle delays, in the captured IICE.
- The procedure described is used for cross triggering the following IICEs:
  - System level IICE can trigger a FPGA IICE
  - FPGA IICE can trigger a system level IICE
  - FPGA IICE can trigger another FPGA IICE

  For cross triggering system-level IICEs only, that is, IICE units in the same partitioned FPGA, use the method described in Setting up an IICE for Cross-Triggering.

Automatic Connection of Triggers to Clock Control Module (CCM)

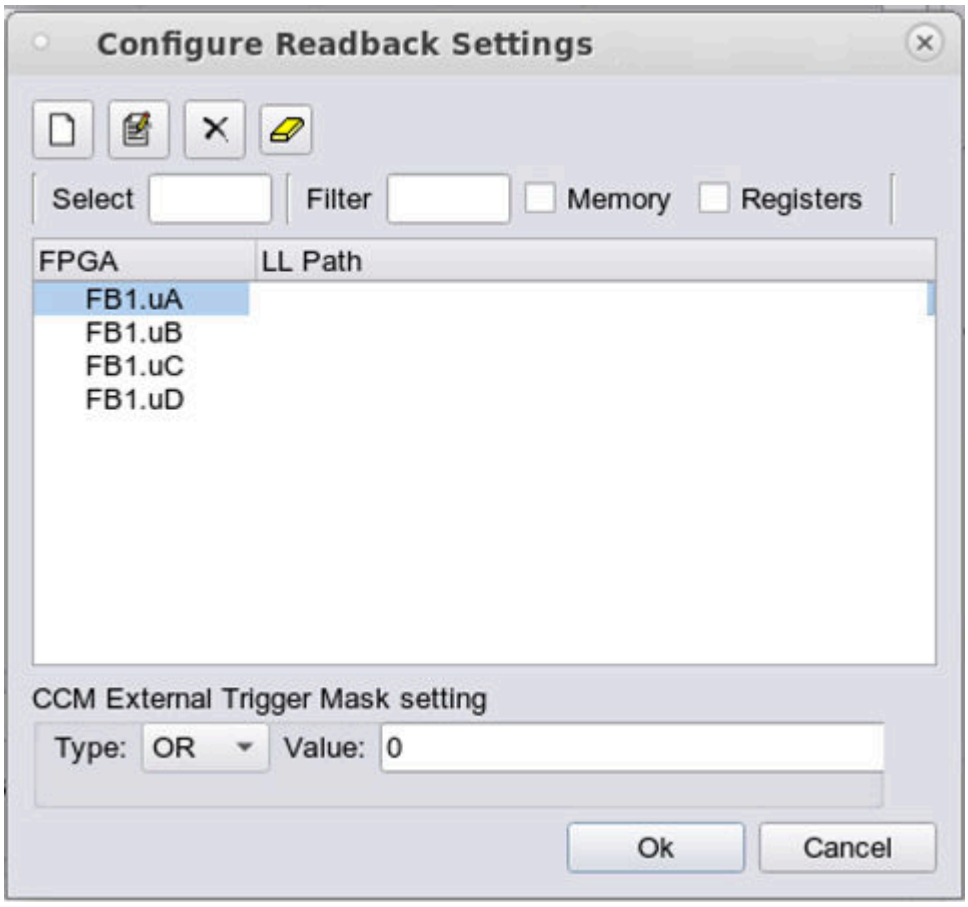
In designs using Clock Control Module (CCM), if fast track debug is enabled, fast track debug IICE is automatically connected to the CCM and it is used to stop controlled clocks and perform *readback*.

For other IICEs, you must instantiate `iice_trigger` instances into the RTL to connect the IICEs to CCM as described in Configuring CCM V3 Clock Generator.

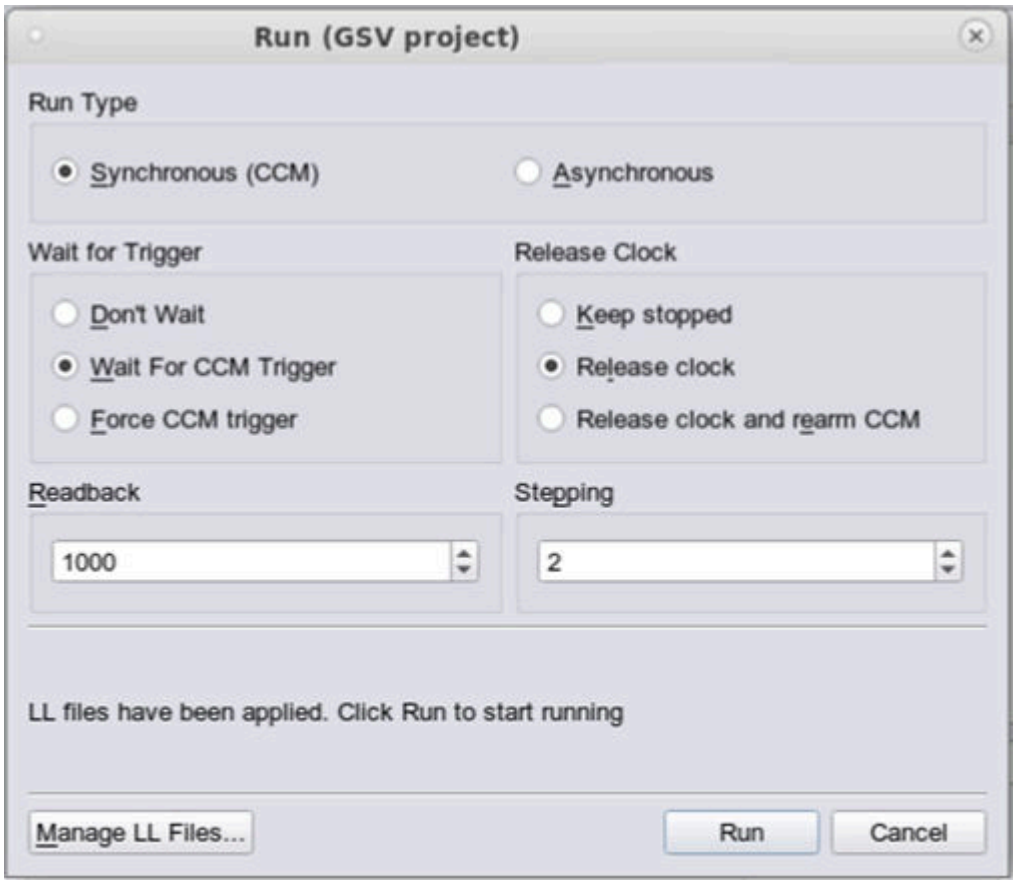
The Hardware Flow

The following procedure describes how to use fast track debug IICE to stop controlled clocks and perform *readback*.

1. After successfully programming the hardware, launch one Runtime window for the *readback* project at `<export directory>/system/readback/debug.prj`and another Runtime window for the fast track debug DTD IICE.
2. In the *readback* project, check the *readback* settings and make sure that the LL files are loaded correctly.



3. Run the *readback* IICE with the required settings. On this dialog, enable Wait for CCM trigger to use fast track debug IICE for external trigger.



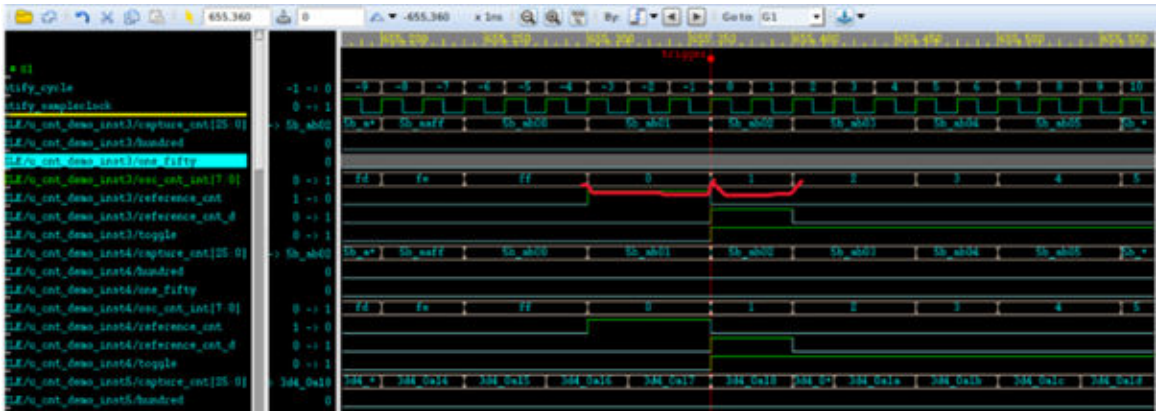
4. In the fast track debug DTD IICE Runtime window, configure the fast track debug DTD IICE state machine and choose the required trigger condition and run the IICE.
5. The trigger is detected in both the fast track debug IICE and the *readback* IICE and the samples are captured.

Limitations

- The fast track debug ECO flow is supported for instrumentation only upto 2k bits per FPGA.
- Fast track debug IICE with ECO only works at the FPGA level.
- The fast track debug flow in CCM designs requires that the clocks are generated locally in each FPGA.
- Oversampling does not support clock frequencies beyond 20MHz. Hence a frequency checker is implemented which issues a warning in the Runtime with the clock driver name and the list of affected registers if the clock exceeds the supported range.

- In the oversampling scenario, there is no alignment between waveforms of different FPGAs. That is if logic clocked with a global clock is sampled on different FPGAs and even if logic is synchronous in the system, there is no alignment ensured between waveforms for this logic.
- In oversampling, since the output waveform is shown with respect to 40MHz clock, for a clock with frequency of X MHz, if the remainder of 40 MHz / X MHz is non-zero, then the period shown in the waveform dump might have a variance of 1 cycle at 40 MHz because that frequency cannot fit within the period of 40MHz. No information is lost in the obtained waveform and the alignment of signals from the same source clock driver is consistent.

  For example, the following waveform shows period variance with a clock frequency 15.62MHz, which is not exactly divisible by 40MHz.



- During cross-triggering between fast track debug IICE and post SLP DTD IICE on another FPGA at lower frequencies (~100kHz), it is expected to see two pulses at the MTRIG signal.
- If there are not enough traces in the TSS file to route the cross-triggering network, the cross-triggering signal is not instrumented.

Enabling Clock Selection With DTD/Readback

You can dynamically select the sampling clock and the edge of interest during runtime. You can switch dynamically between positive, negative or both edges of the controlled clocks during runtime, for DTD buffer type and for readback (GSV/eGSV).

Clock selection is supported for DTD IICE and Readback, but not supported for BRAM buffer type.

1. To enable clock selection, execute the following command or define the constraint in the IDC.

   ```
   iice clock -iice <iice_name> -driver_clk -use_controlled_clks {0|1|2}
   ```

   Specify:

   - *0* (default): Cannot select controlled clocks.
   - *1* to enable controlled clock selection on positive and negative clock edge.
   - *2* to enable controlled clock selection and dual edge (positive, negative and both edge) sampling.

2. During runtime, you can select the sampling clock and the edge by specifying the following command in the *hRun.tcl* file.

   ```
   iice clock [-edge <positive|negative|both>] [-iice <name>] [-select
   <driver_clock|<cclock path>] [-stepping <value>]
   ```

- `[-edge <positive|negative|both>]`: Specify the sample clock edge. For driver clock and composite clock, you can specify only positive edge.
- `[-iice <name>]`: Specify the name of the IICE unit to configure (includes 'all'). If the name is not specified, the option returns the name of the IICE clock.
- `[-select <driver_clock|<cclock path>>]`: Specify the sample clock. Default: *driver_clock*.
- `[-stepping <value>]`: For Readback IICEs, specify the number of sample clock events between each cycle.

To list the available controlled clocks in the design at runtime, specify the `iice clock` command without specifying any parameters in the *hRun.tcl* file.

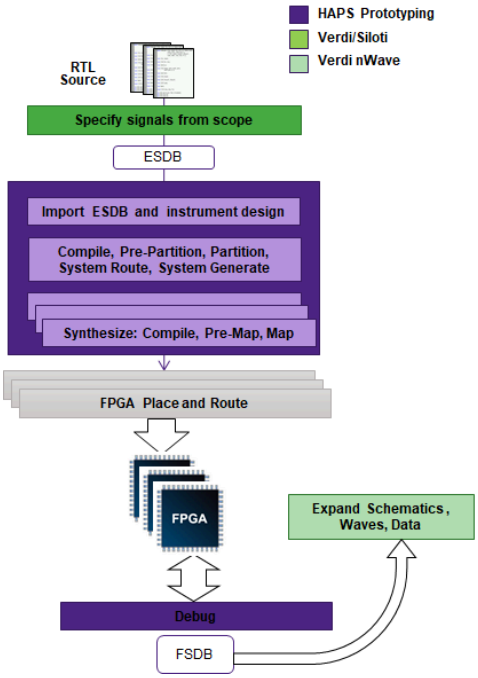
Integrated Debugging with Verdi Software

You can incorporate functionality from the Synopsys Verdi® automated debug system with its powerful data analysis capabilities into the HAPS ProtoCompiler debug flow. Verdi features include waveform viewing, comparison functionality, and waveform correlation to RTL. Use the Verdi capabilities in the prototyping debug flow to take full advantage of deep sample buffers. You can also combine the Verdi Siloti capabilities with the instrumentor to extract only the essential signals from a targeted part of the design for instrumentation and debugging.

The following figure illustrates the integrated Verdi flow. In it, Verdi/Siloti generates an Essential Signal Database (ESDB) which is imported into the instrumentor for signal instrumentation. The instrumented design is then synthesized, placed and routed, and programmed into the FPGA. The debugger samples the data and writes out a Fast Signal

Feedback

Database (FSDB) of the debugged signals, which is then displayed in the Verdi nWave viewer. You can then use the generated FSDB for data expansion and waveform viewing, and to compare and correlate the waveform to the RTL.



Note these points before you start using the flow:

- Integrated debugging with Verdi Software is supported only on Linux.
- HAPS-Verdi flow does not support SV interfaces.
- Do not reuse the correlation database (CRDB) generated using an earlier version of Verdi. Use the recommended version of Verdi to generate CRDB and to export DE.

For more detail, refer to these links:

- Using Verdi and Siloti for Debugging
- Instrumenting Verdi Signals (ESDB)
- Generating the Fast Signal Database (FSDB)
- Debugging with the Waveform Viewer
- Using the Verdi Flow with a UC Design

Launch Verdi

The following steps describe how to launch Verdi:

1. Load the original *.ll* file from Vivado.
2. Once the samples are downloaded, generate the FSDB file using the following command:

   ```
   write fsdb <filename> -iice <value>
   ```

3. Once the FSDB file is generated, launch Verdi.

For details on the commands, see `launch verdi`in the *HAPS Prototyping Command Reference* manual and `launch_verdi`command in the HAPS Prototyping Debugging Environment Reference manual.

Limitations

- Correlation for VHDL constructs is sub-optimal, so data expansion percentage is low for VHDL, approximately 70-80%.
- SystemVerilog interfaces are not supported in HAPS Verdi flow.
- Data expansion is not supported on designs with inferred shift register that are fractured.
- Data expansion is not supported for the internals of SystemVerilog interfaces, DesignWare, and VHDL generates.
- The maximum number of hierarchies data expansion can handle is 50.

The following topics describe how to launch the Verdi tool from the command line and GUI.

- Launching Verdi Through Commands
- Launching Verdi from the HAPS-200 ProtoCompiler GUI

Launching Verdi Through Commands

1. Make sure the environment variables for *VCS_HOME, VERDI_HOME* and *XILINX_VIVADO* are set correctly.
2. After the FSDB file is generated, start the Verdi software with the *launch verdi*command specifying the EDIF or Verilog flow, according to what was used for place and route.

EDIF example:

```
launch_verdi -run_dir debug_data -fsdbtype edif
```

Feedback

Verilog example:

```
launch_verdi -run_dir debug_data -fsdbtype verilog
```

Launching Verdi from the HAPS-200 ProtoCompiler GUI

1. Open the HAPS-200 ProtoCompiler tool and select **State > Verdi RTL Debug** from the menu.

   The **Verdi RTL Debug Setup** window is displayed.

2. Configure the following parameters in the **Required Setup** section:

Table 7 Verdi RTL Debug Setup: Required Setup Parameters

Parameter	Description
Run Directory	Absolute path to the runtime directory.
VCS_HOME	To ensure compatibility, configure the same VCS_HOME path that was used to generate the UC database. All users must have access to RTL simv.daidir directory from their location.
VERDI_HOME	To ensure compatibility, configure the same VERDI_HOME path that was used to generate the CRDB. Change in versions can cause incorrect results during data expansion.
XILINX_VIVADO	Configure the path to Xilinx Vivado tools.

If *VERDI_HOME* and *XILINX_VIVADO* are not set correctly, these fields are left blank. By default, the tool searches for the UC results (*VCS simv.daidir*) in the current database. It is recommended that you do not override this default database that is generated by the tool.

3. Configure the following parameters in the **Miscellaneous Options** section:

Table 8 Verdi RTL Debug Setup: Miscellaneous Options Parameters

Parameter	Description
UC RTL simv.daidir	Path to the VCS simv.daidir directory.
On Demand RTL Loading	Select this checkbox to enable on-demand RTL loading.
Explore RTL	Select this checkbox to enable RTL exploration.

Parameter	Description
Data Expansion Force Value File	Specify the force value file. This file is used for incremental debug without changing FSDB. ◦ Forces constant values on specific signals through a file <RTL_bit_path> => 1‘b0. ◦ Data expansion uses the forced values.
Data Expansion RTL Signal List File	Specify the RTL signal list file for data expansion only on those signals.
Include Combinational Signals	Select this checkbox to include combinational signals.
Data Expansion FSDB Scope	Specify the scope for data expansion.
Data Expansion FSDB Levels	Specify the level for data expansion if the scope is provided.
Select FPGA for Debug	Select the FPGAs for vertical sliced debug. Debugs the scope mapped to the specific FPGAs with its FSDB.

4. Configure the following parameters in the **FSDB** section:

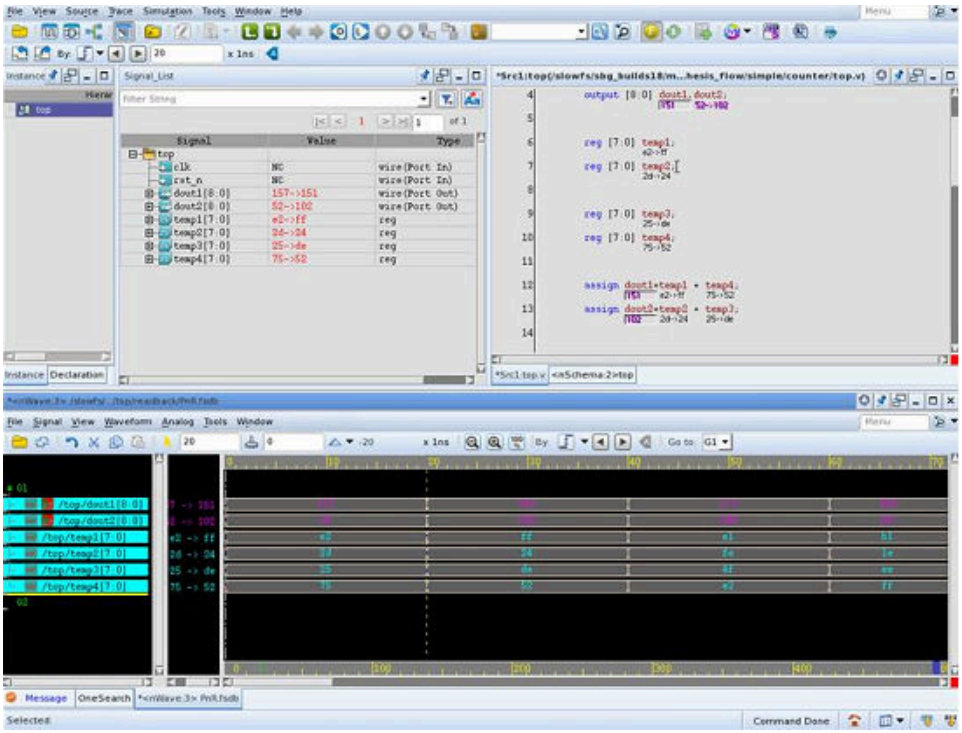
Table 9 Verdi RTL Debug Setup: FSDB Parameters

Parameter	Description
FSDB Files	Specify the path to the generated FSDB files.
Top Module	Specify the top module name of the FSDB.
Type	Select the netlist type generated by the ProtoCompiler tool (input format used for place and route): ◦ *edif* ◦ *verilog* For example, if you have used the command *option set write_verilog 1*, then use *launch verdi -run_dir ../verdi -fsdbtype verilog*.
Only Run Data Expansion	Select this checkbox to run only data expansion.
Expand Top Level Signals	Select this checkbox to enable automatic data expansion of top-level signals
Data Expansion for Cycle Shifted Registers	Select this checkbox to enable automatic data expansion with cycle shifts for retimed registers.

Feedback

5. Click **Export DE Setup** to generate folders that allows multiple users to perform data expansion from the same design correlation database. The tool creates a folder in the user specified path and copies correlation database (CRDB) along with necessary scripts to launch data expansion. See *export_verdi_de* for command line syntax information.
6. Click the appropriate button to start the tool.
   - Click *Launch Batch* to open the Verdi tool in batch mode and run the flow
   - Click *Launch GUI* to start the Verdi GUI and run the flow.

     The Verdi GUI opens and automatically displays the data expansion for the RTL signals.



Using Verdi and Siloti for Debugging

The following procedure describes the steps illustrated in the diagram in Integrated Debugging with Verdi Software for Standard Compile.

► Set up the project.

- Connect the HAPS hardware system to a Linux machine.
- Ensure that the Verdi software is installed on a Linux machine.

Feedback

- Configure the Verdi license:

Binary File
setenv VERDI_HOMEPathToNovasInstallationset path = ($VERDI_HOME/bin $path)
License File
*setenv NOVAS_LICENSE_FILElicense_file:$NOVAS_LICENSE_FILE*Alternatively use *LM_LICENSE_FILE* or *SNPSLMD_LICENSE_FILE*

The tool searches for licenses in this order: *SNPSLMD_LICENSE_FILE, NOVAS_LICENSE_FILE, LM_LICENSE_FILE*

► Generate the Essential Signals Database (ESDB).

The ESDB comprises the essential set of signals you want to capture for debug visibility into the block or area of interest.

- Load the RTL design in the Verdi tool.
- Run behavioral analysis in the Verdi software. Use the visibility analysis engine to determine the essential set of signals you need for later processing with the data expansion engine.
- Generate ESDB by specifying this command: *run_generate_esdb*

You can also use the *vericom* and *esa* commands to specify the working scope and the directory. This example generates the ESDB for the specified working scope, which is the top module P_*lddt_Struct_tbtop*, and saves it in a directory called *es*.*esdb*++. The essential signal database is called *es*.

```
vericom -2009 -sverilog +v2k -lib work -f files.f -incdir ./../src
+incdir+./../src

esa -lib work -top P_lddt_Struct_tbtop -db es -bas P_lddt_Struct_tbtop
-libPath work -bdb_file work
```

► Start the prototyping software and instrument the ESDB signals.

See Instrumenting Verdi Signals (ESDB) for details.

► Use hRun to download the design and generate the fast signal database.

See Generating the Fast Signal Database (FSDB) for details.

► Export files for Verdi debugging.

See Exporting Files for Verdi for details.

► Use the Verdi waveform viewer to debug the design.

See Debugging with the Waveform Viewer for details.

Instrumenting Verdi Signals (ESDB)

You can import signals from a Verdi database directly into the instrumentor by following the steps.

1. Generate the essential signal database (ESDB) as described in Using Verdi and Siloti for Debugging
2. Start the prototyping tool, and instrument the design (*run pre_instrument* and *edit idc -mode* commands).
3. Parse the essential signal list from the ESDB using this command:

```
verdi getsignalsESDBpath
```

ESDBpath is the location of the *es.esdb++* directory created previously. For example: *verdi getsignals myDir/es*

This command extracts the signals from the ESDB.

► Instrument the essential signal list using the *verdi instrument*command.

This command instruments the essential signals. The signals are automatically instrumented as sample and trigger.

► Instrument the sample clock.

Consult the built-in instrumentor documentation for details about instrumenting and configuring IICE.

► Configure the IICE and instrument the design.

You can now return to the procedure described in Using Verdi and Siloti for Debugging, and complete the FPGA implementation and generate the bit file.

Generating the Fast Signal Database (FSDB)

Once you have instrumented the Verdi database and implemented the FPGA, use the debugger to sample the data and generate the fast signal database (FSDB).

To generate the FSDB database, follow this procedure:

1. Instrument the Verdi database and implement the FPGA as described in Using Verdi and Siloti for Debugging.
    - Use the generated bit file to program the FPGAs on the HAPS system.

2. Start hRun and execute the following command to generate the FSDB.

```
<fid> dump fsdb [-iice <value>] <file>
```

After the trigger occurs, the tool generates an FSDB file for the captured samples of the essential signals, and saves it in the current working directory. The next step is to export the files (see Exporting Files for Verdi) so that you can display the results in the Verdi nWave viewer.

Exporting Files for Verdi

► Generate an FSDB database.

See Generating the Fast Signal Database (FSDB) for details.

1. Set the *verdi_mode* option to 1.
2. Run the *export runtime* command and specify the FSDB database.

The *export runtime* command exports all scripts, setup and correlation data needed to run Verdi. For the complete syntax, see export runtime in the Command Reference.

This is an example of the commands used:

```
option set verdi_mode 1

run compile -filelist addFilessynth.txt

run pre_map run map export runtime -path <fsdbFilename>

launch verdi -fsdb <fsdbFilename>
```

► After exporting the files, you can launch Verdi and use the waveform viewer for debugging.

For details, see Debugging with the Waveform Viewer.

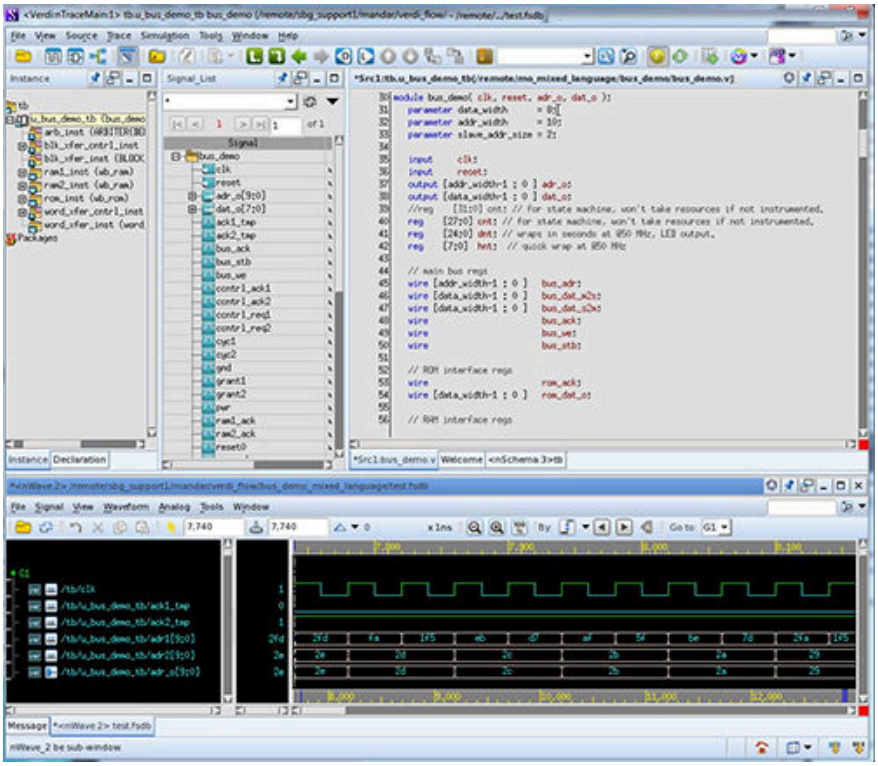
Debugging with the Waveform Viewer

You can use the Verdi nWave waveform viewer for data expansion and debugging once you have generated the FSDB.

1. Export the files, as described in Exporting Files for Verdi.
2. Launch the Verdi software from the shell.

```
launch_verdi -fsdb <filename>
```

Feedback

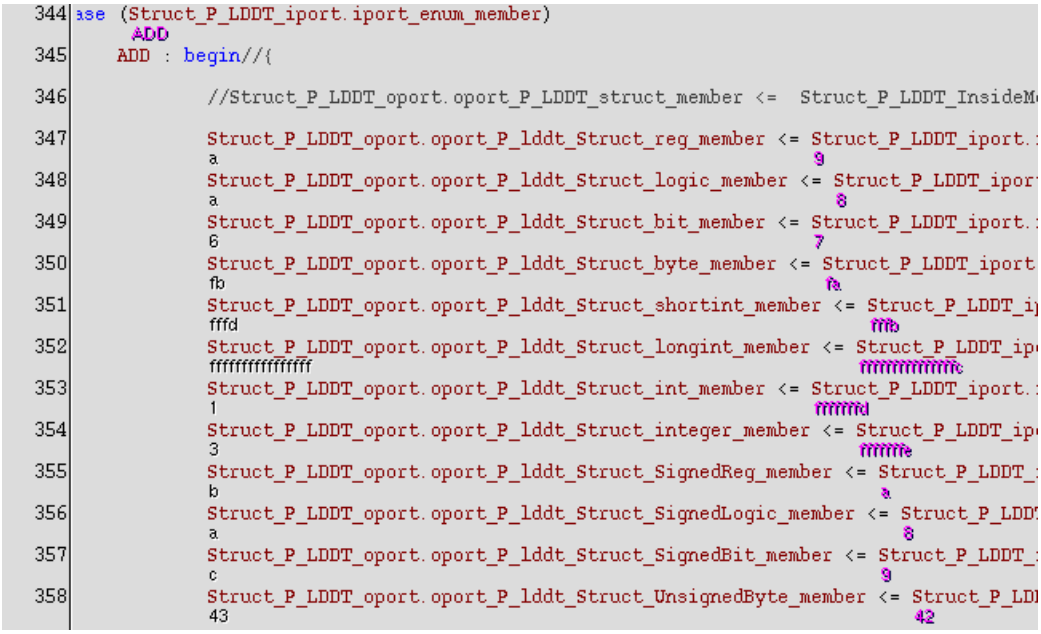


► Open the nWave viewer and view signals that are not in the FSDB with the Data Expansion engine.

- Click the *New Waveform* toolbar icon ( ) in the main Verdi window
- Click the *Get Signals* icon ( ) or select *Signal -> Get Signals* in the nWave window.
- In the *Get Signals* hierarchy browser pane, click the + icon to expand items. The signals in bold are signals that are not in the FSDB, and which can be expanded (calculated on the fly) by the Data Expansion engine.
- Select the signals you want to view in the waveform viewer and click OK to add them to the waveform. Drag left to select several signals; make sure to include the top-level I/O port.

Feedback

The values for all signals are now annotated. The ones in purple are expanded signals which did not come from the FSDB file, but were calculated. This is an example of the results:



Using the Verdi Flow with a UC Design

The Verdi software is integrated with the prototyping software to provide a seamless design debug environment to validate UC designs. The prototyping tool generates the required scripts, and debug environment, which provides a "push-button" way to load the design and debug with the Verdi viewer.

Use the Verdi tool in conjunction with the Global State Visibility (GSV) functionality. GSV leverages the Xilinx readback feature and provides a snapshot of all registers in a configured design at a particular clock cycle, thus offering full visibility. It does not require instrumentation and is instantly available. There are two GSV modes: legacy GSV and enhanced GSV (eGSV). The eGSV mode is implemented only when the design is instrumented but there are no other changes in the use model.

With this approach, you can perform Verdi data expansion using the FSDB that is generated from the GSV flow using the Vivado *.ll* files.

The application also checks the tool version compatibility and prints the message in the *novas.log* file. The tools prints an error if the versions are not matching.

```
#######################
Current running Verdi version: U-2023.12
Verdi version used to generate CRDB: T-2022.06-SP1
```

Feedback

```
HAPS VCS Version Data: VCS S-2021.09-SP2-3_Full64
Error: Current running Verdi version: (U-2023.12) does not match Verdi
 version used to generate CRDB: (T-2022.06-SP1)
*****************************************************************
        End Tool Version Compatibility Report
*****************************************************************
Error: Version compatibility check failed, DE Setup cannot be completed.
Info: DE setup fails.
########################
*****************************************************************
        Begin Tool Version Compatibility Report
*****************************************************************
Current running Verdi version: U-2023.12
Verdi version used to generate CRDB: T-2022.06-SP1
HAPS VCS Version Data: VCS S-2021.09-SP2-3_Full64
Warning: Current running Verdi version (U-2023.12) does not match Verdi
 version used to generate CRDB (T-2022.06-SP1)
*****************************************************************
        End Tool Version Compatibility Report
*****************************************************************
```

See these topics:

- Running the Verdi Flow For Designs with GSV
- Running the Verdi Flow For Designs with Instrumented/Probe Signals
- Troubleshooting Verdi Flow Issues
- Running Formal Verification with Checkpoints

Running the Verdi Flow For Designs with GSV

The following steps describe how to implement the design and export the Verdi debug environment for designs with GSV. See the Release Notes for the compatible versions of VCS and Verdi.

1. Set the following options before executing the synthesis or prototyping design flow:

   ```
   option set verdi_mode 1
   option set prepare_readback 1 | 2
   ```

   The *readback* option is only required for debug with GSV. Set *prepare_readback* to *1* for GSV and to *2* for eGSV. Setting this option generates the logic location (*.ll*) files when the bit files are generated. The *.ll*file contains mapped register information from the FPGA to design signal locations, and GSV uses these signal information.

2. Compile the design using the Unified Compile flow (*launch uc* and *run compile*). The following are some of the recommended settings:
   - Preserve registers at both synthesis. and place and route using the directives *syn_preserve* and *pnr_preserve_regs*.
   - Map memories and DSPs to its logic if it is going to be a part of GSV.
   - Disable new DSP inference using the directive *option set new_dsp_inference_beta 0*.
   - Disable retiming for a specific instance, if required. Retiming is not supported for the GSV flow.
3. Continue with the design flow through place and route.

   Select the logic location (*.ll*) files generated by Vivado from the respective place-and-route folders when the GSV operation runs during runtime (see step 5). You can then launch Verdi from runtime, check correlation in the Verdi-generated *crdb_summary.log* file and monitor the data expansion results.
4. Run the *export runtime* command to create the Verdi database.

   ```
   export runtime
   ```

   The command creates a *readback* folder and a *verdi* folder that contains all the scripts required to run the Verdi.
5. Run debug and capture GSV.
   - Start debug and launch the Verdi software. Details are described in Launch Verdi.
   - Open the readback project.
   - Click the **Readback** icon () to open the dialog box. Load the GSV IICE.
   - Load the *.ll* files, using the Readback IICE dialog box. FPGA names must follow the naming convention for individual FPGAs.
   - Click **Run** and capture the GSV in an FSDB file.

     ```
     write fsdb <fsdbName>
     ```

Make sure not to edit or modify scripts generated during the flow.

Feedback

Running the Verdi Flow For Designs with Instrumented/Probe Signals

You can export instrumented or non-instrumented designs to the Verdi environment for debug. The following steps describe how to implement the design and export the Verdi debug environment. See the Release Notes for the compatible versions of VCS and Verdi.

1. Set the following option before executing the synthesis or prototyping design flow:

   ```
   option set verdi_mode 1
   ```

2. Compile the design using the Unified Compile flow (*launch uc* and *run compile*).
   - Identify the essential signals using the *report debug_essential_signals* command (report debug_essential_signals in the *Command Reference*). Include the signals in an IDC file, so that they are visible to the Verdi tool.
   - Continue with the regular design flow through place and route, with or without instrumenting the design.

3. Run the *export runtime* command to create the Verdi database.

   ```
   export runtime
   ```

   The command creates a *verdi* folder that contains all the scripts required to run the Verdi.

4. Run debug with Verdi.
   - Start debug and launch the Verdi software. Details are described in Launch Verdi.
   - Enable the trigger condition for GSV capture.
   - Run Readback capture.
   - Capture the FSDB waveform.

     launch verdi -fsdb
   - Check the Verdi-generated *crdb_summary.log* file for correlation.
   - Monitor the data expansion results.

Troubleshooting Verdi Flow Issues

You can refer the following log files to troubleshoot issues related to the Verdi flow:

- *stdout.log*—For issues related to CRDB generation.
- *verdiLog*/*novas.log*—For issues related to DE.

Feedback

Running Formal Verification with Checkpoints

This verification flow is based on the creation of check points, which can be compared to the original Verilog using the Formality or VC Formal (VCF) tool to run formal verification. Refer to the Formality and VCF documentation for details on how to use those tools.

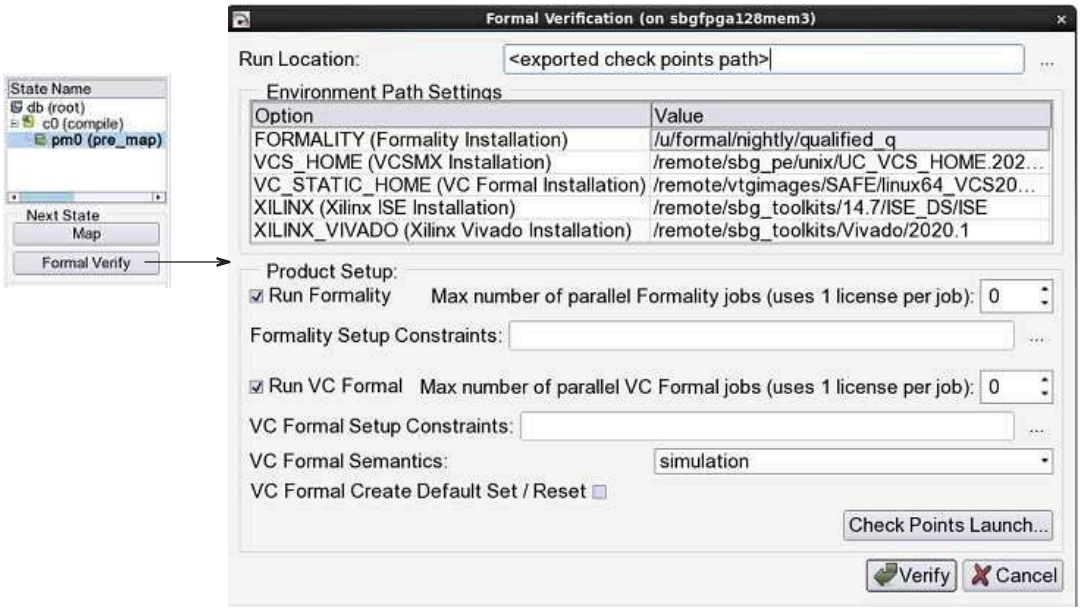
1. Set environment variables for the verification tools:
   - Set the Formality variable (*FORMALITY*).
   - Set the VCS variable (*VCS_HOME*).
   - Set the VCS Static variable (*VCS_STATIC_HOME*).
   - Set the Xilinx Vivado installation (*XILINX_VIVADO*).
2. Set verification options from the GUI or the command line:
   - Enable verification mode: *option set verification_mode 1*. In the GUI, set *Verification Mode*.
   - Enable the creation of checkpoints: *option set verify_check_points 1.* In the GUI, set *Verification Check Points*. When this option is enabled, the tool automatically creates checkpoints based on the design modules (SCMs).

   After the compile stage, the tool writes out checkpoints for the modules. Each verification check point contains the following data that the verification tools can use:
   - Complete environment setup, formal tool setup. and launching data
   - Complete reference and implementation Verilog designs, and other data
   - Formal tool constraints such as clocks, black boxes etc.
   - Guidance and matching data for achieving maximum formal convergence
   - Interface for controlling various capabilities of specific formal tools based on the check point requirements
   - Report generation mechanisms
   - Handshaking mechanisms between the prototyping and verification tools
3. Export files for formal verification.
   - Go to a valid database state, such as compile or pre-map. For a list of the database states, refer to export verification.
   - Run the *export verification* command.

     ```
     export verification -check_points -path dir
     ```

Feedback

4. Open the dialog box for formal verification options.
   - From the pre-map or other valid stage in the GUI, click *Formal Verify*, as shown in the next figure.
   - To use the command line, specify the *formal_verify* command from a valid database state. For the syntax of this command, see formal_verify.



5. Specify options for, and run formal verification with checkpoints.
   - Enable the verification tool to use for the formal verification run. You can choose Formality, or VC Formal, or both. See Selecting a Verification Tool for Checkpoint Analysis for some guidelines.
   - Specify options for the verification tool. For Formality, specify the number of parallel jobs and a file with setup constraints. For VCF , specify the maximum number of parallel jobs, setup constraints file, default set/reset, and semantics mode. See Selecting a Verification Tool for Checkpoint Analysis for more information.
   - Click *Check Points Launch.* This runs verification and opens a window with the results.
6. Analyze results in this window.

   See Analyzing Checkpoint Verification Results.

Feedback

Selecting a Verification Tool for Checkpoint Analysis

Both the VC Formal and Formality tools can be used for checkpoint analysis. The table compares some criteria for the two tools.

VCF	Formality
Use for all sequential equivalence checking, especially for GCC.	Use for combinational equivalence checking.
Use for global prep, because it can handle cross-group, cross-iteration optimizations for distributed synthesis.	Not as well suited for global prep, because it does not handle cross-group optimizations.
Requires several machine resources for formal convergence.	Requires machines with larger memory; possible long runtimes.

- VCF Options
  - Specify the number of formal workers for each VC Formal run. The recommendation is to specify a minimum of 50 workers for GCC verification. Specify an optional setup constraint file with additional information. This in an example of typical commands included in this file:

    ```
    set_grid_usage -type rsh=50
    set_fml_var fml_max_time 24H
    set_fml_var fml_max_mem 64GB
    ```

  - There are no default sets and resets. This could lead to formal failures in some cases. Some designs might require that a default set/reset be defined, and that all design inputs other than clocks be treated as free inputs for verification.
  - Define formal semantics correctly. In some cases, default *simulation* semantics could cause failures, because the formal behavior might try to match the simulation behavior for ProtoCompiler output. In such cases, rerun the failed jobs using *synthesis* or *auto* semantics. The former would better match the ProtoCompiler synthesis output, while *auto* runs the simulation semantics followed by synthesis semantics on failed assertions.
- Formality Options
  - Specify additional information for machine configurations in an optional setup constraints file. The file contains commands like the following:

    ```
    set_host_usage -max_cores 8
    ```

Example 4 Verification Checkpoints After Compile

```
database load db -autocreate -technology HAPS-80
option set verification_mode 1
```

Feedback

```
option set verify_check_points 1
launch uc…
run compile…
export verification -check_points -path check_points_data
set fvId [formal_verify -path check_points_data -post_compile….]
run pre_map
run map
job wait $fvId
```

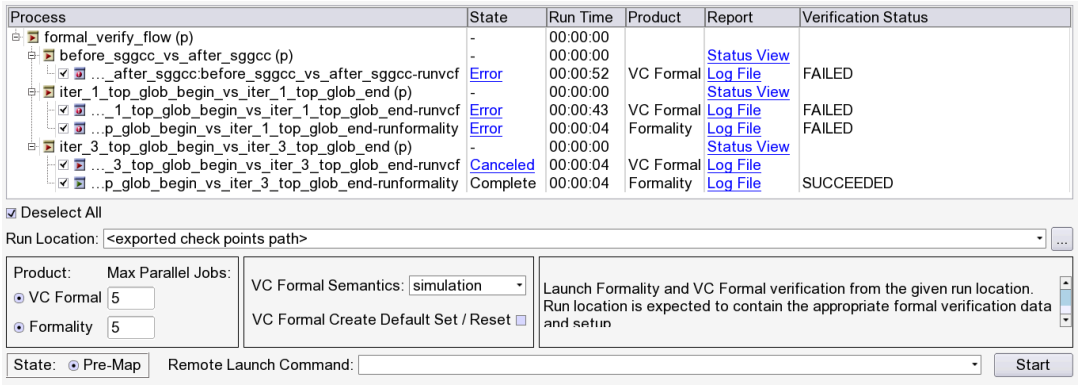
Example 5 Verification Check Points After Pre-Map

```
database load db -autocreate -technology HAPS-80
option set verification_mode 1
option set verify_check_points 1
launch uc…
run compile…
run pre_map…
export verification -check_points -path check_points_data
set fvId [formal_verify -path check_points_data -post_premap…]
run map
job wait $fvId
```

Analyzing Checkpoint Verification Results

1. Set up options and run checkpoint verification as described in Selecting a Verification Tool for Checkpoint Analysis.
2. Analyze results in the results window that opens.

The following figure shows the window when verification was run from the pre-map stage; the window after compile is similar, except for the *State* at the bottom left.



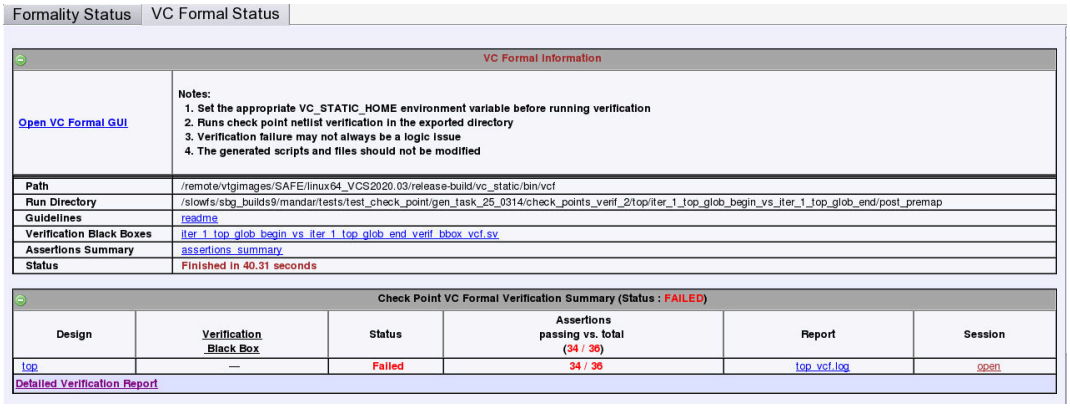
The *Process* column shows all the checkpoints in the design that you can selectively run, and the state of the run. If you specified both VCF and Formality, the jobs are monitored and canceled in whichever tool successfully finishes verification first.

Feedback

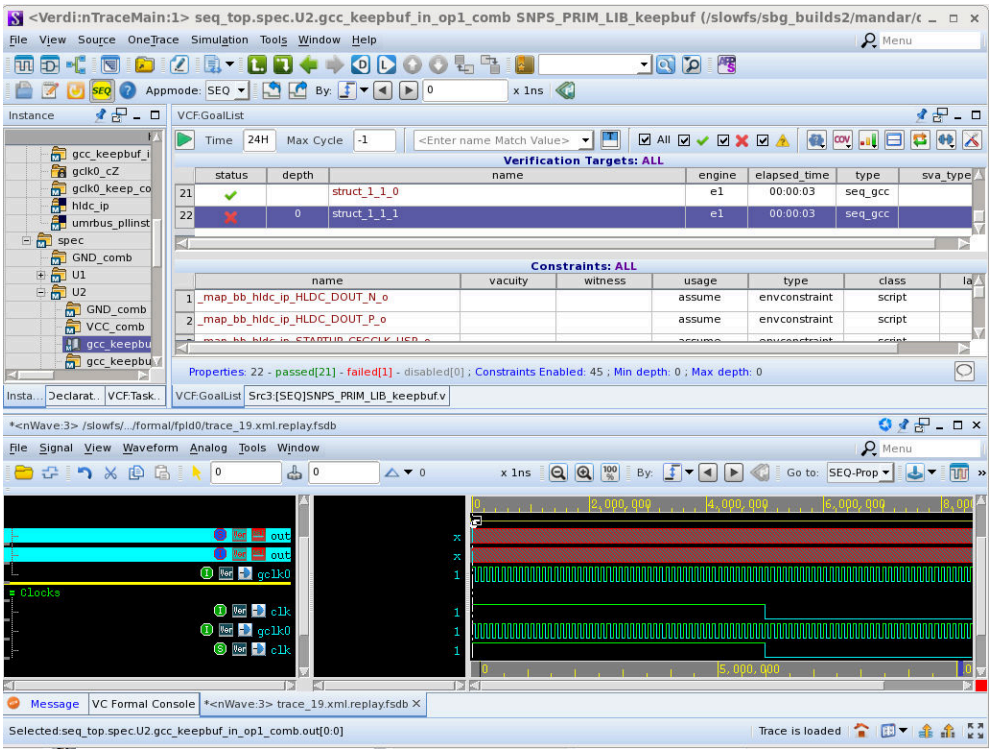
► Check for failures.

- Check the *State* column for status.
- If a job is failing, check the links in the *Reports* column. The *Status View* link shows the complete formal log where you can check details of the run.

The following figure shows the failing assertions for a check point.

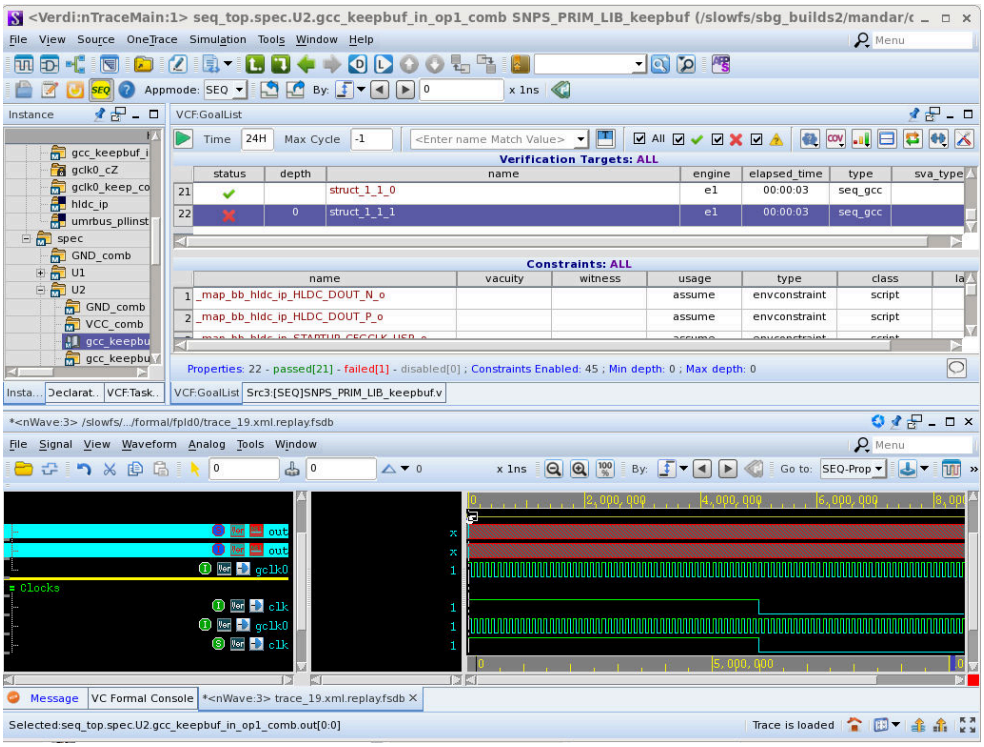


- Click the *open* link (under *Session* on the lower right) to open the corresponding VCF or Formality session.

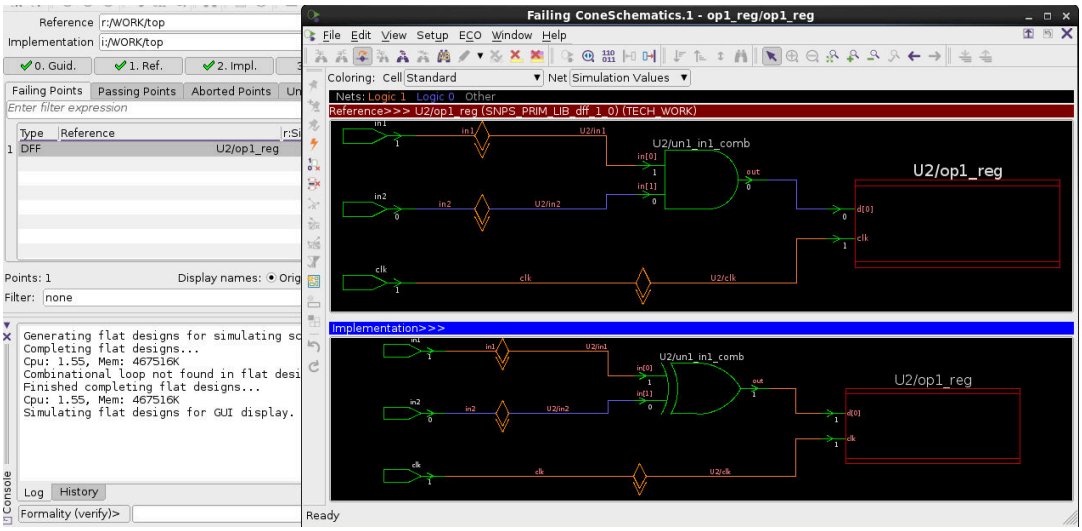


Feedback

► Analyze the results in the VCF session that opens.



If verification was run with the Formality software, that interface opens instead of the VCF one:



For details about using these tools to analyze and verify results, refer to the documentation for the Formality and VC Formal products.

Feedback

Configuring Dynamic Force

Dynamic force flow overrides signal values during prototyping and supports backdoor access to ROM/RAM in the ProtoCompiler100 Runtime tool. It provides ease of use and stops the bit-file regeneration process required to update the ROM/RAM content.

This section provides the steps to perform dynamic force flow.

1. Specify the list of signals to be forced/monitored using *$dumpvars* syntax in the top-level RTL file with the *haps_force* attribute.
2. Enable backdoor initialization for ROM/RAM:
   - Enable backdoor initialization for ROM/single port RAM instance in the design by defining the FDC constraint, or
   - Enable backdoor initialization for ayn type ROM/RAM instance by specifying *$dumpvars* syntax with the *haps_force_mem* attribute for nets connected to ROM/RAM instance in RTL file.

```
define_attribute{i:RAM/ROMinstancename}{syn_allow_dynamic_init}{1}
```

```
initial
begin : IICE_DF
(* haps_force *) $dumpvars (1, <hier_path/signal1_scalar>);
(* haps_force *) $dumpvars (1, <hier_path/signal2_vector>);
(* haps_force_mem *) $dumpvars (1, <hier_path/write port>);
(* haps_force_mem *) $dumpvars (1, <hier_path/read port>);
end
```

Example:

```
initial
begin : IICE_DF
(* haps_force *) $dumpvars (1,
 cnt_demo_top.u_cnt_demo_inst4.capture_cnt[15:0]);
(* haps_force *) $dumpvars (1,
 cnt_demo_top.u_cnt_demo_inst5.capture_cnt[15:0]);
(* haps_force_mem *) $dumpvars (1, top.waddr[7:0]);
(* haps_force_mem *) $dumpvars (1, top.raddr[7:0]);
(* haps_force_mem *) $dumpvars (1, top.we);
(* haps_force_mem *) $dumpvars (1, top.re);
(* haps_force_mem *) $dumpvars (1, top.wr_data[15:0]);
(* haps_force_mem *) $dumpvars (1, top.rd_data[15:0]);
end
```

3. Enable *$dumpvars* in the ProtoCompiler tool using the following options:

```
option set debug_dumpvars 1
```

Feedback

Optionally, enable the global driver tracing to insert the force mux on the real driver of the specified net using the following option:

```
option set dynamic_force_global_driver 1
```

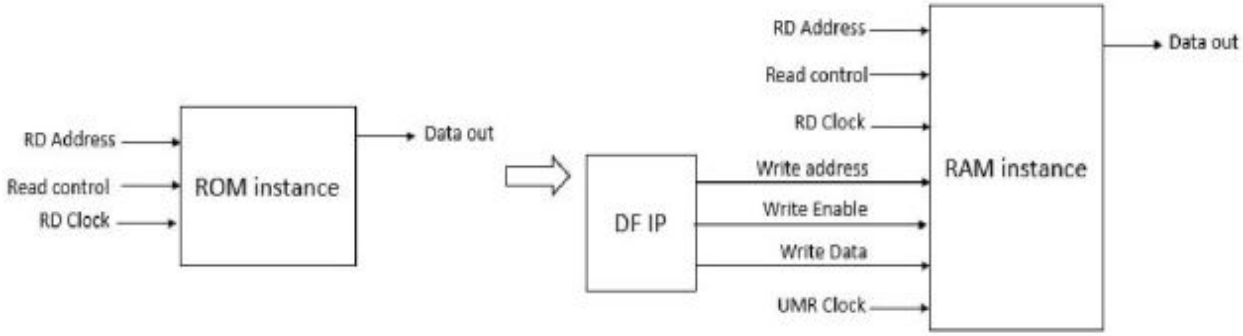
Optionally, to instrument the added signal, specify the IDC file with IICE setting and sample clock to instrument the signals specified for force (*$dumpvars*-based instrumentation is not supported for dynamic force applied signals).

Dynamic Force Backdoor Memory Initialization

This section describes how the ProtoCompiler tool handles the RAM/ROM for Backdoor Memory Initialization.

ROM Backdoor Support

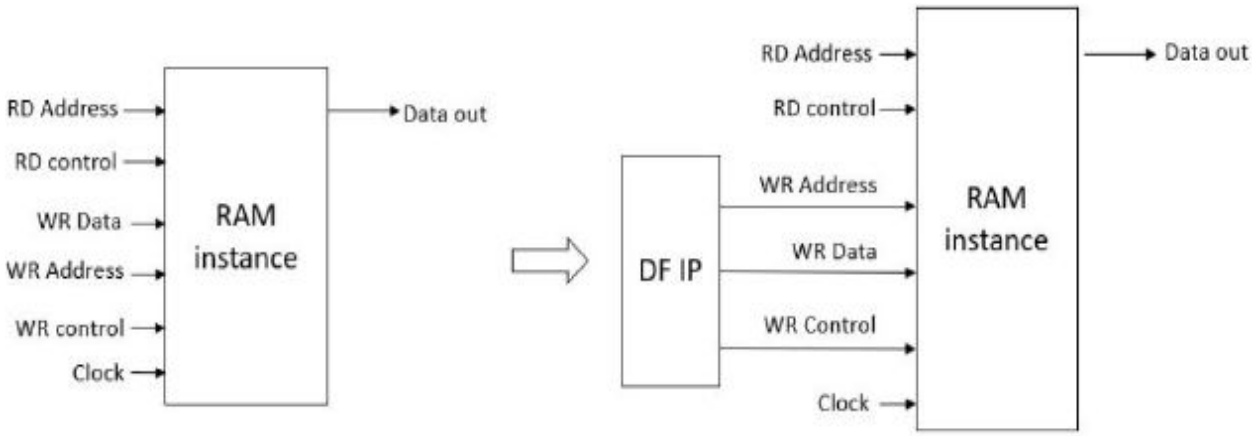
The HAPS-200 ProtoCompiler tool converts ROM instance into RAM instance, adds dynamic force muxes on the RAM write port signals to support backdoor write access through hRun as shown in the following figure. Memory write is performed on the UMR clock.



RAM Backdoor Support

The ProtoCompiler tool adds dynamic force muxes on the RAM write port signals to support backdoor write access through hRun as shown in the following figure. Memory write is performed on the design clock.

Feedback



Dynamic force feature adds dynamic force muxes on memory write ports only. To enable backdoor readback capability on RAM/ROM, specify *$dumpvars* (as given in Step 1 of the procedure) on the following signals:

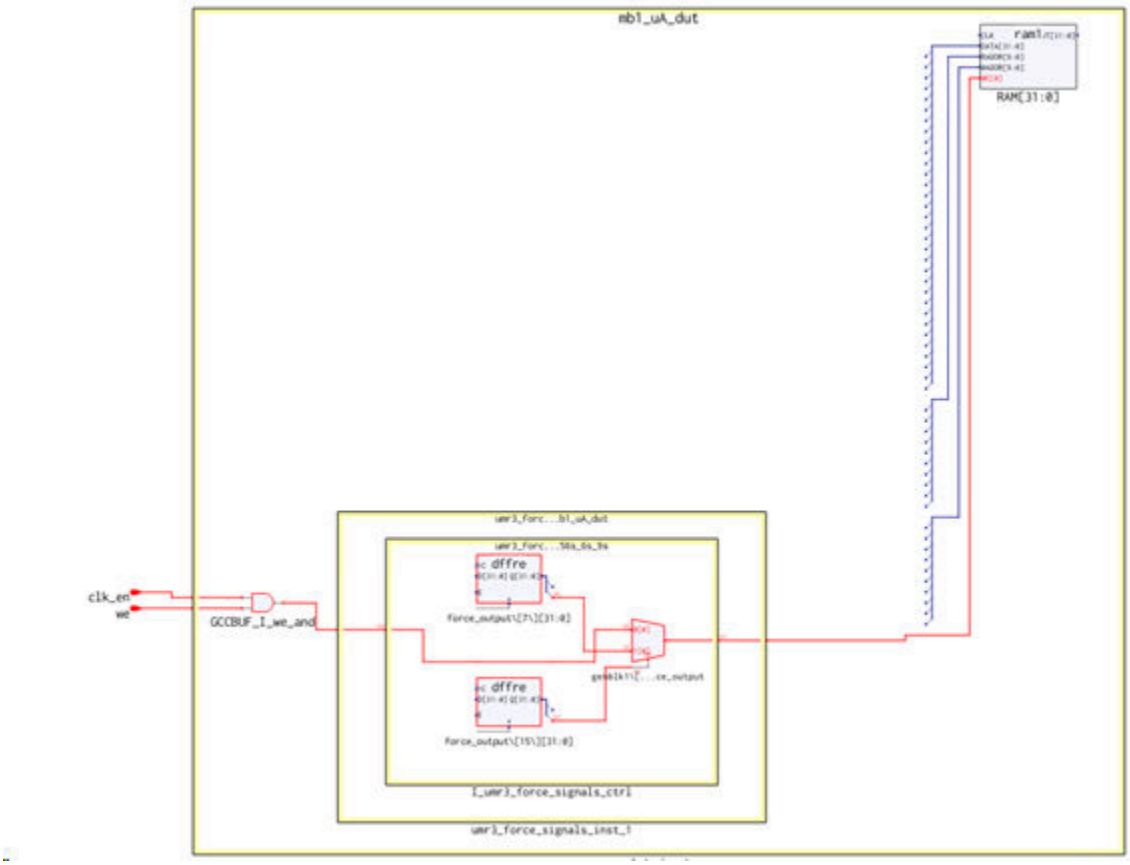
- A memory with read address register
  - Data input or control signals of read address register
  - Memory read data
- A memory with read data register
  - Data output or control signals of read data register
  - Memory read address
- Memory read control signals
- Memory gated clock enable signal

Handling Memories with Gated Clocks

RAM backdoor write using dynamic force depends on the design clock. If the clock is gated, the write operation cannot be performed. To avoid this dependency on the gated clock, the dynamic force IP is inserted after the completion of the gated clock conversion (GCC) process.

When the *syn_allow_dynamic_init* attribute is applied on a RAM primitive with read address or data registers, the tool inserts the dynamic force IP post the GCC making it the immediate driver to the memory write ports. The following image illustrates this implementation.

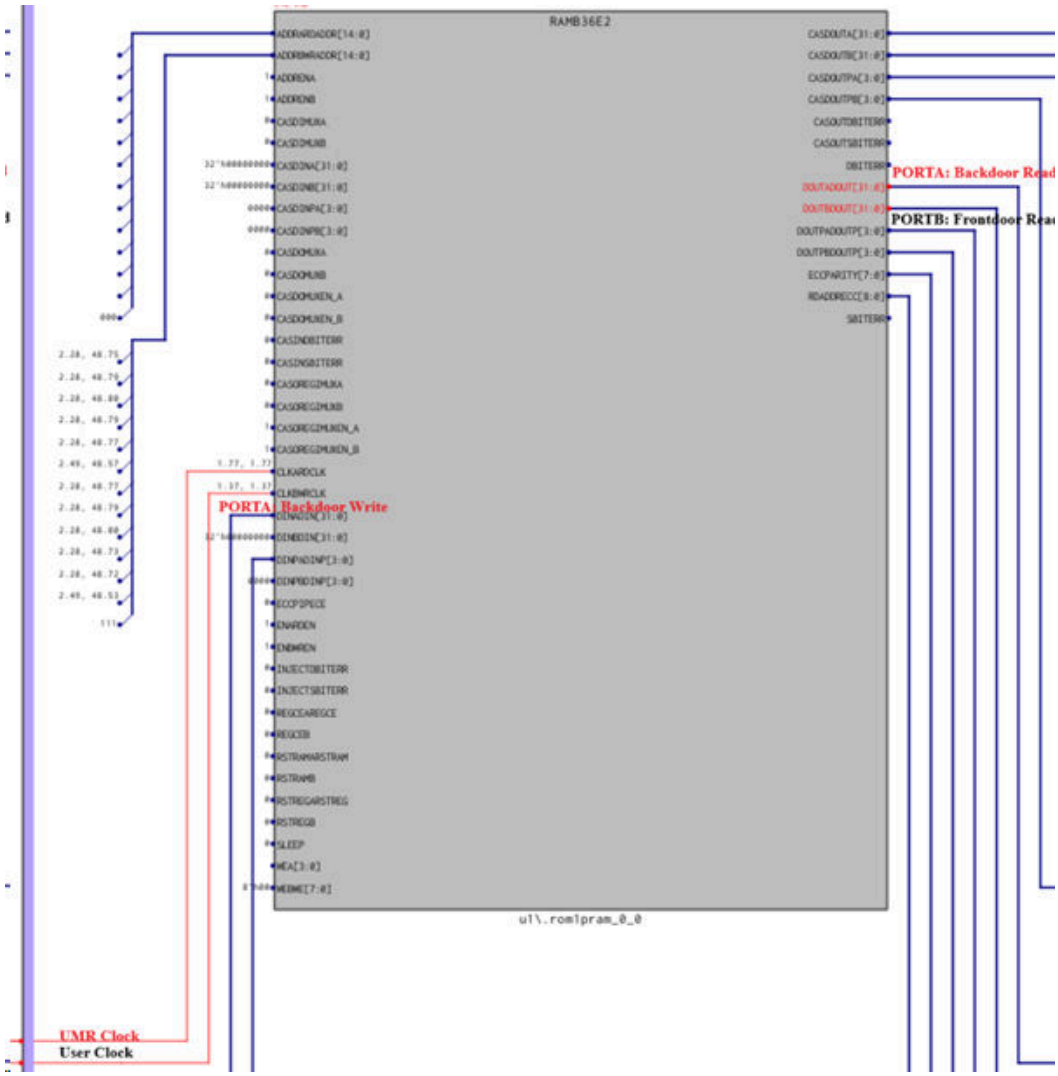
Feedback



Design Clock Independent ROM Backdoor Access

When the *syn_allow_dynamic_init* attribute is specified on a ROM instance, the tool infers the ROM using a TDP RAM structure. The backdoor write and read operation is performed using the UMR clock and the front door read operation is performed using the original memory clock. The following image illustrates this implementation.

Feedback


RAMB36E2
PORTA: Backdoor Write
PORTA: Backdoor Read
PORTB: Frontdoor Rea
UMR Clock
User Clock
u1\.romlpram_0_0

8

Migrating Designs for Synthesis

The tool includes certain features that facilitate the migration of ASIC designs and to improve productivity as you implement the design. See:

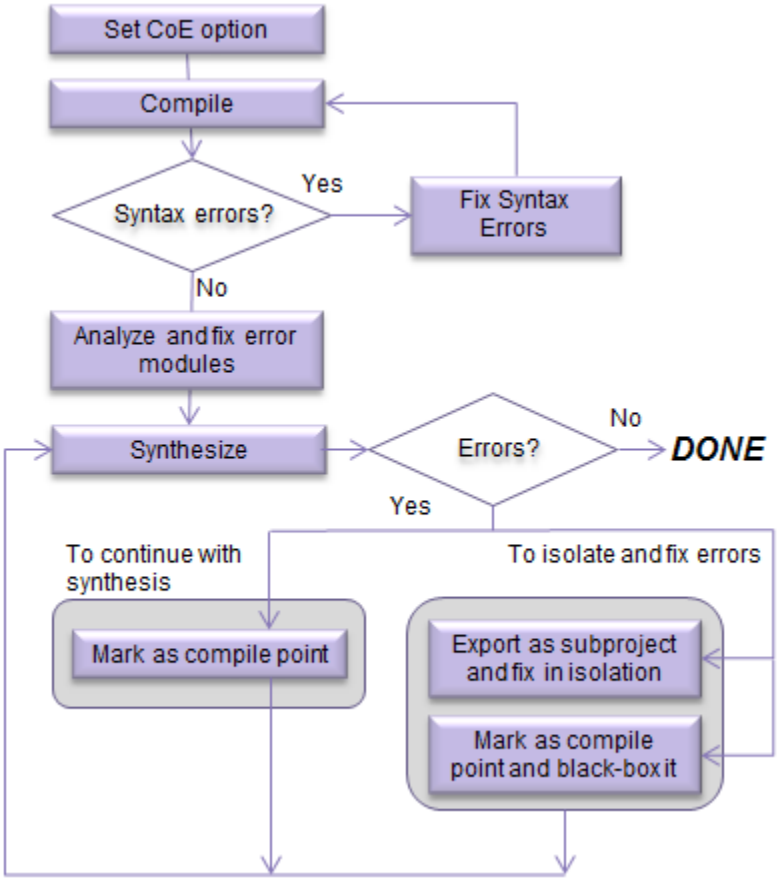
- Running Compilation or Mapping with Errors
- Including Unified Power Format (UPF) Specifications
- Converting Gated Clocks
- Editing Netlists
- Using Cross-Module Referencing (XMR)
- Converting Designs for Unified Compile
- Integrating Memory Models

Running Compilation or Mapping with Errors

The *Continue on Error* (*CoE*) feature significantly reduces overall synthesis runtime by reducing the number of synthesis iterations. This can be a significant advantage when prototyping or handling large designs.

By default, the compiler stops the compilation process as soon as it encounters an error in the design. The *Continue on Error* option allows the compilation process to continue for certain non-syntax compiler errors. As a result, multiple problems might require multiple iterations to identify and correct the issues. This translates to long turnaround

times, especially for complex designs with lengthy compilation times. The following figure summarizes the *Continue on Error* process:



The following procedures describe the use of this feature:

- Running Compilation with Errors
- Analyzing Compilation Errors
- Running Mapping with Errors

Running Compilation with Errors

This procedure shows you how to use the *Continue on Error* feature to reduce compiler iterations.

Feedback

1. Enable the *Continue on Error* option for the design, by including the following command in an options file or by typing it in the console window:

   ```
   option set continue_on_error 1
   ```

2. Compile the design.

   The compiler first parses the syntax and checks for syntax errors. If it encounters syntax errors, it errors out. You must fix these errors before rerunning compilation. Typical syntax errors are RTL code errors, and include typos in keywords, missing semicolons, mismatched beginning and ending brackets, or incorrect syntax.

   After the syntax-checking stage, the compiler does not halt when it encounters an error, but completes a one-pass compilation and reports all the errors together. These non-syntax errors are related to synthesis, and include out-of-range access, missing function or task definitions, improper port or generics mappings, and coding from which sequential elements cannot be extracted.

3. Analyze and fix any syntax errors before proceeding with re-compiling.

4. If you do not have any syntax errors, analyze the non-syntax errors and fix them as described in Analyzing Compilation Errors.

   You can then synthesize the design. For errors found after mapping, you can isolate the modules by creating subprojects or by marking them as compile points and black-boxing them, and then fixing the errors. If you want to continue synthesizing, just mark the error module as a black box and resynthesize.

5. The *Continue on Error* feature works after the first stage of the compilation process is complete, as described in the following table.

Syntax Check/Parse	The compiler parses all files in the project and reports the syntax errors found in each file. If errors are found, the compilation process terminates at this point and does not continue to the next stage. Any errors reported during this stage must be corrected before compilation can advance.
Elaboration/Sizing *Hardware Generation* *Optimization*	If a compilation error is encountered at any of these stages, the compiler converts the design unit containing the error to a black box. The compiler continues processing the remaining design units, automatically continuing to the next compilation stage without stopping at errors. It reports all errors found in these stages together, at the end.

   If errors are found in the latter three stages, these errors must be corrected before any subsequent synthesis operations can be performed.

Feedback

Analyzing Compilation Errors

This procedure describes techniques to analyze errors identified after using *Continue on Error* at the compilation stage, as described in Running Compilation with Errors. These errors are non-syntax errors; the syntax errors must be fixed before compiling with *Continue on Error*.

1. Check the log file for error messages.

```
@E:CG169 : CG169_svlog_1gen_0.v(6) | Fork with a disable is not synthesizable
@N:CG364 : CL224_svlog_1gen_1.v(1) | Synthesizing module CL224_svlog_1

@W:CG361 : CL224_svlog_1gen_1.v(9) | Case tag overlaps with a previous tag (case branches 0 and 1)
@N:CG364 : CG413_svlog_1gen_2.v(1) | Synthesizing module CG413_svlog_2

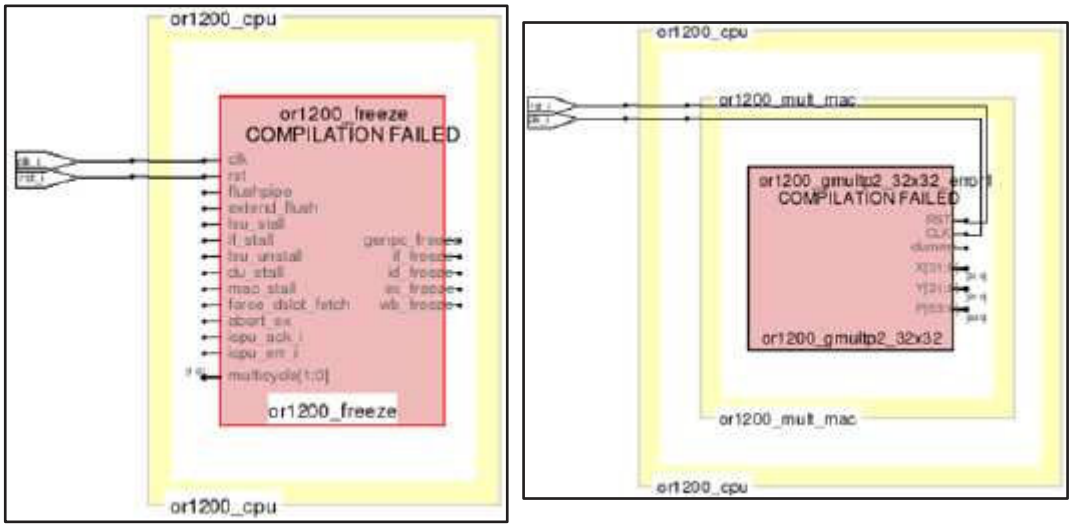
@E:CG413 : CG413_svlog_1gen_2.v(13) | Assignment expressions are valid within procedural statements
@N:CG364 : top.v(1) | Synthesizing module top

@N:CG794 : top.v(69) | Using module pass_and_vhdl2008_0 from library work
@N:CG794 : top.v(74) | Using module pass_nand_vhdl2008_1 from library work
@N:CG794 : top.v(79) | Using module CD729_vhdl2008_0 from library work
@N:CG794 : top.v(84) | Using module CD293_vhdl2008_1 from library work
@N:CG794 : top.v(90) | Using module CD300_vhdl2008_2 from library work
@E:CL224 : CL224_svlog_1gen_1.v(7) | ROM clash detected: address 0000000 with multiple data entries 1011, 1010 found

At c_ver Exit (Real Time elapsed 0h:00m:01s; CPU Time elapsed 0h:00m:00s; Memory used current: 100MB peak: 101MB)
```

2. If you get a message about missing modules, include the modules before rerunning compilation. You can define modules as black boxes with the *Set as Black Box* command in the Design Hierarchy view. Check all modules that were black-boxed by *CoE*. You can use the following techniques to identify the modules in the RTL view:

   - Check all red modules in the RTL view; the color indicates modules with errors. If the module retains its original module name, the error is fully contained in the module. If the module name has an *_error* suffix, the error has to do with port mismatches with the parent module. In the latter case, the parent module is black-boxed by *CoE*.



- Use the Tcl *find* command to search for modules with the *is_error_blackbox* property. The tool attaches this property to all modules with errors. For example, these commands list all modules with errors, and instances with errors, respectively:

```
get_prop -prop inst_of [find -hier -inst * -filter
 @is_error_blackbox==1]
c_list [find -hier -inst * -filter @is_error_blackbox==1]
```

3. After you have identified the errors, either fix them or isolate them.

   You must deal with all *CoE*-identified compiler errors before proceeding with any subsequent synthesis operations. If you cannot immediately fix the errors, use these techniques to isolate and fix modules:

   - Export the affected module as a subproject and fix it later.
   - Mark it as a compile point, and define it as a black box with the *Set as Black Box* command in the Design Hierarchy view.

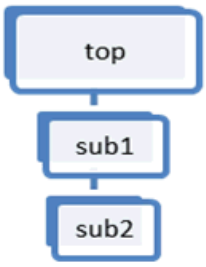
4. Re-compile the design and proceed with synthesis, using one of these approaches:

   - Synthesize only the error-free modules and then merge them into the final netlist.
   - Synthesize the entire design with black boxes for the modules with errors to be resolved.

Examples

Hierarchical Error Reporting

This Verilog example has two sub-modules:



The code highlighted in red indicates the locations of the errors after compilation with *CoE*:

```
module sub2 (input d,rst,set,clk,
 output reg q,
 input [7:0]in, input [2:0] raddr, waddr,
 output [7:0] out3,
 input [3:0] a, b,
 output reg [4:0] q1);
```

Feedback

```
 nomodule i0(in, clk);
always@(posedge clk)
 begin
 if(clk)
  q = d << -3;
 end
 reg [7:0] mem [7:0];
 initial
 begin
  $readmemb("data.dat",in);
 end
always @(posedge clk)
mem[waddr] <= in;
assign out3 = mem[raddr];
wire [2:0] temp;
always@(posedge clk)
 begin
  q1 <= temp | a;
 end
assign temp = a ** b;
endmodule
module sub1 (input d,rst,set,clk,
 output reg q,
 input [7:0]in, input [2:0] raddr, waddr,
 output [7:0] out3,
 input [3:0] a, b,
 output reg [4:0] q1);
always @(posedge clk)
 begin
  q <= data1 | test; // 2 errors reported end
reg q;
always @(posedge clk)
begin
 if (rst)
  q <= 1'b0;
 else
  q=d;
 end
sub2 u1 (.*);
endmodule
module top (input d,rst,set,clk,
 output reg q,
 input [7:0]in, input [2:0] raddr, addr,
 output [7:0] out3,
 input [3:0] a, b,
 output reg [4:0] q1);
sub1 u2 (.*);
endmodule
```

Running Mapping with Errors

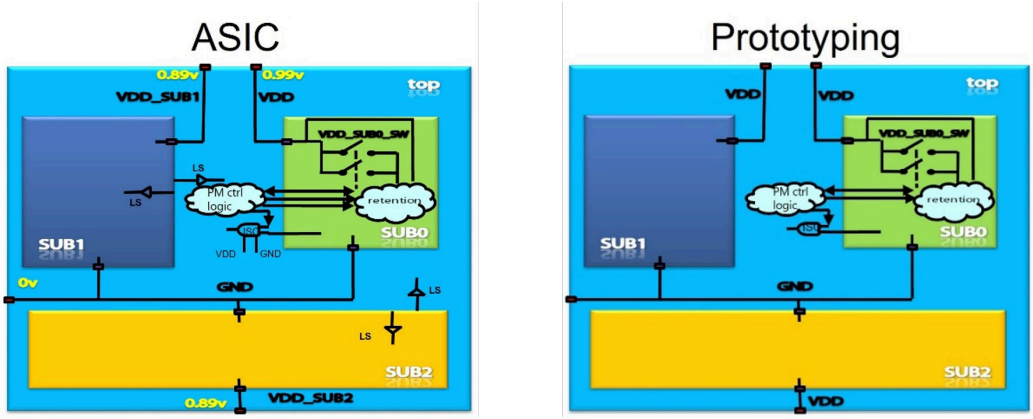
By default, the tool runs continue on error during the mapping stage of synthesis. With this setting, the tool does not stop when it encounters an error, but maps the rest of the design.

To disable CoE from running during mapping, use the following command:

```
option set continue_on_error 0
```

Including Unified Power Format (UPF) Specifications

Before looking at the Unified Power Format (UPF) support in HAPS prototyping, it is important to understand the difference in the lower power intent between ASIC and prototyping:



A typical ASIC design would contain power domains with different voltages, supply network, power switches, isolation logic, level shifter logic and retention logic. When this ASIC low power design is prototyped, due to architectural limitations on a FPGA based hardware, the following UPF functionalities cannot be modeled or validated:

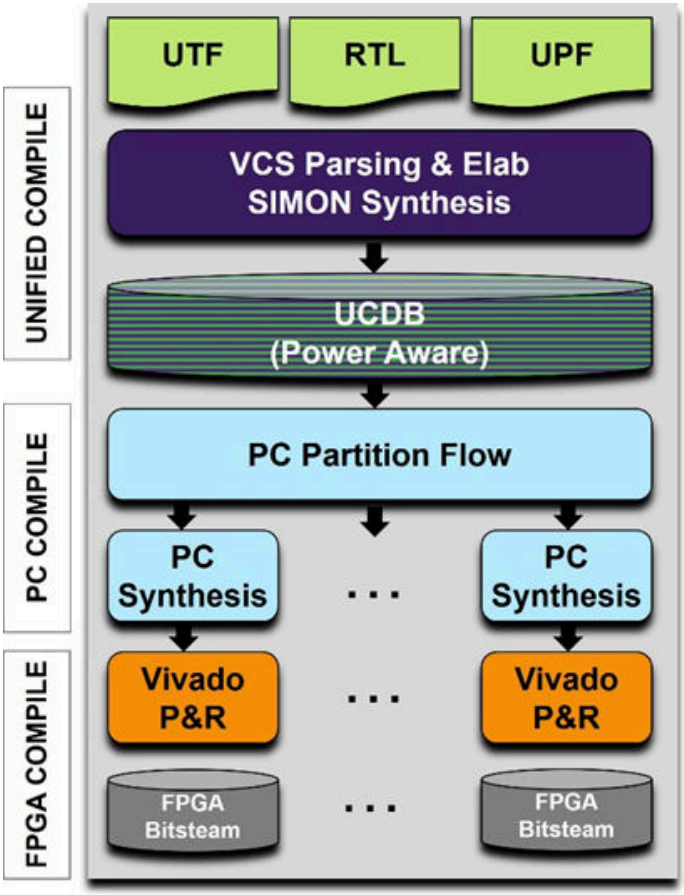
- Multi-voltage domains
- Level shifters between different voltage domains

Any UPF commands that are relevant to these functionalities are parsed and ignored.

HAPS Unified Compile UPF Flow

HAPS UC UPF flow supports the same UPF syntax, UPF command, and error messaging as VCS. The same UPF file is used for both simulation and prototyping. When compiling

for ProtoCompiler, VCS parses and elaborates both the design and UPF files. The following figure illustrates the UPF compilation flow:



In the unified compilation, VCS parses the design source (RTL) along with the UPF files and creates the data-model comprising both the functional and power intent. After VCS execution, the design is synthesized into UCDB (scm) database. ProtoCompiler reads the UCDB database and handles power aware optimizations for enhanced QoR with gated clock conversion and block RAM inference.

The UPF script describes the topology of the power network. HAPS ProtoCompiler supports the following UPF versions:

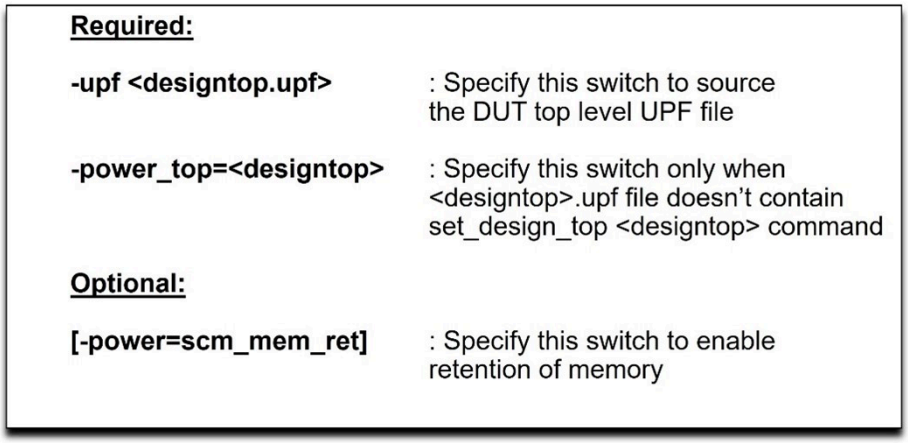
- UPF 1.0 and UPF 2.0 (IEEE 1801-2009 standard)
- Limited set of commands from UPF 2.1 (IEEE 1801-2013 standard)

Power Aware Compilation

The UPF file is specified in the VCS command line (or VCS script) with the `-upf`option:

```
vcs -upf <designtop.upf> -power_top <designtop> [-power=scm_mem_ret]
```

Feedback



By default, flip-flop retention is enabled. Optionally, user can enable memory retention using `scm_mem_ret` value specified using the `-power` option.

UPF for GCC of Clocks

Flip-flops with asynchronous set/reset signals are not aligned with the clock edges, and therefore difficult to be handled by GCC. However, in certain designs if it is guaranteed that the asynchronous set/reset signals are aligned with synchronous edges or, if they remain constant except during set/reset of the design sequence, then GCC conversion is possible. Currently, GCC supports this method by using a global option `force_async_genclk_conv`, which when enabled, declares all the asynchronous flip-flops as synchronous. However, there are scenarios in which you need to select asynchronous flip-flops as synchronous. With the UPF for GCC feature, based on the property `syn_force_async_genclk_conv` set on the specific asynchronous flip-flop, GCC replicates the asynchronous clock paths toward this flip-flop and forces all the asynchronous sequential elements on the paths to be synchronous.

This feature is enabled by the option `gcc_force_async_inst_based` (default is 0):

```
option set gcc_force_async_inst_based 1
```

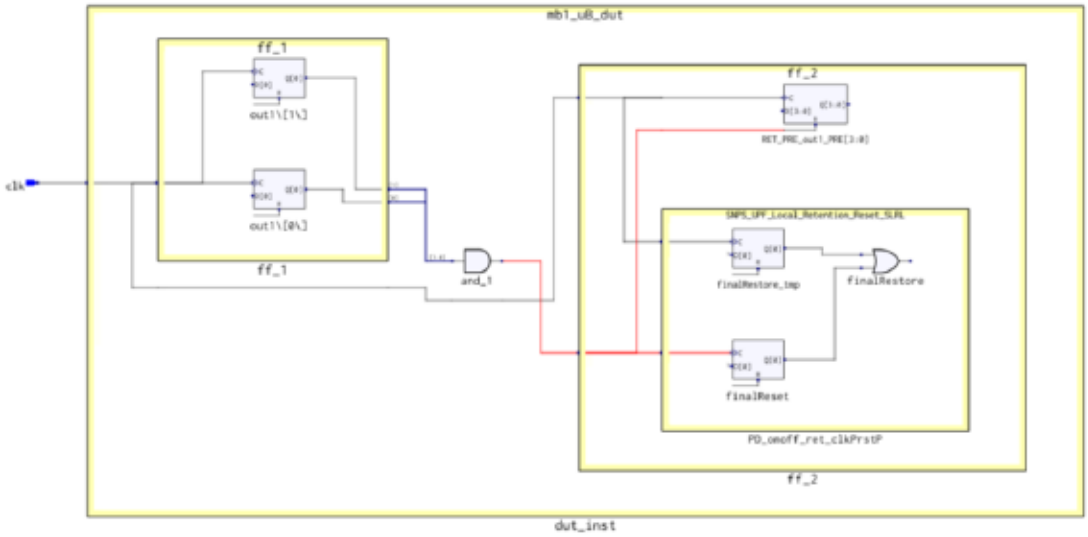
See option for more information.

Example

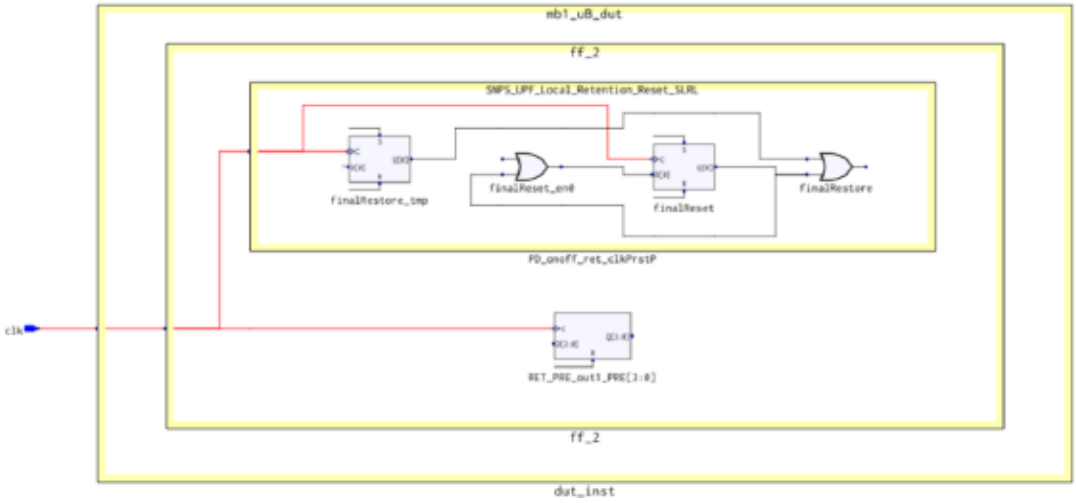
Feedback

With the `gcc_force_async_inst_based` option enabled, GCC converts the logic on the clock path of UPF inserted `finalReset flip-flop` which is marked with the property `syn_force_async_genclk_conv`. The following images describes this feature:

UPF design without this option:



UPF design with this option:



UPF Power Domain Corruption

The tool supports logic corruption and boundary mux-based corruption.

Feedback

Logic corruption

Logic corruption on power domain is enabled using corrupt_pd command specified in a power config file that is sourced on the VCS command line using -power_config option:

```
vcs … -power_config power_config.tcl
```

The following is the corrupt_pd command syntax:

```
corrupt_pd -domain all | domainName [-mode {0 | 1 | I }] [-type {mem |
seq | both}]
```

Logic corruption is supported on the sequential elements (flip-flops and memories) of a power domain with various modes as explained in the following table:

Command	Option	Value	Description
`corrupt_pd`	`-domain`	all	Specifies all the switchable power domains to control corruption.
		Domain Name	Specify the name of the power domain to be corrupted. The power domain name can be simple name or hierarchical name separated by '/'
	`[-mode]`	0 (Default)	All zeros corruption
		1	All ones corruption
		I	Inversion corruption
	`[-type]`	seq (Default)	Implements corruption on all flip-flop elements
		mem	Implements corruption on memory elements
		both	Implements corruption on all the sequential elements

Many a times, user might choose to tradeoff between corruption coverage and QoR (BRAM inference), to match the performance of a non-power aware design. To achieve this, the tool supports excluding certain sequential elements from corruption through corrupt_pd_exclude command specified in a power config file that is sourced on the VCS command line using `-power_config` option. The syntax is:

```
corrupt_pd_exclude -elements {top.I1.I2.elem}
```

Command	Option	Value	Description

Feedback

corrupt_pd_exclude	-elements	Elements/ Instance Name	No corruption is applied to the specified elements or Instance. List of elements should be separated by space. Specified element/Instance name should use “.” as the hierarchy separator.

Corrupt_pd and corrupt_pd_exclude examples

- To corrupt individual power domains with different modes and element type:

```
corrupt_pd - domain PD1 -mode I -type seq

corrupt_pd - domain PD2 -mode 0 -type mem
```

- Conflicting scenarios
  - VCS displays a warning message if the power domain is not found in the input UPF and the command is ignored.

```
Warning-[UPF_PDDNEFC] Domain specified does not exist
Domain 'PD4' provided for 'corrupt_pd' command does not exist in
 design.
```

  - If same power domain is given multiple times, last value of the given mode is taken.
    - In this usage , 1 is applied to “PD1”

```
corrupt_pd -domain PD1 -mode 0

corrupt_pd -domain PD1 -mode 1
```

    - In this usage, 0 is applied to “PD1” since without -mode, 0 is default mode.

```
corrupt_pd -domain PD1 -mode 1
corrupt_pd -domain PD1
```

  - If a power domain name is given along with “all” value, then the mode specified with “all” should be applied to all power domain except the explicit power domain names.
    - In this usage, except PD1 (which is corrupted with 1), all other power domains are corrupted with mode 0.

      corrupt_pd -domain all -mode 0

      corrupt_pd -domain PD1 -mode 1
  - If a power domain name is given with type and mode, corresponding value 0 | 1 | I with types should be applied to power domain.

```
corrupt_pd -domain PD1 -mode I -type both
corrupt_pd -domain PD2 -mode 0 -type seq
corrupt_pd -domain PD3 -mode 1 -type mem
```

- To exclude elements

```
corrupt_pd -domain all -mode 0
corrupt_pd_exclude -elements {top.I1.I2.elem}
corrupt_pd_exclude -elements {top.I1.I2.elem1 top.I1.I2.elem2
 top.I2.I2.elem4}
```

The following are the limitations with the UPF power domain corruption:

- Elements within VHDL hierarchy are not corrupted
- No wild card support on corrupt_pd and corrupt_pd_exclude commands
- No support for part-select or bit-select of vector signals on corrupt_pd_exclude command

Boundary corruption

Boundary mux-based corruption can be enabled by specifying -power option:

```
vcs … -power=enable_boundary_gates
```

Low Power (UPF) Reporting

Low power logic is implemented in VCS after parsing the UPF file. The following report files under `<UCDB>/mvsim_native_reports` directory can be analyzed to sign-off on the implementation:

- Isolation reports
  - isolation_insertion_summary.rpt
  - isolation_association.rpt
- Retention reports (./Zebu)
  - Flop retention: `zebu_retention_flop_implementation.rpt`
  - Memory retention: `zebu_retention_potential_memory.rpt`

Low Power (UPF) Debug

In case of incorrect behavior of any UPF policy or incorrect instrumentation, recompile the design till UCDB stage using the following VCS command line options and share the outputs generated:

- `-power=dumplpconnect (Output file: lpconnect.ev)`
- `-power=dump_tokens+upf_tokens (Output file: tokens.upf)`

In case of compile time degradation, recompile the design till VCS stage using `-power=ctcmdiag` and share the diagnostic reports.

In case of unexpected logic optimizations caused by UPF logic, recompile the design using `-power=keep_upf_ctrl`, to preserve and identify the UPF logic control signals having no driver or constant driver.

Debugging Low Power UPF During Runtime

This feature enables UPF debugging on HAPS-100 hardware during runtime and supports:

- Forcing no-harm mode (Bypasses instrumented isolation or retention logic in the netlist).
- Selectively disabling isolation or retention logic per policy.

To enable the UPF debug feature use the option, option set upf_runtime_debug 1. This option allows the ProtoCompiler tool to add dynamic force logic to all the global or local, isolation or retention controllers that can be controlled at Runtime. By default, this option is disabled (0). See option.

The list of isolation or retention policies are reported in the runtime map file under the export runtime directory:

```
<exportRuntimeDirectory>/<FPGAName>/runtime_upf.txt
```

Sample Runtime UPF File

```
stb_top/top_net/u_wrap/u1/PD1/ISO_LFSR # Isolation policy local
 controller
stb_top/top_net/u_wrap/u1/PD1/ISO_SD # Isolation policy local controller
stb_top/top_net/u_wrap/u1/PD1/RET_PD_SD # Retention policy local
 controller
stb_top/top_net/u_wrap/u2/PD2/ISO_PD_CNT # Isolation policy local
 controller
stb_top/top_net/u_wrap/u2/PD2/RET_PD_BCD # Retention policy local
 controller
stb_top/top_net/u_wrap/u2/PD2/RET_PD_CNT # Retention policy local
 controller
top_net.vcs_xmr_M_ZebuDummyPowerDomain_S_retention_global_control #
 Isolation global controller
top_net.vcs_xmr_M_ZebuDummyPowerDomain_S_isolation_global_control #
 Retention global controller
```

To enable or disable isolation or retention logic at Runtime:

```
force {<signal name in runtime_upf.txt file>} <value>
```

If the isolation or retention (local or global) controllers are not forced using the force command, the controllers drive a value of 1.

To disable all the isolation or retention policies:

```
force
{top_net.vcs_xmr_M_ZebuDummyPowerDomain_S_retention_global_control} 1'b0
force
{top_net.vcs_xmr_M_ZebuDummyPowerDomain_S_isolation_global_control} 1'b0
```

To disable a specific isolation or retention policy:

```
force {stb_top/top_net/u_wrap/u2/PD2/ISO_PD_CNT} 1'b0
force {stb_top/top_net/u_wrap/u2/PD2/RET_PD_CNT} 1'b0
```

Converting Gated Clocks

Gated clocks are clocks where signal propagation is controlled by gates. They are widely used in ASIC design to reduce power when domains are not used. FPGAs have dedicated low-skew clock distribution nets and non-gated clocks. But SoC designs use gated clocks instead of these resources. Generated clocks are clocks created by multiplying, dividing, or time-shifting an existing clock, or by switching inputs.

Gated clock conversion (GCC) simplifies gated and generated clock structures in the ASIC RTL design. GCC converts the clocks to FPGA-friendly clock schemes.

The following sections describe how to use GCC to automatically convert ASIC clocks:

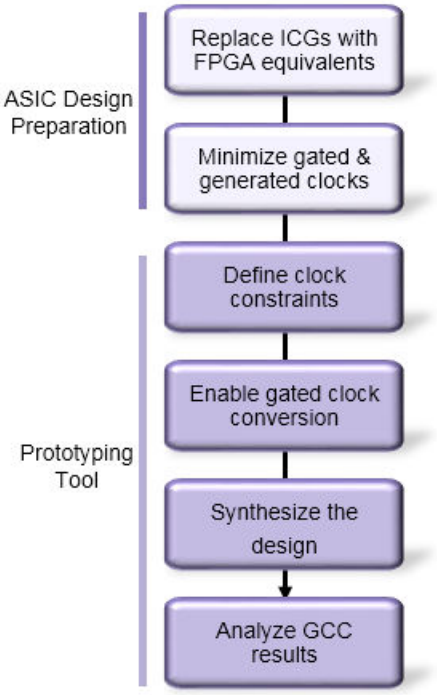
- Gated Clock Conversion Flow
- Guidelines for Generated Clocks
- Guidelines for Gated Clocks
- Guidelines for Integrated Clock Gating Cells (ICG)
- Working with Gated and Generated Clocks
- Defining Clocks for Gated Clock Conversion
- Interpreting Gated Clock Error Messages

Gated Clock Conversion Flow

FPGAs have dedicated low-skew clock distribution nets and un-gated clocks, but SoC designs use gated clocks instead of these resources. The ASIC clock gating logic must be converted before it can use the dedicated clock nets on the FPGA. ‘This process is called gated clock conversion (GCC). GCC creates a logical equivalent without setup and hold violations, by removing the clock gate with the enable signal and routing the enable signal directly to the FPGA logic elements.

Feedback

The following figure summarizes the steps in the GCC process. For details, see the procedure that follows the figure.



1. Replace integrated clock gating cells (ICGs) with FPGA equivalents.

   You can use a netlist editor script to do this. See Guidelines for Integrated Clock Gating Cells (ICG) for further information.

2. Follow guidelines for working with generated clocks and gated clocks.

   See Guidelines for Generated Clocks and Guidelines for Gated Clocks.

3. Enable gated clock conversion and do a preliminary FPGA synthesis run.

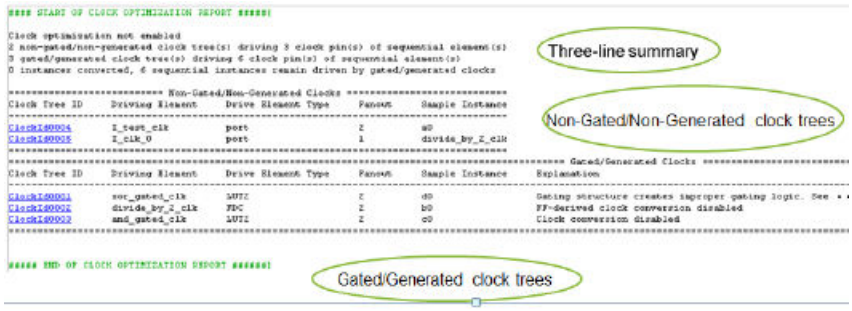
   Refer to Working with Gated and Generated Clocks for details.

4. Check the results in the clock conversion report.

   The clock conversion report generated after mapping shows the generated and gated clocks that were converted. The tool removes clock gating by moving the gated clock logic from the clock pin of a sequential element to the enable pin.

Feedback

This example shows a typical clock conversion report:

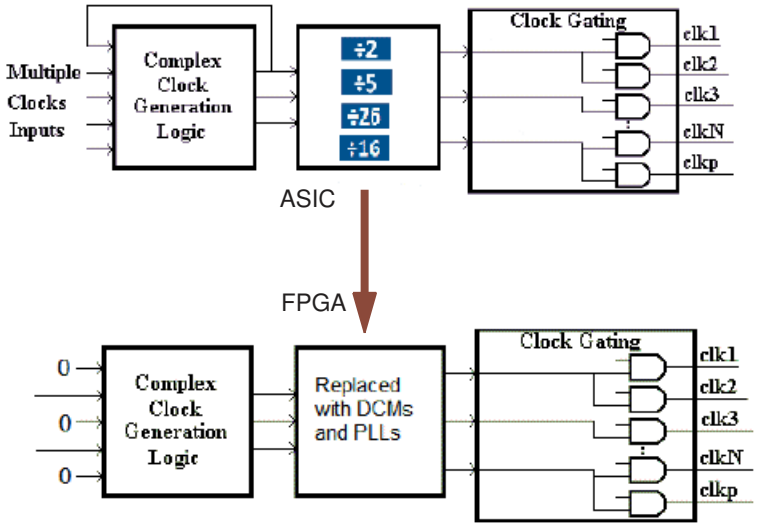


For more information about handling GCC errors, see Interpreting Gated Clock Error Messages.

Guidelines for Generated Clocks

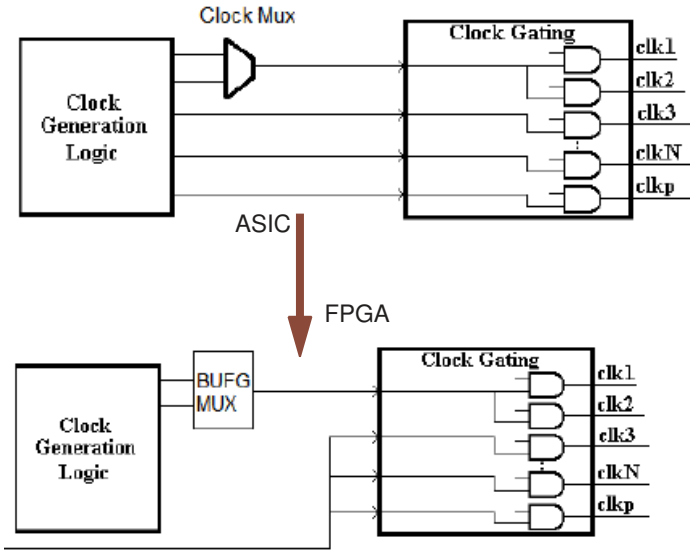
Generated clocks are clocks created by multiplying, dividing, or time-shifting an existing clock, or by switching inputs or another alignment mechanism. Define clocks that run at sub-multiples of a base clock frequency as generated clocks. Using latch-based clock-gating cells or clock dividers to generate sub-multiple clocks causes those parts of the design to be constrained by the sub-multiple clock frequency instead of the base clock frequency, and this results in faster run times and a smaller area.

- Replace generated clocks with equivalent FPGA structures in the target technology, such as clock managers and programmable clock generators like phase-locked loops (PLLs).



- Tie clock inputs that are not going to be used for prototyping to constants. The FPGA prototyping software propagates the constants and optimizes the clock-generating logic, minimizing the clock network. This reduces the number of clocks to be routed inside the FPGAs, and also reduces synthesis and place-and-route runtime.
- When possible, avoid clock muxes. If they cannot be removed or reworked, modify the constraints and declare the clock at the mux output.

  Another technique is to replace clock multiplexing logic with FPGA-specific primitives. The following figure shows the clock mux replaced by a Xilinx BUFGMUX, which, along with some additional glitch removal logic, is logically equivalent. The advantage to this approach is that the prototyping software understands these primitives, so timing analysis can be more accurate. Further, it eliminates the need for additional skew logic, because this primitive directly drives global clock resources.

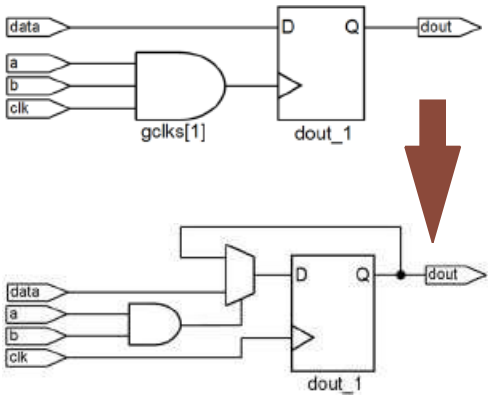


- If you have multiple internally generated clocks for functionality that is not going to be prototyped, tie these clocks to a single external clock source.

Guidelines for Gated Clocks

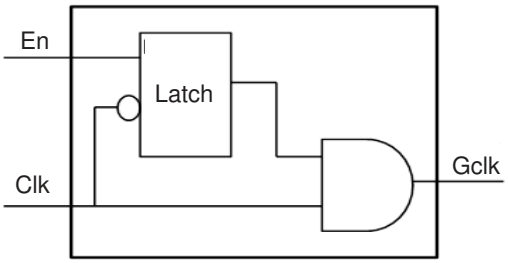
Gated clocks are clocks where signal propagation is controlled by gates. FPGAs have dedicated low-skew clock distribution nets and un-gated clocks, but SoC designs use gated clocks.

To make it possible to use the same RTL for both SoC and FPGA designs, the software provides the functionality to automate the translation. The functionality moves the SoC clock gating from the clock pins of sequential elements to the enable pins. This provides a logically-equivalent FPGA version, without altering the original RTL. For more information about converting gated clocks, see Converting Gated Clocks.



Guidelines for Integrated Clock Gating Cells (ICG)

These ASIC cells combine a latch and a clock gate to cut ASIC clock skew. Designs might contain cascading ICGs. Replace these cells with equivalent FPGA logic.



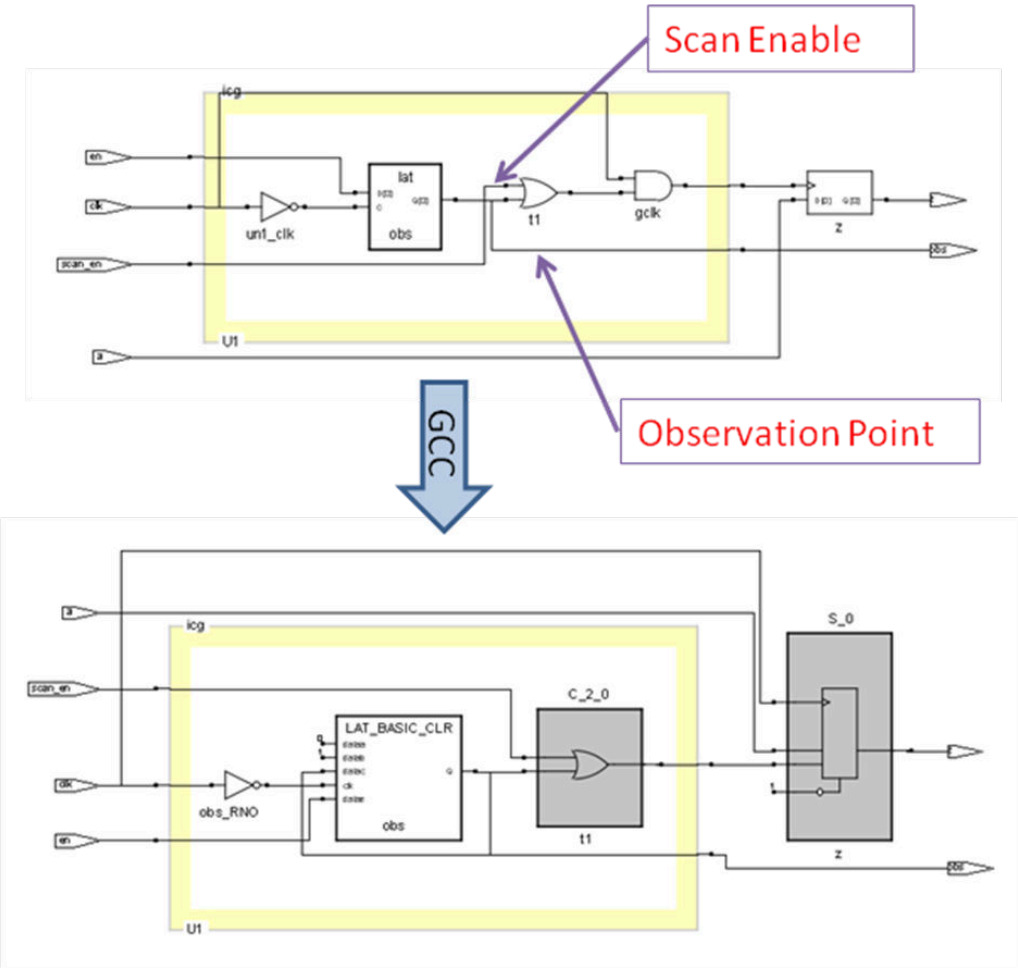
If the ICG cell has ASIC-only pins, such as a scan-enable input or an observation output, remove these pins. These signals force the tool to preserve intermediate signals in the clock-gating circuit so that they cannot be optimized. The following code shows one such example, where the data register is connected directly to the source clock, but the latch and OR gate for the ICG model are part of the enable logic.

```
module icg (clk, en, scan_en, obs, gclk);
input clk, en, scan_en;
output obs, gclk;
wire t0, t1;
assign t0 = (!clk) ? en : t0;
assign obs = t0;
assign t1 = t0 | scan_en;
assign gclk = t1 & clk;
endmodule
```

The following figure shows how the RTL is simplified to remove the intermediate points that are not used in the FPGA. The *scan_en*input and the *obs* observation point output, are tied to a constant, 0. If no other changes are made to the design, this model still fits,

Feedback

because the ports are unchanged. Also, the constant tied to the output propagates, and simplifies downstream logic.



This is the corresponding RTL after modification:

```
module icg (clk, en, scan_en, obs, gclk);
input clk, en, scan_en;
output obs, gclk;
wire t0, t1;
assign t0 = (!clk) ? en : t0;
assign obs = 1'b0 /* t0 */;
assign t1 = t0 | 1'b0 /* scan_en */;
assign gclk = t1 & clk;
endmodule
```

Feedback

Working with Gated and Generated Clocks

The following steps describe in detail the procedure needed to identify the gated and generated clocks in your design and set them up for automatic conversion with GCC.

1. Follow the guidelines for gated and generated clocks.

   See Guidelines for Generated Clocks and Guidelines for Gated Clocks.

2. Define the clocks by setting *create_clock* and *create_generated_clock* constraints.

   See Defining Clocks for Gated Clock Conversion for details.

3. Enable gated clock conversion with this command:

   ```
   option set fix_gated_and_generated_clocks 1
   ```

   This command applies globally to the design.

4. To specifically exclude individual clocked elements on critical paths from the global conversion, add a *syn_keep* directive to the input clock net.

   You can apply *syn_keep* in a source file or by setting a constraint:

   - *syn_keep* set on *gclk* in source file:

     ```
     module and_gate (clk, en, a, z);
     input clk, en, a;
     output reg z;
     wire gclk /* synthesis syn_keep = 1 */;
     assign gclk = en & clk;
     always @(posedge gclk) z <= a;
     endmodule
     ```

   - *syn_keep* set as a constraint on *gclk* in the FDC file:

     ```
     define_attribute {n:gclk} {syn_keep} {1}
     ```

5. To convert gated-clock and generated-clock circuits created from instantiated FPGA cells, make sure this default setting is in place:

   ```
   option set optimize_ngc 1
   ```

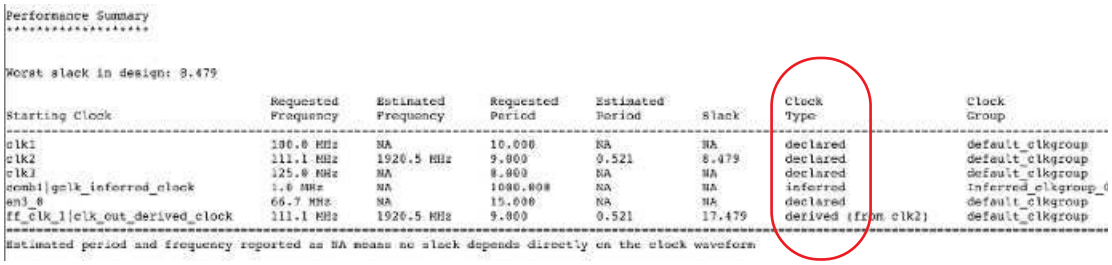
   The tool enables flops that connect directly to the clock. The tool does not convert instantiated cells where there are multiple declared clocks. For example, if a MUX is described using instantiated LUT primitives, the tool does not recognize the structure as a MUX, but sees it as multiple clocks feeding a LUT (for example, a 3-input AND gate).

6. Synthesize the design by running compile and map.

Feedback

For gated clocks, the software converts qualified flip-flops, counters, latches, synchronous memories, and instantiated technology primitives. The tool logically separates the gating from the clock and routes the gating to the clock enables on sequential devices, using the programmable routing resources of the FPGA. The ungated base clock is routed to the clock inputs of the sequential devices using the global clock resources. Because many gated clocks are normally derived from the same base clock, separating the gating from the clock allows a single global clock tree to be used for all gated clocks that reference the same base clock.

7. Analyze results.
   - Open the log file and check the*Performance Summary* section. Check the declared, generated, and derived clocks for issues like incorrect constraints or unsupported structures in clock logic, because gated clock conversion (GCC) is performed on these clocks. GCC is not performed on inferred clocks or on the system clock. Inferred clocks and the system clock are created when no clocks are defined in the fan-in logic of the clock pin of a sequential cell.



Performance Summary
****

Worst slack in design: 8.479

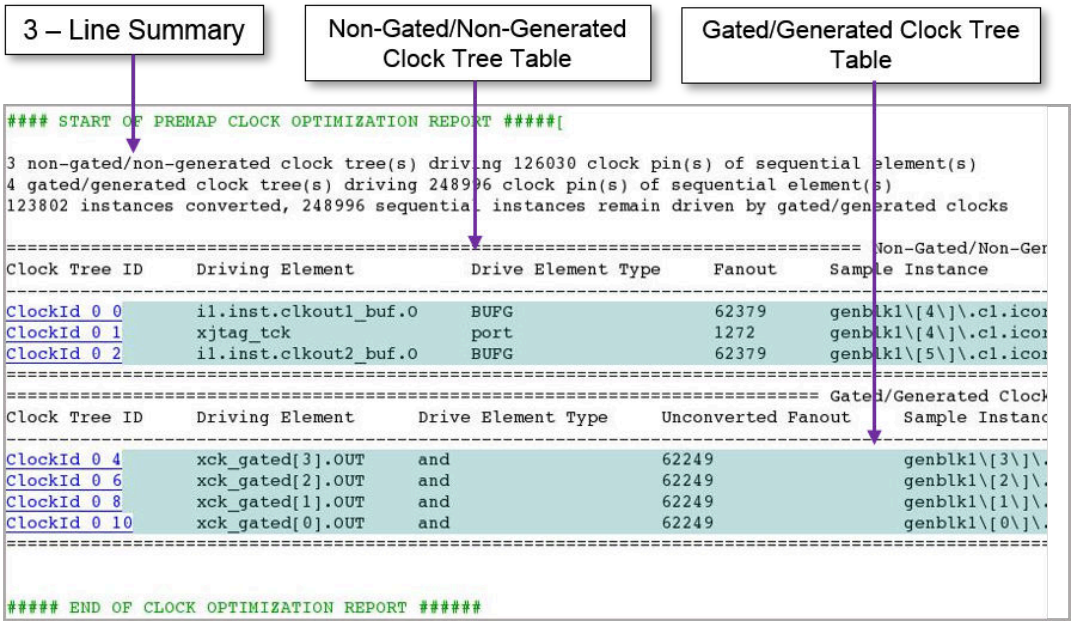
Starting Clock	Requested Frequency	Estimated Frequency	Requested Period	Estimated Period	Slack	Clock Type	Clock Group
clk1	100.0 MHz	NA	10.000	NA	NA	declared	default_clkgroup
clk2	111.1 MHz	1920.5 MHz	9.000	0.521	8.479	declared	default_clkgroup
clk3	125.0 MHz	NA	8.000	NA	NA	declared	default_clkgroup
comb1|gclk_inferred_clock	1.0 MHz	NA	1000.000	NA	NA	inferred	Inferred_clkgroup_0
en3_0	66.7 MHz	NA	15.000	NA	NA	declared	default_clkgroup
ff_clk_1|clk_out_derived_clock	111.1 MHz	1920.5 MHz	9.000	0.521	17.479	derived (from clk2)	default_clkgroup

Estimated period and frequency reported as NA means no slack depends directly on the clock waveform

   - Check for post-compilation warnings about inferred clocks or the system clock that might affect gated clock conversion. The following are examples of these warnings. In both cases, the clocks have no specified timing constraints. This prevents the conversion of gated or generated clocks and adversely impacts design performance.

     @W: MT529 : Found inferred clock clkName which controls *numClkPins* sequential elements including *instanceName*.

     @W: MT531: Found signal identified as System clock which controls *numClkPins* sequential elements including *instanceName*.
   - For a tabular list of some messages and explanations, see Interpreting Gated Clock Error Messages.
   - Check the *START OF CLOCK OPTIMIZATION REPORT* section of the log file for a report on the conversion process.



See Interpreting Gated Clock Error Messages for a description of conversion-specific error messages you might see.

Defining Clocks for Gated Clock Conversion

Follow these guidelines to define gated and generated clocks in the design before you synthesize it.

1. Make sure that the gated clocks you define satisfy these requirements:
   - Gated clock logic must consist only of combinational logic. Derived clocks that are the output of registers are not converted.
   - Additionally, the combinational logic must satisfy the conditions described in the following table. See Combinational Logic and GCC Examples.

Condition 1	For at least one set of gating input values, the value output for the gated clock must be constant and not change as the base clock changes.
Condition 2	For at least one value of the base clock, changes in the gating input must not change the value output for the gated clock.

   - The sequential primitive clocked by the gated clock must be a supported object. The tools support gated-clock conversion for most sequential primitives.

- If you have black boxes driven by gated clocks, use the *syn_force_seq_prim, syn_isclock*, and *syn_gatedclk_en* directives to define the clock and clock enable inputs to the black box. The tool does not check the functionality of the contents of the black box. See Gated Clock Definitions for Black Boxes for examples.

2. To identify a net as a clock, specify a period or frequency constraint for either the gate or the clock using the *create_clock* constraint.
   - Refer to the general guidelines in Converting ASIC Clocks.
   - Set the constraint in the FDC constraint file, using syntax like this:

     ```
     create_clock -name {clk} -freq 10.000 -clockgroup default_clkgroup
     ```

3. Define the relationship between generated clocks and their sources by specifying *create_generated_clock* constraints.

   Do the following to specify the constraint:

   - Specify the master clock source (a clock source pin in the design). Use the schematic viewer to identify the names, and copy and paste them into the constraint.
   - Specify the frequency for the generated clocks as a multiplication or division factor (*-multiply_by* |*-divide_by)*. If you use *-multiply_by*, use *-duty_cycle* to specify the percentage of the high pulse width to use for the generated clock.
   - Specify whether the generated clock signal should be inverted (*-invert*).
   - Specify the clock edges (*-edges*).

   The example shows some generated clock constraints. See GCC Examples for Generated Clocks for additional examples, and create_generated_clock for details about the command syntax.

   ```
   set CLK 48
   create_clock -name CLK -period $CLK [get_ports CLK ]
   create_generated_clock -name CLK1 -add [get_nets CLKOUT1]
     -source [get_ports CLK] -master_clock [get_clocks CLK] -edges {1 2
   5}
   create_generated_clock -name CLKPROCBUS -add [get_nets u_Proc.CLKOUT]
   -source [get_ports CLK] -master_clock [get_clocks CLK]-edges {1 2 5}
   create_generated_clock -name CLK2 -add [get_nets CLKOUT2] -source
   [get_ports CLK] -master_clock [get_clocks CLK] -edges {1 2 9}
   set_clock_groups -asynchronous {CLK CLK1 CLKPROCBUS CLK2}
   ```

Follow these guidelines when setting generated clock constraints:

- Define gated clocks at the nodes to be connected to the clock pins of the sequential cells.
- Identify only one input to the combinational logic for the gated clock as a base clock. If you do not specify an explicit constraint on the clock net or if you set a global frequency constraint, the clocks are not converted during synthesis.
- Define one clock for every fan-in cone clock pin on sequential devices. If you do not do this, the tool cannot trace the clock source. Typically, the tool traces the clock pin back to the first multiple-input gate. If it is faced with a choice of inputs at this point, the tool infers a clock at the output of that cell. The inferred clocks are not user-defined, so the tool does not convert them.

  If you define more than one clock per clock pin, the tool does not perform gated clock conversion.
- Set this constraint if a clock circuit contains both a clock generator and a gating element, to define the relationship.

4. To provide more meaningful names for automatically-generated clocks, use *create_generated_clock*.

   If you have defined the associated master clock, the tool automatically generates clocks at the output of clock-modifying blocks such as MMCMs and PLLs and names them. To rename them, use *create_generated_clock* as shown in example. Renamed clocks are forward-annotated in the *xdc* file, and Vivado recognizes the renamed clocks.

   create_generated_clock -name {mem_clk} [get_pins {mmcm0.CLKOUT0}]

5. To convert datapath latches that are driven by generated clocks and are set or reset by asynchronous signals, follow these steps:

   - Ensure that the combinational logic is driven by flip-flops.
   - Make sure that the input flip-flops do not have an active set or reset (an active-low reset must be tied high, or an active-high set must be tied low). The rules apply to all the input flip-flops in the cone. In addition, all input flip-flops must be driven by the same edge of the same clock.
   - Set the option to force asynchronous flip-flops to be converted:

     ```
     option set force_async_genclk_conv 1
     ```

     The tool does not convert gated or generated clocks if the driving flip-flop has asynchronous sets or resets, unless the flip-flop or datapath latch being converted also has asynchronous controls tied to the same net as the driving flip-flop. With the

option set, the tool ignores asynchronous sets or resets on clocks generated by flip-flops and converts its loads. See Figure 15.

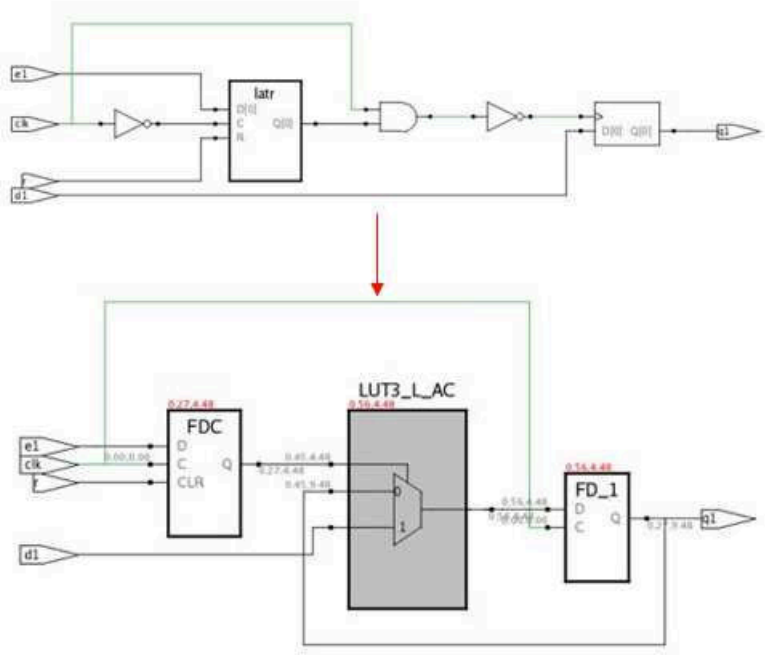
Note that forcing gated clock conversion with this option can miss edges introduced by the asynchronous controls that the original circuit detected. It might also trigger on invalid edges that were masked by the asynchronous controls in the original circuit.

6. Stability latches in the Integrated Clock Gating (ICG) circuit that are moved to the enable path of sequential loads after gated and generated clock conversion (GCC) can cause timing violations for Vivado place and route. Therefore, in the pre-map stage the synthesis tool removes the stability latches in the ICG cell based on the following:
   - A latch on the clock path is converted to a mux and flip-flop.
   - The mux select pin is connected to 1 or 0, depending on the sensitive clock edges of the sequential loads. For a rising clock edge, the mux selects 0. For a falling clock edge, the mux selects 1. Then constant propagation is implemented through the mux.

   To convert ICG latches that are driven by gated and generated clocks and are set or reset by asynchronous signals, follow these steps:
   - Ensure that the ICG latch signals that drive the gated and generated clocks are either asynchronous set or reset. If both asynchronous set and reset signals for the ICG latch are used, then the latch is seen as an unsupported load and is not removed.
   - Make sure that the ICG latch drives only a positive (rising) or negative (falling) edge flip-flop. Do not mix rising and falling clock edges to the loads of the flip-flop.
   - Set the option to force asynchronous flip-flops to be converted: *option set-force_async_genclk_conv 1*

   The following example shows an ICG latch with asynchronous reset signals driving a negedge load and what occurs after GCC optimizations.

Feedback




```
ICG Latch Removal Summary:
Number of ICG latches removed:  1
Number of ICG latches not removed:  0
```

For details and more examples, see Integrated Clock Gating (ICG) Stability Latch Removal Examples.

7. To specify clock latency, follow these steps:
   - Start with an MMCM output pin that drives a global net, and declare a *create_generated_clock* constraint for it.
   - Set the *haps_global_clock* attribute to define clock latency.

   For example:

```
create_generated_clock -name {gen_clk} -source [get_ports {clk}]
-divide_by 2 [get_pins {U_clk_gen.MMCM_i.CLKOUT0}] define_attribute
[get_pins {U_clk_gen.MMCM_i.CLKOUT0}] { haps_global_clock } {1}
```

Defining Generated Clocks with create_generated_clock

Define the relationship between generated clocks and their sources with *create_generated_clock* constraints. The *create_generated_clock* constraint provides

Feedback

a way to specify timing for gated and generated clocks. Use it with the *create_clock* constraint to modify timing.

1. Specify the *create_generated_clock* constraint in the FDC file or through the Clocks tab in the SCOPE GUI.

   Set the *create_generated_clock* constraint if a clock circuit contains both a clock generator and a gating element, to define the relationship. See create_generated_clock for details about the command syntax referred to in this procedure.

2. In the constraint, define a clock source pin as the master clock source (*-source*).

   Use the schematic viewer to identify the source pin names, and copy and paste them into the constraint.

3. Specify the frequency for the generated clocks with *-multiply_by* |*-divide_by.*

   The frequency is set as a multiplication or division factor of the source clock, using the appropriate argument. If you use *-multiply_by*, also use *-duty_cycle* to specify the percentage of the high pulse width to use for the generated clock.

4. Specify whether the generated clock signal should be inverted (*-invert*).

5. Specify the clock edges (*-edges*).

   The example shows some generated clock constraints. See GCC Examples for Generated Clocks for additional examples.

   ```
   set CLK 48
   create_clock -name CLK -period $CLK [get_ports CLK ]
   create_generated_clock -name CLK1 -add [get_nets CLKOUT1] -source
   [get_ports CLK] -master_clock [get_clocks CLK] -edges {1 2 5}
   create_generated_clock -name CLKPROCBUS -add [get_nets u_Proc.CLKOUT]
   -source [get_ports CLK] -master_clock [get_clocks CLK]-edges {1 2 5}
   create_generated_clock -name CLK2 -add [get_nets CLKOUT2] -source
   [get_ports CLK] -master_clock [get_clocks CLK] -edges {1 2 9}
   set_clock_groups -asynchronous {CLK CLK1 CLKPROCBUS CLK2}
   ```

6. To provide more meaningful names for automatically-generated clocks, use *create_generated_clock*.

   If you have defined the associated master clock, the tool automatically generates clocks at the output of clock-modifying blocks such as MMCMs and PLLs and names them. To rename them, use *create_generated_clock* as shown in example. Renamed clocks are forward-annotated in the *xdc* file, and Vivado recognizes the renamed clocks.

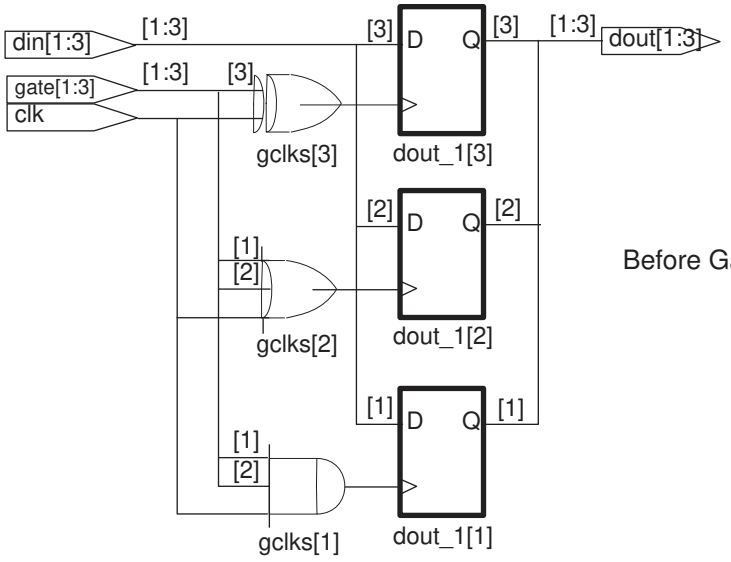
   create_generated_clock -name {mem_clk} [get_pins {mmcm0.CLKOUT0}]

Feedback

Combinational Logic and GCC Examples

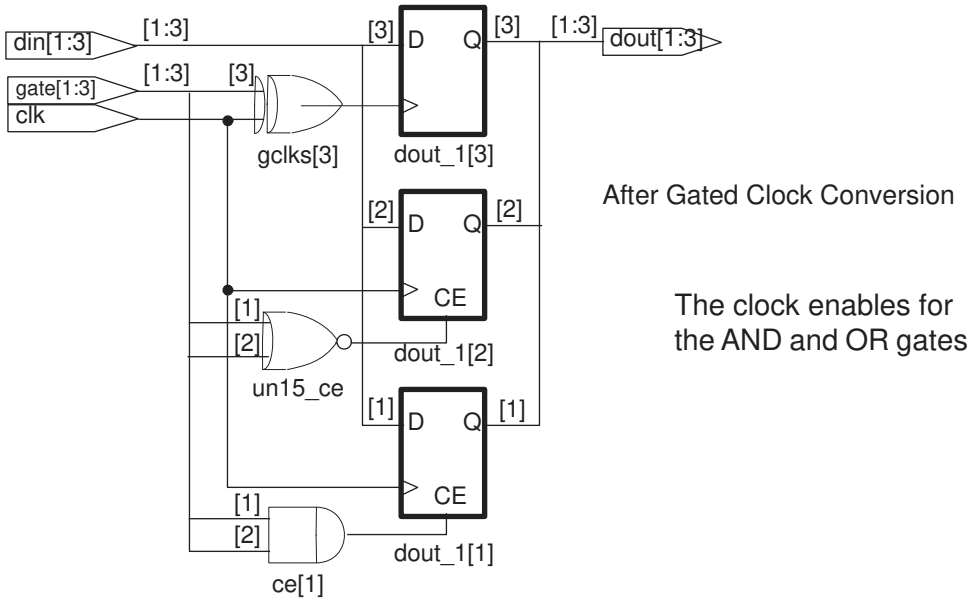
The correct logic format requirements are illustrated with the simple gates shown in the following figures. When the software synthesizes the gated clocks defined, it converts clock enables for the AND and OR gates, but does not convert the exclusive-OR gate shown in the second figure. The following table explains the details.

Gate	Condition1	Condition 2	Result
AND gclks[1]	MetIf either *gate[1]* or *gate[2]* is 0, then *gclks[1]* is 0, independent of the value of *clk*.	MetIf *clk* is 0, then *gclks[1]* is 0, independent of the values of *gate[1]* and *gate[2]*.	*gclks[1]* is successfully converted to clock-enable format.
OR gclks[2]	MetIf either *gate[1]* or *gate[2]* is 1, then *gclks[2]* is 1 independent of the value of *clk*.	MetIf *clk* is 1, then *gclks[2]* is 1 independent of the value of *gate[1]* or *gate[2]* which satisfies the second condition	*gclks[2]* is successfully converted to the clock-enable format
XOR gclks[3]	Not met*gclks[3]* continues to toggle, regardless of the value of *gate[3]*		Exclusive OR, *gclks[3]*, is not converted.



Before Gated Clock

Feedback



Gated Clock Definitions for Black Boxes

The following Verilog example specifies the *syn_force_seq_prim*, *syn_isclock*, and *syn_gatedclk_clock_en* directives, ensuring that the gated clocks that drive the black box get converted during FPGA synthesis.

Verilog

```
module bbe (ena, clk, data_in, data_out)
/* synthesis syn_black_box */
/* synthesis syn_force_seq_prim="clk" */
;
input clk
/* synthesis syn_isclock = 1 */
/* synthesis syn_gatedclk_clock_en="ena" */;
input data_in,ena;
output data_out;
endmodule
```

GCC Examples for Generated Clocks

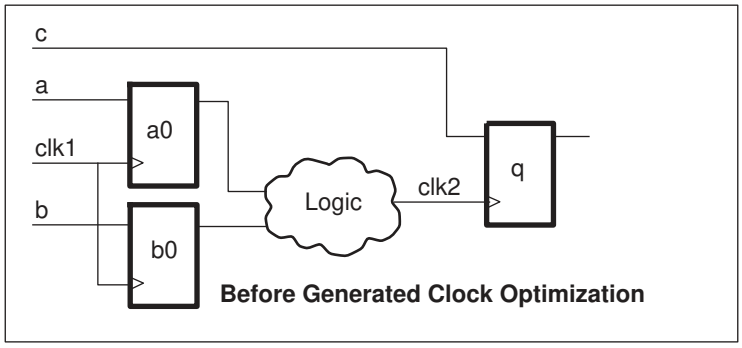
These examples show how gated clock conversion works with generated clocks.

Generated Clock from Combinational Logic

The *q*flip-flop is driven by a generated clock that originates from combinational logic that is driven by the *ao* and *bo* flip-flops. These flip-flops are driven by the initial clock (*clk1*).

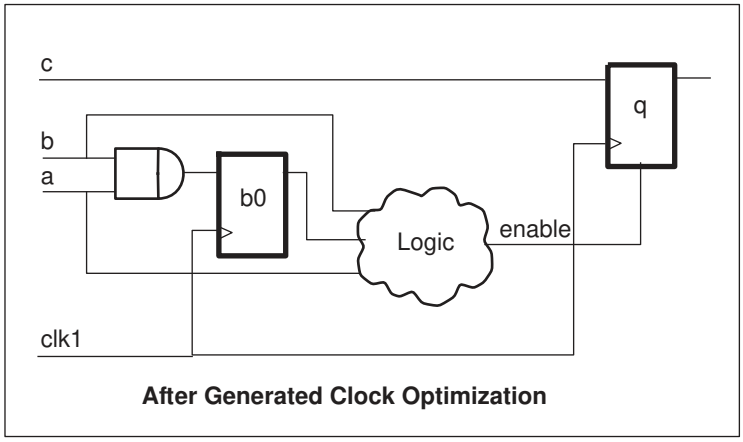
Feedback

Figure 11 Generated Clock from Combinational Logic



The GCC process replaces *q* with an enable flip-flop that is clocked by the initial clock and enabled by combinational logic based on the *a* and *b* inputs:

Figure 12 Optimization After Generated Clock

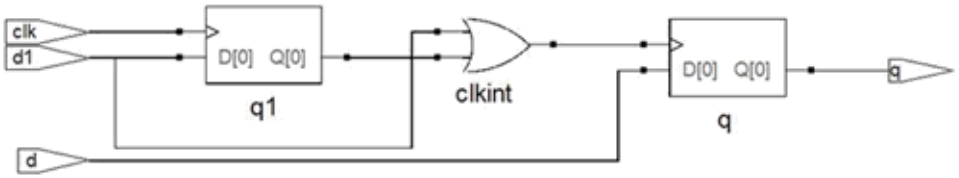


Generated Clock Followed by a Gated Clock

This example has a register that is clocked by a generated clock, then followed by a gated clock. If the generated clock constraint also includes the enable, the register might be optimized away during synthesis and replaced with a constant value. The GCC process converts the generated and gated clock logic into a clock and an enable; the enable might never be active.

Feedback

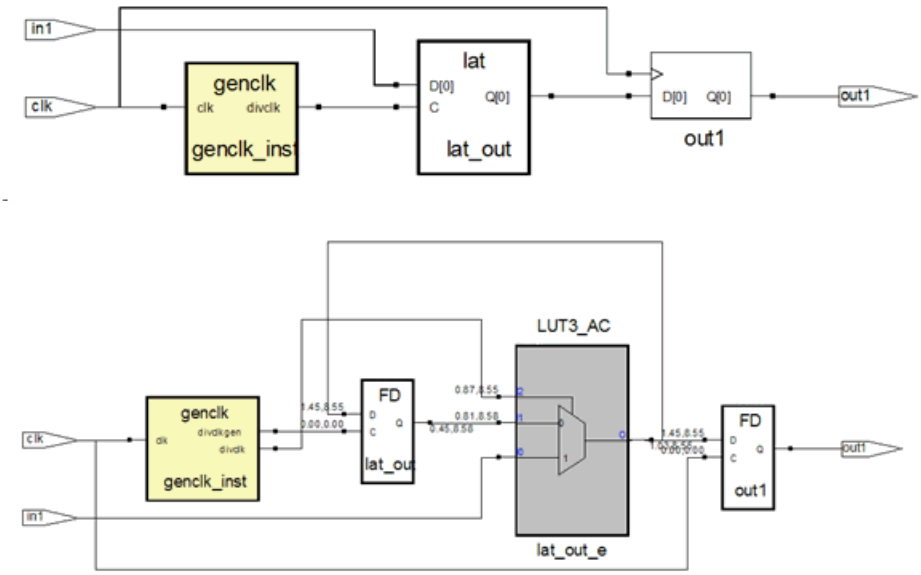
Figure 13 Generated Clock Followed by a Gated Clock



Datapath Latch Conversion

This example shows a latch on the datapath that is clocked by a generated clock. GCC converts the circuit to a flip-flop and multiplexer.

Figure 14 Datapath Latch Conversion

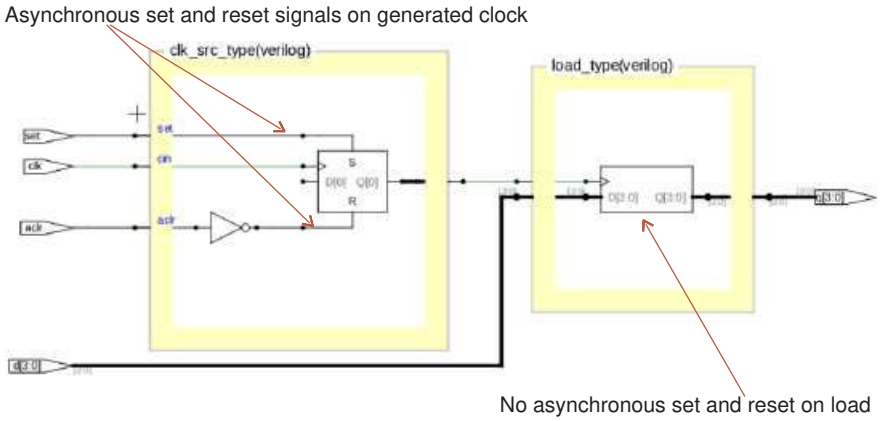


Conversion with Asynchronous Set or Reset Signals

In this example, there are asynchronous set and reset signals on the clock generated from the flip-flop, but the load does not have asynchronous set and reset signals. By default, the tool does not run GCC in this scenario. However, when *force_async_genclk_conv* is set to 1, GCC handles this generated clock scenario and converts the circuit to a flip-flop and multiplexer.

Feedback

Figure 15 Asynchronous Set or Reset Signals on Generated Clock



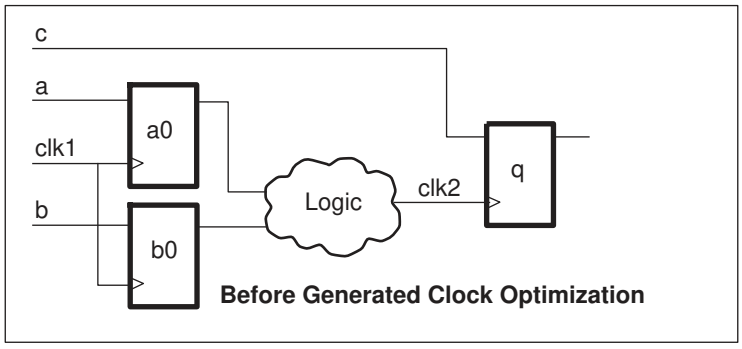
GCC Examples for Generated Clocks

These examples show how gated clock conversion works with generated clocks.

Generated Clock from Combinational Logic

The *q*flip-flop is driven by a generated clock that originates from combinational logic that is driven by the *ao* and *bo* flip-flops. These flip-flops are driven by the initial clock (*clk1*).

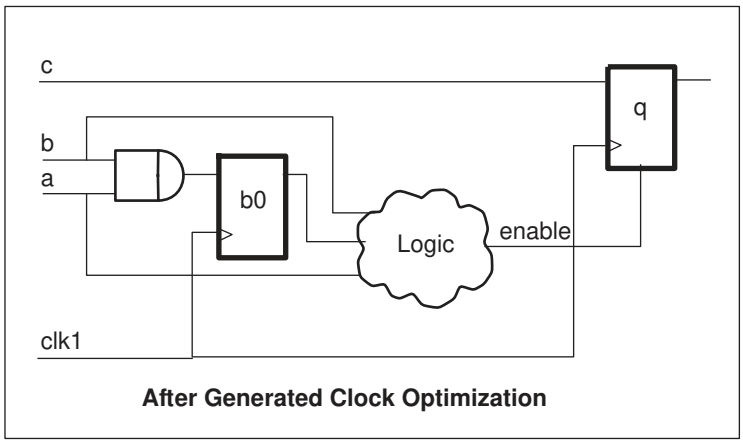
Figure 16 Generated Clock from Combinational Logic



Feedback

The GCC process replaces *q* with an enable flip-flop that is clocked by the initial clock and enabled by combinational logic based on the *a* and *b* inputs:

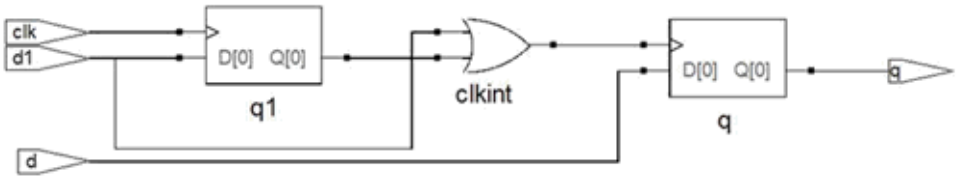
Figure 17 Optimization After Generated Clock



Generated Clock Followed by a Gated Clock

This example has a register that is clocked by a generated clock, then followed by a gated clock. If the generated clock constraint also includes the enable, the register might be optimized away during synthesis and replaced with a constant value. The GCC process converts the generated and gated clock logic into a clock and an enable; the enable might never be active.

Figure 18 Generated Clock Followed by a Gated Clock

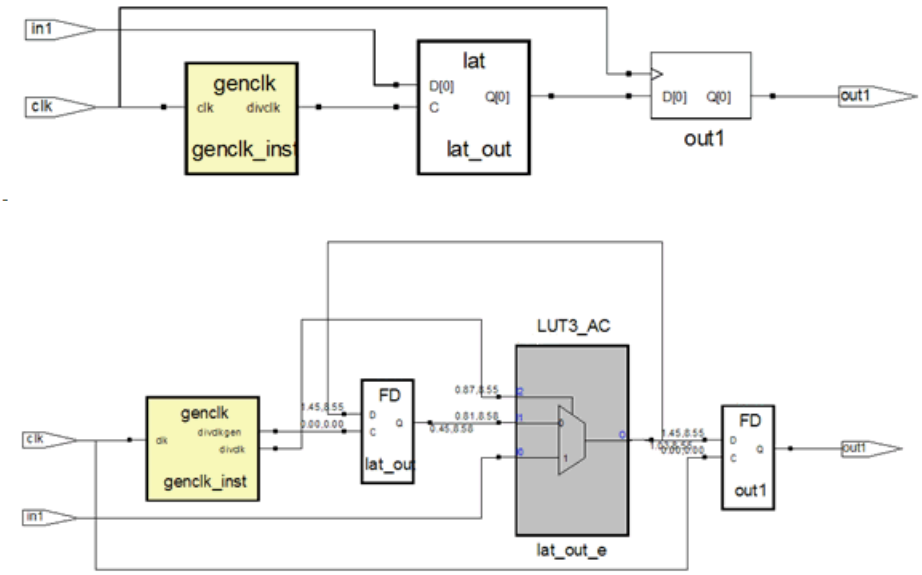


Datapath Latch Conversion

This example shows a latch on the datapath that is clocked by a generated clock. GCC converts the circuit to a flip-flop and multiplexer.

Feedback

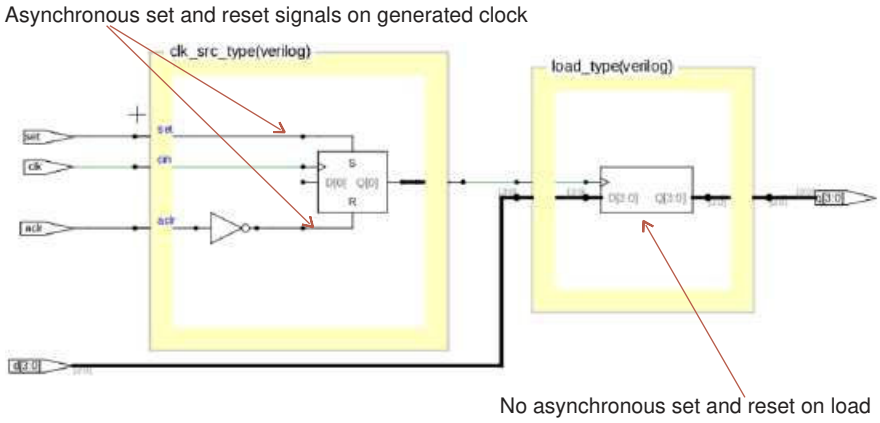
Figure 19 Datapath Latch Conversion



Conversion with Asynchronous Set or Reset Signals

In this example, there are asynchronous set and reset signals on the clock generated from the flip-flop, but the load does not have asynchronous set and reset signals. By default, the tool does not run GCC in this scenario. However, when *force_async_genclk_conv* is set to 1, GCC handles this generated clock scenario and converts the circuit to a flip-flop and multiplexer.

Figure 20 Asynchronous Set or Reset Signals on Generated Clock



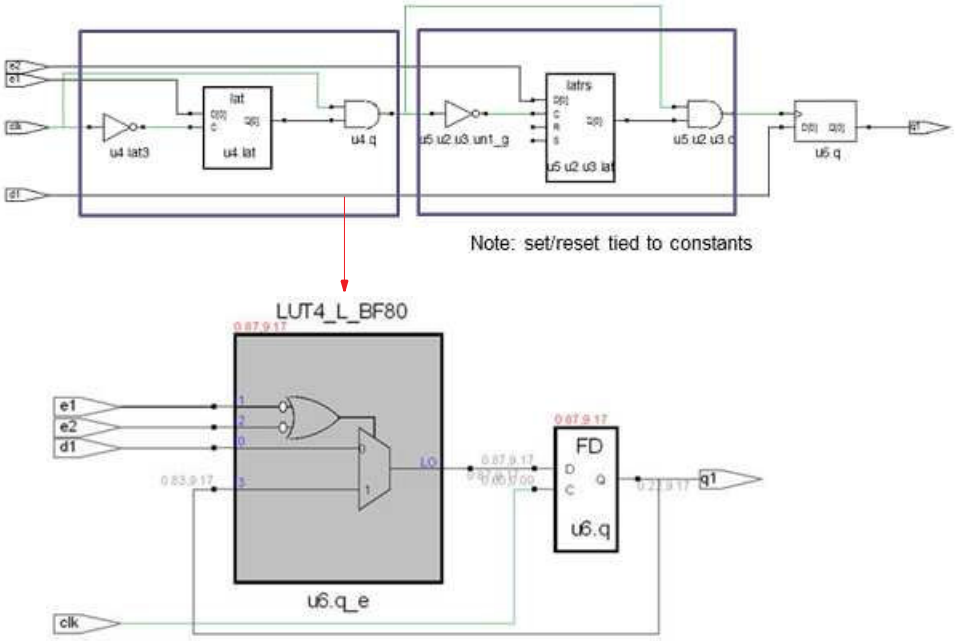
Feedback

Integrated Clock Gating (ICG) Stability Latch Removal Examples

After GCC, the mux is removed and the flip-flop is retained or removed. The following examples show how the ICG stability latches are implemented for various ICG conditions.

The following example shows a cascading ICG that drives a positive edge flip-flop and what occurs after GCC optimizations.

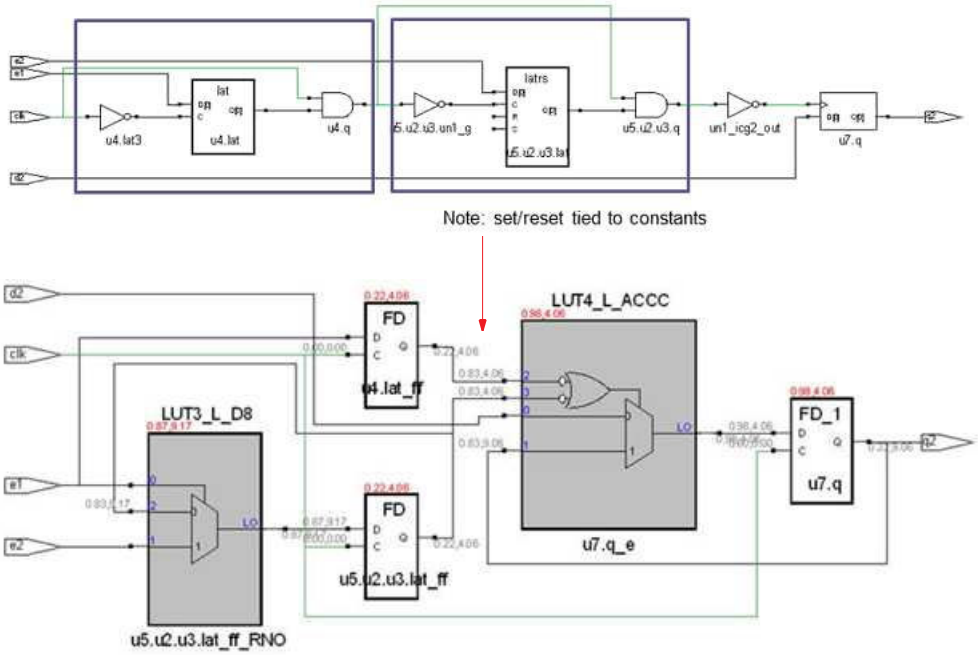
Figure 21 Cascading ICG with positive edge flip-flop



The following example shows a cascading ICG that drives a negative edge flip-flop and what occurs after GCC optimizations.

Feedback

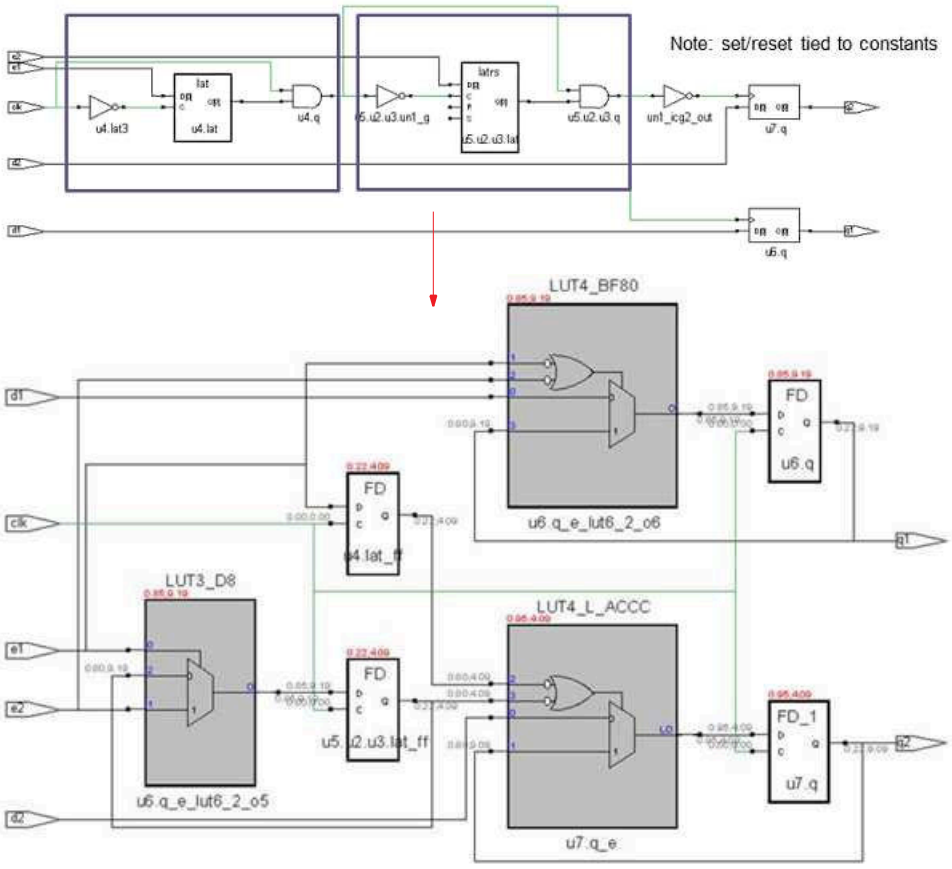
Figure 22 Cascading ICG with negative edge flip-flop



The following example shows a cascading ICG that drives a positive and negative edge flip-flop and what occurs after GCC optimizations.

Feedback

Figure 23 Cascading ICG with positive and negative edge flip-flop



You can check the *ICG Latch Removal Summary* in the pre-map section of the log file for the:

- Number of ICG latches removed
- Number of ICG latches not removed
- Following message that is generated:

```
@N: ICG latches not removed are assigned the property icg_retain set to
1. Use Tcl find to locate the latches with this property.
```

In the *ICG Latch Removal Summary*, note that:

- ICGs that drive black boxes are not counted in the *Number of ICG latches not removed*.
- Latches that are not removed can be optimized away later, after GCC. These latches removed by GCC are not counted in the *ICG Latch Removal Summary*.

Feedback

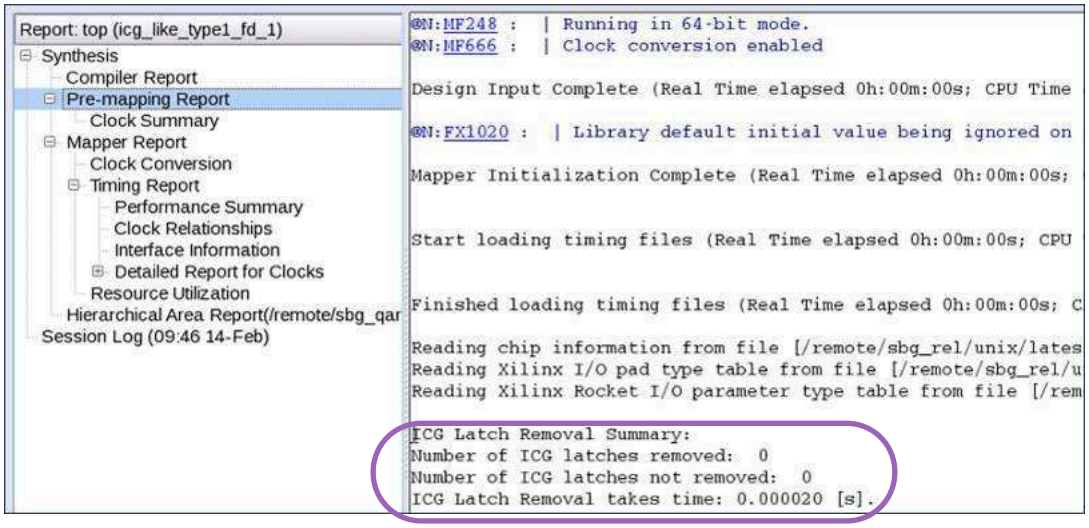
Unsupported Loads for ICG Latches

The following examples provide conditions why ICG stability latches are not removed.

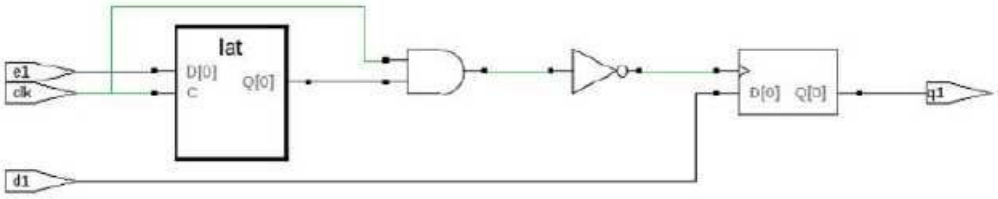
Examples

Example 6 Unsupported latches not considered to be ICG latches

Latches *not* considered to be ICG latches, do not get reported in the *ICG Latch Removal Summary*.

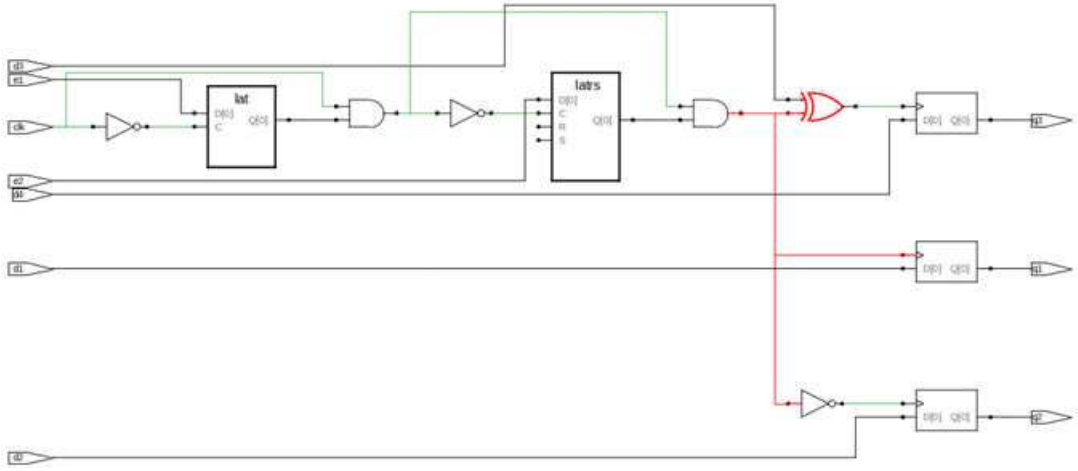


The following figure might include an unsupported latch, where an incorrect phase of the clock is connected to the latch, such that it is not removed.



Feedback

Example 7 ICGs driving unsupported loads



The ICG circuitry drives an unsupported load for an XOR gate. This results in the ICG circuitry not being removed.

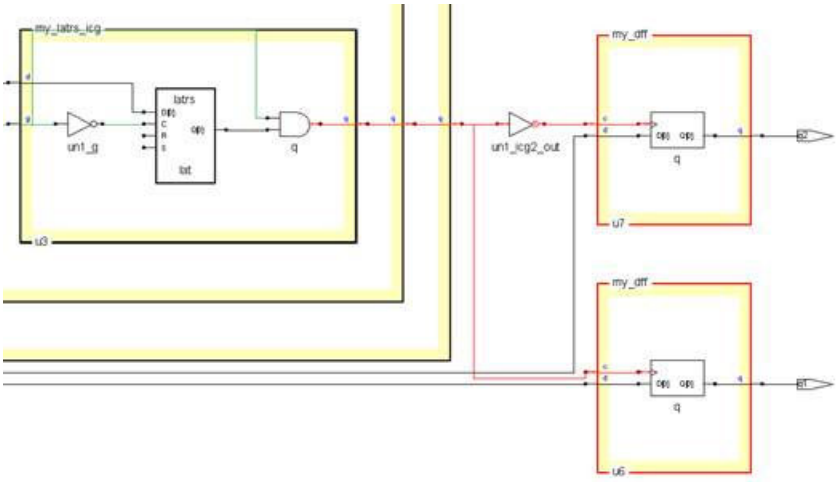
The following warning messages are logged:

```
@W:MF760: | ICG Latch Removal: Unable to remove ICG latch u4.lat because
it drives unsupported instance gclk.

@W:MF760: | ICG Latch Removal: Unable to remove ICG latch u5.u2.u3.lat
because it drives unsupported instance gclk.

ICG Latch Remove Summary: Number of ICG latches removed: 0 Number of ICG
latches not removed: 2
```

Example 8 ICGs driving compile points for loads with mixed clock edges



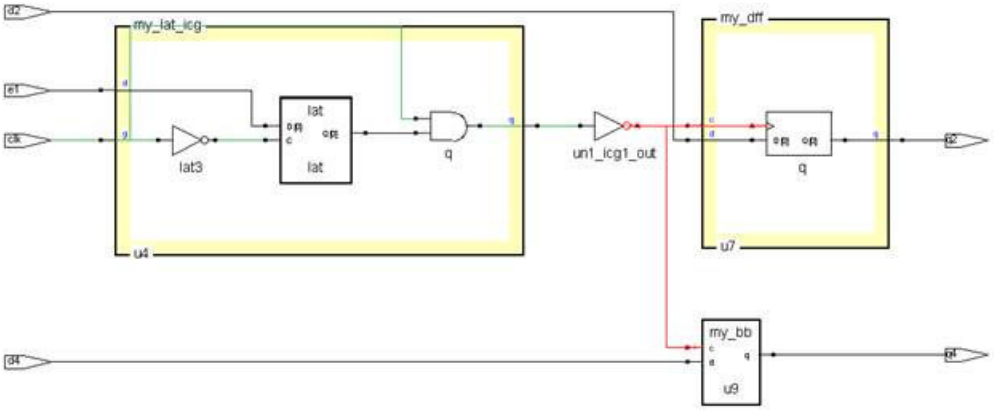
Feedback

The posedge and negedge register loads are inside hard compile points. Since the registers have two different clock edges and are inside compile points, the ICG circuitry is not removed.

The following warning message is logged:

```
@W:MF758: | ICG Latch Removal: Unable to remove ICG latch u5.u2.u3.lat
because it drives a compile point and rising/falling clock edges
```

Example 9 ICGs driving both black boxes and registers

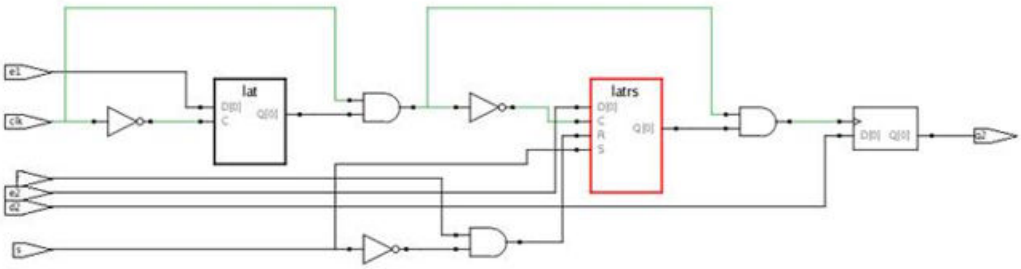


The ICG circuitry driving the black box gets replicated and is not removed. However, the original ICG circuitry that drives *u1.q* is removed.

The following warning message is logged:

```
@N:MF761: | ICG Latch Removal: Blackbox load found on ICG latch u4.lat;
latch replicated and not removed for blackbox load.
```

Example 10 ICG latch with asynchronous set and reset signals




```
ICG Latch Removal Summary:
Number of ICG latches removed:  0
Number of ICG latches not removed :  1
```

Feedback

The ICG latch cannot be removed because the ICG latch has asynchronous set and reset signals.

The following warning message is logged:

```
@W:MF759: | ICG Latch Removal: Unable to remove ICG latch u4.lat because
it drives non-stability latch u5.lat
```

Interpreting Gated Clock Error Messages

This procedure describes how to analyze gated clock conversion (GCC) and troubleshoot the design based on messages reported after the run.

1. Check the log file and GCC report for the results of gated clock conversion at different points in the design flow.

   You can check the design before GCC optimization for a baseline. After GCC, the log file reports the number of keepbufs and constraints as well as an ICG report. The post-GCC report summary contains details of the GCC implementation.

2. Check the messages reported in the *Gated/Generated Clocks* section of the clock conversion report.

   - The general procedure is to trace the clock path and check that the clocks are properly constrained and that the clock structure is one that is supported for gated clock conversion.
   - For information how to deal with some of the specific situations reported, see the following table.

     The following table describes some gated clock conversion error messages reported in the *Gated/Generated Clocks* section of the clock conversion report, in alphabetical order.

Error Message	Explanation
Asynchronous set/reset mismatch prevents generated clock conversion	Unshared asynchronous signals have been detected between a FF-derived clock circuit and its sequential load.
Can't determine input clock driver	Self-explanatory. Specify the driver.
Clock conversion disabled	Gated clocks have been detected, but clock conversion is not enabled.
Clock from cock mux	The mux is fed by a top-level clock on one input and a generated clock (derived from same top-level clock) on the second input. See Clock from Clock Mux.

Feedback

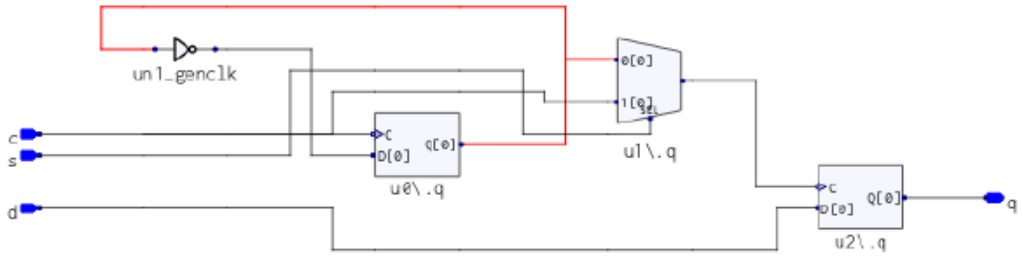
Error Message	Explanation
Clock from gated/generated/ uncosntrained technology primitive	There is a gated or generated clock upstream from the instance. See Clock from Gated/Generated/Unconstrained Technology Primitive.
Clock from generated clock directive	There is a *create_generated_clock* constraint on a mux output. See Clock from Generated Clock Directive.
Clock on mux select line	A clock signal is driving a select pin on a mux. See Clock on Mux Select Line.
Clock propagation blocked by fixed hierarchy	Clock property has been blocked by a fixed hierarchy which prevents clock conversion from propagating upstream.
Clock propagation blocked by hard hierarchy	Clock property has been blocked by a hard hierarchy which prevents clock conversion from propagating upstream.
Clock propagation blocked by locked hierarchy	Clock property has been blocked by a locked hierarchy which prevents clock conversion from propagating upstream.
Clock propagation blocked by syn_keep	Clock property has been blocked by a *syn_keep* property which prevents clock conversion from propagating upstream.
Clock source is constant	Self-explanatory.
Clock source is invalid for GCC	A general introductory note that precedes other messages with specific details.
Combinational loop in clock network	Self-explanatory. Fix the combinational loop.
Derived clock on input	There is an illegal clock structure for GCC, with a generated clock derived from a top-level clock that was not defined. See Derived Clock on Input.
FF-derived clock conversion disabled	FF-derived clocks have been detected, but clock conversion is not enabled.
Gating structure creates improper gating logic	Self-explanatory. Analyze and fix logic.
Illegal instance on clock path	Self-explanatory. Analyze and fix logic.
Incompatible clocks on mux inputs.	There is an illegal clock structure for GCC, with a generated clock derived from a top-level clock that was not defined. See Incompatible Clocks on Mux Inputs.

Error Message	Explanation
Inferred clock from port	Self-explanatory notification. See Inferred Clock from Port for an example.
Input clock depends on output	Self-explanatory.
Instance clocked by multiple variants of clock	The clock tree contains two phases of a clock. See Instance Clocked by Multiple Variants of Clock.
Latch gated by OR originally on clock tree	Self-explanatory.
Multiple clock inputs on sequential instance	Self-explanatory. Analyze and fix logic.
Multiple clocks on generating sequential element	Self-explanatory. Analyze and fix logic.
Multiple clocks on instance	Multiple clocks found feeding an instance of a clock tree. See Multiple Clocks on Instance.
Need declared clock or clock from port to derive clock from ff	Self-explanatory. Specify the clock.
No clocks found on inputs	No clocks found feeding an instance of a clock tree.
No generated or derived clock directive on output of sequential instance	Self-explanatory. Specify the clock
No hierarchical driver	Self-explanatory. Specify the driver.
Signal from port	Self-explanatory.
Unconverted clock gate	Self-explanatory.
Unable to determine clock driver on net	Self-explanatory. Specify the clock driver.
Unable to determine clock input on sequential instance	Self-explanatory. Analyze and fix logic.
Unable to follow clock across hierarchy	Self-explanatory. Analyze and fix logic
Unable to use latch as gated clock generator	Self-explanatory. Analyze and fix logic.

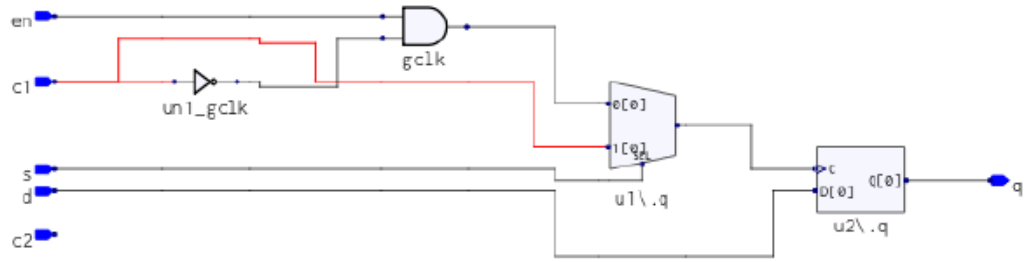
Feedback

Clock from Clock Mux

This message occurs when a mux is fed by a top-level clock on one input and a generated clock (derived from same top-level clock) on the second input:



Another case that generates this message is when a mux is fed by a clock on one input and an inverted clock on the second input.

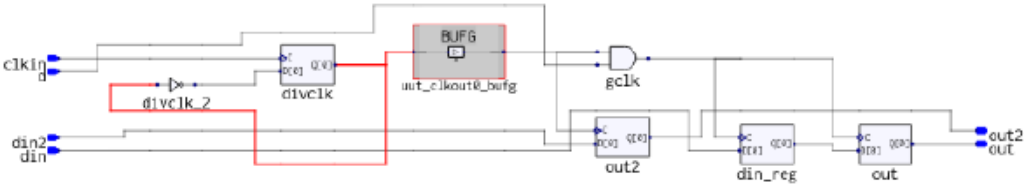


Action

A workaround is to create a faster clock using a generated clock (divide by 2) to replace the top-level clock.

Clock from Gated/Generated/Unconstrained Technology Primitive

This message warns that there is a gated/generated clock upstream from the technology clock primitive.



Action

This scenario can result in clock skew during place and route. Check for hold time violations on this path in the Vivado tool.

Clock from Generated Clock Directive

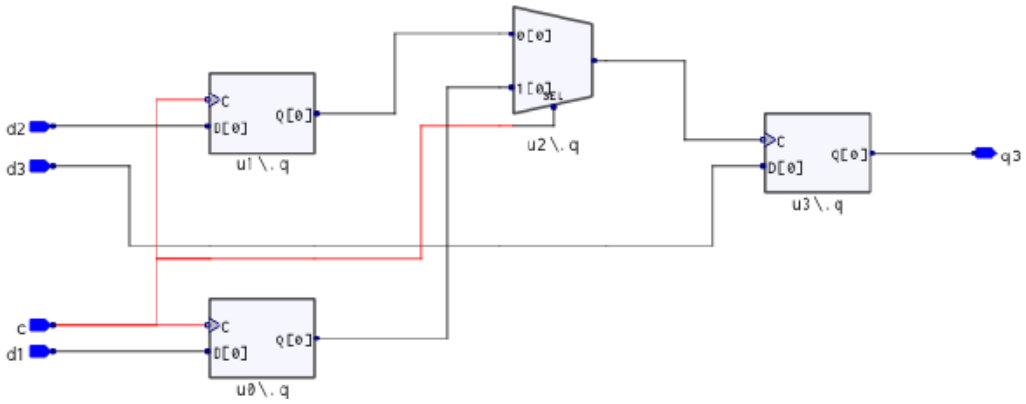
This message indicates that there is a *create_generated_clock* constraint on a mux output because the mux inputs are incompatible.

Action

This message about the constraint is informational. Address the root cause as described in Incompatible Clocks on Mux Inputs.

Clock on Mux Select Line

This message indicates an unsupported clock structure where a clock signal drives a select pin on a mux. When a select pin of a mux is driven by a clock, clock conversion stops at the output of the mux.

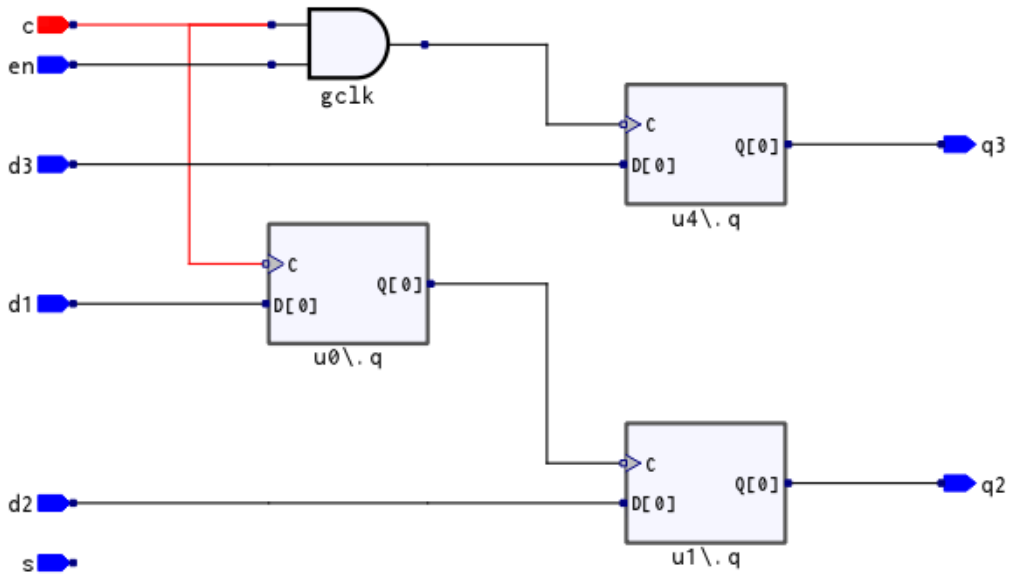


Action

Check the compile netlist to determine whether this structure is present in the RTL. If it is not, check that it is not a GCC tool error.

Derived Clock on Input

This message indicates a clock structure that is not legal for GCC: when the top-level clock is not defined and a generated clock is derived from it. See the example where the top-level clock *c* is not defined.

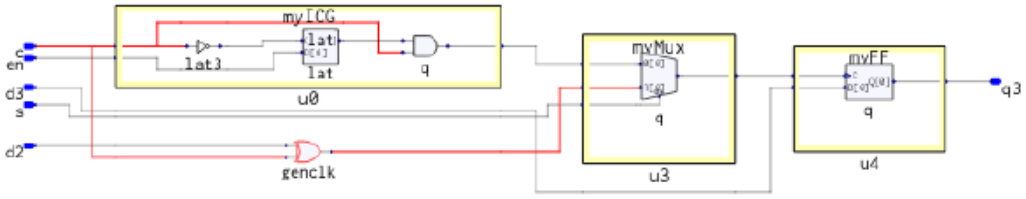


Action

Start with the sample instance (*u1* in the following image) which is reported as failed and trace its clock pin back to clock generator (*u0*). Then trace back the clock pin of clock generator to the undefined top level port *c*. In the FDC file, constrain the top-level port clock with a *create_clock*constraint, and constrain the derived clock with *create_generated_clock*.

Incompatible Clocks on Mux Inputs

This message is reported when the cone of logic driving one input leg of the mux cannot be optimized. The example illustrates this by showing an e XOR gate, which is not a supported structure of GCC, driving an input leg of a mux.

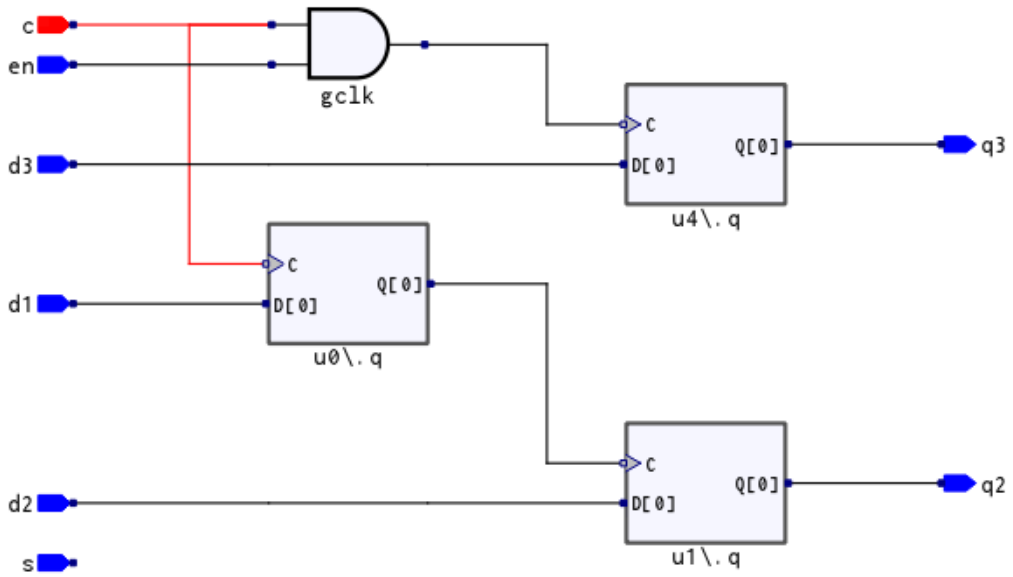


Action

Analyze the cone of logic driving each leg of mux for any unsupported clock structures.

Inferred Clock from Port

This message is reported when the top-level clock is not defined. The following example shows the instance *u4* driven by a top-level clock, *c*, which is not defined.

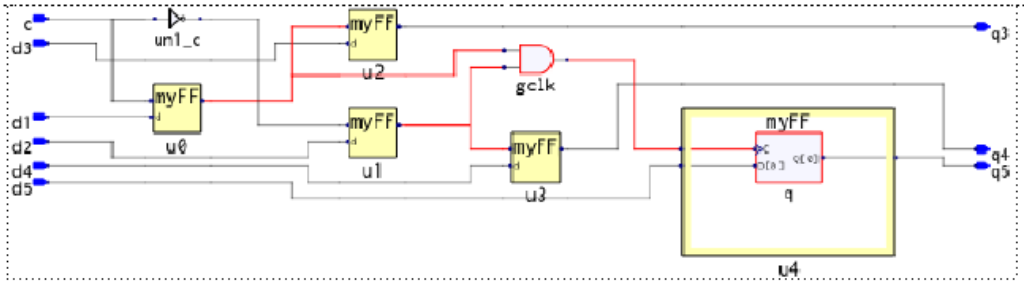


Action

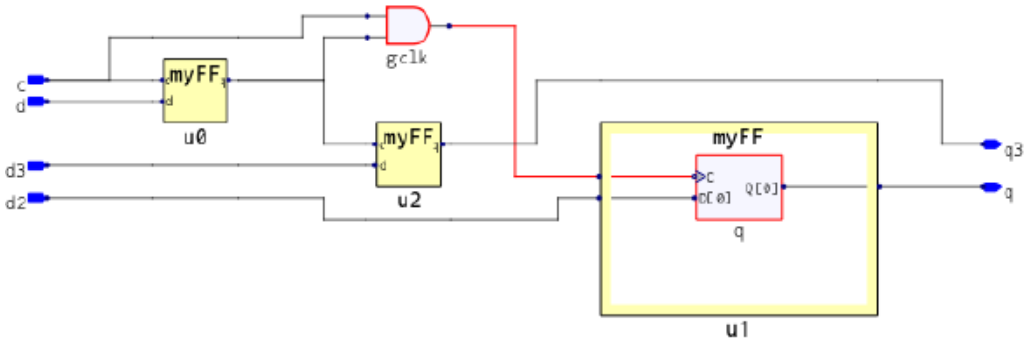
Trace the clock pin back and check the top-level port to make sure it is defined. You can also use the *Clock Summary*report to also locate the undefined clock. Apply the *create_clock*constraint on the top-level port clock in the FDC file.

Instance Clocked by Multiple Variants of Clock

This message is reported when a clock tree contains two phases of a clock. In the example, generated clocks from different phases of clock *c* are gated, and drive the clock pin of a sequential load.



In another scenario, clock *c* and the derived clock from *c* are both gated, and drive the clock pin of a sequential load.

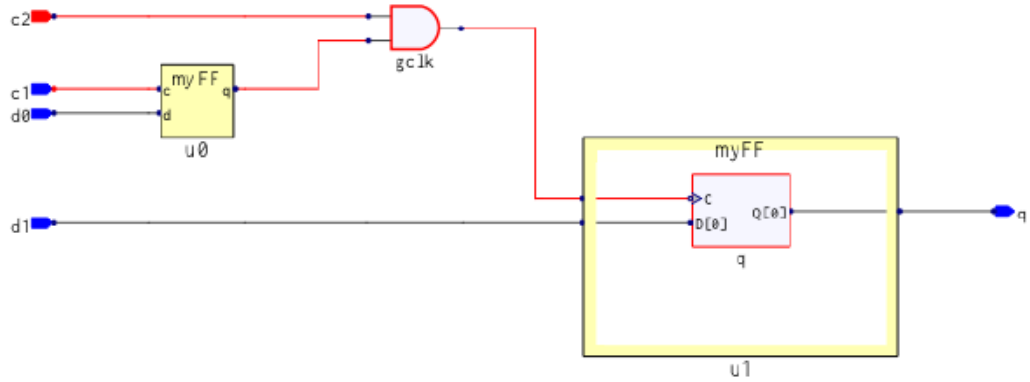


Action

If possible, tie off other test clocks, so that a single source clock feeds the clock tree.

Multiple Clocks on Instance

This message indicates that multiple unique clocks are being fed into a gated clock tree. Such a clock structure is not supported. In the example, clock *c2* and the generated clock from *c1* are both gated, and they drive the clock pin of a sequential load.



There are two cases where a net is considered a generated clock: if a register that generates a clock directly drives a clock pin of a sequential element, or if a *create_generated_clock* constraint is defined for the registered output of a generating clock.

Action

If possible, tie off other test clocks, so that only a single source clock feeds the clock tree, or generate all clocks from a single-source clock.

Editing Netlists

Editing netlists enables you to resolve the architectural differences between ASICs and FPGAs, and make the ASIC netlist suitable for FPGA design.

For more information, see these topics:

- Editing a Verilog Netlist
- Using Gate-Level ASIC Netlists
- Creating Verilog Code with force and bind Statements

Editing a Verilog Netlist

Use the following procedure to edit a Verilog netlist.

1. Load the Verilog database.
2. Load the appropriate *.syn* file for the target vendor from the *lib/xilinx* directory.

   This file is required because the tool cannot work on the Verilog database directly. Use the views from the *xilinx.syn* file to create instances.
3. Create a Tcl file with the netlist editing commands and add it to the synthesis project.
4. Synthesize the design.

Using Gate-Level ASIC Netlists

If you do not have the RTL for the ASIC design, work with a gate-level netlist. It uses more FPGA resources because the resources are mapped at a lower, cell level, and it does not map to high-level resources such as RAM and DSPs.

1. Synthesize the ASIC design and generate a gate-level netlist.
2. Edit the netlist and replace the original ASIC cell library with the FPGA functional equivalents.
3. Use this netlist to run FPGA synthesis.

Creating Verilog Code with *force* and *bind* Statements

Use Verilog *force* and *bind* statements to separate the golden ASIC code from the code for FPGA prototyping. For example, you can use this method to substitute circuitry, change a clock tree, or stub out part of an ASIC design that is not needed in the FPGA prototype.

You can also use *force* and *bind* to and instances to debug in a non-invasive way. You can substitute a module interface definition to isolate it for debugging, or force a constant value to debug using a JTAG port, or create a custom debug port.

force	Replaces existing drivers of internal signals in the hierarchy with new drivers. This statement is only supported for HAPS hardware targets.

Feedback

bind	Inserts an instance into the design hierarchy (for debugging).

1. Create a new file outside the ASIC module.

   Alternatively, you can use the original RTL.

2. Define the alternative logic with the *force* and *bind* statements.

   - Define an initial block with the *force* construct, to override the ASIC-specific golden code in the original file.
   - Add the *bind* construct to insert the parallel instance that replaces the ASIC original.

   See the example for details.

3. In the prototyping tool, specify this command before running pre-map:

   ```
   option set bindandforce 1
   ```

   During pre-mapping, the tool uses the forced values instead of the originals. If you do not specify this option, the tool ignores the *bind* and *force* statements you added.

Example 11 Overriding Original Signal Driver

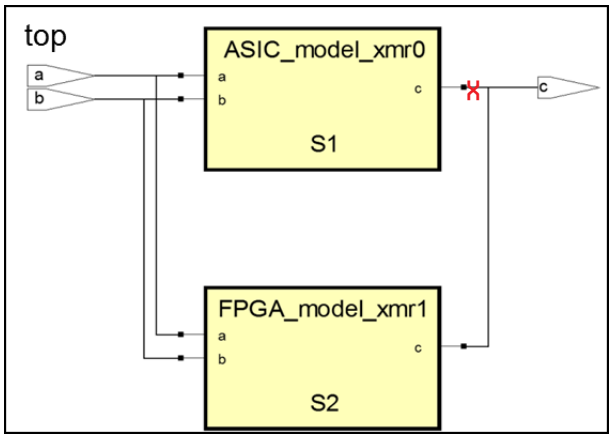
A typical use is to force a logic zero or one to scan flops to remove them from the prototype. In this example, *file2.v* illustrates how a new driver is inserted to override the original:

```
----------file1.v----------

//Golden code which can not be
touched
module top(input a, b, output c);
ASIC_model S1(a, b, c);

----------file2.v----------

module FPGA_model(input a, b,
output c);
 initial begin
 force top.S1.c = top.S2.c;
```



Using Cross-Module Referencing (XMR)

Cross-module referencing (XMR) is a hierarchical reference which provides a way to access signals that would otherwise be internal, without having to bring the signal up

to the top level. It is typically used for verification applications, or to thread nets across modules for time-domain multiplexing (TDM). Cross-module references are also called out-of-module references (OOMR) or XREFs.

XMRs affect distributed processing operations, so you might not see a large runtime advantage in designs with a lot of XMRs.

See the following topics about cross-module references:

- Defining Cross-Module References Using a CDC Constraint
- Defining Cross-Module References in the HDL
- Using Hyper Source Modules for XMR
- Resolving XMR Multi-Driver Errors

Defining Cross-Module References Using a CDC Constraint

The *cdc* constraint method is recommended, but you cannot use it in mixed VHDL and Verilog designs, because the compiler does not support cross-module referencing across languages.

► Specify all the XMRs in a *cdc* file, using *syn_connect_hrefs* constraints, as in this example:

```
syn_connect_hrefs -readers {top.cnt_top} -writer {top.cnt_inst.cnt}
```

See syn_connect_hrefs in the *Compiler Mapper Guide* for details of the syntax.

You can use cross-module references with compile points.

Cross-module referencing has certain limitations:

- No cross-module referencing through an array of instances.
- No cross-module referencing into generate blocks.
- For upward cross-module referencing, the reference must be an absolute path from the top-level module.
- Do not use cross-module reference notation to access functions and tasks.
- You can only use cross-module referencing with Verilog/SystemVerilog elements. You cannot access VHDL elements with hierarchical references.

Defining Cross-Module References in the HDL

The recommended method to define XMRs is to use a constraint (Defining Cross-Module References Using a CDC Constraint) but you can also mark XMRs directly in the HDL,

across languages like Verilog and SystemVerilog. Note that you cannot use this method for mixed Verilog and VHDL designs, because the compiler does not support cross-module referencing across languages.

1. Add the connections manually to the HDL code, using periods (.) as the hierarchical operator for both upward and downward hierarchy in the XMRs, for any SystemVerilog and Verilog data types.
2. To specify a downward XMR read operation, use *assign* statements as shown here:

```
//Downward XMR Read
module top ( input a, input b, output c, output d );
 sub inst1 (.a(a), .b(b), .c(c) );
 assign d = inst1.a;
endmodule
module sub ( input a, input b, output c );
 assign c = a & b;
endmodule
```

3. To specify an upward XMR write operation, use *assign* statements as shown here:

```
//Upward XMR Write
 module top ( input a, input b, output c, output d );
 sub inst1 (.a(a), .b(b), .c(c) );
endmodule
module sub (input a, input b, output c );
 assign c = a & b;
 assign top.d = a;
endmodule
```

Using Hyper Source Modules for XMR

The recommended method to define XMRs is to use a constraint (Defining Cross-Module References Using a CDC Constraint), but you can also use hyperconnect modules. You can use hyperconnect modules to define XMRs for mixed VHDL/Verilog designs, but use them with caution.

Threading Signals with Hyper Source Modules for Prototyping and IP

For prototyping, use Hyper Source to efficiently thread nets across multiple modules to the top-level design to support Time Domain Multiplexing (TDM).

You can also use it to easily replace an ASIC RAM with an FPGA RAM. Follow these guidelines to replace an ASIC RAM with an FPGA RAM:

1. Change the HDL for the RAM instantiation.
2. Add an extra clock signal to all the module interfaces.

   Hyper source reduces the number of modified HDL modules to two, one for the RAM and one for the top level.

Using Hyper Source for IP Designs

For IP designs, Hyper Source is useful for validating and debugging the HDL without directly modifying it. After the HDL has been fully tested with complete QoR results, use Hyper Source to debug, as described in the following cases:

- Add some instrumentation logic that is not part of the original design, such as a cache profiler that counts cache misses or bus monitor that might count statistics about bus contention. The cache or bus might be buried deep inside the HDL; accessing the cache or the bus means ports might need to be added through several levels of hierarchy in the HDL. The instrumentation logic can be included anywhere in the design, so you can use hyper source and hyper connect to easily thread the necessary connections during synthesis.
- Insert other hyper sourcing inside the IP to probe, monitor, and verify correct operation of known signals within the IP.

Threading Signals Through the Design Hierarchy of an IP

Use this procedure to thread a signal through the design hierarchy of a user IP and define an XMR. This signal can be threaded to a top-level port or signal even if the IP source is compiled separately. The tool automatically adds ports and signals between the source and the connection. Otherwise, these connections must be manually added to the HDL code.

However, if you use hyper source to thread signals through an EDIF, you must specify the *Synthesis Strategy advanced mode*.

The following procedure describes a method for using hyper source, using the example HDL shown in Hyper Source Example.

1. Instantiate the XMR source, using syn_hyper_source.
   - Signal *syn_hyper_source (in1)* module defines the source, with a width of 1.
   - The tag name `"`*tag_name*`"`is the global name for the hyper source.
2. Define how to access the hyper source which drives the local signal or port. The following apply to this example:
   - Signal *syn_hyper_connect (out1)* module defines the connection. The signal width of 1 matches the source.
   - Tag name can be the global name or the instance path to the hyper source.

3. In this hierarchical design, note the following about hyper source:
   - Applies to the module *lower_module*.
   - Signal *syn_hyper_source my_source(din)* module is defined for the source with a width of 8.
   - The tag name of "*probe_sig*" must match the name used in the hyper connect block to thread the signal properly.
4. In this hierarchical design, note the following about the hyper connect:
   - Applies to the top-level module `top`, but can be any level of hierarchy.
   - Signal *syn_hyper_connect connect_block (probe)* module is defined for the connection with a width of 8.
   - Tag name of "*probe_sig*"must match the name used in the hyper source block to thread the signal properly.
5. After you run synthesis, the following message appears in the log file:

```
Available hyper_sources - for debug and ip models
HyperSrc label sub2_module.sub1_module.lower_module.probe_sig

Making connections to hyper_source modules
@N: : hyper_example.v(54) | Connected syn_hyper_connect hstdm_training_done_connect, label probe_sig
Finished RTL optimizations (Time elapsed 0h:00m:01s; Memory used current: 122MB peak: 129MB)
```

Hyper Source Example

Examples

```
/* Connect to a signal you want to export example : in1*/
module syn_hyper_source(in1) /*synthesis syn_black_box=1 syn_noprune=1
 */;
parameter w = 1;
parameter tag = "tag_name"; /* global name of hyper_source */
input [w-1:0] in1;
endmodule
/* Use to access hyper_source and drive a local signal or port
 example :out1 */
module syn_hyper_connect(out1) /* synthesis syn_black_box=1 syn_noprune=1
 */;
parameter w = 1; /* width must match source */
parameter tag = "tag_name"; /* global name or instance path to
 hyper_source */
parameter dflt = 0;
parameter mustconnect = 1'b1;
output [w-1:0] out1;
endmodule

/* Example hierarchical design which uses hyper_source */
```

```
module lower_module (clk, dout, din1, din2, we);
output reg [7:0] dout;
input clk, we;
input [7:0] din1, din2;
wire [7:0] din;

syn_hyper_source my_source(din);
defparam my_source.tag = "probe_sig"; /* to thread the signal this
tag_name must match to name used in the hyper connect block */
defparam my_source.w = 8;

always @(posedge clk)
if (we)
 dout <= din;
assign din = din1 & din2;
endmodule

module sub1_module (clk, dout, din1, din2, we);
output[7:0] dout;
input clk, we;
input [7:0] din1, din2;
lower_module lower_module (clk, dout, din1, din2, we);
endmodule

module sub2_module (clk, dout, din1, din2, we);
output [7:0] dout;
input clk, we;
input [7:0] din1, din2;
sub1_module sub1_module (clk, dout, din1, din2, we);
endmodule

module top (clk, dout, din1, din2, we, probe);
output[7:0] dout;
output [7:0] probe;
input clk, we;
input [7:0] din1, din2;

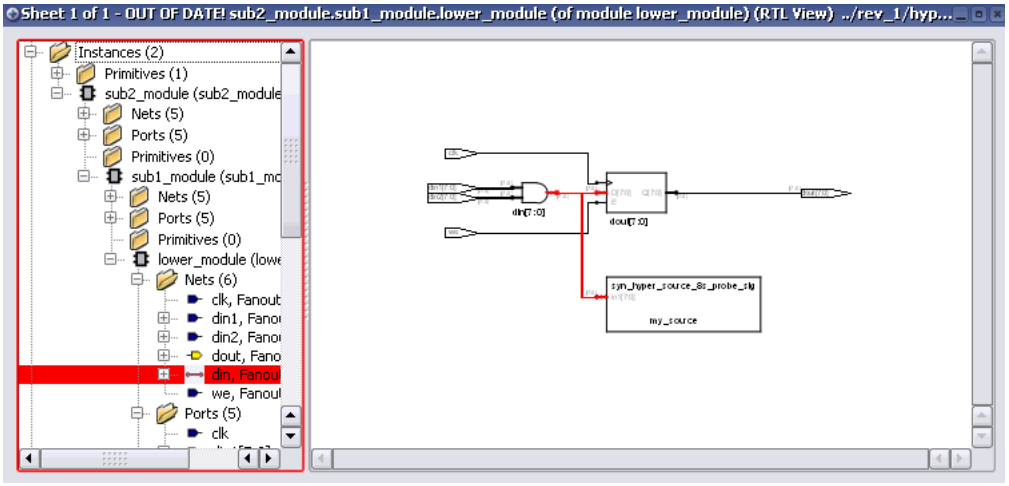
syn_hyper_connect connect_block(probe);
defparam connect_block.tag = "probe_sig"; /* to thread the signal this
 tag_name must match to name used in the hyper connect block */
defparam connect_block.w = 8;

sub2_module sub2_module (clk, dout, din1, din2, we);

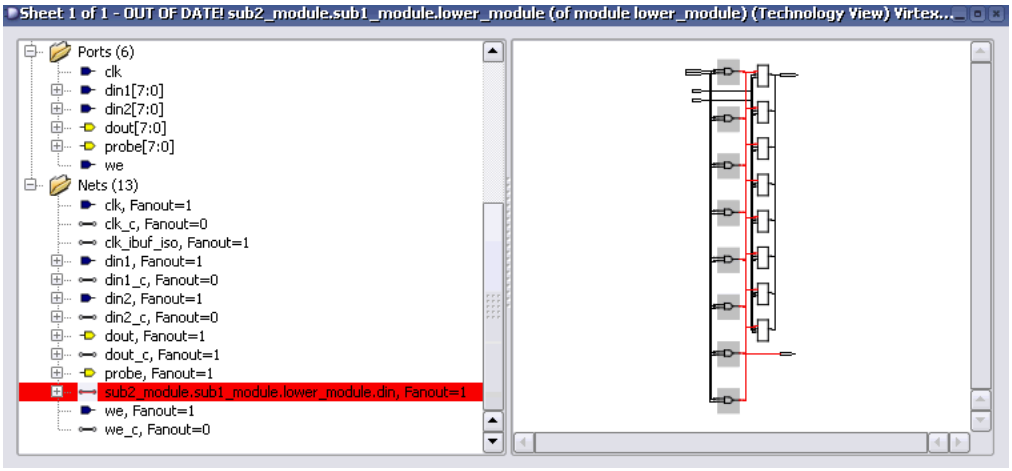
endmodule
```

RTL View

Feedback



Technology View



Resolving XMR Multi-Driver Errors

The updated XMR handling is now compliant with the Language Reference Manual (LRM) and aligns with the simulation behavior. However, during XMR write operation, this compliance can introduce multi-driver errors in the ProtoCompiler flow if the drivers are not optimized. This section explains how to detect these errors, locate the source RTL for the problematic XMR, and apply a Verilog force flow as a workaround.

Enabling XMR Debug Reporting

1. Open your Unified Test File (UTF).

   Add the following option to enable XMR debug reporting:

   ```
   design -report {xmr}
   ```

2. Execute the ProtoCompiler flow.
3. After running the flow, to locate the Debug report, find the `xmr_debug.log` (as shown in the following image) file in the `ucdb/simon.out` directory.

```
MODULE: top
        XMR: top.inst1.d
                USE:            assign top.inst1.d = 1'b1; (./top.v: 9)
                TARGET SCOPE:   sub
                TARGET SIGNAL:  wire                            d; (./top.v: 15)
```

Detecting and Resolving Multi-Driver Errors

1. Review the `globalopt.srr` (as shown in the following image) file for error messages related to multiple drivers. These errors indicate that the XMR has multiple sources that are not optimizable.

```
#   step Error : Mutiple drivers detected on  top.inst1.d
The driver instances are :
top.VDD
top.d
```

2. Cross-reference the error messages in `globalopt.srr` with the XMR entries in `xmr_debug.log` to identify which XMR is causing the error.
3. Use the information in `xmr_debug.log` to trace the XMR back to its RTL source.
4. Modify the RTL source to convert the problematic XMR to use a native force (Verilog force), as appropriate for your design and tool flow.

Converting Designs for Unified Compile

The following topics describe differences between the native compiler and the unified compiler and how to convert designs from the native compiler.

- Converting Older Designs for Unified Compiler
- Handling HDL Differences
- Including DesignWare Foundation Components

Converting Older Designs for Unified Compiler

To reuse a design that was compiled with the native compiler, use the *export* command, as described in the following procedure.

1. If need be, take care of differences in HDL implementation between the two compilers.

   Follow the additional guidelines described in Handling HDL Differences.

2. Load the design database compiled with the native compiler, and export it using the *export uc*command.

   ```
   export uc -path path_to_UC_dir
   ```

   This generates the *UTF*file and the *VCS* script to run UC. The UTF file contains the following command:

   ```
   vcs_exec_command {vcs_cmd.csh}
   ```

   You can also specify the *export uc*command with *-zFmCheck*:

   ```
   export uc -path <value> -zFmCheck
   ```

   The corresponding UTF file contains an additional command to run *zFmCheck*:

   ```
   vcs_exec_command { vcs_cmd.csh}
   synthesis -generate_db_for_fmcheck true
   ```

3. Check the design file library.

   This is a list of exported design files for a mixed language design where all the HDL is compiled in one library. The files include the files needed to run UC:

   ```
   work_cmd_vlog.f
   work_cmd_vhdl.f
   synopsys_sim_setup
   vcs_cmd.csh
   uchaps.utf
   uchaps.tcl
   uc.prj
   ```

   The *work_cmd_vlog.f*and*work_cmd_vhdl.f* files respectively specify that all Verilog files and VHDL files must be compiled in the *work* library. Use the *uc.prj*file as a reference to re-arrange the files in libraries or to create your own UC flow script.

4. Use the generated *UTF* file to launch unified compile from the synthesis tool.

   ```
   launch uc -utf run.utf -ucdb <ucdb_dir>
   ```

5. Compile the *ucdb* generated by the unified compiler.

   ```
   run compile -ucdb <ucdb_dir>
   ```

   Continue with the rest of the flow as usual.

Known Limitations

- This process does not work with designs with configurations.
- The standard compiler implicitly adds the RTL file directory as part of the include path. This is not LRM-compliant and is not supported by unified compile.
- Multi-file compilation unit only works within a single library in VCS. If you need functions/tasks and definitions from different libraries, then the files defining them must be explicitly compiled in each library.

Handling HDL Differences

Generally, the conversion procedure described inConverting Designs for Unified Compile is sufficient for most designs, but some designs might need additional manipulation.

Some of these scenarios are listed alphabetically below:

- Compile Units
- Compilation into Different Libraries
- Configurations in UC
- Cross-Module Referencing
- Designs with VHDL and Macros
- Dynamic Force Statements
- Force and Release Statements
- Library Name Mapping
- RTL Directory Paths
- Virtual Classes

Compile Units

There is a major difference between the native compiler in the prototyping tool and the UC compiler. The native compiler considers the whole design as one unit with Multiple File Compile Units (MFCUs). In the UC flow, each *vlogan* command is a separate compile unit, and you cannot have multiple file compile units across libraries. If a function or a macro is needed in a different library, it must be defined for that library.

To ensure that the compilation units are handled in the same way across both tools, first define a *dut.f* file that contains the RTL, include files, and definitions. Then add a single *vlogan* command like the one shown below to your script:

```
vlogan -full64 -work work -f dut.f
```

Compilation into Different Libraries

During design analysis, the VCS software stores the intermediate files in a design library or a logical library that points to a physical library. This physical library is a physical directory in your UNIX file system.

For details, refer to the VCS documentation.

1. Verify that the logical library is mapped to a physical directory in the *synopsys_sim.setup* file.
2. Specify this mapping in the *synopsys_sim.setup* file as shown in the example below:

```
WORK > DEFAULT
DEFAULT: ./worklib
```

In this example, *WORK*is the default logical library, which is now mapped to the physical library *worklib*in the UNIX file system.With this mapping, the VCS software stores all the intermediate files in the logical library *WORK* by default, and it generates an error if the logical library does not exist in the specified path.

Configurations in UC

Configurations are sets of rules that specify which source code to use in particular cases, as for enabling or disabling clauses; or the libraries from where to pick up modules. In the UC flow, *systemverilog* configurations must be defined with respect to the top-level module.

Configuration File Syntax

```
config config_identifier;
design [library_identifier.]cell_identifier;
config_rule_statement;
endconfig
```

design	Keyword that starts a design statement for specifying the top level of the design
config_rule_statement	One or more of the following clauses:*default* -Specifies logical libraries to search to resolve a default cell instance*instance -Specifies details about a specific instance. These details depend on theliblist.*

Feedback

liblist	Specifies the logical libraries to search to resolve all instances of the cell.
use	Specifies the cell definition is in the specified library

You can specify the configurations in a top-level configuration file or in the RTL files.

Configurations in a Top-Level Configuration File

This is an example of a configuration set up in a single top-level configuration file, in *vcs_cmd.csh:*

```
vcs   -top L1.topcfg  -hw_top=top
vlogan  rtl/low1/lw.v -work Lw1
vlogan  rtl/low2/lw.v -work Lw2
vlogan  rtl/sub2/gt.v -work L2
vlogan  rtl/sub3/gt.v -work L3
vlogan  rtl/top.sv -work L1
vlogan  rtl/top.v -work L1
```

This is the corresponding *synopsys_sim.setup* file. It specifies that module *lw.v* from the *rtl/low1* folder be compiled into the library *Lw1*, module *lw.v* from the *rtl/low2* folder be compiled into the library *Lw2*, and so on.

```
WORK > DEFAULT
DEFAULT : ./work
L1 :  ./lib/lib1
L2 :  ./lib/lib2
L3 :  ./lib/lib3
Lw1 : ./lib/lw1
Lw2 : ./lib/lw2
```

The following example is the configuration definition. The library from which the instance is selected is based on the configuration. With the following configuration, the instance *top* is selected from library *L1*.

```
config topcfg; //Name of the config
design L1.top; //Location of the top-level module
default liblist L3 L2 Lw2; //Default search order for finding
 instantiated modules
instance top.a2 liblist L2; //Explicitly naming library for the following
 module
instance top.a3.i_lw  liblist Lw1;
instance top.a2.g1  liblist Lw2; //Default chosen from Lw2 if not defined
 below
endconfig

library --> L1.top  instance --> top   -- > this is @ top level
library --> L2.gt  rtl instance --> top.a2   -- > this is @ sublevel
library --> LW2.lw  rtl instance --> top.a2.g1   -- > this is @ lower
 level
```

```
library --> L3.gt  rtl instance --> top.a3   -- > this is @ sublevel
library --> LW1.lw  rtl instance --> top.a3.i_lw   -- > this is @ lower
 level
```

In the example above, the *top.a2* instance is selected from library *L2.gt*, the *top.a2.g1* instance is selected from library *LW2.lw*and so on.

Library	Instance	Name	Hierarchical Level
L1.top	Instance	*top*	At top level
L2.gt	RTL instance	*top.a2*	At sub-level
LW2.lw	RTL instance	*top.a2.g1*	At lower level
L3.gt	RTL instance	*top.a3*	At sub-level
LW1.lw	RTL instance	*top.a3.i_lw*	At lower level

Configurations in RTL Files

This is an example of a configuration set up in a single top-level RTL file.

```
config in top.sv  rtl file
config topcfg;
 design L1.top;
 instance top.a2 use L2.gtconfig:config;
 instance top.a3 use L3.gtconfig:config;
endconfig
config in the gt.v  rtl file
config gtconfig;
 design L2.gt;
 instance gt.g1 liblist Lw1;
endconfig
```

The *synopsys_sim.setup* file is set up as follows:

```
WORK > DEFAULT
DEFAULT : ./work
L1 :  ./lib/lib1
L2 :  ./lib/lib2
L3 :  ./lib/lib3
Lw1 : ./lib/lw1
Lw2 : ./lib/lw2
```

The module *lw.v* from the *rtl/low1* folder is compiled into the library *Lw1*, the module *lw.v* from the *rtl/low2* folder is compiled into the library *Lw2*, and so on.

The library from which the instance is selected is based on the configuration. As a result of the configuration, the instance *top* is selected from the library *L1*.

Feedback

```
library --> L1.top  instance --> top   -- > this is @ top level
library --> L2.gt  rtl instance --> top.a2   -- > this is @ sublevel
library --> LW2.lw  rtl instance --> top.a2.g1   -- > this is @ lower
 level
library --> L3.gt  rtl instance --> top.a3   -- > this is @ sublevel
library --> LW1.lw  rtl instance --> top.a3.i_lw   -- > this is @ lower
 level
```

Here, based on the configurations in an individual file, the *top.a2* instance is selected from the library *L2.gt*, the *top.a2.g1* instance is selected from the library *LW2.lw*and so on.

Cross-Module Referencing

Cross-module references (XMRs) are hierarchical references that provide a way to access signals that would otherwise be internal, without having to bring the signal up to the top level.

Use this procedure to create an XMR for any SystemVerilog and Verilog data type:

1. Specify the XMRs in the RTL, not the *.cdc* file.

   The UC flow does not support XMRs defined through directives in the *.cdc* file. Add the connections manually to the HDL code, using a period (.) as the hierarchical separator.

2. To specify a downward XMR read operation, use the *assign* statement as shown:

   ```
   module top ( input a, input b, output c, output d );
    sub inst1 (.a(a), .b(b), .c(c) );
    assign d = inst1.a;
   endmodule
   module sub ( input a, input b, output c );
    assign c = a & b;
   endmodule
   ```

3. To specify an upward XMR write operation, use the *assign* statement as shown:

   ```
   module top ( input a, input b, output c, output d );
    sub inst1 (.a(a), .b(b), .c(c) );
   endmodule
   module sub (input a, input b, output c );
    assign c = a & b;
    assign top.d = a;
   endmodule
   ```

Limitations

- XMRs defined through CDC constraints are not supported.
- In standard compile, XMRs cannot be used in mixed Verilog and VHDL designs, because the compiler does not support cross-module referencing across languages. However, the UC flow supports mixed language XMRs.

Feedback

```
module top ( input a, input b, output c, output d , output e );
  sub inst1 (.a(a), .b(b), .c(c) );
  assign d = inst1.c1; // XMR Read
 endmodule
entity sub is
  port ( a, b : in std_logic ;
     c : out std_logic);
 end sub;
architecture sub_arch of sub is
  signal c1, x2 : std_logic;
  begin
   c1 <= a or b;
   c <= c1;
 end  sub_arch;
```

Designs with VHDL and Macros

If the design contains VHDL, check the following:

- It is recommended that FDC constraints on VHDL paths be converted to upper case. You can also use the *-nocase*switch as shown below, but this method incurs a compile time penalty and all Verilog names will also be matched without considering case.

  ```
  create_clock -name {clk_in1} -period 10 [get_pins
   {top.vhdl_subtopt.clk_in1}]
  -nocase
  ```

- The VCS software does not support VHDL at the top-level. If the design has a top-level VHDL module, create a Verilog wrapper around the top-level VHDL.

Explicitly add required Verilog and VHDL macros in the VCS script.

Dynamic Force Statements

Dynamic force is like the `force` statement (see Force and Release Statements), except that the force value is set dynamically.

1. In the RTL, use *$dumpvars* syntax to specify the signals to be forced.

   This is the syntax:

   ```
   initial
   begin : IICE_DF
   (* haps_force *) $dumpvars (1, <hier_path/signal1_scalar>);
   (* haps_force *) $dumpvars (1, <hier_path/signal2_vector>);
   (* haps_monitor *) $dumpvars (1, <hier_path/signal3>);
   end
   ```

   Example:

   ```
   initial
   begin : IICE_DF
   ```

Feedback

```
(* haps_force *) $dumpvars (1, cnt_top.u_cnt_inst4.capture_cnt[15:0]);
 begin...
```

2. In the prototyping software, enable options for *$dumpvars* and force.
   - Enable the *debug_dumpvars* and *incremental_debug* options.

     ```
     option set debug_dumpvars 1
     option set incremental_debug 1
     ```

   - Optionally, enable global driver tracing and the insertion of a force mux on the real driver of the specified net by enabling the *dynamic_force_global_driver* option.

     ```
     option set dynamic_force_global_driver 1
     ```

3. If the specified signals for force are not to be instrumented, define IICE with *-mode none*.

   For example:

   ```
   iice new {IICE_DF} -type regular -mode {none}
   ```

4. Run through the rest of the synthesis and place-and-route stages.
5. At runtime, force the signal or perform readback:
   - To force a dynamic value for a signal, specify the `force` command, where the format for the value can be decimal, hexadecimal, or binary (*'d* | *'h* | *'b*). The default format is decimal.

     ```
     force {<hier_path/signal1_scalar>} ['d | 'h | 'b]<force_value>
     ```

     The command can also be included in a script:

     ```
     package require DF
     # Open hardware for forcing and provide export folder for runtime
     DF::df_open <export_result_folder> <HW_handle>

     force {<hier_path/signal1_scalar>} <force_value>
     force {<hier_path/signal2_vector>} [<format>]<force_value>
     # value {<hier_path/signal1_scalar>}
     # value {<hier_path/signal2_vector>}
     # value {<hier_path/signal3>}
     ```

   - For readback, optionally reset the default value format; then specify the value for force.

     The default format for the read value is hexbinary (*h*), but you can set it to binary (*b*) using this syntax in the confrposh shell:

     ```
     DF::df_open <export_result_folder> <HW_handle>[h | b]
     ```

It can also be included in a script, following this syntax:

```
package require DF
# To open hardware for forcing and provide export folder for
runtime
DF::df_open <export_result_folder> <HW_handle> [h | b]

# force {<hier_path/signal1_scalar>} <force_value>
# force {<hier_path/signal2_vector>} [<format>]<force_value>
# value {<hier_path/signal1_scalar>}
# value {<hier_path/signal2_vector>}
# value {<hier_path/signal3>}
```

Example of runtime script:

```
set hw_handle [list E025669 E025670]
DF::df_open ./export_results $hw_handle

force {cnt_top.u_cnt_inst4.capture_cnt[15:0]} 16'h00AA
value {cnt_top.u_cnt_inst4.capture_cnt[15:0]}
```

Force and Release Statements

The SystemVerilog `force`and`release` statements are procedural continuous assignments that allow expressions to be driven continuously onto variables or nets. There are differences in support for the `force`and`release` statements.

Force Statement

Both UC and standard compile flows support `force` statements. For example:

```
module top (
 input  a, b,
 output c, c1
);
asic_m s1 (a, b, c);
 asic_m s2 (a, b, c1);
 initial begin
 force s1.a = 1'b1 ;
end
endmodule
```

In standard compile, the *force* statement *s1.a* applies to the local net only, so it is only applies to instance *s1*. In UC, `force` applies to the global driver net, so the statement is applied to the *a* in the top module, so both instances *s1* and *s2*are affected by this `force` statement. Further, in standard compile, the `force` statement can only be specified in an *initial* block.

Release Statement

In standard compile, the `release` statement is ignored; in UC, the `release` statement is applied.

For example:

```
module top (
 input  a, b,
 output c, c1
 );
asic_m s1 (a, b, c);
asic_m s2 (a, b, c1);
initial begin
 force s1.a = 1'b1 ;
 release s1.a ;
end
endmodule
```

Library Name Mapping

The VCS software allows you to create multiple logical libraries, each one pointing to a different physical library. The syntax to map a logical library to a physical library is shown below. Logical library names are case-insensitive.

```
logical_name: physical_name
```

The following example shows two logical libraries, *ALU8* and *ALU16*, mapped to the *alu_8bit* and *alu_16bit*physical libraries, respectively. During design analysis, use the *-work* option to assign the files into the respective libraries.

```
vlogan  alu8.v      -work ALU8
vlogan  alu16.v    -work  ALU16
```

In the *run* directory, specify the logical and physical memory mapping using the *synopsys_sim.setup* file.

```
WORK > DEFAULT
DEFAULT: ./work
```

Library Mapping

```
ALU8: ./alu_8bit
ALU16: ./alu_16bit
```

Feedback

RTL Directory Paths

Check that the *VCS* script explicitly includes the RTL directory path. If it does not, it could result in an error:

```
Error-[SFCOR] Source file cannot be opened
Source file "test.h" cannot be opened for reading due to 'No such file or
 directory'.
Please fix above issue and compile again.
"rtl/top.sv", 7
Source info:`include <test.h>
```

For the ProtoCompiler tool, you do not need to explicitly specify the RTL include directory because by default the tool searches the current HDL directory and then the *include* directories, sequentially.

However, this is not the case with the VCS software, which by default searches only the *include* directories. So, you must explicitly specify the RTL include directory in the *+incdir +* path for the UC flow. *+incdir+* specifies directories to search for include files used in the *`include* compiler directive. In the following example, *rtl* is the path of the RTL include directory, that contains the include files.

```
vlogan -sverilog +incdir+rtl rtl/top.sv
```

Virtual Classes

You can use virtual classes to parameterize SystemVerilog static functions. UC supports a subset of virtual or abstract class features. The following constructs are supported:

- Parameterized abstract classes explicitly declared with the keyword *virtual*.

  ```
  virtual class MyClass #(parameter SIZE=4, parameter SIZE1=1);
  typedef struct {
   logic [SIZE-1:0] elem1;
   logic [SIZE1-1:0] elem2;
   } my_type;
  ```

- Virtual classes declared in packages or by using the *$unit*.
  - Imported from a package.

    ```
    package myPkg;
     virtual class ClassB;
     static function logic [1:0] F2 (logic [1:0] a);
     …
     endfunction
     endclass
     endpackage
    import myPkg::*;
     module test (a,b,y,y1);
     …
    ```

Feedback

```
assign y = ClassB::F2(b);
endmodule
```

  - Referenced using the package scope, the class scope *::* operator

```
package pkg1;
virtual class MyClass;
 static function int F1 ();
  …
endfunction
endclass
endpackage
module test (x,clk,y);
...
y = x + pkg1::MyClass::F1();
endmodule
```

- All legal specialization syntax. When all parameters have defaults, `VC` and `VC#()`.

```
assign y3 =  pkg1::MyClassPkg#(.SIZE(S))::F2(x2);
y1.elem1 = MyClass#()::F1(x.elem1);
```

- Access to the following elements of a specialization using the class scope *::* operator and access using a user-defined type name.
  - Static functions with a default automatic lifetime

```
virtual class ClassA # (parameter SIZE = 32);
static function automatic integer factorial (input [SIZE-1:0]
  operand);
…
endfunction: factorial
endclass
```

  - User-defined type names (e.g a *struct* type)

```
virtual class MyClass #(parameter SIZE=4, parameter type
 mtype=logic );
 typedef struct {
 logic [SIZE-1:0] elem1;
 mtype            elem2;
 } my_type;
endclass
```

  - Parameters and local parameters
- Non-parameterized abstract classes explicitly declared with the keyword *virtual*.

```
virtual class MyClass;
 static function int F1();
 …
 endfunction
endclass
```

- Inherited using the `extends` keyword.

```
virtual class MyClass #(parameter SIZE=4 );
 static function logic [SIZE-1:0] F1 (logic [SIZE-1:0] x);
 endfunction
endclass
class MyClass_ext extends MyClass;
 static function logic [SIZE-1:0] F1 (logic [SIZE-1:0] x);
 return ~x;
 endfunction
endclass
```

- Nested declarations (using a class declaration as a class item)

```
virtual class ClassB # (parameter SIZE = 2);
 typedef struct {
  logic [SIZE-1:0] elem1;
  logic elem2;
 } my_type;
virtual class ClassA;
 static function logic [1:0] F2 (logic [1:0]x);
  return |x;
 endfunction
 endclass
endclass
```

Limitations

The following virtual class constructs are not supported for the UC flow.

- Abstract classes without an explicit *virtual*keyword
- Static data properties
- Static tasks
- Variables of an abstract class type
- Static functions with static lifetime or containing variables with a static lifetime
- Forward class declarations
- Prototypes
- The *implements* keyword

Including DesignWare Foundation Components

You can instantiate DesignWare components from supported foundation libraries, like *gtech, dware, dw01, dw02, dw03, dw04*, and *dw06* and use the unified compiler flow. You can use the *VHDL definitions for these*compon*ents in this flow.*

1. Set the *SYNOPSYS* environment variable to point to a Design Compiler installation.
2. Enable DesignWare in the design by adding a *-syn_dw* argument.
   - In the VCS script, specify the *-syn_dw* option with all *vlogan* and *vcs* commands that use DesignWare components. For example:

     #!/bin/csh -fx vlogan -full64 -syn_dw my_ip.v vlogan -full64 -sverilog top.v vcs -full64 -syn_dw top -hw_top=top
   - *If you are using a separate VCS script to compile the DesignWare library, use the method described in the next step.*
   - If you are using a custom version of a DesignWare library, use *-syn_dw +allow_override* to allow the DesignWare cells to be overridden with user-defined cells during elaboration.

     Include a *dump_dir* option to write the elaborated cells to a specified directory:

     -syn_dw+allow_override+dump_dir=directoryPath
3. *If you are using a custom compile script to compile the DesignWare cells, do the following:*
   - *Either embed the file in your overall compilation script or use it separately.*
   - Add DesignWare library mappings to the *synopsys_sim.setup* file.

   See Custom DesignWare Compile Script Example.
4. To specify physical locations, add the *+dump_dir* argument to the analyze and elaboration commands.

   For example, the following commands change the default local paths for all DWBB libraries:

   % vlogan -syn_dw+dump_dir= directoryPath % vhdlan -syn_dw+dump_dir= directoryPath % vcs -syn_dw+dump_dir= directoryPath elabOptions topModule|entity

   As a result of these commands, the default path for dw01 for example, changes from *dw01* to *directoryPath/dw01*. Do not delete *directoryPath* after analysis, because it is needed for elaboration.
5. Do the following in the *synopsys_sim.setup* file.

   This file is a setup file for VCS, and configures the libraries, mapping them to directory locations.

- Add a *LIBRARY_SCAN=TRUE* statement to the *synopsys_sim.setup* file. This file configures the library locations. This is the recommended setting for DesignWare components.
- *Specify library-to-directory mapping. See Example synopsys_sim.setup File for DesignWare, and refer to the VCS documentation for complete details about this file.*

Custom DesignWare Compile Script Example

VCS Script to Compile DesignWare

The script shows VHDL DesignWare entities. VHDL is only supported when the leaf module is a DesignWare module.

```
#!/bin/csh -fx
mkdir -p gtech dware dw01 dw02 dw03 dw04 dw06
vhdlan -full64 -work gtech
 ${SYNOPSYS}/packages/gtech/src/GTECH_components.vhd
vhdlan -full64 -work gtech
 ${SYNOPSYS}/dw/fpga_ip/fv/dw_foundation/gtech.vhd.e
vhdlan -full64 -work dware
 ${SYNOPSYS}/dw/fpga_ip/fv/dw_foundation/dware_comp.vhd.e
 ${SYNOPSYS}/dw/fpga_ip/fv/dw_foundation/dware.vhd.e
vhdlan -full64 -work dw01
 ${SYNOPSYS}/dw/fpga_ip/fv/dw_foundation/dw01_comp.vhd.e
 ${SYNOPSYS}/dw/fpga_ip/fv/dw_foundation/dw01.vhd.e
vhdlan -full64 -work dw02
 ${SYNOPSYS}/dw/fpga_ip/fv/dw_foundation/dw02_comp.vhd.e
 ${SYNOPSYS}/dw/fpga_ip/fv/dw_foundation/dw02.vhd.e
vhdlan -full64 -work dw03
 ${SYNOPSYS}/dw/fpga_ip/fv/dw_foundation/dw03_comp.vhd.e
 ${SYNOPSYS}/dw/fpga_ip/fv/dw_foundation/dw03.vhd.e
vhdlan -full64 -work dw04
 ${SYNOPSYS}/dw/fpga_ip/fv/dw_foundation/dw04_comp.vhd.e
 ${SYNOPSYS}/dw/fpga_ip/fv/dw_foundation/dw04.vhd.e
vhdlan -full64 -work dw06
 ${SYNOPSYS}/dw/fpga_ip/fv/dw_foundation/dw06_comp.vhd.e
 ${SYNOPSYS}/dw/fpga_ip/fv/dw_foundation/dw06.vhd.e
vlogan -full64 -work dw02
 ${SYNOPSYS}/dw/fpga_ip/fv/dw_foundation/dw_verilog.v.e
```

Example synopsys_sim.setup File for DesignWare

```
LIBRARY_SCAN=TRUE
dw01 : ./dw01
dw02 : ./dw02
dw03 : ./dw03
dw04 : ./dw04
dw05 : ./dw05
```

```
dw06 : ./dw06
dware : ./dware
gtech : ./gtech
TIMEBASE = ns
```

Integrating Memory Models

The HAPS-200 ProtoCompiler tool supports implementation of memory models such as DRAMs, SDRAMs and so on.

For more details, see:

- License Requirement
- Compatible Memory Models
- Memory Model HAPS Flow

License Requirement

You require the following *FLEXlm* license keys, depending on the memory models implemented with the HAPS flow. Contact the marketing team or memory model team to obtain these license keys:

- hw_xtormm_ddr5
- hw_xtormm_hbm4
- hw_xtormm_hbm3
- hw_xtormm_lpddr6
- hw_xtormm_lpddr5
- hw_xtormm_lpddr4
- hw_xtormm_gddr7
- hw_xtormm_gddr6
- hw_xtormm_axi4
- hw_xtormm_dfi
- hw_xtormm_ddr4

Feedback

Compatible Memory Models

The following table lists the memory models compatible with the HAPS hardware:

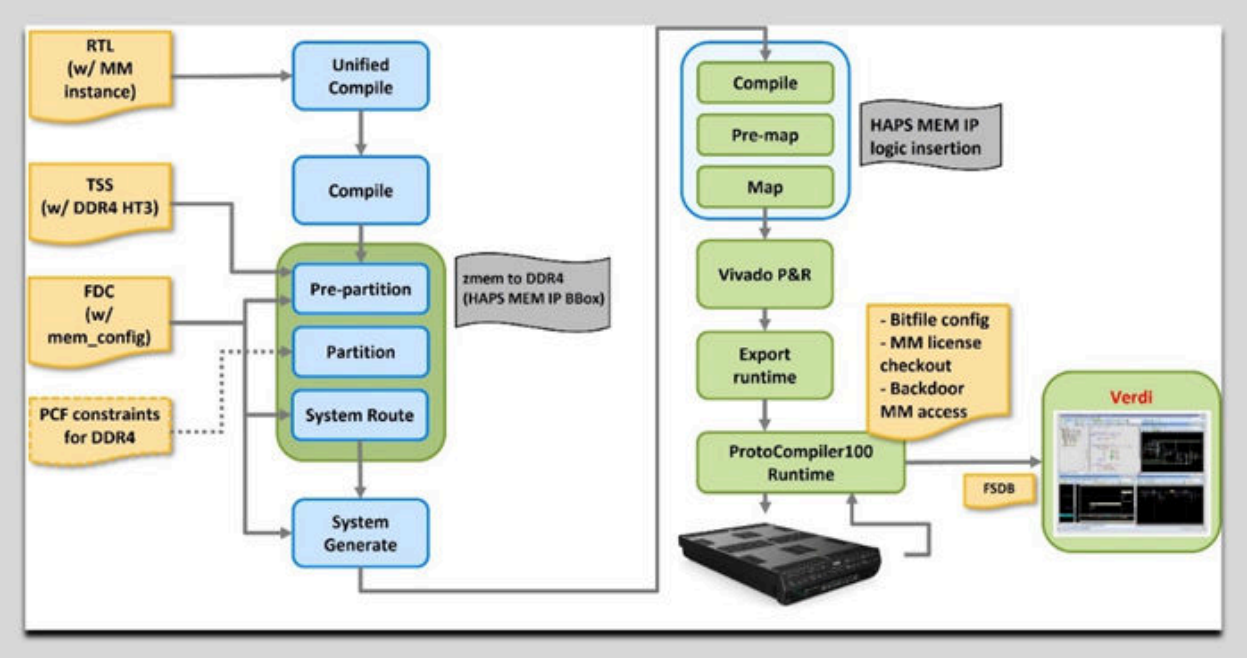
Table 10 Memory Models compatible with HAPS-200 hardware

Memory Model	DFI	HAPS-200 6F			HAPS-200 12F
		DDR4_8GB_HT3	DDR4_16GB_HT3	DDR4_32GB_HT3	
DDR4	Yes	Yes	Yes	Yes	Yes
DDR5	Yes	Yes	Yes	Yes	Yes
HBM4	Yes	Yes	Yes	Yes	Yes
HBM3	Yes	Yes	Yes	Yes	Yes
GDDR7	Yes	Yes	Yes	Yes	Yes
GDDR6	Yes	Yes	Yes	Yes	Yes
LPDDR6	Yes	Yes	Yes	Yes	Yes
LPDDR5	Yes	Yes	Yes	Yes	Yes
LPDDR4	Yes	Yes	Yes	Yes	Yes

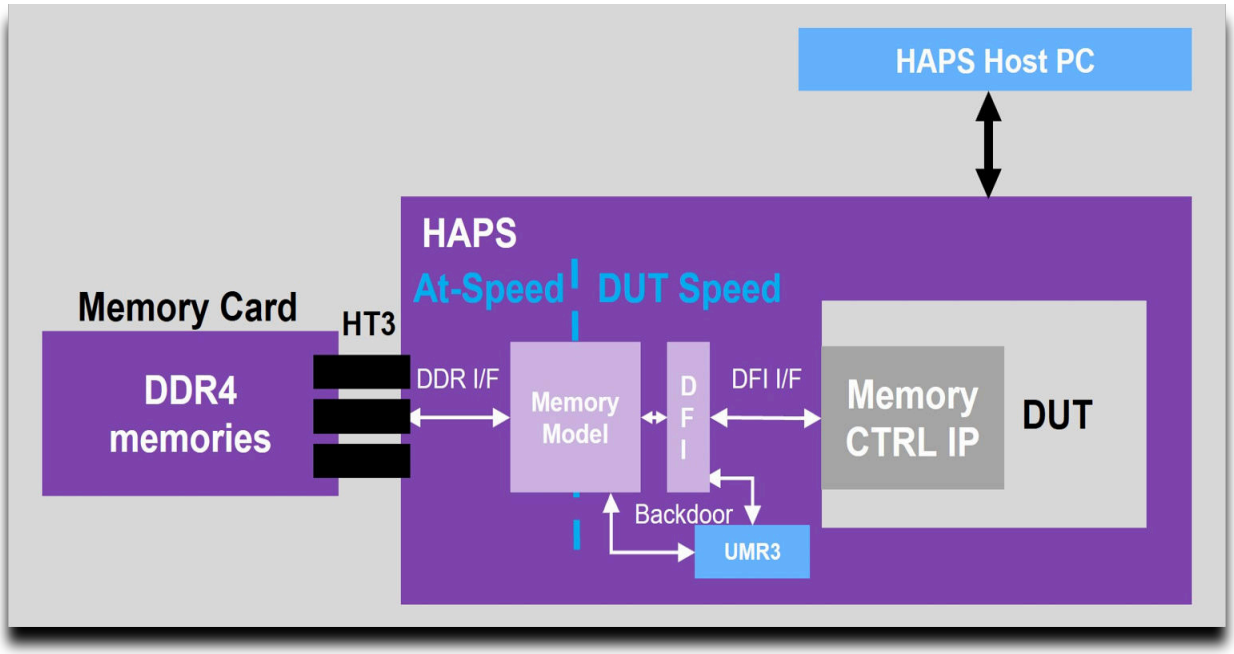
Feedback

Memory Model HAPS Flow

Figure 24 Memory Model HAPS Flow



The following image shows the instantiation of a memory model in the HAPS prototyping system. Data is stored in DDR4 memory cards. The IP provides the DFI interface, to interconnect between the memory model and the controller.



RTL Changes

- Instantiate memory model in RTL from the ZEBU_IP_ROOT
- Debug flags

DDR4_CALIB/INIT_COMPLETE flags:

Instantiate hyper_connect instances to access `DDR4_INIT_COMPLETE` and `DDR4_CALIB_COMPLETE` flags separately for different DDR4 cards:

For example:

```
wire DDR4_CALIB_COMPLETE;  // Flag is enabled when DDR4 calibration
 completes
wire DDR4_INIT_COMPLETE;  // Flag is enabled when DDR4 static
 initialization completes
syn_hyper_connect #(.tag("SNPS_HAPS_MEM_CALIB_COMPLETE_<DDR4 HC TAG
name>"), .w(1), .mustconnect(0), .dflt(1))
snps_haps_mem_calib_complete(DDR4_CALIB_COMPLETE) ;
syn_hyper_connect #(.tag("SNPS_HAPS_MEM_INIT_COMPLETE_<DDR4 HC TAG
name>"), .w(1), .mustconnect(0), .dflt(1))
snps_haps_mem_init_complete(DDR4_INIT_COMPLETE);
```

TSS (Applicable only for HAPS200-6F. HAPS200-12F has embedded 32GB DDR4 cards)

```
board_system_create -interconnect -manual DDR4_U16GB_HT3 -name ddr4_ht3_1
 - connector {mb1.F1_X4 mb1.F1_X5 mb1.F1_X6}
board_system_configure -voltage {mb1.F1_X4 mb1.F1_X5 mb1.F1_X6} 1.2
```

Feedback

```
board_system_create -interconnect -manual DDR4_U16GB_HT3 -name ddr4_ht3_2
 - connector {mb1.F1_X7 mb1.F1_X8 mb1.F1_X9}
board_system_configure -voltage {mb1.F1_X7 mb1.F1_X8 mb1.F1_X9} 1.2
```

FDC

HAPS-200-4F

```
mem_config {i:<hierarchical path to the memory model instance>} -location
{<ddr4_ht3_1>} -ddr_hctag {<DDR_HC_TAG_suffix1>}
mem_config {i:<hierarchical path to the memory model instance>} -location
{<ddr4_ht3_2>} -ddr_hctag {<DDR_HC_TAG_suffix2>}
```

HAPS-200-12F

```
mem_config {i:<hierarchical path to the memory model instance>} -devices
{<FPGA name>} -ddr_hctag {<DDR_HC_TAG_suffix1>}
```

The following code shows the RTL instantiation:

```
syn_hyper_connect
#(.tag("SNPS_HAPS_MEM_CALIB_COMPLETE_DDR_HC_TAG_suffix1"), .w(1),
.mustconnect(0), .dflt(1))
snps_haps_mem_calib_complete_1(DDR4_CALIB_COMPLETE_1);
syn_hyper_connect #(.tag("SNPS_HAPS_MEM_INIT_COMPLETE_
DDR_HC_TAG_suffix1"), .w(1), .mustconnect(0), .dflt(1))
snps_haps_mem_init_complete_1(DDR4_INIT_COMPLETE_1);
syn_hyper_connect #(.tag("SNPS_HAPS_MEM_CALIB_COMPLETE_
DDR_HC_TAG_suffix2"), .w(1), .mustconnect(0), .dflt(1))
snps_haps_mem_calib_complete_2(DDR4_CALIB_COMPLETE_2);
syn_hyper_connect #(.tag("SNPS_HAPS_MEM_INIT_COMPLETE_
DDR_HC_TAG_suffix2"), .w(1), .mustconnect(0), .dflt(1))
snps_haps_mem_init_complete_2(DDR4_INIT_COMPLETE_2);
```

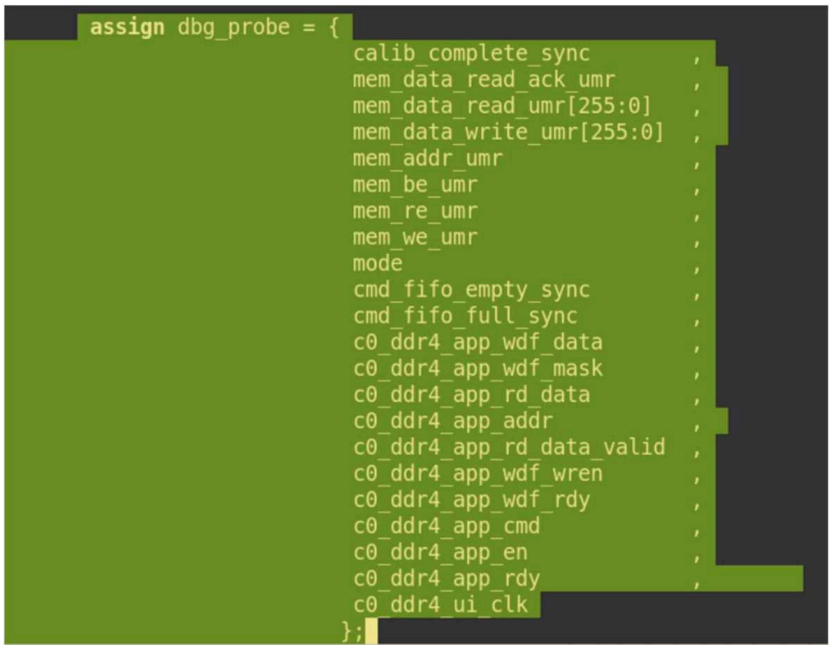
Firmware (MemorymodeltoDDR4) IP Debug Probes

Instantiate hyper_connect instance to access firmware IP debug probes separately for different DDR4 cards:

```
wire [2049:0] DDR4_DBG_PROBE;
syn_hyper_connect #(.tag("SNPS_HAPS_MEM_DBG_PROBE_<DDR4 HC TAG name>"),
.w(2050)) snps_haps_mem_ddr4_dbg_probe(DDR4_DBG_PROBE) ;
```

Feedback

The following list of signals are available as part of this debug probe:



HAPS ProtoCompiler Compile

Generate the bitfile with the following PC compile options:

- Environment variables:

  ```
  setenv ZEBU_IP_ROOT <Memory model installation directory>
  ```

- HAPS ProtoCompiler options

  ```
  option set enable_vs_memory 1 #Enables memory model flow
  ```

- VCS command line:

  ```
  +define+ZEBU_SYNTH \
  +define+ZRM_NEW_LATENCY \
  +define+XTOR_SVS_PROTOCOMPILER_FLOW \
  -y ${ZEBU_IP_ROOT}/HW_IP/vlog/vcs \
  -y ${ZEBU_IP_ROOT}/HW_IP/wrapper_rtl \
  ${ZEBU_IP_ROOT}/HW_IP/wrapper_rtl/<memory model config>_bidir.v \
  ${ZEBU_IP_ROOT}/HW_IP/vlog/vcs/<memory model config>.vp \
  ${ZEBU_IP_ROOT}/HW_IP/vlog/vcs/<DFI config>.vp \ #Required if DFI is
   used
  <Memory model + DFI wrapper>.v #Create a wrapper if DFI is used
  ```

`CK_2x_t` (Applicable for HBM3 only)

Feedback

In addition to the HBM3 JEDEC interface, the ZHBM3 memory model requires a specific clock input named `CK_2x_t`. This clock must be always on, and running and must have:

- Same frequency as `WDQS_t`.
- Same polarity as `WDQS_t`.



The `CK_2x_t` is used to:

- latch incoming DQ data during a write sequence (in replacement of WDQS_t/c strobe).
- Generate the RDQS_t/c strobe during a read sequence.

When using the ZHBM3 along with the ZDFI_HBM3, `CK_2x_t` has to be connected to the `CK_2x_t` of the ZDFI.

Optionally, use the following definition to drive the `CK_2x_t` port internally:

```
+define+XTOR_SVS_HAPS_HBM3_CLK2X
```

FDC constraint:

```
mem_config {i:<hierarchical path to memory model instance>}
-location {<DDR4 daughter card>}
-devices {<FPGA name>}
[-init {0|1}]
[-ddr_hctag {<DDR_HC_TAG_suffix>}]
```

`-location`	DDR4 card name as specified in the TSS file (A4)
`-devices`	FPGA name that has the embedded DDR4 card (A12)
`[-init {<DQbits>'b|h<value>}`	Enable static initialization (Data + ECC)
`{<DQbits>'b0…0}]`	Enable static initialization to all 0s
`{<DQbits>'b1…1}]`	Enable static initialization to all 1s
`{<DQbits>'b<pa..ern>}]`	Enable static initialization to specified pattern
`[-ddr_hctag {<>}]`	Hyper-connect TAG suffix for accessing DDR4_CALIB/INIT_COMPLETE flags

Note:

Do not use these options together, use `-location` HAPS-200 6F and `-devices` with HAPS-200 12F.

Feedback

Example:

```
mem_config {i:zlpddr5_ins} -location {ddr4_ht3_1} -init {40'b0}
 -ddr_hctag
{ddr_hctag_suffix}
mem_config {i:zlpddr5_ins} -devices {mb1.F1} -init {40'b0} -ddr_hctag
{ddr_hctag_suffix}
```

HAPS hRun

Invoke hRun and execute the following script after downloading the bit file, for memory model license checkout and memory model write/read/backdoor access:

```
set s_handle [setup configure \ # HW programming and MM license
-designdir <path to runtime dir> \ # checkout
-hmf <hmf file> -workdir <hrun work dir>]

setup control -handle $s_handle -fpga {fpga name} \
-action usr_local_reset -value <0|1> # Apply design reset
<Backdoor memory and register access>
<Front-door memory model write/read>
<other commands>
setup clear -handle $s_handle # Close/clear the HW and release MM license
```

Backdoor access of memory model and configurable registers

- Listing memory model instances:

  ```
  memory list # Returns list of memory model instances
  ```

- Listing config registers:

  ```
  memory list -registers # Returns list of config registers

  <memory model instance> # within specified memory model instance
  ```

- Register write:

  ```
  memory write_reg \ # Backdoor config register write

  <register name> <value> # Path to config register and value
  ```

- Register read:

  ```
  memory read_reg \ # Backdoor config register read

  <register name> # Path to config register
  ```

- Listing memory logic instances:

  ```
  memory list -memories \ # Returns list of mem logic instances

  <memory model instance> # within specified memory model
  ```

- Memory write:

```
memory write \ # Backdoor memory model write

<filename> \ # Memory model initialization file

<mem logic instance> # Memory logic instance path within memory model
```

Backdoor memory write file format:

```
Address DQ#
0       0
1       ffff
2       ffffffff
:       :
```

Memory read:

```
memory read \# Backdoor memory model read
[-address <value> ] \ # Start address
[-size <value> ] \ # Number of addresses to read
<file name> \ # Memory model read file
<mem logic instance> # Mem logic instance path within memory model
```

References to VS Memory Model ZeBu Documentation

Go to `$ZEBU_IP_ROOT/doc` to access these ZeBu documentation.

- ZeBu Memory Model manual
  - *ZeBu_IP_GDDR6_UM.pdf*
  - *ZeBu_IP_HBM3_UM.pdf*
  - *ZeBu_IP_DDR5_UM.pdf*
  - *ZeBu_IP_LPDDR5_UM.pdf*
  - *ZeBu_IP_LPDDR4_UM.pdf*
- ZeBu DFI interface user manual for various memory models
  - *ZeBu_DFI_DDR5_Interface.pdf*
  - *ZeBu_DFI_LPDDR5_Interface.pdf*
  - *ZeBu_DFI_LPDDR4_Interface.pdf*

9

Verifying the Design

This chapter describes how to formally verify your design using the Formality equivalence checker. You run formal verification after you complete synthesis. See the following for the details:

- Guidelines for Successful Formal Verification
- Verifying Single-FPGA Designs
- Verifying Multi-FPGA Designs
- Configuring Options for Formal Verification
- Running VCS Simulation for Inferred Memories
- Running Formal Verification with Checkpoints

For other methods of verifying your design at different points in the design cycle, see these topics:

- Running Simulation
-
- Instrumenting and Debugging Designs

Guidelines for Successful Formal Verification

Adhere to the following guidelines while checking your design with the Formality tool:

- Generally, non-equivalent points are the result of missing or incorrect mapping rules. Evaluate any non-equivalent points carefully to determine the cause. In most cases, fixing annotations resolves the problem. Search for errors and warnings in the formal verification tool log file and resolve them.
- Although the final goal is a top-down verification run with no non-equivalent points, it is recommended that you begin verification at the submodule level and work up to the top level. Small issues at the submodule level can translate into a large number of unmapped or non-equivalent points at the top level. There is a smaller amount of design logic at the submodule level, so it is much easier to deal with and resolve annotation issues at this level.

- When extracting a submodule from an entire design for verification, preserve the submodule interface as it is in the RTL. During synthesis, the tools optimizes across hierarchies and can alter module level interfaces. For module-level verification, you can either apply synthesis attributes to the module to preserve the interface, or synthesize the module by itself.

Additionally, see these topics:

- General Coding Guidelines for Formal Verification
- Coding Guidelines for Verilog and SystemVerilog Designs
- Limitations of Equivalence Checking

General Coding Guidelines for Formal Verification

The following coding practices are recommended to avoid potential mismatches in formal verification.

Formality Guidelines

- *always_comb* statements
  - Put these statements on lines by themselves; do not share the line with other statement.
  - Avoid using labels for *always_comb* statements, because it leads to a naming mismatch for the datapath operators that contain the statement:

    ```
    always_comb
    begin : label //naming mismatch for datapath operators
    result = a * b;
    end
    ```

  - Avoid using *always_comb* statements inside loops.
- Variable sizes must match when assigned or mapped to one another to avoid any potential formal verification mismatch. Look for warnings that report a port map width mismatch or an index out of range, like CD285 and CD543, for example.
- Do not re-declare parameters and other variables within the same scope.
- Track loop iterations carefully. You can set a loop limit in the Formality setup Tcl script with `set hdlin_while_loop_iterations limit`.
- Always explicitly declare functions as automatic.

- Keep math operations of variables on the same line, and group them as two operands per operator, as shown:

  ```
  (((a + b) + c) + d)
  ```

- Do not use force and bind.

VC Formal Guidelines

- Sizing of variables should match when assign or mapped to one another. Avoid out of bound access or assignments.
- Parameters and other variables should not be redeclared within the same scope.
- When using *'include*, do not give paths, rather use the *include* directives in the project settings.
- Avoid using escaped names.
- Automatic instantiation for block boxes is not supported.
- Latch inferencing.
  - Avoid writing conditional assignments for latches. For example: *assign result = ctrl ? in : result;*

    The synthesis tool infers a latch for the assignment, whereas VC Formal might not infer any latch. This could lead to inconclusive or failing verification. Instead write the assignment in an always block as follows

    ```
    always @(ctrl)
    if (ctrl)
    result = in;
    else
    result = result;
    ```

    This makes both the tools infer latches leading to successful verification.
- Avoid usage of synthesis compiler design constraints (CDC) that cause design changes, such as *syn_append_submodules* for module renaming.
- Avoid usage of supply0/supply1 port types.
- Try to wrap memories into smaller modules. This partitions the module for verification reducing runtime.

Feedback

Coding Guidelines for Verilog and SystemVerilog Designs

Follow these Verilog and SystemVerilog coding guidelines to help the verification process run smoothly:

- Avoid using unions in SystemVerilog.
- When using *'include*, do not use paths, but use *include* directives in the settings, instead.
- The synthesis engine ignores simulation-specific constructs like *$DISPLAY* and *$SETUPHOLD* but the Formality checker does not. Enclose these constructs in *//synthesis translate_off/on* pragmas.
- Always size data types. For example, use *1`b0* instead of *`b0*.
- Do not use duplicate modules or entities in the same design. The synthesis engine supports the *allow multiple modules* option, which allows the the last read definition to be selected when different instances of the same module have different contents. However this option causes ambiguity in the Formality checker and is not permitted. Make sure each design has only one definition of each module or entity.
- When using custom packages, provide the function definition for operators like + and -. Include the package definition file along with the other source files if you use a non-standard package. For example, if you use the non-standard library listed below, you must add the following file: *vhdl_src/std_developerskit/synthreg.vhd*.

  ```
  library std_developerskit; use std_developerskit.synth_regpak.all;
  ```

- Use *full_case* and *parallel_case* with caution. If you specify *full_case* and do not define all the cases, it can cause problems during verification because the Formality checker does not honor this directive. The hardware behavior is not affected.
- Use pragmas and synthesis directives with caution, because they direct the tool to remove logic based on these directives. As these directives are typically specified by the designer, it is assumed that the board behavior is correct. However, simulators do not read these directives and do not do any reachability analysis.
- Do not use *force* and *bind* statements.
- Avoid using multiple drivers, where a net is driven both by a constant and an active signal. Synthesis resolves this to a constant, but the equivalence checker might not do the same. The Formality tool treats this as a wired AND.
- Gated and generated clock structures

  By default, the synthesis flow converts gated and generated clock structures, to ensure that the prototype works on the board. You can help the verification process by identifying and constraining the correct clocks before synthesis.

- RTL RAMs
  - RAMs are typically implemented as block RAM in FPGAs, and these RAMs do not have good verification models. Take the RTL code from which synthesis would infer a RAM and enclose it in a separate hierarchy. Put the RAM code in a separate design hierarchy with a fixed boundary (*syn_hier =fixed*). This allows the tool to black box the RAM and continue verifying of the rest of the design.
  - The tool automatically black boxes all user models with inferred RAM. The RAMs are synthesized, but black-boxed for equivalence checking.
  - If the code would infer register banks instead of a RAM implementation, use the *syn_ramstyle="register"* attribute to force it to be implemented as registers, because registers can be formally verified.
- Multipliers

  The tool does not black box multipliers. Small multiplier structures (<13 bits) that are implemented as logic can be verified formally. Currently, large multipliers mapped to logic could take a long time to verify and the results might still be inconclusive. For larger structures, you might want to enclose them in a separate hierarchy and verify them by other means like simulation.
- DSP blocks

  Logic inferred as DSP blocks is typically difficult to verify because pipelined registers from surrounding logic go into the DSP block. Currently the tool verifies logic mapped to DSPs without registers, and automatically ensures that the mapped DSPs are combinational, unless you have instantiated them in the RTL.

  For DSPs with registers, the current workaround is to direct the tool to implement the DSP blocks as logic using the *syn_dspstyle* attribute, or to enclose the DSP structure in its own hierarchy so it can be black-boxed. Verify these black blocks by some other means like simulation. Future releases include support for DSPs with registers.
- Retiming

  Turn off retiming option whenever possible. Wherever possible, turn it off on specific blocks, not globally. When you enable *Verification Mode*, this optimization is automatically disabled.
- *syn_hier="fixed"*

  When synthesizing, it helps to use the *syn_hier="fixed"* attribute on all levels of hierarchy. Add a simple collection command like the following to the `fdc` file:

  ```
  define_scope_collection all_views {find -hier -view {*}}
  define_attribute {$all_views} {syn_hier} {fixed}
  ```
- Registers with set and reset pins

Feedback

Black-box any blocks containing RTL code that describes registers with both set and reset pins. The Xilinx tool does not have a single equivalent primitive that has both pins. It uses two registers, which causes problems for the Formality checker.

- Latches

  Describe latches explicitly within *always* blocks, instead of assigning constants. Constant assignment is ambiguous, and different tools might interpret the assignments differently. Also, it might result in combinational loops, which you are advised to avoid.

  Use the explicit style:

Explicit Coding Style	Ambiguous Coding Style (Do not use)
module latch_test (input ctrl, data, output reg result1, result2); always @(data or ctrl) if (ctrl) result = data endmodule	*module latch_test (input ctrl, data, output reg result1); assign result1 = ctrl ? data : result1 endmodule*

- XMRs

  You can use downward cross-module reference (XMR) constructs. This example shows a a high-level port (*d*) that refers to a port at the lower level in *inst1*:

```
module top (input a, input b, output c, output d);
sub inst1 (.a(a), .b(b), .c(c);
assign d = inst1.a;
endmodule

module sub (input a, input b, output c);
assign c = a&b;
endmodule
```

  The Formality software does not support the following:

  - Upward references.
  - References to the same level.
  - References to packed structures.
  - XMRs specified with relative paths. Use an absolute path instead: *assign a = top.intst2.b*.

Limitations of Equivalence Checking

There are some limitations to using both the Formality and VC Formal software:

- The tool automatically turns off the following synthesis optimizations.
  - Finite state machines inference
  - Retiming and pipelining
  - High-reliability synthesis

Additional limitations for the Formality software:

- Modules with inferred memories are automatically black-boxed. The tool reports the memories and provides the setup needed to run VCS simulation. See Running VCS Simulation for Inferred Memories.
- VHDL support is limited. VHDL designs that use complex math operators might not get verified properly.
- For single-FPGA designs, verifies the mapped netlist against the RTL language description
- For multi-FPGA designs, verifies the multi-FPGA mapped netlist against the post-partitioning output.
- Verification mode automatically turns off some synthesis optimizations so that the design can be formally verified. Verification mode preserves hierarchy so as to converge in the formal verification flow. For designs with critical timing requirements, after verification, re-synthesize with verification mode turned off, so that the tool can optimize the design.

Additional limitations for the VC Formal software:

- System Verilog bind is not supported.

Prerequisites for Verifying FPGA designs

1. Follow the guidelines described in General Coding Guidelines for Formal Verification, and note the limitations (Limitations of Equivalence Checking).
2. To ensure that the tool does not run out of memory, set virtual memory and *ulimit* in the shell:

   ```
   limit vmemoryuse unlimited
   ulimit unlimited
   ```

3. Configure the environment variables listed in the table. In the GUI mode, you can set the environment variables later. But in the shell mode, set the variables.

Environment Variable	Description
VC_STATIC_HOME	Path to the VC Formal executable; required for VC Formal verification
FORMALITY	Path to the Formality executable; required for Formality formal verification
VCS_HOME	Path to the VCS tool; required to simulate inferred memories or unverifiable blocks
XILINX_VIVADO/XILINX	Path to the Xilinx tools; required for formal verification libraries from the vendor

4. If you have DesignWare® components, you must also set the path to the DesignWare library; otherwise the compiler errors out.

   Either set *dc_root* in the *options.tcl* file to point to the Design Compiler® installation (*option set dc_root*), or specify the path to the installation with the *SYNOPSYS* variable. If both *$SYNOPSYS* and *dc_root*are specified, the tool uses *dc_root*. If only *$SYNOPSYS* is defined, the tool uses that path.

Enabling Formal Verification

Use the following workflow to enable formal verification.

1. Execute the following command to enable formal verification and turn off synthesis optimizations that cannot be formally verified.

   ```
   option set verification_mode 1
   ```

   In designs with critical timing requirements, the design verified with Formality might fail on the board. After verification, resynthesize such designs with verification mode turned off.

2. Execute the following command to perform bottom-up verification or distribute verification among smaller blocks.

   ```
   option set hier_verification
   ```

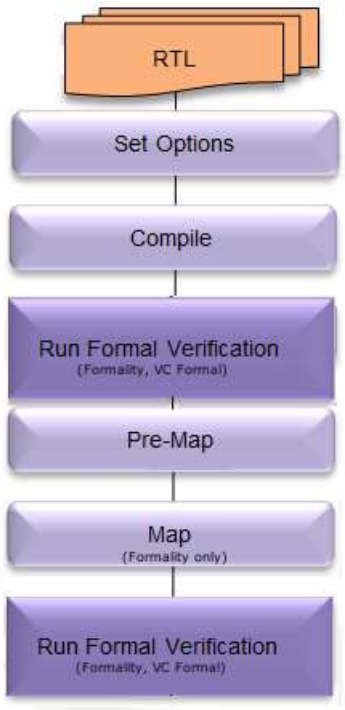
   This command defines the number of verification blocks and do bottom-up verification. The default (*auto*) uses heuristics to automatically divide the design into verification blocks, but you can set it to *manual*, and then specify a percentage to create verification blocks with the `option set hier_verification percent` command. A percentage of 5 specifies that modules that are at least 5 percent of the total area be made into verification blocks. If you set *hier_verification* to *off*, formal verification runs on the whole design. For large designs, it is recommended to use *auto*.

Feedback

Verifying Single-FPGA Designs

For single-FPGA designs, run formal verification (VC Formal™ or Formality) on a mapped design and compare the results to the original RTL.

Figure 25 Verification Flow for single-FPGA designs



Review the prerequisites to verify the design. See Prerequisites for Verifying FPGA designs.

Use the following workflow to verify single-FPGA designs.

1. Enabling Formal Verification
2. Synthesizing a Single-FPGA Design
3. Running Formal Verification
4. Analyzing Formal Verification Results
5. Re-synthesizing a Single-FPGA Design

Feedback

Synthesizing a Single-FPGA Design

Use the following workflow to synthesizing a single-FPGA design.

1. Run synthesis. You can run verification after the compile (VC Formal or Formality) or map (Formality only) stages. The following table shows the sequence of commands required from each database state:

From Compile State	From Map State
option set verification_mode 1	*option set verification_mode 1*
run compile	*run compile run pre_map* (Formality can include UPF files)*run map*
export verification(see step 4) *launch vcf* | *launch formality*(see step 5)	*export verification*(see step 4) *launch formality*(see step 5)

   See General Coding Guidelines for Formal Verification and Coding Guidelines for Verilog and SystemVerilog Designs for information about the source files.

2. You can include UPF and UUM files for formal verification at the pre-map stage. For example:

   ```
   run pre_map -upf test.upf
   ```

3. When you verify post-compile Verilog against HDL for designs with memory blocks, the tool automatically verifies the memories without black-boxing them.

   There are some limitations to this coverage of memories. The tool automatically black boxes some memories that it does not support: multi-process write, partial read and write and bit-wise read memories.

   Verification fails if the design contains memories inferred from multiple register banks, negative index register banks, or a mixture of to and downto register banks; for example *reg [0:15] mem [4095:0]* or *reg [15:0] mem [0:4095].*

   You can encounter long formal verification runtimes because of the time taken for elaboration.

4. ◦ Check for warnings that might cause mismatches during formal verification, then fix them. For example:

   ```
   CD285
   @W: Port map width mismatch (7 => 8) on port <b> of component
    <addern>
   ```

Feedback

```
CD453
@W: Index <8> may be out of range
```

5. Execute the following command to export the synthesized data for verification.

   ```
   export verification -path
   ```

   The directory includes encrypted *svf* guidance files that direct the verification run, based on the synthesis optimizations performed. You cannot modify the files. You can export the files from a system generate, compiled, or mapped state.

   Note:

   After export, do not move the files. The Tcl files in the exported directory require access to the source files. If you transfer the exported files to another location and the Tcl files cannot access the source files, formal verification encounters problems.

Running Formal Verification

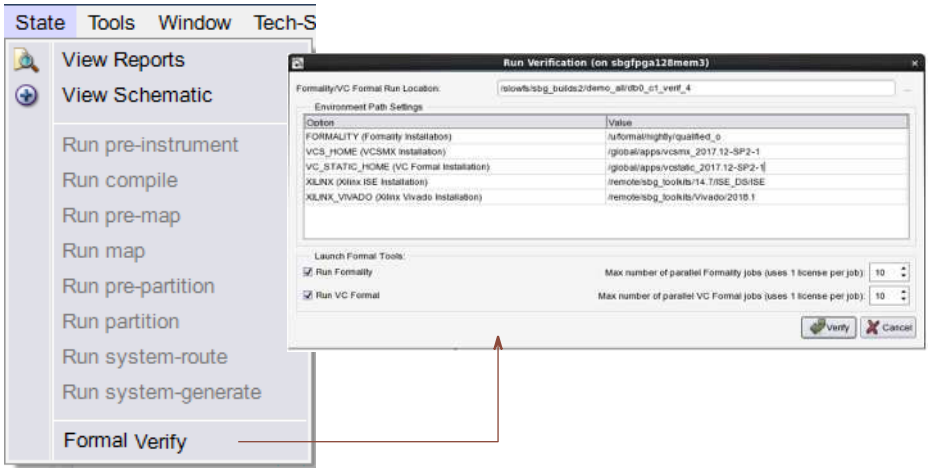
Use this procedure to run formal verification.

Before you begin

- Optionally, set parameters for the run. See Configuring Options for Formal Verification for the options you can set.
- It is recommended that you launch the formal verification tool from within the prototyping tool.
- The formal verification tool does not support CDPL for running verification partitions in parallel, but you can distribute processing on multiple processors on the same machine.

1. Start the VC Formal or Formality tool from the command line or the GUI.
   - To start the tool from the prototyping tool shell, type the following command. The optional *-gui* argument launches the formal verification tool in GUI mode. If you do not specify *-gui*, the tool launches it in shell mode and runs it in the background.
   - ```
     launch formality -path runDir -gui
     launch vcf -pathrunDir -gui
     ```
   - To start the tool from the prototyping GUI, go to a mapped database state and select **Formal Verify** from the **State** popup menu.
2. In the dialog box that opens, specify the directory for the formal verification run. The default location is *workingDir/databaseName_state_verif*. You can also define the environment variables if you did not do so earlier.
     ```

Feedback



3. Select either *Run Formality* or *Run VC Formal* or both.
4. Click **Verify**. The formal verification tool starts equivalence checking in the specified directory and compares the implementation to the reference RTL. It creates the verification partitions and runs the formal verification tool on those. The verification partitions are run individually for better visibility and debugging.

   The formal verification tool does not support CDPL for running these verification partitions in parallel, but you can distribute processing on multiple processors on the same machine.

   If you run equivalence checking in GUI mode, a reporting window opens. The window is updated automatically as the run progresses.

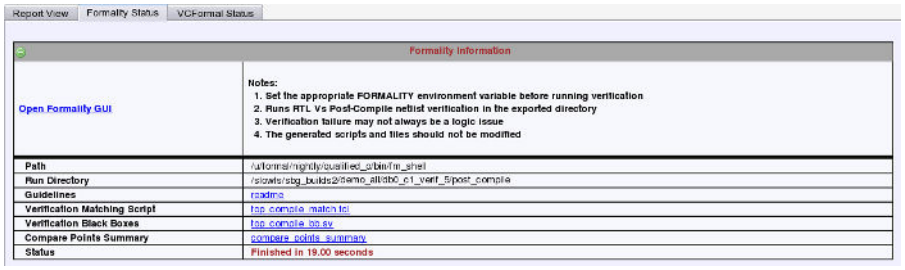
Analyzing Formal Verification Results

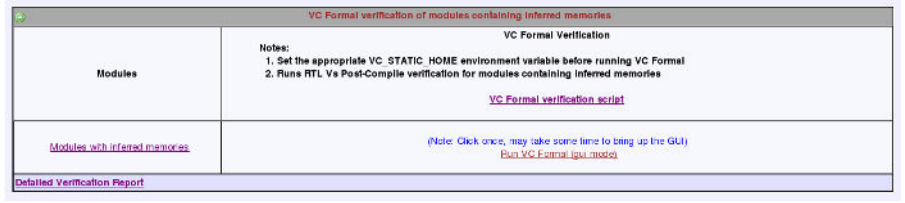
Use this procedure to analyze formal verification results.

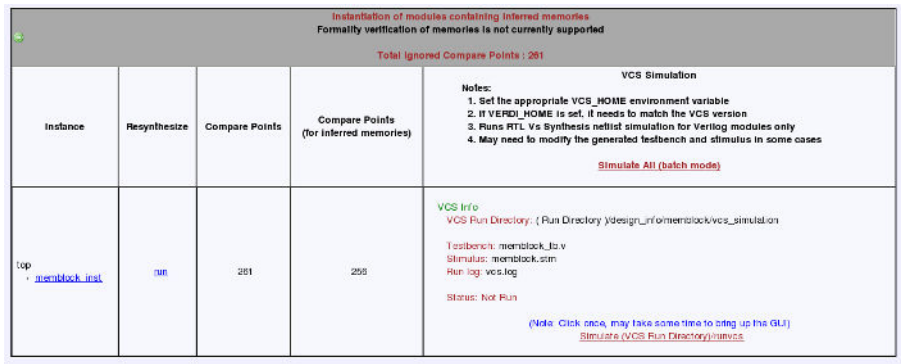
1. To view the results in the log file, use the *view report* command in the prototyping window.
2. Analyze the results in the interactive reporting window.

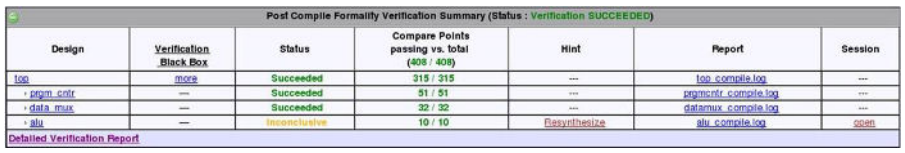
   Formality Status

Feedback

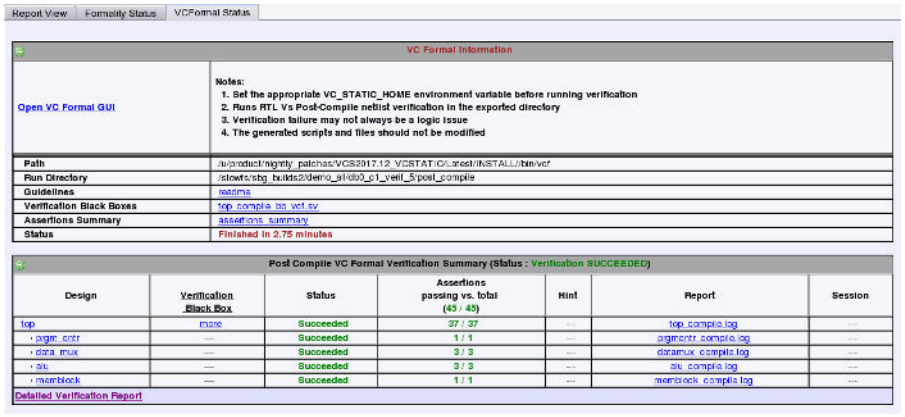








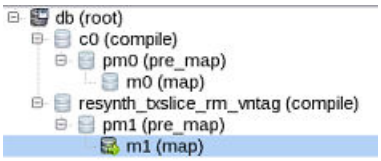
VC Formal Status



- Check the *Verification Summary* status, which updates in real time as the run progresses.
- Check the *Hint* column for *Resynthesize* links. The tool generates these links for verification blocks that fail, and the link lets you resynthesize and verify it separately.
- Check the *Instantiation of Modules Containing Inferred Memories*section. These are verification blocks with inferred memories, which cannot be formally verified with Formality. Instead, verify with VC Formal or use VCS to simulate these blocks. See Running VCS Simulation for Inferred Memories for the procedure.
- Check the black box report for modules that are defined as synthesis black boxes. Modules with generated clocks and modules with RAMs are black-boxed automatically.
- To view the interactive report in a browser, click **Detailed Verification Report**.

3. If the report shows failures, try these techniques to analyze and fix the design:
   - Click the link in the **Report** column to view the log, which shows the points where the design failed.
   - To view the cone of logic, open the formal verification software run from the *Session* column in the report, and view the schematic for the failing logic.
   - For failing verification blocks, try resynthesizing them.
   - If your design fails because it is too complex, use *option set hier_verification_percent* to set a lower percentage, to reduce complexity and create smaller verification blocks.

     Rerun compile, pre-map, and map. When you resynthesize, the tool does not overwrite the originals, but creates parallel database states:

4. To run incremental verification, make adjustments as needed, and rerun the equivalence checker.

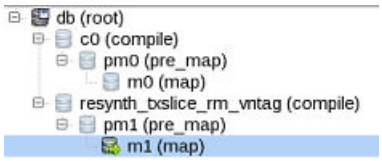
- For information about the changes in guidance you can make in the public *svf* file and other options you can set, see Configuring Options for Formal Verification.
- Do not change the *svf* directory structure or try and edit the encrypted *svf* files.
- Rerun the equivalence checker.

The checker runs incrementally and only verifies the verification partitions that have modifications, so this run is faster.

Re-synthesizing a Single-FPGA Design

Use this procedure to resynthesize the design.

1. If your design fails because it is too complex, use *option set hier_verification_percent* to set a lower percentage, to reduce complexity and create smaller verification blocks.
2. Rerun compile, pre-map, and map. When you resynthesize, the tool does not overwrite the originals, but creates parallel database states:



3. To run incremental verification, make adjustments as needed, and rerun the equivalence checker.
   - For information about the changes in guidance you can make in the public *svf* file and other options you can set, see Configuring Options for Formal Verification.
   - Do not change the *svf* directory structure or try and edit the encrypted *svf* files.
   - Rerun the equivalence checker.

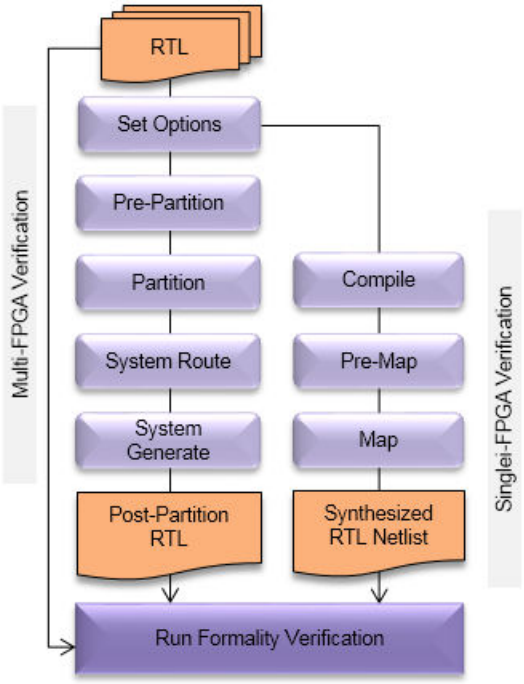
   The checker runs incrementally and only verifies the verification partitions that have modifications, so this run is faster.

Verifying Multi-FPGA Designs

For multi-FPGA designs, you verify the partitioned RTL against the original RTL. You can also verify the mapped RTL for each partitioned FPGA against the post-partitioning RTL netlist.

Feedback

Figure 26 Verification Flow for multi-FPGA designs



The following table summarizes the commands in the flow.

Post-Compile	After System Generate	Post-map
option set verification_mode 1	*option set verification_mode 1*	*option set verification_mode 1*
run compile	*run compilerun pre_partition run partition run system_route run system_generate*	*run compile run pre_map run map*
export verification launch formality	*export verification launch formality*	*export verification launch formality*

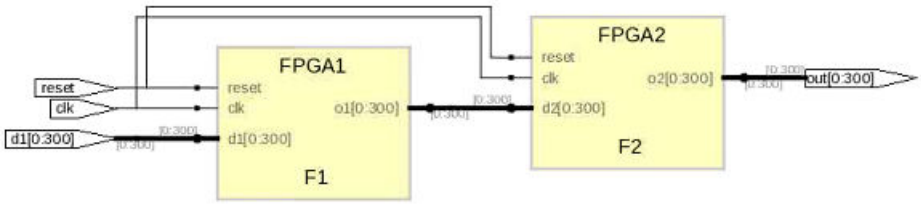
Review the prerequisites to verify the design. See Prerequisites for Verifying FPGA designs.

Use the following workflow to verify multi-FPGA designs.

The following procedure provides step-by-step details.

Feedback

1. To run formal verification after compile, do the following:
   - Compile the design as usual (*run compile*).
   - Follow the directions in step 4 to export the database and launch Formality.
   - Analyze the design as described in steps 5-7.
2. To run formal verification after top-level partitioning, partition the design and generate partitions.
   - Make sure that design flow is set to partition (*option set design_flow partition*).
   - Use the usual sequence of commands to partition: *run compile, run pre_partition, run partition, run system_route*, and *run system_generate*.



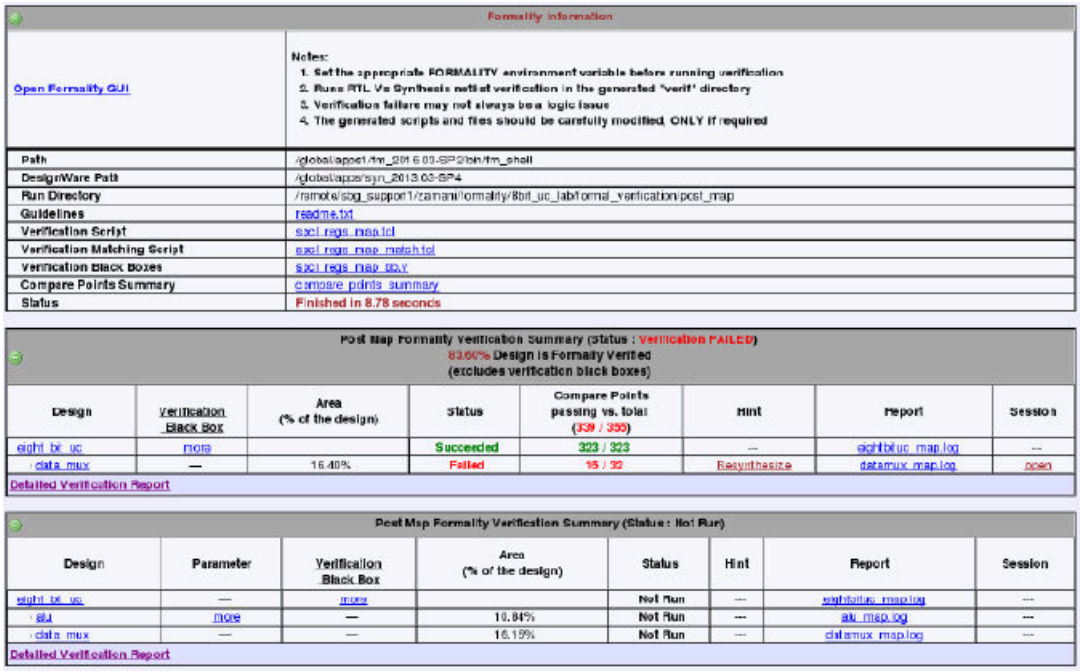
   - You can include UPF files for formal verification by specifying the files when you run pre-partition:

     ```
     run pre_partition -upf test.upf -tss board.tcl
     ```

3. To run verification, after creating partitions with *run system generate*, do the following from the system generate state.
   - In shell mode, use the *export verification* and *launch formality* commands. If you use shell mode, make sure to specify the environment variables for the tools, as you are not prompted for them.
   - Click **Verify** in the GUI to start the Formality run and verify the post-partitioned netlist against the original RTL. If you run equivalence checking in GUI mode, a reporting window opens. The window is updated automatically as the run

Feedback

progresses. To view the results in the log file, use the *view report* command in the prototyping window.



Formality Information

Open Formality GUI	Notes: 1. Set the appropriate FORMALITY environment variable before running verification 2. Runs RTL Vs Synthesis netlist verification in the generated 'verif' directory 3. Verification failure may not always be a logic issue 4. The generated scripts and files should be carefully modified, ONLY if required
Path	/global/apps1/fm_2016.03-SP2/bin/fm_shell
DesignWare Path	/global/apps/syn_2013.03-SP4
Run Directory	/remote/sog_support1/zamani/formality/8bit_uc_lab/formal_verification/post_map
Guidelines	readme.txt
Verification Script	svcl_regs_map.tcl
Verification Matching Script	svcl_regs_map_match.tcl
Verification Black Boxes	svcl_regs_map_bbx
Compare Points Summary	compare_points_summary
Status	Finished in 8.78 seconds

Post Map Formality Verification Summary (Status : Verification FAILED)
83.60% Design is Formality Verified
(excludes verification black boxes)

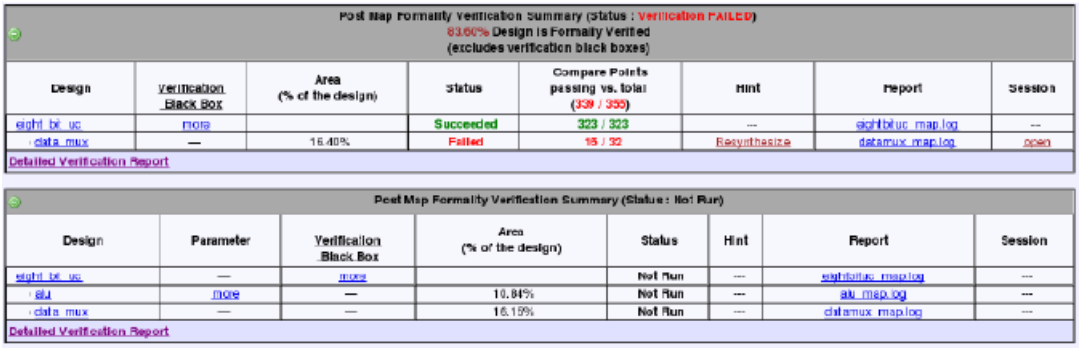
Design	Verification Black Box	Area (% of the design)	Status	Compare Points passing vs. total (339 / 355)	Hint	Report	Session
eight_bit_uc	more		Succeeded	323 / 323	--	eightbituc_map.log	--
↳data_mux	—	16.48%	Failed	16 / 32	Resynthesize	datamux_map.log	open
Detailed Verification Report							

Post Map Formality Verification Summary (Status : Not Run)

Design	Parameter	Verification Black Box	Area (% of the design)	Status	Hint	Report	Session
eight_bit_uc	—	more		Not Run	---	eightbituc_map.log	--
↳alu	more	—	10.84%	Not Run	---	alu_map.log	--
↳data_mux	—	—	16.19%	Not Run	---	datamux_map.log	--
Detailed Verification Report							

4. Analyze the results in the interactive reporting window.

   The figure below shows part of the reporting window, showing the verification partitions and a link to a detailed verification report.



Post Map Formality Verification Summary (Status : Verification FAILED)
83.60% Design is Formality Verified
(excludes verification black boxes)

Design	Verification Black Box	Area (% of the design)	Status	Compare Points passing vs. total (339 / 355)	Hint	Report	Session
eight_bit_uc	more		Succeeded	323 / 323	--	eightbituc_map.log	--
↳data_mux	—	16.48%	Failed	16 / 32	Resynthesize	datamux_map.log	open
Detailed Verification Report							

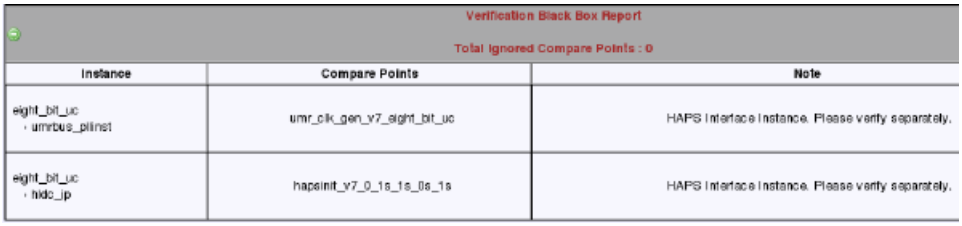
Post Map Formality Verification Summary (Status : Not Run)

Design	Parameter	Verification Black Box	Area (% of the design)	Status	Hint	Report	Session
eight_bit_uc	—	more		Not Run	---	eightbituc_map.log	--
↳alu	more	—	10.84%	Not Run	---	alu_map.log	--
↳data_mux	—	—	16.19%	Not Run	---	datamux_map.log	--
Detailed Verification Report							

   - To view the interactive report in a browser outside the tool, click **Detailed Verification Report**.
   - Check the *Verification Summary* status, which updates in real time as the run progresses.
   - Check the **Hint** column for *Resynthesize* links. The tool generates these links for verification blocks that fail, and the link lets you resynthesize and verify it separately.

Feedback

- Check the *Instantiation of Modules Containing Inferred Memories* section. These are verification blocks with inferred memories, which cannot be formally verified. Instead, use VCS to simulate these blocks. See Running VCS Simulation for Inferred Memories for the procedure.
- Check the black box section of the report for modules that are defined as black boxes. This includes user-defined black boxes or HAPS interfaces. The tool checks the connectivity around the black boxes, but does not verify the insides of the black boxes.

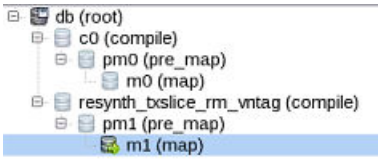


Verification Black Box Report

Total Ignored Compare Points : 0

Instance	Compare Points	Note
eight_bit_uc umrbus_plinst	umr_clk_gen_v7_eight_bit_uc	HAPS Interface Instance. Please verify separately.
eight_bit_uc hidc_ip	hapsinit_v7_0_1s_1s_0s_1s	HAPS Interface Instance. Please verify separately.

5. If the report shows failures, do the following:
   - Click the link in the **Report** column to view the log, which shows the points where the design failed.
   - To view the cone of logic, open the Formality run from the **Session** column in the report, and view the schematic for the failing logic.
   - If your design fails because it is too complex, use *option set hier_verification_percent* to set a lower percentage to reduce complexity and create smaller verification blocks. When you resynthesize, the tool does not overwrite the originals, but creates parallel database states:



6. To run incremental verification, make adjustments as needed, and rerun the equivalence checker.
   - For information about the changes in guidance you can make in the public *svf* file and other options you can set, see Configuring Options for Formal Verification.
   - Do not change the *svf* directory structure or try and edit the encrypted *svf* files.
   - Rerun the equivalence checker.

   The checker runs incrementally and only verifies the verification partitions that have modifications, so this run is faster.

Feedback

Configuring Options for Formal Verification

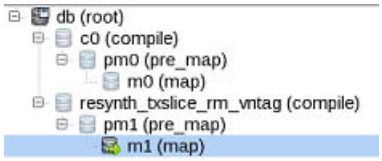
After analyzing the first run, make required adjustments and rerun verification.

1. To fine-tune the automatic verification partitions that the tool creates, specify the *option set hier_verification manual* command, and then use the *option set hier_verification_percent* command to set a percentage value for generating the verification blocks.

   A percentage of 5 specifies that modules that are at least 5 percent of the total area be made into verification blocks. For complex designs that fail, set a lower percentage with *option set verification_hier_percent* to reduce complexity and create smaller verification blocks.

2. Rerun verification with the new option settings.

   When you verify the design again, the tool does not overwrite the original database state, but creates a new parallel database state:

3. Edit formal verification guidance commands in the decrypted public *svf* file and then rerun verification.

   The Setup Verification for Formality (*.svf*) script is a file that contains Formality commands. The *svf* commands provide guidance so that the Formality tool can account for the transformations made by the synthesis optimizations. For example, the tool might duplicate a register to improve drive strength, and this file records the register duplication.

   - Do not try and edit the encrypted private *svf* file. The decrypted public *svf* file that is editable (*formality_svf/design_map.svf*) is generated after a Formality run.
   - You can modify the following commands in the public *svf* file, but make sure you understand the effect your changes have on the verification process:

To...	Use These Commands ...
Modify the design	*guide_uniquify guide_ununiquify guide_ungroup guide_instance_merging*

Feedback

To...	Use These Commands ...
Edit sequential optimization settings	*guide_reg_merging* *guide_reg_constant* *guide_reg_duplicationg* *uide_reg_removal guide_inv_push*
Guide port-matching and black-boxing	*set_user_match remove_user match* *set_direction set_black_box*

For details about these commands, see the Formality or VC Formal documentation.

- Do not edit any *guide_private* commands in the public *svf* file. These commands are placeholders for commands in the private *svf* and must not be altered.

Running VCS Simulation for Inferred Memories

The Formality tool cannot verify modules with inferred memories, user-defined black boxes, or HAPS interface IP, and treats them as black boxes. However, you can verify these black boxes by either using VC Formal or running VCS simulation.

1. Ensure that the *VCS_HOME* environment variable is set to the path of the VCS software. Use the VCS-MX software to run simulation.
2. Run synthesis and formal verification, as described in Verifying Single-FPGA Designs and Verifying Multi-FPGA Designs. The modules must be Verilog modules.

   The tool creates test patterns and a testbench for running VCS simulation on modules with inferred memories. The test pattern is randomly generated. The testbench is generated using the clock and reset information from the design partition that is created for running simulation. The information is listed in the interactive reporting window.

   It also creates a VCS simulation directory called *verif.*

Feedback

3. In the formal verification reporting window, do the following:
   - Check the reported memories or IP:



   - Check the files generated and edit the test pattern or stimulus as needed. Edit the testbench as needed.

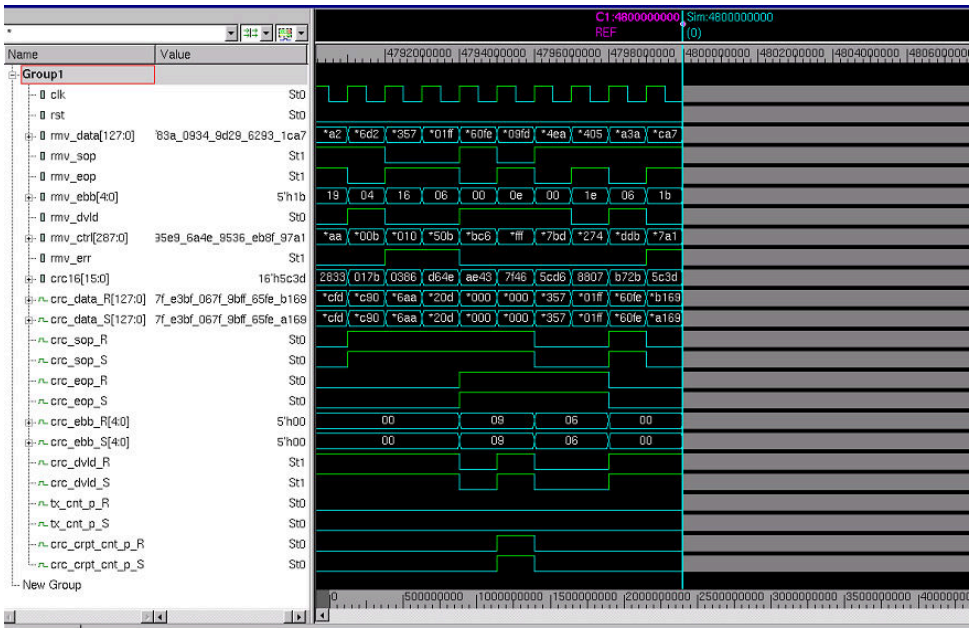
4. From the report window, click the appropriate simulate link in the **VCS Simulation** column once to run simulation for an individual block or for all the blocks.

   Clicking starts the launch script and the VCS run. The tool compiles the individual module or all the synthesized modules as specified, and simulates the synthesized netlist against the original RTL. The VCS GUI opens.

5. Check the results in the VCS run log.

Feedback

Use the GUI for further debugging.



Running Formal Verification with Checkpoints

This verification flow is based on the creation of checkpoints, which can be compared to the original Verilog using the VC Formal tool to run formal verification.

1. Set environment variables for the verification tools:
   - Set the Formality installation (*FORMALITY*).
   - Set the VCS variable (*VCS_HOME*).
   - Set the VCS Static variable (*VCS_STATIC_HOME*).
   - Set the Xilinx Vivado installation (*XILINX_VIVADO*).
2. Set options from the GUI or the command line:
   - Enable verification mode: `option set verification_mode 1`
   - Enable the creation of checkpoints: *option set verify_check_points 1.* When this option is enabled, the tool automatically creates checkpoints based on the design modules (SCMs).

Feedback

After the compile stage, the tool writes out checkpoints for the modules. Each verification check point contains the following data that the verification tools can use:

- Complete environment setup, formal tool setup. and launching data
- Complete reference and implementation Verilog designs, and other data
- Formal tool constraints such as clocks, black boxes etc.
- Guidance and matching data for achieving maximum formal convergence
- Interface for controlling various capabilities of specific formal tools based on the check point requirements
- Report generation mechanisms
- Handshaking mechanisms between the prototyping and verification tools

3. Export files for formal verification.
   - Go to a valid database state, such as compile or pre-map. For a list of the database states, refer to export verification.
   - Run the *export verification* command.

     ```
     export verification -check_points -path dir
     ```

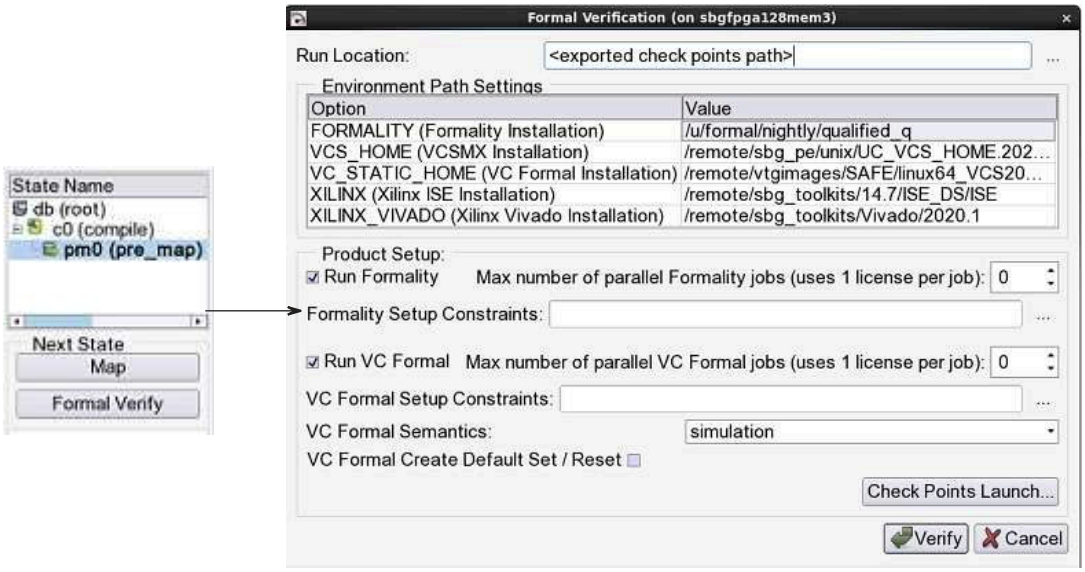
4. Specify verification tool options and launch the verification check from the command line or from the GUI.

   The GUI provides an easy way to enter parameters for the verification check.

   - To use the command line, specify the *formal_verify* command from a valid database state. For the syntax of this command, see formal_verify. You can also do this from the GUI. From the pre-map or other valid stage, click **Formal Verify**, and fill in the verification tool controls in the dialog box that opens.
   - In the GUI or the command, specify controls for the verification tools, such as the maximum number of parallel jobs. You can also specify a setup constraints file for Formality or VC Formal with the verification control commands.

Feedback

- Click **Check Points Launch**. This opens a window for the results.



5. Analyze results.

Examples

Verification Check Points After Compile

```
database load db -autocreate -technology HAPS-80
option set verification_mode 1
option set verify_check_points 1
launch uc
run compile
export verification -check_points -path check_points_data
set fvId [formal_verify -path check_points_data -post_compile]
run pre_map
run map
job wait $fvId
```

Verification Check Points After Pre-Map

```
database load pcucdb -autocreate -technology HAPS-80
option set verification_mode 1
option set verify_check_points 1
launch uc -utf run.utf -ucdb ucdb -v 2.0
run compile -ucdb ucdb
export verification -check_points -path check_points_data
set fvcId [formal_verify -path check_points_data -post_compile ]
job wait $fvcId
```

For more information, see *verification_mode* and *verify_check_points* options in option.

10

Recommended Methodologies

This chapter describes recommended prototyping methodologies to achieve specific goals:

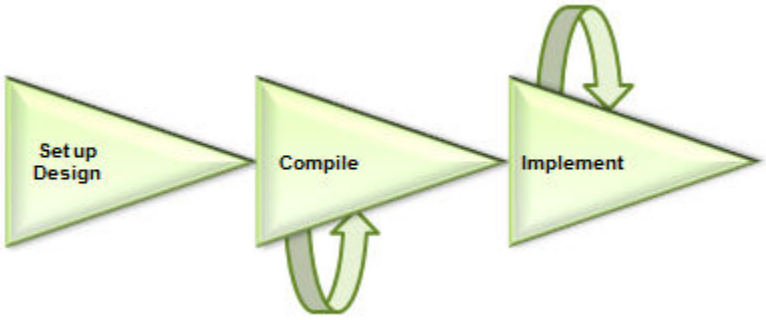
- Methodology for a Quick Prototype
- Configuring a Best-Performance Prototype
- Scripting Design Tasks

Methodology for a Quick Prototype

Typically, you need an initial prototype for proof-of-concept scenarios, such as exploring FPGA architectures, developing prototypes, or validating constraints. In this scenario, the focus is on quickly bringing up an initial FPGA prototype. The priorities are a smooth migration process from ASIC to FPGA, fast turnaround times through iterations, and quick board bring-up.

The prototyping methodology ensures fast bring-up with a three-pronged approach: fast implementation algorithms, incremental runs, and fewer iterations.

The figure shows the phases for implementing a quick prototype. Although setup and compilation are shown as different phases, automatic setup is only implemented when you compile the design. The *Implement* step encompasses pre-mapping, mapping, placement, and routing.



- Setting up the FPGA Design
- Compiling a Quick Prototype

- Synthesizing, Placing, and Routing a Quick Prototype
- Tips to Reduce Turn Around Time (TAT)

A quick prototype is the starting point for a mature prototype focused on better performance. Initial setup time serves to shorten the implementation time for a subsequent, performance-focused prototype.

Setting up the FPGA Design

The HAPS prototyping solution offers many features to automate or speed up the process of making ASIC RTL FPGA-friendly. Many setup features run automatically when you compile the design. This helps to compress the time frame. For details, see Converting ASIC Designs to FPGA and Creating and Compiling Databases.

The following procedure describes the steps and settings to shorten the time taken for design preparation.

- RTL support

  The prototyping tool reads standard Verilog, VHDL, and SystemVerilog.

- ASIC RTL netlist translation for FPGA design
  - There are three broad techniques to use when reconciling ASIC-FPGA RTL differences:

Conversion	Convert ASIC gated clocks with automatic gated clock conversion (run during compilation). Automatically convert generated clocks (run during compilation). For more information about converting ASIC clocking, seeConverting ASIC Clocks.
Substitution	Substitute ASIC memories like system RAM and ROM, system FLASH, system EEPROM, and memory BIST with FPGA equivalents. See Converting ASIC Memories for details.
Exclusion	Exclude ASIC test interfaces such as test TAP, test or JTAG muxing, and BSD. Exclude digital interfaces for analog blocks like ADC, DAC, and voltage regulators.

  - Use the `report syntax_check` command to check the RTL syntax, and then use the `report` command to see the results. Checking for syntax errors up front reduces costly debugging later in the cycle.
  - To quickly run through and validate your design and constraints, your design can have black boxes.

    To go all the way through board bringup, your RTL cannot contain black boxes. You cannot generate a bit file with black boxes.

For netlist editing and other details on converting ASIC RTL, see Converting ASIC Designs to FPGA.

- IP

  You can import Synopsys DesignWare IP directly. You can use the same IP as in the ASIC, and you do not need to verify it again. This reduces the time to bring up. See Intellectual Property.

- Constraints and attributes

  - You can automatically read many existing constraint formats when you implement the FPGA, like *sdc*, *fdc*, *cdc*, and *idc*. This allows you to reuse existing files without having to recreate the constraints. In particular, you can reuse ASIC *sdc* timing constraints, and read in *upf*.
  - The constraints spreadsheet interface provides a quick and convenient way to enter constraints and attributes.
  - Use the Tcl *find*, *expand*, and collection commands to quickly find, group and apply constrains to a set of objects. You can also use the SDC query commands: *get** and *all**.
  - Use the `report constraint_check` command to check the constraints. Use the `report` command to see the results. Checking the constraints reduces the amount of debugging later in the cycle.
  - For a quick prototype, set loose constraints.
  - Avoid setting attributes like *syn_keep* that could inhibit optimizations.

  See Adding Design Files for more information.

Compiling a Quick Prototype

The following procedure outlines the steps needed to generate a quick prototype using command-line commands. You can also put these commands into a script to save you even more time.

You can also use the GUI to specify your design intent and create a quick prototype. See Using Design Intent to Synthesize a Quick Prototype.

1. Set up your RTL, taking advantage of the features that shorten design preparation that are mentioned in Setting up the FPGA Design.
2. Run the diagnostic compiler (*report rtl_diagnostics)* to make sure your RTL is clean.

   This command parses the RTL and checks for syntax errors. By running this syntax check up front, you save time and debugging iterations later on in the flow.

- Check the diagnostic report and fix any errors.
- Rerun the diagnostic compiler with the *-incr 1* option and repeat the process until your RTL is clean. With this option, the diagnostic compiler runs incrementally, which saves time on subsequent runs.

3. Set options for, and run the Time-to-first-prototype (TTFP) fast compiler.

   The following options are some of the options that you can use to reduce runtime. You can also set these options from the GUI by enabling them in the  *Options Editor (.*).

   - In a Tcl options file, set *option set continue_on_error 1* to ensure that the compiler continues to run when it encounters a non-syntax error. The *continue_on_error* option saves runtime by reducing the number of iterations needed. If there are modules with errors, black box the modules and work on them separately. See Running Compilation with Errorsfor details.
   - If you use GCC, add the `option set fix_gated_and_generated_clocks 1` to the options file, to automatically convert ASIC gated and converted clocks.
   - Specify the `option set automatic_compile_point 1` command to automatically define compile points. If you do not want a define a module as an automatic compile point, set the *syn_no_compile_point* attribute. If you have multiple licenses, specify the *max_parallel_jobs* option to process the compile points in parallel on multi-core machines.
   - Specify the TTFP compiler by setting the `option set synthesis_strategy fast|routability` command. The TTFP compiler is designed for fast turnaround. The compiler generates a schematic which you can view to graphically analyze the design.
   - Source the options file.
   - Specify the `run compile` command.

   The tool runs the TTFP compiler and generates a schematic and creates a new compile database state.

4. Analyze the results and fix errors.
   - Use the `view schematic` command to view the results graphically and visualize the RTL, IP design hierarchy, and interconnect.
   - Use the `report` command to check the results in the log file.
   - Fix errors in modules that were identified by the *Continue on Error* feature separately so that they can be compiled.

- Click error messages to get information on fixing them. Check resource utilization, clock timing, I/0 timing, and critical paths to make sure that the constraints are feasible. Adjust constraints if needed.
- Fix any compilation errors.
- Iterate and run the TTFP compiler again. The point of iterating is to clean up the RTL, and the TTFP compile facilitates this by providing quick spins. You can save even more time by rerunning the TTFP compiler incrementally with the `run compile -incr 1` command.

  Use the flexible database model to create different compile database states with each new run. Alternatively, choose to overwrite a previously created database state. The database model lets you stay flexible and iterate through tight design loops for a particular design state.
- Repeat until there are no errors.

Synthesizing, Placing, and Routing a Quick Prototype

Once you have clean RTL and a compiled design, you can complete the synthesis process and run place-and-route. For an initial prototype, you might not want to go beyond this stage because your purpose is exploration or basic validation. However, you can continue on to hardware verification if your design is complete. The following steps provide the details:

1. Start with a clean compiled database state and run pre-map.
   - Set any options you might need in an options file. It is recommended that you use *option set continue_on_error 1*.
   - Go to the compile database state with the clean RTL (*database set*) and run the constraint checker (`run constraint_check`) to check pin I/Os and locations.
   - Use the `run pre_map` command. This command generates a pre-map database state, a schematic, and a log file.
   - Use the schematic and log file (*view schematic* and *report*) to analyze the constraints in addition to the synthesis results, and adjust the constraints if needed. You can save debugging time by using the SCOPE interface to adjust constraints, because it is linked to the schematic and the RTL.
   - Rerun `run pre_map`, leveraging the database model as before to create multiple database states if you are exploring different design options. You can also use `run pre_map incr 1` to rerun it incrementally to retain design stability while also getting a faster turnaround time.
2. Map the design.

- When you are satisfied with the pre-map results, select the pre-mapped database (*database set*) and map the design with `run map`. This command generates a mapped database state, a schematic, and a log file.
- Analyze the schematic and log (*view schematic* and *report*), and fix any errors. Rerun `run map`, using the database model as before to explore the effects of different options. If you use `run pre_map incr 1`, the command runs incrementally and saves more time.

3. Run integrated place and route.
   - Select a mapped database state and use the `run partition` command to run place and route. This command runs place and route and generates a routed database state.
   - If you encounter errors, analyze and fix the errors before rerunning place and route incrementally.

   If your quick prototype was intended to validate RTL feasibility or for design exploration, you can stop at this point. If you want to generate a bit file and validate the design on a board system, remember that your design must be complete with I/O definitions, and you cannot have black boxes.
4. To use the hardware system to verify the prototype, follow these steps:
   - Make sure your routed design is complete, with no black boxes.
   - Use the `export runtime` command on a placed and routed database state to generate a bit file. For an initial prototype you can use a lower frequency.
   - Set up the hardware system. For details, see the hardware documentation.
   - Configure the hardware using the configuration commands described in the *System Configuration Software Handbook*.
   - Verify the design on the hardware system.

Tips to Reduce Turn Around Time (TAT)

- Set accurate design constraints, Use `report constraint_check` to check them.
  - Set realistic clock frequencies; do not overconstrain them.
  - Specify synchronous/asynchronous clock groups and false paths.
- Use distributed compile and distributed synthesis.
- Black-box parts of the design to isolate them and reduce overhead.

- Use incremental flows where possible. Use the ECO flow for small changes. For place and route, use design checkpoint files (DCP) for incremental runs.
- Use multi-FPGA DTD3 and move debug logic from design FPGAs to the debug hub.
- Partition the design across as many FPGAs as possible, while balancing performance and reduced area utilization numbers.
- Disable area estimation when you run pre-partition.
- Balance area utilization across FPGAs using the exploratory approach with the TSS and PCF.
- Work with inter-FPGA connections, trace usage reports, and cable connections to reduce global route ratios so that you do not have everything routed.

Configuring a Best-Performance Prototype

Typically, if the focus for your prototype is performance or optimal QoR, you have created initial prototypes previously. The following procedures are based on the assumption that you have previously worked on initial prototypes and have clean RTL and a feasible design.

Some scenarios where you want a performance-focused prototype are for an IP implementation, benchmarking, or to achieve timing closure.

The best-performance methodology described here assumes that you have cleaned the RTL and checked constraints as part of working on an initial prototype. The best-performance methodology starts with a compiled database state, and does not focus on the initial stages of migrating the design and compiling the design.

The following are some setup guidelines to achieve the best performance in the prototype:

- Set accurate and complete constraints.
- Reduce congestion by not using control signals with small fanouts.
- Avoid or reduce high fanout.

The following procedure outlines the steps needed to generate a performance-based prototype using command-line commands.

As an alternative to the command line, you can use the GUI to quickly specify your design intent to create a prototype with better performance. See Synthesizing for Optimal QoR Using Design Intent. However, the GUI method only covers synthesis, and does not include instrumentation and debug, as described here.

1. Generate an initial prototype to clean up the RTL and check design feasibility.

   See Methodology for a Quick Prototype for details, paying special attention to the information on setup and compiling. The following are some guidelines for setting up the RTL and constraints:

   - Set accurate and complete constraints. Set feasible constraints.
   - Reduce congestion by not using control signals with small fanouts.
   - Avoid or reduce high fanout.

   Based on your initial prototype, you might want to add test instrumentation to debug your RTL. Start with Step 2 if you want to instrument a pre-compiled netlist. If you want to compile your design before adding instrumentation, go to Step 3.

2. To add instrumentation to a pre-compiled netlist, start with the root database and create a new database state for instrumentation.

   - Use the `run pre_instrument` command, and specify the design files. The tool creates a new database state for RTL debugging.
   - Use the `edit idc -mode gui` command to open the GUI and define watchpoints and breakpoints. You can also do this through a Tcl script with the `edit idc -mode tclbatch` command.
   - Save the *idc* instrumentation file. You can now compile the design.

   For more information about instrumenting the design, see Instrumenting the Design for Debug. For complete details, refer to the help built into the instrumentor.

3. Create a new compiled database state from the root database. If you instrumented a pre-compiled netlist in Step 2, start with that database state.

   - Set options like the following for better performance:

     ```
     option set enable_prepacking 1
     option set fix_gated_and_generated_clocks 1
     option set retiming 1
     ```

   - Run the `run compile` command. If you instrumented your design in Step 2, include the *-idc* option and specify the *idc* instrumentation file. This command runs the default compiler, and generates a new, compiled database state. It also generates a log file and a schematic.
   - Analyze the results (*view schematic* and *report list*) and fix any errors. If you instrumented your design, the results reflect this.

- Iterate as needed, using the *-incr 1* option for incremental compile runs. As you are starting with the results of the initial prototype, there should not be many iterations.
- If you want to instrument the design after compilation, use the *-out* option and name a database state. For example: `run compile -out pcInstrument`.

4. To instrument a post-compiled netlist, follow these steps:
   - Generate a compiled database state for instrumentation.
   - Use *edit idc -mode* to open the instrumentor, and instrument the design as described in Step 2.
   - Save the *idc* file.
5. Synthesize, place, and route the design.
   - Set any options you require and run re-map (`run pre_map`). The command generates a pre-mapped database state, as well as log file and a schematic.
   - Analyze the results. Check the log file for the clock report, especially the gated and generated clocks. The prototyping tool is tightly integrated with IP, especially DesignWare IP, so that it can use accurate timing models for the IP and even optimize inside the IP in certain cases.

     Iterate as needed. Isolate areas with problems and work on them independently to save time while you run through the rest of the design.
   - Map the design with `run map`. The command runs timing-driven synthesis with various optimizations. The tool intelligently maps the logic to available resources, including DSPs, RAMs, and LUTs. The results are closely correlated to final timing, because they are optimized for the target architecture and routing.
   - The `run map` command also generates a log file and a schematic that you can use to analyze the results and make adjustments. Iterate as needed. You can view the results of timing adjustments by running the `report timing` command and then checking the results in the log file.
   - Place and route the mapped design by running the `run partition` command. This command launches and runs place and route from the prototyping tool.
   - Analyze the results and iterate through placement and routing until the design meets your goals. If necessary, you can return and rerun a synthesis step based on back-annotation from place and route before working through the flow again.
   - Generate a bit file by specifying the `export runtime bitfile` command. The command creates a bit file, and it also creates a directory with all the files you need for debugging your design, including a *debug.prj* file.

At this point, you can run RTL debug using the instrumentation you inserted earlier (Step 6), or proceed to hardware verification (Step 7).

6. To verify the prototype using the hardware, follow these steps:
   - Make sure your routed design is complete, with no black boxes.
   - Use the `export runtime` command on a placed and routed database state to generate a bit file. For an initial prototype you can use a lower frequency.
   - Set up the hardware. For details, see the hardware documentation.
   - Verify the design on the hardware system.

Scripting Design Tasks

The software comes with some basic scripts for running synthesis. The contents of the scripts are described in the Reference Manual. You can use these scripts as templates and modify them to create scripts of your own to run design tasks.

The *install_dir/examples* directory contains some scripted examples and templates that you can run or adapt for your own designs:

- *hpw_examples* contains examples, including the standard synthesis flow
- *hpw_templates* contains a template that you can fill in to create your own script

  The template includes commands like the following:

```
set OPTION_FILE <optionFile>
set SRC_FILE_LIST <sourceFile>
set CONSTRAINT_FILE <constraintFile>
database create <dbName>
source $OPTION_FILE
run compile -filelist $SRC_FILE_LIST
run pre_map -filelist $CONSTRAINT_FILE
run map
run partition
export runtime -path <outputDir>
```

See the following scripts described in the *Reference Manual*:

- FPGA Synthesis Scripts
- Scripts for Exploring Multiple Databases
- Debug Scripts

In addition to scripts, the FPGA prototyping tool supports Tcl commands that help you traverse the database and identify objects. You can use these commands to run batch conversions of ASIC RTL or for custom database access and reporting.

Tcl Command	Description
find	Finds objects in the RTL database. Offers powerful sifting capabilities when combined with *-filter*.
expand	Finds objects by traversing the design hierarchy
collection	Creates groups of objects based on common attributes, that can then be manipulated together

See Specifying Constraints for details about these commands.